

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME VIII OF EIGHT

The Last Mile

DAYS 171 – 180

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

10 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME VIII · THE LAST MILE

DAYS 171 TO 180

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume VIII of eight: **The Last Mile**. Days 171 to 180.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

PART I	Foundations: how code costs	Days 1–8
PART II	Arrays	Days 9–18
PART III	Strings	Days 19–26
PART IV	Two pointers and sliding window	Days 27–36
PART V	Prefix sums	Days 37–41
PART VI	Binary search	Days 42–50
PART VII	Sorting	Days 51–59
PART VIII	Hashing: maps and sets	Days 60–67
PART IX	Stacks and queues	Days 68–77
PART X	Linked lists	Days 78–86

PART XI	Recursion and backtracking	Days 87–97
PART XII	Trees and binary search trees	Days 98–112
PART XIII	Heaps and priority queues	Days 113–119
PART XIV	Tries	Days 120–124
PART XV	Graphs	Days 125–142
PART XVI	Dynamic programming	Days 143–163
PART XVII	Greedy and intervals	Days 164–170
Part XVIII	Bits and maths	Days 171–176
Part XIX	Final mocks and revision	Days 177–180

What is in this volume

PART XVIII · BITS AND MATHS	1
171 Binary, bits, and why they matter	2
DSA Binary, bits, and why they matter	3
DESIGN Monitoring, metrics, and alerting	31
PRACTICPractice — Binary, bits, and monitoring	55
172 The bit tricks every interview uses	61
DSA The bit tricks every interview uses	62
DESIGN Logging and distributed tracing	89
PRACTICPractice — Bit tricks, logging and tracing	108
173 XOR problems	114
DSA XOR problems	115
DESIGN SLAs, SLOs, and error budgets	141
PRACTICPractice — XOR, SLOs and error budgets	163
174 Primes, GCD, and modular arithmetic	169
DSA Primes, GCD, and modular arithmetic	170
DESIGN Deployments: blue-green, canary, and rollback	199
PRACTICPractice — Number theory and deployments	220
175 The combinatorics you actually need	227
DSA The combinatorics you actually need	228
DESIGN Security in a design interview	258
PRACTICPractice — Combinatorics and security	281
176 Bits and maths revision and mock round	288
DSA Bits and maths revision and mock round	289
DESIGN Cost: the constraint nobody mentions	314
PRACTICPractice — Bits and maths mock, and cost	336
PART XIX · FINAL MOCKS AND REVISION	343
177 The twenty patterns, on one page	344
DSA The twenty patterns, on one page	345
DESIGN Microservices versus monolith, argued both ways	378
PRACTICPractice — The pattern index, and monolith versus microservices	399
178 How to think out loud in a coding round	406
DSA How to think out loud in a coding round	407
DESIGN The system design interview framework, memorised	433
PRACTICPractice — Thinking out loud, and the design framework	455
179 Full mock: two problems, forty-five minutes	462

DSA	Full mock: two problems, forty-five minutes	463
DESIGN	Full mock: one high-level design, one low-level design	489
PRACTICE	Practice — The full mock, coding and design	515
180	Final revision, and the week before the interview	521
DSA	Final revision, and the week before the interview	522
DESIGN	Final revision, and the week before the interview	552
PRACTICE	Practice — The last week, and the questions you ask	571

BACK MATTER

A	Appendix A - The nine sections, and what each is for	578
B	Appendix B - Recall cards	580
C	Appendix C - The complete 180-day map	598
D	Appendix D - Glossary	599
E	Appendix E - References and further reading, by section	618
F	Appendix F - Colophon: how this book was made	620
G	Appendix G - Problem index	621
H	Index	625

List of figures

172.1 System design - Logging and distributed tracing	61
173.1 System design - SLAs, SLOs, and error budgets	114
174.1 System design - Deployments: blue-green, canary, and rollback	169
175.1 System design - Security in a design interview	227
176.1 Data structures and algorithms - Bits and maths revision and mock round	288
176.2 System design - Cost: the constraint nobody mentions	288
177.1 Data structures and algorithms - The twenty patterns, on one page	344
177.2 System design - Microservices versus monolith, argued both ways	344
178.1 Data structures and algorithms - How to think out loud in a coding round	406
178.2 System design - The system design interview framework, memorised	406
179.1 Data structures and algorithms - Full mock: two problems, forty-five minutes	462
179.2 System design - Full mock: one high-level design, one low-level design	462
179.3 System design - Full mock: one high-level design, one low-level design	462
180.1 Data structures and algorithms - Final revision, and the week before the interview	521
180.2 System design - Final revision, and the week before the interview	521

PART XVIII

Bits and maths

Days 171 to 176 · 6 chapters

Alongside, on the system design track:

Reliability, security, and the interview itself

171 Binary, bits, and why they matter

172 The bit tricks every interview uses

173 XOR problems

174 Primes, GCD, and modular arithmetic

175 The combinatorics you actually need

176 Bits and maths revision and mock round

Binary, bits, and why they matter

DATA STRUCTURES AND ALGORITHMS

Binary, bits, and why they matter

SYSTEM DESIGN

Monitoring, metrics, and alerting

1 What this is, and why they ask it

Binary is how numbers actually are inside a machine. Not a representation of them — **the thing itself**. Every number, every character, every colour, every instruction is a run of on-and-off switches.

A bit is one switch: zero or one. Eight of them make a **byte**. That is the entire vocabulary, and everything else in this phase is built from it.

They ask it because **bit questions are common, short, and completely unbluffable**. “Count the set bits.” “Is this a power of two?” “Find the one number that appears once.” **You either know what `n & (n - 1)` does or you do not**, and there is no way to talk around it.

And because it explains things you have already used. Why `Set` lookups are fast. Why a bitmask can stand in for a subset — the trick from bitmask DP. Why hash tables use `& (size - 1)` instead of `% size`. **Those were all bits, and you have been using them without seeing the machinery.**

There is also an honest reason it is worth a day. Bit manipulation is where beginners freeze in interviews — **not because it is hard, but because it is unfamiliar** and the notation looks like line noise. `x ^= 1 << i` is three ideas in eight characters. **Once you can read it slowly and correctly, the freezing stops.**

By the end of this lesson you can convert both ways by hand, read a bit pattern without panic, use all six operators and say what each does in one sentence, explain what shifting really means, handle negative numbers, and know the three tricks that come up again and again.

2 The story

The weights lived in a wooden box under Nagappa’s counter, and there were six of them.

The boy who swept the shop had counted them a hundred times without ever thinking about it. A very small one. One about twice that. Then bigger, then bigger, then bigger, then one so heavy he needed both hands to lift out.

What he could not work out was how six were enough.

Because people came in and asked for all sorts. Eleven of this. Thirty-seven of that. And Nagappa never once said he could not do it. He would reach into the box, put two or three on one side, tip the grain in on the other, and the arms would come level, and that was that.

One slow afternoon the boy asked him how he knew which ones to take.

Nagappa said it like it was nothing. **“Take the biggest one that is not too heavy. Put it on. Now you need less than you did. Take the biggest one that is not too heavy for what is left. Keep going.”**

The boy tried it with thirty-seven, and it worked, and then he noticed the thing that actually surprised him.

He had never once needed the same weight twice.

He asked about that too.

Nagappa laughed at him. **“Of course not. Two of any one of them is the next one up. If you ever wanted two, you would have taken the bigger one instead.”**

The boy went and sat on the step and worked his way through every amount from one up to sixty-three, and there was not one he could not make. **And not one he could make in two different ways.**

Six weights. Sixty-three amounts. Each one either out on the scale or still in the box, and nothing in between.

Years afterwards somebody asked him how he had kept all of it in his head.

He said it was not much to keep.

“You are not remembering the amount,” he said. “You are remembering which ones are out of the box.”

3 The idea in plain English

Nagappa’s weights are binary. Each weight is a **bit**. Out on the scale is **1**, still in the box is **0**. And the reason six weights cover everything up to sixty-three, exactly once each, **is the reason binary works.**

Place value

You already know place value in base ten. In **1101**, the digits mean thousands, hundreds, tens, ones — **each place is ten times the one on its right.**

Binary is the same idea with two instead of ten. Each place is **twice** the one on its right, and each digit is only ever **0** or **1**.

the number 13, in base ten

1	3
tens	ones
10	1

= 10 + 3 = 13

the number 13, in binary

1	1	0	1
8	4	2	1

= 8 + 4 + 0 + 1 = 13

So 13 is 1101.

Those are Nagappa's weights: the 8, the 4 and the 1 out on the scale; the 2 still in the box.

Nagappa's rule — “take the biggest one that is not too heavy” — is exactly how you convert by hand, and it is worth doing that way once because it makes binary feel like counting rather than magic.

Converting, both directions

Decimal to binary, the reliable way: divide by two repeatedly and keep the remainders.

13 / 2 = 6 remainder 1	← the LAST bit
6 / 2 = 3 remainder 0	
3 / 2 = 1 remainder 1	
1 / 2 = 0 remainder 1	← the FIRST bit

READ THE REMAINDERS UPWARDS: 1101

Reading them downwards gives 1011, which is 11.

This is the single most common conversion mistake, and it is why the direction is worth saying out loud every time.

Binary to decimal, the reliable way: left to right, double what you have and add the next bit.

1101

start with 0

see 1: $0 \times 2 + 1 = 1$

see 1: $1 \times 2 + 1 = 3$

see 0: $3 \times 2 + 0 = 6$

see 1: $6 \times 2 + 1 = 13$

No powers to remember, no place values to line up.

One rule, applied four times.

The powers of two, which you should know cold

$2^0 = 1$

$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

$2^4 = 16$

$2^5 = 32$

$2^6 = 64$

$2^7 = 128$

$2^8 = 256$

$2^{10} = 1,024$

$2^{16} = 65,536$

$2^{20} = 1,048,576$

$2^{30} = 1,073,741,824$

$2^{32} = 4,294,967,296$

$2^{64} \sim 1.8 \times 10^{19}$

$\sim$ a thousand

$\sim$ a million

$\sim$ a billion

$\sim$ 4 billion

$2^{10} \sim$ a thousand, $2^{20} \sim$ a million, $2^{30} \sim$ a billion.

Those three cover almost every estimate you will make.

And note what n bits gives you: 2^n different values, from 0 to $2^n - 1$. Six weights, sixty-three amounts — because $2^6 - 1 = 63$. Nagappa's box was a six-bit number.

The six operators

Every one of them works on the bits independently, place by place.

$a = 12 = 1100$

$b = 10 = 1010$

$a \& b$ AND 1 only where BOTH are 1 $1000 = 8$

$a | b$ OR 1 where EITHER is 1 $1110 = 14$

$a \wedge b$ XOR 1 where EXACTLY ONE is 1 $0110 = 6$

$\sim a$ NOT flip every bit (see below)

$a \ll 1$ SHIFT LEFT everything moves left $11000 = 24$

$a \gg 1$ SHIFT RIGHT everything moves right $0110 = 6$

Say each one as a sentence and you will not confuse them. AND is both. OR is either. XOR is exactly one — which is the same as “different”, and that is the property the whole of the next XOR day rests on.

What shifting really means

Shifting left by one doubles. Shifting right by one halves, rounding down.

And it is worth seeing why, rather than memorising it. In base ten, moving every digit one place left multiplies by ten, because each place is ten times its neighbour. **In binary each place is twice its neighbour, so moving left multiplies by two.** It is the same fact.

```
1 << 0 = 1    000001
1 << 1 = 2    000010
1 << 2 = 4    000100
1 << 3 = 8    001000
1 << 4 = 16   010000
1 << 5 = 32   100000
```

`1 << k` IS 2^k . This is how you build a mask with a single bit set, and it appears in almost every bit problem.

Right shift throws away what falls off the end, which is why it is division rounded down: `7 >> 1` is `3`, not `3.5`.

The four edits

Reading and changing one bit are four one-liners, and they are worth knowing by heart.

GET	<code>(n >> position) & 1</code>	shift it to the bottom, mask
SET	<code>n (1 << position)</code>	OR forces a 1
CLEAR	<code>n & ~(1 << position)</code>	AND with all-1s-except-there
TOGGLE	<code>n ^ (1 << position)</code>	XOR flips

Each one builds `1 << position` — a mask with exactly one bit set — and then applies the operator whose sentence matches what you want. OR forces on. AND with a hole forces off. XOR flips. **That is not four things to memorise; it is one thing and three sentences.**

Negative numbers, and two's complement

A fixed-width machine has no minus sign — only bits. So negatives are stored using **two's complement**: flip every bit and add one.

```

in 8 bits

5   = 00000101
flip = 11111010
add 1= 11111011 = -5

check: 5 + (-5) should be 0
  00000101
+ 11111011
-----
 100000000    the ninth bit falls off the end
= 00000000    = 0. Correct.

```

Two properties follow, and both matter. The top bit is the sign — 1 means negative. And **addition just works**: the same circuit adds positives and negatives, which is the entire reason this scheme was chosen over anything more obvious.

Python does something different and it catches people. Python integers are **arbitrary width** — there is no top bit, because there is no top. So ~5 is **not** an eight-bit pattern; it is -6.

```

~x == -x - 1      always, in Python

~5  = -6
~0  = -1
~1  =  0

To get the fixed-width pattern you must MASK:
~5 & 0xFF = 250 = 11111010

```

And 1 << 64 in Python is a perfectly good number, where in C or Java it would overflow. **This makes Python friendly to learn on and slightly misleading**, so when a problem says “32-bit integer”, **mask with & 0xFFFFFFFF** and say that you are doing so.

The three tricks worth knowing today

n & (n - 1) clears the lowest set bit.

```

n      = 10110000
n - 1  = 10101111    borrowing flips the lowest 1 and
                    everything below it
n & (n-1) = 10100000 ← the lowest 1 is gone

```

Two things fall straight out. Counting set bits in one iteration per set bit rather than one per bit — thirty-two iterations become one, for a number like 1024. And $n \& (n - 1) = 0$ tests for a power of two, because a power of two has exactly one set bit, so clearing it leaves zero.

$n \& -n$ isolates the lowest set bit — the same fact wearing the other hat, and the thing a Fenwick tree is built on.

$x \& (size - 1)$ is $x \% size$ when $size$ is a power of two. This is why hash tables and buffers use power-of-two sizes: a mask is one instruction and a division is roughly twenty.

Where you have already met bits

BITMASK DP	a subset of n items as an n-bit number – exactly Nagappa's box
HASH TABLES	$\& (capacity - 1)$ instead of $\% capacity$
PERMISSIONS	read/write/execute as three bits: 7 = rwx
COLOURS	#FF8800 is three bytes: red, green, blue
NETWORK MASKS	/24 means "the top 24 bits identify the network"
FLAGS	one integer carrying 32 independent yes/no answers

All of them are Nagappa's answer: you are not remembering the amount, you are remembering which ones are out of the box.

4 The picture

Place value, side by side:

BASE TEN	BASE TWO
1 3 10 1	1 1 0 1 8 4 2 1 ^ ^ ^ ^
each place is 10x the one on its right	each place is 2x the one on its right
= 10 + 3 = 13	= 8 + 4 + 0 + 1 = 13
SAME IDEA. Only the multiplier changed.	

Nagappa's box, drawn:

	32	16	8	4	2	1	
thirty-seven	[X]	[]	[]	[X]	[]	[X]	= 32 + 4 + 1 = 37 100101
thirteen	[]	[]	[X]	[X]	[]	[X]	= 8 + 4 + 1 = 13 001101
sixty-three	[X]	[X]	[X]	[X]	[X]	[X]	= everything out 111111
zero	[]	[]	[]	[]	[]	[]	= nothing out 000000

SIX weights, $2^6 = 64$ arrangements, covering 0 to 63.
Each amount ONE way only – because two of any weight is the next one up, so you would have taken that instead.

The three operators, bit by bit:

```

a = 1 1 0 0 (12)
b = 1 0 1 0 (10)
-----
a & b = 1 0 0 0 ( 8) BOTH
a | b = 1 1 1 0 (14) EITHER
a ^ b = 0 1 1 0 ( 6) EXACTLY ONE (= "different")

```

Read each column on its own. Nothing carries, nothing borrows – that is what makes these fast and what makes them easy to reason about.

Shifting:


```
12 << 1
```

```
  1 1 0 0      everything moves one place LEFT
  | / | / | /   a 0 comes in at the right
  v v v v
1 1 0 0 0 = 24    DOUBLED
```

```
12 >> 1
```

```
  1 1 0 0      everything moves one place RIGHT
  \ \ \ \      the rightmost bit FALLS OFF
  v v v
0 1 1 0 = 6    HALVED
```

```
7 >> 1
```

```
  0 1 1 1
  \ \ \
  0 0 1 1 = 3    not 3.5 – the 1 that fell off is GONE
```

Right shift is division **ROUNDED DOWN**, and the rounding is simply the bit you dropped.

`n & (n - 1)`, the trick worth the day:

```
n      = 1 0 1 1 0 0 0 0    (176)
n - 1 = 1 0 1 0 1 1 1 1    borrowing flips the lowest 1
                                and turns everything below it to 1
-----
n&(n-1)= 1 0 1 0 0 0 0 0    (160)    the lowest 1 is GONE
```

Keep going until zero, counting:

```
10110000 = 176
10100000 = 160
10000000 = 128
00000000 = 0
```

THREE steps → three set bits.

The naive loop would take EIGHT steps here, and thirty-two on a 32-bit number, every time, regardless.

And `n & (n-1) == 0` means "exactly one bit was set"
→ a POWER OF TWO.

Two's complement:

8 bits, so 256 patterns. Split them down the middle:

```
00000000 = 0
00000001 = 1
...
01111111 = 127      top bit 0 → POSITIVE
-----
10000000 = -128     top bit 1 → NEGATIVE
10000001 = -127
...
11111111 = -1
```

TO NEGATE: flip every bit, add one.

```
5      00000101
flip   11111010
+1     11111011 = -5
```

WHY THIS SCHEME: addition needs no special case.

```
00000101 + 11111011 = 100000000
the ninth bit falls off → 0. Correct, with an
ordinary adder.
```

PYTHON HAS NO TOP BIT – integers are arbitrary width.

~5 is -6, not 11111010.

~5 & 0xFF is 250, which IS 11111010.

When a problem says "32-bit", mask, and say you are masking.

5 The code, built step by step

Converting, by hand

```
def to_binary(number: int, width: int = 8) → str:
    """Repeated division by two. The remainders, read UPWARDS, are the answer."""
    if number == 0:
        return "0".rjust(width, "0")
    bits = ""
    while number > 0:
        bits = str(number % 2) + bits      # newest remainder goes on the LEFT
        number //= 2
    return bits.rjust(width, "0")
```

`str(number % 2) + bits` — **prepending, not appending** — is the whole correctness of this function. The remainders come out in reverse order, so each new one belongs at the front. Appending gives 1011 for 13, which is 11, and is the single most common mistake in this conversion.

```
def from_binary(bits: str) → int:
    """Left to right: double what you have, then add the next bit."""
    value = 0
    for bit in bits:
        value = value * 2 + int(bit)
    return value
```

No powers, no place values, no indexing from the right. One rule applied once per character — **and it is much harder to get wrong under pressure than** `sum(int(b) * 2**i ...)`.

The four edits

```
def get_bit(number: int, position: int) → int:
    return (number >> position) & 1

def set_bit(number: int, position: int) → int:
    return number | (1 << position)

def clear_bit(number: int, position: int) → int:
    return number & ~(1 << position)

def toggle_bit(number: int, position: int) → int:
    return number ^ (1 << position)
```

Every one of them builds `1 << position` — **a mask with exactly one bit set — then applies the operator whose sentence matches the intent.** OR forces on, AND-with-a-hole forces off, XOR flips.

`~(1 << position)` is “all ones except there”, and in Python that is a negative number of unbounded width — **which is fine, because AND against a finite number only ever looks at the bits that exist.**

Counting set bits, two ways

```
def count_bits_loop(number: int) → int:
    """Look at every bit. 32 iterations for a 32-bit number, always."""
    count = 0
    while number:
        count += number & 1
        number >>= 1
    return count
```

This is the honest first answer and you should write it first. It is correct, it is obvious, **and it does work proportional to the position of the highest set bit** — thirty-two steps for `1 << 31`, however few bits are actually set.

```
def count_bits_kernighan(number: int) → int:
    """n & (n - 1) clears the LOWEST set bit. One iteration per SET bit."""
    count = 0
    while number:
        number &= number - 1
        count += 1
    return count
```

One iteration per *set* bit rather than per bit. For 1024 that is one step instead of eleven; for a sparse 64-bit value it is the difference between two steps and sixty-four.

Say the name — Brian Kernighan’s algorithm — because interviewers recognise it, and say what it does in one sentence: **subtracting one flips the lowest set bit and turns everything below it to ones, so ANDing removes exactly that bit.**

Powers of two

```
def is_power_of_two(number: int) → bool:
    """A power of two has exactly one set bit, so clearing it leaves zero."""
    return number > 0 and (number & (number - 1)) == 0
```

The `number > 0` guard is not decoration. `0 & -1` is `0`, so without it zero reports as a power of two — and negative numbers in two’s complement do strange things here too.

Two’s complement, made visible

```
def to_twos_complement(number: int, width: int = 8) → str:
    """How a fixed-width machine stores a negative number."""
    mask = (1 << width) - 1
    return to_binary(number & mask, width)
```

`(1 << width) - 1` is the all-ones mask — eight ones for `width = 8`. And this is the expression whose precedence trap is in section 7, so look at the brackets carefully.

`number & mask` on a negative Python integer produces the fixed-width pattern, because Python’s negatives behave as though they have infinitely many leading ones — which is exactly what two’s complement means.

The complete solution

```
"""Day 171 – reading, writing, and manipulating bit patterns."""

from __future__ import annotations

def to_binary(number: int, width: int = 8) → str:
    """Repeated division by two. The remainders, read UPWARDS, are the answer."""
    if number == 0:
        return "0".rjust(width, "0")
    bits = ""
    while number > 0:
        bits = str(number % 2) + bits      # newest remainder goes on the LEFT
        number //= 2
    return bits.rjust(width, "0")

def from_binary(bits: str) → int:
    """Left to right: double what you have, then add the next bit."""
    value = 0
    for bit in bits:
        value = value * 2 + int(bit)
    return value

def get_bit(number: int, position: int) → int:
    """Is bit `position` set? Shift it down to the bottom and mask."""
    return (number >> position) & 1

def set_bit(number: int, position: int) → int:
    """Turn bit `position` ON. OR with a single 1 in that place."""
    return number | (1 << position)

def clear_bit(number: int, position: int) → int:
    """Turn bit `position` OFF. AND with a mask of all 1s except there."""
    return number & ~(1 << position)

def toggle_bit(number: int, position: int) → int:
    """Flip bit `position`. XOR is the flip operator."""
    return number ^ (1 << position)

def count_bits_loop(number: int) → int:
    """Look at every bit. 32 iterations for a 32-bit number, always."""
    count = 0
    while number:
        count += number & 1
        number >>= 1
    return count

def count_bits_kernighan(number: int) → int:
    """n & (n - 1) clears the LOWEST set bit. One iteration per SET bit."""
    count = 0
    while number:
        number &= number - 1
        count += 1
    return count
```

```

def is_power_of_two(number: int) → bool:
    """A power of two has exactly one set bit, so clearing it leaves zero."""
    return number > 0 and (number & (number - 1)) == 0

def to_twos_complement(number: int, width: int = 8) → str:
    """How a fixed-width machine stores a negative number."""
    mask = (1 << width) - 1
    return to_binary(number & mask, width)

if __name__ == "__main__":
    print("PLACE VALUE - 13 in binary")
    print(f" to_binary(13, 4) = {to_binary(13, 4)} (8 + 4 + 0 + 1)")
    print(f" from_binary('1101') = {from_binary('1101')}")
    print(f" Python agrees: bin(13) = {bin(13)}")

    print()
    print("THE FIVE OPERATORS on 12 (1100) and 10 (1010)")
    a, b = 12, 10
    print(f" a = {to_binary(a, 4)} = {a}")
    print(f" b = {to_binary(b, 4)} = {b}")
    print(f" a & b = {to_binary(a & b, 4)} = {a & b} (both)")
    print(f" a | b = {to_binary(a | b, 4)} = {a | b} (either)")
    print(f" a ^ b = {to_binary(a ^ b, 4)} = {a ^ b} (exactly one)")
    print(f" a << 1 = {to_binary(a << 1, 5)} = {a << 1} (doubled)")
    print(f" a >> 1 = {to_binary(a >> 1, 4)} = {a >> 1} (halved, rounded down)")

    print()
    print("SHIFTING IS DOUBLING")
    for k in range(6):
        print(f" 1 << {k} = {1 << k}>2} {to_binary(1 << k, 6)}")

    print()
    print("THE FOUR EDITS on 0b1010 (10), position 2")
    n = 0b1010
    print(f" start {to_binary(n, 4)} = {n}")
    print(f" get_bit(n, 2) = {get_bit(n, 2)}")
    print(f" set_bit(n, 2) {to_binary(set_bit(n, 2), 4)} = {set_bit(n, 2)}")
    print(f" clear_bit(n,1) {to_binary(clear_bit(n, 1), 4)} = {clear_bit(n, 1)}")
    print(f" toggle_bit(n,0){to_binary(toggle_bit(n, 0), 4)} = {toggle_bit(n, 0)}")

    print()
    print("COUNTING SET BITS")
    for value in (0, 1, 7, 8, 255, 1024):
        print(f" {value:>5} {to_binary(value, 11)} loop={count_bits_loop(value)}"
              f" kernighan={count_bits_kernighan(value)}"
              f" python={value.bit_count()}")

    print()
    print("n & (n - 1) CLEARS THE LOWEST SET BIT")
    n = 0b10110000
    while n:
        print(f" {to_binary(n, 8)} = {n:>3}")
        n &= n - 1
    print(f" {to_binary(0, 8)} = 0")

    print()
    print("POWERS OF TWO")
    for value in (1, 2, 3, 16, 63, 64):
        print(f" {value:>3} → {is_power_of_two(value)}")

```

```

print()
print("NEGATIVE NUMBERS – two's complement in 8 bits")
for value in (5, -5, 1, -1, 127, -128):
    print(f" {value:>5} stored as {to_twos_complement(value)}")
print(f" Python's ~5 = {~5} (that is -5-1, not an 8-bit pattern)")
print(f" masked to 8 bits: {to_binary(~5 & 0xFF, 8)} = {~5 & 0xFF}")

print()
print("VERIFICATION")
bad = 0
for value in range(0, 5000):
    if to_binary(value, 1) != bin(value)[2:]:
        bad += 1
    if from_binary(bin(value)[2:]) != value:
        bad += 1
    if count_bits_loop(value) != value.bit_count():
        bad += 1
    if count_bits_kernighan(value) != value.bit_count():
        bad += 1
    if is_power_of_two(value) != (value > 0 and bin(value).count("1") == 1):
        bad += 1
print(f" {bad} mismatches over 5,000 values, 5 checks each")

```

Running it:

```
PLACE VALUE – 13 in binary
to_binary(13, 4) = 1101    (8 + 4 + 0 + 1)
from_binary('1101') = 13
Python agrees: bin(13) = 0b1101
```

THE FIVE OPERATORS on 12 (1100) and 10 (1010)

```
a      = 1100 = 12
b      = 1010 = 10
a & b = 1000 = 8   (both)
a | b = 1110 = 14  (either)
a ^ b = 0110 = 6   (exactly one)
a << 1 = 11000 = 24 (doubled)
a >> 1 = 0110 = 6   (halved, rounded down)
```

SHIFTING IS DOUBLING

```
1 << 0 = 1   000001
1 << 1 = 2   000010
1 << 2 = 4   000100
1 << 3 = 8   001000
1 << 4 = 16  010000
1 << 5 = 32  100000
```

THE FOUR EDITS on 0b1010 (10), position 2

```
start      1010 = 10
get_bit(n, 2) = 0
set_bit(n, 2) 1110 = 14
clear_bit(n,1) 1000 = 8
toggle_bit(n,0)1011 = 11
```

COUNTING SET BITS

0	000000000000	loop=0	kernighan=0	python=0
1	000000000001	loop=1	kernighan=1	python=1
7	000000000111	loop=3	kernighan=3	python=3
8	000000001000	loop=1	kernighan=1	python=1
255	000111111111	loop=8	kernighan=8	python=8
1024	100000000000	loop=1	kernighan=1	python=1

n & (n - 1) CLEARS THE LOWEST SET BIT

```
10110000 = 176
10100000 = 160
10000000 = 128
00000000 = 0
```

POWERS OF TWO

```
1 → True
2 → True
3 → False
16 → True
63 → False
64 → True
```

NEGATIVE NUMBERS – two's complement in 8 bits

```
5 stored as 00000101
```



```
-5 stored as 11111011
 1 stored as 00000001
-1 stored as 11111111
127 stored as 01111111
-128 stored as 10000000
Python's ~5 = -6 (that is -5-1, not an 8-bit pattern)
masked to 8 bits: 11111010 = 250
```

VERIFICATION

0 mismatches over 5,000 values, 5 checks each

Look at the **255** line: the naive loop takes eight steps and Kernighan also takes eight, because every bit is set. And on **1024** the loop takes eleven and Kernighan takes one. That contrast is the whole argument for the trick, and it is worth having both numbers to hand.

In an interview you would use `n.bit_count()` (Python 3.10 and later), and say so — but write Kernighan first, because the question is almost never really about counting.

6 What it costs

Converting.

```
to_binary(n): one division per bit
13 → 4 divisions
1000 → 10 divisions
10^9 → 30 divisions
```

→ $O(\log n)$ in the VALUE of n , or $O(b)$ in its bit-length.

This is the distinction that matters and interviewers probe it: a loop over the BITS of n is logarithmic in n , not linear.

Counting set bits — count the iterations out loud.

NAIVE LOOP: one iteration per bit position, up to the highest set bit

1024 = 10000000000

→ 11 iterations, to find ONE set bit

1 << 31

→ 32 iterations, to find ONE set bit

KERNIGHAN: one iteration per SET bit

1024 → 1 iteration

255 → 8 iterations (every bit set – no better)

176 → 3 iterations

1 << 31 → 1 iteration

→ naive is $O(b)$, where b is the bit-length: 32 or 64

→ Kernighan is $O(s)$, where s is the number of set bits

WORST CASE THEY ARE THE SAME (all bits set).

TYPICALLY Kernighan is far fewer, and on sparse values it is 1 against 64.

And the honest framing: both are constant time on fixed-width integers, because thirty-two is a constant. **Say that** — it is the more accurate statement, and then say that the constant still matters when you are doing it in a loop over a million values.

The bitmask-DP connection, since the arithmetic is the same.

a subset of n items as an n -bit number:

$n = 20 \rightarrow 2^{20} = 1,048,576$ subsets fine

$n = 25 \rightarrow 2^{25} = 33,554,432$ getting slow

$n = 30 \rightarrow 2^{30} = 1,073,741,824$ no

$n = 64 \rightarrow 2^{64} = 18,446,744,073,709,551,616$

→ which is why bitmask solutions cap around $n = 20$,
and why " $n \leq 20$ " in a problem statement is a hint

That is Nagappa's box again: six weights, 2^6 arrangements.
Twenty items, 2^{20} .

Space.

every function here uses $O(1)$ extra space

except `to_binary`, which builds a string:
 $O(b)$ characters, so ~ 32 or ~ 64 bytes

Note that string CONCATENATION in a loop is $O(b^2)$ work in total, because each `+` copies. For 64 bits that is 2,048 character copies – irrelevant here, and worth knowing about before you write the same loop over a million-character string.

Why powers of two are everywhere.

```
x % size      ~ 20-40 CPU cycles  (integer division is slow)
x & (size-1) ~ 1 CPU cycle       (when size is a power of two)
```

→ a 20-40x difference on an operation a hash table does on EVERY lookup

→ which is why hash tables, ring buffers and memory pages all have power-of-two sizes. It is not aesthetics.

7 The traps

Reading the remainders downwards.

```
13 / 2 = 6 r 1
 6 / 2 = 3 r 0
 3 / 2 = 1 r 1
 1 / 2 = 0 r 1
```

upwards: 1101 = 13 ← correct

downwards: 1011 = 11 ← wrong, and a perfectly plausible number

Nothing errors. You get a valid binary string for a different number. **Say the direction out loud every time you do this by hand** — it is the one step of the conversion that has no self-check.

The shift-precedence trap, which is the real one in Python.

```
n = 4
mask = 1 << n - 1      # meant to be "all ones in n bits"
```

```
1 << n - 1 = 8
(1 << n) - 1 = 15
```

Subtraction binds tighter than shifting, so `1 << n - 1` is `1 << (n - 1)`, which is a single bit rather than a full mask. No error, a plausible number, and every downstream result is quietly wrong.

Always bracket shifts. `(1 << n) - 1`. It costs two characters.

Assuming C's precedence rule. In C, `x & 1 == 0` parses as `x & (1 == 0)` and is a famous bug. In Python `&` binds *tighter* than `=`, so `x & 1 == 0` does mean `(x & 1) == 0` and works. **Know that this differs between languages**, and bracket it anyway so the reader does not have to.

`~` in Python is not the fixed-width complement.

```
~5  = -6
~0  = -1
```

Python integers have no top bit, so `~x` is always `-x - 1`. If you want the eight-bit pattern you must mask:

```
~5 & 0xFF = 250 = 11111010
```

This bites hardest on LeetCode problems that say “32-bit signed integer”. The fix is `& 0xFFFFFFFF`, and converting back to a signed value if the top bit is set — and saying out loud that you are doing it, because otherwise it looks like magic.

Right-shifting a negative number does not do what people expect.

```
-8 >> 1 = -4
-7 >> 1 = -4      not -3
-1 >> 5 = -1      it never reaches zero
```

Python's `>>` is an *arithmetic* shift: it copies the sign bit in from the left. So `while number: number >>= 1` never terminates on a negative input — the value stays `-1` forever. A counting loop written without a guard hangs rather than crashing, which is the worst failure mode there is.

Shifting by a negative amount.

```
1 << -1
```

```
Traceback (most recent call last):
  File "<stdin>", line 8, in <lambda>
ValueError: negative shift count
```

Loud and clear, which puts it in the good category.

Shifting by a float.

```
1 << 2.0
```

```
Traceback (most recent call last):
  File "<stdin>", line 9, in <lambda>
TypeError: unsupported operand type(s) for <<: 'int' and 'float'
```

This is the one that catches you after a division. `n / 2` gives a float in Python 3; `n // 2` gives an integer. Use `//` anywhere near bit work.

Bitwise operators on floats at all.

```
1.5 & 1
```

```
Traceback (most recent call last):
  File "<stdin>", line 12, in <lambda>
TypeError: unsupported operand type(s) for &: 'float' and 'int'
```

Forgetting the base in `int()`.

```
int('1101')      = 1101      ← a thousand one hundred and one
int('1101', 2) = 13         ← correct
```

No error, and the wrong number is a thousand times too big. `int(s, 2)` — the second argument is not optional when you mean binary.

`bin()` on a negative number.

```
bin(-5) = '-0b101'
```

A minus sign and the magnitude — not two's complement. Slicing off the first two characters with `[2:]` gives `'b101'`, which then fails or produces nonsense. Use `bin(x & 0xFF)[2:]` when you want a pattern.

Zero reporting as a power of two. Without the `number > 0` guard, `is_power_of_two(0)` is `True`, because `0 & -1 == 0`. One comparison, and the test cases that catch it are `0` and negative inputs — both of which people forget to try.

8 In the interview

How it gets asked

- “*What is 13 in binary?*” — the warm-up, and it is testing whether you can do it calmly.
- “*What does shifting left by one do?*” — and **why**.
- “*Count the number of set bits.*” — then “can you do better?”
- “*Is this a power of two?*” — a one-liner if you know it.
- “*How are negative numbers stored?*” — two’s complement.
- “*Why do hash tables use power-of-two sizes?*” — the mask against the division.

The first ninety seconds

On “what is 13 in binary, and what does shifting do”:

“Thirteen is 1101.

I would get there by place value. In binary each place is twice the one to its right, so the places are 8, 4, 2, 1. **Thirteen is 8 plus 4 plus 1** — so ones in the 8, 4 and 1 places, and a zero in the 2 place. **1101.**

The mechanical way, if the number is bigger, is repeated division by two, keeping the remainders. Thirteen over two is six remainder one; six over two is three remainder nought; three over two is one remainder one; one over two is nought remainder one. **And you read the remainders upwards — 1101.**

Upwards, not downwards, and I say that out loud every time — reading them downwards gives 1011, which is eleven, **and it is a completely plausible-looking wrong answer with no self-check.**

Shifting left by one doubles. Shifting right by one halves, rounded down.

And the reason is the same fact as place value. In base ten, moving every digit one place left multiplies by ten, because each place is ten times its neighbour. **In binary each place is twice its neighbour, so moving left multiplies by two.**

`1 << k` is `2^k` — that is how you build a mask with one bit set, and it is in essentially every bit problem.

The rounding down in right-shift is just the bit that fell off the end. `7 >> 1` is 3, not 3.5 — **the one you dropped was the half.**

One caveat about Python I would mention: integers are arbitrary width, so `1 << 64` is a perfectly good number here where in C or Java it would overflow. **If the problem says thirty-two bits, I would mask with** `& 0xFFFFFFFF` **and say that I am doing it.**“

The follow-ups

“Count the set bits. Now do it faster.”

“The obvious version looks at every bit: mask the bottom one, add it, shift right, repeat until zero. **I would write that first,** because it is correct and clear.

Its cost is one iteration per bit position, up to the highest set bit — so thirty-two iterations to find one set bit in `1 << 31`. The work depends on where the highest bit is, **not on how many bits are set,** and that is the inefficiency.

The faster version is `n &= n - 1` in a loop, counting iterations. That is Brian Kernighan’s algorithm.

Here is why it works. Subtracting one **flips the lowest set bit to zero and turns everything below it into ones** — that is what borrowing does. **So ANDing the original with it clears exactly the lowest set bit and leaves everything above untouched.**

So you get one iteration per set bit. For `1024` that is one step instead of eleven. For `1 << 31`, one instead of thirty-two.

I would be honest about the worst case: if every bit is set they are identical. `255` takes eight steps either way. **The gain is on sparse values, which is most real data.**

And strictly both are constant time on fixed-width integers, since thirty-two is a constant — **but the constant matters when you are doing this inside a loop over a million values.**

The same identity gives you a power-of-two test for free. A power of two has exactly one set bit, **so clearing it must leave zero:** `n > 0 and (n & (n - 1)) == 0`. **The `n > 0` is not decoration** — without it, zero reports as a power of two.

In production I would write `n.bit_count()` — Python 3.10 and later, and it compiles to a single CPU instruction. **I would say that, and then write Kernighan anyway, because the question is not really about counting.**“

“How are negative numbers stored?”

“Two’s complement: flip every bit and add one.

In eight bits, five is 00000101. Flip it: 11111010. Add one: 11111011, which is minus five.

The check is that they must add to zero. 00000101 plus 11111011 is 100000000 — nine bits, and the top one falls off the end, leaving 00000000. Zero. Correct.

And that falling-off is the entire reason this scheme won, over anything more obvious like a sign bit. The same adder circuit handles positives and negatives with no special case. Sign-and-magnitude would need the hardware to check signs and branch, **and it also gives you two different zeros**, which causes problems everywhere.

Two consequences worth knowing. The top bit is the sign — one means negative. And the range is asymmetric: eight bits gives minus 128 to plus 127, because zero takes up one of the positive slots. That asymmetry is why `abs(-2**31)` overflows a 32-bit integer, which is a real bug people hit.

Python is the exception and it catches people. Python integers are **arbitrary width**, so there is no top bit at all. `~5` is minus six, **not an eight-bit pattern** — the rule is `~x == -x - 1`.

To get the fixed-width pattern you mask: `~5 & 0xFF` is 250, which is 11111010.

And there is a related trap: Python’s right shift is arithmetic — it copies the sign bit in from the left. So `-1 >> 5` is still minus one, **and a `while number: number >>= 1` loop never terminates on a negative input.** It hangs rather than crashing, **which is the worst way for a bug to behave**, so I would guard or mask before any bit loop that might see a negative.”

“Why do hash tables use power-of-two sizes?”

“Because `x % size` becomes `x & (size - 1)` when the size is a power of two, and that is a twenty- to forty-fold speedup on an operation the table does on every single lookup.

The reason it works is place value. In a power-of-two size, the remainder is exactly the low bits — the higher bits are all multiples of the size and contribute nothing to the remainder. So masking off everything above them gives the same answer as dividing.

Concretely: `size = 16` means `size - 1` is 15, which is 1111. ANDing with 1111 keeps the bottom four bits, and the bottom four bits of any number are precisely that number modulo 16.

The cost difference is real. Integer division is roughly twenty to forty CPU cycles; a bitwise AND is one. For an operation on the hot path of every lookup, that is not a micro-optimisation.

It is the same reason ring buffers and memory pages are powers of two. Wrapping an offset is a mask, not a division.

And there is a cost, which is worth naming. Masking only looks at the low bits, so if the hash function produces values whose low bits are poorly distributed — object addresses, or sequential ids multiplied by something — every key lands in a few buckets. Modulo by a prime is more forgiving of a bad hash.

Which is why Java’s HashMap mixes the high bits down into the low ones before masking: `h ^ (h >>> 16)`. It buys the speed of the mask and pays for it with one extra XOR to protect against a weak hash. That trade is the actual answer to the question.”

The model answer

“Explain binary to me as though I have never seen it, then tell me why it matters.”

“Binary is place value with two instead of ten.

In base ten, the places are 1, 10, 100 — each one ten times the last, and each digit runs 0 to 9. In binary the places are 1, 2, 4, 8, 16 — each one twice the last — and each digit is only ever 0 or 1.

So 1101 means 8 plus 4 plus 0 plus 1, which is 13.

The reason it is two and not ten is physical. A wire is either carrying current or it is not. Two states are easy to build reliably and ten are not, so everything above is built on the two.

A bit is one of those switches. Eight bits is a byte. And n bits gives 2^n different values, from 0 to $2^n - 1$ — which is why eight bits covers 0 to 255, and why you see 255 everywhere.

Converting is two rules. Going in: divide by two repeatedly and read the remainders upwards. Coming out: left to right, double what you have and add the next bit. I say ‘upwards’ out loud every time, because reading them downwards gives a plausible wrong number with no self-check.

There are six operators and each is one sentence. AND is both. OR is either. XOR is exactly one — which is the same as ‘different’, and that is the property the whole XOR family of problems rests on. NOT flips everything. Left shift doubles, right shift halves rounding down — and that is the same place-value fact, because moving left multiplies by whatever each place is worth.

Negative numbers are two’s complement: flip and add one. The point of it is that addition then needs no special case — the same circuit handles both signs, and the carry that falls off the end does the work.

Why it matters, in three places I have already used it.

A subset of twenty items is a twenty-bit number — that is what bitmask DP is, and it is why those problems cap around twenty, because 2^{20} is a million and 2^{30} is a billion.

Hash tables use power-of-two sizes so that modulo becomes a mask — $x \& (size - 1)$ — which is one CPU cycle instead of twenty to forty, on an operation done at every lookup.

And $n \& (n - 1)$ clears the lowest set bit, which gives you set-bit counting in one step per set bit and a power-of-two test as a one-liner.

The thing I would want to leave you with is the shift in view. A number is not a quantity that happens to be stored somehow. It is a pattern of switches, and the operators let you work on the pattern directly rather than through arithmetic. Once that lands, these problems stop looking like tricks.”

9 Recall card

Binary is place value with two instead of ten: the places are 1, 2, 4, 8, 16, each twice the last, each digit 0 or 1. $13 = 8 + 4 + 1 = 1101$. n bits gives 2^n values, 0 to $2^n - 1$. Know $2^{10} \approx$ a thousand, $2^{20} \approx$ a million, $2^{30} \approx$ a billion.

Converting, two rules. In: divide by two, read the remainders UPWARDS (downwards gives $1011 = 11$, a plausible wrong answer with no self-check). Out: left to right, double and add the next bit.

Six operators, one sentence each. AND = both. OR = either. XOR = exactly one, which is the same as different. NOT flips. $\ll 1$ doubles, $\gg 1$ halves rounding down — the same place-value fact, and the rounding is just the bit that fell off. $1 \ll k$ is 2^k , the single-bit mask.

The four edits, all built from $1 \ll \text{position}$: GET $(n \gg p) \& 1$; SET $n | (1 \ll p)$; CLEAR $n \& \sim(1 \ll p)$; TOGGLE $n \wedge (1 \ll p)$.

Three tricks. $n \& (n - 1)$ clears the lowest set bit — subtracting one flips it and ones-fills below, so ANDing removes exactly it. That gives Kernighan's count (one step per SET bit: 1 instead of 32 for $1 \ll 31$, but no better when all bits are set) and $n > 0$ and $(n \& (n-1)) = 0$ for powers of two — the guard is not decoration, or zero passes. $x \& (\text{size} - 1)$ is $x \% \text{size}$ for power-of-two sizes: one CPU cycle against twenty to forty, which is why hash tables and ring buffers are sized that way.

Negatives are two's complement — flip and add one — so one adder handles both signs and the carry falls off the end. The range is asymmetric (-128 to 127). Python is arbitrary-width, so $\sim x = -x - 1$, not a pattern — mask with $\& 0xFF$ or $\& 0xFFFFFFFF$. And $\gg$ is arithmetic: $-1 \gg 5$ is still -1 , so a `while n: n  $\gg=$  1` loop HANGS on a negative. Always bracket shifts — $1 \ll n - 1$ is $1 \ll (n-1)$, not $(1 \ll n) - 1$.

Monitoring, metrics, and alerting

1 What this is, and why they ask it

Monitoring is how you find out that the thing you built is broken. Metrics are the numbers you collect. Alerting is the part that wakes someone up.

They ask it because **every high-level design interview ends here** — “what would you measure?”, “what would page you?” — and because **it is the question where prepared candidates most often have nothing**. They can design the system and cannot say how they would know it was working.

And because the answers separate people who have been on call from people who have not. Anyone can say “monitor CPU and memory”. **The interesting answers are the ones that come from having been woken at three in the morning by something that did not matter.**

Three ideas carry almost the whole topic.

Averages lie, and percentiles do not. An average response time of a hundred milliseconds can hide fifty users waiting two seconds. **The average describes nobody.**

Alert on symptoms, not on causes. A machine at ninety percent memory is not a problem if users are fine. **Users getting errors is a problem even if every machine looks healthy.**

And every alert that fires without needing action makes every other alert less effective. This is the one that has a body count — **not because the systems failed, but because the person on call had stopped listening.**

By the end of this lesson you can name the four golden signals, explain why the average is the wrong statistic and why percentiles cannot be averaged either, size a metrics system’s cardinality, write an alert that deserves to wake someone, and answer “what would you put on the dashboard” for any system you have just designed.

2 The story

The bell at the cold storage was wired to everything, and that was the trouble.

Karim had the night shift, in a room with a stool and a heater and a bell on the wall above the door. When something at the plant was not right, the bell rang, and he went and looked.

In the first week he went every time.

The bell rang because the outer gate was open — the loading men had propped it with a crate again. It rang because one of the small motors near the back was running warm, which it always did after midnight and always settled by itself. It rang because a fuse in the lighting had gone. It rang, once, because a cat had got in and sat on something.

By the second week he had counted eleven ringings in a night.

By the third week he went when it rang twice in a row.

By the fifth week he did not go at all until morning, **and he was not being lazy — he had learnt, correctly, that the bell did not mean anything.**

And then there was a night in March.

The bell rang at about half past one, and Karim heard it, and turned over.

By the time the day men came in, everything in the far room had spoiled.

The manager came down and Karim expected to be shouted at, and instead the man stood in the doorway for a long while looking at the bell.

Then he said: “How many times did that thing ring last week?”

Karim told him. Seventy-something. Maybe more.

“And how many of them were worth getting up for?”

Karim thought about it and said one. **The one in March.**

They rewired it that Thursday. **The bell was allowed to ring for exactly one thing — the temperature in the rooms going the wrong way.** Everything else — the gate, the warm motor, the lights — went onto a board on the wall that Karim walked past and looked at when he came on and when he left.

Nobody added anything. They took things away.

And the manager said one more thing on his way out, which Karim repeated for years afterwards.

“The bell was never broken. It rang perfectly, every time. It was just ringing about things that were not the point.”

3 The idea in plain English

Karim's rewiring is the whole topic. The board on the wall is a **dashboard**. The bell is an **alert**. And the fix was not a better bell — **it was deciding which single thing was worth waking a person for.**

The three pillars

Metrics, logs, and traces answer different questions and cost wildly different amounts.

METRICS	numbers over time: requests/second, error rate, p99 CHEAP – a few bytes per sample, aggregated on the way in ANSWERS: "is something wrong, and roughly where?" CANNOT ANSWER: "what happened to this particular request?"
LOGS	one line per event, with detail EXPENSIVE – kilobytes per request ANSWERS: "what exactly happened at 02:14?" CANNOT ANSWER: "is the system healthy right now?" (too slow)
TRACES	one request's journey across every service it touched VERY EXPENSIVE – sampled, typically at 1% ANSWERS: "where did those 800 milliseconds go?" CANNOT ANSWER: anything about the requests you didn't sample

The workflow is metrics to detect, traces to localise, logs to diagnose. You do not choose between them — you use each for the question it answers cheaply.

The four golden signals

These four cover almost every service. They come from Google's SRE practice and they are worth knowing by name.

LATENCY	how long requests take → and SPLIT successful from failed: a fast 500 is not good news, and it drags the average DOWN
TRAFFIC	how many requests, per second
ERRORS	what fraction fail → including the ones that "succeed" with the wrong answer, which is the hard part
SATURATION	how full the system is – the resource nearest its limit → the LEADING indicator: the only one that warns you BEFORE users notice

Two shorthands you will hear. RED — Rate, Errors, Duration — for services. USE — Utilisation, Saturation, Errors — for resources like disks and queues. They are the same four signals viewed from either end.

Percentiles, because the average lies

This is the single most useful idea in the lesson.

```
1,000 requests:
  950 took 10 ms
  50 took 2,000 ms
```

```
AVERAGE = (950 x 10 + 50 x 2,000) / 1,000 = 109.5 ms
```

```
And NO REQUEST TOOK 109.5 ms.
The average describes nobody.
```

```
MEDIAN (p50) = 10 ms      "a typical request is fast"
p95           = 10 ms
p99           = 2,000 ms   "one in a hundred waits two seconds"
```

A dashboard showing 109 milliseconds looks fine. The fifty people who waited two seconds are invisible in it. Percentiles make them visible, and that is the entire argument.

And p99 matters more than “one percent” sounds, because a single page load makes many requests.

```
a page that makes 20 backend calls:
P(all 20 under p99) = 0.99^20 = 0.818
```

```
→ 18% of page loads contain at least one p99-slow call
```

```
at 100 calls: 0.99^100 = 0.366
→ 63% of page loads hit a p99 call
```

So “only one percent of requests are slow” can mean “most page loads feel slow”. That arithmetic is worth having ready — it is the strongest possible argument for caring about the tail.

And here is the part that catches people: you cannot average percentiles.


```
machine A: 1,000,000 requests, p99 = 100 ms
machine B:      100 requests, p99 = 10,000 ms
```

```
average of the two p99s = 5,050 ms
```

```
TRUE combined p99: the slowest 1% of 1,000,100 requests
is about 10,001 requests, essentially all from A
→ ~100 ms
```

```
THE AVERAGE OF THE PERCENTILES IS 50x THE TRUTH.
```

The fix is histograms. Each machine exports **bucket counts** — “how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms” — and **you sum the buckets across machines and compute the percentile from the sum.** That is exactly what Prometheus’s `histogram_quantile` does, and it is why **counters and histograms are the metric types that aggregate and pre-computed percentiles are not.**

SLI, SLO, SLA, and the error budget

Three words that get confused, and the distinction is worth thirty seconds in an interview.

SLI	Service Level INDICATOR	the measurement "99.95% of requests succeeded in the last 30 days"
SLO	Service Level OBJECTIVE	the target you hold yourselves to "99.9% of requests succeed"
SLA	Service Level AGREEMENT	the contract, with money attached "99.5%, or we refund 10%" → ALWAYS looser than the SLO, deliberately

And the error budget is what makes an SLO useful rather than decorative.

```
an SLO of 99.9% is a BUDGET of 0.1% failure
```

```
0.1% of a 30-day month = 43 minutes of downtime
```

```
→ if you have used 5 minutes, ship freely
```

```
→ if you have used 40, freeze and fix
```

```
THE POINT: 100% is not a target. It is not achievable and
aiming for it means never shipping anything. The budget
converts reliability from an argument into arithmetic.
```

Alert on symptoms, not causes

Karim's manager rewired the bell to the temperature in the rooms — the thing that actually mattered — and put the gate and the warm motor on the board. That is the rule.

ALERT (wake someone)

error rate above 1%
p99 above 2 seconds
queue depth growing for
10 minutes
no successful writes in
5 minutes

DASHBOARD (look at it in the morning)

a machine at 90% memory
one node out of fifty unhealthy
cache hit rate down 5%
disk 70% full

THE DIFFERENCE: the left-hand column is what a USER FEELS.
The right-hand column is a CAUSE that may or may not become
a symptom.

A machine at ninety percent memory that is serving users perfectly is not an emergency. Users getting errors while every machine looks healthy is an emergency — and cause-based alerting misses it completely, because you cannot enumerate every cause in advance. You can enumerate the symptoms, and there are about five of them.

The one exception is saturation, which is a cause you *do* alert on — “the disk will be full in four hours” — precisely because the symptom, when it arrives, is unrecoverable.

Every page must pass three tests

- | | |
|---------------|---|
| 1. ACTIONABLE | is there something a human can do right now?
if not, it is a dashboard entry |
| 2. URGENT | must it be done now, or can it wait until morning?
if it can wait, it is a ticket |
| 3. HUMAN | does it need a person, or could it be automated?
if it can be automated, automate it |

Anything failing one of the three is Karim's bell. And every unnecessary page makes every real page less effective, because the person on call is learning — correctly — that the bell does not mean anything.

The measurable version: track your alert-to-incident ratio. If fewer than half of pages correspond to a real problem, the problem is the alerting, not the system.

Cardinality, which is what actually kills metrics systems

Every distinct combination of label values is a separate stored series. This multiplies, and it multiplies faster than anyone expects.

```
http_requests_total{endpoint, status, region, instance}

50 endpoints x 5 statuses x 10 regions x 500 instances
= 1,250,000 series

add user_id, with 10,000,000 users:
= 12,500,000,000,000 series

→ the monitoring system falls over before the system it
   is monitoring does
```

Never put an unbounded value in a label. User ids, request ids, email addresses, full URLs with query strings, error messages. **Those belong in logs and traces, which are built for high cardinality; metrics are not.**

This is the most common real-world monitoring outage, and it is worth naming as a design constraint rather than an operational footnote.

4 The picture

The three pillars, and which question each answers:

```
ALERT FIRES: "error rate above 1%"
|
v
METRICS    "errors started at 02:14, only in the eu-west region,
            only on the checkout endpoint"
            → cheap, aggregated, always on
            |
            v
TRACES     "of the failing requests, 800 ms is spent in the
            payments service, and it then times out"
            → sampled at 1%, one request's full journey
            |
            v
LOGS       "connection pool exhausted: 100/100 in use"
            → expensive, full detail, the actual cause

DETECT with metrics. LOCALISE with traces. DIAGNOSE with logs.
Not a choice – a sequence.
```

Why the average lies:

1,000 requests

```
950 at 10 ms #####  
50 at 2,000 ms ##
```

AVERAGE 109.5 ms ← describes NOBODY. No request
 took anything like this.

p50 10 ms ← "a typical request is fast"

p95 10 ms

p99 2,000 ms ← the fifty people who waited

A dashboard reading "109 ms" looks healthy.

Fifty users waited two seconds and are invisible in it.

AND THE TAIL COMPOUNDS:

one page load = 20 backend calls

$P(\text{all 20 fast}) = 0.99^{20} = 0.818$

→ 18% of PAGE LOADS contain a p99-slow call

→ at 100 calls: 63%

"Only 1% of requests are slow" can mean

"most page loads feel slow".

Why percentiles cannot be averaged:

```
machine A    1,000,000 requests    p99 =    100 ms
machine B           100 requests    p99 = 10,000 ms
```

NAIVE: $(100 + 10,000) / 2 = 5,050$ ms

TRUTH: 1,000,100 requests total
the slowest 1% is ~10,001 requests
essentially ALL of them are A's
→ combined p99 ~ 100 ms

THE NAIVE ANSWER IS 50x WRONG.

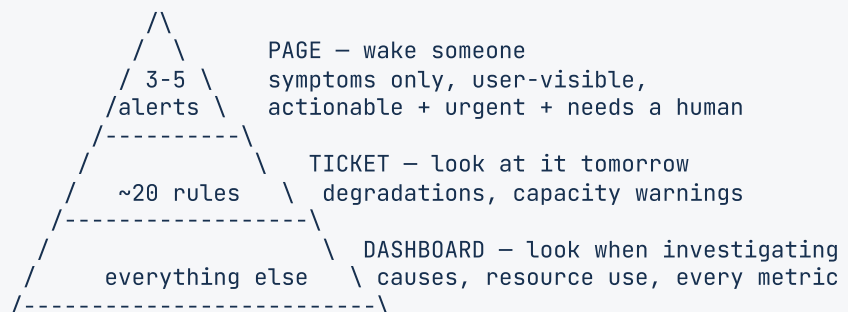
THE FIX – export HISTOGRAM BUCKETS, sum them, then compute:

bucket	A	B	SUM
0-10 ms	900,000	0	900,000
10-100 ms	90,000	5	90,005
100ms-1s	9,900	30	9,930
1-10 s	100	65	165

→ now find the rank-990,099 request in the SUM
→ this is what `histogram_quantile()` does

Counters and histograms AGGREGATE. Pre-computed percentiles DO NOT. That is why the metric type matters.

The alerting pyramid:



KARIM'S RULE: the bell rings for ONE thing.
Everything else goes on the board.

THE FIX IS ALWAYS SUBTRACTION. Nobody has ever improved
an alerting system by adding alerts.

The error budget, as arithmetic:

SL0 99.9% → budget 0.1%

per 30-day month (43,200 minutes):

99%	432 minutes	= 7.2 hours	
99.9%	43 minutes		
99.95%	22 minutes		
99.99%	4.3 minutes		
99.999%	26 seconds		← no human can respond in this; it must be fully automated

USING THE BUDGET:

5 of 43 minutes used → ship freely, take risks

40 of 43 minutes used → freeze, fix reliability

100% IS NOT A TARGET. It is unachievable, and aiming for it means never shipping. The budget turns an argument between product and operations into arithmetic.

Cardinality, which multiplies:

```
http_requests_total{endpoint="/checkout", status="200",  
                    region="eu-west", instance="i-4a2f"}
```

50 endpoints
x 5 statuses
x 10 regions
x 500 instances
= 1,250,000 series → fine, ~9 GB/day

NOW ADD user_id:
x 10,000,000 users
= 12,500,000,000,000 series

→ your monitoring falls over before your system does

NEVER a label: user_id, request_id, email, full URL,
error message, timestamp

Those belong in LOGS and TRACES, which are built for
high cardinality. Metrics are not.

5 How it actually works

The four metric types

Getting these right is most of the practical skill, and the aggregation rules follow from them.

COUNTER	only ever goes up (or resets to zero on restart) requests_total, errors_total, bytes_sent_total → you never read a counter; you read its RATE → rate(requests_total[5m]) → ALWAYS use a counter for "how many things happened", because the reset is detectable and a gauge's missed increment is not
GAUGE	goes up and down queue_depth, memory_bytes, active_connections → read directly, and average or max across machines
HISTOGRAM	counts per bucket, plus a sum and a count request_duration_seconds_bucket{le="0.1"} → the ONLY correct way to get percentiles across machines → cost: one series PER BUCKET, so ~10-15x a gauge
SUMMARY	percentiles computed on each machine → CANNOT be aggregated. Use only when there is exactly one instance, which there almost never is

The counter-versus-gauge choice is where beginners go wrong, and the rule is simple: **if you are counting occurrences, it is a counter, always**. A gauge that you increment loses every increment that happened while a scrape was missed; **a counter's true value is recoverable from any two samples**.

Pull against push

PULL (Prometheus)	the monitoring system scrapes each instance every 15 seconds + the scrape itself is a health check + no client-side buffering or backpressure to design + service discovery tells it what exists - short-lived jobs die before being scraped → a "push gateway" for those, which is an acknowledged wart
PUSH (StatsD, OpenTelemetry, most hosted services)	each instance sends its metrics onward + short-lived jobs and serverless work naturally + works through firewalls and across networks - a burst of clients can overwhelm the collector - "instance stopped sending" is ambiguous: dead, or just idle?

Prometheus's pull model plus Grafana for display is the default open-source stack, and naming it is worth doing — **but the pull-versus-push trade is the part that shows understanding**.

What a good alert looks like

```
- alert: HighErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[5m]))
    / sum(rate(http_requests_total[5m])) > 0.01
  for: 5m
  labels:
    severity: page
  annotations:
    summary: "Error rate {{ $value | humanizePercentage }} (threshold 1%)"
    runbook: "https://wiki/runbooks/high-error-rate"
```

Four things make this a good alert rather than a bad one.

for: 5m — the condition must hold for five minutes. **Without it, a one-second blip at 02:14 pages someone**, and that is Karim's warm motor.

A ratio, not a count. `errors > 100` fires during a traffic spike when the *rate* is unchanged. **The fraction is what users experience.**

The current value in the summary, so the person woken knows whether it is 1.1% or 90% **before opening anything.**

A runbook link. An alert without one is an alert that assumes whoever is woken already knows what to do — and at three in the morning, six months after you wrote it, that person is not you.

The runbook

ALERT: HighErrorRate

WHAT IT MEANS	more than 1% of requests returned 5xx for 5 minutes
WHO IT AFFECTS	all users of the checkout flow
FIRST CHECK	the errors-by-endpoint dashboard — is it one endpoint or all of them?
COMMON CAUSES	1. a deploy in the last 30 minutes → roll back 2. the payments provider is down → enable the degraded mode flag 3. connection pool exhausted → check pool metrics, scale out
ESCALATE TO DO NOT	the payments on-call, if it is provider-side restart the whole fleet; it will make it worse

The **DO NOT** line is the one people forget and the one that saves incidents.

Sampling traces

A trace per request is unaffordable at scale, so the question is which ones to keep.

HEAD SAMPLING	decide at the start: keep 1% at random + cheap, simple, no buffering - you keep 1% of the errors too, which are exactly the ones you wanted
TAIL SAMPLING	buffer the whole trace, decide at the end → keep 100% of errors, 100% of anything slower than a second, 1% of the rest + you keep what matters - must buffer every trace until it completes, which is real memory and real complexity

Tail sampling is what you want and head sampling is what most people run, and saying that honestly is better than pretending otherwise.

Structured logs

```
# bad: unparseable, and the interesting fields are trapped in prose
log.info(f"User {user_id} checkout failed after {ms}ms: {error}")

# good: queryable, and every field can be filtered on
log.info("checkout_failed", extra={
    "user_id": user_id,
    "duration_ms": ms,
    "error_code": error.code,
    "trace_id": trace_id,          # ties this line to the trace
})
```

trace_id on every log line is the join key between the three pillars — it is what turns “metrics, logs and traces” from three separate tools into one workflow.

The real systems

Prometheus	metrics, pull-based; the open-source default
Grafana	dashboards over almost any metrics store
OpenTelemetry	the vendor-neutral standard for all three pillars
Jaeger / Tempo	distributed tracing
Loki / Elasticsearch	log aggregation and search
PagerDuty / Opsgenie	routing pages to whoever is on call
Datadog / New Relic	all of the above, hosted, expensive at scale

6 The numbers

Metrics storage, and where the cost comes from.

1,250,000 series, scraped every 15 seconds:

4 samples/minute x 60 x 24 = 5,760 samples/day per series
1,250,000 x 5,760 = 7,200,000,000 samples/day

Prometheus compresses to ~1.3 bytes/sample
→ $7.2e9 \times 1.3 = \sim 9.4$ GB/day
→ ~280 GB/month

→ AND THE DRIVER IS CARDINALITY, NOT SAMPLE RATE.
Halving the scrape interval doubles the cost.
Adding one label with 100 values multiplies it by 100.

The cardinality explosion, quantified.

base:	1,250,000 series	~9 GB/day
+ user_id (10,000,000):	12,500,000,000,000	~94 PB/day

→ a factor of TEN MILLION, from one label

This is the most common real-world monitoring outage, and the fix is a rule rather than capacity: no unbounded value ever becomes a label.

The three pillars, priced against each other.

100,000,000 requests/day

METRICS aggregated, independent of request count
 ~9 GB/day

LOGS 1 KB per request
 100,000,000 x 1 KB = ~100 GB/day
 compressed ~10:1 = ~10 GB/day stored
 retained 30 days = ~300 GB

TRACES 20 spans x 500 bytes = 10 KB per request
 100,000,000 x 10 KB = 1 TB/day at 100%
 at 1% sampling = 10 GB/day

→ metrics are ~10x cheaper than logs and ~100x cheaper than unsampled traces, PER REQUEST

→ which is exactly why the workflow is metrics first: you detect with the cheap thing and only reach for the expensive ones once you know where to look

Error budgets.

per 30-day month = 43,200 minutes

99%	432 minutes	7.2 hours
99.9%	43 minutes	
99.95%	22 minutes	
99.99%	4.3 minutes	
99.999%	26 seconds	

- at 99.99% a human cannot respond in time. Detection, failover and rollback must ALL be automatic.
- at 99.999% you are not designing an on-call rotation, you are designing a system with no manual steps at all.

Each extra nine costs roughly 10x more to achieve. "What availability do you need?" is therefore a budget question, and asking it is worth doing before promising anything.

The tail, compounded — the argument for caring about p99.

one page load = N backend calls, each independently
p99-slow with probability 0.01

N = 5	→	$1 - 0.99^5$	=	4.9%	of page loads affected
N = 20	→	$1 - 0.99^{20}$	=	18.2%	
N = 50	→	$1 - 0.99^{50}$	=	39.5%	
N = 100	→	$1 - 0.99^{100}$	=	63.4%	

- "only 1% of requests are slow" becomes "most page loads feel slow" at a hundred calls
- AND IT IS THE STRONGEST ARGUMENT FOR FAN-OUT DESIGNS BEING BOUNDED: every extra service in the path multiplies your exposure to everyone else's tail

Alert volume, which is a number worth tracking.

a healthy on-call rotation:
0-2 pages per week
> 50% of pages correspond to a real problem

Karim's plant:
~70 rings/week, 1 worth acting on
→ a 1.4% signal rate
→ the correct human response is to stop listening, and he did

IF YOUR ALERT-TO-INCIDENT RATIO IS BELOW 50%, THE PROBLEM IS THE ALERTING, NOT THE SYSTEM.

7 The trade-offs

More metrics against fewer. Collecting everything means you have the data when something odd happens; **it also means cardinality growth, cost, and dashboards nobody can read.** The practical resolution is **collect broadly, alert narrowly** — the expensive mistake is alerting broadly, not collecting broadly.

Sensitive alerts against specific ones. A low threshold catches problems early **and fires on noise**; a high one fires only on real problems **and lets them run longer first.** `for: 5m` is the cheapest way to buy specificity without losing sensitivity — **it costs you five minutes of detection time and removes almost every blip.**

Symptom alerts against cause alerts. Symptoms catch problems you did not predict **and tell you nothing about why.** Causes point straight at the fix **and only cover the failures you thought of in advance.** Alert on symptoms; **put causes on the dashboard the runbook sends you to** — that combination gets both.

Percentiles against averages. Percentiles describe real users; **averages are cheap and aggregate trivially.** Percentiles need histograms, which cost **one series per bucket** — ten to fifteen times a plain gauge. **Worth it, and it is a real cost that belongs in the cardinality budget.**

Sampling rate for traces. One percent is affordable and **loses the rare failure you most wanted to see.** Tail sampling — keep everything that errored or ran slow, sample the rest — is the right answer **and requires buffering every trace until it completes**, which is genuine complexity most teams do not take on.

Pull against push. Pull makes the scrape a health check and needs service discovery; **push handles short-lived and serverless work and makes “stopped sending” ambiguous** — dead, or idle? **Most large systems end up running both**, and saying so is more honest than defending one.

Build against buy. Prometheus and Grafana are free and **cost you an engineer’s ongoing attention.** Hosted tools are excellent and **priced per host or per gigabyte, which becomes a genuinely large bill at scale** — large enough that companies build their own to escape it, which is how the cycle repeats.

And the one people forget: monitoring must survive the outage. If your metrics run on the same infrastructure as your product, **the incident takes both down and you are debugging blind.** Monitoring lives in a separate failure domain, **and the alerting path in particular should have as few shared dependencies as you can manage.**

8 In the interview

How it gets asked

- *“What metrics would you put on the dashboard for this system?”* — the standard closer.
- *“What would page you at three in the morning?”*
- *“How would you know this was broken?”*
- *“Why percentiles rather than averages?”*
- *“What is an SLO, and how is it different from an SLA?”*
- *“What is the most useless alert you have ever had?”* — a real question, and a good one.

The first ninety seconds

“I would answer this in two halves, because **what you measure and what you get woken for are different questions**, and conflating them is where monitoring goes wrong.

What I measure: the four golden signals. Latency — split into successful and failed, because a fast error is not good news and it drags the average down. **Traffic. Errors**, as a fraction. And **saturation** — how close the tightest resource is to its limit, **which is the only leading indicator of the four**.

And I would measure latency as percentiles, never averages. If nine hundred and fifty requests take ten milliseconds and fifty take two seconds, **the average is a hundred and nine milliseconds and no request took anything like that.** The dashboard looks healthy and fifty people waited two seconds. **p50, p95, p99** — and the p99 is the one I would look at.

What pages me: symptoms, not causes. A machine at ninety percent memory is a dashboard entry. **Users getting errors is a page**, even if every machine looks perfectly healthy.

The reason is that you cannot enumerate the causes in advance — there is always a new one — **but you can enumerate the symptoms, and there are about five of them.**

So: error rate above one percent for five minutes. p99 above two seconds for five minutes. Queue depth growing steadily for ten. No successful writes for five. Three to five paging alerts for a service, and if there are twenty, the on-call has already stopped reading them.

The one cause I would page on is saturation — ‘the disk fills in four hours’ — **because by the time that becomes a symptom it is unrecoverable.**

And every alert needs a runbook link. At three in the morning, six months later, **the person woken is not me and does not have the context I had when I wrote it.**“

The follow-ups

“Why percentiles rather than averages?”

“Because the average describes nobody, and I can show that in one example.

A thousand requests: nine hundred and fifty take ten milliseconds, fifty take two seconds. The average is **a hundred and nine and a half milliseconds** — and **not one request took anything close to that.** It is a number that describes an experience nobody had.

The median is ten milliseconds — ‘typical requests are fast’. **The p99 is two seconds** — ‘one in a hundred waits two seconds’. Both are true, both matter, and the average conceals both of them.

The reason to care about the tail more than ‘one percent’ suggests is that a page load makes many requests. Twenty backend calls, each independently p99-slow one percent of the time: the chance all twenty are fast is 0.99 to the twentieth, which is 82%. So eighteen percent of page loads contain a p99-slow call. At a hundred calls it is sixty-three percent.

‘Only one percent of requests are slow’ can mean ‘most page loads feel slow’ — and that arithmetic is the strongest argument I know for bounding fan-out.

Now the part people get wrong, which is that you cannot average percentiles either.

Machine A serves a million requests with a p99 of a hundred milliseconds. Machine B serves a hundred requests with a p99 of ten seconds. Averaging gives **five thousand and fifty milliseconds.** The true combined p99 is about a **hundred milliseconds**, because the slowest one percent of a million and a hundred requests is essentially all A’s. **The naive answer is fifty times wrong.**

The fix is histograms. Each machine exports **bucket counts**; you **sum the buckets across machines and compute the percentile from the sum.** That is what `histogram_quantile` does.

Which is also why the metric type matters: **counters and histograms aggregate correctly across machines and pre-computed percentiles — summaries — do not.** So I would use a histogram even though it costs one series per bucket, and I would count those buckets in the cardinality budget.”

“What would page you at three in the morning, and what would not?”

“Three tests, and an alert has to pass all three: actionable, urgent, needs a human.

Actionable — is there something a person can do right now? If not, it is a dashboard entry. **Urgent** — must it be done now, or can it wait until morning? If it can wait, it is a ticket. **Needs a human** — if it can be automated, automate it and do not page anybody.

Anything failing one of those three degrades every other alert, because the person on call learns that the alert does not mean anything. **And they are learning correctly** — that is the part worth stressing. It is not carelessness; it is an accurate response to a bad signal.

So: error rate above one percent for five minutes pages. p99 above two seconds for five minutes pages. Queue depth growing steadily for ten minutes pages — that one is a leading indicator of a stall.

A single node unhealthy out of fifty does not page. The system is meant to survive that; **if it does not, the fix is the system, not the alert. Ninety percent memory does not page. Cache hit rate down five percent does not page.**

The `for: 5m` on every one of those is not a detail. Without it, a one-second blip wakes someone, and blips are constant. **It costs five minutes of detection and removes almost all of the noise.**

And every alert must be a ratio, not a count. `errors > 100` fires during a traffic spike when nothing has actually got worse. **The fraction is what users experience.**

The number I would actually track is the alert-to-incident ratio. If fewer than half of pages correspond to a real problem, the problem is the alerting, not the system — and the fix is always subtraction. Nobody has ever improved an alerting system by adding alerts.”

“What is an SLO, and what is an error budget?”

“An SLI is the measurement, an SLO is the target, an SLA is the contract with money attached.

SLI: ‘99.95% of requests succeeded in the last thirty days.’ That is a number you measured.

SLO: ‘99.9% of requests succeed.’ That is what you hold yourselves to internally.

SLA: ‘99.5%, or we refund ten percent.’ Always looser than the SLO, deliberately — you want to breach your internal target and start fixing things long before you owe anyone money.

The error budget is what makes an SLO useful rather than decorative.

An SLO of 99.9% is a budget of 0.1% failure. Over a thirty-day month that is forty-three minutes.

And it is a budget you are meant to spend. Five minutes used: ship freely, take risks, do the risky migration. Forty minutes used: freeze features and fix reliability.

The point is that a hundred percent is not a target. It is not achievable, and aiming for it means never shipping anything — which is its own kind of failure. The budget turns what is otherwise an argument between product and operations into arithmetic.

And the numbers are worth knowing per month: 99% is seven hours, 99.9% is forty-three minutes, 99.99% is four minutes, 99.999% is twenty-six seconds.

That last one is the interesting one, because it changes the design rather than the target. At twenty-six seconds a month, no human can be woken, read a runbook and act. Detection, failover and rollback must all be automatic — you are not designing an on-call rotation, you are designing a system with no manual steps in the recovery path.

Each extra nine costs roughly ten times more. So ‘what availability do you actually need?’ is a question worth asking before promising anything — and ‘as high as possible’ is not an answer.”

The model answer

“You have just designed a URL shortener. What would you monitor, and what would page you?”

“Two questions, and I would keep them apart: what I measure, and what wakes someone.

The four golden signals first, applied to this system specifically.

Traffic, split by the two paths — **redirects and creations** — because they have completely different volumes and completely different failure consequences. A **redirect failing is a broken link somebody published; a creation failing is a user who retries.**

Latency as percentiles: p50, p95, p99, and separately for successes and failures, because a fast 404 would otherwise flatter the numbers.

Errors as a fraction: 5xx rate on redirects, 5xx rate on creations. And separately, 404 rate — which is not an error exactly, **but a sudden spike in 404s means either a broken deploy or somebody enumerating our key space**, and both are worth seeing.

Saturation: the database connection pool, and **how much of the key space is used**, since running out of short codes is a slow-moving disaster with a long lead time.

And two that are specific to this system rather than generic. Cache hit rate, because redirects are read-heavy and cheap only while the cache is working — **a hit rate falling from 95% to 60% is a ten-times increase in database load that has not become a symptom yet.** And **redirect latency at the edge**, since the whole product is ‘this is fast’.

Now what pages, and it is a much shorter list.

Redirect error rate above one percent for five minutes. That is users hitting broken links, and it is the core promise of the product.

Redirect p99 above five hundred milliseconds for five minutes.

No successful creations in five minutes — a total write outage, which the error-rate alert would miss if traffic dropped to zero.

And the one cause I would page on: key space above ninety percent used. That is saturation, and by the time it becomes a symptom every creation fails and there is no quick fix. Four hours of warning is worth a page.

Four paging alerts. Everything else goes on a dashboard.

Cache hit rate does not page — it is a cause, it is on the dashboard, **and the runbook for the error-rate alert points at it as the first thing to check. One node unhealthy does not page. Elevated 404s do not page; they raise a ticket.**

Every one of those four has for: 5m on it, because a one-second blip is not an incident, **and every one has a runbook link** — including a **DO NOT** line, since the instinct during a redirect outage is to restart the fleet, which empties the cache and makes it much worse.

Two things I would add unprompted.

The cardinality rule. The short code must never be a metric label. It is unbounded — millions of values — **and it would take the monitoring system down before the product.** Short codes go in logs and traces, which are built for that.

And the monitoring must not share a failure domain with the product. If they run on the same infrastructure, the outage takes both and we are debugging blind — which is the failure that turns a twenty-minute incident into a three-hour one.”

9 Recall card

Three pillars, and it is a sequence rather than a choice: METRICS to detect (cheap, aggregated), TRACES to localise (sampled ~1%), LOGS to diagnose (expensive, full detail). `trace_id` on every log line is the join key. Roughly 9 GB/day of metrics against 100 GB/day of logs and 1 TB/day of unsampled traces at 100M requests.

Four golden signals: LATENCY (split successful from failed — a fast error drags the average down), TRAFFIC, ERRORS (as a fraction, never a count), SATURATION (the only leading indicator). RED for services, USE for resources.

The average describes nobody: 950 requests at 10 ms and 50 at 2,000 ms averages **109.5 ms**, which no request took. p50 = 10 ms, p99 = 2,000 ms. **And the tail compounds — 20 backend calls means 18% of page loads hit a p99 call; 100 calls means 63%. You cannot average percentiles either** (p99s of 100 ms and 10,000 ms average to 5,050 ms when the truth is ~100 ms — 50× wrong); **sum HISTOGRAM BUCKETS instead.** Counters and histograms aggregate; summaries do not.

Alert on SYMPTOMS, not causes — you cannot enumerate causes in advance but there are about five symptoms. **Every page must be actionable, urgent, and need a human**; anything else is a dashboard entry or a ticket. **Three to five paging alerts per service**, each with `for: 5m`, a ratio not a count, the current value, and a **run-book link with a DO NOT line**. **Saturation is the one cause worth paging on**. **The fix is always subtraction** — and if under half of pages are real problems, the alerting is the problem.

SLI is the measurement, SLO the internal target, SLA the contract (always looser). Error budget: $99.9\% = 43$ minutes a month; $99.99\% = 4.3$; $99.999\% = 26$ seconds, at which no human can respond and recovery must be fully automatic. Each nine costs $\sim 10\times$ more. **And CARDINALITY is what actually kills metrics systems** — one `user_id` label turns 1.25M series into 12.5 trillion — **so no unbounded value is ever a label**. **Monitoring lives in a separate failure domain**, or the outage takes it with you.

DSA topic: Binary, bits, and why they matter **System design topic:** Monitoring, metrics, and alerting

Code these, in this order

One rule for the whole set: **write out the bit pattern before you write the expression**. Eight characters of bit manipulation is three ideas stacked on top of each other, and reading it back afterwards will not tell you whether it is right. **Say the pattern, then write the code that produces it.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of 1 Bits	LeetCode 191 (Easy)	The naive loop, then <code>n & (n-1)</code> .
2	Power of Two	LeetCode 231 (Easy)	The one-liner, and the guard everyone omits.
3	Counting Bits	LeetCode 338 (Easy)	Bits meeting DP — the recurrence is one line.
4	Reverse Bits	LeetCode 190 (Easy)	Fixed width, and why Python needs to be told it.
5	Sum of Two Integers	LeetCode 371 (Medium)	Addition from AND, XOR and shift — and Python's traps.
6	Bitwise AND of Numbers Range	LeetCode 201 (Medium)	The common prefix, and why the answer is a shift.

On problem 1, count the iterations

Instrument both versions. Run them on `1024`, `255` and `1 << 31` and **record the iteration counts in a small table**. Say which input makes them equal and why.

On problem 2, remove the guard

Delete `number > 0` and run on `0` and `-8`. **Record both answers**. Say what `0 & -1` is and why that is the whole bug.

On problem 3, find the one-line recurrence

`bits[i]` can be built from a smaller answer. **Find two different ways** — one using `i >> 1`, one using `i & (i - 1)` — and say what each one is really saying about the number.

On problem 4, discover why Python is awkward

Write the obvious loop. **Run it and see what happens with leading zeros.** Then say what “32-bit” means here and write the version that respects it. **Record what `bin(x)[2:]` gives you for a small number** and why that is the problem.

On problem 5, meet the arbitrary-width trap

Implement addition with XOR for the sum and AND-shifted for the carry. **Run it on two positives** — it works. **Then run it on a negative** and say what happens. Then fix it with a mask, and **say out loud what the mask is standing in for.**

On problem 6, do it by hand first

Take `[5, 7]` and write out `101`, `110`, `111`. **Say what survives the AND and why.** Then say what that has to do with the common prefix, and only then write the shifting solution.

Then the conversion drill

Convert `37`, `100` and `255` to binary by hand, saying “upwards” out loud each time. **Then convert them back using double-and-add.** Check against `bin()`. Then deliberately read one set of remainders downwards and record the wrong number you get.

Then the precedence drill

Write `mask = 1 << n - 1` for `n = 4` and record its value against `(1 << n) - 1`. **Say which one you meant.** Then find every unbracketed shift in code you have written today.

Then the negative-shift drill

Run `while n: n >= 1` on `-5`. **Do not let it run long.** Say what `-1 >> 5` is, why the loop never ends, and why hanging is worse than crashing.

The place-value drill

1. Give the binary places up to 64.
2. Convert 13 both ways and state the direction rule.
3. Say what `n` bits gives you, in values and in range.
4. Give `2^10`, `2^20`, `2^30` and their rough decimal names.
5. Say why the machine uses two and not ten.

The operators drill

1. Give all six with a one-sentence meaning each.
2. Say what XOR is the same as.
3. Say what shifting left does and why, from place value.
4. Say why right shift rounds down, and what the rounding is.
5. Give `1 << k` in words.

The edits drill

1. Give all four one-liners from memory.
2. Say what they have in common.
3. Say which operator forces on, which forces off, which flips.
4. Say what `~(1 << p)` is in Python and why it still works.

The tricks drill

1. Say what `n & (n - 1)` does and why, in terms of borrowing.
2. Give the two things that fall out of it.
3. Give the iteration counts for `1024` and `255`, both methods.
4. Say what `n & -n` gives.
5. Say why hash tables use power-of-two sizes, with the cycle counts.
6. Say what that costs, and what Java does about it.

The negatives drill

1. Give the two's complement rule.
2. Work `-5` in eight bits and check it sums to zero.
3. Say why this scheme was chosen over a sign bit.
4. Give the range in eight bits and say why it is asymmetric.
5. Say what `~x` is in Python and what to do about it.
6. Say what `-1 >> 5` is and what loop that breaks.

The break-it drill

For each, say what happens and whether anything reports it:

1. Reading the remainders downwards.
2. `1 << n - 1` when you meant a mask.
3. `int('1101')` without the base.
4. `bin(-5)[2:]`.

5. `is_power_of_two(0)` without the guard.
 6. `while n: n >= 1` on a negative.
 7. `1 << 2.0` after a `/` division.
-

The pillars drill

1. Name all three, what each answers, and what each cannot.
2. Give the workflow as a sequence.
3. Give the relative cost of each at 100M requests/day.
4. Say what field ties them together.

The signals drill

1. Name the four golden signals.
2. Say what must be split, and why.
3. Say which is the leading indicator.
4. Give RED and USE and say how they relate.

The percentiles drill

1. Give the 950/50 example and compute the average.
2. Say what is wrong with that number in one sentence.
3. Give p50 and p99 for it.
4. Compute the tail compounding at 20 and 100 calls.
5. Give the two-machine example and say why averaging p99s fails.
6. Give the fix and name the metric type that supports it.
7. Say which metric types aggregate and which do not.

The alerting drill

1. Give the three tests every page must pass.
2. Give four things that page and four that do not.
3. Say what `for: 5m` buys and costs.
4. Say why an alert must be a ratio.
5. Say what belongs in a runbook, including the line people forget.
6. Give the alert-to-incident ratio rule.
7. Say what direction the fix always goes in.

The budget drill

1. Define SLI, SLO and SLA and say which is loosest.
2. Give downtime per month at four availability levels.
3. Say what the budget is for and how you spend it.
4. Say what changes in the design at five nines.
5. Say why 100% is not a target.

The cardinality drill

1. Compute the base series count from the four labels.
 2. Compute it again with `user_id`.
 3. Give the storage in GB/day for the base case.
 4. Name five things that must never be a label.
 5. Say where those belong instead.
 6. Say what else must not share a failure domain, and why.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is 13 in binary, and what does shifting left by one do?* Place value, both conversion rules with the direction said aloud, why shifting doubles, and the Python width caveat.
 2. *Count the set bits. Now do it faster.* The naive loop with its iteration count, `n & (n-1)` with the borrowing explanation, the worst case where they tie, and the power-of-two test that falls out.
 3. *What would you put on the dashboard, and what would page you?* The four golden signals, percentiles over averages with the arithmetic, symptoms over causes, the three tests for a page, and the cardinality rule.
-

Before you move on

- [] I can convert to binary by hand and I say “upwards” out loud.
- [] I know what reading the remainders downwards gives, and that nothing errors.
- [] I can convert back with double-and-add without powers.
- [] I know the powers of two to 2^{10} , and 2^{20} and 2^{30} by their rough names.

- [] I can give all six operators with one sentence each.
- [] I know XOR is “exactly one”, which is “different”.
- [] I can explain why shifting left doubles, from place value.
- [] I know right shift rounds down and what the rounding is.
- [] I can write all four bit edits from memory.
- [] I know what they have in common.
- [] I can explain `n & (n - 1)` in terms of borrowing.
- [] I know the iteration counts for `1024` and `255`, both methods.
- [] I know the power-of-two test and why the guard is not decoration.
- [] I know why hash tables use power-of-two sizes, with the cycle counts.
- [] I can do two’s complement by hand and check it sums to zero.
- [] I know why that scheme beat a sign bit.
- [] I know `~x == -x - 1` in Python and how to get a real pattern.
- [] I know `-1 >> 5` is `-1` and which loop that hangs.
- [] I always bracket shifts.
- [] I know `int(s, 2)` needs the base.
- [] I can name the three pillars and what each answers.
- [] I know the workflow is detect, localise, diagnose.
- [] I can name the four golden signals and which is leading.
- [] I can give the 950/50 example and say why the average describes nobody.
- [] I can compute the tail compounding at 20 and 100 calls.
- [] I know why percentiles cannot be averaged, and the fix.
- [] I know which metric types aggregate and which do not.
- [] I can give the three tests every page must pass.
- [] I can name four things that page and four that do not.
- [] I know what `for: 5m` buys and why alerts must be ratios.
- [] I know what belongs in a runbook, including the `DO NOT` line.
- [] I can define SLI, SLO and SLA and give the monthly budgets.
- [] I know what changes in the design at five nines.
- [] I can compute the cardinality explosion and name what is never a label.
- [] I know monitoring must not share a failure domain with the product.
- [] I answered all three questions above out loud.

The bit tricks every interview uses

DATA STRUCTURES AND ALGORITHMS

The bit tricks every interview uses

SYSTEM DESIGN

Logging and distributed tracing

1 What this is, and why they ask it

There are about ten bit expressions that cover almost every bit question ever asked. $n \& (n - 1)$, $n \& -n$, $1 \ll k$, $(1 \ll k) - 1$, $a \wedge b$. That is most of it.

Each one is short, each one has a one-line reason, and the reason is the part that matters. Anybody can memorise $n \& (n - 1)$. Only the person who can say why it works can use it on a problem they have not seen.

Yesterday you learnt what the operators do. Today is the small kit of moves built out of them, and the habit of naming the reason out loud.

They ask it because these questions are short, unbluffable, and used as filters. “Count the set bits” takes four minutes to ask and answer. In a phone screen that is exactly what an interviewer wants: a question where you either have the machinery or you do not, and no amount of talking around it helps.

And because bit tricks appear inside bigger answers. Enumerating subsets in a backtracking problem. The mask in bitmask DP. Hashing into a power-of-two table. The trick is rarely the question — it is a line inside the question, and if you have to stop and derive it, you lose the thread of the real problem.

By the end of this lesson you can write all ten from memory, say the one-line reason for each, use them to solve five named interview problems, and enumerate the subsets of a set and the submasks of a mask — which is where the tricks stop being tricks and start being a technique.

2 The story

The shop had three chairs and two of them had been broken for years, and Mahadevan cut hair in the middle one.

Shivu had swept that floor since he was eleven. What he watched, forty times a day, was the same twenty minutes: the sheet snapped open, the comb, the scissors going like an insect, and the man in the chair standing up looking like a slightly different man.

What Shivu could not work out was how few things Mahadevan actually did.

He counted them once, on a slow Monday afternoon. The comb lifted a section of hair and held it out flat. The scissors went across the top of the comb. The comb came down half its own width and did the same thing again. **That was it. That was most of a haircut.**

There were four or five other moves for the edges — the thing he did at the neck with the back of the blade, the two fingers he put against the ear.

Nine or ten moves in the whole trade, and Shivu had watched every one of them a thousand times.

So one evening he asked if he could try, on a boy from the next lane who was not paying anyway.

It went badly, and it went badly in a way that surprised him. **He knew all the moves. He could do each one on its own, and they looked right.** What he could not do was look at a head and know which move it wanted.

Mahadevan watched from the doorway and did not laugh, which Shivu was grateful for. Then he said a thing that took about two years to land properly.

“You have learnt what my hands do. You have not learnt why they do it.”

He picked up the comb.

“This one is for taking length off the top and keeping it even. And it works because the comb decides how much hair sticks out, not me. My hand only has to stay straight.”

He put it down again.

“Learn why a move works and you know what it is for. After that the head tells you which one.”

3 The idea in plain English

Every trick below is one expression and one sentence. The expression is what your hands do. The sentence is why. **Learn them in pairs, and never one without the other.**

The two core moves

Everything in this lesson is built on two facts about what happens near the lowest **1** in a number.

`n - 1` flips the lowest set bit to zero and turns everything below it into ones.

That is just borrowing, the same borrowing you do in ordinary subtraction.

```
n      = 10110000
n - 1  = 10101111
      ^^^^ the lowest 1 became 0, and everything
            under it became 1
```

So `n & (n - 1)` clears the lowest set bit. Above the lowest one, the two numbers are identical, so AND keeps them. At the lowest one, one number has `1` and the other has `0`, so it goes. Below it, one number has all zeros, so those stay zero.

And `n & -n` keeps only the lowest set bit. `-n` in two's complement is "flip everything and add one", which is exactly `~n + 1` — and adding one to a flipped number ripples up to the first place that was a `1`. **Everything above the lowest set bit ends up flipped, everything below stays zero, and the lowest set bit itself survives in both.**

```
n      = 10110000
-n     = 01010000
n & -n = 00010000    only the lowest 1 is left
```

Those two are opposites and you should say them as a pair. One removes the lowest one. One keeps only the lowest one. **Almost every counting or set-walking trick is one of these in a loop.**

The masks

A mask is a number you AND or OR against, chosen so that it lets through exactly the bits you care about.

<code>1 << k</code>	a single 1, in place k	<code>1 << 4 = 00010000</code>
<code>(1 << k) - 1</code>	k ones at the bottom	<code>(1<<4)-1 = 00001111</code>
<code>~(1 << k)</code>	all ones EXCEPT place k	a hole

`(1 << k) - 1` is the workhorse. It is "the bottom `k` bits", and you use it to cut a number down to a fixed width, to take a remainder, and to say "just this part".

Bracket it. Always. `1 << k - 1` means `1 << (k - 1)` — a single bit rather than a run of ones — because subtraction binds tighter than shifting. **Nothing errors. You get a plausible wrong number.**

The one-line tests

$n > 0 \text{ and } n \& (n - 1) = 0$	n is a POWER OF TWO exactly one bit set, so clearing it leaves 0
$n \geq 0 \text{ and } n \& (n + 1) = 0$	n is ALL ONES: 0, 1, 3, 7, 15, 31 ... adding one carries all the way past the top
$n \& 1$	the last bit: 1 if odd, 0 if even

The `n > 0` guard on the power-of-two test is not decoration. `0 & -1` is `0`, so without it zero passes. It is the single most common one-character bug in this whole topic.

Counting, and the two ways to say it

Set bits are counted by removing them one at a time.

```
count = 0
while n:
    n &= n - 1    removes exactly one set bit
    count += 1
```

One iteration per *set* bit, not per bit position. This is **Brian Kernighan's algorithm** and it is worth knowing by name, because interviewers recognise it.

And there is a second way, which matters when you need the answer for every number up to some limit.

```
bits[i] = bits[i & (i - 1)] + 1
```

Read it as a sentence: the number of set bits in `i` is one more than the number in `i` with its lowest set bit removed. And `i & (i - 1)` is always smaller than `i`, so the smaller answer is already there. That is a one-line dynamic programme, and it is LeetCode 338 in its entirety.

XOR, and what it is really for

XOR is “exactly one”, which is the same as “different”. So `a ^ b` has a `1` in every place where `a` and `b` disagree.

Three consequences, and each one is a named interview problem.

Counting the differences is counting the set bits of the XOR. That is **Hamming distance**, LeetCode 461, one line.

$a \oplus a$ is 0 and $a \oplus 0$ is a . So XORing a whole list together **cancels everything that appears twice** and leaves the one that does not. That is the single-number family, and [tomorrow](#) is entirely about it.

Swapping without a temporary — $a \oplus= b; b \oplus= a; a \oplus= b$ — is a party trick, and you should know it and also know that **it silently zeroes the value if the two are the same variable**. Say both halves.

Walking a set, and the subset trick

Here is where the tricks become a technique. A subset of n items is an n -bit number: **bit i set means item i is in**. You met this in [bitmask DP](#); this is the mechanics under it.

```
for mask in range(1 << n):          every subset of n items
    for i in range(n):
        if mask >> i & 1:          is item i in this subset?
            ...
```

$1 \ll n$ is 2^n , so the outer loop is every possible subset, and $\text{mask} \gg i \ \& \ 1$ is the get-bit one-liner from yesterday.

If a mask is sparse, walk only its set bits instead of all n positions.

```
while mask:
    low = mask & -mask              isolate the lowest set bit
    position = low.bit_length() - 1
    mask &= mask - 1               clear it and go again
```

Both core moves, in four lines. Isolate to read it, clear to move on.

And the one that looks like magic: every submask of a mask, in descending order.

```
sub = mask
while True:
    ...use sub...
    if sub == 0: break
    sub = (sub - 1) & mask
```

$\text{sub} - 1$ **borrow**s into the bits below, and $\& \text{mask}$ throws away everything that was never part of the mask in the first place — **so you land on the next smaller subset without ever visiting a number that is not one**. It is worth having, because it turns an enumeration that looks like $2^n \times 2^n$ into 3^n .

The small ones worth knowing

<code>n >> 1</code>	halve, rounding down
<code>n << 1</code>	double
<code>n & (size - 1)</code>	<code>n % size</code> , when <code>size</code> is a power of two
<code>ord(c) ^ 32</code>	swap the case of an ASCII letter
<code>x (x + 1)</code>	set the lowest zero bit

The case one is a nice example of the whole lesson's point. `'a'` is 97 and `'A'` is 65, and the difference is 32, **which is one bit** — so the cases of every ASCII letter differ in exactly one place. **Knowing that is a trick. Knowing why is a fact about the character table**, and the fact is what lets you answer when someone asks about digits or punctuation.

4 The picture

The two core moves, side by side:

CLEAR the lowest set bit	ISOLATE the lowest set bit
<code>n</code> = 1 0 1 1 0 0 0 0	<code>n</code> = 1 0 1 1 0 0 0 0
<code>n - 1</code> = 1 0 1 0 1 1 1 1	<code>-n</code> = 0 1 0 1 0 0 0 0
-----	-----
<code>n & (n - 1)</code> = 1 0 1 0 0 0 0 0	<code>n & -n</code> = 0 0 0 1 0 0 0 0
^	^
it is GONE	only IT is left
Same bit. One move takes it away, one move keeps only it. Say them as a pair and you will never mix them up.	

Why `n - 1` behaves like that — the borrowing, drawn out:

```
176  1 0 1 1 0 0 0 0
-   1
```

the last place is 0, so you borrow from the left
... and keep borrowing until you find a 1

```
1 0 1 1 0 0 0 0
    ^ this is the first 1 you meet
    it becomes 0
    and every 0 you passed becomes 1
```

```
175  1 0 1 0 1 1 1 1
```

THAT IS THE WHOLE REASON $n \& (n-1)$ WORKS.
Nothing above the lowest 1 changed, so AND keeps it.
The lowest 1 changed, so AND kills it.
Below it, one side is all 0s, so AND gives 0.

The masks, drawn:

place	7	6	5	4	3	2	1	0	
$1 \ll 4$	0	0	0	1	0	0	0	0	one bit, in place 4
$(1 \ll 4) - 1$	0	0	0	0	1	1	1	1	the bottom four bits
$\sim(1 \ll 4)$	1	1	1	0	1	1	1	1	everything EXCEPT place 4
$1 \ll 4 - 1$									
					0	0	0	0	← the trap
					this is	$1 \ll 3$,	not	$(1 \ll 4) - 1$	

Subsets as numbers — three items, eight subsets:

```
items:  [3, 5, 7]
         ^  ^  ^
bit:     0  1  2

mask 000 → []          0
mask 001 → [3]         1
mask 010 → [5]         2
mask 011 → [3, 5]      3
mask 100 → [7]         4
mask 101 → [3, 7]      5
mask 110 → [5, 7]      6
mask 111 → [3, 5, 7]   7
```

Counting from 0 to $2^n - 1$ IS listing every subset.
No recursion, no extra structure - the number is the subset.

5 The code, built step by step

A way to see what happened

```
def show(number: int, width: int = 8) → str:
    """The bit pattern of `number` in `width` bits, so you can see what happened."""
    return format(number & ((1 << width) - 1), f"0{width}b")
```

Write this first, before any of the tricks. Bit code is unreadable when it goes wrong, and the difference between a five-minute bug and a fifty-minute one is whether you can see the pattern.

`number & ((1 << width) - 1)` is the mask trick doing real work: it cuts an arbitrary-width Python integer down to `width` bits, which is also how you make a negative number show its two's complement pattern.

The two core moves

```
def lowest_set_bit(number: int) → int:
    """n & -n keeps ONLY the lowest set bit. -n is ~n + 1, so it flips everything above."""
    return number & -number

def clear_lowest_set_bit(number: int) → int:
    """n & (n - 1) REMOVES the lowest set bit. Borrowing does the work."""
    return number & (number - 1)
```

Two functions, four words of difference, opposite jobs. In an interview you would inline both — the point of naming them here is that **the name is the sentence you have to be able to say.**

Counting, with the guard that matters

```
def count_set_bits(number: int) → int:
    """Kernighan: one iteration per SET bit. Guarded, because negatives never terminate."""
    if number < 0:
        raise ValueError("count_set_bits needs a non-negative number")
    count = 0
    while number:
        number &= number - 1
        count += 1
    return count
```

The guard is the interesting line and section 7 is about it. On a negative Python integer this loop does not crash — it runs forever, drifting to `-16`, `-32`, `-64` and never reaching zero.

The one-line tests

```
def is_power_of_two(number: int) → bool:
    """Exactly one set bit. The `> 0` guard is not decoration:  $0 \& -1 = 0$ ."""
    return number > 0 and (number & (number - 1)) == 0

def is_all_ones(number: int) → bool:
    """0, 1, 3, 7, 15 ... - one less than a power of two. Adding one carries all the way."""
    return number ≥ 0 and (number & (number + 1)) == 0
```

These two are a matched pair and interviewers like the second one, because almost nobody has it. `n + 1` on a run of ones carries all the way past the top, leaving a single high bit that shares nothing with `n`.

Hamming distance, in one line

```
def hamming_distance(a: int, b: int) → int:
    """XOR marks every place they differ; count those places."""
    return count_set_bits(a ^ b)
```

Say the sentence before you write the line. “XOR is ‘different’, so the set bits of the XOR are exactly the disagreements.” **That sentence is the answer to LeetCode 461**, and the code is a formality.

Counting bits for every number at once

```
def counting_bits(limit: int) → list[int]:
    """bits[i] = bits[i with its lowest set bit removed] + 1. One smaller answer, always."""
    bits = [0] * (limit + 1)
    for i in range(1, limit + 1):
        bits[i] = bits[i & (i - 1)] + 1
    return bits
```

`i & (i - 1)` is strictly smaller than `i` for every positive `i`, so the value you need has already been filled in. **That is the whole of the correctness argument**, and it is worth saying out loud because it is what makes this a dynamic programme rather than a coincidence.

There is a second recurrence — `bits[i] = bits[i >> 1] + (i & 1)` — “the bits of `i` are the bits of `i` without its last one, plus that last one”. **Both are one line. Know both, and say what each is claiming.**

Reversing bits, and why width has to be stated

```
def reverse_bits(number: int, width: int = 32) → int:
    """Take the bottom bit off `number` and push it onto the bottom of the answer."""
    result = 0
    for _ in range(width):
        result = (result << 1) | (number & 1)
        number >>= 1
    return result
```

The loop runs exactly `width` times, not “until the number runs out”. That is the point of this problem: reversing `00000001` in eight bits gives `10000000`, and reversing it in “as many bits as it needs” gives `1`. Python has no width, so you must supply one, and saying that out loud is most of the credit.

Subsets, and walking only what is set

```
def subsets(items: list[int]) → list[list[int]]:
    """Every subset of n items is an n-bit number: bit i means 'item i is in'."""
    n = len(items)
    out = []
    for mask in range(1 << n):
        out.append([items[i] for i in range(n) if mask >> i & 1])
    return out
```

Two loops and no recursion. The outer one counts from `0` to `2n - 1`; the inner one asks the get-bit question of each place. `mask >> i & 1` needs no brackets — `>>` binds tighter than `&` — but write them anyway.

```
def set_bit_positions(mask: int) → list[int]:
    """Walk only the set bits. Each step isolates the lowest one and then clears it."""
    positions = []
    while mask:
        low = mask & -mask
        positions.append(low.bit_length() - 1)
        mask &= mask - 1
    return positions
```

Both core moves in one loop, doing different jobs. Isolate to find out *where* the bit is; clear to move past it. `bit_length() - 1` turns a single-bit number into its place, because a lone bit in place 4 is the number 16, whose bit length is 5.

```
def submasks(mask: int) → list[int]:
    """Every subset of a mask, biggest first. (sub - 1) & mask is the next one down."""
    out = []
    sub = mask
    while True:
        out.append(sub)
        if sub == 0:
            break
        sub = (sub - 1) & mask
    return out
```

The `if sub == 0: break` is inside the loop and after the append, and that is deliberate: zero is a legitimate submask and must be visited, but `(0 - 1) & mask` is `mask` again, so testing at the top gives you an endless circle. **This is the one loop in the lesson whose shape you should memorise rather than re-derive under pressure.**

The complete solution

```
"""Day 172 - the bit tricks, all of them, with the reason each one works."""

from __future__ import annotations


def show(number: int, width: int = 8) → str:
    """The bit pattern of `number` in `width` bits, so you can see what happened."""
    return format(number & ((1 << width) - 1), f"0{width}b")


def lowest_set_bit(number: int) → int:
    """n & -n keeps ONLY the lowest set bit. -n is ~n + 1, so it flips everything above."""
    return number & -number


def clear_lowest_set_bit(number: int) → int:
    """n & (n - 1) REMOVES the lowest set bit. Borrowing does the work."""
    return number & (number - 1)


def count_set_bits(number: int) → int:
    """Kernighan: one iteration per SET bit. Guarded, because negatives never terminate."""
    if number < 0:
        raise ValueError("count_set_bits needs a non-negative number")
    count = 0
    while number:
        number &= number - 1
        count += 1
    return count


def is_power_of_two(number: int) → bool:
    """Exactly one set bit. The `> 0` guard is not decoration: 0 & -1 = 0."""
    return number > 0 and (number & (number - 1)) == 0


def is_all_ones(number: int) → bool:
    """0, 1, 3, 7, 15 ... - one less than a power of two. Adding one carries all the way."""
    return number ≥ 0 and (number & (number + 1)) == 0


def low_mask(width: int) → int:
    """`width` ones at the bottom. Bracket the shift or you get a single bit instead."""
    return (1 << width) - 1


def hamming_distance(a: int, b: int) → int:
    """XOR marks every place they differ; count those places."""
    return count_set_bits(a ^ b)


def counting_bits(limit: int) → list[int]:
    """bits[i] = bits[i with its lowest set bit removed] + 1. One smaller answer, always."""
    bits = [0] * (limit + 1)
    for i in range(1, limit + 1):
        bits[i] = bits[i & (i - 1)] + 1
    return bits


def reverse_bits(number: int, width: int = 32) → int:
    """Take the bottom bit off `number` and push it onto the bottom of the answer."""
    result = 0
    for _ in range(width):
        result = (result << 1) | (number & 1)
        number >>= 1
```

```

return result

def subsets(items: list[int]) → list[list[int]]:
    """Every subset of n items is an n-bit number: bit i means 'item i is in'."""
    n = len(items)
    out = []
    for mask in range(1 << n):
        out.append([items[i] for i in range(n) if mask >> i & 1])
    return out

def set_bit_positions(mask: int) → list[int]:
    """Walk only the set bits. Each step isolates the lowest one and then clears it."""
    positions = []
    while mask:
        low = mask & -mask
        positions.append(low.bit_length() - 1)
        mask &= mask - 1
    return positions

def submasks(mask: int) → list[int]:
    """Every subset of a mask, biggest first. (sub - 1) & mask is the next one down."""
    out = []
    sub = mask
    while True:
        out.append(sub)
        if sub == 0:
            break
        sub = (sub - 1) & mask
    return out

def single_number(numbers: list[int]) → int:
    """Pairs cancel under XOR, so what is left is the one that had no pair."""
    answer = 0
    for value in numbers:
        answer ^= value
    return answer

def swap_case(letter: str) → str:
    """Letters differ by exactly one bit between the cases: the 32s place."""
    return chr(ord(letter) ^ 32)

if __name__ == "__main__":
    print("THE TWO CORE MOVES, on 176 = 10110000")
    n = 0b10110000
    print(f"  n                {show(n)} = {n}")
    print(f"  n - 1            {show(n - 1)} = {n - 1}  (lowest 1 flipped, ones below)")
    print(f"  n & (n - 1)      {show(clear_lowest_set_bit(n))} = {clear_lowest_set_bit(n)}")
    print(f"    f"  CLEARS the lowest 1")
    print(f"  -n              {show(-n)} = {-n}  (flip and add one)")
    print(f"  n & -n          {show(lowest_set_bit(n))} = {lowest_set_bit(n)}")
    print(f"    f"  ISOLATES the lowest 1")

    print()
    print("MASKS")
    for k in (1, 3, 4, 8):
        print(f"  low_mask({k})    {show(low_mask(k), 8)} = {low_mask(k):>3}  ({k} ones)")
    print(f"  1 << 4           {show(1 << 4)} = {1 << 4:>3}  (one bit)")
    print(f"  (1 << 4) - 1     {show((1 << 4) - 1)} = {(1 << 4) - 1:>3}  (four ones)")
    print(f"  1 << 4 - 1       {show(1 << 4 - 1)} = {1 << 4 - 1:>3}  THE PRECEDENCE TRAP")

    print()
    print("COUNTING, and the two one-line tests")

```



```

for value in (0, 1, 7, 8, 176, 255, 1024):
    print(f" {value:>5} {show(value, 11)} bits={count_set_bits(value)}"
          f" power_of_two={is_power_of_two(value)} all_ones={is_all_ones(value)}")

print()
print("HAMMING DISTANCE - count where they differ")
for a, b in ((1, 4), (3, 1), (176, 160)):
    print(f" {show(a)} ^ {show(b)} = {show(a ^ b)} → {hamming_distance(a, b)}")

print()
print("COUNTING BITS FOR 0..15 IN ONE PASS")
print(f" {counting_bits(15)}")

print()
print("REVERSE BITS, in 8 bits")
for value in (1, 0b10110000, 0b11111111):
    print(f" {show(value)} → {show(reverse_bits(value, 8))}")

print()
print("SUBSETS OF [3, 5, 7] - the mask IS the subset")
for mask, subset in enumerate(subsets([3, 5, 7])):
    print(f" mask {mask:03b} → {subset}")

print()
print("WALKING ONLY THE SET BITS of 176")
print(f" positions: {set_bit_positions(176)} (8 bits wide, 3 steps taken)")

print()
print("EVERY SUBMASK OF 1011")
print(f" {[format(s, '04b') for s in submasks(0b1011)]}")

print()
print("PAIRS CANCEL")
print(f" single_number([4, 1, 2, 1, 2]) = {single_number([4, 1, 2, 1, 2])}")

print()
print("THE 32s PLACE IS THE CASE BIT")
for letter in ("a", "z", "k"):
    print(f" {letter} = {ord(letter)>3} = {show(ord(letter))}"
          f" → {swap_case(letter)} = {ord(swap_case(letter))>3}"
          f" = {show(ord(swap_case(letter)))}")

print()
print("VERIFICATION")
bad = 0
for value in range(0, 5000):
    if count_set_bits(value) != value.bit_count():
        bad += 1
    if is_power_of_two(value) != (value > 0 and value.bit_count() == 1):
        bad += 1
    if is_all_ones(value) != (value == (1 << value.bit_length()) - 1):
        bad += 1
    if reverse_bits(reverse_bits(value, 16), 16) != value:
        bad += 1
if counting_bits(4999) != [v.bit_count() for v in range(5000)]:
    bad += 1
if len(subsets([1, 2, 3, 4, 5])) != 32:
    bad += 1
if len(submasks(0b1011)) != 2 ** 3:
    bad += 1
print(f" {bad} mismatches over 5,000 values, 4 checks each, plus 3 whole-list checks")

```

Running it:

THE TWO CORE MOVES, on 176 = 10110000

```
n          10110000 = 176
n - 1      10101111 = 175  (lowest 1 flipped, ones below)
n & (n - 1) 10100000 = 160  CLEARS the lowest 1
-n         01010000 = -176  (flip and add one)
n & -n      00010000 = 16   ISOLATES the lowest 1
```

MASKS

```
low_mask(1) 00000001 = 1   (1 ones)
low_mask(3) 00000111 = 7   (3 ones)
low_mask(4) 00001111 = 15  (4 ones)
low_mask(8) 11111111 = 255 (8 ones)
1 << 4      00010000 = 16  (one bit)
(1 << 4) - 1 00001111 = 15  (four ones)
1 << 4 - 1   00001000 = 8   THE PRECEDENCE TRAP
```

COUNTING, and the two one-line tests

```
0 000000000000 bits=0 power_of_two=False all_ones=True
1 000000000001 bits=1 power_of_two=True  all_ones=True
7 000000000111 bits=3 power_of_two=False all_ones=True
8 000000001000 bits=1 power_of_two=True  all_ones=False
176 000101100000 bits=3 power_of_two=False all_ones=False
255 000111111111 bits=8 power_of_two=False all_ones=True
1024 100000000000 bits=1 power_of_two=True  all_ones=False
```

HAMMING DISTANCE - count where they differ

```
00000001 ^ 00000100 = 00000101 → 2
00000011 ^ 00000001 = 00000010 → 1
10110000 ^ 10100000 = 00010000 → 1
```

COUNTING BITS FOR 0..15 IN ONE PASS

```
[0, 1, 1, 2, 1, 2, 2, 3, 1, 2, 2, 3, 2, 3, 3, 4]
```

REVERSE BITS, in 8 bits

```
00000001 → 10000000
10110000 → 00001101
11111111 → 11111111
```

SUBSETS OF [3, 5, 7] - the mask IS the subset

```
mask 000 → []
mask 001 → [3]
mask 010 → [5]
mask 011 → [3, 5]
mask 100 → [7]
mask 101 → [3, 7]
mask 110 → [5, 7]
mask 111 → [3, 5, 7]
```

WALKING ONLY THE SET BITS of 176

```
positions: [4, 5, 7] (8 bits wide, 3 steps taken)
```

EVERY SUBMASK OF 1011

```
['1011', '1010', '1001', '1000', '0011', '0010', '0001', '0000']
```

PAIRS CANCEL

```
single_number([4, 1, 2, 1, 2]) = 4
```

THE 32s PLACE IS THE CASE BIT

```
a = 97 = 01100001 → A = 65 = 01000001
```

```
Z = 90 = 01011010 → z = 122 = 01111010
```

```
k = 107 = 01101011 → K = 75 = 01001011
```

VERIFICATION

0 mismatches over 5,000 values, 4 checks each, plus 3 whole-list checks

Look at the `all_ones` column for `0` and `7`: both true. Zero is the empty run of ones and seven is `111`. And look at `255`: eight set bits, all ones, not a power of two — three tests disagreeing about the same number is a good sign that they are testing different things.

Look at the reverse of `10110000` in eight bits: `00001101`. Reversed in thirty-two bits it would be a number in the hundreds of millions. The width is not a detail.

6 What it costs

Counting set bits — count the iterations out loud.

KERNIGHAN, one iteration per SET bit

```
176 = 10110000 → 3 iterations
```

```
255 = 11111111 → 8 iterations
```

```
1024 = 100000000000 → 1 iteration
```

```
1 << 31 → 1 iteration
```

THE NAIVE SHIFT LOOP, one per bit POSITION

```
176 → 8 iterations
```

```
255 → 8 iterations
```

```
1024 → 11 iterations
```

```
1 << 31 → 32 iterations
```

→ They TIE when every bit is set (255).

→ They differ most when one bit is set high (`1 << 31`): 1 against 32.

Both are $O(1)$ on fixed-width integers, and you should say so — thirty-two is a constant. The constant is the whole argument, and it is real when you do this inside a loop over a million values.

Counting bits for every number to `limit`.

counting_bits(n): one table write per value, each $O(1)$

n = 15 → 15 writes
n = 100,000 → 100,000 writes

→ $O(n)$ time, $O(n)$ space for the table itself.

Compare with calling count_set_bits on each value:
100,000 values x up to 17 iterations = ~1.7 million steps
against 100,000.

Subsets.

```
for mask in range(1 << n): for i in range(n):
```

outer loop: 2^n masks
inner loop: n positions each

→ $n \times 2^n$

n = 10	→	10 x 1,024	=	10,240	
n = 15	→	15 x 32,768	=	491,520	
n = 20	→	20 x 1,048,576	=	20,971,520	about a second
n = 25	→	25 x 33,554,432	=	838 million	no

→ which is why " $n \leq 20$ " in a problem statement is a hint
that the answer is a bitmask.

Walking only the set bits changes the inner loop.

inner loop over all n positions: n steps, always
inner loop with mask $\&=$ mask - 1: s steps, s = set bits

a mask with 3 bits set out of 20:
20 steps → 3 steps

→ nearly 7x, on the inner loop of a 2^n outer loop

Submask enumeration, which is the surprising one.

The naive way: for each mask, loop over all 2^n numbers and keep the ones that are submasks.

2^n masks $\times$ 2^n candidates = 4^n

$n = 12 \rightarrow 4^{12} = 16.7$ million

$n = 16 \rightarrow 4^{16} = 4.3$ billion

With `sub = (sub - 1) & mask`, you only ever visit real submasks. Across the whole enumeration each of the n bits is in exactly one of three states:

- out of the mask;
- in the mask and in the submask;
- in the mask and out of the submask.

Three states per bit:

3^n

$n = 12 \rightarrow 3^{12} = 531,441$

31x better

$n = 16 \rightarrow 3^{16} = 43$ million

100x better

Space.

every trick here is $O(1)$ extra space

`count_set_bits`, `is_power_of_two`, `hamming_distance`,
`reverse_bits`, `set_bit_positions`: a few integers

`counting_bits`: $O(n)$ - the table IS the answer

`subsets`: $O(n \times 2^n)$ - the output itself

`show()`: $O(\text{width})$ - a string

7 The traps

The negative-number infinite loop, which is the worst one here.

```
count_set_bits(-8)    # without the guard
```

This does not raise. It does not stop. In Python, integers are arbitrary width and a negative number behaves as though it has infinitely many leading ones, so clearing the lowest set bit just moves the problem left, forever.

```
-8 & -9 = -16
-16 & -17 = -32
-32 & -33 = -64
...
```

A hang is worse than a crash, because a crash tells you where it was. **Guard, or mask with `& 0xFFFFFFFF` first** — and say which one you are doing and why.

With the guard, you get something you can read:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in count_set_bits
ValueError: count_set_bits needs a non-negative number
```

The precedence trap, again, because it is the one that keeps happening.

```
1 << 4 - 1 = 8      this is 1 << (4 - 1)
(1 << 4) - 1 = 15   this is what you meant
```

No error. A plausible number. Every downstream result quietly wrong. Two brackets.

The power-of-two guard.

```
is_power_of_two(0)  without the `number > 0` guard → True
```

Because `0 & -1` is `0`. Zero is not a power of two, `0` is the input every test suite includes, **and the fix is eight characters.**

Testing at the top of the submask loop.

```
sub = mask
while sub:                # WRONG
    ...
    sub = (sub - 1) & mask
```

This never visits the empty submask, which is a legitimate answer and is usually the base case of whatever you are enumerating. **And if you fix it by moving the test the other way you get an endless circle**, because `(0 - 1) & mask` is `-1 & mask`, which is `mask` again. **The append-then-break shape is the correct one.**

Reversing bits without stating the width.

```
def reverse_bits_wrong(number: int) → int:
    result = 0
    while number:
        result = (result << 1) | (number & 1)
        number >>= 1
    return result
```

```
reverse_bits_wrong(1)      = 1      expected 2147483648
reverse_bits_wrong(0b1011) = 13     expected 3489660928
```

Nothing errors, and small inputs even look right — `0b1011` reversed is `1101`, which is 13, and that is a perfectly sensible answer to a different question. **The problem says thirty-two bits. Loop thirty-two times.**

The swap trick on the same variable.

```
values = [7, 9]
i = j = 0
values[i] ^= values[j]
values[j] ^= values[i]
values[i] ^= values[j]
```

```
values = [0, 0]
```

`x ^ x` is zero, so swapping a slot with itself destroys it. No error, no warning, and the bug only appears when two positions happen to coincide — **which in a sorting routine is exactly the case nobody tests.**

Bitwise operators on things that are not integers.

```
bin(5)[2] & 1
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for &: 'str' and 'int'
```

`bin()` gives a string. This catches people who reach for `bin()` to inspect a number and then keep going with the result.

And after any division:

```
1 << (6 / 2)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for <<: 'int' and 'float'
```

/ gives a float in Python 3. Use // anywhere near bit work.

Shifting by a negative amount:

```
1 << -1
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: negative shift count
```

Loud and immediate, which puts it in the good category.

And the ASCII case trick outside ASCII. `ord('a') ^ 32` is 'A', but `ord('1') ^ 32` is 17, a control character, and on anything outside the plain English letters the trick produces nonsense with no complaint. Say “for ASCII letters” out loud when you use it.

8 In the interview

How it gets asked

- “Count the number of 1 bits.” — LeetCode 191, and the follow-up is always “can you do better?”
- “Is this a power of two?” — a one-liner, and they are watching for the guard.
- “Return a list where the i -th element is the number of 1s in i .” — LeetCode 338.
- “How many bit positions differ between these two numbers?” — Hamming distance.
- “Reverse the bits of a 32-bit unsigned integer.” — LeetCode 190, and width is the whole question.
- “Generate all subsets of this list.” — and the bitmask answer is the one that gets remembered.

The first ninety seconds

On “count the set bits, and can you do better”:

“The obvious version looks at every bit: `n & 1` to read the bottom one, add it, shift right, repeat until the number is zero. I would write that first — it is correct and anybody can read it.

Its cost is one iteration per bit position, up to the highest set bit. So for `1 << 31` it takes thirty-two iterations to find one set bit. The work depends on where the highest bit is, not on how many bits are set, and that is the inefficiency.

The better version is `n &= n - 1` in a loop, counting the iterations. That is Kernighan’s algorithm.

Here is why it works, and this is the part I would want to say properly. Subtracting one from a number flips its lowest set bit to zero and turns everything below that into ones — that is ordinary borrowing. Above the lowest set bit, nothing changed, so ANDing keeps it. At the lowest set bit, one side is now zero, so ANDing removes exactly that bit. Below it, one side is all zeros. So `n & (n - 1)` clears the lowest set bit and nothing else.

That gives one iteration per set bit. `1024` takes one step instead of eleven. `1 << 31` takes one instead of thirty-two.

I would be honest about the worst case: if every bit is set they are identical. `255` is eight steps either way. The gain is on sparse numbers.

And strictly both are constant time on fixed-width integers, since thirty-two is a constant — but the constant is real if this sits inside a loop over a million values.

One Python caveat: this loop never terminates on a negative number, because Python integers have no top bit. I would guard, or mask with `& 0xFFFFFFFF`.

In production I would write `n.bit_count()` .“

The follow-ups

“Now give me the count for every number from 0 to n.”

“Not by calling the counter n times. That is up to seventeen steps per value at `n = 100,000` , so about 1.7 million steps, **when there is a one-line recurrence that does it in 100,000.**

```
bits[i] = bits[i & (i - 1)] + 1.
```

Read it as a sentence: the number of set bits in `i` is one more than the number in `i` with its lowest set bit removed. And that is exactly what `i & (i - 1)` gives me.

The reason this is a valid dynamic programme is that `i & (i - 1)` is strictly smaller than `i` , so by the time I reach `i` , the value I need is already in the table. I would say that out loud, because otherwise it looks like a coincidence.

There is a second recurrence and it is worth knowing both: `bits[i] = bits[i >> 1] + (i & 1)` — “the bits of `i` are the bits of `i` with its last bit chopped off, plus that last bit”. Same cost, and some people find it easier to justify.

$O(n)$ time and $O(n)$ space, and the space is the answer itself, so there is nothing to optimise away.”

“How would you generate every subset of a list?”

“A subset of n items is an n -bit number. Bit i set means item i is in. So counting from 0 to $2^n - 1$ lists every subset exactly once, with no recursion and no bookkeeping.

Outer loop over `range(1 << n)`, inner loop over the n positions, and `mask >> i & 1` asks whether item i is in this one.

The cost is $n \times 2^n$, and I want to be concrete about that. At $n = 20$ that is about twenty-one million operations — a second or so. At $n = 25$ it is 838 million, which is out of reach. So $n \leq 20$ in a problem statement is a strong hint that the intended answer is a bitmask.

If the masks are sparse I would walk only the set bits instead — `low = mask & -mask` to isolate one, `mask &= mask - 1` to move past it. Three bits set out of twenty turns twenty inner steps into three.

And if I need every *subset of a mask* rather than every subset of the list, the loop is `sub = (sub - 1) & mask`, which visits only real submasks and turns 4^n into 3^n — at $n = 16$ that is 4.3 billion against 43 million. The subtraction borrows into the lower bits and the AND throws away anything that was never in the mask.

The one thing I would be careful about is the loop shape: append, then break on zero, then step. Testing at the top skips the empty submask, and `(0 - 1) & mask` is `mask` again, so the obvious fix gives you an endless circle.”

“Is this a power of two? What about a number that is all ones?”

“Power of two: $n > 0$ and $(n \& (n - 1)) = 0$. A power of two has exactly one set bit, so clearing the lowest set bit must leave zero.

The $n > 0$ is not decoration. Without it, `is_power_of_two(0)` returns `True`, because `0 & -1` is `0` — and zero is in every test suite.

All ones — 0, 1, 3, 7, 15, 31 — is the matching pair: $n \& (n + 1) = 0$. Adding one to a run of ones carries all the way past the top, leaving a single high bit that shares no place with the original, so the AND is zero. That is one less than a power of two, and it is the shape of every low mask you build.

The related pair I would mention is $n \& -n$, which keeps only the lowest set bit instead of clearing it. **The reason is two’s complement:** $-n$ is $\sim n + 1$, and adding one to the flipped number ripples up to the first place that was a one — so above the lowest set bit everything is flipped and shares nothing, below it is all zeros, and the lowest set bit itself is one in both. That one is what a Fenwick tree is built on.”

The model answer

“Talk me through the bit tricks you actually know, and why each one works.”

“I would group them, because they are not ten unrelated facts.

First, the two moves on the lowest set bit, and they are opposites. `n & (n - 1)` removes it. `n & -n` keeps only it.

Both come from what subtraction does. `n - 1` borrows: the lowest one becomes zero and everything under it becomes one. So above the lowest one both numbers agree and AND keeps it; at the lowest one they disagree and AND kills it. `-n` is `~n + 1`, which is the same ripple from the other side, so everything above the lowest one is flipped and shares nothing, and only the lowest one survives.

Second, the masks. `1 << k` is one bit in place `k`. `(1 << k) - 1` is `k` ones at the bottom. `~(1 << k)` is all ones with a hole. Those three build every get, set, clear and toggle, and every fixed-width truncation. **And I bracket the shift every time, because** `1 << k - 1` is `1 << (k - 1)` and it fails silently.

Third, the one-line tests. Power of two is `n > 0` and `n & (n - 1) == 0` — exactly one bit, so clearing it leaves nothing, **and the guard matters because zero would otherwise pass.** All ones is `n & (n + 1) == 0` — the carry runs off the top. Odd is `n & 1`.

Fourth, XOR, which is ‘different’. `a ^ b` marks every place they disagree, so Hamming distance is the set-bit count of the XOR. **And** `a ^ a` is zero, so XOR-ing a whole list cancels every pair and leaves the unpaired one.

Fifth, and this is the one that stops being a trick: a subset is a number. Bit `i` set means item `i` is in, so counting from `0` to `2^n - 1` enumerates every subset with no recursion. **That is what makes bitmask DP possible, and it is why those problems cap around** `n = 20` — `2^20` is a million and `2^30` is a billion.

If I had to give you the single most useful line, it is `n & (n - 1)`, because counting set bits, testing powers of two, the counting-bits recurrence and walking a sparse mask are all the same expression **wearing four different hats.**

And the thing I would want to be judged on is not that I remember them. It is that each one is a sentence about borrowing or about flipping, and once you have the sentence, you can rebuild the expression at the desk.“

9 Recall card

Two core moves, and they are opposites. $n \& (n - 1)$ **CLEARs** the lowest set bit — $n - 1$ borrows, so the lowest 1 flips and everything below fills with ones; above it nothing changed. $n \& -n$ **KEEPS ONLY the lowest set bit** — $-n$ is $\sim n + 1$, the same ripple from the other side. Almost every trick is one of these in a loop.

Masks: $1 \ll k$ is one bit; $(1 \ll k) - 1$ is k ones at the bottom; $\sim(1 \ll k)$ is a hole. Always bracket — $1 \ll k - 1$ is $1 \ll (k-1)$, silently wrong.

One-line tests. Power of two: $n > 0$ and $n \& (n-1) = 0$ — the guard is not decoration, $0 \& -1 = 0$. All ones (0, 1, 3, 7, 15): $n \& (n+1) = 0$ — the carry runs off the top. Odd: $n \& 1$.

Counting. Kernighan `while n: n &= n-1` — **one step per SET bit** (1 against 32 for $1 \ll 31$, tied at 255), and it **HANGS on a negative in Python**, so guard or mask. Every count at once: `bits[i] = bits[i & (i-1)] + 1`, valid because $i \& (i-1) < i$. Hamming distance = set bits of $a \oplus b$, because **XOR is “different”**.

A subset IS a number. `for mask in range(1 << n)` lists all 2^n subsets; `mask >> i & 1` tests item i ; $n \times 2^n$ is why bitmask problems stop at $n = 20$. Walk sparse masks with isolate-then-clear. Every submask: `sub = (sub - 1) & mask`, **append-then-break-on-zero** — 3^n , not 4^n , and testing at the top skips the empty submask.

1 What this is, and why they ask it

A log line is one sentence about one thing that happened. A trace is the story of one request across every service it touched. Logs tell you what went wrong; traces tell you where.

Yesterday you learnt that metrics detect a problem and cannot explain it. Today is the other two pillars, and the single field that makes them useful: an identifier carried along with the request from the first service to the last.

They ask it because the modern version of “the site is slow” has no obvious owner. A page load touches eight services. Each one looks healthy. Each one’s own numbers are fine. Somebody has to be able to say which of the eight ate the four seconds, and if nobody wired that up in advance, nobody can.

And because it separates people who have run something from people who have only designed something. The candidate who says “I would add logging” has said nothing. The candidate who says “every log line carries the trace id, we sample traces at one percent but keep every trace that errored, and we never log the authorisation header” has clearly debugged a production incident at two in the morning.

By the end of this lesson you can explain what a span is, describe how the identifier gets from one service to the next, size a logging bill and be shocked by it, choose a sampling strategy and say what it costs you, and name the fields that must never appear in a log line.

2 The story

The trunk was steel, painted green, and it had gone from Chennai on the ninth of June and had not arrived.

Savitri’s son was starting at the college in Guwahati and everything he owned for the next three years was inside it — bedding, a stove, the good plates, a cricket bat that had been his uncle’s. She had sent it by train because everyone said the train was safe, and it was, and it was also gone.

She went to the office at the station on the fourth of July, which was the third time. The same man was behind the counter. He was not unkind. He simply had nothing to tell her, because the trunk was not in Guwahati and that was all his book said.

Then a different clerk came on at four o'clock and asked for the paper.

Savitri had kept it in a plastic folder, folded twice — the long thin receipt they had given her in Chennai, with the number printed at the top in fat black type.

The new man turned it over.

On the back there were marks. Savitri had never looked at the back of it. There were four rubber stamps, in different inks, each one with a place and a date, and a fifth mark in pencil.

He read them out to her as though he were reading a bus timetable.

“Chennai, ninth. Vijayawada, eleventh. Kharagpur, fourteenth.”

Then he stopped.

“And nothing after Kharagpur.”

He said it three weeks earlier than anybody had said anything, and he said it in about eleven seconds, and he had not made a single telephone call.

Savitri asked how he knew.

“Every place it passes through puts its own mark on the back with the same number on the front. So I do not have to ask four stations where your box is. I only have to find the last station that admits it had it.”

He wrote the number on a slip and handed it to a boy.

“It is at Kharagpur. It has been at Kharagpur for twenty days. Somebody there put it against the wrong wall, and until now nobody had all four marks in one hand at the same time.”

3 The idea in plain English

The number printed on the front of Savitri's receipt is the whole lesson. Every place the trunk passed through wrote its own mark, **and every mark carried the same number.** Nobody had to search. They only had to put the marks in one place and read them in order.

Logs, first

A log line is one record of one event, written by one service, at one moment. “Order 8812 accepted.” “Payment declined, code 51.” “Took 340 milliseconds.”

A log is written for a human who is not there yet. That single fact decides everything about how you write one.

Levels say how loud a line is:

DEBUG	only useful when you are already inside the problem
INFO	the normal life of the system: this happened, that finished
WARN	something odd that did not stop anything
ERROR	this request failed
FATAL	the process is going down

In production you keep INFO and above, and turn DEBUG on for one service when you need it. Leaving DEBUG on everywhere is the fastest way to make the whole system expensive and unsearchable at the same time.

Structured logging, which is the one habit that matters

A log line should be a set of named fields, not an English sentence.

BAD	"Order 8812 for user 4471 failed after 340ms: card declined"
GOOD	{"event": "order_failed", "order_id": 8812, "user_id": 4471, "duration_ms": 340, "reason": "card_declined", "trace_id": "a3f1c2", "service": "payments", "level": "error"}

The bad one is readable by a person and useless to a machine. To count how many orders failed for card declines you would have to pull the sentence apart with pattern matching, and the pattern breaks the moment somebody rewords the message.

The good one you can filter, count and group. And critically, **you can ask “show me every line with this trace id”** — which is the entire point of the next section.

The problem: one request, many services

A single page load in a real product touches five to ten services. Each of them logs. Each of their logs is a separate pile.

```
user clicks "buy"
  → gateway      logs: "request in"
  → orders        logs: "order created"
  → inventory     logs: "stock reserved"
  → payments      logs: "charge sent"
  → payments      logs: "charge took 3.9 seconds"
  → notifications logs: "email queued"
```

Six lines, in five different places, written by five different machines, with five different clocks.

NOTHING connects them.

And that is Savitri's problem exactly. Four stations, four books, and no way to ask one question that touches all four.

The fix: one identifier, carried

When a request first enters the system, the front door generates a random identifier — the trace id — and attaches it to the request. Every service that handles that request **puts the trace id on every log line it writes, and passes it on** to every service it calls.

That is the rubber stamp with the same number on the front.

Passing it on means putting it in the request header. The standard is **W3C Trace Context**, and the header is called `traceparent`:

```
traceparent: 00-a3f1c2d4e5f60718293a4b5c6d7e8f90-b7ad6b7169203331-01
              ^^ ^----- trace id -----^ ^--- span id ---^ ^^
              |                                     |
            version                               flags (sampled?)
```

Every service reads that header, logs the trace id, and sends a new one downstream with its own span id.

Spans, and what a trace really is

A span is one unit of work with a start time and an end time. “The payments service handled this call.” “This query ran.” **A span has a name, a duration, a span id, and the span id of its parent.**

A trace is all the spans that share a trace id, arranged by their parent links into a tree.

```

TRACE a3f1c2                                     total 4,010 ms

gateway.handle                                0 ----- 4010
orders.create                                20 ----- 3990
  inventory.reserve                          30 --- 95
    payments.charge                          100 ----- 3960
      bank.authorise                         120 ----- 3890
        notifications.queue                  3965 - 3985

```

Read that and you have your answer in one second. The four seconds is not in the gateway, not in orders, not in inventory. It is `bank.authorise`, and everything above it is just waiting.

That picture is the reason tracing exists. No amount of staring at five separate piles of log lines gets you there as fast.

Sampling, because you cannot keep all of it

Keeping a full trace of every request is unaffordable, and section 6 does the arithmetic. So you keep some fraction.

Head sampling decides at the front door — “keep one request in a hundred” — and puts that decision in the `traceparent` flags so every downstream service agrees. It is cheap and simple, and its flaw is obvious: the one request you want to look at is almost certainly one of the ninety-nine you threw away.

Tail sampling buffers all the spans of a trace until the request finishes, then decides: keep it if it errored, keep it if it was slow, otherwise keep one in a hundred. You get exactly the traces you want, and you pay for it by holding every in-flight trace in memory somewhere.

The practical answer, and the one to say in an interview: head-sample the boring traffic at one percent, and always keep errors and anything over the p99.

What must never be in a log line

Logs get copied, shipped, and kept for months, and far more people can read them than can read the production data itself. So:

NEVER LOG

- passwords, even hashed
- authorisation headers, bearer tokens, session cookies
- full card numbers, CVVs
- one-time passwords
- full addresses, phone numbers, identity numbers

LOG INSTEAD

- user_id (an internal number)
- card_last4
- "auth: present" rather than the header

This is not a compliance box to tick. The most common way a real secret escapes is that somebody logged the whole request object once, during a bug hunt, and never took it out.

4 The picture

One request, one identifier, five services:

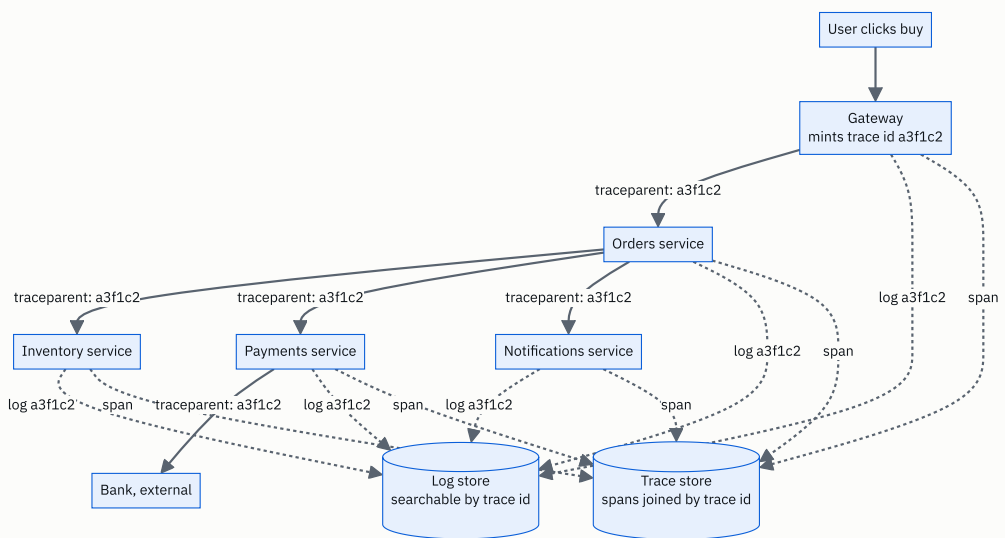


FIGURE 172.1

Notice that the trace id is minted once, at the front door, and never regenerated. Every solid arrow carries it forward in a header; every dotted arrow writes it into a store. **One search on `a3f1c2` returns every line and every span from the whole journey** — which is the clerk turning the receipt over.

The waterfall, which is what you actually look at:

```

TRACE a3f1c2                total 4,010 ms
time (ms)  0      500    1000   1500   2000   2500   3000   3500   4000
           |      |      |      |      |      |      |      |      |

gateway.handle
[=====] 4010

  orders.create
  [=====] 3970

    inventory.reserve
    [=] 65

    payments.charge
    [=====] 3860

      bank.authorise
      [=====] 3770

        notifications.queue
                                     [=] 20

READ IT TOP DOWN AND ASK WHERE THE WIDTH IS.
Everything above bank.authorise is a bar that is wide only
because it is WAITING for the bar below it.

The answer is: the bank took 3.77 seconds.
Nothing you own is slow.

```

That last line is the most valuable output of a tracing system, and it is the one metrics can never give you: the service that looks slowest on a dashboard is usually the one waiting for the real culprit.

5 How it actually works

Getting the identifier from one service to the next

There are two halves, and both are libraries you do not write yourself.

Propagation is reading `traceparent` from the incoming request and putting it on every outgoing one. **The standard is W3C Trace Context**, and the reason a standard matters is that your services are not all in the same language. **A Python service and a Java service and a Go service all read the same header.**

Context storage is the awkward half. Between reading the header and making the outgoing call, **the trace id has to travel through your own code** — through function calls that know nothing about tracing.

```
Python:    contextvars          (works across async/await)
Java:      ThreadLocal + the OpenTelemetry Context
Go:        context.Context, passed explicitly as the first argument
Node:      AsyncLocalStorage
```

This is where propagation actually breaks in practice. The moment work hops onto a background worker, a message queue, or a pool, **the context is gone unless somebody carried it across deliberately.** A trace that stops halfway is almost always this.

What emits the data

OpenTelemetry is the answer to give. It is the vendor-neutral standard — one set of libraries that produces logs, metrics and traces, and one agent (**the OpenTelemetry Collector**) that receives them and forwards them wherever you like.

The reason it matters commercially is lock-in. Instrumenting your code with a vendor's own agent means re-instrumenting everything when you change vendor. **Instrumenting with OpenTelemetry means changing one line of collector configuration.**

And most of the instrumentation is automatic. Auto-instrumentation wraps the common libraries — the HTTP client, the web framework, the database driver — so **a service gets useful spans without anyone editing business code.** Hand-written spans are for the parts that are specific to you.

Where it all goes

```
LOGS
Elasticsearch + Kibana ("ELK")  search anything, expensive to run
Grafana Loki                    indexes only labels, stores the rest
                                compressed. Much cheaper, less flexible
Splunk                          the enterprise default, famously costly
Managed: Datadog Logs, CloudWatch Logs

TRACES
Jaeger                          the open-source default, from Uber
Zipkin                          the original, from Twitter
Grafana Tempo                   cheap: object storage, search by trace id only
Managed: Datadog APM, Honeycomb, AWS X-Ray, Google Cloud Trace

SHIPPING
Fluent Bit / Fluentd / Vector   read log files, forward them
OpenTelemetry Collector         receive, sample, batch, forward
```

The design choice worth knowing is Loki's and Tempo's. Elasticsearch indexes every word of every log line, which is why you can search for anything and why it costs so much. **Loki indexes only a handful of labels — service, level, environment — and keeps the message body compressed and unsearched.** You give up free-text search over everything and you get an order of magnitude off the bill.

How a log line gets out of the process

Write to standard output, and let something else deal with it. That is the modern answer and it is worth knowing why.

```
the process writes JSON lines to stdout
→ the container runtime captures them to a file
→ a shipper (Fluent Bit) tails that file
→ forwards, batched, to the store
```

The service should never make a network call to write a log line. If it does, then a slow log store makes your product slow, **and an unreachable log store makes your product unavailable** — which is a genuinely famous class of outage.

Writing is also asynchronous inside the process: the application appends to a small in-memory buffer and a background writer flushes it. **The cost of that is real and you should name it — if the process crashes hard, the last few lines in the buffer are gone,** and those are exactly the lines about the crash.

What happens when the logging system fails

The rule from yesterday applies here too: the monitoring stack must be able to fail without taking the product with it.

log store unreachable	→ shipper buffers to local disk, then drops the OLDEST. The product keeps serving.
local disk full	→ this one HAS taken sites down. Log files that nobody rotates fill the disk, and then every write in the whole process fails. Rotation and a size cap are not optional.
trace store unreachable	→ spans are dropped. Nothing user-facing notices. This is the correct behaviour.

6 The numbers

Take the same system as yesterday: 100 million requests a day.

```
100,000,000 requests / 86,400 seconds = 1,157 requests/second average
peak is roughly 3x the average         = ~3,500 requests/second
```

Logs, and this is the number that shocks people.

```
services touched per request           8
log lines each service writes per request 3
-----
log lines per request                  24

24 x 100,000,000 = 2,400,000,000 lines/day

a structured JSON line, with a trace id,
timestamps and a dozen fields          ~ 300 bytes

2.4e9 x 300 bytes = 720,000,000,000 bytes
                  = 720 GB PER DAY, raw
```

Compare that with yesterday's metrics: about 9 GB a day. Logs are eighty times the volume.

```
compressed, roughly 10:1           72 GB/day
kept for 30 days                    2,160 GB = 2.16 TB
```

Now price it, because this is the part candidates never do.

```
MANAGED (typical list price, ~$0.50 per GB ingested):
  720 GB/day x $0.50                = $360/day
                                    = $10,800/month
                                    = $129,600/year

SELF-HOSTED on object storage:
  72 GB/day compressed x 30 days = 2.16 TB
  2.16 TB x $0.023/GB-month         = ~$50/month for the storage
  plus the search cluster:          3 machines x $300 = ~$900/month

→ The managed logging bill can genuinely exceed the bill for
  the servers running the product. This is not a joke and it is
  the reason sampling and level discipline exist.
```

Traces.

spans per request (8 services, some with 2 spans) ~12
bytes per span (name, ids, timings, a few tags) ~500 bytes

UNSAMPLED:

$12 \times 100,000,000 = 1,200,000,000$ spans/day

$1.2e9 \times 500 \text{ bytes} = 600,000,000,000 = 600 \text{ GB/day}$

HEAD-SAMPLED AT 1%:

$600 \text{ GB} \times 0.01 = 6 \text{ GB/day}$ → affordable

PLUS every errored trace (say 0.5% of requests):

$600 \text{ GB} \times 0.005 = 3 \text{ GB/day}$

→ ~9 GB/day, and you keep the traces you actually want.

The three pillars, side by side, at this scale:

METRICS	9 GB/day	detect	
TRACES	9 GB/day	localise	(1% + all errors)
LOGS	720 GB/day	diagnose	

→ Logs are 40x the other two COMBINED.

→ Which is why the first cost lever anyone pulls is log level, and the second is retention.

Retention, priced.

30 days of hot, searchable logs	2.16 TB	~\$900/month cluster
7 days hot + 90 days in cold storage		
hot: $7 \times 72 \text{ GB} = 504 \text{ GB}$		smaller cluster, ~\$300/month
cold: $90 \times 72 \text{ GB} = 6.5 \text{ TB}$ in S3 Glacier at \$0.004/GB		
		= ~\$26/month

→ Three-quarters off, and the cost is that a 60-day-old investigation takes hours instead of seconds.

One more, on cardinality — the same trap as yesterday, wearing different clothes.

A trace id is a HIGH-CARDINALITY VALUE: every request has a different one. 100 million distinct values a day.

In a LOG store that is fine - it is a field in the document, and Elasticsearch or Loki look it up.

In a METRICS store it is fatal - it would create 100 million time series a day.

→ Trace ids go in logs and spans. NEVER in a metric label.
This is the single most useful sentence connecting the two days.

7 The trade-offs

Sampling: you save ninety-nine percent of the cost and you lose the exact trace somebody asks about.

Head sampling is decided at the front door and is cheap, stateless, and consistent across services. **It throws away the interesting requests at the same rate as the boring ones**, so when a customer rings up with a request id, the trace is usually gone.

Tail sampling keeps what matters, and to do it the collector must hold **every span of every in-flight trace** until the request completes — typically thirty seconds of buffer. **At 3,500 requests a second with twelve spans of 500 bytes each, that is $3,500 \times 12 \times 500 \times 30 \approx 630$ MB of memory held constantly**, and the collector becomes a stateful thing that can itself fall over.

I would not use tail sampling if the collector fleet has to be simple and stateless, or the traffic is so spiky that the buffer would need to be sized for a peak you cannot predict. **I would use it if** the product has paying customers who report specific failed requests, because head sampling makes those unanswerable.

Log levels: DEBUG everywhere is a 5-10x bill and an unsearchable haystack.

Turning DEBUG on globally is the most common self-inflicted cost incident there is. The right shape is **INFO by default, with the ability to raise one service to DEBUG at runtime, for a while, without a redeploy.** That last clause is worth saying — a debug switch that needs a deployment is a debug switch nobody uses during an incident.

Structured logging: machine-readable costs you human-readable.

A wall of JSON is genuinely harder to read with your eyes than a wall of sentences. The trade is worth it because you almost never read logs with your eyes any more — you filter them. **But keep a human-formatted output for local development,** or every engineer will quietly hate the system.

Free-text search costs an order of magnitude.

Elasticsearch lets you search for any word in any line, and you pay for a full index of every log line ever shipped. Loki indexes a few labels and greps the rest at query time. I would use Elasticsearch if investigations routinely start from an unknown string — an error message somebody pasted into chat. I would use Loki if investigations almost always start from a service, a time window, and a trace id, which in a well-instrumented system they do.

Synchronous versus asynchronous writing.

Synchronous is honest — the line is on disk before the function returns — and it puts disk latency on the request path. Asynchronous is fast and loses the buffered lines if the process dies, which is precisely when you wanted them. The usual compromise: asynchronous for INFO, synchronous flush for ERROR and FATAL.

And the honest limitation of the whole approach: tracing tells you where the time went, not why.

The waterfall says `bank.authorise` took 3.77 seconds. It does not say whether the bank was slow, the connection pool was empty, or a retry happened twice with a backoff between them. **You still need the logs inside that span,** which is why the trace id on every log line is the thing that makes both useful. **Neither pillar is sufficient. That is why there are three.**

8 In the interview

How it gets asked

- “A request took four seconds. How do you find out which service was slow?” — the direct form.
- “How would you debug an issue that only affects one user?” — trace id, and how the user gives it to you.
- “What would you log?” — they are listening for structure and for what you refuse to log.
- “How do you connect logs across microservices?” — correlation id, propagated in a header.

- *“Your logging bill is a hundred thousand a year. Cut it.”* — sampling, levels, retention tiers.

The first ninety seconds

On “a request took four seconds, find the slow service”:

“I would look at the trace, and the reason I can is that the design has one identifier per request that every service carries.

Concretely: the gateway mints a random trace id when the request first arrives. It puts it in the `traceparent` header on every downstream call, and every service does the same — reads it, logs it on every line it writes, passes it on. That is the W3C Trace Context standard, so it works across languages.

Each service also emits a span: a piece of work with a name, a start, an end, its own id, and its parent’s id. All the spans with the same trace id form a tree, and drawn as a waterfall that tree tells you where the four seconds went in about one second of looking.

The thing to read for is which bar is wide because it is working, and which is wide because it is waiting. In a typical case the gateway span is 4,000 milliseconds and so is the orders span and so is the payments span — they are all just waiting on `bank.authorise`, which is 3,770. So the answer is the external bank call, and nothing I own is slow.

Then I switch to logs to find out why, filtered by that same trace id. That is what makes the two work together: the trace localises, the logs explain.

I would mention sampling, because at any real volume I am not keeping every trace. Full traces of 100 million requests a day is about 600 GB a day. So: one percent head sampling for normal traffic, and keep every trace that errored or breached the p99. That way the four-second request is one I still have.”

The follow-ups

“What exactly do you put in a log line, and what do you refuse to put in?”

“Structured, not prose. A JSON object of named fields — `event`, `service`, `level`, `trace_id`, `user_id`, `duration_ms`, and whatever is specific to that event. **Not** `'Order 8812 failed after 340ms'`, because to count those I would have to pull the sentence apart with pattern matching, **and the pattern breaks the day somebody rewords the message.**

The mandatory fields are timestamp, level, service, and trace id. The trace id is the one that turns five separate piles into one story.

What I refuse to log: anything that is a secret or identifies a person more than I need. Passwords — including hashed ones. Authorisation headers, bearer tokens, session cookies. Full card numbers. One-time codes. Full addresses and phone numbers. **I log** `user_id` **and** `card_last4` **and** `'auth: present'`.

The reason is not compliance, it is blast radius. Logs are copied, shipped and kept for months, and far more people can read them than can read the production data. **And the realistic way a token leaks is not a decision — it is somebody logging the whole request object during a bug hunt and never taking it out.** So I would want redaction in the logging library itself, by field name, so that it is the default rather than a thing every engineer has to remember.

One more: levels. INFO and above in production, **with a runtime switch to raise a single service to DEBUG without a redeploy.** DEBUG everywhere is a five- to tenfold bill and an unsearchable haystack, and a debug switch that needs a deployment is one nobody uses during an incident.”

“How does the trace id actually get from one service to the next, and where does that break?”

“Two halves. Between services it is a header — `traceparent` . Inside a service it is a context object.

The incoming middleware reads the header and puts the trace id into a context store: `contextvars` in Python, the OpenTelemetry `Context` on a `ThreadLocal` in Java, an explicit `context.Context` in Go, `AsyncLocalStorage` in Node. The outgoing client reads it back out and sets the header. Both ends are library code — nobody writes this by hand, and auto-instrumentation wraps the common HTTP clients and database drivers so most services get spans without touching business code.

Where it breaks, in practice, is every hop that is not a direct call.

A message queue. The trace id has to be put into the message’s own metadata by the producer and read back out by the consumer. **If nobody did that, the trace stops at the queue** and the consumer’s work looks like an unrelated orphan trace.

A thread pool or a background job. The context lives on the request’s thread or task; hand the work to a pool and it is gone.

A third party. You cannot make the bank propagate your header, **so the trace ends at your side of that call** — which is fine, because your span still measures how long they took.

The symptom is always the same and it is worth recognising: a trace that just stops halfway. That is almost never a broken tracing system. It is a hop where nobody carried the context across.“

“Your logging bill is a hundred and thirty thousand a year. Cut it in half, and tell me what it costs you.”

“Four levers, and I would pull them in this order, because they get progressively more painful.

First, levels. Confirm nothing is running at DEBUG in production. **That alone is often five to ten times the volume of a single service.** Cost: nothing, if the runtime switch exists.

Second, retention tiering. Instead of thirty days of hot searchable storage, keep **seven days hot and ninety days in object storage.** At 72 GB a day compressed, that is 504 GB in the hot cluster instead of 2.16 TB, **and 6.5 TB in Glacier at four-tenths of a cent per gigabyte, which is about twenty-six dollars a month.** Cost: **an investigation into something sixty days old takes hours instead of seconds.** For most products that is the right trade.

Third, sampling the successful requests. Keep every log line for anything that errored, and one in ten for requests that succeeded normally. Successful INFO lines are the bulk of the volume and the least useful. **Cost: you can no longer reconstruct an arbitrary successful request** — which matters if support needs to prove what happened for a specific customer, so I would confirm that before doing it.

Fourth, move off per-gigabyte-ingested pricing. At 720 GB a day, a store that indexes only labels — Loki or Tempo on object storage — is roughly an order of magnitude cheaper. **Cost: no free-text search over the message body. I would only do this if investigations normally start from a service, a time window and a trace id, rather than from an error string somebody pasted into chat.**

The first two are free wins and I would do them today. The last two are real trades and I would want the support team in the room.“

The model answer

“Design the observability for the system you have just drawn.”

“Three pillars, and it is a sequence, not a menu: metrics to detect, traces to localise, logs to diagnose.

Metrics are cheap and aggregated, so they can cover everything and page somebody. The four golden signals per service — latency split by success and failure, traffic, errors as a fraction, saturation — with percentiles rather than averages. **About 9 GB a day at this scale.**

Traces are the middle layer and the one people forget. The gateway mints a trace id per request, every service propagates it in `traceparent` and emits spans with parent links, **and the waterfall shows where the time went across all eight services in one picture.** Unsampled that is 600 GB a day, so: **one per-cent head sampling, plus every trace that errored or breached the p99.** That brings it to about 9 GB a day and keeps the traces anyone will actually ask about.

Logs are the expensive layer, so they are the last resort rather than the first. Structured JSON, INFO and above, **every line carrying the trace id.** At eight services, three lines each, 300 bytes a line and 100 million requests, **that is 720 GB a day — eighty times the metrics volume,** and on managed per-gigabyte pricing it is over a hundred thousand a year. **So seven days hot, ninety days cold, and a runtime DEBUG switch per service rather than DEBUG everywhere.**

The one field that makes all three work together is the trace id. It goes on every span and every log line — and never on a metric label, because it is unbounded and would create a hundred million time series a day.

For emitting it I would use OpenTelemetry, because it is vendor-neutral: one instrumentation, and the collector decides where the data goes. **Changing observability vendor becomes a configuration change rather than re-instrumenting fifty services.**

Two operational points I would not leave out. Services write logs to standard output and a shipper forwards them — never a network call on the request path, or a slow log store makes the product slow and an unreachable one makes it unavailable. **And the whole stack lives in a separate failure domain from the product,** or the outage takes away the thing you were going to use to investigate the outage.

The workflow this buys is the one I would describe last, because it is the point of all of it. The alert fires on a symptom from a metric. The trace tells you which of eight services owns the time. The logs for that trace id tell you why. Three steps, minutes rather than hours, and none of it is possible unless somebody put the identifier in before the incident.“

9 Recall card

One identifier, minted at the front door, carried everywhere. The trace id goes in the `traceparent` header (W3C Trace Context) between services and in a **context object** (`contextvars`, `ThreadLocal`, `context.Context`, `AsyncLocalStorage`) inside one — and it goes on every log line. A trace that stops halfway is almost always a hop nobody carried context across: a queue, a thread pool, a third party.

A SPAN is one unit of work with a name, a duration, an id and a parent id. A TRACE is every span sharing a trace id, as a tree. Read the waterfall and ask which bar is wide because it is working and which is wide because it is waiting — the slow-looking service is usually just waiting for the real culprit.

Structured, not prose: `{"event":..., "trace_id":..., "duration_ms":...}`, because you cannot count sentences. **Mandatory fields:** **timestamp, level, service, trace id.** NEVER log passwords, auth headers, tokens, full card numbers or one-time codes — redact by field name in the library, because the real leak is somebody logging the whole request object during a bug hunt.

The arithmetic that decides everything. $100\text{M requests} \times 8 \text{ services} \times 3 \text{ lines} \times 300 \text{ bytes} = 720 \text{ GB/day of logs}$, against **9 GB/day of metrics — eighty times** — and over **\$100k/year** on per-gigabyte pricing. Traces unsampled are 600 GB/day; **1% head sampling plus every error and every p99 breach** brings it to $\sim 9 \text{ GB/day}$. Cut the log bill with levels, then retention tiers (7 days hot + 90 cold), then sampling successes, then a label-only store like Loki.

Metrics detect, traces localise, logs diagnose — in that order. Trace ids belong in logs and spans and NEVER in a metric label (100M series a day). **Write logs to stdout and let a shipper forward them**, never a network call on the request path; rotate the files, or a full disk takes the process down; and keep the whole stack in a separate failure domain from the product.

DSA topic: The bit tricks every interview uses **System design topic:** Logging and distributed tracing

Code these, in this order

One rule for the whole set: **before you write an expression, say the sentence it stands for.** “Clears the lowest set bit.” “Marks every place they differ.” **If you cannot say the sentence, you have memorised a shape rather than learnt a move**, and it will not survive a problem you have not seen.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Hamming Distance	LeetCode 461 (Easy)	That you know XOR means “different”. One line.
2	Counting Bits	LeetCode 338 (Easy)	The one-line recurrence, and why the smaller answer is already there.
3	Reverse Bits	LeetCode 190 (Easy)	Fixed width, and why Python has to be told what the width is.
4	Subsets	LeetCode 78 (Medium)	A subset is a number. The iterative bitmask answer, not backtracking.
5	Single Number	LeetCode 136 (Easy)	Pairs cancel. Warm-up for tomorrow.
6	Maximum Product of Word Lengths	LeetCode 318 (Medium)	A word as a 26-bit mask, and “no shared letters” as <code>a & b == 0</code> .

On problem 1, resist the loop

Write it in one line. **Then say out loud why counting the set bits of `a ^ b` is the same as counting the places where they disagree.** If that sentence does not come easily, the one-liner is memorised, not known.

On problem 2, find both recurrences

Get it with `bits[i] = bits[i & (i - 1)] + 1`, then again with `bits[i] = bits[i >> 1] + (i & 1)`. **Say what each one claims about the number**, in a sentence each. Then say why both are valid dynamic programmes — the argument is the same three words for both.

On problem 3, break it first

Write the version that loops `while number:` instead of thirty-two times. **Run it on 1 and record what it gives.** Say why that answer is not wrong so much as an answer to a different question. Then fix it.

On problem 4, write it twice

Once with backtracking, once with `for mask in range(1 << n)`. **Time both on a twenty-element list.** Then say which one you would write in an interview and why — and note that the answer is not always the fast one.

On problem 6, invent the mask

Nobody tells you to use a mask here. **Work out for yourself that a lowercase word is twenty-six yes-or-no answers**, so it fits in an integer, and that “these two words share no letter” becomes `a & b == 0`. **Say what that turns an $O(n \times m)$ character comparison into.**

Then the two-core-moves drill

Take 176. Compute `n & (n - 1)` and `n & -n` by hand, writing both bit patterns out. **Say which one removes and which one keeps.** Then explain each in terms of borrowing or flipping — not by what it produces.

Then the guard drill

Delete the `number > 0` from the power-of-two test and run it on 0. **Record the answer.** Say what `0 & -1` is and why that is the whole bug. Then write the all-ones test and check it agrees with itself on 0.

Then the hang drill

Run `count_set_bits(-8)` with the guard removed. **Stop it after two seconds.** Then compute `-8 & -9` and `-16 & -17` by hand and say why the loop can never reach zero. Say why a hang is worse than a crash.

Then the submask drill

Enumerate every submask of 1011 by hand, biggest first. **Then write the loop and check you got the same eight.** Move the test to the top of the loop and record what you lose. Then work out what `(0 - 1) & mask` is, and why that makes the obvious fix an endless circle.

Then the logging drill

Take one log line you have written or seen — any one — and rewrite it as structured fields. **Name the four fields that must be on every line.** Then go through the same line and say what in it must never be logged, and what you would put there instead.

Then the trace drill

Take the six-service request from the lesson and say, out loud, **where the trace id is created, how it reaches the fifth service, and where it is written down.** Then name three places the chain breaks in real systems.

The core-moves drill

1. Give both expressions and say which removes and which keeps.
2. Explain `n & (n - 1)` in terms of borrowing, without saying what it produces.
3. Explain `n & -n` in terms of two's complement.
4. Say what each one is used for, one use each.

The masks drill

1. Give the three masks and what each looks like in eight bits.
2. Say what `1 << k - 1` actually is and what you meant.
3. Say what `(1 << width) - 1` does to a negative Python number.
4. Give `x & (size - 1)` and the condition under which it is a remainder.

The tests drill

1. Give the power-of-two test with its guard and say why the guard exists.
2. Give the all-ones test and say why the carry makes it work.
3. Give the odd test.
4. Say which numbers make all three disagree, and why that is a good check.

The counting drill

1. Give Kernighan and the iteration count for `1024`, `255` and `1 << 31`.
2. Say when it ties with the naive loop.
3. Say what it does on a negative number in Python, and why.
4. Give both counting-bits recurrences and the reason each is valid.

The subsets drill

1. Say what a subset is, as a number.

2. Give the two-loop enumeration and the test for “is item i in”.
3. Give the cost, and the value of `n` at which it stops being possible.
4. Give the sparse-mask walk and what it saves.
5. Give the submask loop, its shape, and `3^n` against `4^n`.

The pillars drill

1. Say what a log line is and what a span is.
2. Say what a trace is, in one sentence.
3. Give the sequence: which pillar detects, which localises, which diagnoses.
4. Say which pillar the trace id must never appear in, and why.

The propagation drill

1. Name the header and the standard.
2. Name the in-process context mechanism in two languages.
3. Give three places propagation breaks.
4. Say what the symptom of a broken chain looks like.

The numbers drill

1. Compute the daily log volume from 100M requests, 8 services, 3 lines, 300 bytes.
2. Compare it with the daily metrics volume and give the ratio.
3. Price it at fifty cents per gigabyte ingested.
4. Compute unsampled trace volume, then at 1% plus errors.
5. Give the retention-tier saving and what it costs you.

The hygiene drill

1. Name six things that must never appear in a log line.
2. Say what you log instead, for three of them.
3. Say where redaction belongs and why there.
4. Say why logs are written to standard output rather than sent over the network.
5. Say what a full disk does to a process, and the two settings that prevent it.

The break-it drill

For each, say what happens and whether anything reports it:

1. `1 << 4 - 1` when you meant a mask of four ones.
2. `is_power_of_two(0)` without the guard.

3. `while n: n &= n - 1` on `-8`.
 4. Reversing bits with `while number:` instead of a fixed width.
 5. The XOR swap when both positions are the same.
 6. `bin(5)[2] & 1`.
 7. Head sampling at 1%, when a customer rings with a specific slow request.
 8. A background worker picking up a job, with nobody carrying the context across.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Count the number of ones in the binary representation.* The naive loop and its iteration count, `n & (n-1)` explained by borrowing, the case where they tie, the Python hang on negatives, and `bit_count()` as the production answer.
 2. *Generate every subset of this list, and tell me when that stops being possible.* A subset is a number, the two-loop enumeration, `n × 2^n` with the figure at `n = 20` and `n = 25`, the sparse walk, and the submask loop with `3^n` against `4^n`.
 3. *A request took four seconds. How do you find out which service was slow?* The trace id minted at the front door, `traceparent` propagation, spans and parent links, reading the waterfall for waiting rather than working, then logs filtered by the same id — and the sampling policy that means you still have the trace.
-

Before you move on

- [] I can give both core moves and say which removes and which keeps.
- [] I can explain `n & (n - 1)` by borrowing, not by its output.
- [] I can explain `n & -n` by two's complement.
- [] I can write all three masks and I bracket every shift.
- [] I know what `1 << k - 1` really is.
- [] I can give the power-of-two test and say why the guard is not decoration.
- [] I can give the all-ones test and explain the carry.
- [] I know Kernighan's iteration counts for `1024`, `255` and `1 << 31`.
- [] I know it hangs on a negative in Python, and why a hang is worse than a crash.
- [] I can give both counting-bits recurrences and justify each.

- [] I know Hamming distance is the set-bit count of the XOR, and why.
- [] I can enumerate every subset with two loops and no recursion.
- [] I know $n \times 2^n$ and the value of n where bitmasks stop working.
- [] I can walk only the set bits of a sparse mask.
- [] I can write the submask loop, in the right shape, and say why that shape.
- [] I know 3^n against 4^n and what it is worth at $n = 16$.
- [] I can say what a log line, a span and a trace each are.
- [] I know the sequence: metrics detect, traces localise, logs diagnose.
- [] I can name the header and the standard that carry the trace id.
- [] I can name three places propagation breaks, and the symptom.
- [] I can compute 720 GB a day from first principles.
- [] I know logs are about eighty times the metrics volume.
- [] I can price a logging bill and name the four levers to cut it.
- [] I know the trace id must never be a metric label, and why.
- [] I can name six things that must never be logged, and what to log instead.
- [] I know why services write to standard output rather than over the network.
- [] I can describe today's request path out loud, from the front door to the last service.
- [] I answered all three questions above out loud.

XOR problems

DATA STRUCTURES AND ALGORITHMS

XOR problems

SYSTEM DESIGN

SLAs, SLOs, and error budgets

1 What this is, and why they ask it

XOR has one property that matters, and it solves a whole family of interview questions: $a \oplus a$ is zero. A thing XORed with itself disappears.

So if you XOR a whole list together, everything that appears twice cancels itself out, and what is left is whatever had no partner. No extra memory. One pass. No sorting.

And because XOR does not care about order — $(5 \oplus 3) \oplus 9$ and $9 \oplus (3 \oplus 5)$ are both 15 — you never have to know which pair cancelled which. That is the part that makes it feel like a trick, and it is the part worth being able to explain.

They ask it because the constraint gives it away and candidates still miss it. “Find the number that appears once, in linear time and constant extra space.” Linear time and constant space rules out sorting and rules out a frequency map, and once those are gone, XOR is essentially the only tool left. An interviewer saying “O(1) space” out loud is saying the word XOR without saying it.

And because the family goes deeper than the famous one. One loner is easy. Two loners is the question that separates people, and it needs a second idea: split the list into two piles on a bit where the two answers differ, then run the easy version on each pile.

By the end of this lesson you can solve the whole family — one loner, one missing, two loners, everything three times, one doubled and one missing — plus the XOR of any range in constant time, prefix XOR for subarray questions, and the biggest XOR pair in a list.

2 The story

There were sixty-one children going to Mysore and Rajeshwari had done this trip eleven times.

The other teachers wrote lists. She had tried lists in her second year and given them up, because a list of who is sitting with whom goes out of date about four minutes after you write it. Children swap. They swap at the tea stop, they swap when somebody feels sick, and they swap for no reason anybody can explain.

So she had one rule instead, and she said it at the gate before anybody got on the bus.

“Find one person. Hold their hand. That is your partner for the whole day.”

At every stop — the tea stall past Channapatna, the zoo gate, the place with the fountain — she did the same thing, and it took under a minute.

“Everybody. Find your partner. Hands up.”

And then she did not count. **Counting sixty-one moving children is hopeless, and she had learnt that the hard way in her first year.**

She looked for the one child standing with nobody.

Sixty-one is an odd number, so there was always exactly one. That year it was a boy called Vishnu, who did not mind at all, and who had worked out by the second stop that his entire job for the day was to stand still and be easy to see.

If Vishnu was there, everybody was there. If Vishnu was not there, or if there was a second child standing loose beside him, something had happened.

At the fountain, that afternoon, there were two children standing alone.

The other teacher started shouting names off her list, one after another, and got to about eleven before she lost her place.

Rajeshwari did something else.

“Blue shirts, this side. Everybody not in blue, that side.”

Two smaller crowds. And in each one, she said the same sentence she had said all day.

“Find your partner. Hands up.”

One child left over on the blue side. One child left over on the other.

Two names, in about ninety seconds, and she had still not written a single thing down.

3 The idea in plain English

Holding hands is XOR. Two children who pair up become invisible to Rajeshwari — they cancel. The one left standing is the answer. **And she never had to know who paired with whom,** which is exactly why she needed no list.

The three facts

Everything in this lesson comes from three lines.

```
a ^ a = 0          a thing XORed with itself vanishes
a ^ 0 = a          zero changes nothing
order does not matter, and grouping does not matter

(5 ^ 3) ^ 9 = 15
5 ^ (3 ^ 9) = 15
9 ^ (3 ^ 5) = 15
```

The first is the pairing. The second is the empty room — start from nothing. The third is the reason you do not need a list: you can XOR the numbers in any order at all and get the same answer, so the pairs find each other without your help.

Say the third one out loud in an interview, because it is the fact that turns the trick into an argument.

One loner

```
[4, 1, 2, 1, 2]

0 ^ 4 = 4
4 ^ 1 = 5
5 ^ 2 = 7
7 ^ 1 = 6
6 ^ 2 = 4

→ 4
```

Look at the middle numbers: 5, 7, 6. They are meaningless. The partial answers are not partial answers — nothing is true until the end, when every pair has cancelled. That is uncomfortable and it is worth saying so, because it is why people do not trust this the first time they see it.

Rearrange the same list and you can see why it must work:

```
[4, 1, 2, 1, 2] → 4 ^ (1 ^ 1) ^ (2 ^ 2)
                → 4 ^ 0 ^ 0
                → 4
```

You are allowed to rearrange it because order does not matter. That is the whole proof, and it is one line.

One missing from 0 to n

Same idea, but now you have to supply the partners yourself.

```
given [3, 0, 1], the full set is 0, 1, 2, 3
```

```
XOR the numbers you HAVE      3 ^ 0 ^ 1
```

```
XOR the numbers you EXPECT    0 ^ 1 ^ 2 ^ 3
```

```
-----  
everything present appears TWICE and cancels  
what is absent appears ONCE
```

```
→ 2
```

Every value that is present gets XORed twice — once from the list, once from the range — so it pairs with itself and disappears. The missing value only appears on the “expected” side, so it survives.

This is worth preferring to the sum formula. $n(n+1)/2 - \text{sum}(\text{numbers})$ also works, and it overflows in languages with fixed-width integers. XOR cannot overflow, because it never carries. Say that.

Two loners, which is the real question

Rajeshwari’s blue shirts. XORing everything gives you $a \oplus b$ — the two answers mixed together — and you cannot pull them apart directly.

But you can split the list.

```
[1, 2, 1, 3, 2, 5]
```

```
XOR everything      → 6 = 0110      this is 3 ^ 5  
lowest set bit      → 2 = 0010
```

A set bit in $a \oplus b$ means the two answers DISAGREE in that place. So pick any one of them — the lowest is easiest, both & -both — and use it as the shirt colour.

```
has the 2s bit set:  [2, 3, 2]      → XOR → 3  
does not:           [1, 1, 5]      → XOR → 5
```

Every duplicate pair lands in the same pile, because two equal numbers agree in every place. And the two loners land in different piles, because they were chosen to disagree exactly there. So each pile is now the easy problem.

That is the whole of LeetCode 260, and it is the moment the XOR family stops being a party trick.

Everything three times except one

Pairing does not help — three of a thing is not zero. So you count instead, **place by place**.

```
[2, 2, 3, 2]          in binary: 10, 10, 11, 10

place 0 (the 1s):    0 + 0 + 1 + 0 = 1    → 1 % 3 = 1    → set it
place 1 (the 2s):    1 + 1 + 1 + 1 = 4    → 4 % 3 = 1    → set it

→ 11 = 3
```

Every number that appears three times contributes three to some columns and zero to others, so every column is a multiple of three, plus whatever the loner contributed. Take the remainder modulo three and you have the loner's bits back.

This generalises: appearing `k` times means counting modulo `k`. Say that — it shows you understand the mechanism rather than the special case.

The XOR of a whole range, in constant time

```
0^1      = 1
0^1^2    = 3
0^1^2^3  = 0
0^1^2^3^4 = 4
... ^5   = 1
... ^6   = 7
... ^7   = 0
```

THE PATTERN REPEATS EVERY FOUR:

```
n % 4 == 0 → n
n % 4 == 1 → 1
n % 4 == 2 → n + 1
n % 4 == 3 → 0
```

And any range is two of those: `low ^ ... ^ high` is `xor_to(high) ^ xor_to(low - 1)`, because everything below `low` appears in both and cancels. That is the same subtraction trick as prefix sums, wearing XOR instead of addition.

Prefix XOR

XOR has the property prefix sums need: it can be undone. `x ^ y ^ y` is `x`. So:

```
prefix[0] = 0
prefix[i] = a[0] ^ a[1] ^ ... ^ a[i-1]

XOR of a[l..r] = prefix[r+1] ^ prefix[l]
```

And the useful consequence: `prefix[i] = prefix[k]` means the whole stretch between them XORs to zero, which is how a family of “count the subarrays where...” problems collapses to counting equal prefix values.

The biggest XOR pair

Build the answer one bit at a time, starting from the top, and at each step ask whether the next bit can be a one.

want the highest bit set? Take every number's top-k bits.
If any two of those prefixes XOR to the target, yes.
Then move down one place and try again.

Thirty-two rounds, each a set lookup. That is LeetCode 421, and the same answer can be written with a trie — and if you say “the trie version stores the numbers bit by bit and walks the opposite branch at every step”, you have said the thing they wanted to hear.

4 The picture

Pairs cancelling, column by column:

```
[4, 1, 2, 1, 2]
```

```
  4   1   2   1   2
100 001 010 001 010
--- --- --- --- ---
```

place 0 (the 1s):	0 + 1 + 0 + 1 + 0	
	two 1s → cancel	→ 0
place 1 (the 2s):	0 + 0 + 1 + 0 + 1	
	two 1s → cancel	→ 0
place 2 (the 4s):	1 + 0 + 0 + 0 + 0	
	one 1 → survives	→ 1

answer 100 = 4

XOR is just "is the count of 1s in this column odd?"
asked independently in every column. Nothing carries,
nothing is remembered between columns.

The split, drawn — Rajeshwari's blue shirts:

[1, 2, 1, 3, 2, 5] the loners are 3 and 5

XOR of everything = 6 = 0110

^
|

pick ANY set bit. The lowest is easiest: both & -both = 0010

Now sort the list by "does it have the 2s bit set?"

2s bit SET

2 = 0010

3 = 0011

2 = 0010

XOR → 3

2s bit CLEAR

1 = 0001

1 = 0001

5 = 0101

XOR → 5

WHY EVERY PAIR STAYS TOGETHER:

two equal numbers agree in EVERY place, so they
always land in the same pile and cancel there.

WHY THE LONERS SPLIT:

the bit was chosen because they DISAGREE there.
That is what a 1 in $a \wedge b$ means.

Counting modulo three:

[2, 2, 3, 2]

	place 1	place 0
2 =	1	0
2 =	1	0
3 =	1	1
2 =	1	0

column	4	1
mod 3	1	1

answer = 1 1 = 11 = 3

The three 2s put a 1 in place 1 three times, and 3 mod 3
is 0 - they vanish. Everything left is the loner.

Appearing k times → count mod k. Twice is just $k = 2$,
and mod 2 is exactly what XOR already does.

The range pattern:

n	0	1	2	3	4	5	6	7	8	9	10	11
xor	0	1	3	0	4	1	7	0	8	1	11	0
	^			^	^			^	^			^
	n%4=0			n%4=3				every four,		back to 0		

$n\%4 = 0 \rightarrow n$ $n\%4 = 2 \rightarrow n+1$
 $n\%4 = 1 \rightarrow 1$ $n\%4 = 3 \rightarrow 0$

Four cases, no loop, and it is the kind of thing an interviewer asks as a follow-up to see whether you look for structure or reach for a loop.

5 The code, built step by step

One loner

```
def single_number(numbers: list[int]) → int:
    """Everything appears twice except one. Pairs cancel; the loner survives."""
    answer = 0
    for value in numbers:
        answer ^= value
    return answer
```

Three lines, and the only decision in them is starting at `0`. That is `a ^ 0 = a` — an empty pile XORs to zero, exactly as an empty pile sums to zero.

In an interview, write it and then immediately say the rearrangement argument: “order does not matter, so I can group the duplicates together, and each group is `x ^ x`, which is zero.”

One missing

```
def missing_number(numbers: list[int]) → int:
    """0..n with one missing. XOR the full range against what you have."""
    answer = len(numbers)
    for i, value in enumerate(numbers):
        answer ^= i ^ value
    return answer
```

Starting at `len(numbers)` is the whole subtlety. The loop covers positions `0` to `n-1` and the values, but the range goes up to `n` — so the top of the range has to be put in by hand. Forget it and you get a plausible wrong answer with no error.

Two loners

```
def single_number_three(numbers: list[int]) → tuple[int, int]:
    """Two loners. XOR everything, then split the list on one bit where they differ."""
    both = 0
    for value in numbers:
        both ^= value
    split = both & -both
    first = 0
    for value in numbers:
        if value & split:
            first ^= value
    return first, both ^ first
```

`split = both & -both` is yesterday's isolate-the-lowest-set-bit, and it is doing real work here: it picks one place where the two answers disagree.

And the last line is the nicest part. Having found one answer, you do not need a third pass — `both` is `a ^ b`, so `both ^ first` is `b`. XOR undoes itself.

Everything three times

```
def single_number_twice_over(numbers: list[int]) → int:
    """Everything appears three times except one. Count each place modulo three."""
    answer = 0
    for position in range(32):
        column = sum((value >> position) & 1 for value in numbers)
        if column % 3:
            answer |= 1 << position
    return answer - (1 << 32) if answer >> 31 else answer
```

Thirty-two columns, each counted independently, each taken modulo three. Slower than the clever two-variable state machine, and far easier to explain under pressure — which is the trade you want in an interview.

The last line is the Python tax. Python integers have no top bit, so a negative answer comes out as a huge positive number and you have to convert it back by hand. Say “the input can be negative, so I have to reinterpret the top bit as a sign” out loud — otherwise that line looks like superstition.

The range formula

```
def xor_to(n: int) → int:
    """0 ^ 1 ^ ... ^ n in O(1). The answer repeats with period four."""
    return (n, 1, n + 1, 0)[n % 4]

def xor_range(low: int, high: int) → int:
    """low ^ ... ^ high. Everything below `low` cancels itself out."""
    return xor_to(high) ^ xor_to(low - 1)
```

Derive the four cases at the desk rather than memorising them — write out the XOR of 0 to 7 and the pattern is obvious in about twenty seconds. `xor_range` is the prefix-sum subtraction with XOR in place of minus, and it works for the same reason: XOR is its own inverse.

The biggest XOR pair

```
def maximum_xor_pair(numbers: list[int], width: int = 32) → int:
    """Build the answer one bit at a time, highest first, and ask if it is reachable."""
    best = 0
    for position in range(width - 1, -1, -1):
        best <= 1
        wanted = best | 1
        prefixes = {value >> position for value in numbers}
        if any(wanted ^ prefix in prefixes for prefix in prefixes):
            best = wanted
    return best
```

Greedy from the top, and greedy is safe here for the reason greedy is ever safe: a higher bit outweighs every lower bit combined. `1000` is bigger than `0111`. So if the top bit can be a one, take it, and never reconsider.

`wanted ^ prefix in prefixes` is the trick inside the trick. You are asking “is there a partner that would give me this target?” — and `x ^ y = target` means `y = target ^ x`, so the question is a set lookup rather than a second loop.

The complete solution

```
"""Day 173 - the XOR family. Pairs cancel, and that one fact solves all of it."""

from __future__ import annotations

def single_number(numbers: list[int]) → int:
    """Everything appears twice except one. Pairs cancel; the loner survives."""
    answer = 0
    for value in numbers:
        answer ^= value
    return answer

def missing_number(numbers: list[int]) → int:
    """0..n with one missing. XOR the full range against what you have."""
    answer = len(numbers)
    for i, value in enumerate(numbers):
        answer ^= i ^ value
    return answer

def single_number_three(numbers: list[int]) → tuple[int, int]:
    """Two loners. XOR everything, then split the list on one bit where they differ."""
    both = 0
    for value in numbers:
        both ^= value
    split = both & -both
    first = 0
    for value in numbers:
        if value & split:
            first ^= value
    return first, both ^ first

def single_number_twice_over(numbers: list[int]) → int:
    """Everything appears three times except one. Count each place modulo three."""
    answer = 0
    for position in range(32):
        column = sum((value >> position) & 1 for value in numbers)
        if column % 3:
            answer |= 1 << position
    return answer - (1 << 32) if answer >> 31 else answer

def find_duplicate_and_missing(numbers: list[int]) → tuple[int, int]:
    """1..n with one number doubled and one absent. Same split trick as two loners."""
    both = 0
    for i, value in enumerate(numbers, start=1):
        both ^= i ^ value
    split = both & -both
    first = 0
    for value in numbers:
        if value & split:
            first ^= value
    for i in range(1, len(numbers) + 1):
        if i & split:
            first ^= i
    duplicate = first if numbers.count(first) == 2 else both ^ first
    return duplicate, both ^ duplicate

def xor_to(n: int) → int:
    """0 ^ 1 ^ ... ^ n in 0(1). The answer repeats with period four."""
    return (n, 1, n + 1, 0)[n % 4]

def xor_range(low: int, high: int) → int:
    """low ^ ... ^ high. Everything below `low` cancels itself out."""
    return xor_to(high) ^ xor_to(low - 1)
```

```

def count_equal_split_triplets(numbers: list[int]) → int:
    """Count  $i < j \leq k$  with  $a[i..j-1] = a[j..k]$ . That means  $prefix[i] = prefix[k+1]$ ."""
    prefix = [0]
    for value in numbers:
        prefix.append(prefix[-1] ^ value)
    total = 0
    for i in range(len(prefix)):
        for k in range(i + 1, len(prefix)):
            if prefix[i] == prefix[k]:
                total += k - i - 1
    return total

def maximum_xor_pair(numbers: list[int], width: int = 32) → int:
    """Build the answer one bit at a time, highest first, and ask if it is reachable."""
    best = 0
    for position in range(width - 1, -1, -1):
        best <= 1
        wanted = best | 1
        prefixes = {value >> position for value in numbers}
        if any(wanted ^ prefix in prefixes for prefix in prefixes):
            best = wanted
    return best

def gray_code(bits: int) → list[int]:
    """Every number  $0..2^bits-1$ , each differing from the last in exactly one place."""
    return [i ^ (i >> 1) for i in range(1 << bits)]

if __name__ == "__main__":
    print("THE THREE FACTS")
    print(f" 7 ^ 7      = {7 ^ 7}      a ^ a = 0")
    print(f" 7 ^ 0      = {7 ^ 0}      a ^ 0 = a")
    print(f" (5^3)^9    = {(5 ^ 3) ^ 9}    order does not matter")
    print(f" 5^(3^9)    = {5 ^ (3 ^ 9)}")
    print(f" 9^(3^5)    = {9 ^ (3 ^ 5)}")

    print()
    print("ONE LONER")
    print(f" single_number([4,1,2,1,2])      = {single_number([4, 1, 2, 1, 2])}")
    print(f" single_number([2,2,1])          = {single_number([2, 2, 1])}")
    print(f" single_number([7])              = {single_number([7])}")

    print()
    print("ONE MISSING FROM 0..n")
    print(f" missing_number([3,0,1])          = {missing_number([3, 0, 1])}")
    print(f" missing_number([0,1])           = {missing_number([0, 1])}")
    print(f" missing_number([9,6,4,2,3,5,7,0,1]) = {missing_number([9, 6, 4, 2, 3, 5, 7, 0, 1])}")

    print()
    print("TWO LONERS - the split bit does the work")
    values = [1, 2, 1, 3, 2, 5]
    both = 0
    for v in values:
        both ^= v
    print(f" list                {values}")
    print(f" XOR of everything    {both} = {both:04b} (this is 3 ^ 5)")
    print(f" lowest set bit      {both & -both} = {both & -both:04b} they differ HERE")
    print(f" single_number_three  {single_number_three(values)}")

    print()
    print("EVERYTHING THREE TIMES EXCEPT ONE")
    print(f" [2,2,3,2]           = {single_number_twice_over([2, 2, 3, 2])}")
    print(f" [0,1,0,1,0,1,99]    = {single_number_twice_over([0, 1, 0, 1, 0, 1, 99])}")
    print(f" [-2,-2,1,-2]        = {single_number_twice_over([-2, -2, 1, -2])}")

    print()
    print("ONE DOUBLED, ONE MISSING, IN 1..n")
    print(f" [1,2,2,4]           = {find_duplicate_and_missing([1, 2, 2, 4])} (dup, missing)")
    print(f" [1,1]                = {find_duplicate_and_missing([1, 1])}")

    print()

```

```

print("XOR OF A RANGE, IN CONSTANT TIME")
for n in range(0, 9):
    brute = 0
    for i in range(n + 1):
        brute ^= i
    print(f" 0..{n}>2} formula={xor_to(n):>3} brute={brute:>3} n%4={n % 4}")
print(f" xor_range(3, 9) = {xor_range(3, 9)}")

print()
print("PREFIX XOR - counting equal splits")
print(f" [2,3,1,6,7] → {count_equal_split_triplets([2, 3, 1, 6, 7])}")
print(f" [1,1,1,1,1] → {count_equal_split_triplets([1, 1, 1, 1, 1])}")

print()
print("BIGGEST XOR IN THE LIST")
print(f" [3,10,5,25,2,8] → {maximum_xor_pair([3, 10, 5, 25, 2, 8])} (5 ^ 25)")
print(f" 5 ^ 25 = {5 ^ 25}")

print()
print("GRAY CODE - one bit changes each step")
for i, value in enumerate(gray_code(3)):
    print(f" i={i} i>>1={i >> 1} i^(i>>1) = {value} = {value:03b}")

print()
print("VERIFICATION")
import random

bad = 0
for _ in range(2000):
    n = random.randint(1, 40)
    loner = random.randint(0, 10_000)
    pool = [random.randint(0, 10_000) for _ in range(n)]
    data = pool + pool + [loner]
    random.shuffle(data)
    if single_number(data) != loner:
        bad += 1

    size = random.randint(1, 60)
    gone = random.randint(0, size)
    if missing_number([v for v in range(size + 1) if v != gone]) != gone:
        bad += 1

    a, b = random.sample(range(0, 10_000), 2)
    pool = [random.randint(0, 10_000) for _ in range(n)]
    data = pool + pool + [a, b]
    random.shuffle(data)
    if sorted(single_number_three(data)) != sorted([a, b]):
        bad += 1

    triple = [random.randint(0, 10_000) for _ in range(n)]
    data = triple * 3 + [loner]
    random.shuffle(data)
    if single_number_twice_over(data) != loner:
        bad += 1

    top = random.randint(1, 500)
    brute = 0
    for v in range(top + 1):
        brute ^= v
    if xor_to(top) != brute:
        bad += 1

    sample = [random.randint(0, 255) for _ in range(random.randint(2, 12))]
    best = max(x ^ y for x in sample for y in sample)
    if maximum_xor_pair(sample, 8) != best:
        bad += 1
codes = gray_code(6)
if len(set(codes)) != 64 or any((codes[i] ^ codes[i + 1]).bit_count() != 1 for i in range(63)):
    bad += 1
print(f" {bad} mismatches over 2,000 random cases, 6 checks each, plus gray code")

```

Running it:

THE THREE FACTS

$7 \wedge 7$	= 0	$a \wedge a = 0$
$7 \wedge 0$	= 7	$a \wedge 0 = a$
$(5 \wedge 3) \wedge 9$	= 15	order does not matter
$5 \wedge (3 \wedge 9)$	= 15	
$9 \wedge (3 \wedge 5)$	= 15	

ONE LONER

<code>single_number([4,1,2,1,2])</code>	= 4
<code>single_number([2,2,1])</code>	= 1
<code>single_number([7])</code>	= 7

ONE MISSING FROM 0..n

<code>missing_number([3,0,1])</code>	= 2
<code>missing_number([0,1])</code>	= 2
<code>missing_number([9,6,4,2,3,5,7,0,1])</code>	= 8

TWO LONERS - the split bit does the work

<code>list</code>	[1, 2, 1, 3, 2, 5]
XOR of everything	6 = 0110 (this is $3 \wedge 5$)
lowest set bit	2 = 0010 they differ HERE
<code>single_number_three</code>	(3, 5)

EVERYTHING THREE TIMES EXCEPT ONE

[2,2,3,2]	= 3
[0,1,0,1,0,1,99]	= 99
[-2,-2,1,-2]	= 1

ONE DOUBLED, ONE MISSING, IN 1..n

[1,2,2,4]	= (2, 3) (dup, missing)
[1,1]	= (1, 2)

XOR OF A RANGE, IN CONSTANT TIME

0.. 0	formula= 0	brute= 0	$n\%4=0$
0.. 1	formula= 1	brute= 1	$n\%4=1$
0.. 2	formula= 3	brute= 3	$n\%4=2$
0.. 3	formula= 0	brute= 0	$n\%4=3$
0.. 4	formula= 4	brute= 4	$n\%4=0$
0.. 5	formula= 1	brute= 1	$n\%4=1$
0.. 6	formula= 7	brute= 7	$n\%4=2$
0.. 7	formula= 0	brute= 0	$n\%4=3$
0.. 8	formula= 8	brute= 8	$n\%4=0$

`xor_range(3, 9) = 2`

PREFIX XOR - counting equal splits

[2,3,1,6,7]	→ 4
[1,1,1,1,1]	→ 10

BIGGEST XOR IN THE LIST

[3,10,5,25,2,8]	→ 28 ($5 \wedge 25$)
$5 \wedge 25$	= 28

GRAY CODE - one bit changes each step

```

i=0  i>>1=0  i^(i>>1) = 0 = 000
i=1  i>>1=0  i^(i>>1) = 1 = 001
i=2  i>>1=1  i^(i>>1) = 3 = 011
i=3  i>>1=1  i^(i>>1) = 2 = 010
i=4  i>>1=2  i^(i>>1) = 6 = 110
i=5  i>>1=2  i^(i>>1) = 7 = 111
i=6  i>>1=3  i^(i>>1) = 5 = 101
i=7  i>>1=3  i^(i>>1) = 4 = 100

```

VERIFICATION

0 mismatches over 2,000 random cases, 6 checks each, plus gray code

Look at `single_number([7])`: the answer is 7. A list of one is the base case and it comes out right for free, because you started at zero and $7 \oplus 0$ is 7.

Look at the two-loner trace. The XOR of everything is `0110`, which is $3 \oplus 5$ — the two answers mixed together and unusable on their own. The isolated lowest bit, `0010`, is the only extra thing needed, and it turns one unsolvable problem into two easy ones.

And look at the gray code column. `i` counts 0, 1, 2, 3, 4 — but $i \oplus (i \gg 1)$ gives 0, 1, 3, 2, 6, and every consecutive pair differs in exactly one place. One XOR, and an ordering that would otherwise take real work.

6 What it costs

One loner.

`single_number`: one XOR per element

```

n = 5          → 5 operations
n = 1,000,000 → 1,000,000 operations

```

→ $O(n)$ time, $O(1)$ extra space.

Compare with the two answers people reach for first:

```

SORT then scan:       $O(n \log n)$  time, and it MUTATES the input
FREQUENCY MAP:        $O(n)$  time but  $O(n)$  space -
                      1,000,000 entries at ~50 bytes each = 50 MB

```

→ the constraint " $O(1)$ space" is what rules out the map, and it is the interviewer telling you the answer.

Two loners — three passes, still linear.

pass 1: XOR everything	n operations
pass 2: XOR the matching pile (one comparison each)	n operations

	2n operations

→ $O(n)$ time, $O(1)$ space. The extra pass costs a constant factor of two and buys the entire problem.

Everything three times — count the loops out loud.

for position in range(32):	32 iterations
sum over every value	n operations each

	32 x n

$n = 1,000,000 \rightarrow 32,000,000$ operations

→ $O(32n)$, which is $O(n)$, but the constant 32 is REAL.

The two-variable state-machine version is a single pass:
n operations, so 32x fewer.

→ In an interview, write the column version and SAY the state machine exists. Correct and explainable beats clever and shaky.

The range formula.

xor_to(n): one modulo, one lookup

$n = 10 \rightarrow 1$ operation

$n = 1,000,000,000 \rightarrow 1$ operation

the loop it replaces:

$n = 1,000,000,000 \rightarrow 1,000,000,000$ operations

→ $O(1)$ against $O(n)$. This is the largest single saving in the lesson, and it comes from noticing a pattern rather than from a technique.

Prefix XOR.


```
build the table:      n operations, n+1 slots
answer one query:    1 XOR
```

1,000 queries on a 100,000-element list:

naive: $1,000 \times 100,000 = 100,000,000$

prefix: $100,000 + 1,000 = 101,000$

→ ~1,000x, for $O(n)$ extra space.

The biggest XOR pair.

```
for each of 32 bit positions:
    build a set of n prefixes      n operations
    check each prefix against it  n lookups
-----
                                32 x 2n
```

$n = 200,000 \rightarrow \sim 12,800,000$ operations fine

against the brute force:

every pair: $n(n-1)/2$

$n = 200,000 \rightarrow 20,000,000,000$ no

→ $O(32n)$ against $O(n^2)$. The trie version has the same cost and a nicer explanation.

Space, across the whole lesson.

```
single_number, missing_number,
single_number_three, xor_to:      0(1)
single_number_twice_over:         0(1)
maximum_xor_pair:                 0(n) for the set
prefix XOR:                       0(n) for the table
```

→ Every headline problem in this family is $O(1)$ space, and that is precisely why they are asked.

7 The traps

Assuming “appears twice” when the problem said something else.

```

single_number([2, 2, 3])      → 3      correct - two copies cancel
single_number([2, 2, 2, 2, 3]) → 3      correct - four copies cancel
single_number([4, 4, 4, 4, 7]) → 7      correct

single_number([2, 2, 2, 3])   → 1      WRONG, and it is a
                                perfectly plausible number
single_number([1, 1, 1, 3])   → 2      WRONG
single_number([5, 5, 5, 5, 5, 9]) → 12  WRONG

```

An even number of copies cancels; an odd number leaves one behind. So the plain XOR answer happens to be right whenever every repeat count is even, and quietly wrong otherwise. The wrong answers are small, ordinary-looking numbers — 1, 2, 12 — **not obvious failures**. It passes the hand-made test with two copies and fails the real input with three, **which is the worst shape a bug can have**. Read the repeat count out of the problem statement and say it out loud before you write a line.

Forgetting the top of the range in the missing-number version.

```

def missing_number_wrong(numbers: list[int]) → int:
    answer = 0                # should start at len(numbers)
    for i, value in enumerate(numbers):
        answer ^= i ^ value
    return answer

```

```

missing_number_wrong([3, 0, 1])      → 1      correct answer is 2
missing_number_wrong([0, 1])         → 0      correct answer is 2
missing_number_wrong([9,6,4,2,3,5,7,0,1]) → 1      correct answer is 8

```

Positions run 0 to n-1 and values run 0 to n, so the value `n` has no position to pair with. Seeding the accumulator with `len(numbers)` supplies it. Nothing errors; you just get a number from the right range that is not the answer.

The split bit when there is nothing to split.

```

single_number_three([1, 1, 2, 2])

```

```

both = 0
split = 0 & -0 = 0
first = 0                (nothing has bit 0 set)
returns (0, 0)

```

No error, and two answers that were never in the list. The precondition — exactly two loners — is doing real work, and if the input might not satisfy it you have to check `both != 0` yourself.

The negative-number tax in the modulo-three version.

without the final sign correction:

```
single_number_twice_over([-2, -2, 1, -2]) → 1           fine
single_number_twice_over([2, 2, -3, 2])   → 4294967293
                                           expected -3
```

Python integers are arbitrary width, so setting bit 31 does not make a number negative — it makes it a large positive one. The correction is `answer - (1 << 32)` when bit 31 is set, and it exists purely because of Python. In Java or C++ this line does not appear at all, and saying that is worth a mark.

Trying to recover both numbers from `a ^ b`.

```
a ^ b = 6

a = 3, b = 5 → 6
a = 1, b = 7 → 6
a = 0, b = 6 → 6
a = 2, b = 4 → 6
```

The XOR of two numbers does not determine them. People try to “unmix” it and there is nothing to unmix. You need the split, and the split needs the original list — that is why the two-loner solution has a second pass and cannot avoid one.

XOR is not addition, and the swap trick proves it.

```
5 + 3 = 8      5 ^ 3 = 6
```

XOR is “addition without carrying”. In a single column it is the same; across columns it is not. This matters when somebody offers you the sum-based missing-number formula: it is correct, and it overflows. XOR never overflows because it never carries — say that as the reason you prefer it.

Mixing types, which is how this fails at the keyboard.

```
answer = 0
answer ^= "3"
```

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
TypeError: unsupported operand type(s) for ^=: 'int' and 'str'
```

Reading numbers from input and forgetting to convert them is the single most common way this loop fails.

```
3 ^ 2.0
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for ^: 'int' and 'float'
```

Anything that has been through a `/` division is a float. Use `//`.

The range formula on a negative input.

```
xor_to(-1) → 0      no error at all
```

Python's `%` is always non-negative, so `-1 % 4` is `3` and you get the `n % 4 = 3` case. A silently plausible answer to a question that had no meaning. The formula is defined for `n ≥ 0`; guard it, or say so.

8 In the interview

How it gets asked

- “Every number appears twice except one. Find it, in linear time and constant space.” — LeetCode 136.
- “An array of 0 to n with one missing. Find it, without extra space.” — LeetCode 268.
- “Now two numbers appear once and the rest twice.” — LeetCode 260, and this is the real question.
- “Every number appears three times except one.” — LeetCode 137.
- “Find the maximum XOR of any two numbers in the array.” — LeetCode 421.
- “What is $1 \text{ XOR } 2 \text{ XOR } \dots \text{ XOR } n$?” — the constant-time follow-up.

The first ninety seconds

On “every number appears twice except one, $O(1)$ space”:

“The constraint tells me the answer. Constant extra space rules out a frequency map, and linear time rules out sorting. **That leaves XOR, and XOR is exactly right here.**

The property I need is that $a \oplus a$ is zero. A number XORed with itself vanishes. And $a \oplus 0$ is a , so starting from zero costs nothing.

So I XOR the whole list together and return the result.

The reason that works — and this is the part I want to say properly — is that XOR does not care about order or grouping. So I am allowed to imagine the list rearranged with every duplicate pair side by side. **Then it reads as $\text{loner} \oplus (x \oplus x) \oplus (y \oplus y) \oplus \dots$, every bracket is zero, and the loner is all that is left.**

I would point out one thing that makes people distrust this. The intermediate values are meaningless — on `[4,1,2,1,2]` you pass through 4, 5, 7, 6 and land on 4. **Nothing is true until the last element**, because the pairs have not all cancelled yet.

One pass, one integer, so $O(n)$ time and $O(1)$ space.

A useful way to see why it works at all: XOR is asking, independently in each bit position, ‘is the number of ones in this column odd?’ Duplicates contribute an even number to every column. Only the loner makes a column odd.“

The follow-ups

“Now two numbers appear once and everything else twice.”

“XORing everything still works, but it gives me $a \oplus b$ — the two answers mixed together — and there is nothing I can do with that directly. $a \oplus b = 6$ could be 3 and 5, or 1 and 7, or 2 and 4. The XOR does not determine the pair.

So I split the list into two piles and run the easy version on each.

The insight is what a set bit in $a \oplus b$ means: it means the two answers DISAGREE in that place. So I pick any set bit — the lowest, with $\text{both} \& -\text{both}$, which isolates it in one operation — and I use it to decide which pile each number goes into.

Two things make this work, and I would state both. Every duplicate pair lands in the same pile, because two equal numbers agree in every place, so they cancel wherever they land. And the two loners land in different piles, because I chose a place where they differ. So each pile is exactly the original problem: one loner, everything else in pairs.

Then a small nicety: I do not need a third pass. Once I have one answer, the other is $\text{both} \wedge \text{first}$, because both is $a \oplus b$ and XOR undoes itself.

Two passes, so still $O(n)$ time, and $O(1)$ space.

And I would name the assumption: this needs exactly two loners. If there were none, both would be zero, the split bit would be zero, and it would return a pair of zeros without complaining.

“Every number appears three times except one. Now what?”

“Pairing is no use — three of a thing does not cancel. So I stop thinking about the numbers and think about the bit positions instead.

For each of the thirty-two places, I count how many numbers have a one there. Every value that appears three times contributes either three or zero to that column, so the whole column is a multiple of three plus whatever the loner contributed. Take the count modulo three and you have the loner’s bit.

Do that thirty-two times and you have rebuilt the loner.

The general statement is nicer than the special case: appearing `k` times means counting modulo `k`. And the twice case is just `k = 2` — modulo two is precisely what XOR already does, which is why the first problem needed no counting at all.

Cost: thirty-two passes over the list, so $O(32n)$ — linear, but with a real constant. There is a single-pass version using two variables as a small state machine that tracks ‘seen once’ and ‘seen twice’, and it is thirty-two times faster. I would mention it and still write the column version, because I can explain the column version under pressure and the state machine is the kind of code that is easy to write subtly wrong.

One Python-specific point: if the input can be negative I have to fix the sign at the end, subtracting `1 << 32` when bit 31 is set, because Python integers have no top bit and would otherwise return four billion instead of minus three.“

“What is 1 XOR 2 XOR ... XOR n?”

“In constant time, and the way to get there is to write out the first eight and look.

0, 1, 3, 0, 4, 1, 7, 0. It resets to zero every fourth step, and the four cases are:

`n % 4 == 0` gives `n`; `1` gives `1`; `2` gives `n + 1`; `3` gives `0`.

I would derive that at the desk rather than claim to remember it — it takes about twenty seconds and it is much more convincing than reciting.

And an arbitrary range is two of those: `xor_to(high) ^ xor_to(low - 1)`, because everything below `low` appears in both terms and cancels. That is the prefix-sum subtraction with XOR instead of minus, and it works for the same reason — XOR is its own inverse.

That inverse property is worth one more sentence, because it is what makes prefix XOR a real technique. `prefix[r+1] ^ prefix[l]` is the XOR of a subarray, so a thousand range questions on a hundred-thousand element list go from a hundred million operations to about a hundred thousand. And `prefix[i] == prefix[k]` means the stretch between them XORs to zero, which is how a whole family of ‘count the subarrays where...’ problems collapses into counting equal prefix values.”

The model answer

“Talk me through the XOR family of problems and what connects them.”

“They are all one fact: $a \oplus a$ is zero, so things that come in pairs disappear.

The base case is one loner in a list of pairs. XOR everything; the pairs cancel; the loner survives. **Linear time, constant space, and the reason it works is that XOR ignores order — so I can imagine the duplicates grouped together and every group is zero.**

The second case is one number missing from zero to n . Now I have to supply the partners: XOR the list together with the full range. Everything present appears twice and cancels; the absent one appears once and survives. **The sum formula also works and can overflow. XOR cannot overflow, because it never carries.**

The third case is two loners, and this is the one that is actually a question. XORing everything gives $a \oplus b$, which does not determine either. **So I split.** A set bit in $a \oplus b$ marks a place where the two answers disagree, **so I isolate the lowest one with both & -both and use it to divide the list.** Equal numbers agree everywhere, so pairs stay together and still cancel. **The loners are guaranteed to separate.** Two easy problems, and the second answer falls out as `both ^ first`.

The fourth case breaks the pattern deliberately: everything three times. Pairing cannot help, so I count each bit position modulo three. The general rule is that k copies means counting modulo k , and XOR is just the $k = 2$ case done for free.

Then two things that are the same idea at a different scale. The XOR of a range is periodic with period four, so it is constant time rather than a loop. And prefix XOR gives subarray XOR in one operation, because XOR is its own inverse — the same trick as prefix sums.

What ties it together is a habit rather than a formula. Whenever a problem says ‘constant extra space’ and involves things that repeat, I ask what cancels. If the answer is ‘pairs’, it is XOR. If it is ‘triples’, it is counting modulo three. If nothing cancels, it is a different technique entirely and I stop trying.

The failure mode I watch for is assuming the repeat count. Plain XOR gives a right-looking answer whenever the repeats happen to be even, so a hand-made test with two copies passes and the real input with three copies fails. I read

the repeat count out of the problem statement and say it out loud before I write a line.“

9 Recall card

One fact runs the whole family: $a \oplus a = 0$, $a \oplus 0 = a$, and order does not matter. So XORing a list cancels everything that comes in pairs. The rearrangement argument IS the proof — group the duplicates, every bracket is zero. XOR is “is the count of ones in this column odd?”, asked independently per column, so nothing carries and nothing overflows.

One loner: XOR the list. One missing from $0..n$: XOR the list against the range (seed the accumulator with `len(numbers)`, or the top of the range has no partner). Prefer this to the sum formula, which overflows.

Two loners is the real question. XORing everything gives $a \oplus b$, which does not determine the pair — 6 could be 3^5 or 1^7 . A set bit in $a \oplus b$ means they DISAGREE there, so isolate one with `both & -both` and split the list on it: duplicates agree everywhere so pairs stay together; the loners are forced apart. Then `second = both ^ first` — no third pass. Needs exactly two loners, or it silently returns `(0, 0)`.

k copies means counting modulo k . Three times each: count every bit position mod 3, 32 passes, $O(32n)$. XOR is the $k = 2$ case for free. In Python, fix the sign at the end (`answer - (1 << 32)` when bit 31 is set) or minus three comes back as 4,294,967,293.

XOR is its own inverse, so it does prefix sums. `xor(l..r) = prefix[r+1] ^ prefix[l]`, and `prefix[i] = prefix[k]` means that stretch XORs to zero. The XOR of $0..n$ has period four: $n, 1, n+1, 0$ for $n\%4 = 0, 1, 2, 3$. Biggest XOR pair: build the answer from the top bit down and ask `wanted ^ prefix in prefixes` — $O(32n)$, not $O(n^2)$. And the trap that passes small tests: plain XOR looks correct whenever every repeat count happens to be even.

1 What this is, and why they ask it

An SLI is something you measure. An SLO is the target you hold yourself to. An SLA is the promise you make to a customer, with money attached. Three words, three different audiences, and candidates mix them up constantly.

The error budget is the idea underneath all three, and it is the one worth having. If your target is 99.9 percent, you are saying **0.1 percent of the time it may be broken** — and that 0.1 percent is a resource you are allowed to spend.

Which means the right number of outages is not zero. A month with no failures at all does not mean you were careful. It usually means you were too slow, too cautious, or spending money on capacity you did not need.

They ask it because “how available should this be?” is a question every design interview reaches, and because the honest answer involves arithmetic that most candidates cannot do on the spot. “Highly available” is not an answer. “Three nines, which is forty-three minutes a month, and here is what that rules out” is one.

And because it exposes whether you have ever shipped anything. The candidate who wants five nines for a photo-sharing app has not priced five nines. **Twenty-six seconds of downtime a month is a budget no human can respond inside** — it means every recovery must be automatic, which changes the whole design and multiplies the bill.

By the end of this lesson you can define all three terms without hesitating, convert any number of nines into minutes, compute an error budget in failed requests, work out what a chain of dependencies does to your availability, and explain why a target of 100 percent is a mistake rather than an ambition.

2 The story

The lorry had to be at the harbour by four in the morning, and that was the only line in the agreement anybody ever read out.

Mustafa had been making ice for the fish sellers at Malpe for nineteen years. The boats came in between two and half past three. **If the ice was there, the catch went into the boxes cold and went up to Bangalore. If it was not, the fish sat in the**

open, and by nine o'clock it was worth about half.

The agreement he signed every year had one sentence about time and one about money. **Ice by four. Late, and he took ten percent off that morning's bill.**

What took him about eleven years to work out was that the agreement was not his target.

His own target, which he never told anybody, was half past three.

The half hour was not politeness. It was for the things that happen. The lorry not starting on a cold morning. The boy who loaded it not turning up. The one Tuesday a year when the machine tripped and everything had to be shifted by hand.

And he had a second number in his head, which surprised his son when he finally explained it.

Two.

Two late mornings a month were acceptable. Not good. Acceptable. He had watched it for nineteen years, and the seller who shouted loudest about lateness still bought from him every week, **because two was inside what everyone had quietly decided was normal.**

His son wanted to make it none.

So Mustafa worked it through for him, out loud, standing by the shed.

"To never be late I need a second lorry. And a second driver for it. And that lorry stands there doing nothing three hundred and sixty-three days a year."

He asked the boy what a second lorry cost. Then he asked what two late mornings cost. **The boy did the two sums and stopped arguing.**

"The two late mornings are not a failure," Mustafa said. "The two are what I am spending in order to run this on one lorry."

And then the line his son repeated for years afterwards.

"And if I have not been late once all month, I have not been careful. I have been slow."

Because a month with nothing late meant he had been leaving at half past two every single morning, **and standing about at the harbour for an hour, paying a boy to stand about with him.**

3 The idea in plain English

Mustafa has all three of today's terms and he never named any of them.

"the time the lorry arrives"	← the SLI: the measurement
"half past three, my own target"	← the SLO: the internal goal
"by four, or ten percent off"	← the SLA: the contract, with money
"two late mornings a month"	← the ERROR BUDGET

SLI — the indicator

An SLI is a number you actually measure, expressed as a fraction of good events over valid events.

AVAILABILITY	successful requests / all valid requests
LATENCY	requests under 300 ms / all requests
FRESHNESS	records updated within 5 minutes / all records
CORRECTNESS	orders billed correctly / all orders

Writing it as good-over-valid rather than as a count is the discipline that matters, and it is the same point as yesterday's "errors as a fraction, never a count".

Two details decide whether the number means anything.

Where you measure. Measured at your load balancer, a request that never arrived because DNS was broken is invisible — **your graph says 100 percent while every user sees an error.** Measured in the client, you see the user's reality **and you also see their bad wifi**, which is not your fault and not something you can fix. **Say which one you chose and why.**

What counts as valid. A request that returns 400 because the client sent nonsense is not your failure. **Excluding it is correct; excluding too much is how a team makes its numbers look good and its users miserable.**

SLO — the objective

An SLO is the target for that indicator, over a stated window.

"99.9% of requests succeed, measured over a rolling 28 days"
"99% of requests complete in under 300 ms, over 28 days"

The window is part of the objective, not decoration. A daily window makes one bad hour a catastrophe; a quarterly window lets you hide a whole bad week. **Twenty-eight days is the common choice because it contains exactly four of every week-day**, so weekly traffic patterns do not distort it.

The SLO is internal. Nobody outside sees it. You are allowed to miss it — that is what makes it a useful management tool rather than a promise.

SLA — the agreement

An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always **service credits** — money back off the next bill — rather than damages.

The SLA is always looser than the SLO, and the gap is deliberate. Mustafa promised four o'clock and aimed for half past three. **If your promise and your target are the same number, then the first time you miss your target you are also paying a penalty**, and you have no room to detect a problem before it becomes a liability.

typical shape:

SLA to customers	99.9%	(43 min/month, with credits)
SLO internally	99.95%	(22 min/month, no money involved)
alerting fires at	99.97%	(13 min/month)

→ three layers, each tighter than the last, so a problem is noticed, then breaches an internal goal, and only then costs anybody money.

Real ones are worth knowing. AWS EC2 promises 99.99 percent for a region and gives credits of 10, 25 or 100 percent as the number falls. Google Cloud and Azure are shaped the same. And every one of them requires you to notice and to **file a claim** — nobody sends the money automatically, which tells you something about how these are really used.

The error budget, which is the useful idea

If the SLO is 99.9 percent, then 0.1 percent is yours to spend.

30 days = 43,200 minutes

99.9% available → 0.1% unavailable
→ $43,200 \times 0.001 = 43.2$ minutes a month

THAT IS THE BUDGET. Not an accident allowance - a resource.

And you spend it on things you want:

shipping a risky change
a migration with a chance of going wrong
an experiment
turning off a redundant machine to save money
a deliberate failure test

The rule the budget buys is simple and it settles arguments that otherwise never end. Budget remaining → ship it. Budget spent → stop shipping features and fix reliability until it recovers. The engineers and the product managers stop negotiating with each other and start reading the same number.

And Mustafa’s last line is the part people find genuinely counter-intuitive. An unspent budget is waste. If you finished the month at 99.99 percent against a 99.9 target, you were over-cautious or over-provisioned, and both cost money that bought nothing anybody asked for.

Why 100 percent is the wrong target

Three reasons, and all three are worth saying.

It is unachievable. The user’s phone, their network, DNS, the undersea cable — most of the path is not yours, and a user on a train sees failures you cannot prevent.

It is unaffordable. Each additional nine costs roughly ten times more than the last, and the gain is progressively less visible to anybody.

It is invisible. If the user’s own connection is 99 percent reliable, they cannot tell the difference between your 99.99 and your 99.999. You would be paying ten times as much for something no one can perceive.

What the nines actually mean

Learn this table. It is the single most quoted thing in this whole phase.

AVAILABILITY	PER MONTH (30 days)	PER YEAR
99%	7 hours 12 min	3.65 days
99.5%	3 hours 36 min	1.83 days
99.9%	43.2 minutes	8.76 hours
99.95%	21.6 minutes	4.38 hours
99.99%	4.32 minutes	52.6 minutes
99.999%	25.9 seconds	5.26 minutes

Look at the last row and think about what it means operationally. Twenty-six seconds a month is less time than it takes a person to read an alert and open a laptop. At five nines, no human is in the recovery path. Everything must fail over

automatically, which means the design changes: **health checks in seconds, automatic promotion of replicas, traffic that reroutes itself.** That is why the price is what it is.

Burn rate, so you find out early

The problem with a monthly budget is that you can burn all of it in twenty minutes and only notice at the end of the month. So you alert on how fast you are spending it.

```
burn rate = actual error rate / allowed error rate
```

```
SLO 99.9% → allowed error rate 0.1%
```

errors at 0.1%	→ burn rate 1	budget lasts exactly the month
errors at 1%	→ burn rate 10	budget gone in 3 days
errors at 10%	→ burn rate 100	budget gone in 7 hours

The standard shape is two alerts, and it is worth knowing because it is a genuinely good design.

```
FAST: burn rate 14.4x sustained for 1 hour  
→ 2% of the monthly budget in an hour. Page someone.
```

```
SLOW: burn rate 6x sustained for 6 hours  
→ 5% of the budget. A ticket, not a page.
```

The fast one catches the outage. The slow one catches the leak — the 0.6 percent error rate that nobody notices and that eats the whole month. **One alert cannot do both**, because a threshold tight enough to catch the leak fires constantly on ordinary noise.

4 The picture

Where the indicator is measured, and what it sees:

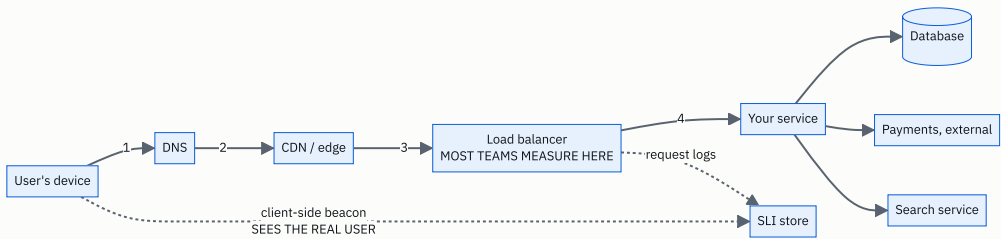


FIGURE 173.1

Measuring at the load balancer is easy and slightly dishonest. It cannot see requests that never arrived — a DNS failure or a dead edge shows as 100 percent availability while every user gets nothing. Measuring in the client sees the truth and also sees the user’s own broken wifi, which you cannot fix and should not be judged on.

The answer to give: measure server-side for the SLO you manage against, and keep a client-side signal to catch the failures the server-side view is blind to.

The budget, drawn as a month:

```

ERROR BUDGET, 99.9% SLO, 30-day month
Total budget: 43.2 minutes

day 1                                day 15                                day 30
|-----|-----|-----|-----|-----|
[=====] 43.2 min
^
a healthy month spends it gradually:

[#####.....] 9 min used
                                     34 spare
→ ship freely

a bad deploy on day 4:

[#####.....] 41 min used
^
one 38-minute outage
→ 1.2 minutes left for 26 days. FREEZE features.

a month with nothing spent:

[.....] 0 min used
→ NOT a triumph. You were too cautious, or you are
  paying for capacity nobody needed.
  
```

Dependencies multiply, and this is the diagram people find alarming:

Your service calls five things to serve one request.
Each one is independently 99.9% available.

```
you ---> auth      99.9%
        → database  99.9%
        → payments  99.9%
        → search    99.9%
        → storage   99.9%
```

IN SERIES, availability MULTIPLIES:

$0.999^5 = 0.995$

→ 99.5%, which is 3 hours 36 minutes a month.

You promised 43 minutes. You have built 216.

THE FIX IS NOT "ask them to try harder":

- make calls optional (search fails → hide the box)
- cache the last good answer
- put redundancy IN PARALLEL, where it helps:

two independent replicas, each 99%:

fails only if BOTH fail: $0.01 \times 0.01 = 0.0001$
→ 99.99%

IN SERIES you multiply availabilities and get WORSE.

IN PARALLEL you multiply failure rates and get BETTER.

That one sentence is the whole of reliability arithmetic.

5 How it actually works

Choosing the indicator

Start from the user, not from the machine. The question is “what does a bad minute feel like to somebody using this?” — **and the answer is almost never CPU.**

a checkout service	→ can they pay?	availability + correctness
a video player	→ does it start, and does it stutter?	startup latency + rebuffer ratio
a search box	→ does it answer, and is it fast?	availability + p95 latency
a data pipeline	→ is today's data there by 9 a.m.?	freshness

Three to five SLIs per service is the practical number. More than that and nobody remembers them, **and an SLO nobody remembers is not a control on anything.**

Measuring it

Two honest sources, and they answer different questions.

Request logs — count the good ones and the valid ones. **Cheap, accurate, and blind to anything that failed before it reached you.**

Synthetic probes — a robot somewhere else on the internet asking for your home page every thirty seconds. **It catches the DNS and edge failures the logs cannot see,** and it says nothing about real users, because it is not one.

Real-user monitoring — a small piece of code in the app that reports what actually happened to actual people. **The most honest signal and the most contaminated one,** because it includes every broken phone and every train tunnel.

In practice: SLO from server-side logs, synthetics as a safety net for total outages, and real-user data to argue about what the SLO should be.

Where the numbers live

Prometheus + Grafana	recording rules compute the ratio; Grafana's SLO panels or Sloth generate the burn-rate alerts for you
Nobl9 / Datadog SLOs	managed: define the SLI, get budget tracking and burn alerts
Google Cloud Monitoring OpenSLO	SLOs are first-class objects a YAML spec for defining SLOs portably, if you dislike lock-in

The mechanical part matters less than one design decision: the budget must be visible on the same dashboard as the feature work. A budget nobody looks at changes nobody's behaviour, and then the whole exercise is paperwork.

What happens when the budget runs out

This is the part that has to be agreed in advance, in writing, while everybody is calm.

```
BUDGET HEALTHY (> 25% left)
  ship normally, take risks, run experiments

BUDGET LOW (< 25%)
  no risky changes; deploys need a reviewer;
  reliability work is prioritised

BUDGET EXHAUSTED (0%)
  FEATURE FREEZE. Only reliability work and
  critical security fixes ship until the rolling
  window recovers.
```

The freeze is the entire point of the mechanism. Without a consequence, the SLO is a wish. With one, the decision to stop shipping is made by arithmetic rather than by whoever argues hardest in the meeting — and that is genuinely why teams adopt this.

And there has to be an escape hatch, agreed in advance: who can override the freeze, and what they have to write down when they do. A rule with no override gets ignored the first time it is inconvenient, and then it is dead.

The SLA side

The contract is a legal document and it is written defensively.

```
what it defines:
  the measurement, precisely (whose clock, which requests count)
  the exclusions (scheduled maintenance, force majeure,
    the customer's own misuse, beta features)
  the credit schedule
  the claim process and its deadline

what a credit schedule looks like (typical cloud shape):
99.99% or better    no credit
99.0% - 99.99%      10% of the monthly bill
95.0% - 99.0%        25%
below 95.0%          100%
```

Notice what the money actually is. Ten percent of one month's bill is not compensation for a business that lost a day's trading. The SLA is not insurance. It is a statement of seriousness, and it exists so that both sides have a number to point at.

And notice the exclusion for scheduled maintenance, which is how a provider gets to take a system down on purpose without breaching anything. If you are asked to design an SLA, that clause is the one to remember, because without it you have accidentally promised never to do planned work.

The nines, computed rather than recited.

a 365-day year = 525,600 minutes

$$= 5.26 \text{ min/year}$$

→ that single incident spent 46% of the monthly budget.

Burn rate, with the arithmetic shown.

allowed error rate at 99.9% = 0.001

observed 1% errors:

burn rate = $0.01 / 0.001 = 10x$

43.2 minutes of budget / 10 = the month's budget lasts

30 days / 10 = 3 DAYS

observed 10% errors:

burn rate = $100x$

30 days / 100 = 7.2 HOURS

observed 0.15% errors (barely visible on any graph):

burn rate = $1.5x$

30 days / 1.5 = 20 days

→ you run out on day 20 and nobody ever noticed a problem.

This is exactly what the slow-burn alert is for.

The standard two-alert configuration, and where its numbers come from.

FAST alert: 14.4x burn for 1 hour

1 hour is $1/720$ of a 30-day month

$14.4 \times (1/720) = 2\%$ of the budget consumed

→ page

SLOW alert: 6x burn for 6 hours

6 hours is $1/120$ of the month

$6 \times (1/120) = 5\%$ of the budget consumed

→ ticket

The 14.4 is not magic: it is the number that makes one hour equal 2% of a monthly budget.

Dependencies in series.

n dependencies, each 99.9%:

n = 1 $0.999^1 = 0.9990$ → 43 min/month

n = 3 $0.999^3 = 0.9970$ → 130 min/month

n = 5 $0.999^5 = 0.9950$ → 216 min/month

n = 10 $0.999^{10} = 0.9900$ → 432 min/month (7 hours)

n = 20 $0.999^{20} = 0.9802$ → 855 min/month (14 hours)

→ A twenty-service request path cannot be more than 98% available if every hop is three nines.

→ This is a real and often decisive argument in the microservices-versus-monolith discussion.

Redundancy in parallel, which is the counter-move.

two independent instances, each 99%:
both fail: $0.01 \times 0.01 = 0.0001$
available: 99.99%

three, each 99%:
 $0.01^3 = 0.000001 \rightarrow 99.9999\%$

BUT "independent" is doing enormous work in that sentence.
Two machines in the same rack share a power supply.
Two regions share a deployment pipeline and a DNS provider.

Real-world correlation is why measured availability is
always worse than the multiplication says.

The cost of a nine, roughly.

99%	one machine, restart it when it breaks
99.9%	two machines, a load balancer, on-call, monitoring
99.99%	multi-zone, automatic failover, replicated storage, tested runbooks, staged deploys
99.999%	multi-region active-active, no human in the recovery path, chaos testing, and a team that does only this

Each step is roughly 10x the engineering and infrastructure
cost of the one before.

Concretely, for a service costing \$10,000/month at 99.9%:
99.99% $\rightarrow$ ~\$40,000-100,000/month
99.999% $\rightarrow$ a dedicated team, so add salaries

$\rightarrow$ Ask what the downtime actually costs before buying a nine.
If an hour of downtime costs \$5,000, spending \$90,000 a
month to prevent 39 minutes of it is a bad trade.

7 The trade-offs

A tighter SLO buys confidence and costs velocity.

Every nine you add takes engineering time away from features and spends it on redundancy, testing and automation. That is the correct trade for a payments system and the wrong one for an internal dashboard. **I would not set a 99.99 percent SLO on anything whose users would not notice the difference** — and the way to find out is to look at what they actually complain about.

A long measurement window is forgiving and slow; a short one is honest and jumpy.

A 28-day rolling window absorbs a bad afternoon and tells you about a bad fortnight. A 24-hour window makes every incident a crisis and drives teams to hide small failures. A quarterly window lets a genuinely bad month be averaged away. I would use 28 days by default and add a 7-day view for the team's own awareness, without attaching consequences to the short one.

Measuring server-side is cheap and flatters you. Measuring client-side is honest and unfair.

Server-side logs cannot see what never arrived — and total outages are exactly the case where they go silent and your dashboard reads 100 percent. Client-side telemetry sees the real experience and includes every failure of the user's own network. I would set the SLO on server-side data, because that is what the team controls, and keep synthetic probes as the tripwire for the failures that make the logs go quiet.

Excluding requests makes the number meaningful, or makes it meaningless.

Excluding 4xx client errors is right — a malformed request is not your outage. Excluding “planned maintenance” can be right too. But every exclusion is a place where the number stops describing the user's experience, and there is a real failure mode where a team's SLO is green for a year while support tickets climb. The test I would apply: if the SLI is healthy and users are complaining, the SLI is wrong, and the exclusions are the first place to look.

The error budget policy only works if the freeze is real.

A budget with no enforcement is a dashboard. The whole value is that the decision to stop shipping is made by arithmetic instead of by argument. But a rigid freeze with no override gets ignored the first time something urgent appears, and after that nobody believes any of it. I would want the override written down in advance: who can grant it and what they have to record.

And the honest limit of the whole framework: it says nothing about how bad a failure was.

Three million failed requests spread thinly over a month is a mild annoyance. Three million in one twenty minute window, all on the checkout page, is a front-page incident. Same budget consumption, wildly different reality. I would not use error budgets as the only measure of health — they sit alongside incident reviews, not instead of them.

8 In the interview

How it gets asked

- *“What does three nines mean in minutes per month?”* — the direct one, and it is checked instantly.
- *“What availability would you target for this system, and why?”* — they want a number and a justification.
- *“What is the difference between an SLA and an SLO?”* — a definitions question, and a filter.
- *“You have a 99.9% target and you just had a 20-minute outage. Now what?”* — budget arithmetic.
- *“Your service calls six others. What is your availability?”* — the multiplication.
- *“Why not aim for 100%?”* — the question that finds out whether you have ever operated anything.

The first ninety seconds

On “what does three nines mean, in minutes”:

“Forty-three minutes a month.

The arithmetic is worth doing out loud: a thirty-day month is 43,200 minutes, and 0.1 percent of that is 43.2 minutes. Over a year it is 8.76 hours.

And the rest of the table, because the shape matters more than any single row. 99 percent is seven hours a month. 99.9 is forty-three minutes. 99.99 is four minutes. 99.999 is twenty-six seconds.

Twenty-six seconds is the row that changes the design rather than the budget. It is less time than a person needs to read a page and open a laptop, so at five nines no human is in the recovery path. Everything must fail over automatically. That is why each nine costs roughly ten times the last one.

I would also give the budget in requests, because it is more useful in an argument. At a hundred million requests a day, a month is three billion, so 99.9 percent permits three million failed requests. And a twenty-minute outage at about 1,150 requests a second is 1.4 million requests — nearly half the month’s budget in one incident.

The thing I would want to add is the framing. That 0.1 percent is not an accident allowance, it is a budget I am allowed to spend — on risky deploys, migrations, experiments, or running with less redundancy than the paranoid option. And if I end the month having spent none of it, that is not a triumph. It means I was too cautious or paying for capacity nobody needed.”

The follow-ups

“What is the difference between an SLA, an SLO and an SLI?”

“An SLI is the measurement. An SLO is the internal target for it. An SLA is the external promise, with money attached.

The SLI is a ratio of good events over valid events — successful requests over all valid requests, or requests under 300 milliseconds over all requests. **A ratio, never a count**, so that it means the same thing on a quiet Sunday and a busy Monday.

The SLO adds a target and a window: 99.9 percent over a rolling 28 days. The window is part of the objective — a daily window makes one bad hour a catastrophe and a quarterly one lets you hide a bad week. **Twenty-eight days is the usual choice because it contains four of each weekday**, so weekly patterns do not distort it.

The SLA is the contract, and the important structural fact is that it is always looser than the SLO. Typically the SLA is 99.9 and the SLO is 99.95, with alerting tighter still. If the promise and the target are the same number, then the moment you miss your target you are already paying penalties — you have left yourself no room to notice a problem before it becomes a liability.

The consequence in an SLA is service credits, not damages — the cloud shape is ten percent of the bill, then twenty-five, then a hundred as it gets worse. **And the credit is not compensation.** Ten percent of a monthly bill does not cover a customer’s lost trading day. **The SLA is a statement of seriousness rather than insurance**, and you usually have to file a claim to get anything at all.”

“Your service depends on six others. What is your availability?”

“Worse than any of them, and this is arithmetic rather than pessimism.

Calls in series multiply. If every dependency is 99.9 percent available and I need all six to answer, **0.999 to the sixth is 0.994 — about 99.4 percent, which is roughly four and a half hours a month.** I promised forty-three minutes and I have built two hundred and sixty.

So the first question I would ask is: do I actually need all six? Most of them are not really required. If search is down I can hide the search box. If recommendations are down I can show the popular list. **Every dependency I can make optional comes out of the multiplication entirely.**

The second lever is caching the last good answer, which turns a dependency’s outage into stale data rather than an error.

The third is redundancy, and the key point is that it has to be in parallel to help. In series you multiply availabilities and it gets worse. In parallel you multiply failure rates and it gets better. Two independent instances at 99 percent each fail together only 0.01 times 0.01 of the time — **99.99 percent.**

And I would flag the word ‘independent’, because it is doing enormous work. Two machines in one rack share a power supply. Two regions share a deployment pipeline and a DNS provider. **Correlated failure is why measured availability is always worse than the multiplication predicts, and the honest version of the answer includes that.**

One last point, because it is a real design argument. A twenty-hop request path cannot beat 98 percent if each hop is three nines. That is one of the strongest concrete arguments against splitting a system into very many services, and it is worth having ready for tomorrow’s kind of question.”

“You are at 99.9%, you just had a twenty-minute outage on the fourth of the month. What happens now?”

“First the arithmetic, because everything else follows from it. The monthly budget is 43.2 minutes. Twenty minutes is 46 percent of it, spent on day four.

So we have about 23 minutes left for 26 days, and the burn rate we can afford from here is less than half of normal.

What that triggers should already be written down, agreed while everyone was calm. My policy would be: above 25 percent budget remaining, ship normally. Below 25 percent, no risky changes and deploys need a second reviewer. At zero, feature freeze — only reliability work and critical security fixes.

We are at 54 percent remaining, so not frozen, but into the cautious band. Concretely: postpone the migration that was scheduled for next week, keep shipping small low-risk changes, and put whatever caused the outage at the top of the list.

The reason to have this written in advance is that it takes the decision away from whoever argues hardest on the day. The budget number decides, not the meeting. That is genuinely why teams adopt this — it ends a recurring fight between shipping and stability by making it arithmetic.

I would also want a burn-rate alert so that the next one is caught in an hour rather than at month end. The standard shape is two alerts: 14.4 times burn sustained over an hour, which is two percent of the monthly budget, and that pages someone; and 6 times over six hours, which is five percent, and that files a ticket. The fast one catches an outage; the slow one catches a leak — a 0.15 percent error rate that looks like nothing on a graph but eats the whole month by day twenty. One alert cannot do both jobs.

And the honest caveat I would add: the budget says how much, not how bad. Three million failures spread over a month is an annoyance; three million in twenty minutes, all on checkout, is a front-page incident. Same number, different reality — so the budget sits alongside incident reviews rather than replacing them.”

The model answer

“What availability should this system have, and how would you manage it?”

“I would start by refusing to answer in the abstract, and ask what a bad minute costs.

If this is checkout for a business doing a hundred thousand a day, an hour of downtime costs about four thousand, and paying for another nine is easy to justify. If it is an internal reporting dashboard, nobody notices an hour — and a 99.99 percent target would be spending real money on something no user can perceive.

Then I would name the indicator before the target, because the target is meaningless without it. For a checkout: successful payment attempts over valid payment attempts, measured at the service, over a rolling 28 days. Probably a latency SLI beside it — 99 percent of attempts completing in under two seconds — because a checkout that works and takes eleven seconds is a failure that no availability number captures.

Say the target is 99.9 percent. That is 43 minutes a month, or three million failed requests out of three billion. The SLA I would offer customers would be looser — 99.5 percent, with service credits — so that missing my internal target does not immediately cost money and I keep room to react.

The 0.1 percent is then an error budget I actively spend: on risky deploys, on migrations, on running one fewer replica than the paranoid option. And the policy attached to it is the part that makes it work. Above 25 percent remaining, ship freely. Below, no risky changes. At zero, feature freeze until the rolling window recovers — with a named person who can override it and a note saying why.

I would alert on burn rate, not on the budget being gone. 14.4 times over an hour pages; 6 times over six hours makes a ticket. Otherwise you find out at month end.

Two pieces of arithmetic I would put on the table unprompted.

First, dependencies multiply in series. Six dependencies at three nines each puts my ceiling at 99.4 percent, so I cannot promise three nines until I have made most of those calls optional or cached. Redundancy only helps in parallel — two independent instances at 99 percent give 99.99, because failure rates multiply rather than availabilities.

Second, each nine is roughly ten times the cost. 99.999 percent is 26 seconds a month, which is less time than a human needs to react, so it means no person in the recovery path at all — automatic failover, multi-

region, chaos testing, and a team whose whole job is that. **I would only propose it where downtime is measured in millions.**

And I would close with the sentence that gets the least agreement and is the most useful. A month where I spent none of the budget is not a good month. It means I was too cautious, or I was paying for capacity nobody needed, and both of those are money that bought nothing.“

9 Recall card

SLI is the MEASUREMENT (good events / valid events — a ratio, never a count). SLO is the INTERNAL TARGET plus a window (“99.9% over a rolling 28 days”). SLA is the EXTERNAL CONTRACT with money attached, and it is ALWAYS LOOSER than the SLO — same number means the first internal miss is already a penalty. Typical layering: SLA 99.9 / SLO 99.95 / alert at 99.97. Credits are a statement of seriousness, not insurance.

The table, from 43,200 minutes in a 30-day month. $99\% = 7h12m \cdot 99.9\% = 43.2 \text{ min} \cdot 99.99\% = 4.32 \text{ min} \cdot 99.999\% = 25.9 \text{ seconds}$. Twenty-six seconds is less time than a human needs to read an alert, so at five nines nothing human is in the recovery path — and each nine costs roughly $10\times$ the last.

The error budget is a RESOURCE, not an accident allowance. 99.9% of 3 billion monthly requests = 3,000,000 failures allowed; a 20-minute outage at 1,157 req/s spends 46% of the month in one incident. Spend it on risky deploys, migrations and experiments. Policy: $>25\%$ left ship freely; $<25\%$ no risky changes; 0% feature freeze — with a named override, or the rule dies the first time it is inconvenient. An unspent budget means you were too cautious or over-provisioned.

Alert on BURN RATE, or you find out at month end. burn = actual error rate / allowed. $1\% \text{ errors against a } 0.1\% \text{ allowance} = 10\times = \text{budget gone in 3 days}$. Standard pair: $14.4\times$ for 1 hour (2% of budget) pages; $6\times$ for 6 hours (5%) tickets — the fast one catches outages, the slow one catches the 0.15% leak that eats the month invisibly.

IN SERIES YOU MULTIPLY AVAILABILITIES AND GET WORSE; IN PARALLEL YOU MULTIPLY FAILURE RATES AND GET BETTER. Six dependencies at 99.9% $\rightarrow 0.999^6 = 99.4\%$, four and a half hours a month; twenty hops caps you at 98%. Fix by making calls optional, caching the last good answer, or adding independent parallel

replicas (two at 99% → 99.99%) — and “independent” is doing enormous work, because shared racks, pipelines and DNS correlate failures. **100% is the wrong target: unachievable, unaffordable, and invisible behind the user’s own connection.**

DSA topic: XOR problems **System design topic:** SLAs, SLOs, and error budgets

Code these, in this order

One rule for the whole set: **before you write the loop, say what cancels.** “Pairs cancel.” “Everything present appears twice.” **If nothing cancels, XOR is the wrong tool and you should stop looking for a clever line.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Single Number	LeetCode 136 (Easy)	Pairs cancel, and that the constraint named the technique.
2	Missing Number	LeetCode 268 (Easy)	Supplying the partners yourself, and why XOR beats the sum.
3	Single Number III	LeetCode 260 (Medium)	The split bit. This is the one that separates people.
4	Single Number II	LeetCode 137 (Medium)	Counting modulo three, and Python’s sign problem.
5	Set Mismatch	LeetCode 645 (Easy)	One doubled, one missing — the split trick again, in disguise.
6	Maximum XOR of Two Numbers in an Array	LeetCode 421 (Medium)	Greedy from the top bit, with a set lookup instead of a second loop.

On problem 1, distrust it first

Trace `[4, 1, 2, 1, 2]` by hand and **write down every intermediate value.** Notice that 5, 7 and 6 mean nothing. **Then say the rearrangement argument out loud** — that is what turns the trick into a proof.

On problem 2, do it three ways

XOR, the sum formula, and a set. **Say the space cost of each.** Then say what happens to the sum formula at `n = 100,000` in a language with 32-bit integers, and why XOR is immune.

On problem 3, find the split bit by hand

Take `[1, 2, 1, 3, 2, 5]`. XOR everything and write out the bit pattern. Then say what a set bit in that number means. Pick it, split the list into two piles on it by hand, and check that both copies of every duplicate landed in the same pile. Only then write the code.

On problem 3 again, break the precondition

Run your solution on `[1, 1, 2, 2]` — a list with no loners at all. Record what comes back. Say why, in terms of what `both` and `split` become.

On problem 4, meet Python's sign

Write the column-counting version without the final correction. Run it on `[2, 2, -3, 2]` and record the number. Say why it is 4,294,967,293 and not -3 , and what one line fixes it. Then say why that line does not exist in the Java version.

On problem 6, do the small case by hand

Take `[3, 10, 5, 25, 2, 8]`. Work out the answer by trying all fifteen pairs. Then work it out again the greedy way, top bit first, and check you reached the same 28. Say why greedy is safe here — the reason is one sentence about place value.

Then the three-facts drill

Say all three facts. Then use them to explain, in one sentence each, why the one-loner solution works and why `both ^ first` gives you the second answer without another pass.

Then the repeat-count drill

Run plain XOR on `[2,2,3]`, `[2,2,2,3]`, `[2,2,2,2,3]` and `[1,1,1,3]`. Record all four answers. Say which are right, which are wrong, and what rule decides it. Then say why this is a dangerous bug rather than an obvious one.

Then the range drill

Write out the XOR of `0..n` for `n` from 0 to 11. Find the period yourself. State the four cases. Then compute `xor_range(3, 9)` two ways and check they agree. Finally run `xor_to(-1)` and say why it returns something rather than complaining.

Then the nines drill

From 43,200 minutes, compute the monthly downtime for 99%, 99.9%, 99.99% and 99.999% without looking at the table. Then say what changes about the design at the last one, and why.

Then the budget drill

Take 100 million requests a day. **Compute the monthly budget in failed requests at 99.9 percent.** Then compute what a 20-minute outage costs, as a fraction of it. Then say what your policy does at that point.

Then the dependency drill

Six dependencies, each 99.9 percent, all required. **Compute your ceiling and convert it to minutes.** Then compute two independent replicas at 99 percent each. **Say which multiplication applies to which arrangement,** and what the word “independent” is hiding.

The facts drill

1. Give the three facts of XOR.
2. Say which one makes the list order irrelevant.
3. Say why starting the accumulator at zero is correct.
4. Say what XOR is asking, per column.

The family drill

1. One loner: the code and the proof, in two sentences.
2. One missing: what you XOR against, and the seeding detail.
3. Two loners: why the plain XOR is not enough, and what the split bit means.
4. Three times each: what replaces cancelling, and the general rule.
5. Say what all four have in common.

The split-bit drill

1. Say what a set bit in `a ^ b` tells you.
2. Give the expression that isolates one, and why that one.
3. Say why duplicates cannot be separated by the split.
4. Say why the loners must be separated by it.
5. Give the second answer without a third pass, and why that works.

The range and prefix drill

1. Give the four cases of `xor_to(n)`.
2. Say how you would rediscover them in twenty seconds.
3. Give `xor_range(low, high)` and why the lower part cancels.
4. Give the subarray XOR formula from a prefix table.

5. Say what `prefix[i] = prefix[k]` means about the stretch between them.

The terms drill

1. Define SLI, SLO and SLA in one sentence each.
2. Say which is looser and why the gap exists.
3. Say what the window is for, and why 28 days.
4. Say what an SLA's consequence actually is, and what it is not.
5. Name the exclusion clause that matters most, and why.

The nines drill

1. Give downtime per month at five availability levels.
2. Say which row changes the architecture rather than the budget, and why.
3. Give the rough cost multiplier per nine.
4. Give the three reasons 100 percent is the wrong target.

The budget drill

1. Define the error budget from an SLO.
2. Give it in minutes and in requests for a stated scale.
3. Name four things you spend it on.
4. Give the three-band policy and what happens in each.
5. Say why the policy needs a written override.
6. Say what an unspent budget means.

The burn-rate drill

1. Give the formula.
2. Give how long the budget lasts at 1 percent and 10 percent errors.
3. Give the two standard alerts with their windows.
4. Say where 14.4 comes from.
5. Say what each alert is for, and why one alert cannot do both.

The arithmetic drill

1. Say what happens to availability in series, and compute six at 99.9 percent.
2. Say what happens in parallel, and compute two at 99 percent.
3. Give the sentence that separates the two cases.
4. Say what "independent" is hiding, with two concrete examples.
5. Say what a twenty-hop path implies, and which design debate that feeds.

The break-it drill

For each, say what happens and whether anything reports it:

1. Plain XOR on a list where every value appears three times.
 2. `missing_number` without seeding the accumulator with `len(numbers)`.
 3. The two-loner solution on a list with no loners.
 4. The modulo-three solution on a negative input, in Python.
 5. `xor_to(-1)`.
 6. `answer ^= "3"`.
 7. An SLI measured only at the load balancer, during a DNS failure.
 8. An SLA and an SLO set to the same number.
 9. An error budget policy with no consequence attached.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Every number appears twice except one. Find it, in $O(1)$ space.* Why the constraint names the technique, the three facts, the rearrangement proof, the meaningless intermediate values, and the column view.
 2. *Now two numbers appear once.* Why `a ^ b` is not enough, what a set bit in it means, `both & -both`, why duplicates stay together and loners split, and `both ^ first` for the second answer.
 3. *What does three nines mean in minutes per month?* The arithmetic from 43,200 minutes, the whole table, the budget in requests, what changes at five nines, and why an unspent budget is waste.
-

Before you move on

- ☐ I can give the three facts of XOR without hesitating.
- ☐ I can prove the one-loner solution by rearrangement, in one sentence.
- ☐ I know the intermediate values are meaningless, and why that unsettles people.
- ☐ I can explain XOR as an odd-count question per column.
- ☐ I can solve missing-number and say why XOR beats the sum formula.
- ☐ I know why the accumulator starts at `len(numbers)`.

- [] I can say what a set bit in `a ^ b` means.
- [] I can isolate the split bit and say why that one is easiest.
- [] I can explain why duplicates stay together and loners separate.
- [] I can get the second answer without a third pass.
- [] I know what the two-loner code does when the precondition fails.
- [] I can count modulo three and state the general `k` rule.
- [] I know why Python needs a sign correction there, and Java does not.
- [] I can derive the four cases of the range XOR in twenty seconds.
- [] I can give the subarray XOR formula and what equal prefixes mean.
- [] I know why greedy is safe in the maximum-XOR problem.
- [] I know plain XOR silently succeeds when repeat counts happen to be even.
- [] I can define SLI, SLO and SLA in one sentence each.
- [] I know why the SLA must be looser than the SLO.
- [] I can give the downtime table from 43,200 minutes.
- [] I know what changes in the design at five nines, and why.
- [] I can compute an error budget in failed requests.
- [] I can price a 20-minute outage against a monthly budget.
- [] I can give the three-band budget policy and why it needs an override.
- [] I know why an unspent budget is waste.
- [] I can give the burn-rate formula and the two standard alerts.
- [] I know where 14.4 comes from.
- [] I know series multiplies availabilities and parallel multiplies failure rates.
- [] I can compute six dependencies at 99.9 percent and convert it to minutes.
- [] I know what “independent” is hiding, with examples.
- [] I can give the three reasons 100 percent is the wrong target.
- [] I answered all three questions above out loud.

Primes, GCD, and modular arithmetic

DATA STRUCTURES AND ALGORITHMS

Primes, GCD, and modular arithmetic

SYSTEM DESIGN

Deployments: blue-green, canary, and rollback

Primes, GCD, and modular arithmetic

1 What this is, and why they ask it

A prime is a number with exactly two divisors: one and itself. A GCD is the largest number that divides two others exactly. Modular arithmetic is arithmetic that wraps round, the way a clock does.

Those three ideas cover almost all the mathematics an interview will ask you for, and each of them has one technique attached that is far faster than the obvious approach.

The sieve, instead of testing each number. Euclid's algorithm, instead of listing divisors. Square and multiply, instead of multiplying a billion times. Each one turns something impossible into something instant, and each one is about six lines.

They ask it because these appear as sub-steps inside larger problems far more often than as questions of their own. A counting problem says "answer modulo $10^9 + 7$ " — that is this lesson. A problem about repeating patterns turns out to be about a GCD. A problem about arrangements needs a fast power.

And because they are a fair test of whether you reach for the obvious loop. "Find all primes below a million" has an answer that takes a second and an answer that takes twenty minutes, and the difference is a single change of viewpoint: stop asking about each number, and start crossing off.

By the end of this lesson you can test primality properly, sieve a million primes, factorise any number instantly after one preparation pass, compute a GCD and an LCM without overflowing, do arithmetic under a modulus including division, and say why $10^9 + 7$ is on every competitive problem you have ever read.

2 The story

There were about sixty names in Ashfaq's phone under a group called SUNDAY, and getting eleven of them onto the maidan by six in the morning was the hardest thing he did all week.

For two years he did it by asking. He would start on Thursday evening and go down the group, one call at a time. **“Are you free on Sunday?”** Sixty calls, of which perhaps forty were answered, of which about eleven said yes — **and by the time he got to the end it was Friday night and two of the eleven had changed their minds.**

What fixed it was a conversation at the tea stall with a man who ran the shifts at the mill.

Ashfaq was complaining, at some length. The man listened and then asked something that sounded almost rude.

“Why are you asking the people? Ask the reasons.”

And once he said it, it was obvious. **Almost everybody who could not come was unable for one of about five reasons, and each reason had one person who already knew the whole list.**

The mill ran a Sunday shift, and the supervisor could say who was on it. The eight or nine who drove for the travel company were all on one roster, and one man had it. The students had their tuition on Sunday mornings, all of them, at the same place. **Four or five of them were the sort whose in-laws visited, and their wives were all in one group that knew everything.**

So on Thursday, Ashfaq sent five messages instead of making sixty calls.

Each one came back with names. He took those names out of the group as they arrived. **The mill supervisor alone removed fourteen.**

What was left was nineteen names, and those he actually rang. Eleven yeses in about forty minutes, and he was finished before the shop shut.

The thing that stayed with him was what the man said afterwards, when Ashfaq told him it had worked.

“You were asking sixty questions to find eleven answers. Ask five questions that each rule out ten people, and you are done before the tea goes cold.”

3 The idea in plain English

Ashfaq’s five messages are a sieve. He did not examine each of the sixty. **He took whole groups of them off the list at once, using something that was already known.** That is the difference between testing every number for primality and crossing off multiples.

What a prime is

A prime has exactly two divisors: 1 and itself. 2, 3, 5, 7, 11, 13, 17.

Two things people get wrong and interviewers check. 1 is not prime — it has one divisor, not two. **2 is prime, and it is the only even prime**, which is why every prime routine has a special case for it.

Testing one number

The obvious test tries every divisor from 2 up to $n - 1$. For 1,000,003 that is a million divisions.

You only need to go up to the square root, and the reason is worth saying out loud.

If $n = a \times b$, then a and b cannot BOTH be bigger than $\sqrt{n}$ - because then $a \times b$ would be bigger than n .

So EVERY composite number has a divisor at or below its square root. If you find none there, there is none.

$n = 1,000,003$
testing to $n-1$ → 1,000,002 divisions
testing to $\sqrt{n}$ → 1,000 divisions
→ 1,000x fewer

And you can do better with almost no effort. After ruling out 2 and 3, **every prime is one either side of a multiple of six** — 5, 7, 11, 13, 17, 19. Anything else is divisible by 2 or by 3.

$6k$ divisible by 6
 $6k + 1$ ← can be prime
 $6k + 2$ divisible by 2
 $6k + 3$ divisible by 3
 $6k + 4$ divisible by 2
 $6k + 5$ ← can be prime (same as $6k - 1$)

→ step by 6 and test two candidates each time:
a third of the work again.

The sieve, which is the real idea

Ashfaq's move. To find every prime below a million, **do not ask a million questions. Cross off.**

```

Write down 2 to 30.
Take 2. Cross off 4, 6, 8, 10 ... everything left is
    not divisible by 2.
Take 3, the next survivor. Cross off 9, 15, 21, 27.
Take 5. Cross off 25.
Stop - 7 x 7 is 49, past the end.

```

```

WHAT SURVIVES IS THE ANSWER:
2 3 5 7 11 13 17 19 23 29

```

This is the Sieve of Eratosthenes, and it is about two thousand two hundred years old.

Two details make it fast, and both are asked about.

Start crossing off at $p \times p$, not at $2p$. Every multiple of 5 below 25 — 10, 15, 20 — was already crossed off by 2 and by 3. Starting at the square skips work you have already done.

Stop when $p \times p$ exceeds the limit. If a number below the limit has any factor at all, it has one at or below the square root, so by the time you have sieved up to the square root, everything composite is already gone.

Factorising, after one sieve

A small variation makes factorising instant. Instead of storing “is it prime”, store the smallest prime that divides each number.

```

spf[12] = 2      spf[35] = 5      spf[97] = 97

```

To factorise 360:

```

spf[360] = 2  → 360 / 2 = 180
spf[180] = 2  → 180 / 2 = 90
spf[90]  = 2  → 90 / 2 = 45
spf[45]  = 3  → 45 / 3 = 15
spf[15]  = 3  → 15 / 3 = 5
spf[5]   = 5  → 5 / 5 = 1

```

```

→ 360 = 2^3 x 3^2 x 5

```

Each step at least halves the number, so it takes at most $\log_2(n)$ steps — about twenty for a million. One sieve, then any number factorised in twenty operations.

The GCD, and Euclid

The greatest common divisor of 1071 and 462 is the biggest number that divides both.

The naive answer tries every number from the smaller one downwards. The good answer is one line, and it is about as old as the sieve.

Euclid's insight: the common divisors of `a` and `b` are exactly the common divisors of `b` and `a mod b`.

Here is why, and it is worth being able to say. If some number `d` divides both `a` and `b`, then it also divides whatever is left over when you take `b` out of `a` as many times as you can — because you removed a pile of things that `d` already divided. So the set of common divisors never changes, and each step makes the numbers smaller.

```
gcd(1071, 462) → 1071 mod 462 = 147
gcd( 462, 147) →  462 mod 147 = 21
gcd( 147,  21) →  147 mod  21 = 0
gcd(  21,   0) →   21
```

Three steps. The naive search would have tried 462 candidates.

And the LCM comes free: `a × b = gcd(a, b) × lcm(a, b)`, so `lcm = a / gcd × b`.

Divide first. `a * b // gcd` computes the product before dividing, and in any language with fixed-width integers that product overflows for large inputs. `a // gcd * b` never gets bigger than the answer.

Modular arithmetic

Arithmetic that wraps. On a clock, 10 o'clock plus 5 hours is 3 o'clock: `15 mod 12 = 3`.

Three rules survive the wrap, and one does not.

<code>(a + b) % m</code>	<code>=</code>	<code>((a % m) + (b % m)) % m</code>	yes
<code>(a - b) % m</code>	<code>=</code>	<code>((a % m) - (b % m)) % m</code>	yes*
<code>(a * b) % m</code>	<code>=</code>	<code>((a % m) * (b % m)) % m</code>	yes
<code>(a / b) % m</code>	<code>=</code>	<code>((a % m) / (b % m)) % m</code>	NO

Addition, subtraction and multiplication all pass through the modulus. Division does not, and that is the whole reason modular inverses exist.

The asterisk on subtraction is a language difference, not a maths one. In Python, `-7 % 10` is `3` — the result always has the sign of the modulus. In C, C++, Java and Go it is `-7`. So in those languages you write `((a - b) % m + m) % m`, and

forgetting it produces negative answers on a problem that promised non-negative ones.

Why $10^9 + 7$

Because counting problems produce enormous answers, and the setter wants a fixed-width answer.

Three properties, and it is worth knowing all three. It is prime, which is what makes division work via Fermat's theorem below. It is just over a billion, so it fits in a 32-bit signed integer. And two values under it multiply to just under 10^{18} , which still fits in a 64-bit integer — so `a * b % m` is safe in C++ and Java without any special handling. That last one is why it is exactly this number and not some other prime.

Fast power: square and multiply

Computing 3^{13} by multiplying thirteen times is fine. Computing $2^{1000000000}$ that way is not.

Use the binary expansion of the exponent.

$13 = 1101$ in binary $= 8 + 4 + 1$

$3^{13} = 3^8 \times 3^4 \times 3^1$

and you get 3^1 , 3^2 , 3^4 , 3^8 by squaring repeatedly:

$3^1 = 3$

$3^2 = 9$

$3^4 = 81$

$3^8 = 6561$

→ four squarings and two multiplications,
instead of thirteen multiplications.

For an exponent of a billion that is thirty squarings instead of a billion multiplications. The loop is “if the low bit is set, multiply it in; then square the base and shift the exponent right” — which is yesterday's bit walk doing real work.

Division under a modulus

You cannot divide, so you multiply by an inverse. The inverse of `a` is the number `x` with `a * x == 1 (mod m)` — the modular version of $1/a$.

Fermat's little theorem gives it in one line when the modulus is prime: `a(m-2) mod m` is the inverse of `a`. And you compute that with the fast power you just wrote.

```
inverse of 3 mod 1,000,000,007 = 3^1000000005 mod m
                                = 333,333,336
```

```
check: 3 x 333,333,336 mod m = 1      correct
```

```
so "10 / 2" under the modulus is
10 x inverse(2) mod m = 5
```

Two conditions, and both matter. The modulus must be prime for Fermat. And a must not be a multiple of the modulus, or no inverse exists at all.

If the modulus is not prime, use the extended Euclidean algorithm instead — it finds x and y with $ax + my = \gcd(a, m)$, and when the gcd is 1, that x is the inverse. This is the safe general answer, and it is worth naming even if you write the Fermat version.

4 The picture

The sieve, crossing off:

```
2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
-----
```

take 2, cross from $2 \times 2 = 4$, step 2:

```
2  3  X  5  X  7  X  9  X 11  X 13  X 15  X 17  X 19  X
```

take 3, cross from $3 \times 3 = 9$, step 3:

```
2  3  X  5  X  7  X  X  X 11  X 13  X  X  X 17  X 19  X
```

take 5? $5 \times 5 = 25$, past the end. STOP.

survivors: 2 3 5 7 11 13 17 19

TWO THINGS TO NOTICE

Starting at $p \times p$, not $2p$: when we reached 3, the numbers 6 and 3×2 were already gone - 2 killed them. Every multiple of p below $p \times p$ has a smaller prime factor, so it is already crossed off.

Stopping at sqrt: any composite below 20 has a factor at or below 4.47, so 2 and 3 between them have already killed every composite here.

Euclid, as squares cut off a rectangle:

`gcd(1071, 462)` - lay out a rectangle 1071 long and 462 high and cut off the biggest squares you can.

```
+-----+-----+-----+
|  462   |  462   | 147   |   1071 = 2 x 462 + 147
|         |         |         |
+-----+-----+-----+
```

←→

what is left is 147 high... no,
147 WIDE and 462 high. Turn it:

```
+-----+-----+-----+-----+
| 147 | 147 | 147 | 21 |   462 = 3 x 147 + 21
+-----+-----+-----+-----+
```

```
+-----+-----+-----+-----+
| 21 | 21 | 21 | 21 | 21 | 21 | 21 |   147 = 7 x 21 + 0
+-----+-----+-----+-----+
```

NOTHING LEFT OVER.

→ 21 is the largest square that tiles the original
rectangle exactly. That is the GCD.

Each step: replace the rectangle with the leftover strip.
Three steps to go from 1071 to 21.

Square and multiply, drawn:

3^{13} , and 13 = 1 1 0 1
 8 4 2 1

exponent	bit	base	result	
13	1	3	3	(multiply in)
6	0	9	3	(skip)
3	1	81	3 x 81 = 243	(multiply in)
1	1	6561	243 x 6561	(multiply in)
			= 1,594,323	

check: $3^{13} = 1,594,323$

The base column is repeated squaring: 3, 9, 81, 6561.
The bit column is just the binary of 13, read from the
bottom up.

→ exponent of 1,000,000,000 → 30 rows, not a billion.

The three-way relationship worth carrying:

```
a x b = gcd(a, b) x lcm(a, b)
```

```
1071 x 462 = 494,802
```

```
gcd = 21, lcm = 23,562
```

```
21 x 23,562 = 494,802                      correct
```

```
→ lcm(a, b) = a / gcd x b
```

DIVIDE FIRST. $a \times b$ overflows a 64-bit integer for a, b around 10^{10} ; $a / \text{gcd} \times b$ never exceeds the answer.

5 The code, built step by step

Testing one number

```
def is_prime(n: int) → bool:
    """Trial division, but only up to the square root, and only over 6k +/- 1."""
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    factor = 5
    while factor * factor ≤ n:
        if n % factor == 0 or n % (factor + 2) == 0:
            return False
        factor += 6
    return True
```

The first three lines are the whole edge-case story: 1 is not prime, 2 and 3 are, everything else even or divisible by three is not. Write those before the loop, always.

`factor * factor ≤ n` rather than `factor ≤ sqrt(n)` keeps you in integers.

One multiplication per iteration is cheaper than a square root, and there is no floating-point rounding to worry about. If you prefer the square root, use `math.isqrt(n)`, which is exact.

The sieve

```
def sieve(limit: int) → list[int]:
    """Every prime below `limit`. Do not test each number - cross off the multiples."""
    if limit < 3:
        return []
    prime = [True] * limit
    prime[0] = prime[1] = False
    p = 2
    while p * p < limit:
        if prime[p]:
            prime[p * p:limit:p] = [False] * len(range(p * p, limit, p))
            p += 1
    return [i for i, still in enumerate(prime) if still]
```

`prime[p*p:limit:p] = [False] * ...` is a slice assignment, and it is the reason this is fast in Python. A plain `for` loop over the multiples does the same work in interpreted code; the slice does it in C. On a million-element sieve that is roughly a fivefold difference.

`while p * p < limit` is the stopping rule from section 3, and `p * p` is the starting point. Both are the same fact seen from two ends.

Factorising with a prepared table

```
def smallest_prime_factors(limit: int) → list[int]:
    """spf[n] is the smallest prime dividing n. One sieve, then factorise anything fast."""
    spf = list(range(limit))
    p = 2
    while p * p < limit:
        if spf[p] == p:
            for multiple in range(p * p, limit, p):
                if spf[multiple] == multiple:
                    spf[multiple] = p
            p += 1
    return spf
```

`spf[p] == p` is how you ask “is `p` still prime?” — nothing has claimed it yet. And `if spf[multiple] == multiple` is what keeps the *smallest* factor: once something has written there, a later, larger prime must not overwrite it.

```
def factorise(n: int, spf: list[int]) → dict[int, int]:
    """Divide out the smallest prime factor, over and over. At most log2(n) steps."""
    factors: dict[int, int] = {}
    while n > 1:
        p = spf[n]
        factors[p] = factors.get(p, 0) + 1
        n /= p
    return factors
```

Each division at least halves `n`, because the smallest prime factor is at least 2. So the loop runs at most `log2(n)` times — twenty for a million — regardless of how ugly the number is.

Euclid, and the LCM

```
def gcd(a: int, b: int) → int:
    """Euclid: the common measure of (a, b) is the common measure of (b, a mod b)."""
    while b:
        a, b = b, a % b
    return a

def lcm(a: int, b: int) → int:
    """Divide FIRST, or a * b overflows in every fixed-width language."""
    return a // gcd(a, b) * b
```

Two lines, and the tuple assignment does the shuffle without a temporary. It works with the arguments in either order — if `a < b`, the first iteration simply swaps them, at the cost of one wasted step.

`gcd(a, 0)` is `a`, and that is correct rather than a special case: every number divides zero, so the greatest common divisor of `a` and nothing is `a` itself. `gcd(0, 0)` is `0`, which is the conventional answer.

Fast power

```
def power_mod(base: int, exponent: int, modulus: int) → int:
    """Square and multiply. 30 steps for an exponent of a billion, not a billion steps."""
    result = 1
    base %= modulus
    while exponent > 0:
        if exponent & 1:
            result = result * base % modulus
        base = base * base % modulus
        exponent >>= 1
    return result
```

Read it against the table in section 4 and it maps line for line. `exponent & 1` asks whether this power of the base is one of the ones you need. `base = base * base` climbs to the next power. `exponent >>= 1` moves to the next bit.

The `% modulus` after every multiplication is not optional. Without it the numbers grow to millions of digits — Python will not overflow, it will simply get slower and slower until it stops being useful.

The modular inverse

```
def inverse_mod(a: int, modulus: int = MOD) → int:
    """Fermat: when the modulus is prime,  $a^{(m-2)}$  is a's inverse. Division, at last."""
    return power_mod(a, modulus - 2, modulus)

def extended_gcd(a: int, b: int) → tuple[int, int, int]:
    """Returns (g, x, y) with  $a*x + b*y = g$ . The x is the modular inverse when g is 1."""
    if b == 0:
        return a, 1, 0
    g, x, y = extended_gcd(b, a % b)
    return g, y, x - (a // b) * y
```

One line, and it costs one fast power. In an interview, say the precondition out loud as you write it: “this needs the modulus to be prime, which $10^9 + 7$ is.”

The extended version is the general answer — it works for any modulus coprime with a , and it does not need a prime. The swap in the return line is where people get lost, so if you write it, write the base case first and trace one small example.

The complete solution

```
"""Day 174 - primes, GCD, and modular arithmetic. The maths interviews actually use."""

from __future__ import annotations

MOD = 1_000_000_007


def is_prime(n: int) → bool:
    """Trial division, but only up to the square root, and only over 6k +/- 1."""
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    factor = 5
    while factor * factor ≤ n:
        if n % factor == 0 or n % (factor + 2) == 0:
            return False
        factor += 6
    return True


def sieve(limit: int) → list[int]:
    """Every prime below `limit`. Do not test each number - cross off the multiples."""
    if limit < 3:
        return []
    prime = [True] * limit
    prime[0] = prime[1] = False
    p = 2
    while p * p < limit:
        if prime[p]:
            prime[p * p:limit:p] = [False] * len(range(p * p, limit, p))
            p += 1
    return [i for i, still in enumerate(prime) if still]


def smallest_prime_factors(limit: int) → list[int]:
    """spf[n] is the smallest prime dividing n. One sieve, then factorise anything fast."""
    spf = list(range(limit))
    p = 2
    while p * p < limit:
        if spf[p] == p:
            for multiple in range(p * p, limit, p):
                if spf[multiple] == multiple:
                    spf[multiple] = p
            p += 1
    return spf


def factorise(n: int, spf: list[int]) → dict[int, int]:
    """Divide out the smallest prime factor, over and over. At most log2(n) steps."""
    factors: dict[int, int] = {}
    while n > 1:
        p = spf[n]
        factors[p] = factors.get(p, 0) + 1
        n /= p
    return factors


def gcd(a: int, b: int) → int:
    """Euclid: the common measure of (a, b) is the common measure of (b, a mod b)."""
    while b:
        a, b = b, a % b
    return a
```

```

def lcm(a: int, b: int) → int:
    """Divide FIRST, or a * b overflows in every fixed-width language."""
    return a // gcd(a, b) * b

def extended_gcd(a: int, b: int) → tuple[int, int, int]:
    """Returns (g, x, y) with a*x + b*y = g. The x is the modular inverse when g is 1."""
    if b == 0:
        return a, 1, 0
    g, x, y = extended_gcd(b, a % b)
    return g, y, x - (a // b) * y

def power_mod(base: int, exponent: int, modulus: int) → int:
    """Square and multiply. 30 steps for an exponent of a billion, not a billion steps."""
    result = 1
    base %= modulus
    while exponent > 0:
        if exponent & 1:
            result = result * base % modulus
            base = base * base % modulus
            exponent >>= 1
    return result

def inverse_mod(a: int, modulus: int = MOD) → int:
    """Fermat: when the modulus is prime, a^(m-2) is a's inverse. Division, at last."""
    return power_mod(a, modulus - 2, modulus)

if __name__ == "__main__":
    print("IS IT PRIME - stop at the square root")
    for n in (1, 2, 17, 91, 97, 1_000_003):
        root = int(n ** 0.5)
        print(f" {n:>9} prime={str(is_prime(n)):<5} sqrt={root:>4}"
              f" trial divisions saved: {n} → ~{root // 3}")

    print()
    print("THE SIEVE - cross off multiples instead of testing numbers")
    print(f" primes below 50: {sieve(50)}")
    print(f" how many below 1,000,000: {len(sieve(1_000_000))},}")
    print(f" how many below 10,000,000: {len(sieve(10_000_000))},}")

    print()
    print("CROSSING OFF, STEP BY STEP, BELOW 30")
    limit = 30
    flags = [True] * limit
    flags[0] = flags[1] = False
    for p in (2, 3, 5):
        killed = [m for m in range(p * p, limit, p) if flags[m]]
        for m in killed:
            flags[m] = False
        print(f" start at {p}x{p}={p * p}, cross off every {p}: {killed}")
    print(f" what survives: {[i for i, f in enumerate(flags) if f]}")

    print()
    print("FACTORISING WITH A SIEVE")
    spf = smallest_prime_factors(1000)
    for n in (12, 97, 360, 999):
        print(f" {n:>4} = {factorise(n, spf)}")

    print()
    print("EUCLID - each step is one remainder")
    a, b = 1071, 462
    steps = 0
    x, y = a, b
    while y:
        print(f" gcd({x:>5}, {y:>4}) → {x} mod {y} = {x % y}")
        x, y = y, x % y
        steps += 1

```

```

print(f" gcd = {x}, in {steps} steps")
print(f" lcm(1071, 462) = {lcm(1071, 462)}")
print(f" gcd(0, 5) = {gcd(0, 5)} gcd(5, 0) = {gcd(5, 0)} gcd(0, 0) = {gcd(0, 0)}")

print()
print("FAST POWER - square and multiply")
print(f" 3^13 the long way = {3 ** 13}")
print(f" power_mod(3, 13, 1000000007) = {power_mod(3, 13, MOD)}")
print(f" 13 = 1101 in binary, so:")
print(f" 3^13 = 3^8 * 3^4 * 3^1 = 6561 * 81 * 3 = 1594323")
print(f" power_mod(2, 1000000000, MOD) = {power_mod(2, 1_000_000_000, MOD)}")
f" in {1_000_000_000.bit_length()} squarings")

print()
print("MODULAR ARITHMETIC - what survives and what does not")
big_a, big_b = 987_654_321_987, 123_456_789_123
print(f" (a + b) % m = (a%m + b%m) % m → "
      f"{(big_a + big_b) % MOD == (big_a % MOD + big_b % MOD) % MOD}")
print(f" (a * b) % m = (a%m * b%m) % m → "
      f"{(big_a * big_b) % MOD == (big_a % MOD * (big_b % MOD)) % MOD}")
print(f" (a - b) % m works in Python → {(3 - 10) % MOD}")
print(f" ... but in C/Java (3 - 10) % m is -7, so you add m and take % again")
print(f" (a / b) % m is NOT (a%m / b%m) → division needs an INVERSE")

print()
print("DIVISION UNDER A MODULUS, VIA FERMAT")
inv3 = inverse_mod(3)
print(f" inverse of 3 mod 1e9+7 = {inv3}")
print(f" check: 3 * inv3 % MOD = {3 * inv3 % MOD}")
print(f" 10 / 2 under the modulus = {10 * inverse_mod(2) % MOD}")
print(f" 1 / 7 under the modulus = {inverse_mod(7)}")
f" and 7 * that = {7 * inverse_mod(7) % MOD}")

print()
print("EXTENDED EUCLID gives the same inverse without a prime modulus")
g, x, y = extended_gcd(3, MOD)
print(f" 3*x + MOD*y = {g}, x = {x % MOD}")
print(f" matches Fermat: {x % MOD == inv3}")

print()
print("VERIFICATION")
import math
import random

bad = 0
primes_below = set(sieve(20_000))
for n in range(2, 20_000):
    if is_prime(n) != (n in primes_below):
        bad += 1
spf_big = smallest_prime_factors(50_000)
for _ in range(2000):
    n = random.randint(2, 49_999)
    product = 1
    for p, e in factorise(n, spf_big).items():
        product *= p ** e
    if product != n:
        bad += 1
    a = random.randint(0, 10 ** 12)
    b = random.randint(1, 10 ** 12)
    if gcd(a, b) != math.gcd(a, b):
        bad += 1
    if lcm(a, b) != math.lcm(a, b):
        bad += 1
    base = random.randint(1, 10 ** 9)
    exp = random.randint(0, 10 ** 6)
    if power_mod(base, exp, MOD) != pow(base, exp, MOD):
        bad += 1
    if base % MOD and base * inverse_mod(base) % MOD != 1:
        bad += 1
print(f" {bad} mismatches: 20,000 primality checks and 2,000 random cases, 5 checks each")

```

Running it:

IS IT PRIME - stop at the square root

```
1 prime=False sqrt= 1 trial divisions saved: 1 → ~0
2 prime=True sqrt= 1 trial divisions saved: 2 → ~0
17 prime=True sqrt= 4 trial divisions saved: 17 → ~1
91 prime=False sqrt= 9 trial divisions saved: 91 → ~3
97 prime=True sqrt= 9 trial divisions saved: 97 → ~3
1000003 prime=True sqrt=1000 trial divisions saved: 1000003 → ~333
```

THE SIEVE - cross off multiples instead of testing numbers

```
primes below 50: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
how many below 1,000,000: 78,498
how many below 10,000,000: 664,579
```

CROSSING OFF, STEP BY STEP, BELOW 30

```
start at 2x2=4, cross off every 2: [4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28]
start at 3x3=9, cross off every 3: [9, 15, 21, 27]
start at 5x5=25, cross off every 5: [25]
what survives: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

FACTORISING WITH A SIEVE

```
12 = {2: 2, 3: 1}
97 = {97: 1}
360 = {2: 3, 3: 2, 5: 1}
999 = {3: 3, 37: 1}
```

EUCLID - each step is one remainder

```
gcd( 1071, 462) → 1071 mod 462 = 147
gcd( 462, 147) → 462 mod 147 = 21
gcd( 147, 21) → 147 mod 21 = 0
gcd = 21, in 3 steps
lcm(1071, 462) = 23562
gcd(0, 5) = 5 gcd(5, 0) = 5 gcd(0, 0) = 0
```

FAST POWER - square and multiply

```
3^13 the long way = 1594323
power_mod(3, 13, 1000000007) = 1594323
13 = 1101 in binary, so:
3^13 = 3^8 * 3^4 * 3^1 = 6561 * 81 * 3 = 1594323
power_mod(2, 1000000000, MOD) = 140625001 in 30 squarings
```

MODULAR ARITHMETIC - what survives and what does not

```
(a + b) % m == (a%m + b%m) % m → True
(a * b) % m == (a%m * b%m) % m → True
(a - b) % m works in Python → 1000000000
... but in C/Java (3 - 10) % m is -7, so you add m and take % again
(a / b) % m is NOT (a%m / b%m) → division needs an INVERSE
```

DIVISION UNDER A MODULUS, VIA FERMAT

```
inverse of 3 mod 1e9+7 = 333333336
check: 3 * inv3 % MOD = 1
10 / 2 under the modulus = 5
1 / 7 under the modulus = 142857144 and 7 * that = 1
```

EXTENDED EUCLID gives the same inverse without a prime modulus

```
3*x + MOD*y = 1, x = 333333336
matches Fermat: True
```

VERIFICATION

```
0 mismatches: 20,000 primality checks and 2,000 random cases, 5 checks each
```

Look at the crossing-off trace. Two crossed off thirteen numbers, three crossed off four more, and five crossed off exactly one. Each prime does progressively less work, because the earlier ones have already killed most of its multiples — and that is the intuition behind the sieve’s surprising cost in section 6.

Look at the prime counts: 78,498 below a million and 664,579 below ten million. Ten times the range gives only 8.5 times as many primes. They thin out, slowly, and the density near n is about $1 / \ln n$.

And look at the modular subtraction line: $(3 - 10) \% \text{MOD}$ is 1,000,000,000 in Python. A large positive number, which is the mathematically correct representative. In C or Java the same expression gives -7 . Same maths, different language convention, and it is a real source of wrong answers.

6 What it costs

Testing one number.

```
naive: divide by 2, 3, 4, ..., n-1

n = 1,000,003 → 1,000,001 divisions

to the square root:

n = 1,000,003 → ~1,000 divisions
n = 1012      → ~1,000,000 divisions
n = 1018      → ~1,000,000,000 - too slow

only 6k +/- 1:

a third again: ~333 divisions for 1,000,003

→ O(sqrt(n)) time, O(1) space.
→ Fine up to about 1014. Beyond that you need
  Miller-Rabin, and it is enough to name it.
```

The sieve — and the counting here is the interesting part.


```

for p = 2: cross off limit/2 numbers
for p = 3: cross off limit/3
for p = 5: cross off limit/5
...

```

total = limit $\times$ (1/2 + 1/3 + 1/5 + 1/7 + 1/11 + ...)

The sum of the reciprocals of the primes below n grows like $\ln(\ln(n))$ - very slowly indeed.

→ $O(n \log \log n)$, which for any practical n is "a small constant times n ".

limit = 1,000,000	→ ~2.8 million operations	~0.1 s
limit = 10,000,000	→ ~30 million operations	~1-2 s
limit = 100,000,000	→ memory becomes the problem first	

COMPARE: testing each number separately
 1,000,000 numbers $\times$ ~333 divisions each
 = 333,000,000 operations, roughly 100x slower.

Space is what actually stops the sieve.

a Python list of booleans: ~8 bytes per entry (a pointer)

limit = 1,000,000	→ ~8 MB	fine
limit = 10,000,000	→ ~80 MB	fine
limit = 100,000,000	→ ~800 MB	painful
limit = 1,000,000,000	→ ~8 GB	no

a bytearray: 1 byte per entry
 limit = 100,000,000 → 100 MB workable

a bitset: 1 bit per entry
 limit = 1,000,000,000 → 125 MB workable

→ The time is fine long before the memory is.
 If asked for primes below 10^9 , say "segmented sieve":
 sieve in blocks that fit in cache, using the primes
 below $\sqrt{10^9} = 31,623$.

Factorising.

with the spf table, each step at least halves n :

$n = 1,000,000 \rightarrow$ at most 20 steps
 $n = 2^{20} \rightarrow$ exactly 20 steps (all 2s)
 $n = 999,983$ (prime) $\rightarrow$ 1 step

$\rightarrow O(\log n)$ per number, after an $O(n \log \log n)$ sieve.

without the table, trial division per number:
 $O(\sqrt{n})$ each $\rightarrow$ 1,000 steps for a million.

$\rightarrow$ 50x per query, so the table pays for itself
after about 3,000 queries.

Euclid.

$\text{gcd}(1071, 462)$: 3 steps
 $\text{gcd}(10^{18}, 10^{18} - 1)$: 2 steps

WORST CASE is consecutive Fibonacci numbers - each step reduces by the smallest possible amount:

$\text{gcd}(89, 55) \rightarrow 55, 34 \rightarrow 34, 21 \rightarrow 21, 13 \rightarrow 13, 8$
 $\rightarrow 8, 5 \rightarrow 5, 3 \rightarrow 3, 2 \rightarrow 2, 1 \rightarrow 1, 0$
9 steps for numbers under 100

$\rightarrow O(\log \min(a, b))$. Concretely, at most about 5 times
the number of decimal digits.

a, b around $10^{18} \rightarrow$ at most ~ 90 steps

Compare with "try every divisor downwards":
 10^{18} candidates. Not a competition.

Fast power.

one squaring per bit of the exponent:

exponent 13 $\rightarrow$ 4 steps
exponent 1,000 $\rightarrow$ 10 steps
exponent 1,000,000,000 $\rightarrow$ 30 steps
exponent 10^{18} $\rightarrow$ 60 steps

$\rightarrow O(\log \text{exponent})$ multiplications, each on numbers
below the modulus.

the naive loop for 10^9 : 1,000,000,000 multiplications
 $\rightarrow$ about 33 million times more work.

Modular inverse via Fermat is one fast power, so $O(\log m)$ — about thirty multiplications for $10^9 + 7$. Extended Euclid is $O(\log m)$ too and is usually a little faster, so the choice between them is about preconditions, not speed.

Space, across the lesson.

```
is_prime, gcd, lcm, power_mod, inverse_mod:  O(1)
sieve:                                       O(n)
smallest_prime_factors:                     O(n)
factorise:                                  O(log n) for the result
extended_gcd:                               O(log n) stack frames
```

7 The traps

Forgetting that 1 is not prime, and that 2 is.

```
without the n < 2 check: is_prime(1) → True    WRONG
without the n < 4 check: is_prime(2) → the loop
                                never runs
                                → True, by luck
                                is_prime(3) → True, by luck
```

The 1 case is a genuine wrong answer and it is in every test suite. The 2 and 3 cases happen to come out right here because the loop starts at 5 and never runs — but rely on luck and the version you write next week will not have it. Write the base cases first.

Starting the sieve at $2p$ instead of $p \times p$.

This is not wrong — it is just slower, and an interviewer will ask about it specifically. Every multiple of p below $p \times p$ has a prime factor smaller than p , so it has already been crossed off. Starting at $p \times p$ skips that duplicated work.

Sieving the wrong size.

```
sieve(1000)      # primes below 1000, so 997 is the last one
sieve(1001)      # if you want 1000 itself considered
```

Off-by-one on whether the limit is included is the commonest bug in this function, and it produces an answer that is right except at one end. Decide out loud which convention you are using and put it in the docstring.

Building a sieve that does not fit.

```
prime = [True] * (10 ** 11)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
MemoryError
```

No message, no advice, just `MemoryError`. The time was never the problem — the memory was. If the limit is large, say “segmented sieve” and explain that you sieve a block at a time using the primes below the square root.

`lcm` written the obvious way.

```
lcm = a * b // gcd(a, b)      overflows  
lcm = a // gcd(a, b) * b     safe
```

In Python both give the right answer, because integers are arbitrary width. In C++, Java, Go or Rust, `a * b` overflows silently for `a, b` around `3 × 109`, and you get a plausible wrong number. Say “divide first” as you write it, because that is a sentence the interviewer is listening for.

Negative results from `%` in other languages.

```
Python:      (3 - 10) % 7 → 3  
C, Java, Go: (3 - 10) % 7 → -7 % 7 → 0 ... and  
              (2 - 10) % 7 → -8 % 7 → -1  
→ the fix in those languages: ((a - b) % m + m) % m
```

Python’s `%` always returns something with the modulus’s sign, so it is non-negative for a positive modulus. Most other languages take the sign of the dividend. A negative answer on a problem that asked for a value in `[0, m)` is this, essentially every time.

Forgetting the modulus inside the fast-power loop.

```
base = base * base          # missing % modulus
```

In Python this does not crash. It gets catastrophically slow. `2^(2^30)` has about 323 million digits, and each squaring doubles that. The program does not fail — it stops finishing, which is a much harder thing to debug than an exception.

Using Fermat’s inverse when the modulus is not prime.

```
inverse of 3 mod 10:
  the true answer is 7, because 3 x 7 = 21 = 1 mod 10
  Fermat gives 3^8 mod 10 = 1
  check: 3 x 1 mod 10 = 3, not 1          WRONG

inverse of 5 mod 12:
  Fermat gives 5^10 mod 12 = 1
  check: 5 x 1 mod 12 = 5, not 1        WRONG
```

No error. A small, plausible number that is simply not the inverse. Fermat needs a prime modulus, and `10^9 + 7` is prime, which is exactly why problems use it. If the modulus is composite, use extended Euclid — and if `a` shares a factor with the modulus, there is no inverse at all.

Dividing by zero, under any modulus.

```
5 % 0
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer modulo by zero
```

Loud, immediate, and the good kind of failure.

Using a float step or a float bound.

```
list(range(2, 10, 0.5))
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'float' object cannot be interpreted as an integer
```

This is what happens when a square root gets into a loop bound. Use `factor * factor ≤ n`, or `math.isqrt(n)`, and keep floats out of number theory entirely.

8 In the interview

How it gets asked

- “Find all the primes below one million.” — the sieve, and they will ask why you start at `p × p`.
- “Is this number prime?” — trial division to the square root, and the reason for the square root.

- “Find the GCD of two numbers.” — Euclid, and why each step preserves the answer.
- “Compute $a^b \bmod m$ for very large b .” — square and multiply.
- “The answer may be large, so return it modulo $10^9 + 7$.” — everything in section 3.
- “How would you compute nCr modulo a prime?” — inverses, and tomorrow’s lesson.

The first ninety seconds

On “find all the primes below one million”:

“I would sieve, and the reason is a change of viewpoint: instead of asking a million questions, I cross off.

Concretely, the Sieve of Eratosthenes. Start with every number from 2 to a million marked as possibly prime. Take 2 — cross off every multiple of 2. Take 3, the next survivor — cross off every multiple of 3. Keep going. Whatever survives is prime.

Two details make it fast, and I would mention both because they are what gets asked.

I start crossing off at $p \times p$, not at $2p$. Every multiple of p below $p \times p$ has a prime factor smaller than p , so it has already been crossed off by that smaller prime. For 5, the numbers 10, 15 and 20 were killed by 2 and 3 already.

And I stop when $p \times p$ exceeds the limit. Any composite number below the limit has a factor at or below its square root, so once I have sieved past the square root, everything composite is gone.

The cost is the interesting part. For each prime I cross off n/p numbers, so the total is n times the sum of the reciprocals of the primes — and that sum grows like $\ln \ln n$, which is tiny. So it is $O(n \log \log n)$, which for practical purposes is a small constant times n . About 2.8 million operations for a million, well under a second.

Testing each number individually would be about a thousand divisions each, so 333 million operations — roughly a hundred times slower.

The thing that actually limits it is memory, not time. A Python list of a hundred million booleans is about 800 megabytes. A `bytearray` is one byte each, and a bitset is one bit. And if you asked for primes below a billion, I would use a segmented sieve — sieve in blocks that fit in cache, using the primes below 31,623.

There are 78,498 primes below a million, if you want the answer as a check.”

The follow-ups

“Compute the GCD, and tell me why your method works.”

“Euclid’s algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, until the second one is zero.

The reason is one sentence and I would want to say it properly. If some number divides both a and b , then it also divides what is left over after you take b out of a as many times as possible — because you removed a pile of things it already divided. So the set of common divisors is exactly the same before and after the step, and each step makes the numbers strictly smaller. It has to end, and it ends at the answer.

There is a picture that makes it obvious. Lay out a rectangle 1071 by 462 and cut off the biggest squares you can. You get two 462-squares and a strip 147 wide. Do the same to the strip: three 147-squares and a 21-strip. Then seven 21-squares and nothing left. 21 is the largest square that tiles the original exactly — that is the GCD.

The cost is $O(\log \min(a, b))$ — at most about five times the number of decimal digits. The worst case is consecutive Fibonacci numbers, because each step then removes as little as possible. For numbers around 10^{18} that is under about ninety steps, against 10^{18} candidates for the naive search.

And the LCM comes free, because $a \times b = \text{gcd} \times \text{lcm}$. So $\text{lcm} = a / \text{gcd} \times b$ — and I divide first. In Python it makes no difference, but in C++ or Java $a * b$ overflows a 64-bit integer for values around three billion, and dividing first never produces anything larger than the answer.

One edge case: $\text{gcd}(a, 0)$ is a , and that is correct rather than a special case — every number divides zero.”

“Compute $a^b \bmod m$ where b can be a billion.”

“Square and multiply, using the binary expansion of the exponent.

The idea: 13 is 1101 in binary, which is $8 + 4 + 1$, so $3^{13} = 3^8 \times 3^4 \times 3^1$. And I get $3^1, 3^2, 3^4, 3^8$ by repeatedly squaring — four squarings rather than thirteen multiplications.

The loop is: if the low bit of the exponent is set, multiply the current base into the result; then square the base and shift the exponent right. One iteration per bit.

For an exponent of a billion that is thirty iterations rather than a billion multiplications — about thirty-three million times less work.

The line that must not be forgotten is the `% m` after every multiplication. Without it the numbers grow without limit — $2^{(2^{30})}$ has hundreds of millions of digits — and in Python that does not overflow, it just stops finishing. A program that gets slower and slower is harder to debug than one that crashes, so that modulus is doing more work than it looks.

Python has `pow(base, exp, mod)` built in and it does exactly this, and in production I would use it. I would still write the loop, because the question is testing whether I know why it is thirty steps.”

“The answer may be large — return it modulo $10^9 + 7$. What does that change?”

“Three things pass through a modulus and one does not. Addition, subtraction and multiplication are all safe: I can take the modulus at every step and the answer is the same. Division is not.

So wherever the formula divides, I multiply by a modular inverse instead. The inverse of a is the x with $a \times x \equiv 1$, which is what $1/a$ means when you are wrapping round.

When the modulus is prime — and $10^9 + 7$ is prime — Fermat’s little theorem gives it in one line: $a^{(m-2)} \bmod m$, which is one fast power, about thirty multiplications. The precondition matters and I would say it out loud: prime modulus, and a not a multiple of it.

If the modulus is composite, Fermat quietly gives the wrong answer — the ‘inverse’ of $3 \bmod 10$ comes out as 1, and 3×1 is 3, not 1. No error, just a wrong number. The general answer is the extended Euclidean algorithm, which finds x and y with $ax + my = \gcd(a, m)$; when that gcd is 1, x is the inverse. And if a shares a factor with the modulus, there is no inverse at all.

Why this particular number, since I am usually asked: three reasons. It is prime, which makes Fermat work. It is just over a billion, so it fits in a signed 32-bit integer. And two values below it multiply to just under 10^{18} , which still fits in a signed 64-bit integer — so $a * b \% m$ is safe in C++ or Java with no special handling. That third reason is why it is exactly $10^9 + 7$ and not some other prime.

One language trap worth naming: in C, Java and Go, $\%$ takes the sign of the dividend, so a subtraction can produce a negative result on a problem that wanted something in $[0, m)$. The fix is $((a - b) \% m + m) \% m$. Python is the exception — its $\%$ is always non-negative for a positive modulus.“

The model answer

“What number theory do you actually need for interviews, and why?”

“Four things, and each one replaces an obvious loop with something logarithmic or near-linear.

Primality, and the square-root bound. To test one number I divide only up to its square root, because if $n = a \times b$ then they cannot both exceed the square root. For a million that is a thousand divisions instead of a million. Skipping even numbers and multiples of three cuts it by another third — every prime above 3 is one either side of a multiple of six.

The sieve, when I need many primes rather than one. Do not test each number; cross off the multiples. Start at $p \times p$ because everything smaller has already been struck out by a smaller prime, and stop at the square root of the limit. $O(n \log \log n)$, about a hundred times faster than testing each number, and memory rather than time is what limits it — a bitset or a segmented sieve if the range is huge.

Euclid, for the GCD. $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, because a common divisor of the two also divides the remainder, so the set of common divisors never changes. Logarithmic — worst case consecutive Fibonacci numbers. LCM from $a \times b = \text{gcd} \times \text{lcm}$, dividing first to avoid overflow.

Modular arithmetic, because half of all counting problems end with ‘modulo $10^9 + 7$ ’. Addition, subtraction and multiplication pass through the modulus; division does not. So division becomes multiplication by an inverse, and the inverse comes from Fermat when the modulus is prime — one fast power — or extended Euclid when it is not.

And the fast power itself, which is the piece all of that rests on. Square and multiply, one step per bit of the exponent — thirty steps for an exponent of a billion. It is really yesterday’s bit walk: read the exponent’s binary digits and decide whether to multiply this power in.

What I would want you to take from that is that they are not five separate facts. They are the same move each time: stop doing the thing once per unit, and find the structure that lets you do it once per group, or once per digit. Ask about the reasons instead of about each person, and sixty questions become five.

The two things I would be most careful about in a real answer are the preconditions. Fermat needs a prime modulus and silently lies otherwise. And `%` is negative for negative inputs in most languages, which quietly breaks

answers that were supposed to be non-negative. Both are wrong answers rather than crashes, and that is what makes them worth saying out loud.“

9 Recall card

Primality: test only to the SQUARE ROOT — if $n = a \times b$, they cannot both exceed $\sqrt{n}$, so 1,000 divisions instead of a million. **Then step by 6 and test $6k \pm 1$** , a third again. 1 is not prime; 2 is, and is the only even prime — write the base cases first. Beyond $\sim 10^{14}$, name **Miller–Rabin**.

The SIEVE is the change of viewpoint: cross off multiples instead of testing numbers. Start at $p \times p$ (everything smaller already has a smaller prime factor) and stop when $p \times p > \text{limit}$. $O(n \log \log n)$ — $\sim 2.8\text{M}$ operations for a million, about $100 \times$ faster than testing each — and **memory limits it before time does** (8 MB / 80 MB / 800 MB in Python; use a bytearray, a bitset, or a **segmented sieve** for 10^9). **78,498 primes below a million.** A **smallest-prime-factor** sieve then factorises anything in $\leq \log_2 n$ steps.

Euclid: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, because a common divisor of both also divides the remainder — so the set of common divisors never changes. The picture is cutting the largest squares off a rectangle. $O(\log \min(a, b))$, worst case consecutive Fibonacci numbers; $\text{gcd}(a, 0) = a$. $\text{lcm} = a // \text{gcd} * b$ — **DIVIDE FIRST**, or $a * b$ overflows 64 bits around 3×10^9 .

Under a modulus: $+$, $-$ and $\times$ pass through; $\div$ does NOT. Division becomes multiplication by an inverse. Fermat, when the modulus is prime: $a^{(m-2)} \bmod m$ — one fast power. On a composite modulus it silently returns a wrong number (the “inverse” of 3 mod 10 comes out as 1); use **extended Euclid** there, and note there is no inverse at all when a shares a factor with m . In C/Java/Go $\%$ takes the dividend’s sign, so write $((a-b) \% m + m) \% m$; Python’s $\%$ is already non-negative.

Fast power: square and multiply, one step per BIT of the exponent — 30 steps for 10^9 , 60 for 10^{18} . Forgetting $\% m$ inside the loop does not crash, it just stops finishing. $10^9 + 7$ is used because it is prime (Fermat works), fits in a signed 32-bit integer, and two values under it multiply to under 10^{18} so $a * b \% m$ is safe in 64 bits.

1 What this is, and why they ask it

A deployment is the act of replacing running software with different software while people are using it. Nothing stops. The traffic keeps arriving. **And somewhere in the middle, two versions of your system are running at the same time.**

That last sentence is the whole topic. Every strategy below is an answer to “how much of the traffic sees the new version, for how long, and how fast can I undo it?”

And undoing it is the part that matters most. The single most valuable number in this lesson is how long it takes you to get back to the previous version — because the majority of production incidents are caused by a change, and the fastest cure for a bad change is not fixing it. **It is putting the old one back.**

They ask it because **it is where design meets operations**, and because it is the part of the job that candidates who have only built side projects have never had to think about. **On your laptop you stop the program and start it again. With live traffic you cannot.**

And because the follow-up is always the database, which is the one thing you cannot keep two copies of. **Anybody can say “blue-green”.** The interesting answer is what happens to the schema when the old code and the new code both have to read it.

By the end of this lesson you can describe five deployment strategies and say when each is right, explain what a readiness check does that a liveness check does not, migrate a database schema without downtime, size the extra capacity a blue-green switch needs, and say how long a one-percent canary must run before it means anything.

2 The story

Nagaraj had printed wedding cards for twenty-six years, and the thing he was known for was that he had never once had to reprint a whole order.

He had reprinted single cards. Everybody does. A name spelled the way the family said it and not the way they wanted it. **What he had never done was run off a thousand and then find that all thousand were wrong.**

His method was three things, and he had never written any of them down.

The first was that before a run he pulled one card. One. He inked the plate, ran a single sheet, and carried it out to the front where the light from the street was better, **and he would not print the second one until somebody from the family had read the first one out loud.**

The second was that he never changed two things in the same job. New ink and new card stock together — no. **If the gold came out looking flat, he wanted to know which of the two had done it,** and with both changed at once there was no way to know.

The third one the boys in the shop thought was superstition. He kept the old plate. Even after the new one was made and approved and already running. **The old plate stayed on the shelf behind him until the finished order was boxed and gone.**

His nephew asked him about that once. If the new plate was approved, why keep the old one taking up the shelf?

Nagaraj told him about a job in 1994.

A new plate, checked, approved, read out loud by the bride's father himself. **And a hairline crack in it that did not show at all until about the four hundredth impression,** when a thin white line began appearing through one corner of the border.

Four hundred cards were spoiled. But the old plate was on the shelf, and he was printing again in eleven minutes.

“Approved is not the same as proved,” he said. “Approved means it looked right on one card. The four hundredth card is a different question, and nobody can answer it in advance.”

Then the part his nephew remembered.

“I do not keep the old plate because I think the new one is bad. I keep it because eleven minutes is the difference between a mistake and a disaster.”

3 The idea in plain English

Nagaraj has the three ideas of this lesson and he has them in the right order. The single proof card is a canary. One change at a time is the rule that makes a failure diagnosable. And the old plate on the shelf is rollback — the thing that turns a bad release into an inconvenience.

The five strategies

Every one of them is a different answer to “who sees the new version, and for how long?”

Recreate. Stop the old, start the new. **Simple**, and it means **downtime** — as long as the new version takes to start. **It is the right answer for a batch job, an internal tool, or anything that genuinely cannot run two versions at once.**

Rolling update. Replace the machines a few at a time. **No extra capacity, no downtime**, and it is the **default in Kubernetes**. The cost is that both versions run side by side for the whole rollout, and going back is another slow rollout.

Blue-green. Run two complete environments. **Blue is live; green has the new version and no traffic.** Test green, then **switch all traffic at once**. Rollback is **switching back — seconds**. The cost is **double the infrastructure for the duration**, and the database is shared, which is where it gets hard.

Canary. Send a **small slice of real traffic** — one percent — to the new version. Watch its error rate and latency against the old one. Increase slowly: 1%, 5%, 25%, 50%, 100%. **This is the only strategy that finds problems that need real traffic to appear**, which is most of them. The cost is that it is slow, and it only works if your metrics are good enough to tell one percent apart from noise.

Feature flags. Ship the code with the new behaviour switched off, **then turn it on separately**. Deployment and release become two different events, which is the single most useful idea in the list. **You can enable it for internal staff first, then one percent of users, then everyone — with no deploy at all.**

The idea underneath all of them

During any of these, two versions of your code are running at the same time and talking to the same data.

So every change must work with the version before it. This is called **backward compatibility**, and in practice it means:

```
NEW code must work with OLD data
OLD code must not break on NEW data
NEW code must handle responses from OLD instances of other services
```

And that is why some changes cannot be made in one step at all, which is the next section.

Health checks, and the two kinds

A rolling update needs to know when a new instance is ready for traffic. That is a readiness check.

LIVENESS	"are you alive, or should I kill and restart you?" A failed liveness check RESTARTS the container.
READINESS	"should I send you traffic right now?" A failed readiness check REMOVES you from the load balancer but leaves you running.

The difference matters during a deployment. A service that takes forty seconds to warm its caches is **alive but not ready** — and if you only have a liveness check, the load balancer sends real traffic to it immediately and those users get errors. If you wire the liveness check to the same slow condition, the platform kills it before it ever finishes starting, and you get a restart loop.

A readiness check must test the things the service needs to serve, and must not test its downstream dependencies too aggressively — or one slow dependency takes your whole fleet out of the load balancer at once.

Rolling back, and rolling forward

Rollback: put the previous version back. Roll forward: fix the bug and deploy again.

During an incident, roll back. Rolling forward means writing code under pressure and shipping it untested, and it takes as long as the fix takes. Rollback takes as long as a deploy — or seconds, if you are blue-green.

The number worth having is: how long from “this is bad” to “the old version is serving”? A team that can answer “under five minutes” can afford to take risks. A team that answers “about an hour” cannot, and will compensate by deploying rarely, which makes each deployment bigger and more dangerous. That loop is real and it is worth naming in an interview.

The database, which is the hard part

You can have two versions of your code. You cannot have two versions of your data.

So schema changes are done in stages, and the pattern is called expand and contract.

Say you want to rename `name` to `full\_name`.

THE OBVIOUS WAY - and it breaks everything:

rename the column, deploy the new code
→ between those two moments, the old code is
querying a column that no longer exists

EXPAND AND CONTRACT - five deploys, no downtime:

1. EXPAND add `full\_name`, leave `name` alone.
Old code is unaffected.
2. DUAL WRITE deploy code that writes BOTH columns
and still reads `name`.
3. BACKFILL copy existing rows into `full\_name`,
in batches, in the background.
4. READ NEW deploy code that reads `full\_name`.
Still writing both, so rollback is safe.
5. CONTRACT once nothing reads `name`, stop writing
it, then drop it - weeks later.

Five steps, and the reason for each is that at no moment is any running version of the code looking at something that is not there.

The rule that generates all of it: every schema change must be additive first and destructive last, with a gap in between long enough that you could still roll back. Dropping a column is the last thing you do, and it should feel boringly overdue when you do it.

4 The picture

Blue-green, and the moment of the switch:

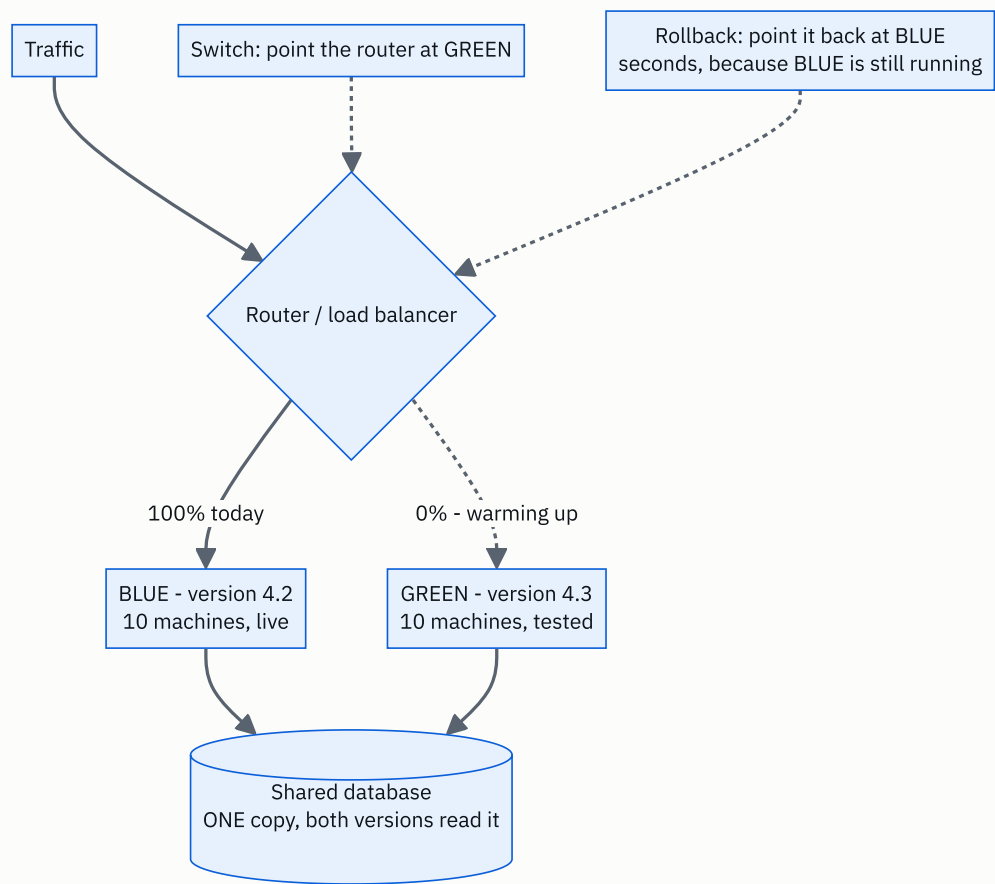


FIGURE 174.1

The important box is the shared database. Blue-green duplicates the compute and cannot duplicate the data, so the schema must work for both versions at once — which is exactly why expand-and-contract exists. And blue stays running after the switch, unused, for as long as you want the ability to go back in seconds.

Canary, as a sequence of decisions:

TIME →

```
00:00  1% to new      watch error rate, p99, saturation
      |
      |--- 30 min --- enough requests to say anything
      v
00:30  5% to new      watch again
      |
01:00  25% to new
      |
01:30  50% to new
      |
02:00  100%          old version stays running a while
```

AT ANY POINT: shift back to 0% and stop.
That is the whole value of the shape.

AUTOMATIC ABORT RULES, agreed in advance:

new version's error rate > 2x old version's	→ abort
new version's p99 > 1.5x old version's	→ abort
any burn-rate alert fires	→ abort

Compare the canary against the old version, not against a fixed threshold. If Monday morning is naturally noisier than Sunday night, a fixed threshold either fires constantly or never fires. The old version is serving the same traffic at the same moment, and it is the only fair control.

Rolling update, machine by machine:

10 machines, maxSurge 2, maxUnavailable 0

```
step 0  [0000000000]          10 old, serving
step 1  [0000000000] + [NN]    add 2 new, wait for READY
step 2  [00000000..] + [NN]    remove 2 old
step 3  [00000000..] + [NNNN]   add 2 more
...
step 10 [.....] + [NNNNNNNNNN] done
```

0 = old version N = new version

maxSurge 2 → you may temporarily run 12 machines
maxUnavailable 0 → you never drop below 10 serving

→ zero capacity loss, at the price of 20% extra
machines during the rollout.

AND NOTE: between step 1 and step 10, BOTH VERSIONS ARE
SERVING REAL USERS. Anything that is not compatible with
the previous version breaks here, not in testing.

Expand and contract, drawn as a timeline:

```
deploy:      1          2          3          4          5
             |          |          |          |          |
column       add        write      backfill  read        drop
`full_`      it         both       old rows  new          `name`
name`

old code     works      works      works      ---        ---
new code     ---        works      works      works      works
```

^ AT EVERY POINT, BOTH ROWS ARE SAFE.
That is the property the whole dance exists to preserve.

The gap between step 4 and step 5 is deliberate and long
- days or weeks - because until you drop the column you
can still roll back to any earlier version.

5 How it actually works

The platform pieces

Kubernetes does rolling updates natively. A `Deployment` has `maxSurge` and `maxUnavailable`, and it waits for each new pod's readiness probe before removing an old one. Rollback is one command — `kubectl rollout undo` — because the previous `ReplicaSet` definition is still stored.

Blue-green and canary need something that can split traffic.

Argo Rollouts	canary and blue-green as first-class Kubernetes objects, with analysis steps that query Prometheus and abort on their own
Flagger	the same idea, driven by a service mesh
Istio / Linkerd	weighted traffic splitting at the mesh layer
AWS CodeDeploy	blue/green for ECS and Lambda, with automatic rollback on a CloudWatch alarm
Spinnaker	multi-cloud pipelines, from Netflix
Nginx / Envoy	weighted upstreams, if you want no platform

Feature flags are a different product entirely — **LaunchDarkly, Unleash, Flagsmith, or a table in your own database**. They live in the application, not the infrastructure, which is exactly why they can turn a feature on for one customer.

What “watch the canary” actually means

Automatic analysis is what makes canaries useful rather than theatre. A human staring at a dashboard for two hours will miss things and will get bored.

```
every 60 seconds, for the duration of the step:
  query: error rate of the canary
  query: error rate of the stable version
  if canary_errors > 2 x stable_errors → ABORT
  if canary_p99 > 1.5 x stable_p99      → ABORT
  else                                  → continue
```

The comparison is against the stable version at the same moment, for the reason in section 4. **And the abort action is “shift traffic back to zero percent”, which takes effect as fast as the load balancer can be reconfigured** — usually seconds.

Session affinity, and the thing people forget

If a user’s first request goes to the new version and their second goes to the old one, they can see something appear and then vanish. For a stateless read that is harmless. For a multi-step flow it is a bug.

The fix is sticky routing during a canary — hash the user id and send the same user consistently to the same side. That also makes the comparison more honest, because you are measuring “one percent of users had a complete experience of the new version” rather than “one percent of requests were random”.

Connection draining

When you take a machine out, it still has requests in flight.

1. stop sending NEW requests to it (remove from the load balancer)
2. WAIT for in-flight requests to finish - typically 30 seconds
3. send SIGTERM; the process stops accepting and finishes work
4. after a grace period, SIGKILL

Skip step 2 and every request in flight becomes a 502.
At 1,000 requests/second with a 200 ms average, that is
about 200 requests killed PER MACHINE, and with 10 machines
in a rolling update that is 2,000 failed requests per deploy
- invisible in testing, obvious in the error graph.

Automatic rollback, tied to yesterday’s numbers

The strongest version of this is to wire the deployment to the error budget.

```
deploy → canary at 1% → burn-rate alert fires → abort
                                                    automatically
```

```
and after full rollout:
  if the fast burn-rate alert (14.4x for 1 hour) fires
  within 2 hours of a deploy, roll back FIRST and
  investigate afterwards.
```

“Roll back first, investigate afterwards” is the correct default and it is worth saying explicitly, because the instinct under pressure is to understand the problem before undoing it. **Understanding takes an hour. Undoing takes five minutes.**

What cannot be rolled back

Some changes are one-way, and knowing which is a mark of experience.

ROLLBACK-SAFE	NOT ROLLBACK-SAFE
-----	-----
code	a dropped column
configuration	a data migration that rewrote rows
a feature flag	a message published to a queue
	that consumers already processed
	an email sent
	a payment taken
	a schema change the old code
	cannot read

For the right-hand column the answer is not a better rollback. It is not doing it in one step — expand and contract, feature flags, and a long gap before anything destructive.

6 The numbers

How long a canary must run before it means anything.

100,000,000 requests/day = 1,157 requests/second
canary at 1% = 11.6 requests/second

Suppose the new version has a 0.5% error rate.
To see 100 errors - enough to be confident it is not noise:

$100 \text{ errors} / (11.6 \text{ req/s} \times 0.005) = 1,724 \text{ seconds}$
= ~29 minutes

→ A 1% canary needs about half an hour to say anything about a moderately rare failure.

For a RARER bug, at 0.05%:

$100 / (11.6 \times 0.0005) = 17,241 \text{ s} = \sim 4.8 \text{ HOURS}$

→ This is the honest limitation of canaries and it is worth stating: a 1% canary running for 10 minutes detects an outage, not a subtle regression.

At 5% traffic the same 0.5% bug shows in ~6 minutes.

→ which is why the ramp exists, rather than sitting at 1% for hours.

Blue-green capacity.

fleet: 100 machines at \$0.10/hour

steady state	$100 \times \$0.10 = \$10/\text{hour}$
during blue-green	$200 \times \$0.10 = \$20/\text{hour}$

deploy window: 1 hour of overlap, 20 deploys a month
extra cost = $100 \text{ machines} \times \$0.10 \times 1 \text{ h} \times 20$
= \$200/month

→ On a fleet costing \$7,200/month, that is under 3%. Cheap.

fleet: 2,000 machines

extra = $2,000 \times \$0.10 \times 1 \text{ h} \times 20 = \$4,000/\text{month}$

→ still small against \$144,000/month, but now large enough that people argue about it, and it is why big fleets use rolling or canary rather than blue-green.

Rolling update duration.

```

100 machines, batches of 10, each batch:
  start new instances      20 s
  wait for readiness      40 s
  drain and stop old      30 s
-----
                              90 s per batch

10 batches x 90 s = 900 s = 15 minutes to roll forward
                  = 15 minutes to roll BACK

→ The rollback time is the same as the deploy time,
    and that is the weakness of rolling updates.

Blue-green rollback: one router change, ~5 seconds.
Canary abort:         one weight change, ~5 seconds.

```

Requests lost to a deploy without connection draining.

```

1,000 requests/second, 200 ms average duration
→ in-flight at any instant = 1,000 x 0.2 = 200 requests

per machine killed abruptly:      200 failed requests
100-machine rolling update:       20,000 failed requests

against a 99.9% monthly budget of 3,000,000 failures:
  20,000 / 3,000,000 = 0.67% of the month's budget
  x 20 deploys/month = 13% of the budget

→ 13% of your entire error budget spent on nothing but
  not waiting 30 seconds. This is a real and common
  finding when teams first measure it.

```

The database migration, sized.

```

backfilling a new column across 500,000,000 rows

naive: UPDATE users SET full_name = name;
→ one enormous transaction, a lock, replication lag
  measured in hours, and a very bad afternoon

batched: 10,000 rows at a time, 100 ms pause between
  500,000,000 / 10,000 = 50,000 batches
  50,000 x (50 ms work + 100 ms pause) = 7,500 s
                                      = ~2 hours

→ Two hours of background work with no lock and no
  user impact, against a single statement that would
  have taken the site down.

```


7 The trade-offs

Blue-green buys the fastest rollback there is and costs double capacity and a shared database.

Switching back is a router change — seconds — and that is genuinely hard to beat. But you pay for two full environments during the window, and the database is not duplicated, so every change still has to work with both versions. I would not use blue-green if the fleet is large enough that doubling it is a real bill, or if the deploy involves any schema change that is not already backward compatible — in that case the fast rollback is an illusion, because the code can go back and the data cannot.

Canary is the only strategy that finds problems real traffic causes, and it is slow.

Nagaraj's crack showed at the four hundredth impression, not the first. Load-dependent bugs, memory leaks, cache-miss storms and lock contention do not appear in staging and do not appear in the first minute. The cost is time — half an hour at one percent to detect a 0.5 percent error rate, nearly five hours for a 0.05 percent one — and good enough metrics to compare the two versions honestly. I would not canary a change with no user-visible signal to measure, because then the canary is just a slower deploy.

Rolling updates are free and their rollback is as slow as their rollout.

No extra capacity, no traffic-splitting infrastructure, and it is the platform default. But if the new version is bad, going back is another fifteen minutes of the same slow process, and for the whole of that time some users are still on the bad version. I would use rolling for routine, low-risk changes and reach for something else when the change is one I am nervous about.

Feature flags decouple deploying from releasing, and they accumulate.

Being able to turn a feature on for internal staff, then one percent, then everyone, with no deploy, is the most flexible control in the list — and it makes rollback instantaneous for anything the flag covers. The cost is flag debt: every flag is a branch in the code, and n flags mean 2^n combinations nobody has tested. I would insist that every flag has an owner and a removal date, and that the number of live flags is a metric somebody looks at.

Automatic rollback is right far more often than it feels.

Rolling back on an alert without understanding the problem feels wrong and is almost always correct. Understanding takes an hour under pressure; undoing takes five minutes. **The exception is the case where the rollback itself is dangerous** — a migration has already rewritten data, or the old version genuinely cannot read the new schema. **Which is the argument for making sure that case never exists.**

And the honest limit of all of it: deployment strategy cannot save you from an irreversible action.

A sent email, a taken payment, a consumed message, a dropped column. None of those come back because you changed a router weight. The answer there is not a better deploy strategy, it is designing the change so that the irreversible step happens last and separately — which is what expand-and-contract is for, and what feature flags are for.

8 In the interview

How it gets asked

- *“How do you deploy a change to a service handling live traffic?”* — the direct one.
- *“What is blue-green, and what does it cost you?”* — definitions plus the capacity answer.
- *“You deployed and the error rate went up. What now?”* — roll back first, and they want to hear it fast.
- *“How do you rename a database column with no downtime?”* — expand and contract, and this is the real test.
- *“How long should a canary run?”* — arithmetic, and most candidates have none.
- *“What is the difference between a liveness and a readiness probe?”* — a Kubernetes-flavoured filter.

The first ninety seconds

On “how do you deploy to a live service”:

“The thing I would say first is that during any deployment, two versions of my code are running at the same time and reading the same data. Every strategy below is an answer to who sees the new one and for how long, and everything that goes wrong comes from that overlap.

For a routine change I would use a rolling update, which is the Kubernetes default. Replace the machines a few at a time, **waiting for each new one’s readiness check before removing an old one. No extra capacity, no down-time.** The weakness is that rolling back is another rollout — if it takes fifteen minutes to go forward, it takes fifteen minutes to come back.

For a change I am nervous about I would canary. Send one percent of real traffic to the new version and compare its error rate and p99 against the old version serving the same traffic at the same moment. Then 5, 25, 50, 100 percent, with automatic abort rules agreed in advance — **error rate more than twice the stable version, or p99 more than one and a half times, and it shifts back to zero.**

The reason to compare against the stable version rather than a fixed threshold is that traffic patterns move; a fixed number either fires constantly or never.

Where I need the fastest possible rollback, blue-green. Two complete environments, switch the router, and **going back is a router change — seconds.** The costs are double capacity for the window and the fact that the database is shared, so the schema still has to work for both versions.

And underneath all of them, feature flags, because they separate deploying from releasing. **The code ships switched off, and turning it on is not a deploy at all.**

The number I would want the team to know is the time from ‘this is bad’ to ‘the old version is serving’. Under five minutes means you can afford to take risks. An hour means you will deploy rarely, which makes every deploy bigger and more dangerous — and that loop is how teams end up frightened of their own release process.”

The follow-ups

“You deployed twenty minutes ago and the error rate has tripled. Walk me through the next ten minutes.”

“I roll back first and investigate afterwards, and I would want to be unambiguous about that order.

The instinct is to understand the problem before undoing it. That instinct is wrong under time pressure. Understanding takes an hour. Undoing takes five minutes, and it stops the bleeding for users while I think.

Concretely: shift traffic back to the previous version. If it is blue-green, that is a router change and the old environment is still running, so it takes seconds. **If it is a rolling update, it is `kubectl rollout undo`, and it takes as long as the rollout did.** If it is behind a feature flag, I turn the flag off and nothing needs deploying at all.

Then I confirm the error rate actually recovered, because if it did not, the deploy was not the cause and I have just wasted five minutes usefully — **I now know it is something else, which is real information.**

Then I check the error budget, because that decides what happens next. Twenty minutes at three times the normal error rate is a meaningful slice of a monthly budget, and if it took us into the low band, the policy says no more risky changes today.

The one case where I would not roll back immediately is when the rollback is itself dangerous — if a migration has already rewritten data the old code cannot read. **And the fact that this case exists is exactly why schema changes are done in additive steps with the destructive one last,** so that rolling back stays a safe, boring option for as long as possible.

Afterwards: roll forward with the fix, through a canary this time, and write down what the deployment process failed to catch.“

“Rename a column in a table with five hundred million rows, with no downtime.”

“Not in one step. Five deploys, and the reason for each is that no running version of the code may ever look at something that is not there.

The obvious approach — rename the column, then deploy the new code — breaks in the gap between those two actions, when the old code is still querying a column that no longer exists. And during a rolling update that gap is minutes long and involves real users.

So: expand and contract.

One, expand. Add `full_name` as a new nullable column. Purely additive; the old code does not know it exists and is completely unaffected.

Two, dual write. Deploy code that writes both columns and still reads the old one. Now every new row is correct in both places, and rollback is still trivial.

Three, backfill. Copy the existing rows across, in batches, in the background. At five hundred million rows I would do ten thousand at a time with a pause between batches — about fifty thousand batches, roughly two hours. A single `UPDATE` across the whole table would hold a lock and put replication hours behind, which is a far worse outage than the deploy I was trying to avoid.

Four, read new. Deploy code that reads `full_name`. Still writing both, so if this deploy is bad I can roll back to step three and everything still works.

Five, contract. Once nothing reads the old column — and I would wait days or weeks, not hours — stop writing it, then drop it. Dropping the column should feel boringly overdue by the time you do it.

The general rule this comes from is worth stating: additive first, destructive last, with a long enough gap that you could still roll back. And the reason the database is the hard part of deployment is simply that you can run two versions of your code and you cannot run two versions of your data.“

“What is the difference between a liveness probe and a readiness probe, and why does it matter here?”

“Liveness asks ‘are you alive, or should I kill and restart you?’ Readiness asks ‘should I send you traffic right now?’ A failed liveness check restarts the container. A failed readiness check takes it out of the load balancer and leaves it running.

It matters during a deployment because a new instance is usually alive well before it is useful. A service that takes forty seconds to warm caches and open connection pools is alive at second one and ready at second forty. With only a liveness check, the load balancer starts sending real traffic at second one and those users get errors or timeouts — and in a rolling update the platform also thinks the batch succeeded and moves on to kill more old machines.

The classic failure in the other direction is worse: wiring the liveness check to the slow condition. Then the platform kills the container before it ever finishes starting, and you get a restart loop that looks like a crash bug and is actually a configuration mistake.

One design point on readiness that people get wrong: it should test what the service needs to serve, but be careful about testing downstream dependencies. If every instance reports not-ready because one shared dependency is slow, the entire fleet leaves the load balancer at the same moment — and a partial outage becomes a total one.

And related, on the way out: connection draining. When you remove a machine, stop sending it new requests, then wait for the in-flight ones — thirty seconds is typical — before stopping the process. At a thousand requests a second with a two-hundred-millisecond average there are two hundred requests in flight at any instant. Killing ten machines without draining is two thousand failed requests per deploy, twenty deploys a month, which is over a tenth of a three-nines error budget spent on nothing but impatience.”

The model answer

“Design the release process for the system you have just drawn.”

“I would start from the number that decides everything else: how fast can I get back to the previous version. My target is under five minutes, because that is what makes it safe to deploy often, and deploying often is what keeps each change small enough to reason about.

The default path is a rolling update with proper readiness checks and connection draining — a new instance takes no traffic until it says it is ready, and a leaving instance finishes its in-flight requests before it stops. Those two settings alone are worth a large slice of the error budget.

Anything user-facing or risky goes through a canary. One percent of traffic, sticky by user id so that a user gets a consistent experience, with automatic analysis comparing the canary against the stable version — not against a fixed threshold, because traffic patterns move. Abort rules agreed in advance: double the error rate, or one and a half times the p99, and it shifts back to zero automatically.

I would be honest about how long that takes. At a hundred million requests a day, one percent is about twelve requests a second, so detecting a 0.5 percent error rate with any confidence takes about half an hour — and a subtler 0.05 percent bug would take nearly five hours. That is why the ramp exists: 1, 5, 25, 50, 100, with the later steps giving statistical power the first one cannot.

Behind all of it, feature flags, so that deploying and releasing are separate events. The code ships off; enabling it is a configuration change with instant rollback. With the discipline that every flag has an owner and a removal date, because otherwise n flags become 2^n untested combinations.

Database changes never ride along with code changes. Expand and contract: add, dual write, backfill in batches, read new, and drop the old thing weeks later. Additive first, destructive last. The backfill is batched — ten thousand rows with a pause — because one large `UPDATE` across five hundred million rows is a longer outage than anything I was trying to prevent.

And I would wire the deployment to the error budget from yesterday. If the fast burn-rate alert fires within two hours of a deploy, roll back automatically and investigate afterwards. Roll back first is the right default, because understanding takes an hour and undoing takes minutes.

The one thing no deployment strategy can fix is an irreversible action — a payment taken, an email sent, a message consumed, a column dropped. For those the answer is not a better rollback, it is arranging the change so the irreversible step is last, small, and separately controlled.

If I had to reduce all of it to one sentence: keep the old version running until you are sure, change one thing at a time, and make sure that going back is always the cheapest option available.“

9 Recall card

Every deployment runs **TWO VERSIONS AT ONCE** against **ONE DATABASE** — that overlap is where everything goes wrong, so every change must be backward compatible with the version before it. **Five strategies:** **RECREATE** (downtime, fine for batch), **ROLLING** (free, no extra capacity, but rollback is as slow as rollout), **BLUE-GREEN** (rollback in seconds, costs $2 \times$ capacity, database still shared), **CANARY** (the only one that finds load-dependent bugs, and it is slow), **FEATURE FLAGS** (deploy $\neq$ release; instant rollback; costs flag debt, n flags = 2^n untested combinations).

The number that decides your culture: time from “this is bad” to “the old version is serving.” Under five minutes and you can take risks; an hour and you deploy rarely, which makes each deploy bigger and more dangerous. **ROLL BACK FIRST, INVESTIGATE AFTERWARDS** — understanding takes an hour, undoing takes minutes.

LIVENESS restarts the container; **READINESS** removes it from the load balancer and leaves it running. A service is alive long before it is ready — only a liveness check means real users hit a cold instance; wiring liveness to the slow condition gives a restart loop. Do not make readiness depend hard on a shared dependency, or the whole fleet leaves the load balancer at once. Drain connections for ~ 30 s: $1,000 \text{ req/s} \times 200 \text{ ms} = 200 \text{ in-flight per machine}$, so an undrained 10-machine rollout throws away 2,000 requests, and twenty deploys a month is **13% of a three-nines error budget**.

Canary arithmetic, which almost nobody has. 1% of 100M requests/day = 11.6 req/s , so detecting a **0.5% error rate** takes ~ 29 minutes, and a 0.05% one takes ~ 4.8 hours. **A ten-minute 1% canary detects an outage, not a regression** — hence the $1/5/25/50/100$ ramp. Compare against the stable version at the same moment, never a fixed threshold.

You can run two versions of code; you cannot run two versions of data.
EXPAND AND CONTRACT: add column → dual write → backfill in batches → read new → drop the old one weeks later. Additive first, destructive last, with a gap long enough that rollback still works. Backfill 500M rows in 10,000-row batches (~2 hours), never one `UPDATE`. **And no strategy rolls back an irreversible action** — a sent email, a taken payment, a consumed message, a dropped column — so arrange for the irreversible step to be last, small, and separately controlled.

DSA topic: Primes, GCD, and modular arithmetic **System design topic:** Deployments: blue-green, canary, and rollback

Code these, in this order

One rule for the whole set: **say the precondition before you write the line.** “This needs the modulus to be prime.” “This needs `a` and `m` to share no factor.” **Every wrong answer in this topic is a violated precondition that nothing checks,** and saying it out loud is the only defence.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Count Primes	LeetCode 204 (Medium)	The sieve, and why you start at <code>p × p</code> .
2	Greatest Common Divisor of Strings	LeetCode 1071 (Easy)	That Euclid is about structure, not just numbers.
3	Ugly Number II	LeetCode 264 (Medium)	Factors as generators, and the three-pointer merge.
4	Pow(x, n)	LeetCode 50 (Medium)	Square and multiply, plus the negative-exponent edge case.
5	Super Pow	LeetCode 372 (Medium)	Fast power with the exponent given as digits.
6	Fraction to Recurring Decimal	LeetCode 166 (Medium)	Remainders repeating, which is modular arithmetic in disguise.

On problem 1, count the crossings

Instrument the sieve so it counts how many numbers each prime actually strikes out, for a limit of 100. **Record the counts for 2, 3, 5 and 7.** Say why they fall so fast, and connect that to why the total is `n log log n` rather than `n log n`.

On problem 1 again, change the start

Change the inner loop to start at `2p` instead of `p × p`. **Count the total crossings both ways at a limit of one million.** Record both numbers. Say why the answer is identical and the work is not.

On problem 2, find Euclid where it is hiding

Nothing in this problem mentions numbers. **Work out for yourself why the answer is the string of length** `gcd(len(a), len(b))`, and say what “divides” means for strings. Then say what property of Euclid you used.

On problem 4, break it on the edge

Run your fast power on `n = -2147483648`. **Record what happens.** Say why negating that particular value is a problem in a fixed-width language, and what you would do about it.

Then the square-root drill

Take 1,000,003. **Say how many divisions the naive test does, how many the square-root test does, and how many the** `6k ± 1` **version does.** Then say, in one sentence, why the square root is enough.

Then the sieve-memory drill

Compute the memory a Python list sieve needs for limits of one million, ten million and a hundred million. **Then do it again for a bytearray and for a bitset.** Say at which limit each one stops being sensible, and what you would say if asked for primes below a billion.

Then the Euclid drill

Compute `gcd(1071, 462)` by hand, writing every remainder. **Count the steps.** Then do `gcd(89, 55)` and count those. Say why the Fibonacci pair is the worst case.

Then the overflow drill

Write `lcm` both ways. **In Python they agree.** Say exactly which values would break the multiply-first version in a 64-bit language, and compute the threshold.

Then the modulus drill

Compute `(3 - 10) % 7` in Python. **Then say what C, Java and Go give.** Write the three-token fix. Then say which of `+`, `-`, `×` and `÷` survive a modulus, and what replaces the one that does not.

Then the Fermat drill

Compute the inverse of 3 modulo 10 using Fermat’s formula. **Check it by multiplying.** Record that it is wrong, say why, and give the true inverse. Then say which two preconditions Fermat needs and which one `109 + 7` satisfies.

Then the deployment drill

Take any service you can imagine and describe out loud how a change reaches production. **Name the strategy, the abort rule, and the rollback time.** Then say what would have to be true for the rollback to be unsafe.

Then the migration drill

Say the five steps of expand and contract from memory, and **for each one, say what would break if you skipped it.** Then size the backfill for five hundred million rows in ten-thousand-row batches.

Then the canary arithmetic drill

At 100 million requests a day and a 1 percent canary, **compute how long you must wait to see a hundred errors at a 0.5 percent error rate.** Then at 0.05 percent. Then say what that tells you about ten-minute canaries.

The primality drill

1. Say what a prime is, and the two numbers people get wrong.
2. Say why testing to the square root is enough.
3. Give the $6k \pm 1$ optimisation and why it works.
4. Give the cost, and the size of number at which you would name Miller–Rabin.

The sieve drill

1. Describe the sieve in three sentences.
2. Say why crossing off starts at $p \times p$.
3. Say why sieving stops at the square root of the limit.
4. Give the cost and where $\log \log n$ comes from.
5. Give the memory for three limits and three representations.
6. Say what a segmented sieve is for.

The factorising drill

1. Say what an spf table stores.
2. Give the factorising loop and its step count.
3. Say why each step at least halves the number.
4. Say after how many queries the table has paid for itself.

The Euclid drill

1. State the identity.
2. Give the one-sentence reason it preserves the answer.
3. Give the rectangle picture.
4. Give the cost and the worst case.
5. Give the LCM relation and the overflow-safe form.
6. Say what $\text{gcd}(a, 0)$ is and why that is not a special case.

The modular drill

1. Say which three operations pass through a modulus and which does not.
2. Give the replacement for the one that does not.
3. Give Fermat's formula and its two preconditions.
4. Say what happens when the modulus is composite.
5. Give the alternative that does not need a prime.
6. Give the three reasons for $10^9 + 7$.
7. Give the negative-modulo fix and the languages that need it.

The fast-power drill

1. Give the idea in one sentence, using the binary expansion.
2. Trace 3^{13} through the four steps.
3. Give the step count for exponents of 10^9 and 10^{18} .
4. Say what goes wrong without the modulus inside the loop, and why that is hard to debug.

The strategies drill

1. Name five deployment strategies.
2. For each: what it costs and what it buys.
3. Say what is true during every one of them.
4. Say which one is the only way to find load-dependent bugs.

The probes drill

1. Define liveness and readiness, and what failing each one does.
2. Say what goes wrong with only a liveness check.
3. Say what goes wrong when liveness is wired to a slow condition.
4. Say why readiness should not depend hard on a shared dependency.

5. Give the connection-draining sequence and the cost of skipping it.

The rollback drill

1. Give the target time and why the number shapes team behaviour.
2. Say the order: roll back or investigate, and why.
3. Give the rollback time for each of three strategies.
4. Name five things that cannot be rolled back.
5. Say what to do about them instead.

The migration drill

1. Give the five steps of expand and contract.
2. Say what breaks if you rename in one step.
3. Say why the backfill is batched.
4. Say why the gap before dropping is deliberately long.
5. Give the rule that generates the whole pattern.

The break-it drill

For each, say what happens and whether anything reports it:

1. `is_prime` without the `n < 2` check.
2. A sieve built for a limit of `10^11`.
3. `lcm = a * b // gcd(a, b)` in Java, with `a` and `b` around `3 × 10^9`.
4. `(a - b) % m` in C, when `b > a`.
5. Fermat's inverse with a composite modulus.
6. Fast power with the `% m` missing inside the loop.
7. A rolling update with no readiness check.
8. A rolling update with no connection draining.
9. Renaming a column and deploying the new code straight afterwards.
10. A ten-minute canary at one percent, against a 0.05 percent regression.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. Find all primes below one million. The sieve, why it starts at $p \times p$ and stops at the square root, the $n \log \log n$ cost with the reciprocal-sum reason, the hundredfold comparison against testing each number, and memory as the real limit.
2. The answer may be large, so return it modulo $10^9 + 7$. What does that change? Which operations survive, why division does not, Fermat with its preconditions, what happens on a composite modulus, extended Euclid as the general answer, and the three reasons for that particular number.
3. How do you deploy a change to a service handling live traffic? Two versions at once against one database, the five strategies with what each costs, the rollback-time number, roll back before investigating, and expand-and-contract for the schema.

Before you move on

- [] I know 1 is not prime and 2 is, and I write the base cases first.
- [] I can say why testing to the square root is enough.
- [] I know the $6k \pm 1$ optimisation and why it works.
- [] I can write the sieve from memory.
- [] I can say why it starts at $p \times p$ and stops at the square root.
- [] I know the cost is $n \log \log n$ and where that comes from.
- [] I know memory limits the sieve before time does.
- [] I know what a segmented sieve is for.
- [] I can factorise with an spf table and give the step count.
- [] I can state Euclid's identity and the one-sentence reason it holds.
- [] I can give the rectangle picture for a GCD.
- [] I know the cost and the Fibonacci worst case.
- [] I write lcm dividing first, and I can say which values break the other form.
- [] I know which three operations pass through a modulus and which does not.
- [] I can compute a modular inverse with Fermat and state both preconditions.
- [] I know what Fermat does on a composite modulus, and that it does not complain.
- [] I know extended Euclid is the general answer.
- [] I can give the three reasons for $10^9 + 7$.
- [] I know the negative-modulo fix and which languages need it.
- [] I can write fast power and give the step count for 10^9 .

- [] I know what happens without the modulus inside the loop.
- [] I can name five deployment strategies with their costs.
- [] I know two versions always run at once, against one database.
- [] I can define liveness and readiness and say what failing each one does.
- [] I know the two probe misconfigurations and what each looks like.
- [] I can give the connection-draining sequence and price skipping it.
- [] I know my rollback target and why that number shapes behaviour.
- [] I roll back before investigating, and I can say why.
- [] I can give the five steps of expand and contract from memory.
- [] I know why the backfill is batched and the gap before dropping is long.
- [] I can do the canary arithmetic for 0.5 percent and 0.05 percent.
- [] I can name five things no rollback can undo.
- [] I answered all three questions above out loud.

The combinatorics you actually need

DATA STRUCTURES AND ALGORITHMS

The combinatorics you actually need

SYSTEM DESIGN

Security in a design interview

The combinatorics you actually need

1 What this is, and why they ask it

Combinatorics is counting without listing. How many ways can these things be arranged? How many ways can you choose five of them? **The answer is a number, and you must produce it without generating every possibility** — because there are usually more possibilities than atoms available.

Almost all of it comes from one rule. If a choice has 4 options and an independent second choice has 6, there are 24 combinations. Multiply. Everything else in this lesson is that rule with a correction applied.

And one question decides which formula you need: does order matter? “Pick three people for a committee” and “pick three people for first, second and third place” are different questions with different answers, **and they differ by exactly a factor of $3!$.**

They ask it because **counting problems are everywhere and the arithmetic goes wrong quietly.** “How many unique paths across this grid?” is a combinatorics question wearing a dynamic programming costume. **“Return the answer modulo $10^9 + 7$ ” is a counting problem by definition** — nobody asks for a modulus unless the answer is enormous.

And because $n!$ overflows almost immediately. $21!$ does not fit in a 64-bit integer. A candidate who writes `factorial(n) // (factorial(r) * factorial(n - r))` has written something that is mathematically correct and breaks at `n = 21`, **while the answer they wanted was 210.**

By the end of this lesson you can decide in one question which formula a problem needs, compute `nCr` without ever forming a factorial, do it under a modulus with three lookups per query, count arrangements with repeated items, and recognise the four or five standard shapes that hide inside interview problems.

2 The story

The wedding was on the twelfth and Renuka’s daughter had been asking the same question for four days.

“How many do you actually need to take?”

Renuka had six sarees she was prepared to be seen in, and four blouses that went under more than one of them. Her daughter looked at the pile on the bed and saw ten things. **Renuka looked at the same pile and saw twenty-four.**

The girl did not believe her, so Renuka made her count it out loud.

“Take the green one. Which blouses can go under it?”

All four, the girl said.

“So that is four different ways somebody at the wedding sees me. Now the maroon.”

All four again.

Six sarees, four blouses each. Twenty-four.

“So if you put in one more blouse,” the girl said, working it through, “that is twenty-five.”

“Thirty,” said Renuka.

And that was the part the girl actually remembered afterwards. **One extra blouse in the bag was not one more outfit. It was six more — one for every saree.**

On the day itself there was a second argument, and it turned out to be the other half of the same idea.

The photographer wanted the five of them in a line — Renuka, her husband, the two girls, and her mother-in-law. He put them in an order. Then he changed it. Then he tried the old lady in the middle instead of on the end. Then he went back to something close to the first one.

Renuka’s daughter, who by now had started noticing this sort of thing, worked it out while she was standing there in the sun.

Any of the five of them could stand at the left end. Then four people were left for the next place. Then three. Then two. Then the last one had nowhere else to go.

Five times four times three times two.

A hundred and twenty.

She said it out loud. The photographer did not find it as interesting as she did, **and took the fourth arrangement he had tried.**

3 The idea in plain English

Renuka's two arguments are the whole subject. The sarees and blouses are the multiplication principle. The photograph is a permutation. And the difference between them is the one question you always ask first.

The multiplication principle

If one choice has a options and an independent second choice has b options, together they have $a \times b$.

4 breads $\times$ 6 curries $\times$ 3 sweets = 72 different meals

and "no sweet" is a fourth option for the third choice:
 $4 \times 6 \times 4 = 96$

"Independent" is the word doing the work. It means the second choice is not limited by the first. If two of the blouses only go with three of the sarees, you cannot simply multiply — and noticing that is usually where a counting problem becomes hard.

The one question: does order matter?

Ask it before writing anything.

"Pick 3 people for a committee"
→ Asha, Biju, Chetan is the SAME committee as
Chetan, Biju, Asha.
→ ORDER DOES NOT MATTER.

"Pick 3 people for gold, silver and bronze"
→ Asha-Biju-Chetan is a DIFFERENT result from
Chetan-Biju-Asha.
→ ORDER MATTERS.

And there is a second question: can you repeat? Together they give four cases, and every basic counting problem is one of these four.

	ORDER MATTERS	ORDER DOES NOT
NO REPEATS	nPr $n!/(n-r)!$	nCr $n!/(r!(n-r)!)$
REPEATS ALLOWED	n^r (r independent choices from n)	$C(n+r-1, r)$ "stars and bars"

Learn the table as four questions, not four formulas. In an interview you will not remember which formula is which; you will remember “does order matter, can I repeat”, and then derive the right one in ten seconds.

Permutations, from Renuka’s photograph

Arranging n distinct things in a row: $n!$.

The reasoning is the girl’s reasoning and it is better than the formula. Five choices for the first position, then four remain, then three, then two, then one. $5 \times 4 \times 3 \times 2 \times 1 = 120$.

Arranging only r of them: stop early. $nPr = n \times (n-1) \times \dots \times (n-r+1)$, which is r factors.

10 people, 3 prizes:
 $10 \times 9 \times 8 = 720$

Combinations, and where the division comes from

nCr is nPr divided by $r!$, and the reason is worth saying rather than memorising.

Pick 3 of 10 with order: $10 \times 9 \times 8 = 720$

But each GROUP of three has been counted once for every ORDER it could appear in - and three things can be ordered $3! = 6$ ways.

So each committee was counted 6 times.

$720 / 6 = 120$ committees.

“I have counted each answer once per arrangement, so I divide by the number of arrangements” is the most useful sentence in this lesson. It is how you fix over-counting in problems that have no named formula at all.

Two facts about nCr worth having

Symmetry: $C(n, r) = C(n, n - r)$. Choosing 3 to take is the same as choosing 17 to leave. So always compute with the smaller of the two — $C(20, 17)$ in three steps rather than seventeen.

Pascal’s identity: $C(n, r) = C(n-1, r-1) + C(n-1, r)$.

Read it as a sentence about one particular item. Either that item is in the group — then choose the remaining $r - 1$ from the other $n - 1$ — or it is out, and you choose all r from the other $n - 1$. In or out, no overlap, nothing missed.

That sentence is why counting problems turn into dynamic programming. It is the same “take it or leave it” split as the knapsack, and Pascal’s triangle is the DP table.

And the row sums: row n of Pascal’s triangle adds up to 2^n . Because choosing 0, or 1, or 2, ... or n items covers every possible subset, and there are 2^n subsets — which is day 172’s “a subset is a number”, seen from the counting side.

Never compute a factorial

This is the practical heart of the lesson.

```
C(100, 50) = 100! / (50! 50!)
```

100! has 158 digits.

The answer has 30 digits.

→ You would build two enormous numbers and divide them to get something far smaller than either.

Build it up instead, one factor at a time:

```
result = 1
for i in 0 .. r-1:
    result = result * (n - i) // (i + 1)
```

```
C(5, 2):
i=0: 1 * 5 // 1 = 5
i=1: 5 * 4 // 2 = 10
```

The division is exact at every step, which is the part that looks suspicious and is not: after $i + 1$ factors of the numerator, the running value is $C(n, i+1)$, which is a whole number by definition.

And the largest number the loop ever holds is the answer itself. For $C(100, 50)$ that is 30 digits rather than 158.

Arrangements when things repeat

MISSISSIPPI has 11 letters: one M, four I, four S, two P.

If all 11 were distinct: $11! = 39,916,800$

But the four I's are identical, so every arrangement was counted $4! = 24$ times over, once per shuffle of the I's. Same for the four S's, and $2!$ for the P's.

$$11! / (1! \times 4! \times 4! \times 2!) = 34,650$$

Same sentence as before: counted once per arrangement of the identical things, so divide by it.

Stars and bars

Distributing identical things into labelled boxes. Ten identical sweets among four children.

Write the sweets as stars and the divisions as bars:

```
* * * | * * | * * * * | *  
   3     2     4       1
```

Ten stars and three bars, in a row: 13 positions, and you choose which 3 of them are bars.

$$\rightarrow C(13, 3) = 286$$

If every child must get at least one, give each one sweet first and share out the remaining six:

$$\rightarrow C(9, 3) = 84$$

The trick is turning an arrangement problem into a choosing problem, and it is worth recognising because it appears disguised — “how many ways to write n as a sum of k non-negative numbers” is exactly this.

Under a modulus

When the answer must be given modulo $10^9 + 7$, you cannot divide. [Yesterday's](#) inverse does the work.

Precompute once, up to the largest n you will need:

```
fact[i]      = i! mod m  
inv_fact[i] = the inverse of i! mod m
```

Then every query is three lookups and two multiplications:

$$C(n, r) = \text{fact}[n] * \text{inv_fact}[r] * \text{inv_fact}[n-r] \mod m$$

And there is a trick for the inverse factorials that is worth knowing, because the obvious way costs one fast power per entry:

```

inv_fact[N] = power_mod(fact[N], m - 2, m)    ONE fast power
then walk DOWN:
    inv_fact[i-1] = inv_fact[i] * i mod m

because  $1/(i-1)! = 1/i! \times i$ 

```

One fast power for the whole table instead of N of them — at $N = 200,000$ that is thirty multiplications instead of six million.

The shapes to recognise

```

"how many paths across an m x n grid, right and down only"
    → every path is (m-1) downs and (n-1) rights in some
       order; choose which steps are the downs
    →  $C(m+n-2, m-1)$ 

"how many balanced bracket strings of length 2n"
"how many shapes can a binary tree with n nodes have"
"how many ways to fully parenthesise a product"
    → all the SAME NUMBER: the nth Catalan number,
        $C(2n, n) / (n + 1)$ 
    → 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862

"how many subsets"           →  $2^n$ 
"how many subsets of size k" →  $C(n, k)$ 

```

Catalan numbers are worth memorising up to about 42, because recognising 1, 2, 5, 14, 42 in your own small-case working is often how you spot the pattern in the first place.

4 The picture

The multiplication principle as a tree:

6 sarees x 4 blouses

```
green ---+--- blouse A
      +--- blouse B
      +--- blouse C
      +--- blouse D          4 outfits
```

```
maroon --+--- blouse A
        +--- blouse B
        +--- blouse C
        +--- blouse D       4 outfits
```

... four more sarees ...

6 branches, each splitting 4 ways = 24 leaves.

ADD ONE BLOUSE: every branch splits 5 ways instead of 4 $\rightarrow$ 30. The new blouse is worth SIX outfits, not one, because it appears under every saree.

Order matters, or it does not:

Pick 2 from {A, B, C}

ORDER MATTERS ($3P2 = 6$)

AB BA

AC CA

BC CB

ORDER DOES NOT ($3C2 = 3$)

AB $\leftarrow$ same group

AC $\leftarrow$ same group

BC $\leftarrow$ same group

Each unordered pair appears $2! = 2$ times in the left-hand list.

$6 / 2 = 3$.

THAT DIVISION IS THE ONLY DIFFERENCE BETWEEN THE TWO COLUMNS, and it is $r!$ every time.

Pascal's triangle, and the identity drawn on it:

```

      1
    1 1
  1 2 1
1 3 3 1
  4 6 4
1 5 10 10 5 1
  6 15 20 15 6 1

```

THE IDENTITY, on the 20:

```

... 10 10 ...      row 5
    \  /
C(6,3) = 20        row 6

```

```

C(6,3) = C(5,2) + C(5,3)
      ^^^^^^  ^^^^^^
      item is item is
      IN      OUT

```

ROW SUMS:

```

row 0: 1 = 1 = 2^0
row 1: 1+1 = 2 = 2^1
row 2: 1+2+1 = 4 = 2^2
row 3: 1+3+3+1 = 8 = 2^3
row 6: 1+6+15+20+15+6+1 = 64 = 2^6

```

→ every subset, sorted by its size. Which is the same 2^n as counting from 0 to $2^n - 1$ in binary.

Stars and bars:

10 identical sweets, 4 children

```

* * * | * * | * * * * | *
\_____/ \___/ \_____/ \_/
  3       2       4       1

```

The row is 10 stars + 3 bars = 13 symbols.

An arrangement is decided entirely by WHICH 3 of the 13 positions hold bars.

→ $C(13, 3) = 286$

Note what the bars can do:

```

| * * * * * * * * * | |

```

→ first child 0, second 10, third 0, fourth 0

Adjacent bars mean an empty box, which is allowed here.

"At least one each": hand out 4 sweets first, then share the remaining 6 the same way → $C(9, 3) = 84$.

Grid paths, which is nCr in a costume:

3 rows x 7 columns, moving only RIGHT and DOWN

```
+---+---+---+---+---+---+
| S |   |   |   |   |   |
+---+---+---+---+---+---+
|   |   |   |   |   |   |
+---+---+---+---+---+---+
|   |   |   |   |   |   | E |
+---+---+---+---+---+---+
```

EVERY path is exactly 2 downs and 6 rights,
in some order. 8 steps in total.

A path IS a choice of which 2 of the 8 steps
are the downs.

→ $C(8, 2) = 28$

The dynamic programming solution fills 21 cells.
The counting solution is one line. Both are correct,
and knowing they are the same problem is the point.

5 The code, built step by step

The two that are just loops

```
def factorial(n: int) → int:
    """n! - the number of ways to put n distinct things in a row."""
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def permutations_count(n: int, r: int) → int:
    """nPr: pick r from n where ORDER MATTERS. n x (n-1) x ... x (n-r+1)."""
    if r < 0 or r > n:
        return 0
    result = 1
    for i in range(r):
        result *= n - i
    return result
```

`permutations_count` stops after `r` factors rather than computing `n!` and dividing. The formula `n!/(n-r)!` is a description, not an instruction — for `n = 100`, `r = 3` it says build a 158-digit number and divide, when the answer is 970,200.

Returning 0 for out-of-range `r` rather than raising is a choice, and it is usually the right one, because counting problems produce impossible cases naturally and zero is the honest answer.

Combinations, without ever forming a factorial

```
def combinations_count(n: int, r: int) → int:
    """nCr, built up one factor at a time. Never forms n! - so it never explodes."""
    if r < 0 or r > n:
        return 0
    r = min(r, n - r)
    result = 1
    for i in range(r):
        result = result * (n - i) // (i + 1)
    return result
```

Two lines carry all the value here.

`r = min(r, n - r)` uses the symmetry — `C(20, 17)` becomes `C(20, 3)` and the loop runs three times instead of seventeen.

`result * (n - i) // (i + 1)` multiplies before it divides, and the division is exact every time. After `i + 1` steps the running value is exactly `C(n, i+1)`, which is a whole number. If you divide first you get a fraction; if you use `/` instead of `//` you get a float and you lose precision at around 15 digits.

Pascal's triangle, which is addition only

```
def pascal_triangle(rows: int) → list[list[int]]:
    """The whole triangle, by addition only. No multiplication, no division."""
    triangle: list[list[int]] = []
    for n in range(rows):
        row = [1] * (n + 1)
        for i in range(1, n):
            row[i] = triangle[n - 1][i - 1] + triangle[n - 1][i]
        triangle.append(row)
    return triangle
```

The ends are always 1 — `C(n, 0)` and `C(n, n)` — so the inner loop starts at 1 and stops before the end. No multiplication, no division, no overflow risk beyond the numbers themselves.

This is the right tool when you need many values of `nCr` with small `n`, and the wrong one when `n` is large: the triangle for `n = 100,000` is five billion entries.

Under a modulus, with the walk-down trick

```
def build_factorials(limit: int, modulus: int = MOD) → tuple[list[int], list[int]]:
    """Factorials and their inverses, once, so every later nCr is three lookups."""
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % modulus
    inverse_fact = [1] * (limit + 1)
    inverse_fact[limit] = power_mod(fact[limit], modulus - 2, modulus)
    for i in range(limit, 0, -1):
        inverse_fact[i - 1] = inverse_fact[i] * i % modulus
    return fact, inverse_fact
```

One fast power for the whole table. $1/(i-1)!$ is $1/i! \times i$, so once you have the last inverse you can walk backwards multiplying. At `limit = 200,000` that is thirty multiplications instead of six million.

```
def ncr_mod(n: int, r: int, fact: list[int], inverse_fact: list[int],
            modulus: int = MOD) → int:
    """n! / (r! (n-r)!) under a modulus: multiply by inverses instead of dividing."""
    if r < 0 or r > n:
        return 0
    return fact[n] * inverse_fact[r] % modulus * inverse_fact[n - r] % modulus
```

Three lookups and two multiplications, after the one-off preparation. Take the modulus between the multiplications, not only at the end — in C++ the intermediate would overflow otherwise.

The named shapes

```
def multiset_permutations(counts: list[int]) → int:
    """Arrangements of things with repeats: n! divided by the factorial of each count."""
    total = sum(counts)
    result = factorial(total)
    for c in counts:
        result //= factorial(c)
    return result

def stars_andBars(items: int, boxes: int) → int:
    """Ways to split `items` identical things into `boxes` labelled boxes, empties allowed."""
    return combinations_count(items + boxes - 1, boxes - 1)

def catalan(n: int) → int:
    """C(2n, n) / (n + 1). Counts balanced brackets, and shapes of binary trees."""
    return combinations_count(2 * n, n) // (n + 1)

def unique_paths(rows: int, cols: int) → int:
    """Only right and down moves: choose which of the steps are the downward ones."""
    return combinations_count(rows + cols - 2, rows - 1)
```

Each of these is one line because the thinking happened before the code. `unique_paths` is the clearest example: the whole solution is the sentence “a path is a choice of which steps go down”, and once you have said it there is nothing left to

write.

The complete solution

```
"""Day 175 - the combinatorics interviews actually use: count, choose, and do it safely."""

from __future__ import annotations

MOD = 1_000_000_007


def factorial(n: int) → int:
    """n! - the number of ways to put n distinct things in a row."""
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result


def permutations_count(n: int, r: int) → int:
    """nPr: pick r from n where ORDER MATTERS. n x (n-1) x ... x (n-r+1)."""
    if r < 0 or r > n:
        return 0
    result = 1
    for i in range(r):
        result *= n - i
    return result


def combinations_count(n: int, r: int) → int:
    """nC_r, built up one factor at a time. Never forms n! - so it never explodes."""
    if r < 0 or r > n:
        return 0
    r = min(r, n - r)
    result = 1
    for i in range(r):
        result = result * (n - i) // (i + 1)
    return result


def pascal_row(n: int) → list[int]:
    """Row n of Pascal's triangle. Each entry is the sum of the two above it."""
    row = [1]
    for i in range(n):
        row.append(row[-1] * (n - i) // (i + 1))
    return row


def pascal_triangle(rows: int) → list[list[int]]:
    """The whole triangle, by addition only. No multiplication, no division."""
    triangle: list[list[int]] = []
    for n in range(rows):
        row = [1] * (n + 1)
        for i in range(1, n):
            row[i] = triangle[n - 1][i - 1] + triangle[n - 1][i]
        triangle.append(row)
    return triangle


def power_mod(base: int, exponent: int, modulus: int) → int:
    """Square and multiply, from yesterday. Needed for the modular inverse."""
    result = 1
    base %= modulus
    while exponent:
        if exponent & 1:
            result = result * base % modulus
        base = base * base % modulus
        exponent >>= 1
    return result
```

```

def build_factorials(limit: int, modulus: int = MOD) → tuple[list[int], list[int]]:
    """Factorials and their inverses, once, so every later nCr is three lookups."""
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % modulus
    inverse_fact = [1] * (limit + 1)
    inverse_fact[limit] = power_mod(fact[limit], modulus - 2, modulus)
    for i in range(limit, 0, -1):
        inverse_fact[i - 1] = inverse_fact[i] * i % modulus
    return fact, inverse_fact

def ncr_mod(n: int, r: int, fact: list[int], inverse_fact: list[int],
            modulus: int = MOD) → int:
    """n! / (r! (n-r)!) under a modulus: multiply by inverses instead of dividing."""
    if r < 0 or r > n:
        return 0
    return fact[n] * inverse_fact[r] % modulus * inverse_fact[n - r] % modulus

def multiset_permutations(counts: list[int]) → int:
    """Arrangements of things with repeats: n! divided by the factorial of each count."""
    total = sum(counts)
    result = factorial(total)
    for c in counts:
        result /= factorial(c)
    return result

def stars_and_bars(items: int, boxes: int) → int:
    """Ways to split `items` identical things into `boxes` labelled boxes, empties allowed."""
    return combinations_count(items + boxes - 1, boxes - 1)

def catalan(n: int) → int:
    """C(2n, n) / (n + 1). Counts balanced brackets, and shapes of binary trees."""
    return combinations_count(2 * n, n) // (n + 1)

def unique_paths(rows: int, cols: int) → int:
    """Only right and down moves: choose which of the steps are the downward ones."""
    return combinations_count(rows + cols - 2, rows - 1)

if __name__ == "__main__":
    print("THE MULTIPLICATION PRINCIPLE")
    print(" 4 breads x 6 curries x 3 sweets = ", 4 * 6 * 3)
    print(" and with a choice of 'no sweet': 4 x 6 x 4 =", 4 * 6 * 4)

    print()
    print("FACTORIALS GROW FASTER THAN ANYTHING YOU HAVE A NAME FOR")
    for n in (5, 10, 15, 20, 21, 25):
        f = factorial(n)
        fits = "fits in 64 bits" if f < 2 ** 63 else "OVERFLOWS 64 bits"
        print(f" {n}>2)! = {f:>26}, {fits}")

    print()
    print("ORDER MATTERS, OR IT DOES NOT")
    for n, r in ((5, 2), (5, 3), (10, 3), (52, 5)):
        print(f" n={n}>2} r={r} nPr = {permutations_count(n, r)>12,}"
              f" nCr = {combinations_count(n, r)>10,}"
              f" ratio = {r)! = {factorial(r)}")

    print()
    print("PASCAL'S TRIANGLE - every entry is the two above it")
    for row in pascal_triangle(8):
        print(" " + " ".join(f"{v:>4}" for v in row).center(44))

    print()
    print("THE SYMMETRY, AND WHY IT SAVES HALF THE WORK")

```



```

print(f" C(20, 3) = {combinations_count(20, 3):,}")
print(f" C(20, 17) = {combinations_count(20, 17):,} same number")
print(" → so always compute with the SMALLER of r and n-r")

print()
print("THE MULTIPLICATIVE FORM NEVER EXPLODES")
print(f" C(100, 50) = {combinations_count(100, 50):,}")
print(f" 100! has {len(str(factorial(100)))} digits, and we never formed it")

running = 1
biggest = 1
for i in range(50):
    running = running * (100 - i) // (i + 1)
    biggest = max(biggest, running)
print(f" largest intermediate value: {biggest:,} ({len(str(biggest))} digits)")

print()
print("nC r UNDER A MODULUS - precompute once, then three lookups")
fact, inv_fact = build_factorials(200_000)
for n, r in ((5, 2), (100, 50), (200_000, 100_000)):
    print(f" C({n:,}, {r:,}) mod 1e9+7 = {ncr_mod(n, r, fact, inv_fact):,}")
print(f" check small case against exact: C(100,50) mod MOD = "
      f"{combinations_count(100, 50) % MOD:,}")

print()
print("ARRANGEMENTS WITH REPEATS")
print(" MISSISSIPPI: 11 letters, M1 I4 S4 P2")
print(f" 11! / (1! 4! 4! 2!) = {multiset_permutations([1, 4, 4, 2]):,}")
print(f" against 11! = {factorial(11):,} if every letter were distinct")

print()
print("STARS AND BARS - identical things into labelled boxes")
print(f" 10 identical sweets among 4 children = C(13, 3) = "
      f"{stars_and_bars(10, 4)}")
print(f" 10 sweets, 4 children, everyone gets at least one = C(9, 3) = "
      f"{combinations_count(9, 3)}")

print()
print("CATALAN NUMBERS - the same count wearing five hats")
print(f" {[catalan(n) for n in range(10)]}")
print(" balanced bracket strings of length 2n; shapes of a binary tree")
print(" with n nodes; ways to triangulate a polygon; ways to fully")
print(" parenthesise a product.")

print()
print("GRID PATHS - the choose in disguise")
for r, c in ((3, 7), (3, 3), (18, 18)):
    print(f" {r} x {c} grid: C({r + c - 2}, {r - 1}) = {unique_paths(r, c):,}")

print()
print("VERIFICATION")
import math
import random

bad = 0
for _ in range(3000):
    n = random.randint(0, 300)
    r = random.randint(-2, n + 2)
    if combinations_count(n, r) != (math.comb(n, r) if 0 ≤ r ≤ n else 0):
        bad += 1
    if permutations_count(n, r) != (math.perm(n, r) if 0 ≤ r ≤ n else 0):
        bad += 1
    if r ≥ 0 and combinations_count(n, r) % MOD != ncr_mod(n, r, fact, inv_fact):
        bad += 1
for n in range(0, 40):
    if pascal_row(n) != [math.comb(n, k) for k in range(n + 1)]:
        bad += 1
    if catalan(n) != math.comb(2 * n, n) // (n + 1):
        bad += 1

```

```
if multiset_permutations([1, 4, 4, 2]) != 34650:  
    bad += 1  
print(f" {bad} mismatches over 3,000 random pairs (3 checks each) plus 40 rows")
```

Running it:

THE MULTIPLICATION PRINCIPLE

4 breads x 6 curries x 3 sweets = 72
and with a choice of 'no sweet': $4 \times 6 \times 4 = 96$

FACTORIALS GROW FASTER THAN ANYTHING YOU HAVE A NAME FOR

$5! = 120$ fits in 64 bits
 $10! = 3,628,800$ fits in 64 bits
 $15! = 1,307,674,368,000$ fits in 64 bits
 $20! = 2,432,902,008,176,640,000$ fits in 64 bits
 $21! = 51,090,942,171,709,440,000$ OVERFLOWS 64 bits
 $25! = 15,511,210,043,330,985,984,000,000$ OVERFLOWS 64 bits

ORDER MATTERS, OR IT DOES NOT

$n=5$	$r=2$	$nPr = 20$	$nCr = 10$	ratio = $2! = 2$
$n=5$	$r=3$	$nPr = 60$	$nCr = 10$	ratio = $3! = 6$
$n=10$	$r=3$	$nPr = 720$	$nCr = 120$	ratio = $3! = 6$
$n=52$	$r=5$	$nPr = 311,875,200$	$nCr = 2,598,960$	ratio = $5! = 120$

PASCAL'S TRIANGLE - every entry is the two above it

```

      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
```

THE SYMMETRY, AND WHY IT SAVES HALF THE WORK

$C(20, 3) = 1,140$
 $C(20, 17) = 1,140$ same number
→ so always compute with the SMALLER of r and $n-r$

THE MULTIPLICATIVE FORM NEVER EXPLODES

$C(100, 50) = 100,891,344,545,564,193,334,812,497,256$
100! has 158 digits, and we never formed it
largest intermediate value: $100,891,344,545,564,193,334,812,497,256$ (30 digits)

nCr UNDER A MODULUS - precompute once, then three lookups

$C(5, 2) \bmod 1e9+7 = 10$
 $C(100, 50) \bmod 1e9+7 = 538,992,043$
 $C(200,000, 100,000) \bmod 1e9+7 = 879,467,333$
check small case against exact: $C(100,50) \bmod MOD = 538,992,043$

ARRANGEMENTS WITH REPEATS

MISSISSIPPI: 11 letters, M1 I4 S4 P2
 $11! / (1! 4! 4! 2!) = 34,650$
against $11! = 39,916,800$ if every letter were distinct

STARS AND BARS - identical things into labelled boxes

10 identical sweets among 4 children = $C(13, 3) = 286$
10 sweets, 4 children, everyone gets at least one = $C(9, 3) = 84$

CATALAN NUMBERS - the same count wearing five hats

[1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862]
balanced bracket strings of length $2n$; shapes of a binary tree
with n nodes; ways to triangulate a polygon; ways to fully
parenthesise a product.

```
GRID PATHS - the choose in disguise
3 x 7 grid: C(8, 2) = 28
3 x 3 grid: C(4, 2) = 6
18 x 18 grid: C(34, 17) = 2,333,606,220
```

VERIFICATION

0 mismatches over 3,000 random pairs (3 checks each) plus 40 rows

Look at the factorial table: **20!** fits in a 64-bit integer and **21!** does not. That is the entire argument for the multiplicative form, and it is a number worth being able to state.

Look at “largest intermediate value”: it is exactly the answer. The loop for **C(100, 50)** never holds anything bigger than the 30-digit result, while the formula it implements mentions a 158-digit number twice.

And look at the last grid line: an 18 by 18 grid has 2.3 billion paths. Enumerating them is impossible; counting them is one line — which is the whole reason this subject exists.

6 What it costs

The basic three.

```
factorial(n):          n - 1 multiplications
permutations_count(n, r): r multiplications
combinations_count(n, r): min(r, n-r) multiply-divide pairs

C(20, 17) without the symmetry step: 17 iterations
C(20, 17) with it:                3 iterations

→ O(min(r, n-r)) time, O(1) space.
```

But there is a catch that interviewers like, and it is worth naming. These are **O(r)** in the number of operations, *not in the amount of work* — because the numbers themselves grow. **C(100, 50)** is a 30-digit number, and multiplying 30-digit numbers is not one machine instruction. In a fixed-width language the distinction vanishes, because you would have overflowed instead.

Pascal’s triangle.

rows x average row length:

$n \text{ rows} \rightarrow 1 + 2 + 3 + \dots + n = n(n+1)/2 \text{ entries}$

$n = 30 \rightarrow 465 \text{ entries}$

$n = 1,000 \rightarrow 500,500 \text{ entries} \quad \sim 4 \text{ MB in Python}$

$n = 10,000 \rightarrow 50,005,000 \text{ entries} \quad \sim 400 \text{ MB}$

$n = 100,000 \rightarrow 5,000,050,000 \quad \text{no}$

$\rightarrow O(n^2)$ time AND $O(n^2)$ space.

$\rightarrow$ The right tool for many small queries, useless for large n . That crossover is the answer to "which method would you use?"

Precomputed factorials under a modulus.

PREPARATION, limit N :

factorials: N multiplications

the one inverse: ~ 30 multiplications (fast power)

the walk down: N multiplications

$2N + 30$

$N = 200,000 \rightarrow \sim 400,000 \text{ operations, well under a second}$

WITHOUT the walk-down trick:

one fast power per entry: $N \times 30 = 6,000,000$

$\rightarrow 15\times$ slower, and it is a two-line change.

EACH QUERY AFTERWARDS:

3 lookups, 2 multiplications, 2 modulo operations

$\rightarrow O(1)$

1,000,000 queries: $400,000 + 5,000,000 \text{ operations}$

against Pascal's triangle, which cannot even be built.

Space for the modular tables.

two lists of $N+1$ integers

$N = 200,000 \rightarrow 400,002 \text{ entries} \quad \sim 3 \text{ MB in Python}$

$3.2 \text{ MB in C++ (int64)}$

$N = 10,000,000 \rightarrow \sim 160 \text{ MB in C++}$

$\rightarrow$ Size the table to the largest n in the problem statement, and no larger.

The named shapes.

```

multiset_permutations: one big factorial + k small ones
    → O(n) multiplications, but on LARGE numbers

stars_and_bars:    one nCr          → O(min(r, n-r))
catalan(n):        one nCr of size 2n → O(n)
unique_paths:      one nCr          → O(min(m, n))

    unique_paths(18, 18) → 17 iterations
    the dynamic programming version → 324 cells

→ Both are instant here. At 1,000 x 1,000 the DP is a
    million cells and the formula is still 999 iterations.

```

7 The traps

Computing the factorials in the formula.

```

def ncr_naive(n: int, r: int) → int:
    return factorial(n) // (factorial(r) * factorial(n - r))

```

In Python this is correct and wasteful. `C(100, 50)` builds a 158-digit number twice. **In C++, Java or Go it is simply wrong:** `21!` overflows a 64-bit integer, so `C(21, 2)` — whose answer is 210 — produces nonsense. Say “I never form a factorial” as you write the multiplicative version.

And if you reach for floats to escape it:

```

math.factorial(1000) / math.factorial(500)

```

```

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OverflowError: integer division result too large for a float

```

A clear error, which is lucky. The dangerous version is the one that succeeds:

```

def ncr_float(n, r):
    result = 1.0
    for i in range(r):
        result = result * (n - i) / (i + 1)    # / not //
    return result

```

```
ncr_float(50, 25) = 126410606437752.0          correct
ncr_float(30, 15) = 155117520.0                correct
ncr_float(100, 50) = 1.0089134454556418e+29
    exact answer  = 100891344545564193334812497256
```

It looks right on small inputs and silently loses precision past about fifteen digits. One character — `/` instead of `//` — and the failure only appears on the large cases.

Forgetting the symmetry.

`C(1000, 997)` computed with `r = 997` does 997 iterations for an answer that takes three. Not wrong, just foolish, and `r = min(r, n - r)` is one line.

Dividing before multiplying.

```
result = result // (i + 1) * (n - i)          # WRONG
```

Integer division truncates. The running value is only guaranteed to be a whole number **after** the multiplication, so **dividing first throws away a remainder and the answer comes out too small**. Multiply, then divide, in that order, every time.

Building Pascal's triangle for a large `n`.

```
pascal_triangle(100_000)
```

Five billion entries. In Python you will run out of memory long before it finishes; the **honest failure is that the machine stops responding**, which is worse than an exception. Recognise the crossover: Pascal for many queries with small `n`, precomputed factorials for large `n`.

Negative or out-of-range arguments.

```
math.comb(5, -1)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: k must be a non-negative integer
```

The library raises. Your own version should decide deliberately — returning 0 is usually right, because counting problems generate impossible cases and zero is the true count.

Fractional inputs.

```
math.factorial(3.5)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: 'float' object cannot be interpreted as an integer
```

This is what a stray `/` produces two lines earlier.

Assuming independence when there is none.

```
6 sarees, 4 blouses → 24.
```

```
But if two of the blouses only go with three of  
the sarees:
```

```
4 sarees x 4 blouses = 16
```

```
2 sarees x 2 blouses = 4
```

```
--  
20, not 24.
```

The multiplication principle needs the second choice to be unaffected by the first. Nothing will tell you this is violated — the number simply comes out too large, and this is by far the most common conceptual error in real counting problems. Check by counting a tiny case by hand.

`nCr` under a modulus when `n` is bigger than the modulus.

The factorial table approach requires `n < m`, because `fact[m]` and everything after it is zero — `m!` contains `m` as a factor, so it is `0 mod m`. For `n ≥ 10^9 + 7` you need Lucas' theorem, and naming it is enough; the situation is rare and the interviewer mostly wants to know that you noticed.

Double counting, which no formula protects you from.

```
"How many 3-letter strings from {A, B} with at least one A?"
```

```
WRONG: choose one position for the A (3 ways), fill the  
rest freely (2 x 2) → 12
```

```
TRUTH: 2^3 = 8 strings in total, minus the one with no A  
→ 7
```

```
The wrong method counts "AAB" once for the first A and  
again for the second.
```

When a count is suspiciously large, you have double counted. The fix is usually to count the complement — total minus the bad ones — which is nearly always easier than counting the good ones directly.

8 In the interview

How it gets asked

- “How many unique paths are there across an m by n grid?” — LeetCode 62, and the formula answer stands out.
- “How many ways can you arrange these letters?” — the repeats question.
- “Compute nCr . Now for n up to 200,000, modulo $10^9 + 7$.” — precomputed factorials.
- “How many binary search trees with n nodes?” — Catalan, LeetCode 96.
- “How many ways to split n identical items among k people?” — stars and bars.
- “Return the number of ways ... the answer may be large.” — the modulus is the tell.

The first ninety seconds

On “how many ways can you arrange these, and now with repetitions”:

“The first question I ask is whether order matters, and the second is whether repeats are allowed. Those two answers pick the formula, and I would rather derive it than recall it.

If order matters and there are no repeats, it is a permutation. Arranging all n things is $n!$ — and the reasoning is better than the formula: n choices for the first position, $n - 1$ left for the second, and so on down to one. If I only place r of them, I stop after r factors — $n \times (n-1) \times \dots \times (n-r+1)$.

If order does not matter, it is a combination, and it is the permutation divided by $r!$. Because every group of r has been counted once for each order it could appear in, and there are $r!$ orders. That sentence — ‘I counted each answer once per arrangement, so I divide by the number of arrangements’ — is the one I actually use, because it fixes over-counting in problems that have no named formula.

If repeats are allowed and order matters, it is n^r — r independent choices, each with n options. If repeats are allowed and order does not matter, it is stars and bars, $C(n + r - 1, r)$.

On the repeats-in-the-items version — arranging MISSISSIPPI — it is the same division argument. Eleven letters would be $11!$ if they were all distinct, but the four I’s are identical, so every arrangement was counted $4!$ times over; likewise $4!$ for the S’s and $2!$ for the P’s. So $11!$ over $1! \cdot 4! \cdot 4! \cdot 2!$, which is 34,650 against 39,916,800.

And one implementation point I would make immediately, because it is where most answers break: I never compute a factorial. $21!$ overflows a 64-bit integer, so $C(21, 2)$ — which is 210 — would come out as nonsense. I build it up one factor at a time instead, and the largest number the loop holds is the answer itself.”

The follow-ups

“Compute nCr , for n up to 200,000, modulo $10^9 + 7$.”

“Precompute the factorials and their inverses once, then every query is three lookups.

The formula needs a division and I cannot divide under a modulus, so each division becomes multiplication by a modular inverse — which is yesterday’s Fermat, $a^{(m-2)} \bmod m$, valid because $10^9 + 7$ is prime.

So I build two tables up to 200,000: `fact[i] = i! mod m`, and `inv_fact[i]`, the inverse of that. Then `C(n, r) = fact[n] * inv_fact[r] * inv_fact[n-r]`, taking the modulus between the multiplications — in C++ the intermediate would otherwise overflow.

There is a trick for the inverse table that I would definitely use. The obvious way is one fast power per entry — 200,000 fast powers, about six million multiplications. Instead compute only the last one, `inv_fact[N]`, and walk backwards: `inv_fact[i-1] = inv_fact[i] * i`, because $1/(i-1)!$ is $1/i!$ times i . One fast power for the whole table — about fifteen times faster, for a two-line change.

Preparation is roughly $2N$ operations, so under a second, and every query afterwards is constant time. Pascal’s triangle would be the alternative and it is impossible here — the triangle for $n = 200,000$ is twenty billion entries. Pascal is the right answer when n is small and there are many queries; tables are the right answer when n is large.

One limitation worth naming: this needs n to be smaller than the modulus, because $m!$ contains m as a factor and is therefore zero. If n could exceed $10^9 + 7$, that is Lucas’ theorem, which I would mention rather than derive.”

“How many unique paths across a grid, moving only right and down?”

“It is $C(m + n - 2, m - 1)$, and the reason is nicer than the formula.

Every path from the top-left to the bottom-right of an m by n grid is exactly $m - 1$ downward moves and $n - 1$ rightward moves, in some order. Every path has the same length. The only thing that distinguishes one path from another is which of the steps are the downward ones.

So a path is a choice of $m - 1$ positions out of $m + n - 2$. For a 3 by 7 grid that is $C(8, 2) = 28$.

I would also say that the dynamic programming answer is correct and is what most people write — fill a grid where each cell is the sum of the one above and the one to the left. And the interesting thing is that the DP table is Pascal’s triangle, rotated. Which is not a coincidence: Pascal’s identity says $C(n, r) = C(n-1, r-1) + C(n-1, r)$, and ‘the cell above plus the cell to the left’ is the same sentence.

On cost: the formula is $O(\min(m, n))$ and constant space; the DP is $O(mn)$ time and $O(n)$ space with one row. At 1,000 by 1,000 that is 999 operations against a million.

The reason I would still write the DP first in some interviews is that it survives a follow-up. Add an obstacle in the middle of the grid and the closed form is gone, while the DP changes by one line. So I would give the formula, say why it works, and then say which one I would actually ship and why.”

“How many different shapes can a binary tree with n nodes have?”

“The n th Catalan number, $C(2n, n) / (n + 1)$. For n from zero: 1, 1, 2, 5, 14, 42, 132, 429.

The way to get there without recalling the formula is to set up the recurrence. Pick which node is the root; then i nodes go in the left subtree and $n - 1 - i$ in the right, and the shapes multiply. So $T(n) = \text{sum over } i \text{ of } T(i) \times T(n-1-i)$, which is a dynamic programme in about four lines and is what I would write.

Then I would say that this sequence is the Catalan numbers and has a closed form, and that recognising 1, 2, 5, 14, 42 in my own small-case working is usually how I spot it in the first place.

The thing worth adding is how many different problems this same sequence answers, because interviewers follow up along that line. Balanced bracket strings of length $2n$. Shapes of a binary tree with n nodes. Ways to triangulate a polygon. Ways to fully parenthesise a product of $n + 1$ factors. Monotone lattice paths that stay below the diagonal.

They are all the same count because they all have the same recurrence: split into a left part and a right part, and multiply. If a problem’s small cases are 1, 2, 5, 14, that is what is happening, and I would look for the split rather than keep enumerating.”

The model answer

“Talk me through how you approach a counting problem.”

“Four steps, in this order, and I try not to skip the first one even when the answer looks obvious.

First, I count a tiny case by hand. Three items, four items. Not to find the answer, but because an exhaustive count of a small case is the only check I have on a formula that produces a number too large to verify. And if my small cases come out 1, 2, 5, 14, I already know it is Catalan.

Second, I ask the two questions: does order matter, and can things repeat. That picks the shape. Order and no repeats is a permutation. No order and no repeats is a combination. Order with repeats is n^r . No order with repeats is stars and bars. I derive rather than recall, because the derivation is short: n choices, then $n - 1$, and so on.

Third, I look for over-counting, because that is where the real errors are. The sentence I use is: ‘how many times has each answer been counted?’ — and then I divide by that. Combinations divide by $r!$ because each group appears once per ordering. MISSISSIPPI divides by $4! \cdot 4! \cdot 2!$ because each arrangement appears once per shuffle of the identical letters. And when a count comes out suspiciously large, over-counting is nearly always the cause — the usual fix is to count the complement instead: total minus the bad ones.

Fourth, I worry about the arithmetic, and this is where most solutions actually fail. $21!$ overflows a 64-bit integer, so I never compute a factorial: I build nCr one factor at a time, multiplying then dividing, using the smaller of r and $n - r$. The largest value that loop ever holds is the answer itself. And if the problem says ‘modulo $10^9 + 7$ ’, that is the tell that the answer is astronomically large — so precomputed factorial and inverse-factorial tables, with the inverses built by one fast power and a walk down.

The judgement call I would flag is between a formula and a dynamic programme. Grid paths have a closed form and the DP is a million times slower — but put one obstacle in the grid and the closed form disappears while the DP changes by a single line. So I say which one I would write for the problem as stated, and which one I would write if I expected the requirements to move.

And the one assumption I check explicitly is independence. The multiplication principle needs the second choice to be unaffected by the first. Nothing warns you when that fails — the number just comes out too big — and it is the most common conceptual mistake in this whole area.“

9 Recall card

Two questions decide everything: DOES ORDER MATTER, and CAN THINGS REPEAT. Order + no repeats = nPr ; no order + no repeats = nCr ; order + repeats = n^r ; no order + repeats = stars and bars, $C(n+r-1, r)$. Derive, do not recall: n choices, then $n-1$, then $n-2$ — and stop after r factors.

The most useful sentence in the subject: “I counted each answer once per arrangement, so I divide by the number of arrangements.” $nCr = nPr / r!$. MISSISSIPPI = $11!/(1!4!4!2!) = 34,650$. When a count comes out too big you have double counted — usually the fix is to count the COMPLEMENT. And the multiplication principle needs independence; nothing warns you when the second choice is limited by the first.

NEVER COMPUTE A FACTORIAL. $21!$ overflows a 64-bit integer, so $C(21,2) = 210$ comes back as nonsense. Build up: `result = result * (n - i) // (i + 1)`, multiply before you divide (the division is exact only after the multiplication), and use `r = min(r, n-r)` — 3 iterations, not 17, for $C(20,17)$. `//` not `/`: the float version is right on small inputs and quietly wrong past ~15 digits.

$C(n,r) = C(n-1,r-1) + C(n-1,r)$ is “the item is in, or it is out” — which is why counting problems become DP, and why Pascal’s triangle IS the grid-paths table. Row n sums to 2^n , because it is every subset sorted by size. Pascal is $O(n^2)$ time and space — right for many small queries, impossible at $n = 100,000$.

Modulo 10^9+7 is the tell that the answer is astronomical. Precompute `fact[]` and `inv_fact[]`, then $C(n,r) = \text{fact}[n] \cdot \text{inv_fact}[r] \cdot \text{inv_fact}[n-r]$ — three lookups, $O(1)$ per query. Build the inverses with ONE fast power at the top and a walk down (`inv_fact[i-1] = inv_fact[i] * i`), $15\times$ faster than one power each. Requires $n < \text{modulus}$, else Lucas’ theorem. Recognise the shapes: grid paths = $C(m+n-2, m-1)$; Catalan = $C(2n,n)/(n+1) = 1, 1, 2, 5, 14, 42, 132$ for brackets, tree shapes and parenthesisations; subsets = 2^n .

1 What this is, and why they ask it

Security in a design interview is not a separate subject. It is four questions asked about the system you have already drawn. Who are you? What are you allowed to do? Is anybody watching the wire? And what happens when one part of this is compromised?

Two words carry most of it, and candidates run them together. Authentication is proving who you are. Authorisation is deciding what you may do. Knowing somebody's name does not entitle them to the strongroom, and most real breaches are failures of the second one, not the first.

And the organising idea is defence in depth. You assume every single layer will eventually fail, and you arrange things so that no one failure is enough on its own.

They ask it because almost every design interview ends with “and what about security?” — often in the last five minutes, often as a test of whether you have anything at all. A candidate who says “we would use HTTPS and hash the passwords” has said the two things everybody says. A candidate who says “the token is short-lived so a leaked one is worth fifteen minutes, and every query is scoped by tenant id because that is where the real bug always is” has clearly worked on something real.

And because it is the area where the gap between knowing the words and knowing the mechanism is widest. Everybody has heard of SQL injection. Far fewer can say what a parameterised statement actually does differently.

By the end of this lesson you can give a structured three-minute security answer for any system, store passwords correctly and say why, choose between sessions and tokens with the trade-off named, list the attacks that actually happen with the fix for each, and price the things security costs you.

2 The story

The branch had eleven people in it and one strongroom, and the door of the strongroom had two locks that were nothing like each other.

Kannan had the key to one. The manager had the key to the other. Neither would turn on its own. It had been built that way, and the bank had built a great many branches that way for a great many years.

What Kannan noticed in his first month was that **the two locks were not really about trust.** Everybody in that branch trusted everybody else. Devraj, who made the tea and carried the ledgers, had been there thirty-one years and would sooner have walked into the sea than taken ten rupees. **But Devraj could not open the strongroom. Neither could Kannan on his own. Neither could the manager.**

He asked about it once, and got an answer he thought was strange at the time.

“It is not that I do not trust you. It is that if money ever goes missing, I want to be able to say, in front of everybody, that it could not have been you alone.”

Then, after a moment: **“The lock protects you more than it protects the bank.”**

There were other rules and Kannan absorbed them without ever being told why. **Everybody signed the register on the way in, with the time.** Nobody carried a bag through that door. **And the cash at the counter each morning was only what the day was expected to need** — everything else stayed behind the two locks, so that whatever happened at the counter, it could only be that much.

Then the thing that actually taught him something.

For about four months in his second year, the manager kept his key in the top drawer of his desk.

It was easier. He was in and out of the strongroom eleven times a day and hunting for the key each time was tiresome, and the drawer was three feet from where he sat.

Nothing happened. No money went missing. Nobody noticed.

And that was the whole problem, because for four months there had been one lock on that door and not two, and the only reason anybody ever found out was that the manager was transferred and the new one asked, on his first morning, where the second key was kept.

Nothing went wrong is not the same as nothing was wrong.

3 The idea in plain English

Kannan's branch has almost every idea in this lesson. Two locks is separation of duties. The register is an audit log. The small amount of cash at the counter is blast radius. And the key in the drawer is what happens to every control that is

inconvenient.

The two words

```
AUTHENTICATION  "who are you?"  
                  a password, a token, a certificate, a fingerprint  
                  → answered ONCE, at the edge  
  
AUTHORISATION    "are you allowed to do this?"  
                  → answered EVERY TIME, on every request,  
                  for every object
```

Devraj was thoroughly authenticated — everyone knew exactly who he was — and completely unauthorised.

Most real-world breaches are authorisation failures. The attacker is a perfectly legitimate logged-in user **who asks for somebody else's data and gets it.** Which is the next idea.

The bug that actually happens

It has a name — insecure direct object reference — and it is the single most common serious flaw in real systems.

```
GET /invoices/8812    → your invoice. Fine.  
GET /invoices/8813    → somebody else's invoice.  
  
The user is authenticated. The endpoint checked  
that they are logged in. It never checked that  
invoice 8813 BELONGS to them.
```

The fix is not “use unguessable ids”. That is hiding, not securing. **The fix is that every read and every write is scoped by the owner:** `WHERE id = 8813 AND account_id = <the caller's account>`. **The ownership check is in the same statement as the fetch, so it cannot be forgotten separately.**

In a multi-tenant system this is the whole ball game. Every single query carries the tenant id, and the strongest version pushes it below the application entirely — row-level security in Postgres, so a query without a tenant filter returns nothing rather than everything.

Storing passwords

Three rules, and the reasons matter more than the rules.

Never store the password. Store a hash of it — a one-way transformation.

Never use a fast hash. SHA-256 is designed to be fast, and fast is exactly wrong here, because an attacker with a stolen file can try billions of guesses a second. Use a deliberately slow function designed for passwords: **bcrypt, scrypt, or Argon2id.**

Always salt. A salt is a random value stored alongside the hash and mixed in before hashing. Without it, two users with the same password have the same hash — so one crack breaks both, and precomputed tables of common passwords work directly. **bcrypt and Argon2 salt automatically; you do not have to remember.**

```
bcrypt with a work factor of 12 takes about 250 ms  
per hash on ordinary hardware.
```

```
That is deliberate. It is slow for you once per login,  
and catastrophic for an attacker doing it a billion times.
```

And the work factor is a dial you turn up as hardware gets faster, which is why the cost is stored inside the hash string itself.

Sessions or tokens

Two ways to remember that somebody logged in.

SESSION COOKIE

```
the client holds a random id  
the SERVER holds the session data  
→ revoking is instant: delete the row  
→ needs shared session storage across machines
```

JWT (a signed token)

```
the client holds the claims, signed  
the server holds NOTHING  
→ no lookup, scales trivially  
→ CANNOT BE REVOKED before it expires
```

That last line is the whole trade and it is the answer interviewers want. A JWT is valid until it expires, full stop. If somebody's account is compromised at 10:00 and you disable it at 10:01, **their token still works until it expires.**

So the practical shape is a short-lived access token and a long-lived refresh token.

```
access token  15 minutes, a JWT, sent on every request  
refresh token 30 days, stored server-side, revocable
```

```
→ a stolen access token is worth 15 minutes  
→ revoking the refresh token ends the session at the  
  next renewal, so within 15 minutes
```

That is Kannan's counter float. Keep only what the moment needs where it can be taken.

Encryption, in two places

In transit: TLS, everywhere, including inside your own network. The old model — a hard shell and a soft inside — fails the moment anything gets inside, which it eventually does. Service-to-service traffic uses mutual TLS, where both ends present a certificate, so a service proves what it is and not merely where it is.

At rest: disk and field. Whole-disk or whole-volume encryption is nearly free and protects against a stolen disk or a decommissioned drive. It protects against nothing else — an attacker with access to your running application sees decrypted data, because the application has the key.

Say that limitation out loud, because “we encrypt at rest” is said constantly by people who think it means more than it does.

Field-level encryption is the stronger version: encrypt specific columns — identity numbers, card details — with a key held elsewhere. **The cost is that you can no longer index or search those columns**, which is a real design constraint, not a footnote.

Secrets

A secret in the repository is a secret that is public. Repository history is forever, and rotating a leaked key is much harder than never committing it.

```
WHERE SECRETS GO
  a secrets manager: HashiCorp Vault, AWS Secrets Manager,
  GCP Secret Manager, Azure Key Vault
```

```
BETTER: no long-lived secret at all
  the machine has an identity (an IAM role, a workload
  identity) and receives short-lived credentials
  automatically, rotated every few hours
```

The direction of travel is worth stating: fewer secrets, shorter lives, automatic rotation. A credential that lasts an hour needs far less protecting than one that lasts three years.

The attacks that actually happen

Five, with the mechanism of the fix, because naming the attack is not the answer.

SQL injection. Building a query by pasting strings together, so that input can become code. **The fix is parameterised statements**, and the mechanism matters: **the query text and the values travel to the database separately, so the value is never parsed as SQL at all.** Escaping is a weaker fix that people get wrong; parameterisation makes it structurally impossible.

Cross-site scripting. Attacker-supplied text is rendered as HTML, so their script runs in your users' browsers. **The fix is encoding on output, in the context you are outputting into** — HTML body, attribute and JavaScript all need different encoding. **Plus a Content Security Policy, which stops inline scripts running even if one slips through. That is defence in depth in one line.**

Cross-site request forgery. Another site makes the user's browser send an authenticated request to yours. **The fix is SameSite cookies, plus a token the other site cannot read.**

Server-side request forgery. Your service fetches a URL supplied by a user, and the user supplies an internal address — the cloud metadata endpoint, for instance, which hands out credentials. **The fix is an allowlist of destinations, never a blocklist**, and blocking internal address ranges at the network.

Credential stuffing. Not a clever attack at all: **passwords leaked from some other site, tried against yours in bulk. The fix is multi-factor authentication, rate limiting per account and per address, and checking new passwords against known-breached lists.**

Blast radius, and the key in the drawer

Assume each layer fails. Ask what the next one stops.

the edge is breached	→ internal traffic is still authenticated (mTLS)
a service is compromised	→ its credentials only reach its own data (least privilege)
the database is dumped	→ passwords are slow-hashed, sensitive fields separately encrypted
an insider acts alone	→ two people are required for the dangerous operations

And every one of these degrades the way the manager's key degraded. Controls that are inconvenient get worked around, quietly, by well-meaning people, and the system keeps working perfectly the whole time. Which is why the audit log and the periodic review exist — not to catch thieves, but to notice that the second lock stopped being a lock four months ago.

4 The picture

The layers, and what each one stops:

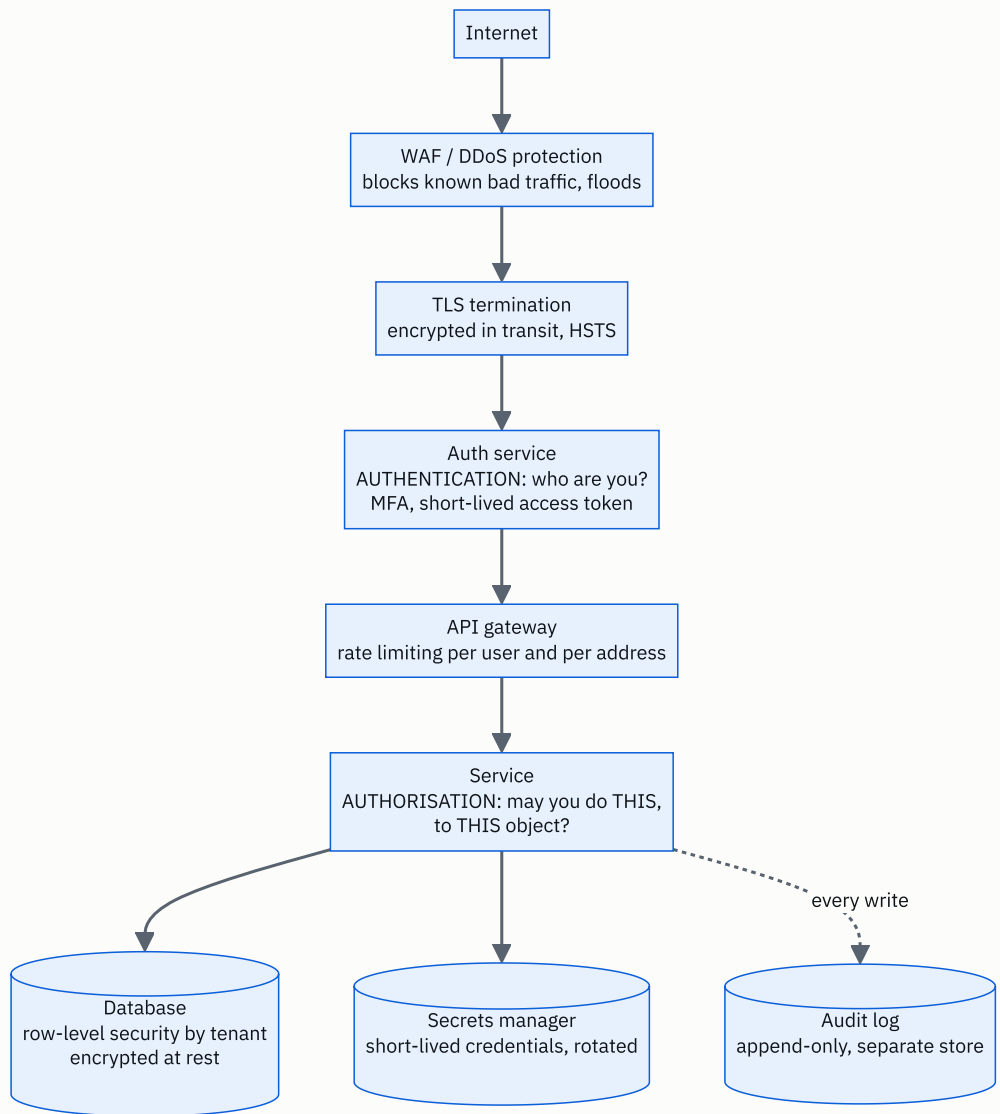


FIGURE 175.1

Read it as a sequence of independent failures. The WAF stops the noise. TLS stops the wire. Authentication answers who. Authorisation answers whether — and that box is where the real bugs live, because it is the only one that has to be

right on every single request for every single object.

And notice the audit log is a separate store. If the same credentials that write your data can also rewrite your audit trail, you do not have an audit trail — you have a log that an attacker edits on the way out.

Authentication against authorisation, drawn as the mistake:

```
REQUEST:  GET /invoices/8813
          Authorization: Bearer <valid token for user 41>
```

WHAT A BROKEN SERVICE CHECKS:

```
[x] is the token valid and unexpired?      YES
[x] is user 41 a real, active user?        YES
→ 200 OK, here is invoice 8813
```

WHAT IT MUST ALSO CHECK:

```
[ ] does invoice 8813 belong to user 41?   NO
```

THE FIX, and note where it lives:

```
SELECT * FROM invoices
WHERE id = 8813
      AND account_id = 41      ← in the SAME statement
```

Not a separate `if` two lines above. In the query.
A separate check is a check somebody can forget to write
on the next endpoint.

The password path, and why slowness is the feature:

REGISTER

```
password ---> bcrypt(password, random salt, cost 12)
               ~250 ms
               ---> $2b$12$<22-char salt><31-char hash>
                     ^       ^       ^
                     algo cost  salt, stored WITH the hash
```

LOGIN

```
candidate ---> bcrypt with the SAME salt and cost
               ---> compare, in constant time
```

WHY SLOW IS THE POINT

SHA-256, fast: ~10,000,000,000 guesses/second on a GPU
bcrypt cost 12: ~4 guesses/second per core

8 lowercase letters = $26^8 = 208,827,064,576$ candidates

```
with SHA-256: 208.8e9 / 1e10 = 21 SECONDS
with bcrypt:  208.8e9 / 4    = 52.2e9 seconds
                               = about 1,650 YEARS
                               on one core
```

Same password. Same attacker. The only difference is
the cost of one guess.

5 How it actually works

Logging somebody in, in practice

Almost nobody should build this themselves any more, and saying so is a mark of judgement rather than laziness.

```
OAuth 2.0      a framework for DELEGATED ACCESS -
                "let this app act on my behalf"
OpenID Connect a thin layer on top of OAuth 2.0 that
                actually does LOGIN, and returns an
                ID token saying who the user is
```

```
→ "Sign in with Google" is OIDC.
→ Managed identity providers: Auth0, Okta, AWS Cognito,
  Firebase Auth, Keycloak if you want to host it.
```

What you get by not building it: password reset flows that are not vulnerable, MFA, breached-password checks, device fingerprinting, and somebody else's security team. What you give up: control, and a dependency whose outage stops all logins.

Multi-factor authentication

something you KNOW	a password
something you HAVE	a phone, a hardware key
something you ARE	a fingerprint, a face

SMS codes	better than nothing, defeated by SIM swapping
TOTP apps	good, phishable (the code can be relayed)
WebAuthn / passkeys	BEST: the key is bound to the site's domain, so a phishing site cannot use it at all

That last point is the one worth knowing. WebAuthn is phishing-resistant by construction — the credential simply will not produce a signature for the wrong domain, **so it does not rely on the user noticing anything.**

Managing keys

Envelope encryption is how this is actually done, and it is a neat idea.

1. generate a random DATA key for this record
2. encrypt the data with the data key
3. encrypt the DATA KEY with a MASTER key held in a key management service (AWS KMS, GCP KMS, Vault)
4. store the encrypted data and the encrypted data key together

THE MASTER KEY NEVER LEAVES THE KMS.

Rotating the master key means re-encrypting only the small data keys - not terabytes of data.

And the access control on the master key is the real control: who may call “decrypt” is a policy, it is logged, and it can be revoked without touching any data.

The audit log

Different from application logs, and the differences are the point.

APPLICATION LOG

for debugging
sampled, dropped freely
30-day retention
engineers can write
whatever

AUDIT LOG

for accountability
never dropped
years, often legally mandated
APPEND-ONLY; nobody can edit
or delete, including admins

EVERY ENTRY:

who (the authenticated identity, not the machine)
what (the action)
which object (the id)
when (a trusted clock)
from where (address, device)
and the outcome - INCLUDING DENIED ATTEMPTS

Denied attempts are the ones with the most information in them. A hundred denials from one account in a minute is the clearest signal you will ever get, and a system that only logs successes is blind to exactly the thing you want to see.

Least privilege, mechanically

the reporting service can READ the orders table
→ and cannot write to it
→ and cannot see the users table at all
the batch job's credentials expire in 1 hour
the admin panel requires a second approval for
refunds over a threshold

Kannan's two locks, generalised. The test is not “do we trust this service?” It is “if this service is completely taken over tonight, what exactly can the attacker reach?”

What to do when it goes wrong

Having an answer here separates people who have been through it.

1. CONTAIN revoke credentials, rotate keys, disable the affected path. Before understanding.
2. PRESERVE snapshot the logs and the machines before restarting anything - restarting destroys the evidence
3. ASSESS what was reachable, what was actually taken, whose data
4. NOTIFY regulators and users, inside the legal window - 72 hours under GDPR
5. FIX and then the honest question: why did no layer stop this, and which layer had quietly stopped working

Step five is the manager's key in the drawer. The interesting finding in most reviews is not that a control was missing. It is that one existed in the design, and had been bypassed for months — and everything kept working the whole time.

6 The numbers

Why a slow hash is the whole defence.

```
ATTACKER SPEED, on stolen hashes, with a GPU rig:
MD5           ~ 100,000,000,000 guesses/second
SHA-256       ~ 10,000,000,000 guesses/second
bcrypt (12)   ~ 4 guesses/second/core

→ bcrypt is roughly 2.5 BILLION times slower per guess.

AN 8-CHARACTER LOWERCASE PASSWORD:
26^8 = 208,827,064,576 possibilities

SHA-256: 208.8e9 / 1e10      = 21 seconds
bcrypt:  208.8e9 / 4         = 52.2e9 seconds
                                = 1,655 years (1 core)
                        even with 10,000 cores: ~2 months

ADD TWO CHARACTERS - 10 lowercase letters:
26^10 = 141,167,095,653,376
SHA-256: 141.2e12 / 1e10 = 3.9 hours
bcrypt:  141.2e12 / 4    = 1.1 MILLION years

→ length beats complexity, by a very long way.
```

What that costs you at the front door.

bcrypt cost 12 = ~250 ms of CPU per login

one core: 4 logins/second

16-core box: 64 logins/second

→ 100,000 users all logging in at 9 a.m. over 10 minutes
= 167 logins/second

→ you need ~3 such machines JUST for password hashing

→ AND IT IS A DENIAL-OF-SERVICE VECTOR: an attacker
sending 1,000 bogus login attempts a second consumes
250 CPU-seconds per second of wall clock.

Which is why login endpoints are rate limited separately
and more strictly than anything else.

Token lifetimes, as blast radius.

a stolen access token is worth its remaining lifetime

lifetime 15 min → at most 15 minutes of access

lifetime 24 h → a working day

lifetime 30 days → a month, and you cannot stop it

with 15-minute access tokens and revocable refresh tokens:
time from "disable this account" to "they are locked out"
= at most 15 minutes

with a 30-day JWT and no denylist:
= 30 days.

→ The entire argument for short expiry, in one comparison.

Audit log volume, since it is kept for years.

100,000,000 requests/day, but only WRITES are audited
say 5% are writes: 5,000,000 events/day
~500 bytes per structured entry

$5,000,000 \times 500 = 2,500,000,000$ bytes = 2.5 GB/day

7-year retention (a common financial requirement):

$2.5 \text{ GB} \times 365 \times 7 = 6,387 \text{ GB} = \sim 6.4 \text{ TB}$

in object storage at \$0.023/GB-month:

$6,387 \text{ GB} \times \$0.023 = \sim \$147/\text{month}$ at full size

→ Cheap, and non-negotiable. Compare with the 720 GB/day
of application logs, which you sample and delete.

TLS, so you can answer “isn’t it expensive?”

full handshake:	~1-2 ms of CPU, two round trips (TLS 1.2)
TLS 1.3:	one round trip
resumed session:	~0 round trips, negligible CPU
bulk encryption:	hardware-accelerated, ~1% overhead

at 3,500 requests/second with 90% session resumption:
350 full handshakes/s x 1.5 ms = 0.5 CPU-seconds/second
→ about half a core.

→ The "TLS is too slow" objection is twenty years out of date, and worth saying so plainly.

Rate limits worth quoting.

login attempts	5 per account per 15 minutes
	20 per IP address per minute
password reset	3 per account per hour
general API	1,000 per user per minute
expensive endpoints	10 per user per minute

→ per ACCOUNT and per ADDRESS, both. Per address alone is defeated by a botnet; per account alone lets one attacker spray one guess across a million accounts, which is exactly how credential stuffing works.

7 The trade-offs

JWTs buy stateless scale and cost you revocation.

No session lookup means any machine can verify a request with only a public key, which is genuinely valuable at scale. The price is that a token is valid until it expires and nothing can stop it. The mitigations all reintroduce state: a denylist of revoked tokens is a lookup on every request, which is the thing you were avoiding. I would use short-lived JWTs with revocable refresh tokens for a large API, and plain server-side sessions for a normal web application — where the session store is a Redis lookup nobody notices and instant revocation is worth having.

Every security control costs usability, and usability failures become security failures.

MFA on every action is more secure and drives people to write codes down. Ninety-day password rotation was industry standard for twenty years and has been withdrawn by NIST, because it made people pick Summer2024! and then

Autumn2024! . The manager's key in the drawer is the general form of this. I would rather have a control people can live with permanently than a stricter one that is quietly bypassed within six months.

Encryption at rest protects against much less than people think.

It defends against a stolen disk, a mislaid backup and a badly decommissioned drive. That is the list. It does nothing against a compromised application, because the application holds the key and reads plaintext. Field-level encryption with keys in a separate service is the real defence for sensitive columns, and it costs you indexing and search on those columns — which frequently means the feature has to be redesigned. I would encrypt whole volumes always, because it is nearly free, and encrypt individual fields only where the data genuinely warrants losing query ability.

A stronger password hash is a denial-of-service surface.

bcrypt at cost 12 is 250 milliseconds of CPU that an unauthenticated stranger can make you spend. A thousand bogus logins a second consumes 250 CPU-seconds every second. So the login endpoint needs tighter rate limiting than anything else in the system, and that limit needs to be per account and per address at once.

Rate limiting logins prevents brute force and enables logout abuse.

Locking an account after five failures stops guessing and lets anybody lock anybody else out by failing five times against their address. The better shape is exponential backoff plus a CAPTCHA rather than a hard lock, and to count failures per address as well as per account.

Buying identity is usually right, and it is a dependency.

A managed identity provider gives you flows that took other people years to get right. The cost is that if it is down, nobody can log in, including your own staff during the incident. I would use one for almost anything, and I would make sure there is a break-glass path for administrators that does not depend on it.

And the honest limit: none of this survives a determined insider or a supply-chain compromise.

Two locks stop one person acting alone; they do not stop two people agreeing. A dependency you pulled in last week can read everything your process can read. The answer there is not another control at the edge — it is least privilege, so that any single compromised component reaches very little, plus an audit

trail an attacker cannot edit. I would say plainly that the goal is not preventing all breaches. It is making sure that no single failure is sufficient, and that you find out.

8 In the interview

How it gets asked

- *“What are the security concerns in this design?”* — the open one, in the last five minutes.
- *“How do you store passwords?”* — and the follow-up is always “why not SHA-256?”
- *“How do you handle authentication between services?”* — mTLS, or signed tokens.
- *“A user’s account is compromised. Walk me through what happens.”* — revocation, and token lifetime.
- *“How do you stop one customer seeing another customer’s data?”* — the tenant question, and the real answer.
- *“What would you log for security purposes?”* — the audit log, and why it is separate.

The first ninety seconds

On “what are the security concerns in this design”:

“I would go through it in four layers, because that keeps it structured rather than a list of buzzwords.

First, identity — authentication. Who is making this request? Login through an identity provider using OpenID Connect rather than building it myself, **multi-factor** available and mandatory for anything privileged, and passwords stored with bcrypt or Argon2id, never a fast hash. A short-lived access token — fifteen minutes — with a revocable refresh token behind it.

Second, permission — authorisation, and this is where the real bugs are. Authentication is answered once at the edge; authorisation has to be answered on every request for every object. The most common serious flaw in real systems is that an endpoint checks you are logged in and never checks the thing you asked for is yours. So the ownership condition goes in the query itself — `WHERE id = ? AND account_id = ?` — not in a separate check two lines above, because a separate check is one somebody forgets to write on the next endpoint.

Third, the data. TLS everywhere including between internal services, because a hard shell with a soft inside fails the moment anything gets in. Volume encryption at rest, which is nearly free — and I would be honest that it only protects against a stolen disk, not against a compromised application. Field-level encryption with keys in a key management service for anything genuinely sensitive, accepting that I lose the ability to index those columns. And no secrets in the repository — a secrets manager, or better, short-lived credentials issued to the machine’s own identity.

Fourth, the attacks that actually happen. Parameterised statements for injection, output encoding plus a content security policy for scripting, SameSite cookies for request forgery, an allowlist for anything that fetches a user-supplied URL, and rate limiting per account and per address for credential stuffing.

And running through all four: assume each layer fails and ask what the next one stops. The question I keep asking is not ‘do we trust this service’ but ‘if this service is taken over tonight, what exactly can the attacker reach?’”

The follow-ups

“How do you store passwords, and why not SHA-256?”

“bcrypt, scrypt or Argon2id, with a salt, and a work factor tuned so one hash takes around a quarter of a second.

Not SHA-256, and the reason is that SHA-256 is fast, which is exactly the wrong property. A GPU rig does about ten billion SHA-256 guesses a second. An eight-character lowercase password is twenty-six to the eighth, about 209 billion possibilities — so twenty-one seconds to try all of them.

bcrypt at cost twelve does about four guesses per second per core. The same 209 billion candidates take around 1,650 years on a core, and even with ten thousand cores it is a couple of months. Same password, same attacker; the only thing that changed is the cost of one guess.

The salt is separate and does a different job. It is a random value stored with the hash, so two users with the same password get different hashes. Without it, one crack breaks every account sharing that password, and pre-computed tables of common passwords work directly. bcrypt and Argon2 handle the salt automatically, which is another reason to use them rather than assemble something.

The work factor is stored inside the hash string, so it can be raised over time — you re-hash on next login.

Two things I would add that people miss. Compare in constant time, or the comparison itself leaks information. And this is a denial-of-service surface: 250 milliseconds of CPU that an unauthenticated stranger can make me spend, so the login endpoint gets stricter rate limiting than anything else in the system — per account and per address.

And the thing that beats all of it: length. Two extra lowercase characters multiply the search space by 676, so the honest advice is a long passphrase and a breached-password check, rather than complexity rules that make people write `Password1!` and rotation policies that make them write `Password2!` .“

“A customer says they can see another customer’s data. What went wrong, and how do you prevent it?”

“An authorisation check that was never written, almost certainly. The user is properly authenticated — this is not somebody breaking in, it is a legitimate user asking for an id that is not theirs and being given it.

The endpoint checked the token was valid and the user was real, and then fetched the record by id without checking who owns it. It has a name — insecure direct object reference — and it is the most common serious flaw in real systems.

The fix that does not work is unguessable ids. That is hiding, not securing — the id still leaks through a URL somebody shares, a support ticket, a referrer header.

The fix that works is that ownership is part of the fetch, not a separate step.

`WHERE id = ? AND account_id = ?`, in the same statement, so there is no version of the code where the fetch succeeded and the check was skipped.

At the design level, the stronger answer is to stop relying on every developer remembering. Push the tenant filter below the application — row-level security in Postgres, where the session carries the tenant id and a query without the filter returns nothing rather than everything. The failure mode becomes ‘no rows’ instead of ‘somebody else’s rows’, which is the right direction for a mistake to fail in.

Then the response, since it has already happened. Contain first: disable the path or revoke the tokens before I understand it. Preserve the logs and any affected machines before restarting anything, because restarting destroys the evidence. Then work out from the audit log exactly which records were read and by whom — which is only possible if reads of sensitive objects were audited, so that decision has to have been made months earlier. Then notify, inside the legal window — seventy-two hours under GDPR.

And the review question I would want asked afterwards is not ‘who wrote the bug’. It is ‘what would have caught this’ — a test that asserts one tenant cannot read another’s row, and a default-deny data layer.“

“How do services authenticate to each other, and how do you manage the secrets?”

“Mutual TLS, and as few long-lived secrets as I can arrange.

For service-to-service, both ends present a certificate, so each side proves what it is rather than merely where it is. The old model assumed anything inside the network was trustworthy, and that fails completely the moment one thing inside is compromised. A service mesh — Istio or Linkerd — will issue and rotate those certificates automatically, which matters because certificate management by hand is where this falls apart.

For secrets, the first rule is that nothing goes in the repository. History is forever, and rotating a leaked key is much harder than never committing it — so a scanner in continuous integration that fails the build on a committed credential is worth having.

Beyond that, the direction of travel is fewer secrets with shorter lives. The best version is no stored secret at all: the machine has an identity — an IAM role, a workload identity — and receives credentials automatically that expire in an hour. A credential that lasts an hour needs far less protecting than one that lasts three years.

Where a real secret is unavoidable — a third-party key — it lives in a secrets manager, Vault or the cloud provider’s, with access controlled per service and every read logged.

For data keys, envelope encryption. Generate a data key per record, encrypt the data with it, encrypt the data key with a master key that never leaves the key management service. Rotating the master key then re-encrypts a few small keys instead of terabytes of data, and the real control is the policy on who may call decrypt — which is logged, and revocable without touching any data.

Underneath all of it, least privilege. The reporting service reads orders and cannot write them and cannot see users at all. The question is never ‘do we trust this service’. It is ‘if this is taken over tonight, what can the attacker reach?’”

The model answer

“What are the security concerns in this design?”

“I would answer in four layers and then say the thing that ties them together, because otherwise this becomes a list of words.

Identity. Login via an identity provider using OpenID Connect rather than built by me — password reset and MFA and breached-password checks are things other people have spent years getting right. Passwords, where I hold them, in bcrypt or Argon2id with a salt and a quarter-second work factor: SHA-256 falls to a GPU in twenty-one seconds for an eight-character password, bcrypt takes over a thousand years on a core. WebAuthn for anything privileged, because it is phishing-resistant by construction — the credential will not sign for the wrong domain, so it does not depend on the user noticing. Fifteen-minute access tokens with revocable refresh tokens, so a stolen token is worth fifteen minutes and disabling an account takes effect within fifteen.

Permission, and I would spend the most time here because this is where real breaches are. Authentication is answered once; authorisation is answered on every request for every object. The ownership condition goes in the query, not in a check beside it. In a multi-tenant system, row-level security below the application, so a missing filter returns nothing instead of everything. Least privilege on every service credential — read-only where reading is all it does, and no access at all to tables it has no business in. And separation of duties for the genuinely dangerous operations, so that no single person or single credential is sufficient.

Data. TLS everywhere including internally, with mutual TLS between services and certificates rotated by the mesh. Volume encryption at rest, which is nearly free — and I would be explicit that it protects against a stolen disk and nothing else, because the application holds the key. Field-level encryption with a key management service for genuinely sensitive columns, accepting that I lose indexing on them. No secrets in the repository; short-lived credentials from a workload identity wherever possible.

Attacks. Parameterised statements, so values never reach the parser as code. Output encoding plus a content security policy. SameSite cookies. An allowlist for any user-supplied URL the server fetches. Rate limiting per account and per address, because per address alone is beaten by a botnet and per account alone lets one guess be sprayed across a million accounts.

And the audit log, which is not the application log. Append-only, in a separate store that the application's own credentials cannot rewrite, kept for years — who, what, which object, when, from where, and the outcome including denials. Denied attempts carry the most signal: a hundred denials from one account in a minute is the clearest alert you will ever get.

What ties it together is defence in depth, and I would define it as an assumption rather than a slogan. Every layer will eventually fail. The design question is what the next one stops. If the edge is breached, internal traffic is still authenticated. If a service is compromised, its credentials reach only its own data. If the database is dumped, the passwords are slow-hashed and the sensitive fields are separately encrypted.

And the failure mode I would want reviewed regularly is not a missing control. It is a control that exists in the design and was quietly worked around six months ago because it was inconvenient — and everything kept working perfectly the entire time, which is exactly why nobody noticed.“

9 Recall card

AUTHENTICATION is who you are, answered **ONCE** at the edge. **AUTHORISATION** is what you may do, answered **ON EVERY REQUEST FOR EVERY OBJECT** — and that is where real breaches live. The classic bug is insecure direct object reference: a valid user asks for `/invoices/8813` and gets it. The fix is not unguessable ids (that is hiding); it is `WHERE id = ? AND account_id = ?` in the **SAME** statement, and in multi-tenant systems **row-level security** below the application, so a missing filter returns nothing rather than everything.

Passwords: `bcrypt` / `scrypt` / `Argon2id`, salted, ~250 ms per hash. Never a fast hash. `SHA-256` at 10^{10} guesses/s cracks an 8-character lowercase password ($26^8 = 209$ billion) in **21 seconds**; `bcrypt` at 4 guesses/s/core takes ~1,650 years. The salt stops one crack breaking every shared password. Length beats complexity — two more characters is $\times 676$. And 250 ms of CPU a stranger can spend is a DoS vector, so rate-limit login hardest, per account AND per address.

JWT = stateless scale, **NO REVOCATION** until expiry. **Session** = a lookup, instant revocation. The practical shape: **15-minute access token** + **revocable refresh token**, so a stolen token is worth 15 minutes and disabling an account takes effect within 15. **TLS everywhere including internally (mTLS between services)** — a hard shell with a soft inside fails the moment anything gets in. Encryption at

rest protects against a stolen disk AND NOTHING ELSE, because the application holds the key; field-level encryption is the real defence and **costs you indexing**. **No secrets in the repo — history is forever; prefer short-lived workload-identity credentials, and envelope encryption with a master key that never leaves the KMS.**

The five that actually happen, with the mechanism: SQL injection → parameterised statements (query and values travel separately, so values are never parsed as SQL); **XSS → context-correct output encoding + CSP**; **CSRF → SameSite cookies + a token**; **SSRF → an allowlist of destinations, never a blocklist**; **credential stuffing → MFA, per-account and per-address limits, breached-password checks**. **WebAuthn is phishing-resistant by construction — it will not sign for the wrong domain.**

Defence in depth is an assumption: every layer will fail; the question is what the next one stops. Ask “**if this service is taken over tonight, what can the attacker reach?**” — not “do we trust it”. **The audit log is NOT the application log:** append-only, a separate store the app cannot rewrite, years of retention, and it records **denied** attempts, which carry the most signal. **5M writes/day × 500 B = 2.5 GB/day, 6.4 TB over seven years, ~\$147/month — cheap and non-negotiable.** **And the finding in most reviews is not a missing control; it is one that was quietly bypassed months ago while everything kept working.**

DSA topic: The combinatorics you actually need **System design topic:** Security in a design interview

Code these, in this order

One rule for the whole set: **count a tiny case by hand before you write a formula.** Three items. Four items. **An exhaustive count of a small case is the only check you have** on an answer too large to verify — and if your small cases come out 1, 2, 5, 14, you already know what you are looking at.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Unique Paths	LeetCode 62 (Medium)	That a path is a choice of which steps go down.
2	Pascal's Triangle II	LeetCode 119 (Easy)	One row in $O(n)$ space, by the multiplicative rule.
3	Unique Binary Search Trees	LeetCode 96 (Medium)	Catalan, arrived at through the split-and-multiply recurrence.
4	Combinations	LeetCode 77 (Medium)	Generating them, not counting them — and the size of the output.
5	Count Sorted Vowel Strings	LeetCode 1641 (Medium)	Stars and bars in disguise, or a four-line DP.
6	Number of Ways to Divide a Long Corridor	LeetCode 2147 (Hard)	The multiplication principle over independent gaps, modulo a prime.

On problem 1, do it twice

Solve it with dynamic programming, then with `C(m+n-2, m-1)`. **Check they agree for a 3 by 7 grid.** Then **add an obstacle in the middle** and say which of the two solutions survives. Say which one you would write in an interview and why the answer is not always the faster one.

On problem 2, watch the intermediate values

Print the running value at every step while computing row 100. **Record the largest one.** Compare it with the largest number in the row. Then rewrite the line as `result // (i+1) * (n-i)` and record what happens.

On problem 3, find Catalan yourself

Write the recurrence first — **pick the root, split the remaining nodes left and right, multiply**. Compute the first eight values. **Recognise the sequence**. Only then look up the closed form and check it agrees.

On problem 4, notice the difference

This asks you to *generate* the combinations, not count them. **Compute $C(20, 10)$ first and say how many lists that is**. Then say why a counting problem and a generating problem have completely different cost profiles, even with the same formula on the page.

Then the overflow drill

Compute $20!$ and $21!$. **Record which fits in a signed 64-bit integer**. Then write `nCr` the naive way and say what `C(21, 2)` would produce in Java. Then write the multiplicative version and record its largest intermediate value for `C(100, 50)`.

Then the float drill

Write `nCr` with `/` instead of `//`. **Run it on $C(30, 15)$, $C(50, 25)$ and $C(100, 50)$** . Record all three against the exact answers. Say at roughly how many digits it starts lying, and why the small cases are the dangerous part.

Then the four-cases drill

Take five items and pick three. **Compute all four cases** — order with and without repeats, no order with and without repeats. Get four different numbers. **Say the question that separates each pair**.

Then the over-counting drill

Count the three-letter strings from $\{A, B\}$ with at least one A, **using the wrong method first** — choose a position for the A, fill the rest freely. Record the wrong answer. Say which string was counted twice. Then do it by complement and say why that is nearly always easier.

Then the modulus drill

Build factorial and inverse-factorial tables to 200,000. **Time the build with one fast power per entry, and again with the walk-down**. Record both. Then say what breaks if `n` is bigger than the modulus.

Then the password drill

Compute how long an exhaustive search of all eight-character lowercase passwords takes **at ten billion guesses a second and at four guesses a second**. Then do it again for ten characters. Say which of length and complexity wins, and by how much.

Then the token drill

Say what a stolen access token is worth at three different lifetimes. **Then say how long it takes to lock somebody out** with a fifteen-minute access token, and with a thirty-day JWT and no denylist.

Then the authorisation drill

Take any endpoint you can imagine that returns a record by id. **Write the check that is usually missing**. Say why putting it in the query rather than beside the query matters, and what the database-level version of the same idea is called.

The four-cases drill

1. Give the two questions that pick the formula.
2. Give all four formulas and one example of each.
3. Derive nPr from first principles in one sentence.
4. Derive nCr from nPr in one sentence.

The safety drill

1. Say which factorial is the last one that fits in 64 bits.
2. Give the multiplicative form of nCr and the order of operations.
3. Say why the division is exact at every step.
4. Say what $\min(r, n-r)$ saves on $C(1000, 997)$.
5. Say what $/$ instead of $//$ does, and when you would notice.

The Pascal drill

1. Give the identity and the sentence it means.
2. Say what the row sums are and why.
3. Give the cost in time and space.
4. Say when Pascal is the right tool and when it is impossible.

The modulus drill

1. Say why the modulus is a hint about the answer's size.

2. Give the three-lookup formula.
3. Give the walk-down trick and what it saves.
4. Give the preparation cost and the per-query cost.
5. Say what constraint on `n` the method has, and what to name if it is violated.

The shapes drill

1. Grid paths, and the one-sentence reason.
2. Catalan, its formula, and five things it counts.
3. Stars and bars, with and without the “at least one” variant.
4. Arrangements with repeats, and the division argument.
5. Subsets, and the connection to counting in binary.

The two-words drill

1. Define authentication and authorisation, and say how often each is checked.
2. Name the most common serious flaw in real systems.
3. Give the fix, and say where it must live.
4. Give the database-level version of the same protection.

The passwords drill

1. Name three acceptable password hashes and one unacceptable one.
2. Say why fast is the wrong property.
3. Give the two crack times for an eight-character lowercase password.
4. Say what the salt does, separately from the slowness.
5. Say why the work factor is stored inside the hash.
6. Say why the login endpoint needs the strictest rate limit.

The tokens drill

1. Give the two ways to remember a login and what each costs.
2. Say the one thing a JWT cannot do.
3. Give the two-token shape and both lifetimes.
4. Say what a stolen access token is worth, and why.

The encryption drill

1. Say what TLS covers and why it applies inside the network too.
2. Say exactly what encryption at rest protects against.
3. Say what it does not protect against, and why.

4. Say what field-level encryption costs you.
5. Describe envelope encryption in four steps and say what rotation then means.

The attacks drill

1. Name five attacks and the mechanism of the fix for each.
2. Say what a parameterised statement does differently from escaping.
3. Say why an allowlist and not a blocklist.
4. Say why rate limits must be per account and per address.

The audit drill

1. Give four differences between an audit log and an application log.
2. Give the six fields every entry carries.
3. Say which entries carry the most signal, and why.
4. Say why it lives in a separate store.
5. Compute its volume and seven-year cost from a stated scale.

The break-it drill

For each, say what happens and whether anything reports it:

1. `factorial(n) // (factorial(r) * factorial(n-r))` in Java at `n = 21`.
2. `nCr` computed with floats, at `n = 100`.
3. Dividing before multiplying in the multiplicative form.
4. Pascal's triangle at `n = 100,000`.
5. Factorial tables under a modulus when `n` exceeds the modulus.
6. Multiplying two choices that are not independent.
7. An endpoint that checks the token but not the object's owner.
8. SHA-256 for passwords, with a salt.
9. A thirty-day JWT after an account is compromised.
10. An audit log the application's own credentials can rewrite.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many ways can you arrange these? Now with repetitions.* The two questions, all four cases derived rather than recalled, the division argument for over-counting, the MISSISSIPPI example, and why you never compute a factorial.
2. *Compute nCr for n up to 200,000, modulo $10^9 + 7$.* Why division needs an inverse, the two tables, the three-lookup query, the walk-down trick and what it saves, and the constraint on n .
3. *What are the security concerns in this design?* The four layers, the authorisation bug that actually happens with the fix in the query, the password arithmetic, token life-time as blast radius, the five attacks with mechanisms, and the audit log.

Before you move on

- [] I ask “does order matter” and “can things repeat” before writing anything.
- [] I can derive all four formulas rather than recall them.
- [] I can explain $nCr = nPr / r!$ in one sentence about over-counting.
- [] I know $21!$ overflows a 64-bit integer.
- [] I never compute a factorial to get nCr .
- [] I multiply before I divide, and I know why the division is exact.
- [] I use $\min(r, n-r)$ every time.
- [] I know what `/` instead of `//` does and when it starts lying.
- [] I can state Pascal’s identity as a sentence about one item.
- [] I know the row sums are 2^n and why.
- [] I know when Pascal’s triangle is right and when it is impossible.
- [] I can build factorial and inverse tables under a modulus.
- [] I know the walk-down trick and what it saves.
- [] I know the method needs n below the modulus, and what to name otherwise.
- [] I can explain grid paths as a choice of steps.
- [] I recognise 1, 2, 5, 14, 42 and know five things it counts.
- [] I can do stars and bars, both variants.
- [] I know over-counting is the usual error, and to count the complement.
- [] I check independence before multiplying.
- [] I can define authentication and authorisation and say how often each is checked.
- [] I know the most common serious flaw and where its fix must live.
- [] I know what row-level security buys.

- [] I can name three good password hashes and say why fast is wrong.
- [] I can give both crack times for an eight-character password.
- [] I know what the salt does, separately from the slowness.
- [] I know why login needs the strictest rate limit.
- [] I can give the trade between sessions and JWTs in one sentence.
- [] I know the two-token shape and both lifetimes.
- [] I know exactly what encryption at rest does and does not protect.
- [] I can describe envelope encryption and what rotation then means.
- [] I can name five attacks with the mechanism of each fix.
- [] I know why rate limits are per account and per address.
- [] I can list four ways an audit log differs from an application log.
- [] I know denied attempts carry the most signal.
- [] I know the review question is “what was quietly bypassed”, not “what is missing”.
- [] I answered all three questions above out loud.

Bits and maths revision and mock round

DATA STRUCTURES AND ALGORITHMS

Bits and maths revision and mock round

SYSTEM DESIGN

Cost: the constraint nobody mentions

Bits and maths revision and mock round

1 What this is, and why they ask it

This is the last day of bits and maths, and it is a **mock round**: four unseen problems under the clock, talked through out loud, with the full solutions afterwards.

The revision half is one thing — a **recognition procedure**. This phase gave you about a dozen tools. **The skill that matters now is not writing them. It is looking at a problem you have never seen and knowing, within thirty seconds, which one it wants.**

Because that is the actual failure mode here. **Nobody forgets what `n & (n - 1)` does.** What happens in the room is that a problem arrives with no bits in it anywhere, and the candidate spends nine minutes on a brute-force answer before noticing that “constant extra space” meant XOR.

They ask it because **these problems are short enough to be used as filters, and the recognition is the whole question.** Once you have said “everything appears twice except one, so XOR”, the code is four lines and two minutes. **The interview was decided in the sentence before the code.**

And because this phase is unusually well-signposted, if you know the signs. “Modulo $10^9 + 7$ ” means counting and modular inverses. “ $n \leq 20$ ” means bitmask. “O(1) extra space” with repeated values means XOR. “All primes below” means sieve. **The problems tell you what they want, and this lesson is about hearing it.**

By the end of this lesson you have the recognition table for the whole phase, a thirty-second procedure for an unseen problem, four solved mock problems with the reasoning written out as it would be spoken, and a self-scoring rule that tells you what to do next.

2 The story

The cooking at the hall was done by four women, and the one everybody deferred to was Sulochana, who was not the fastest and did not do any of the heavy work.

What she did was taste.

At about eleven each morning, whoever was on the sambar would carry a little of it over in a small steel bowl, and she would take some on the back of a spoon and hold it in her mouth for a moment. **Then she would say one thing.**

“Salt.” Or “it wants another ten minutes.” Or, once, in a voice that carried across the whole yard, **“who has put the jaggery in twice?”**

The girl who had come to help that season — somebody’s niece, seventeen — found this maddening. **She could taste it too.** She could tell perfectly well that a thing was not right. **What she could never do was say what.**

She said so, one morning, more sharply than she meant to.

Sulochana was not offended. She put the bowl down and asked a question instead.

“When it is wrong, how many things can it be?”

The girl started to answer, and then stopped, because she had never once thought about it that way.

So Sulochana counted them off on her fingers. **Salt. Sourness. Heat. Not enough time on the fire. Too much time on the fire. The dal not properly cooked before it went in. Too much water.**

Seven.

That was the list. **Twenty-six years of cooking for four hundred people at a sitting, and the whole list was seven things long.**

“You are tasting it and asking what is wrong,” she said. **“That is too big a question. You will stand there until the evening.”**

“I am tasting it and asking which of the seven. That is a small question, and it takes a moment.”

The girl started doing it that way. She was not good at it for about a year. **And then one morning in her second season she tasted a sambar and said “it went on the fire too early” before she had properly thought about it at all.**

3 The idea in plain English

Sulochana’s seven is the whole revision. You do not walk into a bits-and-maths problem asking “what is this?” — **you ask “which of the twelve?”**, and twelve is a small enough question to answer in thirty seconds.

The list

Here is the phase, in one place. Every tool, its trigger, and the one sentence that justifies it.

TRIGGER IN THE PROBLEM	TOOL	THE SENTENCE
"count the set bits" "is it a power of two" "clear the lowest bit"	$n \& (n-1)$ in a loop	subtracting 1 flips the lowest 1 and ones-fills below, so AND removes it
"find the lowest set bit" "walk a sparse mask"	$n \& -n$	$-n$ is $\sim n+1$, the same ripple from the other side
"appears twice except one" "0(1) extra space" + repeats	XOR the whole list	$a \wedge a = 0$, and order does not matter
"TWO appear once"	XOR, then split on both $\& -\text{both}$	a set bit in a^b means they DISAGREE there
"appears three times except one"	count each bit position mod 3	k copies $\rightarrow$ count mod k ; XOR is just $k = 2$
"subsets", " $n \leq 20$ "	for mask in $\text{range}(1 \ll n)$	a subset IS a number; $n \times 2^n$
"every subset of a mask"	$\text{sub} = (\text{sub}-1) \& \text{mask}$	3^n , not 4^n
"all primes below N "	sieve	cross off multiples; start at $p \times p$
"is this one number prime"	trial division to $\sqrt{n}$, step 6	if $n = a \times b$, they cannot both exceed $\sqrt{n}$
"largest common measure" "repeating cycle", "reduce a fraction", "tile exactly"	Euclid	a common divisor of a and b also divides $a \bmod b$
" a^b , b is huge" "modulo $10^9 + 7$ "	square and multiply	one step per BIT of the exponent
"modulo a prime" + a division	$a^{(m-2)} \bmod m$	Fermat; needs a PRIME modulus
"how many ways / arrangements" "the answer may be large"	nPr , nCr , stars and bars	does order matter; can things repeat
"1, 2, 5, 14, 42" in your own small cases	Catalan $C(2n, n)/(n+1)$	split into left and right and multiply

Read that table until the middle column arrives before you have finished reading the left one. That is what the year of tasting bought Sulochana's niece.

The thirty-second procedure

Four questions, in this order, on any unseen problem in this phase.

One: what do the constraints forbid? This is the highest-value question and most candidates skip it.

"0(1) extra space"	→ no map, no sort. If values repeat, XOR.
"n ≤ 20"	→ 2^n is a million. Bitmask.
"n ≤ 10^5, answer modulo 10^9 + 7"	→ counting. Precomputed factorials.
"1 ≤ k ≤ 10^9"	→ no loop over k. Fast power, or a formula.
"up to 10^6 values, many queries"	→ precompute once (sieve, prefix, table).

Two: is there a small set that repeats? Pairs cancel under XOR. Triples need counting modulo three. If the repeats are irregular, it is a map problem and not this phase.

Three: is the work per number, or per group? Sulochana's actual insight. Testing each number for primality is per number; sieving is per group. **Ashfaq's five messages instead of sixty calls is the same move.** If you find yourself planning a loop that asks the same question of every value, look for the crossing-off version.

Four: does the answer need dividing? If yes and there is a modulus, you need an inverse, and that sentence is worth saying out loud before you write anything, because it changes the shape of the whole solution.

The three that are really one idea

Worth noticing, because it compresses the phase.

n & (n-1)	"remove the lowest thing and recurse"
Euclid	"remove the biggest multiple and recurse"
fast power	"halve the exponent and recurse"

All three: make the problem strictly smaller by a structural step rather than by one unit, and you get logarithmic instead of linear.

→ That is the same sentence as binary search, and it is most of what this phase is actually teaching.

The four that fail silently

No exceptions, no crashes, just wrong numbers. These are the ones to check by habit.

1 << n - 1	is 1 << (n-1), not (1 << n) - 1
is_power_of_two(0)	True without the > 0 guard
Fermat on a composite modulus	returns a plausible non-inverse
nCr via factorials	overflows at 21! in any 64-bit language

Plus one that hangs rather than failing: `while n: n &= n - 1` on a negative Python integer.

4 The picture

The decision procedure, drawn:

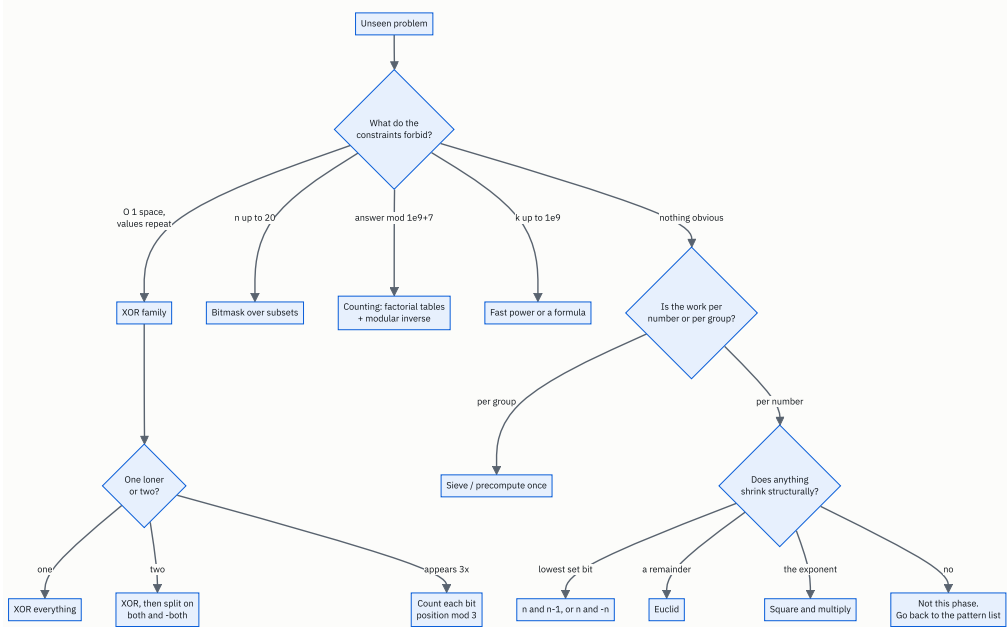


FIGURE 176.1

The most important box is the first one. Reading the constraints before thinking about the problem feels backwards and is the single highest-value habit in this phase, because the constraints are where the setter tells you the intended solution.

The four mock problems, and what each one is testing:

MOCK 1 Bitwise AND of Numbers Range LC 201, Medium
→ can you see that a RANGE of numbers has a common PREFIX, and that removing low bits is $n \& (n-1)$?

MOCK 2 Single Number II LC 137, Medium
→ can you notice that pairing fails, and reach for counting modulo three instead?

MOCK 3 Closest Prime Numbers in Range LC 2523, Medium
→ per group, not per number. And one detail about prime gaps that halves the runtime.

MOCK 4 Count Ways to Make Array With Product LC 1735, Hard
→ the capstone: sieve + factorise + stars and bars + modular inverse, all four in one problem.

RULE FOR ALL FOUR: say the recognition sentence out loud BEFORE writing a line. If you cannot, you are not ready to write.

5 The code, built step by step

Mock 1 — Bitwise AND of Numbers Range

Given two integers `left` and `right`, return the bitwise AND of every number in the inclusive range. $0 \leq \text{left} \leq \text{right} \leq 2^{31} - 1$.

The recognition. `right` can be two billion, so a loop over the range is out — that is the constraints question answering itself. **So the answer must be a property of the two endpoints.**

Say this out loud before writing: “ANDing a range keeps a bit only if that bit is 1 in every number in the range. If any bit position changes anywhere in the range, it becomes 0. So the answer is the common binary prefix of `left` and `right`, with zeros below it.”

```
def range_bitwise_and(left: int, right: int) → int:
    """AND of every number in [left, right]. It is the common binary PREFIX."""
    while right > left:
        right &= right - 1
    return right
```

Three lines, and they look like nothing. Here is why it is right. As long as `right` is still bigger than `left`, there is at least one number between them where the lowest set bit of `right` was 0 — so that bit cannot survive the AND, and `right &=`

`right - 1` removes exactly it. When `right` has been reduced to `left` or below, what remains is the shared prefix.

The more common solution shifts both numbers right until they are equal and then shifts back, and it is equally correct. Say whichever one you can justify — the justification is the answer, not the loop.

Mock 2 — Single Number II

Every element appears three times except one, which appears once. Find it. Linear time, constant extra space.

The recognition. “Constant extra space” rules out a frequency map, and repeats mean the XOR family — but pairs do not cancel here, because three of a thing is not zero.

Say this: “XOR is really ‘count the ones in each bit column modulo two’. The problem is modulo three, so I count the columns myself and take the remainder modulo three.”

```
def single_number_twice_over(numbers: list[int]) → int:
    """Every value appears three times except one. Count each bit position mod 3."""
    answer = 0
    for position in range(32):
        column = sum((value >> position) & 1 for value in numbers)
        if column % 3:
            answer |= 1 << position
    return answer - (1 << 32) if answer >> 31 else answer
```

Every value appearing three times contributes 3 or 0 to each column, so the column total is a multiple of three plus the loner’s bit. The remainder is the loner’s bit, and thirty-two of those rebuild the number.

The last line is the Python tax and you should say why it is there: Python integers have no top bit, so setting bit 31 makes a large positive number rather than a negative one.

There is a one-pass version, and knowing it exists is worth a mark:

```
def single_number_state_machine(numbers: list[int]) → int:
    """One pass. `ones` holds bits seen once, `twos` bits seen twice; three clears both."""
    ones = twos = 0
    for value in numbers:
        ones = (ones ^ value) & ~twos
        twos = (twos ^ value) & ~ones
    return ones
```

Thirty-two times faster and much harder to justify at a whiteboard. Write the column version, mention this one. That is the right trade under a clock, and saying so is itself a good signal.

Mock 3 — Closest Prime Numbers in Range

Given `left` and `right`, find the pair of primes $p < q$ in that range with the smallest $q - p$. If there is no such pair, return `[-1, -1]`. $1 \leq \text{left} \leq \text{right} \leq 10^6$.

The recognition. “Up to a million” and “all the primes in a range” is the sieve, immediately — this is the per-group question from section 3. Testing each of a million numbers individually is about a thousand divisions each; the sieve does the whole range in roughly 2.8 million operations.

```
def closest_primes(left: int, right: int) → list[int]:
    """The closest pair of primes in [left, right]. Sieve once, then one scan."""
    if right < 2:
        return [-1, -1]
    prime = [True] * (right + 1)
    prime[0] = prime[1] = False
    p = 2
    while p * p ≤ right:
        if prime[p]:
            prime[p * p:right + 1:p] = [False] * len(range(p * p, right + 1, p))
        p += 1
```

The sieve, unchanged from day 174. `p * p` to start, `p * p ≤ right` to stop, and the slice assignment so the crossing-off happens in C rather than in interpreted Python.

```
best = [-1, -1]
best_gap = float("inf")
previous = -1
for value in range(max(left, 2), right + 1):
    if not prime[value]:
        continue
    if previous ≠ -1 and value - previous < best_gap:
        best_gap = value - previous
        best = [previous, value]
        if best_gap ≤ 2:
            break
    previous = value
return best
```

One scan, keeping the previous prime seen. And the `if best_gap ≤ 2: break` is the detail worth pointing at. Apart from the pair (2, 3), two primes can never be closer than two apart, because one of any two consecutive numbers above 2 is even. So a gap of 2 is the best possible and you can stop looking. On a range like `[1, 1000000]` that turns a million-step scan into a three-step one.

Saying that out loud is worth more than the code, because it shows you thought about the structure of the answer and not only about the search.

Mock 4 — Count Ways to Make Array With Product

For each query $[n, k]$, count the arrays of length n of positive integers whose product is k . Return each answer modulo $10^9 + 7$. $n, k \leq 10^4$, up to 10^4 queries.

The recognition, and this is the whole problem. Say it before writing anything:

“A product is decided independently for each prime. If $k = 2^3 \times 3^2$, then I have to share three copies of the 2 among n positions, and separately two copies of the 3 among n positions — and the two choices do not affect each other, so I multiply. Sharing e identical copies among n labelled positions is stars and bars: $C(e + n - 1, n - 1)$.”

That sentence is four days of this phase in one paragraph — factorisation, independence, stars and bars, and a modulus.

```
def build_tables(limit: int) → tuple[list[int], list[int], list[int]]:
    """One smallest-prime-factor sieve, and factorials with their inverses."""
    spf = list(range(limit + 1))
    p = 2
    while p * p ≤ limit:
        if spf[p] == p:
            for multiple in range(p * p, limit + 1, p):
                if spf[multiple] == multiple:
                    spf[multiple] = p
        p += 1
```

The smallest-prime-factor sieve from day 174, so that factorising any k afterwards takes at most $\log_2(k)$ steps — fourteen for ten thousand — rather than a hundred trial divisions each, ten thousand times.

```
fact = [1] * (limit + 1)
for i in range(1, limit + 1):
    fact[i] = fact[i - 1] * i % MOD
inverse_fact = [1] * (limit + 1)
inverse_fact[limit] = pow(fact[limit], MOD - 2, MOD)
for i in range(limit, 0, -1):
    inverse_fact[i - 1] = inverse_fact[i] * i % MOD
return spf, fact, inverse_fact
```

Factorials and inverse factorials from day 175, with the walk-down so the whole inverse table costs one fast power. The limit has to cover $e + n - 1$, and since n reaches 10^4 and the largest exponent is 13 (because $2^{13} = 8192$ and 2^{14} exceeds 10^4), 20,000 is comfortably enough.

```
def ways_to_fill_array(queries: list[list[int]]) → list[int]:
    """Arrays of length n whose product is k. Share each prime's copies out by stars and bars."""
    limit = 20_000
    spf, fact, inverse_fact = build_tables(limit)

    def choose(n: int, r: int) → int:
        if r < 0 or r > n:
            return 0
        return fact[n] * inverse_fact[r] % MOD * inverse_fact[n - r] % MOD

    answers = []
    for n, k in queries:
        ways = 1
        while k > 1:
            prime = spf[k]
            power = 0
            while k % prime == 0:
                k //= prime
                power += 1
            ways = ways * choose(power + n - 1, n - 1) % MOD
        answers.append(ways)
    return answers
```

The loop is the sentence. Pull out one prime, count how many copies of it there are, share those copies among the `n` positions with stars and bars, and multiply the ways together because the primes are independent.

Note `k = 1` needs no special case — the `while k > 1` loop simply does not run, `ways` stays 1, and the answer is correct: **there is exactly one array of `n` ones.**

The complete solution

```
"""Day 176 - the bits and maths mock round. Four unseen problems, solved."""

from __future__ import annotations

MOD = 1_000_000_007

# ----- mock 1
def range_bitwise_and(left: int, right: int) → int:
    """AND of every number in [left, right]. It is the common binary PREFIX."""
    while right > left:
        right &= right - 1
    return right

def range_bitwise_and_brute(left: int, right: int) → int:
    """The definition, for checking. Unusable when the range is large."""
    result = right
    for value in range(left, right):
        result &= value
    return result

# ----- mock 2
def single_number_twice_over(numbers: list[int]) → int:
    """Every value appears three times except one. Count each bit position mod 3."""
    answer = 0
    for position in range(32):
        column = sum((value >> position) & 1 for value in numbers)
        if column % 3:
            answer |= 1 << position
    return answer - (1 << 32) if answer >> 31 else answer

def single_number_state_machine(numbers: list[int]) → int:
    """One pass. 'ones' holds bits seen once, 'twos' bits seen twice; three clears both."""
    ones = twos = 0
    for value in numbers:
        ones = (ones ^ value) & ~twos
        twos = (twos ^ value) & ~ones
    return ones

# ----- mock 3
def closest_primes(left: int, right: int) → list[int]:
    """The closest pair of primes in [left, right]. Sieve once, then one scan."""
    if right < 2:
        return [-1, -1]
    prime = [True] * (right + 1)
    prime[0] = prime[1] = False
    p = 2
    while p * p ≤ right:
        if prime[p]:
            prime[p * p:right + 1:p] = [False] * len(range(p * p, right + 1, p))
            p += 1

    best = [-1, -1]
    best_gap = float("inf")
    previous = -1
    for value in range(max(left, 2), right + 1):
        if not prime[value]:
            continue
        if previous ≠ -1 and value - previous < best_gap:
            best_gap = value - previous
            best = [previous, value]
            if best_gap ≤ 2:
                break
        previous = value
    return best
```

```

# ----- mock 4
def build_tables(limit: int) → tuple[list[int], list[int], list[int]]:
    """One smallest-prime-factor sieve, and factorials with their inverses."""
    spf = list(range(limit + 1))
    p = 2
    while p * p ≤ limit:
        if spf[p] == p:
            for multiple in range(p * p, limit + 1, p):
                if spf[multiple] == multiple:
                    spf[multiple] = p
            p += 1

    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    inverse_fact = [1] * (limit + 1)
    inverse_fact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        inverse_fact[i - 1] = inverse_fact[i] * i % MOD
    return spf, fact, inverse_fact

def ways_to_fill_array(queries: list[list[int]]) → list[int]:
    """Arrays of length n whose product is k. Share each prime's copies out by stars and bars."""
    limit = 20_000
    spf, fact, inverse_fact = build_tables(limit)

    def choose(n: int, r: int) → int:
        if r < 0 or r > n:
            return 0
        return fact[n] * inverse_fact[r] % MOD * inverse_fact[n - r] % MOD

    answers = []
    for n, k in queries:
        ways = 1
        while k > 1:
            prime = spf[k]
            power = 0
            while k % prime == 0:
                k /= prime
                power += 1
            ways = ways * choose(power + n - 1, n - 1) % MOD
        answers.append(ways)
    return answers

if __name__ == "__main__":
    print("MOCK 1 - Bitwise AND of Numbers Range")
    for lo, hi in ((5, 7), (0, 0), (1, 2147483647), (12, 15), (600, 700)):
        fast = range_bitwise_and(lo, hi)
        print(f" [{lo}, {hi}] → {fast} ({fast:b} in binary)")
    print(" checking against the definition on small ranges:")
    ok = all(range_bitwise_and(a, b) == range_bitwise_and_brute(a, b)
            for a in range(0, 60) for b in range(a, 60))
    print(f" all pairs in [0, 60): {ok}")

    print()
    print("MOCK 2 - Single Number II")
    for data in ([2, 2, 3, 2], [0, 1, 0, 1, 0, 1, 99], [-2, -2, 1, -2],
                [30000, 500, 100, 30000, 100, 30000, 100]):
        print(f" {str(data):<58} → {single_number_twice_over(data)}"
              f" (state machine: {single_number_state_machine(data)})")

    print()
    print("MOCK 3 - Closest Prime Numbers in Range")
    for lo, hi in ((10, 19), (4, 6), (1, 1000), (19, 31), (999, 1000)):
        print(f" [{lo}, {hi}] → {closest_primes(lo, hi)}")

    print()
    print("MOCK 4 - Count Ways to Make Array With Product")

```

```

qs = [[2, 6], [5, 1], [73, 660], [1, 1], [2, 4], [10, 1024]]
for q, answer in zip(qs, ways_to_fill_array(qs)):
    print(f" n={q[0]:<4} k={q[1]:<6} → {answer}")
print(" n=2, k=6 by hand: [1,6] [2,3] [3,2] [6,1] = 4")
print(" n=2, k=4 by hand: [1,4] [2,2] [4,1] = 3")

print()
print("VERIFICATION")
import random
from math import isqrt

bad = 0
for _ in range(300):
    a = random.randint(0, 3000)
    b = random.randint(a, a + 400)
    if range_bitwise_and(a, b) != range_bitwise_and_brute(a, b):
        bad += 1
    loner = random.randint(-10_000, 10_000)
    triples = [random.randint(-10_000, 10_000) for _ in range(random.randint(1, 20))]
    data = triples * 3 + [loner]
    random.shuffle(data)
    if single_number_twice_over(data) != loner:
        bad += 1

def slow_is_prime(n: int) → bool:
    return n > 1 and all(n % d for d in range(2, isqrt(n) + 1))

for _ in range(200):
    lo = random.randint(1, 500)
    hi = lo + random.randint(0, 300)
    primes = [v for v in range(lo, hi + 1) if slow_is_prime(v)]
    if len(primes) < 2:
        expected = [-1, -1]
    else:
        expected = min(
            ([primes[i], primes[i + 1]] for i in range(len(primes) - 1)),
            key=lambda pair: (pair[1] - pair[0], pair[0]),
        )
    if closest_primes(lo, hi) != expected:
        bad += 1

def brute_ways(n: int, k: int) → int:
    def count(pos: int, remaining: int) → int:
        if pos == n - 1:
            return 1
        total = 0
        for d in range(1, remaining + 1):
            if remaining % d == 0:
                total += count(pos + 1, remaining // d)
        return total
    return count(0, k)

for _ in range(120):
    n = random.randint(1, 4)
    k = random.randint(1, 60)
    if ways_to_fill_array([[n, k]])[0] != brute_ways(n, k) % MOD:
        bad += 1
print(f" {bad} mismatches across 300 + 200 + 120 randomly generated cases")

```

Running it:

```

MOCK 1 - Bitwise AND of Numbers Range
[5, 7] → 4 (100 in binary)
[0, 0] → 0 (0 in binary)
[1, 2147483647] → 0 (0 in binary)
[12, 15] → 12 (1100 in binary)
[600, 700] → 512 (1000000000 in binary)
checking against the definition on small ranges:
  all pairs in [0, 60]: True

MOCK 2 - Single Number II
[2, 2, 3, 2] → 3 (state machine: 3)
[0, 1, 0, 1, 0, 1, 99] → 99 (state machine: 99)
[-2, -2, 1, -2] → 1 (state machine: 1)
[30000, 500, 100, 30000, 100, 30000, 100] → 500 (state machine: 500)

MOCK 3 - Closest Prime Numbers in Range
[10, 19] → [11, 13]
[4, 6] → [-1, -1]
[1, 1000] → [2, 3]
[19, 31] → [29, 31]
[999, 1000] → [-1, -1]

MOCK 4 - Count Ways to Make Array With Product
n=2 k=6 → 4
n=5 k=1 → 1
n=73 k=660 → 50734910
n=1 k=1 → 1
n=2 k=4 → 3
n=10 k=1024 → 92378
n=2, k=6 by hand: [1,6] [2,3] [3,2] [6,1] = 4
n=2, k=4 by hand: [1,4] [2,2] [4,1] = 3

VERIFICATION
0 mismatches across 300 + 200 + 120 randomly generated cases

```

Look at mock 1, `[1, 2147483647]` : the answer is 0. A loop over that range would run two billion times. **The three-line version runs thirty-one times.**

Look at mock 3, `[1, 1000000]` : the answer is `[2, 3]` . The scan finds it in three steps and stops, because a gap of 2 cannot be beaten. **Without that early exit it would walk all 78,498 primes.**

And look at mock 4, `n = 10, k = 1024` : 92,378. `1024` is `2^10` , so it is a single prime with ten copies shared among ten positions — `C(19, 9)` , which is exactly 92,378. One line of stars and bars, and no enumeration of anything.

6 What it costs

The whole phase, in one table.

TOOL	TIME	SPACE
count set bits (naive)	$O(\text{bits}) = O(32)$	$O(1)$
Kernighan	$O(\text{set bits})$	$O(1)$
power-of-two test	$O(1)$	$O(1)$
subset enumeration	$O(n \times 2^n)$	$O(1)$ per mask
submask enumeration	$O(3^n)$	$O(1)$
XOR one loner	$O(n)$	$O(1)$
XOR two loners	$O(n)$, two passes	$O(1)$
count mod 3	$O(32n)$	$O(1)$
XOR of a range	$O(1)$	$O(1)$
maximum XOR pair	$O(32n)$	$O(n)$
primality, one number	$O(\sqrt{n})$	$O(1)$
sieve	$O(n \log \log n)$	$O(n)$
factorise with spf	$O(\log n)$	$O(1)$
Euclid	$O(\log \min(a,b))$	$O(1)$
fast power	$O(\log \text{exponent})$	$O(1)$
modular inverse	$O(\log m)$	$O(1)$
nCr multiplicative	$O(\min(r, n-r))$	$O(1)$
nCr with tables	$O(1)$ per query	$O(n)$ once
Pascal's triangle	$O(n^2)$	$O(n^2)$

The four mock problems, counted.

MOCK 1

the definition: $\text{right} - \text{left} + 1$ iterations
[1, 2147483647] $\rightarrow$ 2,147,483,647 iterations
 $n \ \& \ (n-1)$: one iteration per set bit of `right`
[1, 2147483647] $\rightarrow$ 31 iterations
 $\rightarrow$ about 70 million times less work.

MOCK 2

32 positions $\times$ n values = $32n$
 $n = 30,000 \rightarrow 960,000$ operations
state machine: n operations
 $n = 30,000 \rightarrow 30,000$
 $\rightarrow 32x$, and the slower one is the one to write.

MOCK 3

sieve to 10^6 : ~ 2.8 million operations
scan: stops at the first gap of 2
[1, 10^6] $\rightarrow$ 3 primes examined
[999983, 10^6] $\rightarrow$ the whole (tiny) range
against testing each number:
 $10^6 \times \sim 1,000$ divisions = $10^9 \rightarrow \sim 350x$ slower

MOCK 4

preparation (once):
spf sieve to 20,000 $\sim 60,000$ operations
factorials + inverses $\sim 40,000$ operations
per query:
factorise $k \leq \log_2(10^4) = 13$ steps
one nCr per distinct prime ≤ 5 primes for $k \leq 10^4$
($2 \times 3 \times 5 \times 7 \times 11 = 2,310$;
 $\times 13 = 30,030 > 10^4$)
 $\rightarrow$ about 20 operations

 $10,000$ queries: $100,000 + 200,000 = \sim 300,000$ operations

the brute force - enumerate every array - is
exponential in n and impossible at $n = 10^4$.

One number worth carrying out of this phase.

Almost every tool here turns $O(n)$ into $O(\log n)$,
or $O(n^2)$ into $O(n)$, by finding structure:

2,147,483,647	$\rightarrow$ 31	(bit prefix)
1,000,000,000	$\rightarrow$ 30	(fast power)
10^{18}	$\rightarrow$ 90	(Euclid)
10^9 loop	$\rightarrow$ 1	(range XOR formula)

$\rightarrow$ When a constraint is 10^9 , the intended answer is
almost never a loop. That single observation is
worth more than any individual trick.

7 The traps

The phase's silent failures, gathered in one place. None of these raise.

The shift precedence trap.

```
1 << n - 1      is 1 << (n - 1)    a single bit
(1 << n) - 1    n ones
```

Every mask you build. Two brackets.

The power-of-two guard.

```
is_power_of_two(0)  without `n > 0` → True
```

Because `0 & -1` is `0`.

Assuming the repeat count.

```
single_number([2, 2, 2, 3]) → 1    a plausible wrong number
single_number([2, 2, 3])    → 3    correct
```

An even number of copies cancels; an odd number does not. The hand-made test passes and the real input fails.

Fermat on a composite modulus.

```
inverse of 3 mod 10: Fermat gives 1
                    check: 3 x 1 = 3, not 1
                    the true answer is 7
```

No error. A small plausible number that is not an inverse.

Factorials in `nCr`.

```
21! = 51,090,942,171,709,440,000  overflows a signed 64-bit int
```

So `C(21, 2)` — whose answer is 210 — comes out as nonsense in Java or C++ .
Build up one factor at a time, multiplying before dividing.

`/` instead of `//` .

```
ncr_float(50, 25) = 126410606437752.0          correct
ncr_float(100, 50) = 1.0089134454556418e+29
exact             = 100891344545564193334812497256
```

Right on small inputs, quietly wrong past about fifteen digits.

And the one that hangs instead of failing.

```
count_set_bits(-8)      # while n: n &= n - 1
```

```
-8 & -9 = -16
-16 & -17 = -32
...
```

Python integers are arbitrary width, so a negative number behaves as though it has infinitely many leading ones. The loop never terminates. A hang is worse than a crash, because a crash tells you where it was.

The errors that do raise, so you recognise them fast.

```
1 << -1
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: negative shift count
```

```
1 << (6 / 2)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for <<: 'int' and 'float'
```

```
5 % 0
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer modulo by zero
```

```
prime = [True] * (10 ** 11)
```



```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

The last one is the phase's honest limit: the sieve runs out of memory long before it runs out of time.

And two mock-specific ones.

Mock 1, forgetting that `left` can equal `right`. `range_bitwise_and(7, 7)` must be `7`, and the loop condition `while right > left` handles it by not running. A `while right ≥ left` would loop to zero and return the wrong answer for every input.

Mock 3, forgetting that (2, 3) is the only prime pair with a gap of one. If you break on `gap ≤ 1` you will still be correct, because you cannot do better than 1 — but if you write `if best_gap == 2: break` without considering (2, 3), you have not broken anything, only missed an earlier exit. **The dangerous version is breaking on the first pair you find at all**, which returns the first pair rather than the closest.

8 In the interview

How it gets asked

This is a mock round, so the framing is the framing of a real one.

- “I have two problems for you. Take as long as you need on the first, but talk as you go.”
- “Before you write anything, tell me your approach.” — the sentence, not the code.
- “What is the time complexity? And the space?” — always, after every problem.
- “Can you do better?” — sometimes a real question, sometimes a check on your confidence.
- “What would you test?” — and the edge cases in this phase are always 0, 1, 2, and a negative.

The first ninety seconds

Use the same shape on every problem in this phase. Four sentences, in this order.

“Let me read the constraints first.”

“right goes up to two billion, so a loop over the range is out. Whatever the answer is, it has to come from the two endpoints.”

“Here is what I think the structure is.”

“ANDing a range keeps a bit only where every number in the range has it. So the answer is the common binary prefix of left and right, and everything below the prefix is zero.”

“So the plan is...”

“Strip the lowest set bit off right until it is no bigger than left. right &= right - 1 does exactly that, one bit at a time.”

“And the cost is...”

“One iteration per set bit — at most thirty-one for a 32-bit number — and constant space. I will check left == right and both endpoints zero before I finish.”

That is under a minute, and it is worth more than the code that follows it. An interviewer who hears the constraints question, the structure sentence, the plan and the cost knows what your process is, and can help you if the plan is wrong — which is the whole reason to say it before writing.

The follow-ups

“Every element appears three times except one. Can you do it in constant space?”

“Yes, and the route to it is worth saying rather than the answer.

Constant space rules out a frequency map, so I am in the XOR family. But XOR will not work directly, because three of a thing is not zero — pairing cancels, and these come in threes.

So I go one level down. XOR is really asking, independently in each bit column, whether the count of ones is odd — that is counting modulo two. My problem is modulo three, so I do the counting myself.

For each of the thirty-two positions, I count how many numbers have a one there. Every value that appears three times contributes either three or zero to that column, so the total is a multiple of three plus the loner’s contribution. Take the remainder modulo three and I have the loner’s bit.

Thirty-two columns, one pass over the list each, so $O(32n)$ — linear, with a real constant. Constant space.

The general form is nicer than the special case: k copies means counting modulo k . Twice is $k = 2$, and modulo two is exactly what XOR does for free.

There is a one-pass version using two variables as a small state machine, tracking bits seen once and bits seen twice, and it is thirty-two times faster. I would mention it and still write the column version, because I can justify the column version at a whiteboard and the state machine is easy to write subtly wrong. Under a clock I would rather be explainable than clever.

One Python detail I would say out loud: I have to fix the sign at the end, subtracting 2^{32} when bit 31 is set, because Python integers have no top bit and would otherwise return four billion instead of minus three.”

“How would you find all the primes in a range up to a million, and then the closest pair?”

“Two separate questions, and the second one has a nice ending.

For the primes: sieve, not testing each number. That is the difference between asking a question per number and crossing off per group. Testing each of a million numbers is roughly a thousand divisions each — a billion operations. **The sieve is about 2.8 million,** so a few hundred times faster.

Crossing off starts at $p \times p$, because every smaller multiple of p already has a smaller prime factor and has gone. And it stops when $p \times p$ exceeds the limit, because any composite has a factor at or below its square root.

The thing that limits the sieve is memory rather than time. A boolean list for ten million is fine; for a billion I would use a bit array or a segmented sieve.

For the closest pair: one scan, keeping the previous prime. But there is a fact about the answer that makes it much faster. Apart from 2 and 3, two primes can never be closer than two apart, because one of any two consecutive numbers above two is even. **So a gap of two is the best result possible, and I can stop the moment I see one.**

On a range like one to a million that turns a scan of 78,498 primes into three steps.

I would say that out loud before writing the loop, because it is a statement about the shape of the answer rather than about the search, **and that is the kind of observation the question is actually testing.**“

“For each query n and k , count the arrays of length n whose product is k .”

“The whole problem is one sentence, and I want to get it right before I write anything.

A product is decided independently for each prime. If k is 2^3 times 3^2 , then the array is fixed by deciding how the three copies of 2 are shared among the n positions, and separately how the two copies of 3 are shared. The two decisions do not affect each other, so the number of arrays is the product of the two counts.

And sharing e identical copies among n labelled positions, where a position may get none, is stars and bars: $C(e + n - 1, n - 1)$.

So: factorise k , and for each prime take one nCr and multiply them together.

For the implementation, three pieces from earlier in the phase. A smallest-prime-factor sieve to ten thousand, so factorising any k afterwards is at most fourteen steps rather than a hundred trial divisions. Factorial and inverse-factorial tables under the modulus, with the inverses built by one fast power and a walk down. Then each nCr is three lookups.

The tables have to cover $e + n - 1$. n reaches ten thousand and the biggest exponent is thirteen, since 2^{13} is 8,192 and 2^{14} is past the limit — so twenty thousand is plenty.

Cost: about a hundred thousand operations of preparation, then roughly twenty per query. Ten thousand queries is well under a second. The brute force is enumerating arrays, which is exponential in n and impossible.

And I would check $k = 1$ explicitly — the factorising loop simply does not run and the answer is one, which is right: there is exactly one array of n ones.“

The model answer

“How do you approach a problem in this area when you have not seen it before?”

“I ask which of about twelve it is, rather than asking what it is. That is the difference between standing there for nine minutes and starting in thirty seconds.

My first question is always what the constraints forbid, and I read them before I think about the problem. This is the habit I would most want to be judged on, because in this area the setter tells you the intended solution in the constraints. ‘ $O(1)$ extra space’ with repeated values means XOR. ‘ $n \leq 20$ ’ means a bitmask over subsets. ‘Modulo $10^9 + 7$ ’ means counting and modular inverses. ‘k up to a billion’ means no loop over k — a fast power or a closed form.

The second question is whether the work is per number or per group. Testing each of a million numbers for primality is per number; sieving is per group, and it is hundreds of times faster. Whenever I catch myself planning a loop that asks the same question of every value, I look for the crossing-off version.

The third is whether something shrinks structurally. `n & (n - 1)` removes the lowest set bit. Euclid removes the biggest multiple. Fast power halves the exponent. All three are the same move — make the problem smaller by a structural step rather than by one unit — and all three turn linear into logarithmic. That is the same sentence as binary search, and it is most of what this phase is really teaching.

The fourth is whether the answer needs dividing, because under a modulus that means an inverse, and it changes the shape of the whole solution.

Then I say the recognition sentence out loud before writing a line. *‘ANDing a range keeps only the common prefix.’ ‘A product is decided independently per prime, so I multiply the counts.’* If I cannot say that sentence, I am not ready to write code, and writing anyway is how people lose twenty minutes.

On correctness, I check the same four edge cases every time in this area: zero, one, two, and a negative. Nearly every silent failure here lives at one of those — zero passing the power-of-two test, a negative hanging the set-bit loop, `left == right` in a range problem.

And the last thing, which is a judgement rather than a technique. Where there is a clever version and an explainable version — the state machine against counting columns, the trie against the greedy prefix — I write the

explainable one and say the clever one exists. Under a clock, being able to justify what I wrote is worth more than a constant factor, and the interviewer is grading the reasoning, not the runtime.”

9 Recall card

Ask “which of the twelve”, not “what is this”. The triggers: “O(1) space” + repeats → XOR (one loner: XOR all; two: XOR then split on both & -both; three copies: count each column mod 3). “ $n \leq 20$ ” → bitmask over `range(1 << n)`. “all primes below N” → sieve (per group, not per number). “one number prime?” → trial division to $\sqrt{n}$. “modulo 10^9+7 ” → factorial tables + Fermat inverse. “exponent up to 10^9 ” → square and multiply. “how many ways” → does order matter, can things repeat.

Read the constraints BEFORE thinking about the problem — that is where the setter names the intended solution. Then: is the work per number or per group? (sieve vs test each) — does anything shrink structurally? — does the answer need dividing? `n & (n-1)`, Euclid and fast power are one idea: shrink by a structural step, not by one unit, and linear becomes logarithmic. When a limit is 10^9 , the answer is almost never a loop.

Say the recognition sentence out loud before writing a line. “ANDing a range keeps only the common prefix.” “A product is decided independently per prime, so I multiply the counts.” If you cannot say the sentence, you are not ready to write — and writing anyway is how twenty minutes disappear.

The silent failures, none of which raise. `1 << n - 1` is `1 << (n-1)`. `is_power_of_two(0)` is True without the guard. Plain XOR is right whenever repeat counts happen to be even — `[2,2,2,3]` gives 1. Fermat on a composite modulus returns a plausible non-inverse. `21!` overflows 64 bits, so `C(21,2) = 210` comes out as nonsense. `/` instead of `//` in `nCr` lies past ~15 digits. And `while n: n &= n-1` on a negative hangs, which is worse than crashing.

Edge cases in this phase are always 0, 1, 2 and a negative. Where there is a clever version and an explainable one — the ones/twos state machine against counting columns, a trie against the greedy prefix — write the explainable one and say the clever one exists. Under a clock, being able to justify what you wrote beats a constant factor, because the reasoning is what is being graded.

Cost: the constraint nobody mentions

1 What this is, and why they ask it

Every architecture has a monthly bill, and almost no candidate ever says what it is.

Cost is a design constraint exactly like latency or availability. A design that meets every requirement and costs forty times what the business earns is not a good design that happens to be expensive. **It is a wrong design**, in the same way that a design that loses data is wrong.

And the money is almost never where people expect. It is rarely the compute. It is the data moving between machines, the logs nobody reads, the environment that has been running since a project was cancelled, and the replica that exists because somebody was nervous eighteen months ago.

They ask it because **it separates people who have owned a system from people who have only designed one.** Anybody can add a caching layer. **Very few candidates can say that the caching layer costs about five hundred dollars a month and saves two thousand in data transfer**, which is the sentence that makes the decision obvious rather than aesthetic.

And because it is the last unclaimed differentiator in a design interview. By the final ten minutes you and the other candidates have all drawn a load balancer, a few services, a database and a cache. **“This runs at about thirty thousand a month, and here are the two changes that halve it” is something the interviewer will almost certainly not have heard that week.**

By the end of this lesson you can price an architecture from first principles, name the four line items that dominate real bills, explain why data transfer is the one that surprises everybody, choose between on-demand, committed and spot capacity, and give a cost per user and per thousand requests for a system you have just drawn.

2 The story

The bill came every two months, and for eleven years Ramesh had paid it without reading it.

Then one cycle it went from about four thousand rupees to nearly seven, and he read it very carefully indeed, **and it told him nothing at all.** It was one number, and a date.

He assumed it was the copier. It was the biggest thing in the shop. It ran all day. It got warm enough that he had put a small fan behind it.

His nephew, who fitted air conditioning for a firm in the city, came on a Sunday with a little meter that clipped round a wire. **They went round the shop and measured everything, one thing at a time,** which took most of the afternoon and involved moving a great deal of furniture.

The copier was not the problem. It was large, but it only actually drew anything while it was copying, and that was perhaps two hours spread across the eleven the shop was open.

Three things were the problem, and not one of them was doing any work at all.

The fridge in the back room, which had held cold drinks in 2016 and had held nothing since, and had been running, empty, for four years.

The tube lights over the shelves at the back — eight of them, on from seven in the morning until he locked up — in a part of the shop no customer had walked into in months.

And the second copier. The one he kept in case the first one failed. It was on standby. **It had been on standby for two years and eight months.**

Together they came to more than the copier.

That afternoon he switched the fridge off at the wall, took six of the eight tubes out, and put the standby machine on its own switch that he turned on only when he needed it.

The next bill was three thousand eight hundred. Lower than before the jump.

What he said afterwards, to anybody who would listen, was not really about electricity.

“For eleven years I paid one number. And the whole time, more than half of it was for things that were not doing anything — and I could not see it, because nobody had ever shown me the bill broken up.”

3 The idea in plain English

Ramesh's shop is a cloud bill. One number, no breakdown, and the largest share going to things that are switched on and doing nothing. The meter on the wire is cost allocation. And the empty fridge is the most common finding in every cost review ever conducted.

The four line items

Almost every bill is dominated by four things, and they are worth naming in this order.

COMPUTE	machines, containers, functions → the one everybody thinks about → usually 30-50% of the bill
STORAGE	disks, object storage, backups, snapshots → cheap per gigabyte, and it only ever grows
DATA TRANSFER	bytes moving between places → the one that surprises people → free IN, charged OUT, and charged BETWEEN your own machines
MANAGED SERVICES	databases, queues, search, observability → expensive per unit, cheap in salaries → the observability bill is often the single largest line

And a fifth that is not on the bill at all: people. An engineer costs more per month than a substantial fleet of machines, which is why “we will build it ourselves to save money” is usually wrong, and why saying so is a mark of judgement.

Data transfer, and why it surprises everybody

This is the part worth learning properly, because it is where the shock is.

INTO the cloud	free
OUT to the internet	~\$0.09 per GB
BETWEEN availability zones	~\$0.01 per GB, EACH WAY
BETWEEN regions	~\$0.02 per GB
WITHIN one zone, private address	free

Read the second line again. Storing a gigabyte for a month costs about two and a third cents. Sending that same gigabyte to a user once costs nine cents. Serving it is four times more expensive than keeping it, and if it is popular, it is served thousands of times.

And the third line is the one that catches architects. Every internal call that crosses an availability zone is charged in both directions. A chatty service mesh spread across three zones for availability generates a data-transfer bill that can exceed the cost of the machines themselves — and nothing in the architecture diagram shows it.

Ways to pay for compute

ON-DEMAND	full price, no commitment → the default, and the most expensive
COMMITTED	reserved instances or savings plans: commit for 1 or 3 years → roughly 40% off for 1 year, up to 60-70% for 3 years → you pay whether you use it or not
SPOT	spare capacity, taken back with about two minutes' notice → 70-90% off → only for work that can be interrupted
SERVERLESS	pay per request and per millisecond → near zero when idle → more expensive than committed capacity once traffic is steady and high

The shape to aim for: commit to your steady baseline, use on-demand for the daily peak, and put anything interruptible on spot. A fleet that is 100% on-demand is paying roughly double for its baseline, and that is often the single easiest saving available.

Storage tiers

Storage is cheap and unbounded, which is exactly why it grows without anybody deciding.

OBJECT STORAGE, per GB per month (order of magnitude)

standard	\$0.023	
infrequent access	\$0.0125	+ a retrieval charge
archive, instant	\$0.004	+ a larger retrieval charge
deep archive	\$0.00099	+ hours to retrieve

→ deep archive is about 23x cheaper than standard.

AND THE TRAP: retrieval is charged, and so are requests. Millions of tiny objects in a cheap tier can cost more in request charges than the storage saved.

Lifecycle rules are the mechanism: after 30 days move to infrequent access, after 90 to archive, after a year delete. The decision that actually matters is the delete, and it is the one nobody wants to make.

Where the money really goes

Two findings appear in almost every cost review, and both are Ramesh's shop.

Idle resources. Development and test environments running at nights and weekends — which is 128 of the 168 hours in a week, so switching them off outside working hours removes about three quarters of their cost. Unattached disks. Old snapshots. Load balancers with nothing behind them. The environment for a project that was cancelled and nobody deleted.

Over-provisioning. Machines sized for a peak that never came, or sized by copying whatever the last project used. A fleet running at 8% CPU is paying twelve times what it needs, and it looks perfectly healthy on every dashboard.

And the third, which is newer and now enormous: observability. From day 172, a hundred million requests a day generates about 720 GB of logs, which on managed per-gigabyte pricing is over ten thousand dollars a month. It is routine for a team's logging bill to exceed the bill for the servers that produced the logs.

Making the bill readable

Ramesh could not act until he had the meter. The equivalent is tagging: every resource labelled with a team, an environment and a service, so the single number becomes a table.

WITHOUT TAGS	"\$29,000 this month" → nobody owns it, so nobody reduces it	
WITH TAGS	checkout service, production: \$8,400 checkout service, staging: \$1,900 search service, production: \$6,100 untagged: \$4,600	← always the problem

The “untagged” row is where the empty fridges live, and getting it to zero is usually the highest-value first move in a cost programme. **You cannot switch off what you cannot see, and you cannot ask anybody to switch it off if nobody’s name is on it.**

The order to optimise in

Highest leverage first, because effort is finite and engineering time is expensive.

- | | |
|---|---|
| 1. DELETE what is not used | free, and often 10-30% |
| 2. SWITCH OFF what is not needed right now (non-production outside working hours) | free, ~70% of those |
| 3. RIGHT-SIZE what is oversized | small effort, 20-40% |
| 4. COMMIT to the baseline | a signature, ~40% |
| 5. TIER the storage | a lifecycle rule |
| 6. STOP THE BYTES MOVING | a CDN, compression, co-locating chatty services |
| 7. CHANGE THE ARCHITECTURE | weeks of work. Last. |

Almost everybody starts at seven. The first two are free and frequently bigger.

4 The picture

Where the money goes on a single request path:

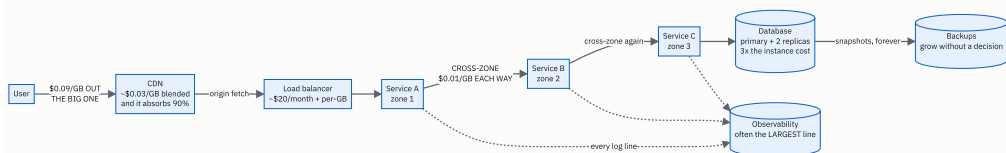


FIGURE 176.2

Two arrows on that diagram cost more than most of the boxes. The one leaving to the user, and the ones crossing between zones — and neither of them appears as a component in any normal architecture drawing, which is exactly why they are missed.

The bill, broken up — the thing Ramesh never had:

MONTHLY BILL, 100M requests/day, 1 TB/day served

observability (managed logs)	\$10,800	36%	← largest
cross-zone data transfer	\$6,420	22%	← invisible
compute, 100 machines on-demand	\$7,300	25%	
internet egress (no CDN)	\$2,700	9%	
database, 3 nodes	\$1,752	6%	
cache, 2 nodes	\$511	2%	
backups and snapshots	\$100	0%	

	\$29,583		

THE TWO LARGEST LINES ARE THINGS NOBODY DRAWS.
Compute - the thing everybody discusses - is a quarter.

Idle cost, drawn as a week:

a non-production environment, 168 hours in a week

Mon Tue Wed Thu Fri Sat Sun

|---|---|---|---|---|---|---|

USED:

###

###

###

###

###

9-6

9-6

9-6

9-6

9-6

= 45 hours

BILLED:

#####

= 168 hours

→ 123 hours a week, 73% of the cost, for an environment with nobody in it.

A schedule that stops it at 7pm and starts it at 8am, weekdays only, costs nothing to implement and removes about three quarters of the bill for every non-production environment you have.

THAT IS RAMESH'S FRIDGE.

5 How it actually works

Reading the bill

Every cloud provider produces a detailed usage export — AWS’s Cost and Usage Report, GCP’s billing export to BigQuery, Azure’s cost exports. It is a row per resource per hour, and it is the only honest source. The console’s summary view aggregates in ways that hide exactly the things you are looking for.

AWS	Cost Explorer for the summary, CUR for the truth
GCP	billing export into BigQuery, then query it
Azure	Cost Management + exports
Kubernetes	Kubecost or OpenCost, because a cluster's bill arrives as one line and you need it per namespace
Terraform	Infracost, which prices a change in the pull request, before it is merged

That last one is the interesting mechanism. Infracost puts “this change adds \$840/month” into the code review, which moves the decision to the moment when changing it is free. Every other tool tells you after you have already been charged.

Tagging, and why it is a policy problem

The technology is trivial: a key-value label on every resource. The difficulty is entirely organisational — a tag that is optional is a tag that is missing on a third of everything.

the enforcement that works:
a policy that REFUSES to create an untagged resource
(AWS Service Control Policies, Azure Policy, or
a Terraform check in CI)
the enforcement that does not work:
a wiki page asking people to remember

And then the bill becomes a table by team, which is where the behaviour changes. Showback is sending each team their number. Chargeback is actually billing it to their budget. Showback changes behaviour almost as much as chargeback and causes far fewer arguments, which is why most organisations stop there.

Anomaly detection

A cost bug does not page anybody, and that is why it runs for a month.

a runaway retry loop	millions of extra requests
a log level left at DEBUG	5-10x the log volume
a test that forgot to delete its resources	a fleet nobody knows about
a misconfigured lifecycle rule	terabytes never expiring

→ All of these are silent, all of them last until the invoice, and all of them are found instantly by a daily spend alert per tag.

A daily budget alert per team, with a threshold on the rate of change rather than the absolute number, is about an hour of setup and routinely catches things worth thousands. It is the same shape as a burn-rate alert from day 173, applied to money.

The cost of a nine, revisited

From day 173: each additional nine costs roughly ten times the last. Cost is where that becomes concrete.

99.9%	two machines, one zone, ordinary on-call ~\$10,000/month
99.99%	multi-zone, automatic failover, replicated storage, tested runbooks ~\$40,000-100,000/month
99.999%	multi-region active-active, no human in the recovery path + a team whose whole job is this

THE QUESTION TO ASK FIRST:

what does an hour of downtime actually cost?

If an hour costs \$5,000 and the budget for 99.9% is 43 minutes a month:

expected loss = $43/60 \times \$5,000 = \sim \$3,600/\text{month}$

Paying \$60,000/month extra to avoid \$3,600/month of expected loss is a bad trade, and saying so out loud is worth more than any amount of redundancy.

Unit economics

The number that makes cost a design input rather than an afterthought.


```
cost per DAU      = monthly bill / daily active users
cost per request  = monthly bill / monthly requests
cost per GB stored
cost per order / per stream / per message
```

→ The point is the DIRECTION over time.

A bill that grows from \$29k to \$35k while users grow 40% is a system getting CHEAPER per user, which is healthy.

A bill that grows 40% while users grow 5% is a leak, and unit cost finds it months before the total does.

And it connects the engineering to the business. If the product earns \$0.30 per active user per month and costs \$0.006 to serve, there is room. If it costs \$0.40, the architecture has to change — and that is a real conversation an engineer can start, with a number.

6 The numbers

Price the system from day 172 and day 173: 100 million requests a day, 1 TB served a day, 5 million daily active users.

Compute.

100 machines, \$0.10/hour, 730 hours in a month

$100 \times \$0.10 \times 730 = \$7,300/\text{month}$ on-demand

with a 1-year commitment at ~40% off:

$\$7,300 \times 0.60 = \$4,380/\text{month}$ saving \$2,920

with 30% of the fleet on spot at ~75% off:

70 machines committed: $70 \times 0.10 \times 730 \times 0.60 = \$3,066$

30 machines spot: $30 \times 0.10 \times 730 \times 0.25 = \548

\$3,614

→ \$7,300 down to \$3,614. A 50% cut, no code changed.

Internet egress — the line people forget.

1 TB/day x 30 days = 30 TB = 30,000 GB

direct from the origin at \$0.09/GB:

$30,000 \times \$0.09 = \$2,700/\text{month}$

through a CDN, blended ~\$0.03/GB at this volume,
absorbing 90% of requests:

$30,000 \times \$0.03 = \$900/\text{month}$

plus the CDN's own request charges, say \$100

\$1,000

→ \$2,700 → \$1,000, and the pages get faster as well.

FOR SCALE: storing that same 30 TB for a month costs

$30,000 \times \$0.023 = \690 .

Sending it out once costs \$2,700.

→ SERVING IS ~4x MORE EXPENSIVE THAN STORING.

Cross-zone transfer — the line nobody draws.

8 internal service calls per request

100,000,000 requests/day

→ 800,000,000 internal calls/day

average response 20 KB:

$800,000,000 \times 20 \text{ KB} = 16,000,000,000 \text{ KB}$
 $= 16,000 \text{ GB/day} = 16 \text{ TB/day}$

with services spread over 3 zones, about two thirds of
calls cross a zone boundary:

$16,000 \times 0.66 = 10,560 \text{ GB/day crossing}$

charged BOTH WAYS at \$0.01/GB = \$0.02/GB:

$10,560 \times \$0.02 = \$211/\text{day}$
 $= \$6,420/\text{month}$

→ MORE THAN THE ENTIRE SAVING FROM COMMITTING THE
COMPUTE, and it appears on no architecture diagram.

THE FIX, and it is not "use fewer zones":

- zone-aware routing: prefer a callee in your own zone,
fall back across zones only when yours is unhealthy
→ cuts the crossing fraction from 66% to ~5%
→ \$6,420 → ~\$490
- batch the calls: 8 calls at 20 KB → 3 calls at 40 KB
is 16 TB → 6 TB
- compress internal responses: 20 KB → 4 KB

Observability, from day 172.

8 services x 3 log lines x 100,000,000 requests
= 2,400,000,000 lines/day
x 300 bytes = 720 GB/day

managed logging at \$0.50/GB ingested:
720 x \$0.50 = \$360/day = \$10,800/month

→ the LARGEST single line on this bill.

with 7 days hot + 90 days in archive, and sampling
successful requests at 1 in 10:

ingested volume falls to ~200 GB/day
200 x \$0.50 = \$100/day = \$3,000/month
→ \$10,800 → \$3,000

The database and the cache.

database: 1 primary + 2 replicas, \$0.80/hour each
3 x \$0.80 x 730 = \$1,752/month
backups, 2 TB of snapshots at \$0.05/GB:
2,000 x \$0.05 = \$100/month
cache: 2 nodes at \$0.35/hour
2 x \$0.35 x 730 = \$511/month

The whole bill, and then the same bill optimised.

BEFORE		AFTER	
observability	\$10,800	\$3,000	sample + tier
compute	\$7,300	\$3,614	commit + spot
cross-zone	\$6,420	\$490	zone-aware
egress	\$2,700	\$1,000	CDN
database	\$1,752	\$1,752	unchanged
cache	\$511	\$511	unchanged
backups	\$100	\$100	unchanged
	-----	-----	
	\$29,583	\$10,467	

→ a 65% reduction, and not one line of product code
changed. Every saving came from configuration,
commitment, or routing.

Unit economics.

5,000,000 daily active users
3,000,000,000 requests/month

BEFORE:

$\$29,583 / 5,000,000 \text{ DAU} = \$0.0059 \text{ per user/month}$
 $\$29,583 / 3,000,000,000 \times 1000 = \$0.0099 \text{ per 1,000 requests}$

AFTER:

$\$10,467 / 5,000,000 = \$0.0021 \text{ per user/month}$
 $= \$0.0035 \text{ per 1,000 requests}$

→ about six-tenths of a cent per user per month,
falling to two-tenths.

If the product earns \$0.30 per active user per month,
infrastructure is 2% of revenue before, 0.7% after.
THAT is the sentence that ends the conversation.

7 The trade-offs

Committing to capacity buys 40% and costs you flexibility for one to three years.

A one-year commitment is roughly 40% off; three years is 60-70%. You pay whether you use it or not. I would not commit if the traffic might halve, if a migration to a different instance family is likely, or if the product is young enough that the whole architecture could change. I would commit to the baseline — the capacity that has been running every hour for six months — and leave the peak on demand. Committing to the peak is how teams end up paying for capacity they stopped using in month three.

Spot capacity is 70-90% off and can be taken away in two minutes.

Right for stateless web servers behind a load balancer, batch jobs, CI runners and anything that can be restarted. Wrong for a database primary, a stateful leader, or anything holding a long-running session. And it needs the application to handle a termination notice gracefully, which is engineering work — so the saving is not free, it is deferred into a different budget.

Managed services cost more per unit and less in total.

A managed database is perhaps two to three times the raw instance cost. An engineer's month is worth more than a substantial fleet. I would run managed by default and self-host only where the scale makes the multiplier genuinely

large and there is a team to own it. The comparison people get wrong is machine cost against machine cost; the honest comparison includes the on-call rota.

Serverless is cheapest when idle and expensive when busy.

At low or spiky traffic, paying per request beats paying for machines that are mostly idle. There is a crossover point — roughly where sustained utilisation would keep a committed instance meaningfully busy — above which committed capacity is several times cheaper. I would use serverless for spiky, low-volume or unpredictable workloads and for glue, and reach for committed capacity for a steady high-traffic path.

Multi-zone redundancy buys availability and costs you data transfer.

Three zones is the standard answer for availability and it puts a per-gigabyte charge on your internal chatter, both ways. The wrong response is to collapse to one zone, which trades a real availability requirement for a bill. The right response is zone-aware routing — prefer a callee in your own zone, cross only when yours is unhealthy — which keeps the redundancy and removes most of the charge.

Cheaper storage tiers cost you retrieval, and sometimes more than they save.

Deep archive is about twenty-three times cheaper per gigabyte. It also charges for retrieval and takes hours. And request charges on millions of small objects can exceed the storage saved entirely — so the tiering decision depends on object size and access pattern, not only on age.

And cost optimisation itself is not free.

Engineering time spent on cost is engineering time not spent on the product. Spending three weeks to save four hundred dollars a month is a loss, and the discipline is to do the free things first — delete, switch off, commit — and only then consider anything that needs a design change. I would say plainly that the last item on the optimisation list is ‘change the architecture’, and that most teams start there.

8 In the interview

How it gets asked

- “What does this system cost to run per month?” — rare, and devastating if you have nothing.

- *“Your bill doubled last month. How do you find out why?”* — tagging, the usage export, anomaly alerts.
- *“How would you cut this bill in half?”* — the ordered list, cheapest effort first.
- *“On-demand, reserved or spot?”* — and they want the reasoning, not the discount numbers.
- *“Is this worth building ourselves?”* — the comparison that includes salaries.
- *“Why not just add more machines?”* — a cost question wearing a scaling costume.

The first ninety seconds

On “what does this system cost to run”:

“Let me price it, and I will do it in four line items because that is where nearly all of it lands.

Compute. About a hundred machines at ten cents an hour, times 730 hours, is \$7,300 a month on demand. Committing to the baseline for a year takes roughly 40% off, and putting the stateless third on spot takes 75% off that portion — so about \$3,600. That is a halving with no code changed.

Data transfer, and this is the one I would want to raise before being asked. A terabyte a day out to users is 30 TB a month, and internet egress is about nine cents a gigabyte — \$2,700. Through a CDN at a blended three cents it is about \$1,000, and the pages get faster too.

For scale: storing those same 30 TB for a month costs about \$690, and sending them out once costs \$2,700. Serving is four times more expensive than storing, which is not most people’s intuition.

Then the line nobody draws: cross-zone transfer. Eight internal calls per request at a hundred million requests a day is 800 million internal calls, at maybe 20 KB each — 16 TB a day. Spread over three zones, about two thirds of that crosses a boundary, and it is charged in both directions at a cent a gigabyte. That is roughly \$6,400 a month, which is more than the entire saving from committing the compute.

And observability, which at this scale is often the largest single line. 720 GB of logs a day at fifty cents a gigabyte ingested is \$10,800 a month — more than the servers that produced them.

All in, about \$29,500 a month. At five million daily active users that is about six-tenths of a cent per user per month, or a cent per thousand requests.

And the two largest lines — logs and cross-zone traffic — are both things that appear nowhere on the architecture diagram, which is exactly why they are the ones that are wrong.”

The follow-ups

“Cut that bill in half.”

“I would go in order of effort, because the free things are usually bigger than the clever ones.

First, delete and switch off, which costs nothing. Non-production environments run 168 hours a week and are used for about 45. A schedule that stops them outside working hours removes about three quarters of their cost. Plus unattached disks, old snapshots, load balancers with nothing behind them, and whatever is in the ‘untagged’ row — which in every organisation I have seen is where the abandoned projects live.

Second, sample and tier the logs. They are the largest line at \$10,800. Keeping every line for errors and one in ten for successful requests, with seven days hot and ninety in archive, takes the ingested volume to about 200 GB a day — roughly \$3,000. That is a \$7,800 saving from a configuration change, and the cost is that reconstructing an arbitrary successful request is no longer possible, so I would check with support before doing it.

Third, zone-aware routing for the internal calls. Right now two thirds of internal traffic crosses a zone boundary and is charged both ways — \$6,400. Preferring a callee in the same zone and only crossing when your zone is unhealthy takes the crossing fraction to a few percent, so about \$490. The redundancy is unchanged; only the routing preference changed.

Fourth, commit the baseline and put the stateless tier on spot — \$7,300 to about \$3,600. A one-year commitment on the capacity that has been running every hour for six months, and on-demand for the peak.

Fifth, a CDN for the egress — \$2,700 to about \$1,000.

That is \$29,500 down to about \$10,500 — a 65% cut — and not one line of product code changed. Every saving came from configuration, commitment or routing.

What I would not do first is redesign anything. That is weeks of work and it is last on the list, and most teams start there because it is the interesting part.”

“On-demand, reserved, or spot?”

“All three, on different parts of the fleet, and the split follows how predictable each part is.

The baseline goes on a commitment. That is the capacity that has genuinely been running every hour for six months. **A one-year commitment is about 40% off; three years is 60 to 70%. The cost is that I pay whether I use it or not, so I commit to the floor and never to the peak** — committing to the peak is how teams end up paying for capacity they stopped using in month three.

The daily peak above that baseline goes on demand. It is the most expensive way to buy an hour and the cheapest way to buy flexibility, and for a few hours a day that is the right trade.

Anything interruptible goes on spot: 70 to 90% off, taken back with about two minutes’ notice. Stateless web servers behind a load balancer, batch jobs, CI runners, anything that can be restarted. Not a database primary, not a stateful leader, not a long-running session.

And I would be honest that spot is not free money — the application has to handle a termination notice gracefully, drain connections and exit. That is engineering work, so the saving is partly moved into a different budget.

The two cases where I would not commit at all: if the traffic might halve, or if the product is young enough that the architecture could change entirely inside a year. A three-year commitment on a six-month-old product is a bet, not a saving.

And I would mention serverless as the fourth option. It is cheapest when idle and more expensive than committed capacity when busy — so it is right for spiky, unpredictable or low-volume work, and wrong for a steady high-traffic path.”

“Your bill doubled last month and nobody knows why. What do you do?”

“First, get the breakdown, because the summary view will not tell me. The detailed usage export — a row per resource per hour — is the only honest source. The console’s grouped view aggregates in exactly the way that hides this.

Then compare month to month by tag, by service and by usage type, and the answer usually falls out of one of four buckets.

Something was created and forgotten — a test that provisioned a fleet and did not clean up. A log level left at DEBUG, which is five to ten times the volume. A retry loop that started firing, which multiplies request counts and data transfer at the same time. Or a lifecycle rule that was wrong, so nothing has been expiring.

The ‘untagged’ row is where I would look first, because that is where anything nobody owns ends up.

The real answer, though, is that this should not have taken a month to notice. A cost anomaly does not page anybody, which is why it runs until the invoice. A daily spend alert per tag, thresholded on the rate of change rather than the absolute number, is about an hour of setup and catches all four of those in a day. It is the same shape as a burn-rate alert, applied to money.

And the preventive version is putting the price in the pull request — a tool like Infracost that comments ‘this change adds \$840 a month’ during code review. That moves the decision to the moment when changing it is free, which is the only time anybody actually reconsiders.”

The model answer

“What does this system cost to run per month, and would you change anything because of it?”

“I would price the four lines that dominate every bill, and then say which two of them are wrong.

Compute is about \$7,300 — a hundred machines at ten cents an hour. Storage and backups are small, a few hundred. The database and cache together are around \$2,300. Egress to users is \$2,700 for 30 TB a month. Cross-zone internal traffic is about \$6,400. And observability is \$10,800. Total, roughly \$29,500 a month.

The first thing worth saying about that list is its shape. Compute — the thing we have spent most of this interview discussing — is a quarter of it. The two largest lines, logging and cross-zone traffic, do not appear anywhere on the diagram I drew. Which is precisely why they are the ones that are wrong.

The unit economics: at five million daily active users, that is about six-tenths of a cent per user per month, or a cent per thousand requests. If the product earns thirty cents per active user, infrastructure is about two percent of revenue, which is healthy. The number I would actually watch is the direction — a bill growing 40% while users grow 5% is a leak, and unit cost finds it months before the total does.

The two changes I would make on cost grounds, and they are both routing rather than redesign.

Zone-aware routing for internal calls. Two thirds of 16 TB a day crosses a zone boundary and is charged both ways. Preferring a callee in the same zone, falling back across zones only when yours is unhealthy, keeps the availability properties completely intact and takes that line from \$6,400 to about \$500.

Sampling and tiering the logs. Every line for errors, one in ten for successful requests, seven days hot and ninety in archive — \$10,800 to about \$3,000. The cost is real and I would name it: reconstructing an arbitrary successful request becomes impossible, so I would confirm with whoever handles customer support before doing it.

Then the things that need no design opinion at all: commit the baseline compute and put the stateless tier on spot, and put a CDN in front of the egress. Together that is about \$29,500 down to \$10,500.

And the sentence I would end on, because it is the one that connects this to the rest of the design. Cost is a constraint like latency and availability, not a report you read afterwards. The reason I would raise it in the last ten minutes of a design conversation is that the cheapest time to change an architecture is before it exists — and the two most expensive lines in this one are both things that would have been free to arrange differently on the whiteboard, and cost weeks to change afterwards.”

9 Recall card

Cost is a design constraint, not a report. Four line items dominate: **COMPUTE** (usually only 30-50%), **STORAGE** (cheap, only grows), **DATA TRANSFER** (the surprise), **MANAGED SERVICES** (observability is often the largest single line) — plus a fifth that is not on the bill: **people**, which is why “build it ourselves to save money” is usually wrong.

Data transfer is where the shock is. **IN** is free; **OUT** to the internet is ~\$0.09/GB; **BETWEEN ZONES** is ~\$0.01/GB EACH WAY. Storing 30 TB for a month is ~\$690; sending it out once is \$2,700 — serving is 4× more expensive than storing. And cross-zone chatter appears on no architecture diagram: 8 internal calls × 100M requests × 20 KB = 16 TB/day, two-thirds crossing, both ways ≈ \$6,420/month — more than the entire saving from committing the compute. Fix it with zone-aware routing, not by collapsing to one zone.

Buy compute three ways at once: **COMMIT** the baseline (~40% off for 1 year, 60-70% for 3, and you pay whether you use it), **ON-DEMAND** for the peak, **SPOT** for anything interruptible (70-90% off, two minutes’ notice, never a stateful leader). Serverless is cheapest idle and dearest busy. Commit to the floor, never the peak.

Optimise in order of effort, because the free things are bigger. 1. Delete what is unused. 2. Switch off non-production outside working hours (used 45 of 168 hours — ~73% saving, and it is Ramesh’s empty fridge). 3. Right-size. 4. Commit. 5. Tier storage. 6. Stop the bytes moving — CDN, compression, co-location. 7. Change the architecture — **LAST**, and where everyone starts. A worked example goes \$29,500 → \$10,500, a 65% cut, with no product code changed.

You cannot reduce what you cannot see. Tag everything and enforce it with a policy that refuses untagged resources — the “untagged” row is where abandoned projects live. Showback changes behaviour almost as much as chargeback with

far fewer arguments. A cost bug pages nobody, so it runs until the invoice — set a daily spend alert per tag on the RATE of change, and price changes in the pull request. Track cost per DAU and per 1,000 requests: the direction matters more than the total, and a bill growing faster than usage is a leak. And before buying a nine, ask what an hour of downtime actually costs.

DSA topic: Bits and maths revision and mock round **System design topic:** Cost: the constraint nobody mentions

Code these, in this order

This is a mock round, so the rule is different today. Set a clock. **Twenty minutes for a medium, thirty for a hard.** Talk out loud the whole time, even alone, even feeling foolish. **And before you write a single line, say the recognition sentence** — the one that names which of the twelve tools this is. **If you cannot say it, do not start typing; go back and read the constraints again.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Bitwise AND of Numbers Range	LeetCode 201 (Medium)	That a range of numbers shares a binary prefix.
2	Single Number II	LeetCode 137 (Medium)	Noticing that pairing fails, and counting modulo three.
3	Closest Prime Numbers in Range	LeetCode 2523 (Medium)	Per group, not per number — plus one fact about prime gaps.
4	Count Ways to Make Array With Product	LeetCode 1735 (Hard)	The capstone: sieve, factorise, stars and bars, modular inverse.

The rules for the round

Read the constraints first, out loud, before the problem statement. Say what they forbid.

Then say the recognition sentence. One sentence, naming the structure. *“ANDing a range keeps only the common prefix.” “A product is decided independently per prime, so I multiply the counts.”*

Then state the plan and the cost, before writing. Time and space.

Then write it. No running it until you have read it back once yourself.

Then test four inputs: zero, one, two, and something negative or degenerate.

Score yourself honestly

For each problem, tick what actually happened:

- ☐ I read the constraints before the problem
- ☐ I said the recognition sentence before writing
- ☐ The sentence was right
- ☐ I stated the cost before writing
- ☐ The first version was correct
- ☐ I found my own bug, before running it
- ☐ I tested 0, 1, 2 and a degenerate case

4 problems x 7 = 28 ticks.

22+ you are ready for this phase in an interview.
15-21 the tools are there and the recognition is not.
Re-read the trigger table until the middle column arrives before you finish the left one.
<15 go back and re-solve days 171-175's problems with the clock off. Speed is not the issue yet.

On problem 1, notice what the constraints forbid

`right` can be 2,147,483,647. **Say out loud, before anything else, what that rules out.** Then say what is left: the answer can only depend on the two endpoints.

On problem 1 again, check the boundary

Run it on `left = right`. **Then change** `while right > left` to `while right ≥ left` **and record what happens** for every input.

On problem 2, write the slow one on purpose

Write the column-counting version even though the state machine exists. **Then write the state machine and time both on a list of thirty thousand.** Record the ratio. **Then say which one you would write in an interview and why the answer is not the fast one.**

On problem 3, find the early exit yourself

Solve it without the early exit first. **Count how many primes the scan examines on** `[1, 1000000]`. Then work out for yourself why a gap of two cannot be beaten, add the exit, and count again. Record both numbers.

On problem 4, say the sentence before anything

Do not write code until you can say, in one breath, **why the primes are independent and what stars and bars is doing.** Then check `n = 2, k = 6` by hand — there should be four arrays — before you trust anything.

Then the trigger-table drill

Cover the middle column. **Read each trigger and name the tool and the justifying sentence.** Do it until nothing takes more than three seconds.

Then the silent-failure drill

Write out all five failures that produce a wrong number with no error. **For each, give the input that exposes it and the one-line fix.**

Then the bill drill

Price the system in the lesson from memory: **compute, egress, cross-zone, observability, database.** Get to a total. Then give the cost per daily active user and per thousand requests.

Then the surprise drill

Compute what it costs to store 30 TB for a month, and what it costs to send it out once. **Say the ratio.** Then compute the cross-zone bill from eight internal calls at twenty kilobytes, and say why it is invisible.

Then the cut-it-in-half drill

Give the optimisation list in order. **For each step, say roughly what it saves and what it costs you.** Then say which step most teams start at, and why that is the wrong end.

The recognition drill

1. Give five triggers and the tool each one names.
2. Give the four questions of the thirty-second procedure, in order.
3. Say which question is highest value and why.
4. Name the three tools that are secretly one idea, and give the idea.

The bits drill

1. Give both core moves and what each one is for.
2. Give the power-of-two test with its guard.
3. Give the subset enumeration and the value of `n` at which it stops.
4. Give the submask loop and its cost against the naive one.

The XOR drill

1. Give the three facts.

2. One loner, two loners, three copies: the method for each.
3. Say what a set bit in $a \wedge b$ means.
4. Give the range-XOR period and the four cases.

The number-theory drill

1. Say why testing to the square root is enough.
2. Give the sieve, including where it starts and where it stops.
3. Give Euclid and the one-sentence reason.
4. Give fast power and its step count for 10^{19} .
5. Give Fermat's inverse and both preconditions.

The counting drill

1. Give the two questions and the four formulas.
2. Say why you never compute a factorial.
3. Give the multiplicative form and its order of operations.
4. Give the three-lookup modular form and the walk-down trick.
5. Give stars and bars, and one problem it solves in disguise.

The line-items drill

1. Name the four line items that dominate a bill.
2. Name the fifth that is not on the bill.
3. Say which is usually the largest at scale, and why that surprises people.
4. Say what fraction compute typically is.

The transfer drill

1. Give the four transfer prices.
2. Compare the cost of storing 30 TB with sending it out once.
3. Compute the cross-zone bill from stated internal traffic.
4. Give the fix, and say why collapsing to one zone is the wrong fix.

The purchasing drill

1. Give the four ways to buy compute and the discount for each.
2. Say what you commit to and what you never commit to.
3. Say what spot is right for and wrong for, with examples.
4. Say where serverless wins and where it loses.

The optimisation drill

1. Give the seven steps in order.
2. Say which two are free.
3. Compute the saving from switching non-production off outside working hours.
4. Say which step teams start at, and why that is wrong.
5. Say when cost optimisation is itself a bad investment.

The visibility drill

1. Say what tagging buys and how it is enforced.
2. Say what the “untagged” row usually contains.
3. Give the difference between showback and chargeback.
4. Say why a cost bug runs for a month, and the alert that catches it in a day.
5. Say where the cheapest place to see a price is.

The break-it drill

For each, say what happens and whether anything reports it:

1. `while right ≥ left` in the range-AND loop.
 2. Plain XOR on a list where values appear three times.
 3. Breaking on the first prime pair found rather than the closest.
 4. Factorial tables sized to `n` rather than `e + n - 1`.
 5. A log level left at DEBUG in production for a month.
 6. A three-year commitment on a six-month-old product.
 7. A database primary running on spot capacity.
 8. Millions of tiny objects moved into deep archive.
 9. An untagged resource created by a test that never cleaned up.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* Read the constraints, say what they forbid, say the recognition sentence, state the plan and the cost, then write. Practise the shape, not the answer.

2. *How do you approach a bits or maths problem you have not seen?* Which of the twelve rather than what is this; constraints first; per number or per group; does anything shrink structurally; does the answer need dividing.
3. *What does this system cost to run per month?* The four line items with real multiplication, the two lines that appear on no diagram, the total, cost per user, and the two changes you would make.

Before you move on

- ☐ I read constraints before I read the problem.
- ☐ I can name the tool for any trigger in the table within three seconds.
- ☐ I say the recognition sentence out loud before writing.
- ☐ I know the three tools that are secretly one idea.
- ☐ I scored myself on the four mock problems honestly.
- ☐ I know which of 0, 1, 2 and a negative breaks each tool.
- ☐ I can list the five silent failures with the input that exposes each.
- ☐ I know `while n: n &= n-1` hangs on a negative, and why hanging is worse.
- ☐ I would write the explainable version and mention the clever one.
- ☐ I can name the four line items and the fifth that is off the bill.
- ☐ I know observability is often the largest single line.
- ☐ I know egress is ~\$0.09/GB and cross-zone is ~\$0.01/GB each way.
- ☐ I can say why serving is four times more expensive than storing.
- ☐ I can compute the cross-zone bill from internal call volume.
- ☐ I know zone-aware routing is the fix, not collapsing to one zone.
- ☐ I can give the four ways to buy compute with their discounts.
- ☐ I know to commit to the floor and never to the peak.
- ☐ I know what spot is right for and wrong for.
- ☐ I can give the seven optimisation steps in order.
- ☐ I know the first two are free and often the biggest.
- ☐ I can compute the non-production idle saving.
- ☐ I know what tagging buys and how it has to be enforced.
- ☐ I know a cost bug pages nobody, and the alert that fixes that.
- ☐ I can give cost per user and per thousand requests for a stated system.
- ☐ I know to ask what an hour of downtime costs before buying a nine.

- [] I answered all three questions above out loud.

PART XIX

Final mocks and revision

Days 177 to 180 · 4 chapters

Alongside, on the system design track:

Reliability, security, and the interview itself

177 The twenty patterns, on one page

178 How to think out loud in a coding round

179 Full mock: two problems, forty-five minutes

180 Final revision, and the week before the interview

The twenty patterns, on one page

DATA STRUCTURES AND ALGORITHMS

The twenty patterns, on one page

SYSTEM DESIGN

Microservices versus monolith, argued both ways

1 What this is, and why they ask it

There are about twenty patterns, and almost every interview problem is one of them.

Not twenty algorithms — twenty shapes. “A contiguous stretch with a constraint” is one shape, and it does not matter whether the elements are letters or numbers or costs. **Once you have named the shape, the code is something you have already written many times.**

This lesson is the index. Every pattern, the sentence that gives it away, the template, and what it costs. **Nothing here is new.** You have met all twenty across the last hundred and seventy-six days. **What is new is having them in one place, in a form you can search from a one-line problem description.**

They ask it because **recognition is the actual skill, and it is the one that fails under pressure.** Nobody forgets how to write a sliding window. **What happens in the room is that a problem arrives, the candidate does not recognise it, and spends nine minutes on a brute force before the shape arrives** — by which point there is no time to write it.

And because interviewers can hear the difference. “This is asking for the longest contiguous stretch with at most two distinct values, so it is a sliding window” **is a different answer from “let me try some loops”,** even when both eventually produce the same code.

By the end of this lesson you have all twenty patterns with their triggers and templates, a procedure for matching an unseen problem to one of them, a table that turns a constraint into a technique, and the honest list of patterns that get confused with each other.

2 The story

The enquiry counter at the station was a window about two feet across with a brass grille in it, and the man behind it was called Raghavan, and he almost never opened the book.

There was a book. A thick one, with a soft cover and a rubber band round it, holding every train that passed through. **It lived on the shelf behind him and he touched it perhaps twice a week.**

People came to that window with a sentence.

"I have to be in Nagpur by Friday morning." "My mother is old, she cannot change in the middle of the night." "Can I go to Belgaum and come back the same day?"

And Raghavan would look at them for about two seconds and say a train, and a time, and which side of the platform.

His son, who was doing a diploma and had started thinking about how things worked, asked him one evening how many trains he had memorised.

Raghavan said he had not memorised any trains.

"Then what?"

He said there were only about twenty questions. Not twenty trains. **Twenty questions.**

And he counted some of them off on his hand. Somebody who has to arrive by a particular hour. Somebody who cannot change trains. Somebody who wants the cheapest thing available. Somebody travelling with a small child, who needs a day train and does not know to ask for one. Somebody going and returning the same day. **Somebody who does not want a train at all and wants the bus, and has not worked that out yet.**

"Once I know which of the twenty it is, there are only two or three trains it can be. Then I look."

His son said that still meant knowing all the trains.

"No," Raghavan said. "It means knowing which two to check."

And then the thing his son remembered for a long time afterwards.

"The people who are slow at this counter are not slow because they do not know the trains. They are slow because they are still listening to the whole sentence."

"I stop listening after about six words. By then I know which question it is."

3 The idea in plain English

Raghavan's twenty questions are the twenty patterns. You are not matching a problem to a solution. You are matching a problem to a shape, and the shape has two or three known solutions attached to it.

Here they are. Read the middle column first — that is the sentence that gives each one away.

1. Two pointers

The tell: the input is sorted, and you want a pair — or you are rearranging in place.

```
left, right = 0, len(a) - 1
while left < right:
    if condition(a[left], a[right]): ...
    elif too_small: left += 1
    else:           right -= 1
```

Why it works: sorting makes the decision unambiguous. Sum too small, the only way up is to move `left` in. $O(n)$ time, $O(1)$ space. *Two Sum II, Container With Most Water, 3Sum, Trapping Rain Water.*

2. Sliding window

The tell: a CONTIGUOUS subarray or substring, with a constraint — “longest with at most k distinct”.

```
left = 0
for right in range(len(a)):
    add a[right] to the window
    while window is invalid:
        remove a[left]; left += 1
    best = max(best, right - left + 1)
```

Why it works: the right edge only moves forward and so does the left, so each element is added once and removed once. $O(n)$, not $O(n^2)$. *Longest Substring Without Repeating Characters, Minimum Window Substring, Sliding Window Maximum (with a monotonic deque).*

3. Fast and slow pointers

The tell: a cycle, a middle, or an n th-from-the-end — in one pass and constant space.

```

slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
if slow is fast: cycle found

```

Why it works: two speeds close a gap of one per step, so if there is a loop they must meet. *Linked List Cycle, Middle of the Linked List, Find the Duplicate Number.*

4. Prefix sums and difference arrays

The tell: many range queries over unchanging data, or “how many subarrays sum to k”.

```

prefix[0] = 0
prefix[i] = prefix[i-1] + a[i-1]
sum(l..r) = prefix[r+1] - prefix[l]

```

Why it works: a range is the difference of two prefixes. $O(n)$ once, $O(1)$ per query. *Subarray Sum Equals K, Range Sum Query, Product of Array Except Self, Corporate Flight Bookings.*

5. Binary search on a sorted array

The tell: sorted, and you want a position.

```

low, high = 0, len(a)
while low < high:
    mid = (low + high) // 2
    if a[mid] < target: low = mid + 1
    else: high = mid
return low

```

Why it works: half the search space goes every step. $O(\log n)$. *Search in Rotated Sorted Array, First Bad Version, Find First and Last Position.*

6. Binary search on the answer

The tell: “minimise the maximum”, “smallest capacity such that”, “minimum speed to finish in h hours”.

```

low, high = smallest_possible, largest_possible
while low < high:
    mid = (low + high) // 2
    if feasible(mid): high = mid
    else: low = mid + 1
return low

```

Why it works: feasibility is monotone — if a capacity of 15 works, so does 16. **You are searching the answers, not the input.** *Koko Eating Bananas, Capacity to Ship Packages, Split Array Largest Sum.*

7. Sort, then sweep

The tell: intervals, or a greedy choice that only works in the right order.

```

events.sort(key=the_right_key)
for event in events:
    one decision, never revisited

```

Why it works: sorting removes the interactions, so a local decision becomes a global one. The whole difficulty is the sort key: by END time for “most non-overlapping”, by START for “merge”. *Merge Intervals, Non-overlapping Intervals, Meeting Rooms II, Minimum Number of Arrows.*

8. Hashing: maps, sets and frequency

The tell: “have I seen this before”, or counting how often things occur.

```

seen = {}
for i, value in enumerate(a):
    if target - value in seen: return seen[target-value], i
    seen[value] = i

```

Why it works: it trades memory for lookups. $O(n)$ time, $O(n)$ space — and that space is what a “constant extra space” constraint is forbidding. *Two Sum, Group Anagrams, Longest Consecutive Sequence, Top K Frequent.*

9. Stack, and the monotonic stack

The tell: matching pairs, or “the next greater / previous smaller element”.

```

stack = []
for i, value in enumerate(a):
    while stack and a[stack[-1]] < value:
        answer[stack.pop()] = value
    stack.append(i)

```

Why it works: each position is pushed once and popped once, so the nested loop is still $O(n)$ in total. *Valid Parentheses, Daily Temperatures, Largest Rectangle in Histogram, Next Greater Element.*

10. Heap: top-k and k-way merge

The tell: the k largest, a running median, or merging sorted sequences.

```
import heapq
heap = []
for value in a:
    heapq.heappush(heap, value)
    if len(heap) > k: heapq.heappop(heap)
```

Why it works: you never need the whole thing sorted — only the boundary. $O(n \log k)$, not $O(n \log n)$. *Kth Largest Element, Top K Frequent Elements, Merge k Sorted Lists, Find Median from Data Stream.*

11. Linked list pointer surgery

The tell: reversing, reordering or splitting a list, in place.

```
dummy = Node(0, head)
previous, current = None, head
while current:
    nxt = current.next
    current.next = previous
    previous, current = current, nxt
```

The dummy node is the trick: it removes the special case where the head itself changes. *Reverse Linked List, Merge Two Sorted Lists, Remove Nth Node From End, Reorder List.*

12. Tree traversal, depth-first and breadth-first

The tell: anything about a tree that is not specifically about a BST.

```
def walk(node):
    if not node: return base_case
    left = walk(node.left)
    right = walk(node.right)
    return combine(left, right, node.val)
```

Why it works: a tree is defined recursively, so the solution usually is too. DFS for paths and depths, BFS for levels and shortest hops. *Maximum Depth, Diameter, Path Sum, Level Order Traversal, Lowest Common Ancestor.*

13. Binary search tree properties

The tell: the word BST — which means inorder is sorted, and you can prune by bounds.

```
def valid(node, low, high):
    if not node: return True
    if not (low < node.val < high): return False
    return valid(node.left, low, node.val) and valid(node.right, node.val, high)
```

The mistake this prevents: checking only against the parent. *Validate BST, Kth Smallest Element in a BST, Convert Sorted Array to BST.*

14. Backtracking

The tell: “return ALL” — every permutation, every subset, every valid board.

```
def walk(state):
    if complete(state): record(state); return
    for choice in options(state):
        apply(choice)
        walk(state)
        undo(choice)          ← the line people forget
```

Why it works: you explore a tree of choices and prune branches that cannot succeed. The output is exponential, so the cost is too — and that is not a failure, it is the question. *Subsets, Permutations, N-Queens, Word Search, Combination Sum.*

15. Graph traversal and topological sort

The tell: things connected to things, or an order with prerequisites.

```
BFS for fewest hops, DFS for reachability.
Topological sort: repeatedly take a node with
in-degree 0, remove it, and decrement its neighbours.
```

A grid is a graph — that is the recognition people miss most often. $O(V + E)$. *Number of Islands, Course Schedule, Clone Graph, Rotting Oranges, Word Ladder.*

16. Shortest paths

The tell: weighted edges and a cheapest route.

```
Dijkstra: a heap of (distance, node), pop the nearest
unfinished node, relax its edges. NO NEGATIVE WEIGHTS.
0-1 weights: a deque instead of a heap.
Negative weights: Bellman-Ford.
```

BFS is Dijkstra when every edge costs the same, which is worth saying out loud. *Network Delay Time, Cheapest Flights Within K Stops, Path With Minimum Effort.*

17. Union-Find

The tell: “are these two connected”, asked many times, while things are being joined.

```
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # path halving
        x = parent[x]
    return x
```

Why it works: near-constant time per operation, with path compression and union by rank. Use it instead of repeated traversal when the graph is growing. *Number of Provinces, Redundant Connection, Accounts Merge, Kruskal's MST.*

18. Trie

The tell: prefixes of strings — or bits of numbers treated as a path.

```
node = root
for letter in word:
    node = node.children.setdefault(letter, {})
node["$"] = True
```

Why it works: shared prefixes are stored once, so lookup costs the length of the word rather than the size of the dictionary. *Implement Trie, Word Search II, Design Add and Search Words, Maximum XOR of Two Numbers.*

19. Dynamic programming

The tell: “how many ways”, or “the best value”, with subproblems that repeat.

```
dp[state] = best_or_count over transitions of
                dp[smaller_state]
```

Why it works: the same subproblem is solved once and reused. The whole difficulty is naming the state. *Climbing Stairs, House Robber, Coin Change, Longest Common Subsequence, Edit Distance, Unique Paths.*

20. Bits and maths

The tell: constant space with repeated values, subsets of a small set, or an answer modulo a prime.

```
XOR for pairs that cancel; n & (n-1) for set bits;  
1 << n for subsets; the sieve for many primes;  
Euclid for a GCD; square and multiply for a big power.
```

Single Number, Counting Bits, Subsets, Count Primes, Pow(x, n), Unique Paths as nCr.

The procedure

Four questions, in this order, and the first one is the one people skip.

One: what do the constraints forbid? $n \leq 20$ means exponential is intended. $n \leq 10^3$ means $O(n^2)$ is fine and a 2D table is in range. $n \leq 10^5$ means $O(n \log n)$ at worst. “O(1) extra space” forbids the hash map, which is usually the obvious answer.

Two: what is the shape of the output? “Return all” is backtracking — the output itself is exponential. “Return the number of ways” is DP or counting, never enumeration. “Return the best value” is DP or greedy. “Return a position” is binary search.

Three: what is the structure of the input? Sorted → two pointers or binary search. A tree → traversal. A grid → a graph. Prefixes of strings → a trie. Things being joined → union-find.

Four: is the obvious answer too slow, and in a fixable way? Nested loops recomputing the same sums → prefix sums. Nested loops finding the next bigger thing → monotonic stack. Recursion re-solving the same subproblem → DP. Sorting everything to get the top five → a heap.

4 The picture

The whole index, on one screen:

#	PATTERN	THE TELL	COST
1	Two pointers	sorted + a pair, or in place	$O(n)$ / $O(1)$
2	Sliding window	CONTIGUOUS stretch + constraint	$O(n)$ / $O(k)$
3	Fast and slow pointers	cycle, middle, nth from end	$O(n)$ / $O(1)$
4	Prefix sums	many range queries; sums to k	$O(n)$ then $O(1)$
5	Binary search	sorted, want a position	$O(\log n)$
6	Binary search on answer	"minimise the maximum"	$O(n \log \text{range})$
7	Sort, then sweep	intervals; greedy needing order	$O(n \log n)$
8	Hashing	"seen before"; counting	$O(n)$ / $O(n)$
9	Monotonic stack	next greater / previous smaller	$O(n)$ / $O(n)$
10	Heap	top-k, running median, k-merge	$O(n \log k)$
11	Linked list surgery	reverse, reorder, split in place	$O(n)$ / $O(1)$
12	Tree traversal	anything tree-shaped	$O(n)$ / $O(h)$
13	BST properties	inorder is sorted; prune bounds	$O(h)$
14	Backtracking	"return ALL"	$O(b^d)$
15	Graph traversal / topo	connected; prerequisites; grids	$O(V + E)$
16	Shortest paths	weighted, cheapest route	$O(E \log V)$
17	Union-Find	"connected?" while joining	$\sim O(1)$ each
18	Trie	prefixes; bits as a path	$O(\text{length})$
19	Dynamic programming	ways / best, overlapping subs	$O(\text{states} \times \text{trans})$
20	Bits and maths	$O(1)$ space + repeats; mod p	$O(n)$ or $O(\log n)$

The constraint table, which is the setter telling you the answer:

THE CONSTRAINT SAYS	THE INTENDED SOLUTION IS ABOUT
$n \leq 12$	permutations: $n!$ is 479 million
$n \leq 20$	subsets: 2^n is a million. Bitmask or backtracking.
$n \leq 100$	$O(n^3)$ is fine. Interval DP, Floyd-Warshall.
$n \leq 1,000$	$O(n^2)$ is fine. A 2D DP table.
$n \leq 100,000$	$O(n \log n)$. Sort, heap, binary search.
$n \leq 1,000,000$	$O(n)$. One pass, prefix sums, counting.
$n \leq 10^9$	$O(\log n)$ or a formula. NOT a loop.
" $O(1)$ extra space"	no map, no sort-copy. Two pointers, or XOR if values repeat.
"in place"	two pointers, or index-as-a-marker
"answer modulo $10^9 + 7$ "	counting: DP or combinatorics
"already sorted"	two pointers or binary search - and if you were going to sort anyway, ask why they told you
"return all"	backtracking; the output is exponential
"streaming / one pass"	heap, or running counters

The confusable pairs, drawn as the question that separates them:

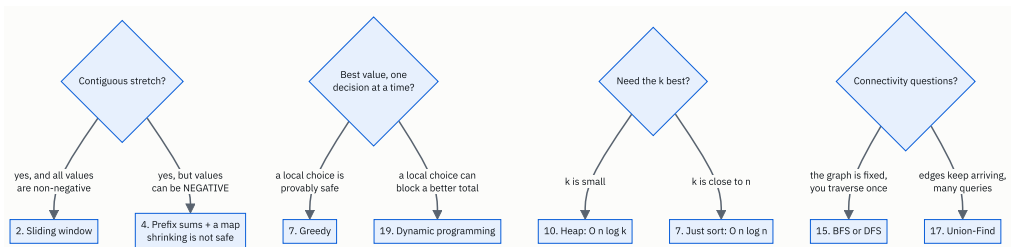


FIGURE 177.1

Every one of those four splits is a place where the wrong choice still produces plausible-looking code, and they are the four confusions worth being able to state out loud.

5 The code, built step by step

The patterns are data, so let us write them as data — and then a small tool that ranks them against a problem description. It is a keyword matcher and nothing cleverer, which is exactly the point: the recognition is a lookup, not an insight.

The patterns, as records

```
@dataclass(frozen=True)
class Pattern:
    number: int
    name: str
    tell: str
    cost: str
    triggers: tuple[str, ...] = field(default=())
```

`tell` is the sentence that gives the pattern away; `triggers` are the phrases that appear in problem statements. Writing them out is itself the revision — if you cannot list five trigger phrases for a pattern, you do not really have it.

Matching whole words, and why

```
def mentions(text: str, phrase: str) → bool:
    """Whole words only. Without the boundaries, 'substring' matches 'bst'."""
    return re.search(rf"(?<\w){re.escape(phrase)}(?!\w)", text) is not None
```

The first version of this used plain substring matching, and it classified “longest substring without repeating characters” as a binary-search-tree problem — because the letters `b`, `s`, `t` appear inside `substring`. That is a real bug from

writing this lesson, and it is a decent illustration of why the recognition step in your own head needs boundaries too. **“Sorted” in a problem statement is a trigger. “Assorted” is not.**

The classifier

```
def classify(description: str, top: int = 3) → list[tuple[Pattern, list[str]]]:
    """Score every pattern by how many of its trigger phrases appear. Keyword matching."""
    text = description.lower()
    scored = []
    for pattern in PATTERNS:
        hits = [phrase for phrase in pattern.triggers if mentions(text, phrase)]
        if hits:
            scored.append((pattern, hits))
    scored.sort(key=lambda pair: (-len(pair[1]), pair[0].number))
    return scored[:top]
```

Sorting by the number of matched triggers, then by pattern number for a stable tie-break. Returning the top three rather than one is deliberate — in a real interview you should have two or three candidate shapes in mind and choose between them, **not lock onto the first thing that fires.**

The constraint reader

```
def constraint_hints(description: str) → list[str]:
    """The constraints usually name the intended solution. Read them first."""
    text = description.lower()
    return [f"{phrase!r} → {advice}" for phrase, advice in CONSTRAINT_TELLS
            if mentions(text, phrase)]
```

Separate from the pattern matcher, because it is a separate habit. The constraints are read first and they often decide the answer on their own — $n \leq 20$ and “return all” between them rule out fifteen of the twenty patterns before you have thought about the problem at all.

Seven templates, verbatim

These are the ones worth being able to type without thinking. Each is short enough to write from memory under a clock.

```
def two_pointers_pair_sum(numbers: list[int], target: int) → tuple[int, int] | None:
    """Sorted input. Too big → move right in. Too small → move left out."""
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return left, right
        if total < target:
            left += 1
        else:
            right -= 1
    return None
```

```
def sliding_window_longest_unique(text: str) → int:
    """Grow the right edge always; pull the left edge in only while the rule is broken."""
    last_seen: dict[str, int] = {}
    best = left = 0
    for right, letter in enumerate(text):
        if letter in last_seen and last_seen[letter] ≥ left:
            left = last_seen[letter] + 1
        last_seen[letter] = right
        best = max(best, right - left + 1)
    return best
```

`last_seen[letter] ≥ left` is the line that matters. A letter seen before the window started is not in the window, and forgetting that comparison is the classic sliding-window bug.

```
def binary_search_on_answer(weights: list[int], days: int) → int:
    """Guess a capacity, ask 'is it enough', and halve the range. The answer is monotone."""
    def days_needed(capacity: int) → int:
        used, current = 1, 0
        for weight in weights:
            if current + weight > capacity:
                used += 1
                current = 0
            current += weight
        return used

    low, high = max(weights), sum(weights)
    while low < high:
        middle = (low + high) // 2
        if days_needed(middle) ≤ days:
            high = middle
        else:
            low = middle + 1
    return low
```

`low = max(weights)` and not `min` — a capacity smaller than the heaviest single item can never work. Getting the bounds right is most of this pattern, and the feasibility function is usually the easy half.

```
def monotonic_stack_next_greater(numbers: list[int]) → list[int]:
    """Keep a stack of positions still waiting for an answer. Each is pushed and popped once."""
    answer = [-1] * len(numbers)
    waiting: list[int] = []
    for i, value in enumerate(numbers):
        while waiting and numbers[waiting[-1]] < value:
            answer[waiting.pop()] = value
        waiting.append(i)
    return answer
```

The stack holds positions, not values, because you have to write the answer back where it belongs.

```
def bfs_shortest_unweighted(graph: dict[int, list[int]], start: int, goal: int) → int:
    """Unweighted → BFS, and the first time you reach a node is the shortest way."""
    from collections import deque
    seen = {start}
    queue = deque([(start, 0)])
    while queue:
        node, distance = queue.popleft()
        if node == goal:
            return distance
        for neighbour in graph.get(node, []):
            if neighbour not in seen:
                seen.add(neighbour)
                queue.append((neighbour, distance + 1))
    return -1
```

Mark as seen when you enqueue, not when you dequeue. Marking on dequeue lets the same node enter the queue many times, which is the commonest BFS bug and turns a linear traversal into something much worse.

```
def backtracking_subsets(items: list[int]) → list[list[int]]:
    """Choose, recurse, un-choose. The un-choose is the line people forget."""
    out: list[list[int]] = []
    chosen: list[int] = []

    def walk(start: int) → None:
        out.append(chosen[:])
        for i in range(start, len(items)):
            chosen.append(items[i])
            walk(i + 1)
            chosen.pop()

    walk(0)
    return out
```

`chosen[:]` copies. Appending `chosen` itself stores a reference to a list that is about to change, and you end up with a result full of identical empty lists.

```
def dp_one_dimension(coins: list[int], amount: int) → int:
    """Fewest coins. State = the amount; transition = one coin. Overlapping subproblems."""
    best = [0] + [float("inf")] * amount
    for value in range(1, amount + 1):
        for coin in coins:
            if coin ≤ value:
                best[value] = min(best[value], best[value - coin] + 1)
    return -1 if best[amount] == float("inf") else int(best[amount])
```

Say the state out loud before writing the loop. “`best[v]` is the fewest coins that make exactly `v`.” If you cannot say that sentence, the loop will be wrong in a way that is very hard to find.

The complete solution

```
"""Day 177 - the twenty patterns as data, plus a classifier that ranks them by trigger."""

from __future__ import annotations

import re
from dataclasses import dataclass, field

@dataclass(frozen=True)
class Pattern:
    number: int
    name: str
    tell: str
    cost: str
    triggers: tuple[str, ...] = field(default=())

PATTERNS: list[Pattern] = [
    Pattern(1, "Two pointers",
        "sorted input, and you are looking for a PAIR or moving in place",
        "O(n) time, O(1) space",
        ("sorted", "pair", "two numbers", "in place", "palindrome",
         "remove duplicates", "reverse", "container", "closest")),
    Pattern(2, "Sliding window",
        "a CONTIGUOUS subarray or substring, with a constraint",
        "O(n) time, O(k) space",
        ("subarray", "substring", "contiguous", "consecutive", "at most",
         "longest", "shortest", "window", "without repeating")),
    Pattern(3, "Fast and slow pointers",
        "a cycle, a middle, or an nth-from-the-end, in one pass",
        "O(n) time, O(1) space",
        ("cycle", "middle", "loop", "nth from the end", "linked list",
         "duplicate number", "happy number")),
    Pattern(4, "Prefix sums and difference arrays",
        "many RANGE queries, or 'subarray sums to k'",
        "O(n) build, O(1) per query",
        ("range sum", "subarray sum", "sums to", "between indices",
         "range update", "running total", "equals k")),
    Pattern(5, "Binary search on a sorted array",
        "sorted, and you want a position",
        "O(log n) time, O(1) space",
        ("sorted", "search", "find the position", "insert position",
         "rotated", "first occurrence", "last occurrence")),
    Pattern(6, "Binary search on the answer",
        "MINIMISE THE MAXIMUM, or 'smallest capacity such that'",
        "O(n log range) time",
        ("minimise the maximum", "minimum largest", "smallest capacity",
         "minimum capacity", "minimum speed", "k days", "split into k",
         "maximise the minimum")),
    Pattern(7, "Sort, then sweep",
        "intervals, or a greedy choice that needs the right order",
        "O(n log n) time",
        ("intervals", "meetings", "merge", "overlap", "schedule",
         "non-overlapping", "earliest", "deadline", "rooms")),
    Pattern(8, "Hashing: maps, sets and frequency",
        "'have I seen this', or counting occurrences",
        "O(n) time, O(n) space",
        ("count", "frequency", "duplicate", "anagram", "seen before",
         "group", "unique", "two sum")),
    Pattern(9, "Stack, and the monotonic stack",
        "matching pairs, or NEXT GREATER / previous smaller",
        "O(n) time, O(n) space",
        ("next greater", "previous smaller", "brackets", "parentheses",
         "valid", "nesting", "histogram", "temperatures", "undo")),
    Pattern(10, "Heap: top-k and k-way merge",
        "the k largest, a running median, or merging sorted lists",
        "O(n log k) time, O(k) space",
        ("k largest", "k smallest", "top k", "most frequent", "median",
         "merge k", "closest points", "priority")),
    Pattern(11, "Linked list pointer surgery",
        "reversing, reordering, or splitting a list in place",
        "O(n) time, O(1) space",
        ("linked list", "reverse", "reorder", "swap nodes", "dummy",
         "merge two lists", "partition list")),
    Pattern(12, "Tree traversal, depth-first and breadth-first",
```

```

        "anything about a tree that is not specifically a BST",
        "O(n) time, O(h) or O(width) space",
        ("tree", "depth", "path", "level order", "leaves", "diameter",
         "lowest common ancestor", "serialize")),
    Pattern(13, "Binary search tree properties",
            "a BST - inorder is sorted, and you can prune by bounds",
            "O(h) time",
            ("binary search tree", "bst", "inorder", "kth smallest",
             "validate", "insert into", "delete from")),
    Pattern(14, "Backtracking",
            "GENERATE all valid arrangements, with pruning",
            "O(branches^depth)",
            ("all permutations", "all combinations", "all subsets", "generate",
             "n-queens", "sudoku", "word search", "partition into")),
    Pattern(15, "Graph traversal and topological sort",
            "things connected to things; ordering with prerequisites",
            "O(V + E) time",
            ("graph", "connected", "islands", "course", "prerequisite",
             "dependency", "cycle detection", "reachable", "shortest path unweighted")),
    Pattern(16, "Shortest paths",
            "weighted edges and a cheapest route",
            "Dijkstra O(E log V)",
            ("shortest path", "cheapest", "minimum cost", "weighted",
             "network delay", "flights", "cheapest flights")),
    Pattern(17, "Union-Find",
            "'are these two connected', many times, with things joining",
            "near O(1) per operation",
            ("connected components", "union", "merge accounts", "redundant connection",
             "number of provinces", "minimum spanning tree", "kruskal")),
    Pattern(18, "Trie",
            "PREFIXES of strings, or bits treated as a path",
            "O(length) per operation",
            ("prefix", "autocomplete", "dictionary", "word search ii",
             "starts with", "maximum xor", "replace words")),
    Pattern(19, "Dynamic programming",
            "count the ways, or the best value, with OVERLAPPING subproblems",
            "O(states x transitions)",
            ("how many ways", "number of ways", "minimum cost", "maximum profit",
             "longest", "can you reach", "coin change", "make change", "knapsack",
             "edit distance", "stock")),
    Pattern(20, "Bits and maths",
            "constant space with repeats, subsets of a small set, or a modulus",
            "usually O(n) or O(log n)",
            ("xor", "bits", "set bits", "power of two", "prime", "gcd",
             "modulo", "10^9 + 7", "appears twice", "n ≤ 20")),
]

CONSTRAINT_TELLS: list[tuple[str, str]] = [
    ("o(1) extra space", "with repeats → XOR (20). Otherwise two pointers (1)."),
    ("constant extra space", "with repeats → XOR (20). Otherwise two pointers (1)."),
    ("n ≤ 20", "bitmask over subsets (20), or backtracking (14)."),
    ("n ≤ 25", "meet in the middle, or bitmask (20)."),
    ("modulo 10^9 + 7", "counting: DP (19) or combinatorics (20)."),
    ("already sorted", "two pointers (1) or binary search (5)."),
    ("sorted", "two pointers (1) or binary search (5)."),
    ("o(log n)", "binary search (5) or a heap (10)."),
    ("n up to 10^5", "O(n log n) at worst - so sorting is affordable, O(n^2) is not."),
    ("n up to 10^3", "O(n^2) is affordable - a 2D DP table is in range (19)."),
    ("return all", "backtracking (14) - the output itself is exponential."),
    ("return the number of ways", "DP (19) or combinatorics (20), never enumeration."),
]

def mentions(text: str, phrase: str) → bool:
    """Whole words only. Without the boundaries, 'substring' matches 'bst'."""
    return re.search(rf"(?<\w){re.escape(phrase)}(?!\w)", text) is not None

def classify(description: str, top: int = 3) → list[tuple[Pattern, list[str]]]:
    """Score every pattern by how many of its trigger phrases appear. Keyword matching."""
    text = description.lower()
    scored = []
    for pattern in PATTERNS:
        hits = [phrase for phrase in pattern.triggers if mentions(text, phrase)]
        if hits:
            scored.append((pattern, hits))
    scored.sort(key=lambda pair: (-len(pair[1]), pair[0].number))
    return scored[:top]

```

```

def constraint_hints(description: str) → list[str]:
    """The constraints usually name the intended solution. Read them first."""
    text = description.lower()
    return [{"phrase!r" → {advice}" for phrase, advice in CONSTRAINT_TELLS
            if mentions(text, phrase)]

# ----- templates

def two_pointers_pair_sum(numbers: list[int], target: int) → tuple[int, int] | None:
    """Sorted input. Too big → move right in. Too small → move left out."""
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return left, right
        if total < target:
            left += 1
        else:
            right -= 1
    return None

def sliding_window_longest_unique(text: str) → int:
    """Grow the right edge always; pull the left edge in only while the rule is broken."""
    last_seen: dict[str, int] = {}
    best = left = 0
    for right, letter in enumerate(text):
        if letter in last_seen and last_seen[letter] ≥ left:
            left = last_seen[letter] + 1
        last_seen[letter] = right
        best = max(best, right - left + 1)
    return best

def binary_search_on_answer(weights: list[int], days: int) → int:
    """Guess a capacity, ask 'is it enough', and halve the range. The answer is monotone."""
    def days_needed(capacity: int) → int:
        used, current = 1, 0
        for weight in weights:
            if current + weight > capacity:
                used += 1
                current = 0
            current += weight
        return used

    low, high = max(weights), sum(weights)
    while low < high:
        middle = (low + high) // 2
        if days_needed(middle) ≤ days:
            high = middle
        else:
            low = middle + 1
    return low

def monotonic_stack_next_greater(numbers: list[int]) → list[int]:
    """Keep a stack of positions still waiting for an answer. Each is pushed and popped once."""
    answer = [-1] * len(numbers)
    waiting: list[int] = []
    for i, value in enumerate(numbers):
        while waiting and numbers[waiting[-1]] < value:
            answer[waiting.pop()] = value
        waiting.append(i)
    return answer

def bfs_shortest_unweighted(graph: dict[int, list[int]], start: int, goal: int) → int:
    """Unweighted → BFS, and the first time you reach a node is the shortest way."""
    from collections import deque
    seen = {start}
    queue = deque([(start, 0)])
    while queue:
        node, distance = queue.popleft()
        if node == goal:
            return distance
        for neighbour in graph.get(node, ()):
            if neighbour not in seen:

```



```

        seen.add(neighbour)
        queue.append((neighbour, distance + 1))
    return -1

def backtracking_subsets(items: list[int]) → list[list[int]]:
    """Choose, recurse, un-choose. The un-choose is the line people forget."""
    out: list[list[int]] = []
    chosen: list[int] = []

    def walk(start: int) → None:
        out.append(chosen[:])
        for i in range(start, len(items)):
            chosen.append(items[i])
            walk(i + 1)
            chosen.pop()

    walk(0)
    return out

def dp_one_dimension(coins: list[int], amount: int) → int:
    """Fewest coins. State = the amount; transition = one coin. Overlapping subproblems."""
    best = [0] + [float("inf")] * amount
    for value in range(1, amount + 1):
        for coin in coins:
            if coin ≤ value:
                best[value] = min(best[value], best[value - coin] + 1)
    return -1 if best[amount] == float("inf") else int(best[amount])

if __name__ == "__main__":
    print("THE TWENTY PATTERNS")
    for p in PATTERNS:
        print(f" {p.number:>2}. {p.name:<44} {p.cost}")

    print()
    print("THE CLASSIFIER, ON EIGHT REAL PROBLEM DESCRIPTIONS")
    problems = [
        "Given a sorted array, find two numbers that add up to a target.",
        "Find the length of the longest substring without repeating characters.",
        "Given an array, return the number of ways to make change for an amount.",
        "Find the minimum capacity of a ship to carry all weights within k days.",
        "Return all subsets of a list where n ≤ 20.",
        "Given a list of intervals, merge all overlapping intervals.",
        "For each temperature, find how many days until a warmer one. Next greater.",
        "Every number appears twice except one. Find it with O(1) extra space.",
    ]
    for text in problems:
        print(f"\n \ {text}\n")
        for hint in constraint_hints(text):
            print(f"    constraint: {hint}")
        for pattern, hits in classify(text):
            print(f"    → {pattern.number:>2}. {pattern.name:<38} (matched: {' ', ' '.join(hits)})")

    print()
    print("SEVEN TEMPLATES, RUN")
    print(f" two pointers      two_pointers_pair_sum([1,3,4,6,8,11], 10) = "
          f"{two_pointers_pair_sum([1, 3, 4, 6, 8, 11], 10)}")
    print(f" sliding window    longest unique in 'abcabcbb' = "
          f"{sliding_window_longest_unique('abcabcbb')}")
    print(f" search the answer capacity for [1..10] in 5 days = "
          f"{binary_search_on_answer(list(range(1, 11)), 5)}")
    print(f" monotonic stack  next greater of [73,74,75,71,69,72,76,73] = "
          f"{monotonic_stack_next_greater([73, 74, 75, 71, 69, 72, 76, 73])}")
    graph = {1: [2, 3], 2: [4], 3: [4, 5], 4: [6], 5: [6], 6: []}
    print(f" BFS              shortest 1 → 6 = {bfs_shortest_unweighted(graph, 1, 6)}")
    print(f" backtracking     subsets of [1,2,3] = {backtracking_subsets([1, 2, 3])}")
    print(f" 1-D DP           fewest coins for 11 from [1,2,5] = "
          f"{dp_one_dimension([1, 2, 5], 11)}")

    print()
    print("VERIFICATION")
    bad = 0
    if two_pointers_pair_sum([1, 3, 4, 6, 8, 11], 10) ≠ (2, 3):
        bad += 1
    if sliding_window_longest_unique("abcabcbb") ≠ 3:
        bad += 1
    if sliding_window_longest_unique("bbbbbb") ≠ 1:

```

```

        bad += 1
    if sliding_window_longest_unique("") != 0:
        bad += 1
    if binary_search_on_answer([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5) != 15:
        bad += 1
    if monotonic_stack_next_greater([73, 74, 75, 71, 69, 72, 76, 73]) != [74, 75, 76, 72, 72, 76, -1, -1]:
        bad += 1
    if bfs_shortest_unweighted(graph, 1, 6) != 3:
        bad += 1
    if len(backtracking_subsets([1, 2, 3, 4, 5])) != 32:
        bad += 1
    if dp_one_dimension([1, 2, 5], 11) != 3 or dp_one_dimension([2], 3) != -1:
        bad += 1
    if len(PATTERNS) != 20:
        bad += 1
    if len({p.number for p in PATTERNS}) != 20:
        bad += 1
    print(f" {bad} failures across 11 checks, including that there are exactly 20 patterns")

```

Running it:

THE TWENTY PATTERNS

1. Two pointers	$O(n)$ time, $O(1)$ space
2. Sliding window	$O(n)$ time, $O(k)$ space
3. Fast and slow pointers	$O(n)$ time, $O(1)$ space
4. Prefix sums and difference arrays	$O(n)$ build, $O(1)$ per query
5. Binary search on a sorted array	$O(\log n)$ time, $O(1)$ space
6. Binary search on the answer	$O(n \log \text{range})$ time
7. Sort, then sweep	$O(n \log n)$ time
8. Hashing: maps, sets and frequency	$O(n)$ time, $O(n)$ space
9. Stack, and the monotonic stack	$O(n)$ time, $O(n)$ space
10. Heap: top-k and k-way merge	$O(n \log k)$ time, $O(k)$ space
11. Linked list pointer surgery	$O(n)$ time, $O(1)$ space
12. Tree traversal, depth-first and breadth-first	$O(n)$ time, $O(h)$ or $O(\text{width})$ space
13. Binary search tree properties	$O(h)$ time
14. Backtracking	$O(\text{branches}^{\text{depth}})$
15. Graph traversal and topological sort	$O(V + E)$ time
16. Shortest paths	Dijkstra $O(E \log V)$
17. Union-Find	near $O(1)$ per operation
18. Trie	$O(\text{length})$ per operation
19. Dynamic programming	$O(\text{states} \times \text{transitions})$
20. Bits and maths	usually $O(n)$ or $O(\log n)$

THE CLASSIFIER, ON EIGHT REAL PROBLEM DESCRIPTIONS

"Given a sorted array, find two numbers that add up to a target." constraint: 'sorted' $\rightarrow$ two pointers (1) or binary search (5).	
$\rightarrow$ 1. Two pointers	(matched: sorted, two numbers)
$\rightarrow$ 5. Binary search on a sorted array	(matched: sorted)
"Find the length of the longest substring without repeating characters." $\rightarrow$ 2. Sliding window (matched: substring, longest, without repeating) $\rightarrow$ 19. Dynamic programming (matched: longest)	
"Given an array, return the number of ways to make change for an amount." constraint: 'return the number of ways' $\rightarrow$ DP (19) or combinatorics (20), never enumeration. $\rightarrow$ 19. Dynamic programming (matched: number of ways, make change)	
"Find the minimum capacity of a ship to carry all weights within k days." $\rightarrow$ 6. Binary search on the answer (matched: minimum capacity, k days)	
"Return all subsets of a list where $n \leq 20$." constraint: ' $n \leq 20$ ' $\rightarrow$ bitmask over subsets (20), or backtracking (14). constraint: 'return all' $\rightarrow$ backtracking (14) - the output itself is exponential. $\rightarrow$ 14. Backtracking (matched: all subsets) $\rightarrow$ 20. Bits and maths (matched: $n \leq 20$)	
"Given a list of intervals, merge all overlapping intervals." $\rightarrow$ 7. Sort, then sweep (matched: intervals, merge)	
"For each temperature, find how many days until a warmer one. Next greater." $\rightarrow$ 9. Stack, and the monotonic stack (matched: next greater)	
"Every number appears twice except one. Find it with $O(1)$ extra space." constraint: 'o(1) extra space' $\rightarrow$ with repeats $\rightarrow$ XOR (20). Otherwise two pointers (1). $\rightarrow$ 20. Bits and maths (matched: appears twice)	

SEVEN TEMPLATES, RUN

two pointers	two_pointers_pair_sum([1,3,4,6,8,11], 10) = (2, 3)
sliding window	longest unique in 'abcabcbb' = 3
search the answer	capacity for [1..10] in 5 days = 15
monotonic stack	next greater of [73,74,75,71,69,72,76,73] = [74, 75, 76, 72, 72, 76, -1, -1]
BFS	shortest 1 $\rightarrow$ 6 = 3
backtracking	subsets of [1,2,3] = [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
1-D DP	fewest coins for 11 from [1,2,5] = 3

VERIFICATION

0 failures across 11 checks, including that there are exactly 20 patterns

Look at the second problem: sliding window is first, and dynamic programming also fires on “longest”. That is correct behaviour, not noise. “Longest” genuinely appears in both families — longest substring without repeating characters is a window; longest increasing subsequence is DP — and the thing that separates them is whether the answer has to be contiguous. The classifier cannot know that. You have to.

Look at the fifth: two constraint hints fire before any pattern does, and between them they have already told you the answer. That is the habit the whole lesson is trying to build.

And look at the last one: “appears twice” plus “ $O(1)$ extra space”. Neither phrase mentions bits at all, and together they name the technique exactly.

6 What it costs

The complexity ladder, which is the thing to have in your head when you read a constraint.

at roughly 10^8 simple operations per second:

$O(1)$	any n	
$O(\log n)$	any n	$n = 10^9 \rightarrow 30$ steps
$O(n)$	$n \leq 100,000,000$	
$O(n \log n)$	$n \leq 5,000,000$	sorting is affordable
$O(n \sqrt{n})$	$n \leq 1,000,000$	
$O(n^2)$	$n \leq 10,000$	
$O(n^2 \log n)$	$n \leq 2,000$	
$O(n^3)$	$n \leq 500$	
$O(2^n)$	$n \leq 22$	
$O(n * 2^n)$	$n \leq 20$	
$O(n!)$	$n \leq 11$	

Read that table backwards to use it. $n = 500$ in the statement means $O(n^3)$ is intended, and that is Floyd-Warshall or interval DP. $n = 20$ means exponential is intended, and that is a bitmask.

What each pattern costs, and what it replaced.

PATTERN	COST	REPLACES	GAIN AT $n = 100,000$
Two pointers	$O(n)$	$O(n^2)$ nested loops	$10^{10} \rightarrow 10^5$
Sliding window	$O(n)$	$O(n*k)$ re-scanning	depends on k
Prefix sums	$O(n) + O(1)/q$	$O(n)$ per query	1,000 queries: $10^8 \rightarrow 10^5$
Binary search	$O(\log n)$	$O(n)$ scan	$100,000 \rightarrow 17$
Search the answer	$O(n \log \text{range})$	$O(\text{range} * n)$	$10^9 \text{ range} \rightarrow 30$ feasibility checks
Sort then sweep	$O(n \log n)$	$O(n^2)$ pairwise	$10^{10} \rightarrow 1.7 \times 10^6$
Hashing	$O(n)$	$O(n^2)$ search	$10^{10} \rightarrow 10^5$
Monotonic stack	$O(n)$	$O(n^2)$ look-ahead	$10^{10} \rightarrow 10^5$
Heap top-k	$O(n \log k)$	$O(n \log n)$ full sort	$k=10$: 3.3x fewer comparisons
Union-Find	$\sim O(1)$ each	$O(V+E)$ per query	1,000 queries: $10^8 \rightarrow 10^3$
Trie	$O(\text{length})$	$O(\text{dictionary size})$	50,000 words $\rightarrow 10$
DP	$O(\text{states})$	$O(2^n)$ recomputation	$2^{40} \rightarrow 10^6$
Bitmask	$O(n * 2^n)$	$O(n!)$	$n=20$: $10^{18} \rightarrow 10^7$

Every row is the same move: notice that work is being repeated, and stop repeating it.

Space, which is asked less and matters as much.

$O(1)$ two pointers, fast/slow, binary search, XOR, in-place reversal, bit tricks
 $O(k)$ sliding window (the window's contents), heap of size k
 $O(n)$ hashing, prefix sums, union-find, 1-D DP, BFS queue, recursion on a linked list
 $O(h)$ tree recursion - h is the HEIGHT, so $O(\log n)$ balanced and $O(n)$ in a stick
 $O(n^2)$ 2-D DP tables, adjacency matrices
 $O(2^n)$ the output of a subsets problem, which is not a cost you can avoid

→ When a problem says $O(1)$ space, it is forbidding the hash map. That is nearly always the point.

And the cost of the recognition itself, which is the real subject.

```
NOT recognising the pattern:  
~9 minutes of brute force  
then the shape arrives with 20 minutes left  
→ a rushed implementation, and no time to test
```

```
Recognising it in 30 seconds:  
3 minutes stating the plan and the cost  
10 minutes writing it carefully  
5 minutes testing  
→ and time left for the follow-up
```

Same knowledge. Same code. Entirely different interview.

7 The traps

These are not coding bugs. They are recognition failures — the pattern that looks right and is not.

Sliding window on an array containing negative numbers.

```
"longest subarray with sum at most k", a = [4, -1, 2, 1], k = 4
```

The window logic says: when the sum exceeds k, shrink from the left.

With negatives that is **WRONG** - shrinking can **INCREASE** the sum, because you might remove a negative number.

→ the answer is prefix sums plus a map, not a window.

The window pattern needs the invariant that removing an element moves you in a predictable direction. Non-negative values give you that; negatives do not. This produces plausible wrong answers on some inputs and correct ones on others, which is the worst kind of failure.

Greedy where a local choice can block a better total.

```
coins = [1, 3, 4], amount = 6
```

```
greedy, biggest first: 4 + 1 + 1 = three coins  
optimal:              3 + 3      = two coins
```

Greedy is right only when you can argue that taking the best-looking option now never prevents a better total later. If you cannot make that argument in a sentence, it is DP. And “it passed the examples” is not the argument.

Two pointers on unsorted input.

The entire justification is that when the sum is too small, the only way to increase it is to move `left` inward. On unsorted data that is not true, and the loop happily walks past the answer with no error at all.

BFS marking as seen on dequeue rather than enqueue.

```
while queue:
    node = queue.popleft()
    seen.add(node)          # TOO LATE
    for neighbour in graph[node]:
        if neighbour not in seen:
            queue.append(neighbour)
```

The same node gets enqueued many times before it is ever dequeued. The answer is still correct. The queue can grow to $O(E)$ instead of $O(V)$, and on a dense graph the program simply stops finishing.

Dijkstra on negative edges.

Dijkstra assumes that once a node is popped, its distance is final — which is only true if edges never reduce a distance later. With a negative edge that assumption breaks and you get a wrong answer, silently. Bellman-Ford handles it; say the name.

Backtracking without the undo.

```
chosen.append(items[i])
walk(i + 1)
# chosen.pop() ← missing
```

Every branch inherits the previous branch's choices, and the output is a growing list of nonsense. The undo is one line and it is the line people forget under pressure.

Appending the mutable state instead of a copy.

```
out.append(chosen)          # WRONG
out.append(chosen[:])       # right
```

```
[[[]], [], [], [], [], [], [], []]
```

Eight identical empty lists, because every entry is the same object and it was emptied on the way back up. **No error, and the shape of the output is right, which makes it confusing.**

Recursion depth on a linked list or a degenerate tree.

```
def length(node):  
    return 0 if node is None else 1 + length(node.next)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 9, in <module>  
  File "<stdin>", line 8, in length  
  File "<stdin>", line 8, in length  
  File "<stdin>", line 8, in length  
  [Previous line repeated 996 more times]  
RecursionError: maximum recursion depth exceeded
```

Python's default limit is 1,000. A tree that is really a chain of 10,000 nodes hits it, and so does a linked list. Use an iterative version, and say why.

Integer division on the binary search midpoint.

`(low + high) // 2` is safe in Python because integers are unbounded. In C++ or Java it can overflow when both are near the maximum, and the standard fix is `low + (high - low) // 2`. Worth knowing because it gets asked, and because it was a real bug in the JDK for nine years.

And the meta-trap: locking onto the first pattern that fires.

"Longest" fires both sliding window and DP. **"Sorted"** fires both two pointers and binary search. **"Minimum cost"** fires both Dijkstra and DP. Hold two or three candidates and choose between them with a second question — is it contiguous? is the graph weighted? — **rather than committing to the first match and forcing the problem to fit.**

8 In the interview

How it gets asked

- *"How would you approach this?"* — before any code. The pattern name belongs in the first sentence.
- *"Have you seen a problem like this before?"* — they mean the shape, not the exact problem.

- “Why that approach?” — the justification, which is what separates recognition from guessing.
- “Can you do better?” — usually means a different pattern entirely, not a tweak.
- “What if the array were not sorted?” — checking whether you know why your solution works.

The first ninety seconds

Use this shape on every problem, whatever it is. Four moves.

“Let me read the constraints first.”

“n is up to a hundred thousand, so $O(n^2)$ is out — that would be ten billion operations. $O(n \log n)$ is fine. And it says the array is already sorted, which is unusual to mention unless it matters.”

“So the shape I think this is...”

“It is asking for a pair with a given sum, in a sorted array. That is two pointers — one at each end, moving inward.”

“And the reason that works is...”

“Because sorting makes the decision unambiguous. If the sum is too small, the only way to increase it is to move the left pointer in — the right one is already at the largest value available. So each step eliminates a possibility for certain, and nothing is missed.”

“Cost, before I write...”

“ $O(n)$ time, since each pointer only moves inward, and $O(1)$ extra space. The alternative is a hash map, which is also $O(n)$ time but $O(n)$ space — and given the array is sorted, two pointers is strictly better.”

That is under a minute, and it does four things: it shows you read the constraints, it names the pattern, **it justifies the pattern rather than reciting it**, and it states the cost before writing rather than being asked afterwards.

The follow-ups

“Given this problem, which pattern, and why that one?”

“I work through four questions, and they are quick.

First, what do the constraints forbid? That is the highest-value question and it is the one people skip. $n \leq 20$ means exponential is intended, so it is a bit-mask or backtracking. $n \leq 1,000$ means $O(n^2)$ is fine and a two-dimensional table is in range. $n \leq 10^9$ means no loop over n at all — a formula or something logarithmic. And ‘constant extra space’ is almost always forbidding a hash map, which is the obvious answer they are steering me away from.

Second, what shape is the output? ‘Return all’ is backtracking — the output itself is exponential, so the cost is not a failure. ‘Return the number of ways’ is dynamic programming or counting, never enumeration. ‘Return the best value’ is DP or greedy. ‘Return a position’ is binary search.

Third, what structure does the input have? Sorted means two pointers or binary search. A tree means traversal. A grid is a graph — that is the one people miss most often. Prefixes of strings means a trie. Things being joined together, with repeated connectivity questions, means union-find.

Fourth, if the obvious answer is too slow, what exactly is it repeating? Re-summing the same range means prefix sums. Re-scanning ahead for the next larger element means a monotonic stack. Re-solving the same subproblem means DP. Sorting everything to get the top five means a heap.

And I would hold two or three candidates rather than one. ‘Longest’ fires both sliding window and DP, and the question that separates them is whether the answer has to be contiguous. ‘Minimum cost’ fires both Dijkstra and DP, and the question is whether it is a graph with weights. Committing to the first match and forcing the problem to fit is how people lose twenty minutes.“

“What if I told you the array was not sorted?”

“Then two pointers is gone, and I want to say why rather than just switching.

The entire justification for two pointers is that when the sum is too small, moving the left pointer inward is the only way to increase it — because the right pointer is already on the largest available value. On unsorted data that is not true, and the loop walks straight past the answer with no error at all. That is a silent wrong answer, which is the worst kind.

So I have two options and the choice depends on the constraints.

Sort it first, then two pointers: $O(n \log n)$ time and $O(1)$ extra space — though sorting destroys the original positions, so if the answer is a pair of indices I would need to sort pairs of value and index.

Or a hash map in one pass: $O(n)$ time and $O(n)$ space. For each value, ask whether `target - value` has been seen already.

If the problem said ‘constant extra space’, that sentence has just ruled out the map and told me to sort. If it said ‘ $O(n)$ time’, it has ruled out the sort and told me to use the map. Which is really the answer to your question: the constraints choose, not my preference.

And the general habit this points at: when a problem tells me something is sorted, I ask why they mentioned it. If they went to the trouble of saying it, the intended solution almost certainly uses it.”

“You said sliding window. What would break that?”

“Negative numbers, and it is worth being precise about why.

The window pattern relies on one invariant: removing an element from the left moves the window’s value in a predictable direction. With non-negative values, **shrinking always reduces the sum**, so ‘while the window is invalid, shrink’ is a correct rule.

With negatives, shrinking can increase the sum, because the element you removed was pulling it down. **The ‘while invalid, shrink’ loop then either shrinks past the answer or stops in the wrong place** — and the failure is input-dependent, so it passes some tests and fails others.

The replacement is prefix sums with a map. Compute a running total, and for each position ask how many earlier prefix values would give the target difference. That works regardless of sign, because it never assumes anything about direction.

The same class of question applies elsewhere and I would look for it. Greedy needs an exchange argument — that taking the best-looking option now never blocks a better total later; `coins = [1, 3, 4]` and `amount = 6` breaks it, because greedy takes $4 + 1 + 1$ and the answer is $3 + 3$. Dijkstra needs non-negative edge weights, because it assumes a popped node is final. Binary search needs monotonicity — if 15 works, 16 must work too.

In every case the pattern has a precondition, the precondition is one sentence, and the failure when it is violated is a wrong answer rather than a crash. So I state the precondition out loud when I name the pattern, which is also the fastest way for you to stop me if I have got it wrong.“

The model answer

“You have solved a few hundred problems. What did you actually learn?”

“That there are about twenty shapes, and almost everything is one of them.

Not twenty algorithms — twenty shapes. ‘A contiguous stretch with a constraint’ is one shape, and it does not matter whether the elements are letters, numbers or prices. ‘Every subset of a small set’ is another. ‘The best value with overlapping subproblems’ is another. Once I have named the shape, the code is something I have already written many times, and the interview stops being about invention.

The skill that actually improved is recognition, not implementation. Nobody forgets how to write a sliding window. What happens under pressure is that the problem arrives, I do not recognise it, and I spend nine minutes on a brute force — and then the shape arrives with twenty minutes left and a rushed implementation. Same knowledge, same code, entirely different result.

So the habit I built is to read the constraints before the problem. $n \leq 20$ means exponential is intended. $n \leq 1,000$ means a two-dimensional table is in range. $n \leq 10^9$ means no loop at all. ‘Constant extra space’ is almost always forbidding the hash map that would otherwise be the obvious answer. The setter tells you the intended solution in the constraints, and most candidates read them last.

The second habit is holding two or three candidate shapes rather than one. ‘Longest’ fires both sliding window and DP; the separating question is whether the answer must be contiguous. ‘Minimum cost’ fires both Dijkstra and DP; the question is whether it is weighted graph. Committing to the first thing that fires is how you spend twenty minutes forcing a problem to fit.

The third is stating the precondition when I name the pattern. Two pointers needs sorted input. Sliding window needs values that do not change direction on you — negatives break it. Greedy needs an exchange argument. Dijkstra needs non-negative weights. Binary search needs monotonicity. Every one of those, when violated, gives a wrong answer rather than a crash — so saying the precondition out loud is both how I check myself and how you can stop me early if I am wrong.

And the thing underneath all of it, which took much longer to learn than any pattern. Almost every one of the twenty is the same move: notice that work is being repeated, and stop repeating it. Prefix sums stop re-summing.

A monotonic stack stops re-scanning ahead. DP stops re-solving. A heap stops sorting things you were going to throw away. A sieve stops asking each number a question the group could answer.

So when I meet something genuinely new, and none of the twenty fit, that is the question I fall back on: what is this doing more than once?"

9 Recall card

Twenty shapes, not twenty algorithms. 1 **two pointers** (sorted + a pair) · 2 **sliding window** (contiguous + constraint) · 3 **fast/slow** (cycle, middle) · 4 **prefix sums** (range queries, sums to k) · 5 **binary search** (sorted, want a position) · 6 **binary search on the ANSWER** ("minimise the maximum") · 7 **sort then sweep** (intervals, greedy) · 8 **hashing** ("seen before", counting) · 9 **monotonic stack** (next greater) · 10 **heap** (top-k, k-merge) · 11 **linked-list surgery** · 12 **tree traversal** · 13 **BST properties** · 14 **backtracking** ("return ALL") · 15 **graph traversal** + **topo sort** (a grid IS a graph) · 16 **shortest paths** · 17 **union-find** ("connected?" while joining) · 18 **trie** (prefixes) · 19 **DP** (ways/best, overlapping) · 20 **bits and maths**.

Read the constraints BEFORE the problem — the setter names the intended solution there. $n \leq 20 \rightarrow$ exponential/bitmask. $n \leq 1,000 \rightarrow O(n^2)$, a 2-D table. $n \leq 10^5 \rightarrow O(n \log n)$. $n \leq 10^9 \rightarrow$ a formula, never a loop. "**O(1) extra space**" is forbidding the hash map. "**Return all**" $\rightarrow$ backtracking. "**Number of ways**" $\rightarrow$ DP or counting. "**Best value**" $\rightarrow$ DP or greedy. "**A position**" $\rightarrow$ binary search.

Then: what structure does the input have (sorted $\rightarrow$ two pointers/binary search; tree $\rightarrow$ traversal; grid $\rightarrow$ graph; prefixes $\rightarrow$ trie; things joining $\rightarrow$ union-find), **and what is the slow version repeating?** Re-summing $\rightarrow$ prefix sums. Re-scanning ahead $\rightarrow$ monotonic stack. Re-solving $\rightarrow$ DP. Sorting to take five $\rightarrow$ heap. **Almost every pattern is the same move: notice repeated work and stop repeating it.**

Say the PRECONDITION when you name the pattern, because violating one gives a wrong answer, not a crash. **Two pointers needs sorted. Sliding window breaks on NEGATIVES** (shrinking can raise the sum — use prefix sums + a map). **Greedy needs an exchange argument** (`coins=[1,3,4]` , `amount=6` : greedy 4+1+1, optimal 3+3). **Dijkstra needs non-negative weights. Binary search needs monotonicity.**

Hold two or three candidates, never one. "Longest" fires window AND DP — the separating question is *contiguous?*. "Minimum cost" fires Dijkstra AND DP — *weighted graph?*. "Sorted" fires two pointers AND binary search. **Recognition is the whole game: thirty seconds to the shape leaves ten minutes to write and five to test;**

nine minutes of brute force leaves a rushed implementation of the same code. And the perennial bugs: **BFS marks seen on ENQUEUE**, backtracking needs the **undo**, and you append `chosen[:]` , not `chosen` .

Microservices versus monolith, argued both ways

1 What this is, and why they ask it

A monolith is one deployable thing. Microservices are many deployable things, each owning its own data. That is the whole definition, and everything else follows from it.

The question is almost never “which is better”. It is “which is right for this system, this team, and this year” — and the honest answer changes as any of those three change.

What is being tested is whether you can argue both sides. A candidate who says “microservices, because they scale” has recited something. A candidate who says “microservices here, because these two parts have genuinely different scaling profiles and different teams — but I would keep these four together, because splitting them would put a network call in the middle of a transaction” has actually thought about it.

They ask it because it is the highest-stakes architectural decision most teams make, and the most commonly got wrong in both directions. Teams of eight split a working system into forty services and spend two years regretting it. Teams of two hundred keep one repository long past the point where nobody can deploy without coordinating six other people.

And because the interviewer usually asks you to argue the opposite afterwards. “Now convince me you are wrong.” That is not hostility — it is the actual question, and having only one side rehearsed is how it goes badly.

By the end of this lesson you can define both properly, give four real arguments for each, name the point at which splitting becomes right, describe the modular monolith and why it is the usual correct answer, and put numbers on what a split costs in availability, latency, money and human attention.

2 The story

The family had run one restaurant on the main road for thirty-one years, and there was one kitchen at the back of it.

The son came home from Bangalore in 2019 with an idea he had watched work there. **Six stalls instead of one kitchen.** One for biryani. One for chaat. One for the tandoor. One for sweets. One for juices. One for dosas. **Each with its own man in charge, its own money box and its own fire.**

His father said no for two years, and then said yes.

The first year was very good.

The biryani man could change his prices on a Tuesday without asking anybody, and did, twice. **When the dosa grinder broke, the other five kept working** — in the old days a broken grinder had once shut the entire place for a day and a half. **And the tandoor man, who was ambitious, nearly doubled his takings, because for the first time nothing was stopping him.**

The second year was harder, and it was harder in a way nobody had predicted.

A customer who wanted biryani and a lassi now queued twice. Paid twice. And if the lassi came first he stood there holding it while he waited for the rest.

There was an argument every single evening about the cold room, because six people wanted things out of it and it belonged to none of them.

And the thing that actually hurt: nobody could say how much the restaurant had taken that day until all six boxes had been counted and matched up, which ran until about eleven at night **and came out wrong roughly twice a week.**

The father, who had been quiet about the whole business for two years, said one thing at the end of that second year, and his son wrote it down.

“Before, it was one kitchen and one problem. Now it is six kitchens — and one problem between every two of them.”

They kept the stalls. It was clearly right for the biryani and the tandoor, which were each big enough to stand on their own. **And they put the sweets and the juices back together into one counter, because between them they were one man’s work — and two boxes to count.**

3 The idea in plain English

The father’s sentence is the whole lesson. Splitting one thing into six does not give you six problems. It gives you six things plus the relationships between them, and relationships grow much faster than things do.

The definitions, properly

MONOLITH

- one codebase, one build, one deployment
- one database, one transaction boundary
- calls between parts are FUNCTION CALLS
- to change anything, you deploy everything

MICROSERVICES

- many codebases, many builds, many deployments
- EACH SERVICE OWNS ITS OWN DATA
- calls between services are NETWORK CALLS
- each part deploys on its own schedule

The line that actually matters is the second one in each block: who owns the data. Many services sharing one database is not microservices — it is a monolith with extra network calls, and it has the costs of both and the benefits of neither. That arrangement has a name, and it is the most common failure in this whole area: the distributed monolith.

The case FOR splitting

Four arguments, and each one has a condition attached.

Independent deployment. The biryani man changes his prices without asking anybody. In a monolith, every change waits for the whole build, the whole test suite and one shared release. **This is the strongest argument, and it only bites when several teams are contending for the same pipeline.**

Independent scaling. Image processing needs eight times the CPU of everything else. In a monolith you scale the whole thing, so you pay for eight times the memory and eight times the database connections you did not need. **Real, and it only matters when the profiles genuinely differ by a lot.**

Fault isolation. The dosa grinder breaks and five stalls keep working. In a monolith, a memory leak in the reporting code takes down checkout with it. **But this only holds if the calls are optional** — if checkout cannot complete without inventory, splitting them has not isolated anything.

Team autonomy. Conway's law says a system ends up shaped like the organisation that built it. With one team, one deployable is natural. With twelve teams, one deployable means twelve teams negotiating every release, and the architecture is fighting the organisation chart.

The case AGAINST splitting

Four arguments, and these are the ones candidates rarely have ready.

You lose transactions. This is the big one. In a monolith, “reserve stock and take payment” is one database transaction: **both happen or neither does, and the database guarantees it. Across services there is no such guarantee.** You are into sagas, compensating actions, and eventual consistency — **which is not a library you import, it is a permanent increase in the difficulty of every feature that touches two services.**

Availability multiplies downward. From [day 173](#): in series, availabilities multiply. Eight services at 99.9% each gives 99.2% — five and a half hours a month instead of forty-three minutes. **You did not build anything less reliable; you just needed all eight of them to work.**

Every call gets a hundred thousand times slower. A function call is about ten nanoseconds. A network call in the same zone is about a millisecond. And the network call can fail, time out, or succeed twice, none of which a function call can do.

Operational overhead per service, forever. A pipeline, a dashboard, alerts, a runbook, a repository, dependency updates, a security patch cycle, and somebody on call. Multiply by the number of services. That is the counting of six money boxes at eleven at night.

The middle answer, which is usually the right one

A modular monolith: one deployable, with hard internal boundaries.

```
one build, one deployment, one transaction boundary
BUT:
  clear modules with published interfaces
  no module reaching into another's tables
  enforced by tooling, not by good intentions
```

You get most of the benefit of the boundaries — clear ownership, independent reasoning, easy extraction later — without the network, the transactions or the operations. And when a module genuinely needs to be split out, the seam is already there.

This is what several well-known companies do deliberately. Shopify runs a large modular monolith on purpose. Segment famously split into microservices and then moved back, publishing why — the operational load on a small team outweighed everything they had gained.

The order that works: monolith first, modular monolith as it grows, and extract services one at a time when a specific pressure justifies a specific extraction. Not “we are going to be big one day, so let us start with forty services.”

When splitting is actually right

Look for a specific pressure, not a general aspiration.

SPLIT WHEN	NOT WHEN
several teams are blocked on one release train	one team of six, and the release train works fine
one component's scaling profile is genuinely 10x different	"it might need to scale one day"
one part has a much higher reliability requirement	everything has the same requirement
one part changes weekly and another changes yearly	everything changes at about the same rate
a bounded context is already obvious in the domain	you have not found a real boundary and are splitting by technical layer

That last row is the one to say out loud. Split by business capability, never by technical layer. “Orders”, “payments”, “inventory” are services. “The API layer”, “the business logic layer”, “the data layer” are not — three services that must all be deployed together for any change is the distributed monolith again, with three times the operations and none of the independence.

4 The picture

The two shapes, side by side:

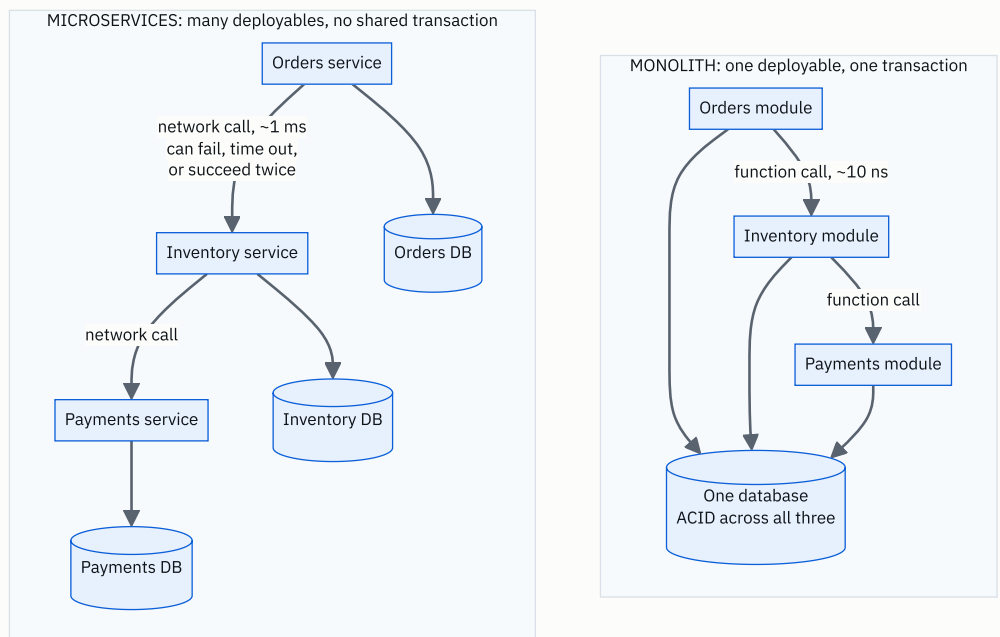


FIGURE 177.2

The important difference is not the number of boxes. It is the arrows and the databases. In the monolith, “reserve stock and take payment” is one transaction the database guarantees. In the split version there is no such guarantee anywhere, and getting it back is the hardest ongoing cost of the decision.

Why six kitchens is not six problems:

THINGS versus RELATIONSHIPS

services	possible pairs = $n(n-1)/2$	
-----	-----	
1	0	
2	1	
3	3	
6	15	← the restaurant
10	45	
20	190	
40	780	

You did not create 40 problems. You created 40 things and up to 780 relationships, each of which is:

- a network call that can fail
- a contract that can drift
- a deployment order that can matter
- an argument about who owns the cold room

In practice not every pair talks - but the number that DO grows much faster than the count of services, and nobody plans for it.

The availability arithmetic, which is the strongest single argument:

every service is a very good 99.9%
a request needs ALL of them

services	availability	downtime per month
-----	-----	-----
1	0.999	43 minutes
2	0.998	86 minutes
4	0.996	2 hours 53 min
8	0.992	5 hours 44 min
16	0.984	11 hours 30 min

NOBODY BUILT ANYTHING WORSE.

Each service is exactly as reliable as before.

You just need all of them at once.

THE FIXES, in order of value:

- make calls OPTIONAL (search down → hide the box)
- cache the last good answer
- make calls ASYNCHRONOUS (a queue, so a slow service delays rather than fails)
- add redundancy IN PARALLEL within each service

The latency arithmetic:

```
in-process function call    ~10 nanoseconds
network call, same zone    ~0.5-1 millisecond
                           = 50,000 to 100,000x slower
```

a request that made 8 internal calls:

```
as a monolith    8 x 10 ns    = 80 nanoseconds
as services      8 x 1 ms     = 8 milliseconds
```

→ 8 ms added to EVERY request, before any work is done.

And that is the happy path. Each call can also:

```
time out, and be retried (doubling the work)
succeed but lose the response (so the caller retries
    something that already happened - hence idempotency)
succeed slowly, holding a connection open
```

5 How it actually works

Getting transactions back, and what it costs

You cannot have ACID across services, so you replace it with a saga: a sequence of local transactions, each with a compensating action if a later step fails.

ORDER SAGA

1. orders: create order (pending)
2. inventory: reserve stock → compensate: release stock
3. payments: charge card → compensate: refund
4. orders: mark order confirmed

If step 3 fails, run step 2's compensation.

WHAT THIS COSTS YOU, and it is not small:

- there is a window where stock is reserved and payment has not happened. Someone can see it.
- compensations can THEMSELVES fail, so they need retries, and the retries need to be idempotent
- a refund is not the inverse of a charge - the money moved, and the customer saw it
- you now need a state machine per business process, and somewhere to run it

Two ways to run one. Choreography — each service emits an event and others react. **Simple to start, and nobody can see the whole flow.** **Orchestration** — one coordinator drives the steps. **Visible and debuggable, and it is a component that can itself fail.** **Temporal**, AWS Step Functions and Camunda exist because this is genuinely hard.

The honest sentence for an interview: “splitting these two services means every feature that touches both becomes a saga, and I would rather keep them together than pay that on every future feature.”

Communication styles

```
SYNCHRONOUS (HTTP/REST, gRPC)
  simple, immediate, and it COUPLES AVAILABILITY -
  if the callee is down, the caller fails
  → use for reads you cannot proceed without

ASYNCHRONOUS (Kafka, SQS, RabbitMQ)
  the caller does not wait; the callee catches up
  → the callee being down becomes a DELAY, not a failure
  → availability stops multiplying
  → the cost is eventual consistency: for a while, two
    services disagree, and the product has to tolerate it
```

Turning a synchronous call into an asynchronous one is the single most effective way to stop availability multiplying, and it is worth naming as a concrete lever rather than a style preference.

Data, which is where it really gets hard

Each service owns its data, so nobody else may read its tables. Which immediately raises: how does the orders service show a customer's name?

```
OPTION 1  call the users service on every request
          → couples availability, adds latency

OPTION 2  keep a local copy of the fields you need,
          updated by events
          → fast and resilient
          → DUPLICATED DATA, eventually consistent,
            and it can drift

OPTION 3  the caller joins two responses
          → pushes the problem outward, often to the
            gateway or the client
```

There is no clean answer, and that is the point. In a monolith this was a `JOIN`. Every one of those three options is worse than a `JOIN`, and you are choosing which kind of worse.

Reporting across services is the same problem at a larger scale, and the usual answer is to stream everything into a warehouse and query it there — which is a whole additional system that the monolith did not need.

The extraction pattern

Nobody rewrites. The pattern is the strangler fig.

1. put a facade in front of the monolith
2. build ONE new service alongside it
3. route just that functionality to the new service
4. when it is proven, delete the old code
5. repeat, one capability at a time - for years

→ the system works throughout
→ each step is individually reversible
→ and you can stop at any point, which matters,
because you usually should stop before "everything"

“We will rewrite it as microservices” is the answer that has destroyed the most companies in this area. “We will extract the payment integration first, because it changes weekly and everything else changes monthly” is a plan.

What you have to build before the first split

This is the part that sinks small teams, and naming it is a strong signal.

centralised logging with a trace id	(day 172)
distributed tracing	(day 172)
metrics and alerting per service	(day 171)
a deployment pipeline per service	(day 174)
service discovery	
a contract-testing story	
a local development story - how does an engineer run this on a laptop?	
an on-call rota that covers all of it	

→ With three services, you can improvise.
With thirty and no platform team, this IS the job,
and feature work stops.

6 The numbers

Availability, in series.

n services, each 99.9%, all required:

$0.999^1 = 0.9990 \rightarrow 43 \text{ minutes/month}$
 $0.999^2 = 0.9980 \rightarrow 86 \text{ minutes}$
 $0.999^4 = 0.9960 \rightarrow 173 \text{ minutes (2h 53m)}$
 $0.999^8 = 0.9920 \rightarrow 344 \text{ minutes (5h 44m)}$
 $0.999^{16} = 0.9841 \rightarrow 690 \text{ minutes (11h 30m)}$

→ Eight services turns a three-nines promise into roughly two-and-a-half nines, with nothing built worse.

To keep 99.9% end to end with 8 required services, each one needs about 99.9875% - which is between three and four nines, and from day 173 each nine is ~10x the cost.

Latency.

in-process call ~10 ns
same-zone network call ~500,000 ns (0.5 ms)
→ 50,000x

a request making 8 internal calls:
monolith: 8 x 10 ns = 0.00008 ms
services: 8 x 0.5 ms = 4 ms

with serialisation, TLS and connection handling,
1 ms per call is more realistic: 8 ms per request

→ On a 50 ms budget that is 16% spent on the architecture before any work happens.
→ And it compounds: if each of the 8 has a p99 of 20 ms, the chance a request avoids ALL of them is $0.99^8 = 92\%$, so 8% of requests hit at least one slow call.

Money, from day 176.

8 internal calls x 100,000,000 requests/day x 20 KB
= 16 TB/day of internal traffic

spread across 3 zones, 2/3 crossing, \$0.01/GB each way:
10,560 GB/day x \$0.02 = \$211/day = \$6,420/month

THE MONOLITH'S EQUIVALENT COST: zero.
Function calls do not appear on a bill.

Human cost, which is the one that actually decides it.

PER SERVICE, PER MONTH, roughly:

dependency and security updates	1 hour
pipeline maintenance	0.5 hour
dashboards and alert tuning	0.5 hour
on-call share, incidents, runbooks	1 hour

~3 hours

40 services x 3 hours = 120 hours/month
= 0.75 of a full-time engineer,
spent entirely on upkeep

With 12 engineers, that is 6% of the entire team's
capacity before anybody writes a feature.

AND THE COGNITIVE COST:

40 services / 12 engineers = 3.3 services each
→ nobody can hold the system in their head
→ "who owns this?" becomes a daily question

Team size, which is the real trigger.

1 team of 6	→ a monolith. Splitting adds pure cost.
3 teams of 6	→ a modular monolith. Boundaries in the code, one deployment.
8 teams of 6	→ the release train is now the bottleneck. Extract along team lines.
50 teams	→ microservices, and a platform team whose whole job is the paved road.

THE RULE OF THUMB: a service per TEAM, not per developer,
and never more services than you have people to own them.

And the counting-the-boxes number.

relationships between n services = $n(n-1)/2$ possible

6 services → 15
20 services → 190
40 services → 780

Even if only 10% of pairs actually talk:

40 services → 78 real integrations
each one: a contract, a failure mode, a version,
a deployment ordering question

7 The trade-offs

This section is the lesson, so here is each side stated as strongly as it deserves.

The case for microservices, at its strongest.

When several teams are contending for one release train, the release train becomes the bottleneck and it does not get better. Every change waits for the slowest test suite and the most nervous reviewer. **Independent deployment is not a performance argument, it is an organisational one, and it is the one that actually matters.**

When one component genuinely needs ten times the resources of the rest, scaling the whole thing is real money. And when one part must be far more reliable than the rest — payments against recommendations — keeping them together means either over-engineering everything or under-protecting the important part.

And fault isolation is real when the calls are optional. A memory leak in the reporting code taking down checkout is a genuine monolith failure, and it happens.

The case for the monolith, at its strongest.

Transactions. A single database guarantees that stock is reserved and payment is taken together, or neither. Splitting them turns that guarantee into a saga with compensating actions, a visible inconsistency window, and refunds that are not really the inverse of charges. Every future feature that touches two services pays that tax, forever.

Availability. Eight services at three nines is two-and-a-half nines end to end, with nothing built worse.

Simplicity, which is not a soft argument. One repository, one deployment, one place to look, and a `JOIN` instead of three bad options for showing a customer's name. An engineer can run the whole thing on a laptop, and that single fact is worth more to a small team's velocity than almost anything else on this page.

And speed of change, early on. Boundaries drawn before you understand the domain are boundaries in the wrong places — and a wrong boundary between two services is enormously more expensive to move than a wrong boundary between two modules.

Which is why the middle option is usually right.

A modular monolith gives you the boundaries without the network. Ownership, independent reasoning, and a seam to extract along later — with one transaction, one deployment and no distributed anything. Shopify does this deliberately. Segment split and moved back, publicly, because the operational load outweighed the gains for the size of team they had.

I would not start with microservices unless the organisation is already large enough that the release train is a real bottleneck on day one. I would not stay with a single undivided monolith past the point where several teams are blocked on each other. And I would extract one service at a time, along team and domain lines, when a specific pressure names a specific extraction.

And the thing I would refuse to do in either direction: split by technical layer. Three services that must all be deployed together is a distributed monolith — every cost of the split, none of the independence.

8 In the interview

How it gets asked

- “Would you build this as microservices?” — and there is always a follow-up.
- “Now argue the opposite.” — the actual question. Having one side rehearsed is how this goes badly.
- “When would you split a monolith?” — they want a trigger, not an aspiration.
- “How do you handle a transaction across two services?” — sagas, and what they cost.
- “Your eight services are each 99.9% available. What is the system’s availability?” — arithmetic.
- “How would you migrate?” — strangler fig, and “not a rewrite”.

The first ninety seconds

On “would you build this as microservices”:

“My default would be a modular monolith, and I would split later along a specific pressure. Let me say why, and then I am happy to argue it the other way.

The definition I am working from is that microservices means many independently deployable services, each owning its own data. The data ownership is the part that matters — many services sharing one database is a distributed monolith, which has the costs of both and the benefits of neither.

The strongest argument for splitting is organisational, not technical. When several teams are contending for one release train, the train becomes the bottleneck and it does not improve on its own. With one team of six, that pressure does not exist, and splitting is pure cost.

The strongest argument against is transactions. In one database, ‘reserve stock and take payment’ is atomic — both or neither, guaranteed. Across services there is no such guarantee anywhere, so it becomes a saga with compensating actions, a window where stock is reserved and payment has not happened, and a refund that is not really the inverse of a charge. Every future feature touching both services pays that tax.

And there is arithmetic I would put on the table unprompted. Availabilities multiply in series. Eight services at 99.9% each gives 99.2% end to end — five and three-quarter hours a month instead of forty-three minutes. Nothing was built worse; you just need all eight at once. And every internal call goes from about ten nanoseconds to about a millisecond, so eight calls add eight milliseconds to every request before any work happens.

So my answer is: modular monolith now — hard internal boundaries, published interfaces, no module touching another’s tables — and extract a service when something specific justifies it. A component whose scaling profile is genuinely ten times different, or a team that is blocked, or a part that changes weekly while everything else changes yearly.“

The follow-ups

“Now argue the opposite. Convince me microservices are right here.”

“Happily, and there are four arguments, each with a condition.

Independent deployment, which is the real one. Right now every change — a one-line copy fix — waits for the whole build, the whole test suite and one shared release. With twelve teams, that means twelve teams negotiating every release, and the coordination cost grows faster than the team does. Conway’s law says the system ends up shaped like the organisation; if the organisation is twelve teams and the architecture is one deployable, the architecture is fighting the organisation and the organisation always wins.

Independent scaling. If image processing needs eight times the CPU of everything else, in a monolith I scale the whole thing — paying for eight times the memory, eight times the database connections and eight times the licence count that nothing needed. Splitting that one component out is often the cheapest change available.

Fault isolation, with the condition stated. A memory leak in the reporting code should not take down checkout, and in a monolith it does. But this only holds if the calls are optional — if checkout cannot complete without inventory, splitting them has isolated nothing and only added a network hop.

And technology freedom, which I would rank last but not at zero. The one component that genuinely belongs in a different language — a machine-learning model, something that needs a specific runtime — can have it, without dragging the whole codebase along.

The honest version of my position is that both answers are right at different sizes, and the mistake is treating it as a permanent identity rather than a decision you revisit. The father in the restaurant kept the stalls that deserved to be stalls and put the sweets and the juices back together, because between them they were one man’s work and two money boxes to count. That is the right shape of answer.”

“How do you handle a transaction that spans two services?”

“You do not — you replace it with a saga, and I want to be honest about what that costs rather than making it sound like a pattern you just apply.

A saga is a sequence of local transactions, each with a compensating action. Create the order, reserve the stock, charge the card, confirm the order — and if the charge fails, run the compensation for the reservation and release the stock.

Four costs, and I would name all four.

There is a visible window where stock is reserved and payment has not happened. Somebody can observe that state, so the product has to have an answer for what it means.

Compensations can themselves fail, so they need retries, and retries mean every step must be idempotent — which is real work on every operation, not a library.

Compensations are not true inverses. A refund is not the opposite of a charge: the money moved, the customer saw it, and there may be a fee. Some things cannot be compensated at all — an email that has been sent.

And you now need a state machine per business process and somewhere to run it. Choreography — each service emits events and others react — is easy to start and nobody can see the whole flow. Orchestration — one coordinator drives it — is visible and debuggable, and is itself a component that can fail. Temporal and Step Functions exist precisely because this is hard.

Which is why my design rule is the other way round. I look at which operations must be atomic, and I keep those inside one service. Orders and order-items belong together. Orders and recommendations do not. If a proposed boundary would put a network call in the middle of something that has to be atomic, that is the strongest possible argument that the boundary is in the wrong place.”

“How would you migrate an existing monolith, and how do you know when to stop?”

“Strangler fig, one capability at a time, and the stopping question is the more interesting half.

The pattern: put a facade in front of the monolith, build one new service alongside it, route just that functionality across, prove it, then delete the old code. The system works throughout, and every step is individually reversible. What I would not do is a rewrite — ‘we will rebuild it as microservices’ is the plan that has destroyed the most projects in this area, because it means a long period where you are maintaining two systems and shipping nothing.

I would choose the first extraction by pressure, not by architecture diagram. The part that changes weekly while everything else changes yearly. The part with a genuinely different scaling profile. The part a separate team already owns in practice. And I would prefer something at the edge with few dependencies, because the first extraction is where you build the platform.

Which is the thing I would flag before starting: there is a fixed cost that arrives with service number two. Centralised logging with a trace id, distributed tracing, per-service metrics and alerts, a pipeline per service, service discovery, contract testing, and a story for how an engineer runs this on a laptop. With three services you can improvise; with thirty and no platform team, that becomes the job and feature work stops.

And when to stop, which almost nobody asks. I would stop when the next extraction has no specific justification. Concretely: about three hours per service per month of upkeep — dependency updates, pipeline, dashboards, on-call share. Forty services is a hundred and twenty hours a month, three-quarters of an engineer, before anybody writes a feature. With twelve engineers that is six percent of capacity spent on existing.

The rule I would actually apply is a service per team, not per developer — and never more services than there are people to own them. Segment split and then publicly moved back for exactly this reason. Putting two small services back together is a perfectly respectable outcome, and treating the split as irreversible is how teams end up with forty things and nobody who understands any twenty of them.“

The model answer

“Would you build this as microservices? Now argue the opposite.”

“I will give you my answer, the strongest version of the other side, and then the thing that actually decides it.

My answer is a modular monolith now, extracting services later along specific pressures. One deployable, one transaction boundary, but hard internal module boundaries with published interfaces and no module reaching into another’s tables — enforced by tooling rather than good intentions. That gives me ownership and clear reasoning and a seam to extract along, without the network, the sagas or the operations.

The argument for splitting, at full strength. It is organisational before it is technical. When several teams contend for one release train, the train is the bottleneck and it does not improve. Conway’s law says the system takes the shape of the organisation, and if that fight is on, the organisation wins. Add genuinely different scaling profiles — one component needing eight times the CPU — and fault isolation where a leak in reporting currently takes down checkout, and that is a real case, not a fashionable one.

The argument against, at full strength. You lose transactions, and that is **permanent**. ‘Reserve stock and take payment’ stops being atomic and becomes a saga with compensating actions, an observable inconsistency window, refunds that are not inverses of charges, and idempotency required on every operation. Every future feature that touches both services pays it. Availability multiplies downward: eight services at 99.9% is 99.2% end to end, five and three-quarter hours a month against forty-three minutes, with nothing built worse. Every call goes from ten nanoseconds to a millisecond, so eight calls add eight milliseconds before any work happens — and about six and a half thousand dollars a month of cross-zone traffic that function calls never generated. And roughly three hours per service per month of upkeep, which at forty services is three-quarters of an engineer doing nothing but maintenance.

What actually decides it is not the architecture. It is the number of teams. One team of six: a monolith, and splitting is pure cost. Three teams: a modular monolith. Eight teams blocked on one release train: extract along team lines. Fifty teams: microservices, and a platform team whose whole job is making that survivable.

Two things I would refuse in either direction. I would not split by technical layer — an API layer, a logic layer and a data layer as three services must all be deployed together, which is every cost of the split and none of the

independence. And I would not treat the decision as permanent. Segment split and publicly moved back because the operational load outweighed the gains at their size. Putting two services back together is a respectable outcome.

If I had to reduce it to one sentence, I would use the restaurant's. Splitting one thing into six does not give you six problems — it gives you six things and a problem between every two of them, and the number of relationships grows much faster than the number of services. So each split has to be paid for by something specific, and 'we might be big one day' does not pay for it."

9 Recall card

A **MONOLITH** is one deployable with one transaction boundary; **MICROSERVICES** are many deployables, each **OWNING ITS OWN DATA**. The data ownership is the definition — many services sharing one database is a **distributed monolith**: every cost of the split, none of the independence. **Never split by technical layer; split by business capability.**

FOR (each with a condition): independent DEPLOYMENT (the real one — a release train several teams contend for), **independent SCALING** (only when profiles differ $\sim 10\times$), **FAULT ISOLATION** (only if the calls are optional), **TEAM AUTONOMY** (Conway's law: the architecture fights the org chart and the org chart wins). **AGAINST: you LOSE TRANSACTIONS** (sagas, compensations, an observable inconsistency window, refunds that are not inverses, idempotency everywhere — paid on every future feature); **availability MULTIPLIES DOWNWARD**; **every call is $\sim 100,000\times$ slower and can now fail, time out or succeed twice; ~ 3 hours/service/month of upkeep, forever.**

The arithmetic to have ready. 8 services at $99.9\% = 0.999^8 = 99.2\% \rightarrow 5\text{h } 44\text{m/month}$ instead of 43 minutes, with nothing built worse. In-process call ~ 10 ns against a network call ~ 1 ms $\rightarrow 8$ calls add 8 ms to every request. 16 TB/day of internal traffic $\approx \$6,420/\text{month cross-zone}$, where function calls cost nothing. **40 services $\times 3$ h = 120 h/month = 0.75 of an engineer on upkeep alone.** And relationships are $n(n-1)/2$ — 6 services is 15 pairs, 40 is 780.

The usual right answer is the **MODULAR MONOLITH**: one deployable and one transaction, with hard internal boundaries enforced by tooling. Shopify does it deliberately; **Segment split and publicly moved back. Migrate with the STRANGLER FIG** — a facade, then one capability at a time, each step reversible — never a

rewrite. And there is a **fixed platform cost that arrives with service number two:** tracing, per-service metrics and pipelines, service discovery, contract tests, and a laptop story.

Team count decides it, not taste. 1 team → monolith. 3 teams → modular monolith. 8 teams blocked on one release train → extract along team lines. 50 teams → microservices plus a platform team. **A service per TEAM, never per developer, and never more services than there are people to own them. Splitting one thing into six gives you six things and a problem between every two of them** — so every split must be paid for by a specific pressure, and “we might be big one day” does not pay for it.

PRACTICE

Practice — The pattern index, and monolith versus microservices

DSA topic: The twenty patterns, on one page **System design topic:** Microservices versus monolith, argued both ways

Code these, in this order

Today the drill is recognition, so the rule changes. For every problem below: **read the constraints, say what they forbid, name the pattern and its precondition, and state the cost — all before writing a line. Give yourself ninety seconds for that part and mean it.** The code is the easy half and you have written all four shapes before.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Sliding window, and the “seen before the window started” line.
2	Koko Eating Bananas	LeetCode 875 (Medium)	Binary search on the answer — the pattern people fail to recognise.
3	Daily Temperatures	LeetCode 739 (Medium)	Monotonic stack, and that it holds positions rather than values.
4	Course Schedule	LeetCode 207 (Medium)	That “prerequisites” means a graph, and cycles mean impossible.
5	Subarray Sum Equals K	LeetCode 560 (Medium)	Why this is prefix sums and NOT a sliding window.
6	Coin Change	LeetCode 322 (Medium)	Naming the state before writing the loop, and why greedy fails.

On problem 2, notice what you are searching

Nothing here is sorted and nothing is an array lookup. **Say out loud what the search space actually is** — the possible answers, not the input — **and say why feasibility is monotone.** That sentence is the entire recognition, and it is the pattern candidates miss most often.

On problem 5, compare it with problem 1

Both mention subarrays. **Problem 1 is a window and problem 5 is not.** Work out for yourself which property problem 5 lacks. **Then construct an input with a negative number where the window approach gives the wrong answer,** and record it.

On problem 6, break greedy on purpose

Run greedy — largest coin first — on `coins = [1, 3, 4]`, `amount = 6`. **Record what it gives and what the answer is.** Then say the state sentence for the DP version out loud before you write the loop.

On all six, score the ninety seconds

```
For each problem:
[ ] I read the constraints first
[ ] I said what they forbid
[ ] I named the pattern before writing
[ ] I named its precondition
[ ] I stated time and space before writing
[ ] The pattern was right

6 problems x 6 = 36 ticks.
30+ the recognition is there.
20-29 you know the code and are still finding the shape
    by writing. Do the classifier drill below.
<20 re-read section 3 until the tell arrives before you
    finish reading the problem.
```

Then the classifier drill

Take twenty problems you have already solved. **Read only the first line of each, and say the pattern out loud.** Time yourself. **Anything over five seconds goes on a list,** and that list is what you revise.

Then the confusable-pairs drill

For each pair, give the one question that separates them:

1. Sliding window against prefix sums.
2. Greedy against dynamic programming.
3. Heap against sorting.
4. BFS/DFS against union-find.
5. Two pointers against binary search.
6. Backtracking against dynamic programming.

Then the precondition drill

Name the precondition for two pointers, sliding window, greedy, Dijkstra and binary search. **For each, give the input that breaks it** and say whether the failure is a crash or a wrong answer.

Then the constraint drill

Cover the right-hand column of the constraint table. **Read each constraint and name the intended technique.** Then do it in reverse: given a technique, say the constraint that would have hinted at it.

Then the both-sides drill

Pick any system you know. **Argue for splitting it into services for two minutes.** Then argue against, for two minutes. **Use different arguments, not the same ones inverted.** Then say which you would actually do and what would change your mind.

Then the arithmetic drill

Compute end-to-end availability for four, eight and sixteen services at 99.9 percent each. **Convert each to minutes a month.** Then compute the latency added by eight network calls, and the monthly cross-zone cost from sixteen terabytes a day.

Then the boundary drill

Take a system with orders, payments, inventory, recommendations, search and notifications. **Say which of those must be atomic together** and therefore must not be split apart. Then say which are safe to split first, and why.

The index drill

1. Name all twenty patterns, in any order.
2. For any five, give the tell in one sentence.
3. For any five, give the cost.
4. Name the pattern for: “next greater element”, “minimise the maximum”, “return all subsets”, “are these two connected”.

The procedure drill

1. Give the four questions, in order.
2. Say which is highest value and why.
3. Give four constraints and what each one names.
4. Give four output shapes and what each one names.

5. Say what to do when two patterns both fire.

The template drill

Write from memory, without looking:

1. The two-pointer loop.
2. The sliding-window skeleton.
3. Binary search on the answer, including the bounds.
4. The monotonic stack loop.
5. BFS with a distance.
6. The backtracking skeleton, including the undo.
7. A one-dimensional DP loop, with the state said aloud first.

The definitions drill

1. Define monolith and microservices, and say which line is the real definition.
2. Say what a distributed monolith is and why it is the worst option.
3. Say what a modular monolith is and what it buys.
4. Say what “split by business capability, not technical layer” rules out.

The both-sides drill

1. Four arguments for splitting, each with its condition.
2. Four arguments against, with what each costs.
3. Say which argument for is strongest, and why it is organisational.
4. Say which argument against is strongest, and why it is permanent.

The saga drill

1. Give a four-step saga with its compensations.
2. Name four things sagas cost you.
3. Say why a refund is not the inverse of a charge.
4. Give the difference between choreography and orchestration.
5. Give the design rule this implies about where boundaries go.

The arithmetic drill

1. Availability for 1, 4, 8 and 16 services at three nines, in minutes.
2. What each service would need for the whole to stay at 99.9 percent.
3. Latency added by eight network calls, against eight function calls.

4. Monthly cross-zone cost from stated internal traffic.
5. Upkeep hours for forty services, as a fraction of a twelve-person team.
6. Relationships between six, twenty and forty services.

The migration drill

1. Describe the strangler fig in five steps.
2. Say why a rewrite fails.
3. Say how you choose the first extraction.
4. Name the platform pieces that must exist before service number two.
5. Say when to stop extracting, with a number.

The break-it drill

For each, say what happens and whether anything reports it:

1. Two pointers on unsorted input.
2. A sliding window on an array containing negatives.
3. Greedy on `coins = [1, 3, 4]`, `amount = 6`.
4. Dijkstra with a negative edge.
5. BFS marking nodes as seen on dequeue.
6. Backtracking without the undo.
7. `out.append(chosen)` instead of `out.append(chosen[:])`.
8. Recursion over a ten-thousand-node linked list.
9. Eight services sharing one database.
10. Splitting orders and payments apart.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Given this problem, which pattern, and why that one?* Constraints first and what they forbid; the output shape; the input structure; what the slow version repeats; two candidates rather than one; and the precondition stated with the pattern.
2. *What did you actually learn from solving a few hundred problems?* Twenty shapes not twenty algorithms; recognition is the skill that fails, not implementation; constraints name the intended solution; and almost every pattern is “stop repeating work”.

3. *Would you build this as microservices? Now argue the opposite.* Both sides at full strength, the availability and latency arithmetic, the transaction cost, the modular monolith as the usual answer, and team count as the thing that actually decides.

Before you move on

- ☐ I can name all twenty patterns.
- ☐ For any pattern, I can give the tell in one sentence.
- ☐ I read the constraints before the problem.
- ☐ I can turn six constraints into six intended techniques.
- ☐ I know “return all” means backtracking and “number of ways” means DP.
- ☐ I know a grid is a graph.
- ☐ I hold two or three candidate patterns, not one.
- ☐ I can separate window from prefix sums with one question.
- ☐ I can separate greedy from DP with one question.
- ☐ I can separate heap from sorting, and traversal from union-find.
- ☐ I state the precondition when I name the pattern.
- ☐ I know which preconditions fail silently rather than crashing.
- ☐ I can write all seven templates from memory.
- ☐ I know BFS marks seen on enqueue.
- ☐ I know backtracking needs the undo and the copy.
- ☐ I can give the complexity ladder and read it backwards from a constraint.
- ☐ I can define monolith and microservices, and say which line is the real definition.
- ☐ I know what a distributed monolith is and why it is the worst option.
- ☐ I can give four arguments for splitting, each with its condition.
- ☐ I can give four arguments against, with what each costs.
- ☐ I know why the deployment argument is organisational.
- ☐ I know why the transaction argument is permanent.
- ☐ I can compute availability for eight services and convert it to minutes.
- ☐ I can compute the latency and the cross-zone cost of a split.
- ☐ I know the per-service upkeep cost in hours.
- ☐ I can describe a saga and name four things it costs.

- [] I know the design rule that follows from sagas being expensive.
- [] I can describe the modular monolith and name two companies that use one.
- [] I can describe the strangler fig and say why a rewrite fails.
- [] I know team count decides this, not taste.
- [] I know putting two services back together is a respectable outcome.
- [] I answered all three questions above out loud.

How to think out loud in a coding round

DATA STRUCTURES AND ALGORITHMS

How to think out loud in a coding round

SYSTEM DESIGN

The system design interview framework, memorised

How to think out loud in a coding round

1 What this is, and why they ask it

A coding interview is graded on what you say, not on what you were thinking. The interviewer cannot see inside your head. **If you solve it silently and correctly, you get a lower score than someone who solves it out loud and correctly** — and often a lower score than someone who talks well and does not quite finish.

That is not unfair, and it is worth understanding why. They are not testing whether you can produce working code alone in a room. **They are testing what it would be like to work through a hard problem with you in the next chair,** because that is the job. **Silence is the answer to a different question.**

This lesson is a script. The first five minutes, said in a fixed order, every time. **Having a script is what stops the freeze,** because when a hard problem lands and your head goes blank, **you do not need an idea — you need the next sentence,** and the script always has one.

They ask it because **it is the difference between two candidates with identical knowledge. One spends nine minutes silently trying things and then writes hurriedly. The other spends ninety seconds naming the shape, three minutes agreeing an approach, and then writes calmly with time left to test. Same person, same ability, different outcome** — and the second one also gets helped when they go wrong, because the interviewer could hear where they were.

By the end of this lesson you have the opening script word for word, the four clarifying questions that apply to almost every problem, a time budget for a forty-five minute round, three specific moves for when you are stuck, a phrasebook of what to say and what never to say, and a full worked example with the spoken words alongside the code.

2 The story

There was a table outside the taluk office, under a blue tarpaulin tied to the railing, and a man sat at it called Shivanna, and people paid him ten rupees to fill in their forms.

He had been there nineteen years. The forms themselves had changed four times.

What people noticed, if they queued long enough to watch, was that **he did not start writing when somebody sat down**. Sometimes he did not start for two or three minutes. **And there was always a queue, and the man in the chair would shift about and look at it.**

He asked questions instead. **Always the same ones, always in the same order.**

Whose name goes on it. Which office it is going to. What date it is needed by. Whether they had the previous receipt. **And then the last one, which annoyed people more than the other four put together: “Now tell me, in your own words, what you are actually trying to get.”**

His nephew, who helped in the afternoons during the holidays, asked him one day why he wasted the time. **The form was right there. He could simply start.**

Shivanna said something the boy thought was strange.

“The form is easy. Anybody can fill in a form.”

“Then why all the questions?”

“Because a form filled in wrongly comes back in six weeks. By then the date has passed and the man has to start again.” He put the pen down. **“And in nineteen years I have never once seen a form come back because of bad handwriting. They come back because we answered a different question from the one he was asking.”**

The nephew said that the man in the chair does not like waiting.

“He does not like waiting three minutes. He likes it a great deal better than waiting six weeks.”

And then the last thing, which the nephew only properly understood a season later, after he had done one badly wrong himself.

“Also — I say all five questions out loud, even when I already know the answers. Because if I have got it wrong in my head, that is when he stops me.”

“Then. Not afterwards.”

3 The idea in plain English

Shivanna's five questions are the opening script, and his last sentence is the whole reason for it. You say your understanding out loud so that the interviewer can correct you in the first two minutes rather than the last two.

The arc of a forty-five minute round

0-2 RESTATE the problem in your own words
2-5 CLARIFY: the four questions, plus anything specific
5-7 EXAMPLES: work one small one by hand, out loud
7-10 APPROACH: brute force, its cost, then the better idea
 and its cost. AGREE it before writing.
10-30 CODE, narrating decisions
30-38 TEST by hand: the example, then edge cases
38-45 COMPLEXITY, and the follow-up question

→ A third of the round happens before you write anything.
That feels wrong and it is correct.

The most common failure is spending minutes 0 to 9 writing code that solves the wrong problem. The second most common is spending them silently.

Restating, which takes twenty seconds and saves ten minutes

“So — I am given a list of integers, in no particular order, and I need to return the length of the longest run of consecutive whole numbers that appears in it. They do not have to be next to each other in the list. Have I got that right?”

Two things happen here. You find out immediately if you have misread it. And the interviewer learns that you read carefully, which is a thing they are marking.

The four questions that always apply

Ask these on every problem, even when you can guess the answers. Shivanna asks all five out loud even when he knows.

1. HOW BIG? "What is the range of n? And the values?"
 → this decides the algorithm, and they
 usually answer with the intended one
2. WHAT WEIRD INPUTS?
 "Can it be empty? Can there be duplicates?"
 "Negative numbers? Is it already sorted?"
3. WHAT DO I RETURN WHEN THERE IS NO ANSWER?
 "Empty list, or -1, or an exception?"
4. CAN I MODIFY THE INPUT?
 "May I sort it in place, or does the caller
 still need it?"

Then one or two specific to the problem. For a string problem: character set, case sensitivity. For a graph: directed or not, can there be cycles, is it connected.

And listen to the answers, because they are hints. “n can be up to a hundred thousand” has just told you $O(n^2)$ is out. “Assume it fits in memory” has just told you it is not a streaming problem.

Working an example by hand

Take the smallest interesting input and say what the answer is and why.

“So with [100, 4, 200, 1, 3, 2], the runs are: 100 on its own, 200 on its own, and 1, 2, 3, 4. So the answer is 4. Let me also check what happens with an empty list — I would return 0.”

This is where you notice things. Half the time you find an edge case the problem statement did not mention, and finding it here is worth far more than finding it in minute forty.

Stating the approach before writing it

Always say the brute force first, with its cost, even when you already know the good answer.

“The obvious approach is: for each number, keep walking upwards while the next one exists, and track the longest. With a set for the lookups, each step is constant. But in the worst case — a single long run — I walk the whole run from every starting point, so that is $O(n^2)$.”

“The fix is one line. I only start walking from a number that has no predecessor in the set. Every run then gets walked exactly once, so the total is $O(n)$.”

“ $O(n)$ time and $O(n)$ space for the set. Shall I write that?”

Three things that paragraph does. It shows you can find a working answer — a candidate who leaps to a clever solution and gets it wrong has shown nothing. It shows you can improve one. And “shall I write that?” gives the interviewer a place to redirect you before you spend twenty minutes.

If they say “can you do better?”, they mean it. If they say “sounds good”, start typing.

Narrating while you code

Narrate decisions, not keystrokes.

NOT THIS	"Now I'm writing a for loop... i equals zero... opening a bracket..."
THIS	"I'm using a set rather than a list so the lookups are constant time." "I'll handle the empty case first so the main loop doesn't have to." "I'm iterating over the set rather than the original list, which also removes duplicates for free."

Every sentence should be a reason. And silence for more than about twenty seconds should be filled — even with “give me a second, I am working out whether this needs to be strictly less than or less than or equal.” That sentence is genuinely useful to the interviewer; a blank stare is not.

The three moves when you are stuck

Being stuck is normal and is not the failure. Being stuck and silent is the failure. Say that you are stuck, then use one of these.

Shrink the problem. *“Let me solve it for the case where all the numbers are distinct and positive, and then add the rest back.”*

Solve a special case by hand. *“Let me work through a four-element example completely and see what I actually do.”* This finds the algorithm surprisingly often, because you already know how to do it — you just have not watched yourself doing it.

Say what you know and what is blocking. *“I know I need constant-time lookups, so a set. And I know the brute force repeats work. What I have not found is how to avoid walking the same run twice.”*

That third one is the most valuable thing you can say when stuck, because it is precisely the sentence that lets the interviewer give you a hint. **They want to.** A hint costs you far less than five silent minutes.

Taking a hint

When a hint arrives, take it. Do not defend the previous idea.

“Ah — so if I only start from numbers that have no predecessor, each run is walked once. Yes, that makes it linear. Let me change that.”

Repeat the hint back in your own words before acting on it. It confirms you understood it, and it is the difference between “took the hint” and “was told the answer”, which are marked very differently.

Testing out loud

Do not say “I think that is right.” Run it, with your mouth.

1. the example from the problem, traced through the code
2. the empty input
3. one element
4. all elements the same
5. the case you were worried about while writing

And say the values as you go. *“present is the set of all six. First value 100 — is 99 in the set? No, so this is a start. Walk up: is 101 in? No. Length 1. Best is 1.”*

Finding your own bug during this is a strong positive, not a negative. **It is the single most reliable way to convert “wrote code” into “wrote correct code” in the interviewer’s notes.**

What is actually being graded

Most companies score four axes separately, and knowing them changes what you do with your time.

PROBLEM SOLVING	did you get to a good approach, and how?
CODING	is it clean, correct, and would it compile?
COMMUNICATION	could I follow you? did you take hints?
TESTING	did you verify it, or did you hope?

Two of those four are things you can be excellent at even on a problem you do not finish. Which is why “almost finished, communicated superbly, found their own bug” beats “finished, said nothing” far more often than candidates expect.

4 The picture

The round as a timeline, with what the interviewer is writing:

MINUTES	YOU	THEY ARE NOTING
0 - 2	restate in your own words	"read it carefully"
2 - 5	the four questions	"thinks about edge cases before, not after"
5 - 7	work a small example aloud	"checks understanding against a concrete case"
7 - 10	brute force + cost, then better + cost, then "shall I write that?"	"can find A solution, then improve it. Not guessing." "gave me a chance to redirect"
10 - 30	code, narrating DECISIONS	"I can follow the reasoning" "clean, named things well"
30 - 38	trace the example by hand, then edge cases	"tests without being asked" "found their own bug"
38 - 45	state complexity, answer the follow-up	"knows what it costs" "has depth beyond the first answer"

NOTE WHERE THE FIRST LINE OF CODE IS.
Minute ten. Nearly a quarter of the round has gone, and that is the CORRECT allocation.

The phrasebook — what to say, and what it replaces:

INSTEAD OF

(silence)

"This is easy."

"I've seen this one."

"It should work."

"I don't know."

"No, because..."
(defending a hint)

"Is this right?"

"I'd need to look it up."

SAY

"Give me a second, I'm working out whether this bound is inclusive."

"I think this is the sliding window shape. Let me check that against an example."

"I've seen something like this. Let me restate it to be sure it's the same problem."

"Let me trace the example. present is {100, 4, 200, 1, 3, 2}. First value 100 - is 99 in the set? No..."

"I'm stuck on how to avoid walking the same run twice. I know I need constant-time lookups, so a set. What I don't have yet is..."

"That's a good point - let me think about it. So if I only start from numbers with no predecessor..."

"I believe this is correct. Let me verify with the example, and then the empty case."

"I'd use a heap here. The API is heappush and heappop; let me write it and you can correct the exact names."

The stuck procedure, as a flow:

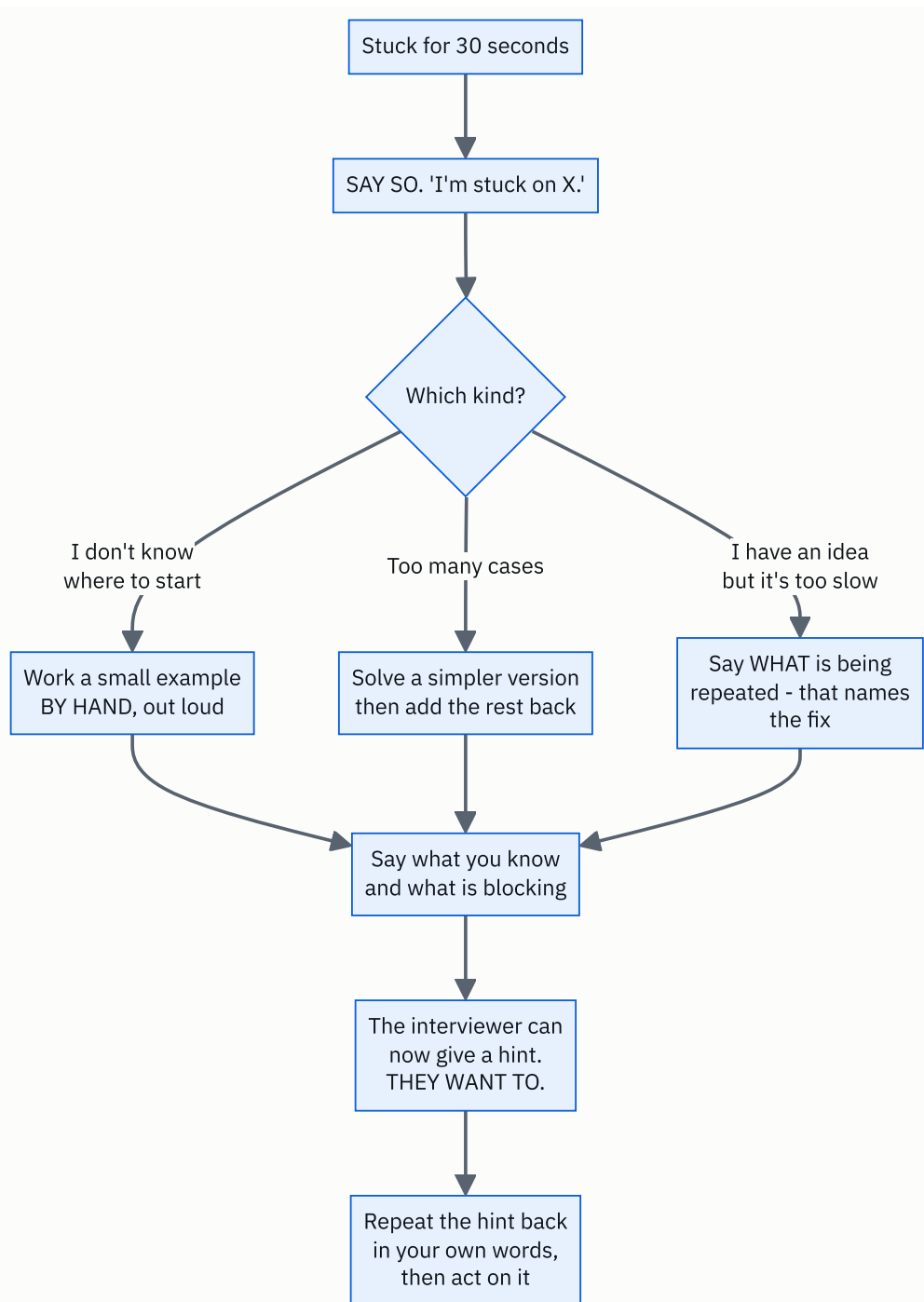


FIGURE 178.1

The box that matters is the first one. Thirty seconds of silence is the limit — after that, saying “I am stuck on X” is strictly better than continuing to look thoughtful, because it is the only thing that lets anybody help you.

5 The code, built step by step

Here is a complete worked example, with the spoken words next to each piece. The problem: *given an unsorted list of integers, return the length of the longest sequence of consecutive integers. It must run in $O(n)$.*

Minute 0-2: restate

“So I have a list of integers in no particular order, possibly with duplicates, and I need the length of the longest run of consecutive whole numbers that appears anywhere in it — they do not have to be adjacent in the list. For [100, 4, 200, 1, 3, 2] the answer would be 4, from 1, 2, 3, 4. Is that right?”

Minute 2-5: the four questions

“A few things before I start. How large can the list be? ... A hundred thousand, right, so $O(n^2)$ is about ten billion operations and is out.

Can the list be empty? ... Then I return 0.

Can there be duplicates? ... Yes — so I should be careful that a repeated number does not inflate a run.

Negative numbers? ... Yes, fine, that does not change anything.

And am I allowed to modify the input, or use extra space? ... The problem says $O(n)$ time, which rules out sorting, so I will need extra space anyway.”

Notice that “it must be $O(n)$ ” has already eliminated the sort-and-scan answer, and saying so out loud shows you understood why the constraint was there.

Minute 5-7: a small example, by hand

“Let me take [100, 4, 200, 1, 3, 2]. Putting them in a set: 100, 4, 200, 1, 3, 2. The runs I can see are 100 on its own, 200 on its own, and 1, 2, 3, 4 together. So the longest is 4.”

Minute 7-10: the brute force, then the fix

“The straightforward approach: put everything in a set for constant-time lookups. Then for each value, walk upwards — value plus one, value plus two — while the next one is present, and keep the longest.”

```
def longest_consecutive_brute(numbers: list[int]) → tuple[int, int]:
    """The honest first answer. Returns (answer, steps taken) so we can count the cost."""
    present = set(numbers)
    steps = 0
    best = 0
    for value in present:
        length = 1
        current = value
        while current + 1 in present:
            current += 1
            length += 1
            steps += 1
        best = max(best, length)
    return best, steps
```

“That is correct, but let me look at its cost, because I do not think it is linear. If the input is one long run — nought to n minus one — then starting from 0 I walk n steps, from 1 I walk n minus one, and so on. That is $n(n-1)/2$, so it is $O(n^2)$ in the worst case. For a thousand elements that is half a million inner steps rather than a thousand.”

“And the reason is that I walk the same run over and over from every point inside it.”

“So the fix is: only start walking from a number that is the beginning of a run — one whose predecessor is not in the set. Then each run is walked exactly once, and the total work across all runs is at most n .”

“That is $O(n)$ time and $O(n)$ space. Shall I write it?”

That is the whole interview, and it happened before any real code was written.

Minute 10-30: the code, with the decisions said

```
def longest_consecutive(numbers: list[int]) → tuple[int, int]:
    """Only start walking from a number with no predecessor. One line, and it is  $O(n)$ ."""
    present = set(numbers)
    steps = 0
    best = 0
    for value in present:
        if value - 1 in present:
            continue
        # not the start of a run - skip it
        current = value
        while current + 1 in present:
            current += 1
            steps += 1
        best = max(best, length)
```

“A set, not a list, because I need constant-time membership. Iterating over the set rather than the original list also deduplicates for free, which handles the duplicates question from earlier.”

“And this `continue` is the entire optimisation — if `value - 1` is present, this is the middle of a run and somebody else will walk it.”

```
length = 1
current = value
while current + 1 in present:
    current += 1
    length += 1
    steps += 1
best = max(best, length)
return best, steps
```

“Starting `length` at 1 because the value itself counts. And `best` starts at 0 so that the empty input returns 0 without a special case — let me double-check that: if the set is empty, the loop body never runs and we return 0. Good.”

Saying “let me double-check that” and then checking it is worth more than getting it right silently.

Minute 30-38: testing, out loud

“Let me trace the example. The set is {1, 2, 3, 4, 100, 200}.

Take 1: is 0 in the set? No, so this is a start. Walk: 2 present, 3 present, 4 present, 5 not. Length 4. Best is 4.

Take 2: is 1 in the set? Yes — skip. Same for 3 and 4.

Take 100: is 99 in? No. Walk: 101 not present. Length 1.

Take 200: same, length 1.

Answer 4. Correct.

Now the edge cases. Empty list: the set is empty, the loop never runs, returns 0. Single element: no predecessor, walk finds nothing, length 1. All duplicates, `[1, 1, 1, 1]`: the set is just {1}, so length 1 — good, that was the duplicates worry. Negatives, `[-3, -2, -1, 5]`: -3 has no predecessor, walks to -1, length 3. Correct.”

Minute 38-45: complexity, said properly

“Time is $O(n)$. That deserves a sentence, because there is a nested loop and it looks quadratic. The inner `while` only runs for values that begin a run, and across all runs it visits each element at most once — so the total inner work is bounded by n , not multiplied by it.

Space is $O(n)$ for the set.

If I were not allowed extra space, I would sort and scan, which is $O(n \log n)$ time and $O(1)$ extra — and the problem’s $O(n)$ requirement is exactly what rules that out.”

The complete solution

```
"""Day 178 - one problem, solved the way it should be solved out loud."""

from __future__ import annotations

def longest_consecutive_brute(numbers: list[int]) → tuple[int, int]:
    """The honest first answer. Returns (answer, steps taken) so we can count the cost."""
    present = set(numbers)
    steps = 0
    best = 0
    for value in present:
        length = 1
        current = value
        while current + 1 in present:
            current += 1
            length += 1
            steps += 1
        best = max(best, length)
    return best, steps

def longest_consecutive(numbers: list[int]) → tuple[int, int]:
    """Only start walking from a number with no predecessor. One line, and it is O(n)."""
    present = set(numbers)
    steps = 0
    best = 0
    for value in present:
        if value - 1 in present:
            continue # not the start of a run - skip it
        length = 1
        current = value
        while current + 1 in present:
            current += 1
            length += 1
            steps += 1
        best = max(best, length)
    return best, steps

def longest_consecutive_clean(numbers: list[int]) → int:
    """The version you would actually submit, without the step counter."""
    present = set(numbers)
    best = 0
    for value in present:
        if value - 1 in present:
            continue
        length = 1
        while value + length in present:
            length += 1
        best = max(best, length)
    return best

if __name__ == "__main__":
    print("THE EXAMPLE FROM THE PROBLEM")
    data = [100, 4, 200, 1, 3, 2]
    print(f" {data}")
    print(" runs: 100 | 200 | 1,2,3,4 → answer 4")
    print(f" brute force: {longest_consecutive_brute(data)} (answer, inner steps)")
    print(f" with the guard: {longest_consecutive(data)}")

    print()
    print("THE EDGE CASES I WOULD TEST OUT LOUD")
    for case, expected in (
```

```

([], 0),
([7], 1),
([1, 1, 1, 1], 1),
([1, 2, 0, 1], 3),
([-3, -2, -1, 5], 3),
([9, 1, 4, 7, 3, -1, 0, 5, 8, -1, 6], 7),
):
    got = longest_consecutive_clean(case)
    print(f" {str(case):<36} → {got:<3} expected {expected}"
          f" {'ok' if got == expected else 'WRONG'}")

print()
print("WHY THE ONE-LINE GUARD IS THE WHOLE ANSWER")
print(" a single run of consecutive numbers, 0..n-1:")
print(" n      brute steps      guarded steps")
for n in (10, 100, 1000, 5000):
    run = list(range(n))
    _, brute_steps = longest_consecutive_brute(run)
    _, good_steps = longest_consecutive(run)
    print(f" {n:>5} {brute_steps:>12} {good_steps:>14}")
print()
print(" brute force walks the whole run from EVERY starting point")
print(" → n(n-1)/2 steps. The guard walks it once → n-1 steps.")

print()
print("VERIFICATION")
import random

bad = 0
for _ in range(3000):
    size = random.randint(0, 40)
    sample = [random.randint(-20, 20) for _ in range(size)]

    present = set(sample)
    expected = 0
    for value in present:
        if value - 1 not in present:
            length = 1
            while value + length in present:
                length += 1
            expected = max(expected, length)

    if longest_consecutive_clean(sample) != expected:
        bad += 1
    if longest_consecutive(sample)[0] != expected:
        bad += 1
    if longest_consecutive_brute(sample)[0] != expected:
        bad += 1
print(f" {bad} mismatches over 3,000 random lists, 3 implementations each")

```

Running it:

THE EXAMPLE FROM THE PROBLEM

```
[100, 4, 200, 1, 3, 2]
runs: 100 | 200 | 1,2,3,4 → answer 4
brute force: (4, 6) (answer, inner steps)
with the guard: (4, 3)
```

THE EDGE CASES I WOULD TEST OUT LOUD

```
[] → 0 expected 0 ok
[7] → 1 expected 1 ok
[1, 1, 1, 1] → 1 expected 1 ok
[1, 2, 0, 1] → 3 expected 3 ok
[-3, -2, -1, 5] → 3 expected 3 ok
[9, 1, 4, 7, 3, -1, 0, 5, 8, -1, 6] → 7 expected 7 ok
```

WHY THE ONE-LINE GUARD IS THE WHOLE ANSWER

a single run of consecutive numbers, $0..n-1$:

n	brute steps	guarded steps
10	45	9
100	4,950	99
1000	499,500	999
5000	12,497,500	4,999

brute force walks the whole run from EVERY starting point
→ $n(n-1)/2$ steps. The guard walks it once → $n-1$ steps.

VERIFICATION

0 mismatches over 3,000 random lists, 3 implementations each

Look at the step counts: at $n = 5,000$ it is 12.5 million against 5,000 — two and a half thousand times. And the difference in the code is one `continue`.

That contrast is the thing to say out loud in the room. Not “I added a check”, but “the check means each run is walked once instead of once per element inside it, which is $n(n-1)/2$ against n .”

6 What it costs

The solution.

building the set:	n insertions	$O(n)$
the outer loop:	n values	$O(n)$
the inner while:	ACROSS ALL RUNS, each element is visited at most once	$O(n)$ TOTAL

		$O(n)$ time
		$O(n)$ space for the set

The inner loop is the part that needs saying properly, because it looks nested and quadratic. **The guard means only run-starters enter the inner loop, and the runs are disjoint** — so all the inner work together is bounded by n .

The brute force, counted.

one run of length n , no guard:

```
start at 0:  $n$  steps
start at 1:  $n-1$  steps
...
start at  $n-1$ : 1 step
```

$\text{total} = n(n-1)/2$

```
n = 10    →          45
n = 100   →        4,950
n = 1,000 →     499,500
n = 5,000 →  12,497,500
```

with the guard: $n - 1$ steps, always.

```
n = 5,000 →          4,999
→ 2,500x
```

The time budget, which is the arithmetic nobody does.

A 45-MINUTE ROUND, minus 5 for introductions and 5 for your questions at the end = 35 minutes of problem.

SPENT WELL

```
2  restate
3  clarify
2  example by hand
3  approach + agree
20 code, narrating
5  test out loud
---
```

35

SPENT BADLY

```
0  (starts coding)
9  silent trial and error
0
1  "I'll use a hash map"
18 code, silent
2  "I think that works"
---
```

30, and 5 minutes of the interviewer asking questions you should have answered

SAME PERSON. SAME CODE.

The left column also leaves room for the follow-up question, which is where the strongest signal is.

The cost of silence, made concrete.

Interviewers typically score four axes.
Suppose each is out of 4:

CANDIDATE A: finishes, says almost nothing
problem solving 3, coding 3,
communication 1, testing 1 = 8/16

CANDIDATE B: does not quite finish, narrates
throughout, finds own bug, takes a hint well
problem solving 3, coding 2,
communication 4, testing 4 = 13/16

→ This is not a trick. Two of the four axes are
things you control completely, on any problem,
including one you do not finish.

And the cost of one clarifying question.

asking "how large can n be?" ~8 seconds
discovering in minute 25 that your
 $O(n^2)$ solution was never going
to be accepted ~25 minutes

→ The four questions cost about a minute in total
and are the cheapest insurance in the round.

7 The traps

These are not code bugs. They are the things that lose rounds that the candidate could have won.

Starting to code in minute one.

It feels productive and it is the single most expensive mistake. You are optimising for looking busy in front of somebody who is marking whether you think before acting. And if you have misread the problem, every minute after that is wasted.

Going silent for two minutes.

From the other chair, silence is indistinguishable from being lost. The interviewer cannot tell whether you are three seconds from the answer or have no idea, so they either interrupt — breaking your thought — or wait and mark you down. Both are avoidable with one sentence: "I am working out whether the bound is inclusive."

Narrating keystrokes instead of decisions.

```
"Now I'm going to write a for loop, i from zero to n..."
```

This is worse than silence in one specific way: it fills the time without carrying information, and the interviewer stops listening. Then when you do say something important, they miss it.

Defending an idea after a hint.

A hint means they have seen the problem hundreds of times and you have seen it once. “No, because...” is the most expensive two words in the round. The correct response is to repeat the hint back in your own words and act on it — that reads as collaboration, and the alternative reads as something they will have to manage every week if they hire you.

Saying “this is easy”.

If you finish quickly, it sounded arrogant. If you then get stuck, it sounds much worse. And it tells the interviewer to make the follow-up harder, which is not what you wanted.

Not testing, or “testing” by asserting.

```
"I think that's right."           ← not a test  
"It should handle the empty case" ← not a test
```

A test is you saying the values. “The set is empty, so the loop never runs, so we return best, which is 0.” Anything else is a hope with a confident voice.

Asking no questions at all.

Every problem statement is deliberately incomplete. A candidate who never asks about empties, duplicates or size is telling the interviewer they will build from an ambiguous ticket without checking, and that is a real and specific concern about how someone works.

Announcing that you have seen the problem before, and stopping there.

Say it — hiding it is worse — but say it correctly. “I have seen something like this, so let me restate it to be sure it is the same problem, and I will explain the reasoning rather than just recalling the code.” Reciting a memorised solution without being able to justify it is very visible, and it is worse than not having seen it.

Leaving the complexity until you are asked.

Stating it unprompted is a small thing that reads as professional. Being asked “and what is the complexity?” after you have said you are done reads as an omission, even when your answer is right.

And the one nobody warns you about: not managing the clock.

At minute 25 with no code, say so. “I want to make sure I get something working, so let me code the straightforward version now and optimise if there is time.” **That sentence turns a probable fail into a pass surprisingly often**, because it demonstrates exactly the judgement the job requires.

8 In the interview

How it gets asked

- “Talk me through your approach before you write any code.” — the explicit version, and a gift.
- “What are you thinking?” — you have been silent too long. Answer it, then keep narrating.
- “Are you sure?” — usually means no. Go back and check rather than defending.
- “Can you do better?” — they mean it. There is a better answer and they expect you to find it.
- “How would you test this?” — say specific inputs and their expected outputs, not categories.

The first ninety seconds

This is the script. It is the same shape on every problem.

Restate. “So — I am given an unsorted list of integers, and I need the length of the longest run of consecutive whole numbers appearing anywhere in it, not necessarily adjacent. With `[100, 4, 200, 1, 3, 2]` that would be 4, from 1, 2, 3, 4. Have I understood it?”

Clarify. “Four quick things. How big can the list be? Can it be empty? Can there be duplicates or negatives? And is there a space constraint, or may I sort it?”

Listen to the answers. “A hundred thousand — so $O(n^2)$ is ten billion operations and is out. And it must be $O(n)$, which also rules out sorting.”

Approach, brute force first. “The obvious version puts everything in a set and, from each value, walks upwards while the next one is present. Correct, but on a single long run I walk it from every starting point, so that is $O(n^2)$.”

Then the fix, with its reason. “The improvement is one line: only start walking from a value whose predecessor is not in the set. Then each run is walked exactly once, so the total inner work is bounded by n . $O(n)$ time, $O(n)$ space.”

Then hand over. “Shall I write that?”

Under ninety seconds, and it has done six things: shown you read carefully, surfaced the edge cases, extracted the constraints, produced a working solution, improved it with a stated reason, **and given the interviewer a place to redirect you before you spend twenty minutes.**

The follow-ups

“You’ve been quiet for a while. What are you thinking?”

“Sorry — let me say where I am.

What I know: I need constant-time membership checks, so a set. And I know the brute force repeats work, because on a long run it walks the same run from every element inside it.

What I am stuck on is how to avoid that repetition without doing extra bookkeeping.

What I was about to try is working a small example completely by hand — `[1, 2, 3, 4]` — and watching what I actually do, because I suspect I naturally start from the 1 and not from the 3, and I want to see why.”

Then, having done it: *“Right — I start from the 1 because there is nothing below it. So the rule is: only begin a walk at a value whose predecessor is absent.”*

That answer does three things. It states knowledge, blockage and next move separately, which is the most useful possible thing for the interviewer to hear. It shows a concrete technique for getting unstuck rather than just more staring. And it gives them a precise place to offer a hint — which they would like to do, because an interview where the candidate never gets anywhere produces no signal for them either.

“Can you do better?”

“Let me look at where the work is going rather than guessing at a different structure.

The set lookups are already constant, so that is not it. The cost is in the inner walk, so the question is: what am I doing more than once?

On a single long run — nought to n minus one — I start at 0 and walk n steps. Then I start at 1 and walk n minus one steps, over the same elements. That is $n(n-1)/2$, which for a thousand elements is half a million inner steps instead of a thousand.

So the repetition is walking a run from every point inside it, when I only need to walk it from the beginning.

The fix is to skip any value whose predecessor is in the set — that value is in the middle of a run and somebody else will cover it. One `continue` .

And I would want to justify why that is genuinely linear, because the code still looks nested. Only run-starters enter the inner loop, the runs are disjoint, and together they contain at most n elements — so all the inner work summed is bounded by n rather than multiplied by it.

$O(n)$ time, $O(n)$ space.“

“How would you test this?”

“Specific inputs with specific expected outputs, and I would say them out loud rather than describing categories.

First the given example, traced through my actual code. The set is {1, 2, 3, 4, 100, 200}. Take 1 — is 0 present? No, so it is a start; walk 2, 3, 4, stop at 5; length 4. Take 2 — 1 is present, skip. Take 100 — 99 absent, walk, 101 absent, length 1. **Answer 4, correct.**

Then the empty list. The set is empty, the loop body never runs, `best` is still 0. **Returns 0.**

Then a single element, `[7]` : no predecessor, nothing above, length 1.

Then all duplicates, `[1,1,1,1]` : the set collapses to {1}, so 1. **That is the duplicates case I asked about at the start, and it works because I iterate over the set rather than the list.**

Then negatives, `[-3, -2, -1, 5]` : -3 has no predecessor, walks to -1, length 3.

And then the case I was actually nervous about while writing: a run that is broken in the middle — `[9,1,4,7,3,-1,0,5,8,6]` , which is -1 to 9 with 2 missing. **Two runs: -1,0,1 of length 3, and 3 to 9 of length 7. Answer 7.**

The last one is the useful habit: test the thing you were unsure about while you were writing it, not just the standard edge cases. That is where my own bugs actually live.“

The model answer

“Talk me through your approach before you write any code.”

“Gladly — and I would want to do that even if you had not asked, because I would rather find out now that I have misunderstood something than in twenty minutes.

First, let me say the problem back. I have an unsorted list of integers, possibly with duplicates, and I need the length of the longest run of consecutive whole numbers appearing anywhere in it — not necessarily next to each other in the list. For `[100, 4, 200, 1, 3, 2]`, that is 4.

Second, four things I always want to know. How large can the input be — because that decides the approach. Whether it can be empty, and what I return then. Whether duplicates and negatives are possible. And whether there is a space constraint, or whether I may sort.

And I would listen carefully to those answers, because they are usually hints. ‘A hundred thousand’ means $O(n^2)$ is ten billion operations and is out. ‘It must be $O(n)$ ’ also rules out sorting, which is the other obvious approach — so the constraint has already told me quite a lot about the intended solution.

Third, I would work the example by hand before proposing anything. Runs of 100, of 200, and 1 through 4 — so the answer is 4. That takes twenty seconds and it is where I usually notice something the statement did not say.

Fourth, the brute force, out loud, with its cost — even though I can see the better answer. Put everything in a set, and from each value walk upwards while the next is present. Correct, but on a single long run I walk it from every starting point, which is $n(n-1)/2$ — half a million steps for a thousand elements instead of a thousand.

I say the brute force first deliberately. It proves I can produce a working solution before optimising one, and a candidate who jumps straight to something clever and gets it slightly wrong has shown less.

Fifth, the improvement, with the reason rather than the trick. The repetition is walking a run from every point inside it. So: only begin a walk at a value whose predecessor is absent from the set. One `continue`. Each run is then walked exactly once, so the total inner work across all runs is bounded by n . $O(n)$ time and $O(n)$ space for the set.

Then I would stop and hand it back: ‘shall I write that?’ — because if you were going to steer me somewhere else, this is the cheapest possible moment for it.

While I write, I will say the decisions rather than the keystrokes — why a set rather than a list, why I iterate the set rather than the input, what the `continue` is doing. And when I have finished I will trace the example out loud with actual values, then the empty list, one element, all duplicates and negatives, before I say the word ‘done’.

9 Recall card

You are graded on what you SAY, not what you thought. The interviewer cannot see inside your head, and they are not testing whether you can code alone in a room — they are testing what it is like to solve a hard problem sitting next to you. Most companies score four separate axes: problem solving, coding, COMMUNICATION and TESTING — and two of those you can be excellent at even on a problem you do not finish.

The arc of 45 minutes: 0-2 restate · 2-5 the four questions · 5-7 an example by hand · 7-10 brute force with its cost, then the better idea with its cost, then “shall I write that?” · 10-30 code · 30-38 test out loud · 38-45 complexity and the follow-up. The first line of code is at minute TEN, and that is correct. Always say the brute force first — it proves you can find a solution before improving one.

The four questions, on every problem, even when you can guess: HOW BIG? WHAT WEIRD INPUTS (empty, duplicates, negatives, already sorted)? WHAT DO I RETURN WHEN THERE IS NO ANSWER? MAY I MODIFY THE INPUT? Listen to the answers — “n up to 100,000” has just ruled out $O(n^2)$. They cost about a minute and are the cheapest insurance in the round.

Narrate DECISIONS, not keystrokes. “I’m using a set so lookups are constant” — not “now I open a bracket”. Thirty seconds is the silence limit; after that say what you are doing, even “I’m working out whether this bound is inclusive”. When stuck: say so, then shrink the problem, or work a small case by hand, or state what you know and what is blocking — that last sentence is what lets the interviewer give you a hint, and they want to. Take hints by repeating them back in your own words; “no, because...” is the most expensive phrase in the round.

Test by SAYING THE VALUES, not by asserting. “The set is empty, so the loop never runs, so it returns 0” is a test; “I think that’s right” is a hope. **Test the thing you were unsure about while writing**, which is where your bugs actually live — **finding your own bug is a strong positive. State the complexity unprompted. And manage the clock: at minute 25 with nothing working, say “let me get the straight-forward version down first and optimise if there is time”** — that sentence turns a probable fail into a pass surprisingly often.

The system design interview framework, memorised

1 What this is, and why they ask it

A system design interview is **forty-five minutes with no structure in it unless you bring one**. The interviewer says “design a photo-sharing service” and then stops talking. **What happens next is entirely up to you, and that is the test.**

So you bring a fixed order and you run it every time. Requirements. Estimation. Interface. Data model. Architecture. Deep dive. Six beats, in that order, out loud, with the clock in mind.

The order is not arbitrary. Each beat’s output is the next beat’s input. You cannot choose a database before you know the read-to-write ratio, **and you cannot know the read-to-write ratio before you have asked how many users there are.** Starting with the architecture — which is what almost everybody does, because it is the enjoyable part — means guessing at things you could have known.

They ask it because **the job is ambiguous and the interview is a sample of the job. Nobody at work hands you a complete specification either.** The skill being measured is whether you can take four vague words, turn them into a scoped problem with numbers attached, and then make defensible decisions.

And because a framework is what stops the two classic failures. Going too broad — forty minutes of boxes with nothing examined — **and going too narrow** — thirty minutes on the caching layer before anybody has agreed what the product does.

By the end of this lesson you have all six beats with what belongs in each, the sentences that open and close them, a time budget, the reference numbers to have memorised, a worked estimation from a hundred million users down to gigabytes per second, and the closing checklist that most candidates never reach.

2 The story

Kondappa had cooked at weddings for thirty-four years, and he had one question he asked before any other, and he would not move until somebody answered it.

“How many plates?”

Not the menu. Not the date. Not who the bride's people were and what they expected. **How many plates.**

Families found this irritating, particularly the ones who had come to talk about food and wanted to talk about food. **They would start telling him about the aunt who needed the paneer done a particular way,** and he would let them finish, and then say it again.

How many plates.

Because everything came out of that number. **Four hundred plates and eleven hundred plates are not the same wedding with more of it.** Four hundred could be done on three fires. **Eleven hundred needed six** — and six fires needed a different sort of yard, and men who could work six fires, and a lorry for the vessels, **and the thing nobody ever thought of, which was somewhere to wash eleven hundred plates between the two sittings.**

Once he had the number, the rest went in a fixed order and he never varied it. **Plates. Then sittings — one or two. Then the menu. Then the vessels. Then the fires. Then the men. Then the timings, worked backwards from when the food had to be on the table.**

His son-in-law watched him do this for two seasons and then asked whether the order ever changed.

“Once,” Kondappa said.

There had been a wedding in 1998 where the family were old customers and he liked them, **and they had begun with the menu, because the grandmother had views,** and he had gone along with it because it would have been rude not to.

Two days before, somebody mentioned the second sitting.

“We had planned the whole thing for six hundred. It was six hundred twice.”

They managed. He did not sleep for two nights, and he sent a boy on a scooter to three villages for extra vessels.

“The order is not a habit,” he said. “The order is so that the thing you cannot change is decided before the things you can.”

3 The idea in plain English

“How many plates” is scale estimation, and Kondappa’s fixed order is the framework. The reason it is fixed is his last sentence: decide the things you cannot change first.

The six beats

1. REQUIREMENTS	5 min	what are we building, and for whom
2. ESTIMATION	5 min	how many plates
3. INTERFACE	5 min	what can a caller actually do
4. DATA MODEL	5 min	what is stored, and how is it read
5. ARCHITECTURE	10 min	the boxes, and the request path
6. DEEP DIVE	10-15 min	one hard thing, properly

45 min		

Announce this at the start. It takes fifteen seconds and it changes the whole round.

“Before I start drawing — here is how I would like to use the time. About five minutes agreeing requirements and scope, five on scale estimates, then the API and data model, then the architecture, and I would like to leave fifteen minutes to go deep on whichever part you find most interesting. Does that work?”

Now you are driving. And the interviewer has been given a chance to say “actually, skip the API, I want to spend the time on the feed” — which is enormously useful information you would otherwise have to guess.

Beat 1 — Requirements

Two kinds, and candidates routinely give only the first.

Functional: what it does. Get to three to five core features and explicitly park the rest.

“For a photo-sharing service, I will assume the core is: a user can post a photo, a user can follow another user, and a user can see a feed of photos from the people they follow. I am going to leave out comments, direct messages, stories and search unless you want them in. Is that the right core?”

Naming what you are excluding is as valuable as naming what you are including. It shows you can scope, and it stops the round sprawling.

Non-functional: what it must be like. These are the ones that actually shape the design.

SCALE	how many users, how many actions each
READ:WRITE	the single most design-shaping ratio
LATENCY	"the feed loads in under 200 ms"
AVAILABILITY	three nines? four? (day 173)
CONSISTENCY	must a new post appear instantly for everyone, or is a few seconds acceptable?
DURABILITY	may we ever lose a photo? (no)

The consistency question is the one that separates people. "A like count can be a few seconds stale, but a payment cannot" is the kind of sentence that shows you know consistency is a product decision, not a technical default.

Beat 2 — Estimation

How many plates. The point is not precision — it is that the numbers decide the architecture.

DAU	→ actions per user per day	→ per second
	→ peak = 2-3x average	
	→ bytes per action	→ storage per day, per year
	→ bytes served	→ bandwidth

And the conclusion has to be said out loud, or the arithmetic was decoration.

"So that is about 23,000 reads a second against 230 writes — a hundred to one. That ratio is what tells me to build this read-optimised: heavy caching, read replicas, and probably fan-out on write so that reading a feed is a single lookup."

That last sentence is the entire reason for the beat.

Beat 3 — Interface

Five or six operations, with their parameters. It takes three minutes and it forces precision.

POST	/photos	(image, caption)	→ photo_id
GET	/feed	(cursor, limit)	→ [photo]
POST	/users/{id}/follow		→ ok
GET	/users/{id}/photos	(cursor, limit)	→ [photo]

Two details worth including because they show experience. Pagination by cursor rather than by page number — offsets get slower as they grow and give inconsistent results when things are being inserted. **And an idempotency key on anything that creates something**, so a retry does not double-post.

Beat 4 — Data model

The entities, their key fields, and — the part that matters — how they are read.

```
users      (user_id, name, created_at)
photos     (photo_id, user_id, url, caption, created_at)
follows    (follower_id, followee_id, created_at)
feed       (user_id, photo_id, created_at) ← materialised

ACCESS PATTERNS, which decide the store:
"photos by one user, newest first" → partition by user_id,
                                   sort by created_at
"the feed for one user"           → partition by user_id
"who follows this user"           → a second table with the
                                   other key order
```

Design from the access patterns, not from a tidy entity diagram. In a key-value or wide-column store you duplicate data so that every read is one lookup, and saying that deliberately — rather than apologising for it — is what shows you have used one.

Then choose, with a reason. *“Photos are blobs, so object storage plus a CDN, never a database. Metadata is relational and small, so Postgres. The feed is enormous, append-only, and read by user id, so a wide-column store partitioned by user.”*

Beat 5 — Architecture

Now draw. Client, load balancer, services, stores, caches, queues. And then walk the request path for the main flows, out loud.

“A write: the client uploads directly to object storage using a pre-signed URL — the photo never passes through my service, which saves a great deal of bandwidth. The service then writes the metadata row, and publishes an event. A worker fans the photo out into the feed table of each follower.”

“A read: the feed service looks up the user’s feed rows from the cache, falls back to the store on a miss, and returns photo ids. The client fetches the images from the CDN.”

Walking the paths is what turns a diagram into a design. A drawing with no narrated path is just boxes.

Beat 6 — Deep dive

This is where the actual signal is, and where many candidates never arrive because they spent thirty minutes on beats one to five.

Offer a choice. *“There are three things I find interesting here: the fan-out problem for users with millions of followers, how the feed cache is kept fresh, and how we shard the metadata. Which would you like?”*

And if they leave it to you, pick the one with a real trade-off, not the one you find easiest.

THE STANDARD DEEP DIVES
fan-out on write vs on read, and the hybrid
hot keys / celebrity users
caching: what, where, and how it is invalidated
sharding: the key, and what happens when it is uneven
the failure story: what happens when this box dies
consistency: what a user sees during the gap

The closing checklist

Five minutes at the end, and almost nobody does this. It is the cheapest way to stand out.

MONITORING	the four golden signals, and what pages	(day 171)
SLOs	the target, in minutes, and the budget	(day 173)
SECURITY	auth, authz, the data you would not log	(day 175)
COST	what this costs a month, roughly	(day 176)
FAILURE	what happens when each box dies	
WHAT I'D DO		
DIFFERENTLY	one honest limitation of the design	

That last item is worth more than it looks. *“The weakness of this design is the fan-out worker — if it falls behind, feeds go stale and nothing alerts on it, so I would put a lag metric on it and page on that.”* **A candidate who can name their own design’s weak point is telling you they have operated something.**

4 The picture

The round, with what belongs in each beat:

1. REQUIREMENTS - 5 min
3-5 features IN, the rest explicitly OUT
scale, read:write, latency, availability, consistency

scope

2. ESTIMATION - 5 min
DAU to QPS to storage to bandwidth
and SAY what the ratio implies

the ratio

3. INTERFACE - 5 min
5-6 operations with parameters
cursor pagination, idempotency keys

the operations

4. DATA MODEL - 5 min
entities, keys, and ACCESS PATTERNS
then choose the store, with a reason

the access patterns

5. ARCHITECTURE - 10 min
boxes and arrows, then WALK the
read path and the write path out loud

the hard part

6. DEEP DIVE - 10-15 min
offer three, let them choose
this is where the signal is



CLOSE - 5 min
monitoring, SLO, security, cost,
failure, and one honest weakness

FIGURE 178.2

Follow the arrow labels. Each beat hands the next one its input, which is why the order is fixed: you cannot choose a store before you know the access patterns, and you cannot know the access patterns before you know what the API does.

The whiteboard, laid out before you start:

+-----+-----+	
REQUIREMENTS	
IN: post, follow,	
feed	
OUT: comments, DMs,	
search, stories	
NON-FUNCTIONAL	
100M DAU	
reads:writes 100:1	
p99 < 200 ms	
99.9%	
feed may be seconds	
stale	
+-----+-----+	
NUMBERS	
23k reads/s	
230 writes/s	
40 TB/day uploaded	
9.3 GB/s served out	
+-----+-----+	
API	DATA MODEL
POST /photos	photos(photo_id, user_id...)
GET /feed	follows(follower, followee)
POST /users/{id}/	feed(user_id, photo_id, ts)
follow	
+-----+-----+	

KEEP THE REQUIREMENTS AND THE NUMBERS VISIBLE ALL ROUND.
You will refer back to them constantly, and so will they.

The two failure shapes:

TOO BROAD

40 minutes of boxes
every component named
nothing examined
no numbers anywhere

"Breadth without depth. Could
not tell whether they had
ever built any of it."

TOO NARROW

30 minutes on the cache
nothing else exists
requirements never agreed
no idea of the scale

"Went deep immediately on
a part that may not even
be the hard part here."

THE FIX IS THE SAME FOR BOTH: the fixed order, with a
clock. Beats 1-5 are DELIBERATELY shallow and quick.
Beat 6 is deliberately deep. Neither works without the
other.

5 How it actually works

Driving the round

The interviewer will not structure it for you, and their silence is not disapproval — it is the exercise.

Three sentences that keep you in control.

At the start: *"Here is how I would like to use the time..."*

At each transition: *"I think we have enough on requirements — shall I move to some numbers?"*

When you are unsure whether to go deeper: *"I could go into how the fan-out handles celebrity accounts, or move on to the data model. Which is more useful to you?"*

That last one is the most useful sentence in the round, because the interviewer has a rubric with specific things on it, **and they will usually steer you towards them if you give them the chance.**

Estimation, mechanically

Do it in the same order every time and it takes three minutes.

1. USERS	"let us say 100 million daily active users"
2. ACTIONS	reads per user per day, writes per user per day
3. PER SECOND	divide by 86,400 (call it 100,000)
4. PEAK	multiply by 2 or 3
5. BYTES	per record, per photo, per response
6. STORAGE	per day, then per year
7. BANDWIDTH	bytes served per second

Round aggressively and say that you are. *"I will call a day 100,000 seconds, which is close enough and much easier to divide by."* **Nobody wants four significant figures; they want to see that the order of magnitude drives a decision.**

The numbers worth memorising

TIME	
1 day	= 86,400 s (round to 100,000)
1 month	= 2.6 million s
1 year	= 31.5 million s
LATENCY, roughly	
memory read	100 ns
SSD read	100 microseconds (1,000x memory)
network, same zone	0.5 ms
network, same region	1-2 ms
network, cross-region	50-150 ms
disk seek (spinning)	10 ms
THROUGHPUT, order of magnitude	
one machine, simple HTTP	~10,000 req/s
one Postgres instance	~5,000 writes/s
one Redis instance	~100,000 ops/s
one Kafka partition	~10 MB/s
SIZES	
a UUID	16 bytes
a timestamp	8 bytes
a short text row	~100 bytes - 1 KB
a compressed photo	~2 MB
a thumbnail	~200 KB
one minute of 1080p video	~50 MB
COSTS (day 176)	
object storage	\$0.023 per GB-month
egress	\$0.09 per GB
cross-zone	\$0.01 per GB, each way

These are not trivia. **"A single Postgres instance does about five thousand writes a second, and I need seven hundred, so one primary is fine and I do not need to shard yet"** is a real design decision made in eight seconds — and the alternative is

sharding something that did not need it.

Scoping, and how to say no

The interviewer often adds features to see whether you will accept everything.

“Comments as well? I can add them, though I would rather finish the feed properly first — if we have time at the end I will come back to it. Which would you prefer?”

Accepting every addition is not agreeableness, it is a failure to prioritise, and it is being marked as such.

Working with the interviewer’s steering

"How would you handle X?"	→ a genuine question. Answer it.
"Are you sure about that?"	→ usually no. Re-examine, do not defend.
"What if traffic went up 100x?"	→ they want to see whether the design degrades gracefully or falls over
"Why did you choose Y over Z?" (long silence after you speak)	→ the trade-off, both directions → keep going; they are letting you drive

The variants, so the framework does not break

Not every design round is “design Twitter”.

HIGH-LEVEL DESIGN	the six beats as written
LOW-LEVEL DESIGN	"design a parking lot" - classes, relationships, patterns. Requirements and interface still come first, but beats 2 and 5 shrink to almost nothing.
A DEEP-DIVE ROUND	"design a rate limiter" - go straight to beat 6 after a short beat 1
A DEBUGGING ROUND	"the p99 has doubled" - not this framework. Metrics, then traces, then logs. (day 171-172)

Recognising which round you are in, out loud, in the first minute, is itself a signal. *“This sounds like a low-level design question, so I will spend most of the time on the class model rather than on scale — tell me if you would rather I went the other way.”*

6 The numbers

A full worked estimation, for a photo-sharing feed.

```
ASSUMPTIONS, stated out loud
100,000,000 daily active users
each reads their feed 20 times a day
each posts 0.2 photos a day (1 photo every 5 days)
an average photo is 2 MB; a thumbnail is 200 KB
metadata per photo is about 1 KB
the average user follows 200 people
```

Reads and writes per second.

```
READS
100,000,000 x 20 = 2,000,000,000 feed reads/day
2e9 / 86,400 = 23,148 reads/second average
peak x3      = ~70,000 reads/second
```

```
WRITES
100,000,000 x 0.2 = 20,000,000 photos/day
2e7 / 86,400 = 231 writes/second average
peak x3      = ~700 writes/second
```

```
RATIO: 23,148 / 231 = 100 : 1
```

```
→ READ HEAVY. That single number decides:
  cache aggressively
  read replicas, not write sharding, first
  fan-out ON WRITE (do the work once, at write
    time, so a read is one lookup)
```

Storage.

PHOTOS

$20,000,000/\text{day} \times 2 \text{ MB} = 40,000,000 \text{ MB} = 40 \text{ TB/day}$
 $\times 365 = 14.6 \text{ PB/year}$

→ obviously not a database. Object storage plus a CDN.

METADATA

$20,000,000/\text{day} \times 1 \text{ KB} = 20 \text{ GB/day}$
 $\times 365 = 7.3 \text{ TB/year}$
 $\times 5 \text{ years} = 36.5 \text{ TB}$

→ comfortably a sharded relational store, or one big one with archiving.

THE FEED TABLE, if fanning out on write

$20,000,000 \text{ photos/day} \times 200 \text{ followers each}$
 $= 4,000,000,000 \text{ feed rows/day}$
at 50 bytes a row = $200 \text{ GB/day} = 73 \text{ TB/year}$

→ and THAT number is why you cap the feed at, say, the most recent 1,000 entries per user and expire the rest.
 $100,000,000 \text{ users} \times 1,000 \text{ rows} \times 50 \text{ bytes} = 5 \text{ TB total.}$

Bandwidth, which is the line people forget.

SERVED TO USERS

$2,000,000,000 \text{ feed reads/day}$
each showing ~10 thumbnails at 200 KB
→ but with caching, assume 2 new images per read
 $2e9 \times 2 \times 200 \text{ KB} = 800,000,000 \text{ MB} = 800 \text{ TB/day}$
 $800 \text{ TB} / 86,400 \text{ s} = \sim 9.3 \text{ GB/second}$

→ This is a CDN problem, not an origin problem.
At \$0.09/GB direct egress that is 800,000 GB/day
 $\times \$0.09 = \$72,000/\text{DAY}$. With a CDN and a 95% hit rate it is a small fraction of that.

UPLOADED

$20,000,000 \times 2 \text{ MB} = 40 \text{ TB/day} = \sim 460 \text{ MB/second}$
→ and this is why the client uploads DIRECTLY to object storage with a pre-signed URL, rather than through your service.

Machines, from the throughput numbers.

70,000 peak reads/second
one application machine handles ~10,000 simple req/s
→ 7 machines, so 15-20 with headroom and redundancy

700 peak writes/second
one Postgres primary handles ~5,000 writes/s
→ ONE primary is enough. Do not shard yet.
Say that out loud - it is a decision, and the
wrong instinct is to shard because it sounds
impressive.

the feed cache: 100,000,000 users x 1,000 entries
x 50 bytes = 5 TB
→ too big for one Redis. Shard by user_id, or cache
only the active users:
20% active in an hour → 1 TB → ~15 nodes

And the time budget of the round itself.

45 minutes, minus 3 for introductions and 5 for your
questions = ~37 minutes of design.

BEAT	MINUTES	CUMULATIVE
1	5	5
2	5	10
3	4	14
4	5	19
5	9	28
6	9	37

IF YOU ARE AT MINUTE 25 AND STILL DRAWING BOXES:
say so, and cut. "I am going to stop adding components
and go deep on the fan-out, since that is the hard part
here." That sentence rescues the round.

7 The trade-offs

A framework buys structure and costs flexibility, and it is worth being honest about where it does not fit.

The six beats are wrong for a low-level design round. "Design a parking lot" needs requirements and an interface, and then classes, relationships and patterns — the estimation and architecture beats shrink to almost nothing. Running the full framework there wastes ten minutes on queries per second nobody cares about. Say which round you think you are in, in the first minute, and let them correct you.

Estimation can become theatre. The point is that a number changes a decision. If you compute a storage figure and then never refer to it, you have spent five minutes on arithmetic that bought nothing — and the interviewer notices. Every number should be followed by “so...”.

Beats one to five are deliberately shallow, and that feels wrong. You will want to explain the caching strategy while drawing the cache. Do not — you have nine minutes of deep dive reserved, and spending it early means arriving at minute forty with no time for the part that carries the most signal. “I will come back to how that cache is invalidated in the deep dive” is the sentence.

Announcing the plan can sound rehearsed. It is rehearsed, and that is fine — but say it in your own words and adapt it to what they answer. A candidate who recites a framework while ignoring the interviewer’s steering is worse than one with no framework at all, because now they are unresponsive as well as disorganised.

And the framework does not choose anything for you. It guarantees you will reach the fan-out question at minute twenty-eight. It has no opinion about whether fan-out on write is right for this product — that comes from the hundred and seventy days behind you. The structure is what buys you the time to think about the interesting part; it is not a substitute for having something to say about it.

The honest summary: the framework is worth perhaps a third of the score. It prevents both classic failures, it gets you to the deep dive with time left, and it makes you legible. The other two thirds are the actual engineering judgement — and those are not something a running order can supply.

8 In the interview

How it gets asked

- “Design a photo-sharing service.” — four words, then silence. That silence is the exam.
- “Where would you like to start?” — a gift. Answer with the plan.
- “How would you scale this?” — they mean a specific bottleneck; find it before answering.
- “What happens if this box dies?” — the failure story, for every box.

- “What would you change if you had more time?” — name a real weakness of your own design.

The first ninety seconds

“Before I draw anything, let me say how I would like to use the time, and then agree what we are building.

I would spend about five minutes on requirements and scope, five on some rough numbers, then the API and the data model fairly quickly, then the architecture — and I would like to keep about fifteen minutes at the end to go deep on whichever part you find most interesting. Does that work for you?

On scope: I am going to assume the core is three things. A user can post a photo. A user can follow another user. A user can see a feed of recent photos from the people they follow. I am explicitly leaving out comments, direct messages, search and stories — tell me if any of those should be in, but otherwise I would rather do those three properly.

On the non-functional side, five questions. How many daily active users are we designing for? What latency should a feed load in? What availability are we targeting? And the one that shapes this most: when somebody posts, does it have to appear in their followers’ feeds immediately, or is a few seconds acceptable?

I would push on that last one, because it decides the architecture. If a few seconds of staleness is acceptable, I can do the expensive work asynchronously at write time and make reads a single lookup. If it must be immediate, that option is gone.

Let us say a hundred million daily active users, a feed in under 200 milliseconds at the 99th percentile, three nines of availability, and a few seconds of staleness is fine. Shall I put some numbers on that?”

The follow-ups

“Give me the numbers, and tell me what they change.”

“I will do it in the same order I always do, and I will round hard — I want the order of magnitude, not four significant figures.

Users and actions. A hundred million daily active users. Say each reads their feed twenty times a day and posts a photo every five days, so 0.2 posts each.

Per second. Two billion feed reads a day, divided by 86,400 — I will call it a hundred thousand seconds — is about twenty-three thousand reads a second. Twenty million photos a day is about 230 writes a second. Peak is two to three times average, so seventy thousand reads and seven hundred writes.

And here is the number that matters: the ratio is a hundred to one.

That single figure decides three things. It is read-heavy, so I cache aggressively. I reach for read replicas before write sharding. And I fan out on write — do the expensive work once when a photo is posted, so that reading a feed is a single lookup rather than a query across two hundred followees.

Storage. Twenty million photos a day at two megabytes is forty terabytes a day, about fifteen petabytes a year — obviously object storage and a CDN, never a database. Metadata at a kilobyte each is twenty gigabytes a day, seven terabytes a year, which is comfortable for a relational store.

The feed table is the interesting one. Twenty million photos times two hundred followers is four billion feed rows a day — seventy-three terabytes a year, which is not sustainable. So I cap each user’s feed at the most recent thousand entries: a hundred million users times a thousand rows times fifty bytes is about five terabytes total, which is fine.

And machines. Seven hundred peak writes a second against roughly five thousand a Postgres primary can take — so one primary is enough and I would not shard yet. I want to say that explicitly, because the instinct is to shard because it sounds impressive, and adding a shard key you do not need is a decision you live with for years.”

“Go deep on the fan-out.”

“Good — that is the hard part here, and there are three approaches with a real trade-off between them.

Fan-out on write. When a photo is posted, a worker writes one row into the feed table of every follower. **Reads become a single lookup by user id, which is exactly what a hundred-to-one read ratio wants. The cost is write amplification: one post becomes two hundred writes on average.**

And it breaks completely on celebrities. A user with fifty million followers means fifty million writes for one post. At 231 posts a second overall that is fine on average, but a single celebrity post is a fifty-million-row burst, and the feed workers fall behind for everybody.

Fan-out on read. Store nothing; at read time, query the photos of everyone the user follows and merge. **No write amplification, and no celebrity problem. But every feed read becomes a scatter-gather across two hundred partitions, at seventy thousand reads a second — which is fourteen million partition queries a second. Not viable.**

So the answer is the hybrid, and this is the design I would propose. Fan out on write for ordinary users. For accounts above a threshold — say a hundred thousand followers — do not fan out at all. At read time, take the user’s pre-computed feed and merge in the recent posts of the handful of celebrities they follow.

The merge is cheap because the number of celebrities any one person follows is small — typically under fifty — and their recent posts are in a hot cache that every reader shares.

Three things I would want to name about it. The threshold is a tuning knob and it will be wrong at first, so it must be configurable without a deploy. The fan-out worker’s lag is the thing that actually breaks this — if it falls behind, feeds go stale silently and no user-facing metric moves, so I would put a lag metric on the queue and alert on it, because it is the failure with no natural symptom. And there is a consistency gap: for a few seconds after posting, some followers have it and some do not. We agreed that was acceptable in the requirements, which is why I asked at the start.”

“What would you change, and what worries you about this design?”

“Three things, and I would rather raise them than have you find them.

The fan-out worker is the weakest point. It is asynchronous, so when it falls behind nothing user-facing breaks immediately — feeds just quietly get older. There is no natural symptom, which is exactly the kind of failure that runs for hours. So: a queue-lag metric, an alert on it, and a user-visible check that the newest item in a feed is not more than a minute old.

The celebrity threshold is a guess. A hundred thousand followers is a number I made up, and the right value depends on the actual follower distribution, which is heavily skewed. I would make it a configuration value, measure the fan-out cost against the merge cost at read time, and tune it — and I would expect the first value to be wrong.

And I have optimised entirely for the read path, which is right at a hundred to one, but it makes some things awkward. Deleting a photo now means removing it from up to a thousand materialised feeds. I would handle that with a tombstone and filtering at read time rather than a synchronous delete fan-out — but it is real complexity that the fan-out-on-read design would not have.

If I had more time, the two things I would add are the operational ones. The four golden signals per service with the feed staleness as a custom one; an availability target of three nines, which is forty-three minutes a month, with an error budget attached. And I would put a cost estimate on it — the bandwidth alone is around eight hundred terabytes a day, which at direct egress prices would be seventy thousand dollars a day, and is the single strongest argument for the CDN being in the design rather than an after-thought.”

The model answer

“Design something. Show me your process, not just your answer.”

“My process is a fixed order, and the reason it is fixed is that each step decides the next one.

I start by announcing the plan, because the interviewer has a rubric and this gives them a chance to redirect me before I spend twenty minutes in the wrong place. *Five minutes on requirements, five on numbers, then API and data model quickly, then architecture, and fifteen minutes kept back to go deep.*

Then requirements, in two halves. Functional: three to five features in, and — just as important — everything else explicitly out. Non-functional: scale, read-to-write ratio, latency, availability, and consistency. I push hardest on consistency, because ‘must a post appear instantly’ is a product question whose answer changes the entire architecture, and it is the one nobody asks.

Then estimation, which is the beat I would not skip under any circumstances. Users, actions, per second, peak, bytes, storage, bandwidth. Rounded hard. And every number followed by ‘so...’ — because a number that does not change a decision was decoration. A hundred-to-one read ratio tells me to cache and to fan out on write. Seven hundred writes a second against five thousand a Postgres primary can take tells me not to shard, which is a decision I want to make deliberately rather than by instinct.

Then the interface and the data model, quickly. Five or six operations, cursor pagination rather than offsets, idempotency keys on creates. And the data model designed from the access patterns rather than from a tidy entity diagram — in a wide-column store I duplicate data on purpose so every read is one lookup.

Then the architecture — and I draw it, and then I walk the read path and the write path out loud. A drawing nobody walks through is just boxes.

Then the deep dive, and I offer a choice: ‘the fan-out for celebrity accounts, the cache invalidation, or the sharding — which is most useful to you?’ That is where the real signal is, and getting there with fifteen minutes left is the whole reason for keeping beats one to five shallow.

And I close with the checklist most candidates never reach. Monitoring and what pages. The availability target in minutes and its error budget. Authentication, authorisation, and what I would refuse to log. A rough monthly cost. What happens when each box dies. And one honest weakness of my own design.

That last one matters most. Every design has a soft spot, and a candidate who names their own — ‘the fan-out worker fails silently, so I would alert on queue lag’ — is telling you they have run something in production. A candidate whose design has no weaknesses has simply not looked.

The framework is worth maybe a third of it, and I would not pretend otherwise. It stops me being too broad or too narrow, and it buys me the time to think about the interesting part. The other two thirds are whether I actually have something to say when I get there.“

9 Recall card

Six beats, fixed order, because each one’s output is the next one’s input. 1 REQUIREMENTS (5) · 2 ESTIMATION (5) · 3 INTERFACE (5) · 4 DATA MODEL (5) · 5 ARCHITECTURE (10) · 6 DEEP DIVE (10-15). Announce the plan in the first fifteen seconds — it puts you in control and lets the interviewer redirect you before you waste twenty minutes. Beats 1-5 are deliberately shallow; beat 6 is deliberately deep. The two classic failures are **too broad** (40 minutes of boxes, nothing examined) and **too narrow** (30 minutes on the cache before anyone agreed what the product does).

Requirements has two halves and candidates give only the first. Functional: 3-5 features IN and the rest explicitly OUT — naming what you exclude shows you can scope. Non-functional: **scale, read:write, latency, availability, CONSISTENCY, durability**. Push hardest on consistency — “must a post appear instantly?” is a product question that changes the whole architecture.

Estimation is “how many plates”, and every number must be followed by “so...”. DAU → actions → ÷ 86,400 (call it 100,000) → **peak** × 2-3 → bytes → storage/year → bandwidth. 100M DAU × 20 reads = 23,000 reads/s against 230 writes/s = 100:1 — SO cache hard, read replicas before sharding, fan out on write. 20M photos × 2 MB = 40 TB/day, so object storage and a CDN, never a database. 700 peak writes against ~5,000/s for one Postgres primary — SO do not shard yet, and say that deliberately.

Memorise the reference numbers: memory 100 ns · SSD 100 µs · same-zone network 0.5 ms · cross-region 50-150 ms · one app machine ~10,000 req/s · one Postgres ~5,000 writes/s · one Redis ~100,000 ops/s · photo 2 MB · row ~1 KB · egress \$0.09/GB. They turn “should I shard?” into an eight-second decision.

Design the data model from ACCESS PATTERNS, not a tidy entity diagram; walk the read path and the write path out loud (a drawing nobody walks through is just boxes); offer the interviewer a choice of deep dive. Close with the checklist almost nobody reaches: monitoring and what pages, the SLO in minutes with its error budget, auth and what you would refuse to log, a monthly cost, what happens when each box dies — and one honest weakness of your own design. “*The fan-out worker fails silently, so I would alert on queue lag.*” A design with no named weakness is a design nobody looked at.

PRACTICE

Practice — Thinking out loud, and the design framework

DSA topic: How to think out loud in a coding round **System design topic:** The system design interview framework, memorised

Code these, in this order

Today the code is not the exercise. The talking is. For every problem below, **record yourself** — a phone is fine — **and solve it out loud from the first second to the last.** Then listen back. **That is the drill, and it is uncomfortable, and it is the single most effective hour in this whole course.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Longest Consecutive Sequence	LeetCode 128 (Medium)	The full arc: brute force, its cost, the one-line fix, and why it is linear.
2	Product of Array Except Self	LeetCode 238 (Medium)	Clarifying the constraint — no division, $O(1)$ extra — before writing.
3	Merge Intervals	LeetCode 56 (Medium)	Stating the sort key and its reason before touching the loop.
4	Word Break	LeetCode 139 (Medium)	Naming the state out loud before writing the DP.

The rule for all four

Say every one of these, in this order, before you write a line:

1. restate the problem in your own words
2. the four questions: how big, weird inputs, what to return when there is no answer, may I modify the input
3. work the given example by hand, aloud
4. the brute force AND its cost
5. the better idea AND its cost
6. "shall I write that?"

Then code, narrating decisions. Then test by saying values. Then state the complexity unprompted.

Then listen back, with this checklist

- [] Did I restate before doing anything?
- [] Did I ask all four questions?
- [] Did I work an example by hand?
- [] Did I say a brute force before the clever answer?
- [] Did I state a cost before writing?
- [] Was there any silence longer than 30 seconds?
- [] Was I narrating DECISIONS or keystrokes?
- [] Did I test by saying actual values?
- [] Did I state the complexity without being asked?
- [] Did I say "this is easy" or "it should work"?

Score the first four problems, then do four more and score again. The gap between the two scores is the only measure that matters here.

On problem 1, count the steps

Instrument both versions and **record the inner-step counts for a single run of 10, 100, 1,000 and 5,000 consecutive numbers**. Then say the ratio out loud, and say why the guarded version is genuinely linear despite the nested loop.

On problem 2, let the constraint choose

The problem forbids division and asks for $O(1)$ extra space. **Say what each of those rules out, before you think about the answer**. Then say what is left.

On problem 4, say the state first

Do not write anything until you can say, in one sentence: "`ok[i]` means the first `i` characters can be broken into dictionary words." **If the sentence does not come, the loop will be wrong in a way that is very hard to find.**

Then the silence drill

Solve any problem with a timer visible. **Every time you go quiet, note the timestamp**. Add up the silent seconds. **Anything over thirty seconds in one stretch is a gap the interviewer would have felt.**

Then the stuck drill

Pick a problem genuinely above your level. **When you get stuck, say the three-part sentence out loud** — what I know, what is blocking me, what I am about to try. **Record it**. Then listen back and ask whether somebody listening could have helped you from that sentence alone.

Then the hint drill

Have somebody give you a hint mid-problem, or read one from a solution. **Practise repeating it back in your own words before acting on it.** Notice the urge to say “yes, but I was going to...” and do not.

Then the framework drill

Take any well-known product and run the six beats out loud with a clock. **Five, five, five, five, ten, fifteen.** Stop at each boundary whether or not you are finished. **Getting to the deep dive with fifteen minutes left is the whole exercise.**

Then the estimation drill

From memory: **100 million daily active users, twenty reads and 0.2 writes each per day.** Get to reads per second, writes per second, the ratio, storage per day and per year, and bandwidth. **Then say what the ratio implies.** Do it in under three minutes.

Then the numbers drill

Recite from memory: memory read, SSD read, same-zone network, cross-region network. One machine’s requests per second, one Postgres instance’s writes per second, one Redis instance’s operations per second. **Then use them to answer “do I need to shard?” for 700 writes a second.**

Then the weakness drill

Take a design you have already produced. **Name its single weakest point and the failure that would have no obvious symptom.** Then say the metric you would alert on.

The script drill

1. Give the seven things you say before writing a line.
2. Say why the brute force goes first even when you know the answer.
3. Give the sentence that hands control back to the interviewer.
4. Say what “shall I write that?” buys you.

The questions drill

1. Give the four questions that apply to every problem.
2. For each, say what the answer would change.
3. Give two problem-specific questions for a string problem and two for a graph problem.

4. Say what “n is up to a hundred thousand” has just told you.

The narration drill

1. Give three examples of narrating a decision.
2. Give one example of narrating a keystroke, and say why it is worse than silence.
3. Give the sentence to use when you need ten quiet seconds.
4. Say the silence limit.

The stuck drill

1. Give the three moves.
2. Give the three-part sentence, in order.
3. Say why that sentence is the most valuable thing to say when stuck.
4. Say how to take a hint, and the two words never to say.

The testing drill

1. Say what a test is, and what it is not.
2. Give the five inputs you always check.
3. Say which one finds your own bugs most often, and why.
4. Say what finding your own bug does to the score.

The grading drill

1. Name the four axes.
2. Say which two you control completely.
3. Explain why “almost finished, communicated well” can beat “finished, silent”.
4. Say what to do at minute 25 with nothing working.

The six-beats drill

1. Name all six, in order, with their minutes.
2. For each, say what it hands to the next one.
3. Give the opening sentence that announces the plan.
4. Say why beats one to five are deliberately shallow.
5. Name the two classic failures and say what the framework prevents.

The requirements drill

1. Give both halves and say which candidates forget.
2. Say why naming exclusions matters.

3. List the six non-functional questions.
4. Say which one shapes the architecture most, and why.

The estimation drill

1. Give the seven steps, in order.
2. Do the worked example from 100M DAU to reads and writes per second.
3. Give the ratio and the three decisions it makes.
4. Compute photo storage per day and per year.
5. Compute the feed table's size, and say what caps it.
6. Give the bandwidth, and say what that argues for.

The numbers drill

1. Give the four latency figures.
2. Give the four throughput figures.
3. Give the four size figures.
4. Use two of them to make a sharding decision in one sentence.

The closing drill

1. Name the six items on the closing checklist.
2. Say why naming your own design's weakness is worth the most.
3. Give an example of a failure with no natural symptom, and the metric for it.

The break-it drill

For each, say what happens and how it is scored:

1. Writing code in minute one.
 2. Two minutes of silence in the middle of a round.
 3. Narrating keystrokes.
 4. "No, because..." after a hint.
 5. "This is easy."
 6. "I think that's right" instead of a trace.
 7. Asking no clarifying questions.
 8. Reciting a memorised solution without justifying it.
 9. Forty minutes of boxes with no numbers.
 10. Thirty minutes on the cache before agreeing the requirements.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Talk me through your approach before you write any code.* Restate, the four questions, an example by hand, the brute force with its cost, the better idea with its cost, and hand back control.
 2. *You've been quiet. What are you thinking?* The three-part sentence: what I know, what is blocking me, what I am about to try — said so precisely that somebody could hand you exactly the right hint.
 3. *Design something. Show me your process, not just your answer.* The six beats with their minutes, why the order is fixed, an estimation with “so...” after every number, the deep dive offered as a choice, and the closing checklist including one honest weakness.
-

Before you move on

- ☐ ☐ I have recorded myself solving a problem out loud and listened back.
- ☐ ☐ I restate the problem before doing anything else.
- ☐ ☐ I ask all four questions every time, even when I can guess.
- ☐ ☐ I know what “n up to 100,000” has already told me.
- ☐ ☐ I work a small example by hand, aloud, before proposing anything.
- ☐ ☐ I say the brute force and its cost before the clever answer.
- ☐ ☐ I state a cost before writing, not after being asked.
- ☐ ☐ I hand control back with “shall I write that?”
- ☐ ☐ I narrate decisions, never keystrokes.
- ☐ ☐ I never go silent for more than thirty seconds.
- ☐ ☐ I can give the three moves for being stuck.
- ☐ ☐ I can say what I know, what is blocking, and what I will try.
- ☐ ☐ I repeat hints back in my own words and never say “no, because”.
- ☐ ☐ I test by saying values, not by asserting.
- ☐ ☐ I test the thing I was unsure about while writing.
- ☐ ☐ I know finding my own bug is a positive.
- ☐ ☐ I know the four grading axes and which two I fully control.
- ☐ ☐ I know what to say at minute 25 with nothing working.

- [] I can name the six design beats with their minutes.
- [] I can say what each beat hands to the next.
- [] I announce the plan in the first fifteen seconds.
- [] I give both halves of requirements and name the exclusions.
- [] I push on the consistency question.
- [] I can go from 100M DAU to reads, writes, storage and bandwidth in three minutes.
- [] I follow every number with “so...”.
- [] I know the latency, throughput and size reference numbers.
- [] I can make a sharding decision from two of them in one sentence.
- [] I design the data model from access patterns.
- [] I walk the read path and the write path out loud.
- [] I offer the deep dive as a choice.
- [] I close with monitoring, SLO, security, cost, failure and one honest weakness.
- [] I answered all three questions above out loud.

Full mock: two problems, forty-five minutes

DATA STRUCTURES AND ALGORITHMS

Full mock: two problems, forty-five minutes

SYSTEM DESIGN

Full mock: one high-level design, one low-level design

Full mock: two problems, forty-five minutes

1 What this is, and why they ask it

This is the round itself. Two problems — one medium, one hard — forty-five minutes, talking the whole time, **and no running the code.**

Performing is a separate skill from knowing. You have solved several hundred problems. **You have almost certainly not solved two of them, consecutively, under a clock, out loud, while somebody watched and said nothing.** Those are different activities, and the second one has to be practised on its own.

The thing that goes wrong is never the technique. It is the four seconds where you cannot remember what comes after the fourth line — **and what makes those four seconds survivable is having been in the room before.**

They ask it because **this is the last checkpoint before the real thing.** A mock **does not teach you anything new.** What it does is find the gap between what you know and what you can produce, **and that gap is only visible under the conditions that create it.**

And because the two-problem shape is deliberate. The second problem is **harder and you have less time.** Whether you can reset after the first one — whether you arrive at the second still thinking clearly, or still worrying about a line you would have written differently — **is a real thing the round measures, and it is mostly about how you handle the transition.**

By the end of this lesson you have the rules for running a mock properly, two real problems with the reasoning written out exactly as it should be spoken, both full solutions, a scoring rubric, and a rule for what to do with whatever the score turns out to be.

2 The story

Meera had played the piece two hundred times, and she knew it was two hundred because her teacher made her keep count.

It was for the school's annual programme, on the stage in the hall with the ceiling fans that squeaked. Twenty minutes of music, in front of about four hundred people — **most of whom were somebody's parents, and almost none of whom were listen-**

ing very hard.

She was not nervous about the notes. She knew the notes the way you know your own name.

What happened was this.

She sat down and the lights were hotter than she had expected. Somebody in the third row was unwrapping something, slowly, in the quiet. **The tuning had drifted a little during the wait**, and she heard it in the first phrase — and that was not a problem, it was a quarter turn on one peg, and she had done that a thousand times.

And then she could not remember what came after the fourth line.

Not because she had forgotten it. She had not forgotten it. **It came back in about four seconds, and it felt like an hour**, and afterwards she could not have told anybody what happened in those four seconds.

She got through the piece. It was fine. Nobody in the hall noticed anything at all.

Her teacher, an old woman who did not say much and never said it warmly, listened to the whole thing standing at the side of the hall. Afterwards she said one sentence.

“Now you have played it in a room.”

Meera said, a little sharply, that she had played it two hundred times.

“You have played it two hundred times in your house, where nothing was happening. That is a different piece.”

And then she made her do the only thing that actually fixed it. **Eleven more times over the next two months — in front of her cousins, the neighbours, the watchman, whoever happened to be there. Never alone in the house. Always with somebody sitting in front of her.**

By the fourth time, the four seconds had gone.

Not because the piece had changed. Because the room had stopped being new.

3 The idea in plain English

Meera's four seconds are the whole reason for a mock. The notes were never the problem. The room was. And the only thing that fixes a room is being in one.

The rules

Follow these exactly, or you are practising something else.

- | | |
|------------------------------------|--|
| 1. A CLOCK, VISIBLE. | 45 minutes total.
20 for the first problem, 25 for the second. |
| 2. TALK THE WHOLE TIME. | Out loud. Even alone. Even feeling ridiculous. |
| 3. NO RUNNING THE CODE. | Not once. You get one editor with no run button, and you verify by reading and tracing. |
| 4. NO LOOKING ANYTHING UP. | Not the heap API. Not the sort key syntax. If you do not know it, say what you would look up and carry on. |
| 5. NO PAUSING THE CLOCK. | Not for tea, not to think, not to check one thing. |
| 6. RECORD IT. | Audio is enough. You will hear things you cannot feel. |
| 7. SCORE IT IMMEDIATELY, HONESTLY. | While it is fresh. The rubric is in section 8. |

Rule three is the one people break, and it is the most important one. Running the code is how you avoid learning to read your own code, and in the real round there is no run button. The habit you need is tracing, and it only develops when running is impossible.

The shape of the round

```
0:00 - 0:20  PROBLEM 1 (medium)
              0:00-0:03  restate, clarify, example
              0:03-0:06  brute force + cost,
                        better idea + cost, agree
              0:06-0:15  code, narrating
              0:15-0:19  trace and edge cases
              0:19-0:20  complexity, stated

0:20 - 0:21  THE RESET. Stand up. Breathe.
              Whatever happened, it is finished.

0:21 - 0:45  PROBLEM 2 (hard)
              0:21-0:25  restate, clarify, example
              0:25-0:30  brute force + cost, the idea
              0:30-0:40  code
              0:40-0:44  trace
              0:44-0:45  complexity
```

The minute at 0:20 is not padding. Carrying the first problem into the second is the commonest way a strong candidate produces a weak second half — and it is entirely avoidable by physically standing up.

What to do when the clock is against you

Three decisions, each with a sentence.

At minute 10 of a 20-minute problem, with nothing written: *“I am going to write the straightforward version now so that something works, and optimise it if there is time.”* **A working $O(n^2)$ beats an unwritten $O(n)$ every single time.**

At minute 17, with code that has a known gap: *“I have not handled the empty case — let me add that now rather than leave it.”* **Naming a gap and fixing it beats hoping it goes unnoticed, and both beat silence.**

At minute 19, unfinished: *“I have not finished, but the remaining piece is the merge step, and it would look like this.”* **Say what the missing part would be. Partial credit is real, and it is only available to people who describe what is missing.**

The reset between problems

Sixty seconds, and it has three parts.

Stand up. Physically. It breaks the posture and the state of mind together.

Say one sentence about the first problem and then stop. *“That went fine”* or *“I lost four minutes on the wrong idea”*. **One sentence. Not an analysis.**

Say what you will do differently in the next twenty-five minutes. *“I will state the cost before I start writing this time.”* **One thing, not a list.**

Then start the second problem as though the first had not happened.

What the mock is actually for

Not to see whether you can solve the problems. You can. It is to find where the gap is between knowing and producing, and there are only about four places it can be.

RECOGNITION	the shape did not arrive in the first two minutes → revise day 177's index, not the topics
ARTICULATION	you knew the answer and could not narrate it while writing → more mocks. This is the one that only responds to repetition.
IMPLEMENTATION	you knew the approach and the code came out wrong → write the seven templates from memory until they are automatic
COMPOSURE	you froze, or you rushed, or you gave up on the hard one before starting → Meera's answer: eleven more rooms

Diagnosing which one it is, is the entire value of doing this. “That went badly” is not a diagnosis.

4 The picture

The round, and where the minutes actually go:

PROBLEM 1 (medium), 20 minutes

```
0      3      6      15      19 20
|-----|-----|-----|-----|
talk  approach      CODE      trace  0()
&    & agree      & edges
clarify
```

PROBLEM 2 (hard), 25 minutes

```
21     25      30      40      44 45
|-----|-----|-----|-----|
talk  approach      CODE      trace  0()
&    & agree
clarify
```

NOTICE: coding is 9 minutes of the first and 10 of the second. Less than half of each. That is correct, and it is the allocation almost nobody uses.

The rubric, which you fill in immediately afterwards:

FOR EACH PROBLEM, score 0-4:

PROBLEM SOLVING

- 0 never found a workable approach
- 1 found one with heavy hints
- 2 found a working approach unaided
- 3 found it, and improved it with a stated reason
- 4 went straight to the right shape and justified it

CODING

- 0 did not produce running code
- 1 code with a bug I did not find
- 2 correct, but messy or over-long
- 3 clean, correct, well named
- 4 clean, correct, and handled edges without prompting

COMMUNICATION

- 0 long silences; could not follow
- 1 spoke, but narrated keystrokes
- 2 narrated decisions, some gaps
- 3 continuous, clear, took hints well
- 4 a conversation - I knew exactly where they were at every moment

TESTING

- 0 none
- 1 "I think that works"
- 2 ran the given example mentally
- 3 the example plus real edge cases
- 4 found and fixed my own bug during the trace

16 per problem, 32 total.

26+ ready. Book the interview.

20-25 ready for mediums; the hard one needs more rooms.

14-19 the knowledge is there and the performance is not.

Do four more mocks before anything else.

<14 go back to the phase that fell over, with the clock off. Speed is not yet the problem.

The two problems, and what each is for:

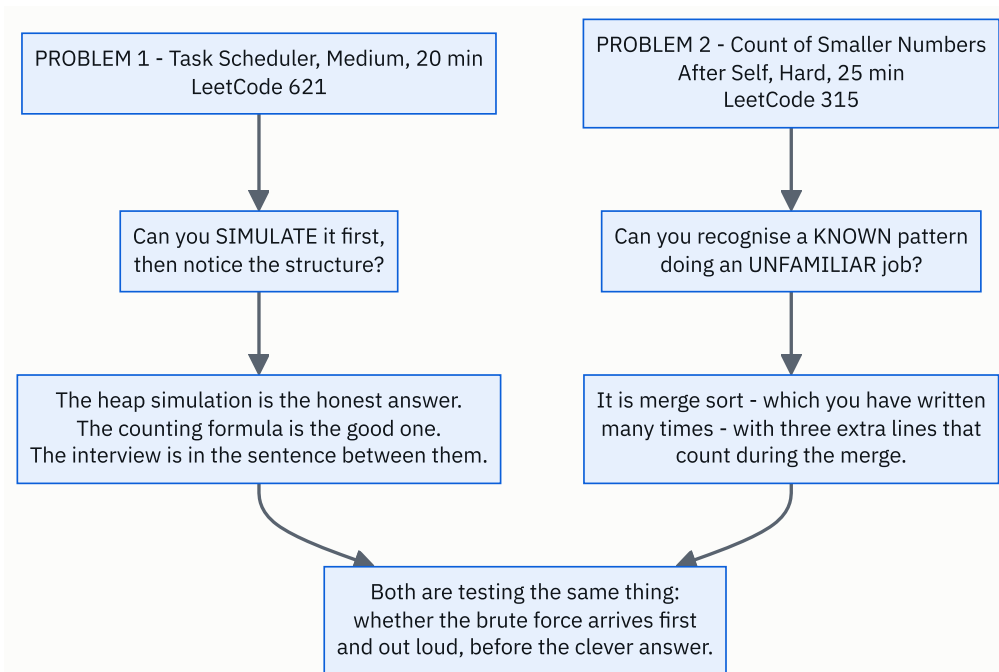


FIGURE 179.1

5 The code, built step by step

Problem 1 — Task Scheduler (Medium, 20 minutes)

Given a list of task labels and an integer n , each identical task must be separated by at least n units of time. The processor can idle. Return the minimum number of time units to finish everything.

Minute 0-3, out loud:

“So I have some tasks, and two identical ones cannot run within n units of each other. I can insert idle slots. I want the shortest total time.

For $[A, A, A, B, B, B]$ with $n = 2$: A, then B, then something else — there is nothing else, so idle. Then A, B, idle, A, B. That is 8.

Questions: how many distinct task labels — is it just A to Z? ... Yes. How large can the list be? ... Ten thousand. And can n be zero? ... Yes, in which case there is no constraint at all and the answer is just the number of tasks.”

Minute 3-6, the brute force and then the structure:

“The honest approach is to simulate the clock. At each unit, run whichever available task has the most copies remaining — a max-heap by count — and put it in a waiting queue until it becomes available again n units later. If nothing is available, idle.

That is correct. Its cost is one heap operation per time unit, and the total time can be up to about $n \times \text{tasks}$, so it is fine here but it is doing a lot of work to produce one number.

Now let me look at the structure, because I think there is a formula.

The task that appears most often decides the shape. If A appears three times and n is 2, A's copies must be at least 3 apart, so the timeline is chunks of size $n + 1$: [A _ _] [A _ _] [A] .

That is $(\text{most} - 1)$ full chunks of $(n + 1)$, plus a final chunk. And the final chunk holds one of every task that ties for the most — so $+ \text{ties}$.

For [A,A,A,B,B,B] with $n = 2$: A and B both appear three times, so $\text{most} = 3$ and $\text{ties} = 2$. That gives $(3-1) \times 3 + 2 = 8$. Which matches what I worked out by hand.

And there is one more case: if there are lots of distinct tasks, the gaps all fill up and there is never any idling. Then the answer is simply the number of tasks. So the answer is the larger of the two.

That is $O(n)$ time and $O(1)$ space, since there are at most 26 labels. Shall I write it?”

Minute 6-15, the code:

```
def task_scheduler(tasks: list[str], cooldown: int) → int:
    """The commonest task sets the skeleton; everything else fills the gaps or extends it."""
    counts = Counter(tasks)
    most = max(counts.values())
    ties = sum(1 for count in counts.values() if count == most)
    skeleton = (most - 1) * (cooldown + 1) + ties
    return max(len(tasks), skeleton)
```

Five lines, and every one of them was justified before it was written. That is the whole point of the six minutes that came first.

And write the simulation too, if there is time — or say that you would:

```
def task_scheduler_simulation(tasks: list[str], cooldown: int) → int:
    """The honest first answer: simulate the clock with a heap and a waiting queue."""
    counts = Counter(tasks)
    available = [-count for count in counts.values()]
    heapq.heapify(available)
    waiting: deque[tuple[int, int]] = deque()      # (ready_at_time, -remaining)
    time = 0
    while available or waiting:
        time += 1
        if waiting and waiting[0][0] == time:
            heapq.heappush(available, waiting.popleft()[1])
        if available:
            remaining = heapq.heappop(available) + 1
            if remaining:
                waiting.append((time + cooldown + 1, remaining))
    return time
```

-count because Python's **heapq** is a **min-heap** and you want the largest count first. Say that as you write it — it is exactly the kind of small thing that reads as fluency.

Minute 15-19, the trace, out loud:

“*[A,A,A,B,B,B]*, *n = 2*. Counts: A three, B three. *most* is 3. *ties* is 2, since both are at three. Skeleton is $(3-1) \times 3 + 2$, which is 8. Length is 6. $\max(6, 8)$ is 8. Correct.

n = 0: skeleton is $(3-1) \times 1 + 2 = 4$, length is 6, so *max* gives 6. Correct — with no cooldown you just run everything.

A single task, *["A"]* with *n = 5*: *most* 1, *ties* 1, skeleton $0 \times 6 + 1 = 1$, length 1. Answer 1. Correct — the cooldown after the last task does not count.

That last one is the case I was worried about while writing, and it comes out right because of the *most - 1*.”

Problem 2 — Count of Smaller Numbers After Self (Hard, 25 minutes)

Given a list of integers, return a list where each entry is the number of values to its right that are smaller than it.

Minute 21-25:

“For `[5, 2, 6, 1]` : to the right of 5 are 2, 6, 1, of which 2 and 1 are smaller — so 2. Right of 2 are 6 and 1; only 1 is smaller — so 1. Right of 6 is 1 — so 1. Right of 1 is nothing — 0. The answer is `[2, 1, 1, 0]` .

Questions: how long can the list be? ... A hundred thousand. Can values be negative? ... Yes. Duplicates? ... Yes — and I want to be careful there, because ‘smaller’ should be strict, so equal values do not count.

A hundred thousand rules out the obvious $O(n^2)$, which would be five billion operations.”

Minute 25-30, the brute force and the recognition:

“The definition is the brute force: for each position, count the smaller values after it. $O(n^2)$.

So I need something that counts these while doing less work. The phrase ‘how many smaller values come later’ is really ‘how many inversions involve this element’ — and counting inversions is the classic merge sort application.

Here is the idea. Merge sort splits the list in half, sorts each half, and merges. During the merge, when I take an element from the LEFT half, every element I have already taken from the RIGHT half was smaller than it and started to its right in the original list. So I add the number of right-hand elements taken so far to that element’s answer.

For that to work I have to carry the original positions along, so I sort pairs of (position, value) rather than bare values.

Cost is $O(n \log n)$ time — it is exactly merge sort, plus one addition per merged element — and $O(n)$ space for the working lists and the answer.

There is also a Fenwick tree solution: walk from the right, query how many values less than this one have been seen, then insert it. Same complexity, and it needs the values compressed into a small range first. I find the merge sort easier to justify, so I will write that one — shall I?”

Minute 30-40, the code:

```
def count_smaller(numbers: list[int]) → list[int]:
    """For each position, how many later values are smaller. Counted during a merge sort."""
    n = len(numbers)
    answer = [0] * n
    indexed = list(enumerate(numbers))          # (original position, value)
```

`enumerate` is the whole setup. The values will move during the sort and the positions must not, so they travel together.

```
def sort(part: list[tuple[int, int]]) → list[tuple[int, int]]:
    if len(part) ≤ 1:
        return part
    middle = len(part) // 2
    left = sort(part[:middle])
    right = sort(part[middle:])
```

Ordinary merge sort so far. Say that out loud — “this half is just merge sort, and the counting goes in the merge”.

```
merged: list[tuple[int, int]] = []
i = j = 0
while i < len(left) or j < len(right):
    if j == len(right) or (i < len(left) and left[i][1] ≤ right[j][1]):
        # everything already taken from the right half is smaller
        answer[left[i][0]] += j
        merged.append(left[i])
        i += 1
    else:
        merged.append(right[j])
        j += 1
return merged

sort(indexed)
return answer
```

`answer[left[i][0]] += j` is the entire algorithm and it is one line. `j` is how many elements have already been taken from the right half, and every one of them was both smaller and later.

And `≤` rather than `<` is the duplicates decision you flagged in the questions. With `≤`, an equal value on the right is not taken first, so it is not counted — which is what “strictly smaller” requires. Say that as you type the operator, because it is exactly the kind of thing an interviewer probes.

Minute 40-44, the trace:

“ $[5, 2, 6, 1]$ becomes $[(0, 5), (1, 2), (2, 6), (3, 1)]$.

Split: left $[(0, 5), (1, 2)]$, right $[(2, 6), (3, 1)]$.

Sorting the left: 5 and 2. Merging $[(0, 5)]$ with $[(1, 2)]$ — 2 is smaller, so take it from the right, j becomes 1. Then take 5 from the left and add $j = 1$ to $\text{answer}[0]$. So $\text{answer}[0]$ is 1 so far. Left is now $[(1, 2), (0, 5)]$.

Sorting the right: 6 and 1. Same thing — 1 is taken first, j becomes 1, then 6 takes $j = 1$, so $\text{answer}[2]$ is 1. Right is $[(3, 1), (2, 6)]$.

Final merge: left $[(1, 2), (0, 5)]$, right $[(3, 1), (2, 6)]$. 1 is smallest, take from right, $j = 1$. Then 2 from the left, add 1 to $\text{answer}[1]$ — $\text{answer}[1]$ is 1. Then 5 from the left, add 1 to $\text{answer}[0]$ — $\text{answer}[0]$ is now 2. Then 6.

Answer: $[2, 1, 1, 0]$. Correct.

Edge cases: empty list gives an empty answer. One element gives $[0]$. All duplicates, $[-1, -1]$, gives $[0, 0]$ because of the $\leq$. Already ascending gives all zeros; already descending gives $n-1, n-2, \dots, 0$.”

The complete solution

```
"""Day 179 - the mock round. Two problems, forty-five minutes, both solved fully."""

from __future__ import annotations

import heapq
from collections import Counter, deque

# ===== PROBLEM 1
def task_scheduler_simulation(tasks: list[str], cooldown: int) → int:
    """The honest first answer: simulate the clock with a heap and a waiting queue."""
    counts = Counter(tasks)
    available = [-count for count in counts.values()]
    heapq.heapify(available)
    waiting: deque[tuple[int, int]] = deque()  # (ready_at_time, -remaining)
    time = 0
    while available or waiting:
        time += 1
        if waiting and waiting[0][0] == time:
            heapq.heappush(available, waiting.popleft()[1])
        if available:
            remaining = heapq.heappop(available) + 1
            if remaining:
                waiting.append((time + cooldown + 1, remaining))
    return time

def task_scheduler(tasks: list[str], cooldown: int) → int:
    """The commonest task sets the skeleton; everything else fills the gaps or extends it."""
    counts = Counter(tasks)
    most = max(counts.values())
    ties = sum(1 for count in counts.values() if count == most)
    skeleton = (most - 1) * (cooldown + 1) + ties
    return max(len(tasks), skeleton)

# ===== PROBLEM 2
def count_smaller(numbers: list[int]) → list[int]:
    """For each position, how many later values are smaller. Counted during a merge sort."""
    n = len(numbers)
    answer = [0] * n
    indexed = list(enumerate(numbers))  # (original position, value)

    def sort(part: list[tuple[int, int]]) → list[tuple[int, int]]:
        if len(part) ≤ 1:
            return part
        middle = len(part) // 2
        left = sort(part[:middle])
        right = sort(part[middle:])

        merged: list[tuple[int, int]] = []
        i = j = 0
        while i < len(left) or j < len(right):
            if j == len(right) or (i < len(left) and left[i][1] ≤ right[j][1]):
                # everything already taken from the right half is smaller
                answer[left[i][0]] += j
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
        return merged

    sort(indexed)
    return answer
```

```

def count_smaller_brute(numbers: list[int]) → list[int]:
    """The definition.  $O(n^2)$  - fine for checking, useless at  $n = 100,000$ ."""
    return [sum(1 for later in numbers[i + 1:] if later < numbers[i])
            for i in range(len(numbers))]

if __name__ == "__main__":
    print("PROBLEM 1 - Task Scheduler (Medium)")
    cases = [
        (["A", "A", "A", "B", "B", "B"], 2, 8),
        (["A", "C", "A", "B", "D", "B"], 1, 6),
        (["A", "A", "A", "B", "B", "B"], 0, 6),
        (["A", "A", "A", "A", "A", "B", "C", "D", "E", "F", "G"], 2, 16),
        (["A"], 5, 1),
    ]
    for tasks, cooldown, expected in cases:
        formula = task_scheduler(tasks, cooldown)
        simulated = task_scheduler_simulation(tasks, cooldown)
        print(f" {''.join(tasks):<14} n={cooldown} formula={formula:<3} "
              f"simulation={simulated:<3} expected={expected}"
              f" {'ok' if formula == simulated == expected else 'WRONG'}")

    print()
    print(" WHY THE FORMULA WORKS, on AAABBB with n=2:")
    print(" both A and B appear 3 times, so most=3 and ties=2")
    print(" the 3 copies of the commonest task need 2 gaps of")
    print(" size n=2 between them, so 2 chunks of (n+1)=3 slots:")
    print(" [A _ _] [A _ _] then a final chunk")
    print(" the final chunk holds one of EACH tied task: A B")
    print(" skeleton = (3-1) x (2+1) + 2 = 8")
    print(" filled in: A B idle | A B idle | A B = 8 slots")
    print()
    print(" AND WHY THE max(): A A A B B B C C C D D E, n=2")
    many = list("AAABBBCCCCDE")
    print(" most=3, ties=3 → skeleton = 2x3 + 3 = 9")
    print(f" but there are {len(many)} tasks, so nothing can idle")
    print(f" → max(12, 9) = {task_scheduler(many, 2)}")

    print()
    print("PROBLEM 2 - Count of Smaller Numbers After Self (Hard)")
    for data, expected in [
        ([5, 2, 6, 1], [2, 1, 1, 0]),
        ([-1], [0]),
        ([-1, -1], [0, 0]),
        ([], []),
        ([2, 0, 1], [2, 0, 0]),
        ([1, 2, 3, 4], [0, 0, 0, 0]),
        ([4, 3, 2, 1], [3, 2, 1, 0]),
    ]:
        got = count_smaller(data)
        print(f" {str(data):<16} → {str(got):<16} expected {expected}"
              f" {'ok' if got == expected else 'WRONG'}")

    print()
    print("VERIFICATION")
    import random

    bad = 0
    for _ in range(2000):
        size = random.randint(0, 30)
        sample = [random.randint(-20, 20) for _ in range(size)]
        if count_smaller(sample) != count_smaller_brute(sample):
            bad += 1

    for _ in range(500):
        letters = [chr(ord("A") + random.randint(0, 5)) for _ in range(random.randint(1, 25))]
        cooldown = random.randint(0, 4)

```

```

if task_scheduler(letters, cooldown) != task_scheduler_simulation(letters, cooldown):
    bad += 1
print(f" {bad} mismatches over 2,000 counting cases and 500 scheduling cases")

```

Running it:

PROBLEM 1 - Task Scheduler (Medium)

AAABBB	n=2	formula=8	simulation=8	expected=8	ok
ACABDB	n=1	formula=6	simulation=6	expected=6	ok
AAABBB	n=0	formula=6	simulation=6	expected=6	ok
AAAAAABCDEFG	n=2	formula=16	simulation=16	expected=16	ok
A	n=5	formula=1	simulation=1	expected=1	ok

WHY THE FORMULA WORKS, on AAABBB with n=2:

both A and B appear 3 times, so most=3 and ties=2
the 3 copies of the commonest task need 2 gaps of
size n=2 between them, so 2 chunks of (n+1)=3 slots:

[A _ _] [A _ _] then a final chunk

the final chunk holds one of EACH tied task: A B

skeleton = (3-1) x (2+1) + 2 = 8

filled in: A B idle | A B idle | A B = 8 slots

AND WHY THE max(): A A A B B B C C C D D E, n=2

most=3, ties=3 → skeleton = 2x3 + 3 = 9

but there are 12 tasks, so nothing can idle

→ max(12, 9) = 12

PROBLEM 2 - Count of Smaller Numbers After Self (Hard)

[5, 2, 6, 1]	→ [2, 1, 1, 0]	expected [2, 1, 1, 0]	ok
[-1]	→ [0]	expected [0]	ok
[-1, -1]	→ [0, 0]	expected [0, 0]	ok
[]	→ []	expected []	ok
[2, 0, 1]	→ [2, 0, 0]	expected [2, 0, 0]	ok
[1, 2, 3, 4]	→ [0, 0, 0, 0]	expected [0, 0, 0, 0]	ok
[4, 3, 2, 1]	→ [3, 2, 1, 0]	expected [3, 2, 1, 0]	ok

VERIFICATION

0 mismatches over 2,000 counting cases and 500 scheduling cases

Look at the two columns in problem 1: the formula and the simulation agree on every case, including `n = 0`. Writing both and checking them against each other is a genuinely good use of spare minutes if the round has any left — and saying “let me verify the formula against a simulation on the examples” is a strong thing to say out loud.

And look at `[-1, -1]` in problem 2: `[0, 0]`. That is the `≤` doing its job. With `<` it would have been `[1, 0]`, which is a wrong answer that only appears on duplicates — the exact case you asked about in minute 23.

6 What it costs

Problem 1.

FORMULA

Counter over the tasks:	n
max over at most 26 counts:	26
the tie count:	26

O(n) time, O(1) space (26 labels is a constant)

n = 10,000 → ~10,052 operations

SIMULATION

one loop iteration per time unit
total time can reach (most - 1) x (cooldown + 1) + ties
each iteration is a heap push or pop: log 26

worst case: 1 task type repeated 10,000 times, n = 100
→ (10,000 - 1) x 101 + 1 = 1,009,900 iterations

→ O(total_time x log 26), which is fine here and is
doing a million steps to produce a number the formula
gets in ten thousand.

Problem 2.

BRUTE FORCE

for each i, scan everything after it
→ n(n-1)/2

n = 100,000 → 5,000,000,000 operations NO

MERGE SORT

log₂(n) levels, each merging every element once
→ n log n

n = 100,000 → 100,000 x 17 = 1,700,000 fine

→ ~3,000x fewer operations.

SPACE

the (position, value) pairs:	O(n)
the merge buffers:	O(n)
the recursion depth:	O(log n)
the answer:	O(n)

→ O(n) total.

Note: recursion depth is log n = 17 at 100,000, so no
recursion limit problem. If it were a linked structure
of depth n, it would be a different conversation.

The clock, which is the other cost in this lesson.

45 minutes, two problems.

WHERE IT ACTUALLY GOES, when it goes well:

talking before writing	7 min	(16%)
writing	19 min	(42%)
tracing and edges	8 min	(18%)
complexity and questions	3 min	(7%)
the reset	1 min	(2%)
slack	7 min	(15%)

WHERE IT GOES WHEN IT GOES BADLY:

silent trial and error	12 min
writing	22 min
"I think that works"	1 min
the interviewer finding your bug for you	6 min
no slack, no reset, and no second problem finished	

SAME PERSON. The seven minutes of slack in the first column came ENTIRELY from the seven minutes of talking.

7 The traps

These are the mock-specific ones — the failures that only appear under a clock with somebody watching.

Running the code.

The single most common way a mock is wasted. You cannot run it in the real round, so a mock where you ran it has not tested the thing that will actually be tested: whether you can find your own bug by reading. Close the run button. Trace with your mouth.

Looking one thing up.

"I will just check the `heapq` argument order." That is a paused clock and a broken mock. In the room you would say "I would use `heapq.heappush(heap, item)` — correct me if the argument order is the other way round", and carry on. Practise saying that sentence, because it works and hiding the gap does not.

Carrying problem one into problem two.

A candidate who spent nineteen minutes on a medium arrives at the hard one already behind and already rattled, and the second half is worse than their ability. The one-minute reset exists precisely for this. Stand up. One sentence. One thing

to change. Then start clean.

Freezing on the hard one and not starting.

Twenty-five minutes of not knowing beats twenty-five minutes of silence. Say the brute force. Say its cost. Say what makes it slow. On problem 2, “it is $O(n^2)$ because for every element I rescan everything after it” is already most of the way to the merge sort, because it names the repeated work.

Optimising before anything works.

At minute 10 with no code, the correct move is to write the $O(n^2)$ version. Partial credit is real. An unwritten optimal solution scores zero on two of the four axes.

Testing by asserting.

```
"I think that handles duplicates."
```

Not a test. A test is: “ `[-1, -1]` . The merge takes the right-hand `-1` first because of the `≤` , so `j` is 1 when... no, wait — with `≤` the LEFT one is taken first, so `j` is still 0. Answer `[0, 0]` . Correct.” Notice that the trace caught a moment of confusion and resolved it. That is what tracing is for.

The off-by-one that a trace catches and a glance does not.

```
skeleton = most * (cooldown + 1) + ties      # WRONG
skeleton = (most - 1) * (cooldown + 1) + ties # right
```

```
task_scheduler(["A"], 5) with the wrong version → 7
                        correct answer          → 1
```

There is no cooldown after the last task, and the `most - 1` is what says so. The single-element case finds it instantly and nothing else does.

And the strict-versus-equal comparison in problem 2.

```
left[i][1] < right[j][1]    → [-1, -1] gives [1, 0]    WRONG
left[i][1] ≤ right[j][1]    → [-1, -1] gives [0, 0]    right
```

One character, and it is only wrong when there are duplicates. Which is why “can there be duplicates?” was worth asking in minute 23 — you asked it, so you knew to check it.

Scoring yourself kindly.

A mock you score generously has told you nothing. The rubric is only useful if a 2 means 2. The point is to find the gap, and a gap you have talked yourself out of will still be there on the day.

8 In the interview

How it gets asked

- *“I have two problems for you today. Let us start with this one.”* — and the clock starts.
- *“Take your time.”* — they do not mean it literally. They mean “do not panic”.
- *“Would that work?”* — usually no. Go back and check rather than defending it.
- *“How would you improve it?”* — the invitation to the second half of the problem.
- *“We have about ten minutes left.”* — a warning, and an instruction. Wrap up and test.

The first ninety seconds

Problem 1, as it should be spoken:

“Let me restate it. I have a list of tasks, and two identical tasks must be at least n units apart. The processor can idle if nothing is available. I want the minimum total time.

Working the example: $[A, A, A, B, B, B]$ with $n = 2$. A, B, idle, A, B, idle, A, B. That is 8 units.

A few questions. How large can the list be? ... Ten thousand. How many distinct labels — uppercase letters only? ... Yes, so at most 26, which means anything I do per-label is effectively constant. And can n be zero? ... Yes, and then there is no constraint and the answer is just the number of tasks.

My first thought is to simulate: at each time unit, run whichever available task has the most copies left, using a max-heap, and hold cooling tasks in a queue. That is correct and it walks the whole timeline.

But I think there is a formula, and let me say why before I commit to either. The most frequent task fixes the skeleton — its copies are forced to be $n + 1$ apart, so the timeline is $(\text{most} - 1)$ chunks of size $n + 1$, plus a last chunk containing one of each task that ties for the most.

And if there are lots of distinct tasks, every gap fills up and nothing idles — so the answer is the larger of that skeleton and the total number of tasks.

$O(n)$ time, constant space. Shall I write that, and then check it against the simulation on the examples if we have time?”

The follow-ups

“You have ten minutes left and problem two is not working. What do you do?”

“I say so, and then I make a specific decision out loud rather than continuing quietly.

First I would state exactly where I am. *“The merge sort structure is right and I am not confident about the counting line. Let me trace a four-element example rather than keep staring at it.”*

Tracing is the correct move at that point and rereading is not. Rereading finds nothing after the second pass — I have already looked at those lines with the same assumption three times. **A trace with actual values breaks the assumption**, which is exactly what is needed.

If the trace finds it, I fix it and I say what it was. *“j was being read after the increment, so it was counting one too many.”* **Finding my own bug is worth more than never having it.**

If the trace does not find it with four minutes left, I switch to describing. *“I have not got this correct. What it should do is: during the merge, when I take an element from the left half, add the number of right-hand elements already taken to that element’s answer — because each of those was smaller and came later. The structure is right and the bug is in how I am tracking that count.”*

That last paragraph is worth real credit and most candidates never say it, because it feels like admitting defeat. It is the opposite: it demonstrates that I understand the algorithm independently of whether my typing was correct, and those are exactly the two things being scored separately.

What I would not do is go silent for the last four minutes, which is what most people do, and which converts a partially working answer into no information at all.

“How do you get better at this, specifically?”

“By diagnosing which of four things went wrong, rather than concluding that it went badly.

Recognition — if the shape did not arrive in the first two minutes, the fix is the pattern index, not more problems. **I would re-read the trigger table until the pattern arrives before I finish reading the problem.**

Articulation — if I knew the answer and could not narrate it while writing, that is the one that only responds to repetition. **More mocks, recorded, listened back.** It is genuinely a separate motor skill.

Implementation — if I had the approach and the code came out wrong, the fix is writing the standard templates from memory until they are automatic: the two-pointer loop, the window, binary search on the answer, the monotonic stack, BFS, backtracking with the undo, a one-dimensional DP.

Composure — if I froze, or rushed, or did not start the hard one, **that is the room rather than the material, and the only cure is more rooms.**

The reason I would separate them is that they have completely different fixes, and the default response to a bad mock — solve fifty more problems — only helps with one of the four.

And I would set a rule for the score, so it is a decision rather than a feeling. Above 26 out of 32, book the interview. 20 to 25, mediums are fine and the hard one needs more rooms. 14 to 19, four more mocks before anything else. Below that, go back to whichever phase fell over, with the clock off — because speed is not the problem yet.”

“You froze for thirty seconds in the middle. What was happening?”

“Honestly — I lost the thread of my own plan while typing, which is a specific and fixable thing rather than a general nervousness.

What happened is that I was holding two things at once: the merge logic and the counting logic, and I started the loop before I had decided which one the `j` belonged to. So halfway through the line I could not remember which of the two I was writing.

The fix is not to be calmer. It is to have said the invariant out loud before typing the loop: ‘*`j` is how many elements I have taken from the right half so far, and that is exactly the count I add.*’ With that sentence said, the line writes itself; without it, I was deriving it and typing it at the same time.

And the general version of that is the thing I would take away. Every loop has one invariant, and saying it before writing the loop costs eight seconds and removes the class of freeze where you are constructing and transcribing simultaneously.

The other half of the fix is the silence itself. Thirty seconds of nothing is invisible from your side and very long from mine. What I should have said is ‘give me a moment — I am working out exactly what `j` counts’, and that sentence costs nothing and keeps you with me.”

The model answer

“One medium, one hard. The clock is running. Begin.”

“Before I start — may I take about two minutes on each problem before writing? I find I write much less nonsense that way, and it gives you a chance to redirect me early if I have misread something.

(Problem 1, minutes 0-3.) Let me restate it, work the given example by hand, and ask my four questions: how large, what weird inputs, what to return when there is nothing, and whether I may modify the input.

(Minutes 3-6.) Then the brute force with its cost, out loud, even though I can see a formula — because having a working answer before optimising one is worth more than leaping to something clever and getting it subtly wrong. Then the structure, then the improved cost, and then ‘shall I write that?’

(Minutes 6-19.) Code, narrating decisions rather than keystrokes — why a Counter, why most - 1 and not most, why the max at the end. Then trace the example with actual values, then the edge cases: $n = 0$, a single task, and every task distinct. Then state the complexity without being asked.

(Minute 20.) Then I stand up for a moment, whatever happened. One sentence about the first problem and one thing I will do differently — and then I start the second one as though the first had not occurred. Carrying the first problem into the second is how a good candidate produces a weak second half.

(Problem 2.) Same shape, more time, and the difference is that I expect not to see the answer immediately. So I will spend longer describing what the brute force repeats, because naming the repeated work is usually what names the technique — ‘for every element I rescan everything after it’ is most of the way to counting during a merge.

And two rules for myself on the clock. At minute ten with nothing written, I write the straightforward version so that something works. At the end, if it is not finished, I describe precisely what the missing piece would do rather than going quiet — that is worth real credit and silence is worth none.

The thing I would want you to see across both problems is not whether I finish. It is that you always knew where I was — what I had decided, what I was unsure about, and what I was about to try. If I go wrong, you should be able to stop me in one sentence rather than watching me for four minutes.

Shall we start?”

9 Recall card

Performing is a separate skill from knowing, and it only improves in the room. Meera's four seconds were never about the notes. **The rules of a real mock: a visible clock (20 + 25), talk continuously, NO RUNNING THE CODE, no looking anything up, no pausing, record it, and score it honestly straight afterwards. Rule three matters most** — running the code is how you avoid learning to read your own code, and in the room there is no run button.

Less than half of each problem is coding. 3 min restate and clarify · 3 min brute force with its cost then the better idea with its cost then “shall I write that?” · 9 min code · 4 min trace and edges · 1 min complexity. **The seven minutes of slack at the end come entirely from the seven minutes of talking at the start.**

The minute at 0:20 is the RESET and it is not padding. Stand up. **One sentence about the first problem, one thing to change, then start the second as though the first had not happened** — carrying it across is how a strong candidate produces a weak second half. **Three clock decisions, each with a sentence: at minute 10 with nothing written, write the brute force (“so that something works”); with a known gap, name it and fix it; at the end unfinished, DESCRIBE the missing piece.** Partial credit is real and only available to people who say what is missing.

Score four axes out of four, per problem: problem solving, coding, communication, testing — 32 total. 26+ book the interview · 20-25 mediums fine, the hard one needs more rooms · 14-19 four more mocks · under 14 go back to the phase that fell over with the clock off. A mock scored generously has told you nothing.

Diagnose WHICH of four things failed, because they have different fixes. **Recognition** → re-read the pattern index, not more problems. **Articulation** → more recorded mocks; it is a motor skill. **Implementation** → write the seven templates from memory until automatic. **Composure** → more rooms. **The default response to a bad mock — solve fifty more problems — only helps with one of the four.** And say every loop's invariant out loud before writing the loop: constructing and transcribing at the same time is what produces the freeze.

Full mock: one high-level design, one low-level design

1 What this is, and why they ask it

This is the design loop as it actually happens: two rounds, back to back, testing different things.

The high-level round asks whether you can take four vague words and produce a system. Scale, storage, architecture, trade-offs. The low-level round asks whether you can take a small, well-understood problem and produce clean, extensible objects. Classes, responsibilities, interfaces.

They are genuinely different skills and candidates are usually much stronger at one. The loop is designed to find out which — which is why on-site days almost always include both, often with the same interviewer watching how you switch.

They ask it because the second round is the one where people fall apart, and not for a reason anyone predicts. You arrive at the low-level round still thinking about the fan-out problem you did not quite resolve. Or the first round went well and you are careless. Either way you are designing the previous system instead of this one.

And because the low-level round catches a specific gap. A candidate who can size a petabyte pipeline and cannot say what belongs in a class has learned system design from articles rather than from building software — and the low-level round is the one that finds it, in about ten minutes.

By the end of this lesson you have both mock rounds worked through in full — a video streaming service at high level and an expense-sharing app at low level — the different shape each round takes, a rubric for both, and the sixty seconds in between that decides how the second one goes.

2 The story

Sharada made forty pots on a good morning, and the ones she sold were all supposed to be the same size, which was the whole difficulty.

The clay came in a heap by the door. She wedged it, cut it into forty lumps with a wire, and weighed the first three on a hand balance until her hand knew what the weight felt like. After that she did not weigh anything.

Then she sat down at about four in the morning and threw pots until half past nine.

Her daughter-in-law, who had come from a family that did not do this work, watched for a week and then asked the question everybody asks.

“How do you not get tired?”

Sharada said she did get tired. That was not the interesting part.

“The interesting part is that the eleventh pot has to be as good as the first. And the thirty-second has to be as good as the eleventh.”

And the way she managed it was a thing the girl had watched forty times a morning without ever seeing.

Between every single pot, she stopped.

Not for long — four or five seconds. She lifted the finished pot off, set it on the board beside her, wiped both hands down the cloth over her knee, **and looked at the empty wheel for a moment before she put the next lump on.**

The girl had assumed this was tidiness.

“If I carry the last pot into the next one, the next one is wrong,” Sharada said. **“If the last one went badly, I press too hard. If it went well, I get careless.”**

“Either way I am making the previous pot again, instead of this one.”

She put the next lump down and started centring it.

“So I put it on the board. The board is over there. It is finished.”

And then the thing the girl repeated to her own children years later.

“Forty pots is not one long job. It is forty jobs — and the wiping of the hands is what makes them separate.”

3 The idea in plain English

Sharada’s five seconds are the whole lesson. A design loop is not one long round. It is two separate rounds, and what makes them separate is a deliberate reset.

The two rounds are not the same exercise

HIGH-LEVEL DESIGN -----	LOW-LEVEL DESIGN -----
"design a video streaming service"	"design an expense-sharing app"
boxes, arrows, data stores scale, storage, bandwidth trade-offs between components "how does this survive a zone failure?"	classes, methods, interfaces responsibilities, coupling trade-offs between designs "what happens when they add a new kind of split?"
THE OUTPUT IS AN ARCHITECTURE	THE OUTPUT IS A CLASS MODEL
THE FAILURE: too broad, no numbers, nothing examined	THE FAILURE: one enormous class that does everything

What is shared is the opening. Both start with requirements, and both are ruined by starting to draw before agreeing what is being built.

What differs is everything after minute five. The high-level round wants numbers before architecture. The low-level round wants entities and responsibilities, and almost no arithmetic at all.

The high-level round, in six beats

Exactly the framework from [day 178](#).

1. REQUIREMENTS	5	features in, features out, scale, latency, availability, consistency
2. ESTIMATION	5	users → QPS → storage → bandwidth and SAY what each number implies
3. INTERFACE	5	5-6 operations
4. DATA MODEL	5	entities and ACCESS PATTERNS
5. ARCHITECTURE	10	boxes, then WALK the paths
6. DEEP DIVE	10+	one hard thing, offered as a choice

The low-level round, in six different beats

Same length, completely different content.

1. REQUIREMENTS	5	what the thing does, in verbs. "add an expense", "settle up", "show what I owe"
2. ENTITIES	5	the nouns, and what each one KNOWS
3. RELATIONSHIPS	5	who holds whom, one-to-many, what owns what's lifetime
4. INTERFACE	5	the public methods, with signatures
5. THE HARD PART	10	the bit that will change - and the pattern that absorbs the change
6. EXTENSION	10	"now add X" - and your design either absorbs it or does not

Beat 6 is the whole round. They will ask for a feature you did not plan for, and the score is entirely about whether your design absorbs it with a new class or requires you to edit five existing ones.

Which is why the low-level round is really a test of one idea: what changes, and what does not.

The reset, which is the sixty seconds that matters

Sharada's cloth.

1. PHYSICALLY STOP. Stand up, get water, look away.
2. ONE SENTENCE about the round that has finished.
"That went well." "I never got to the deep dive."
ONE. Not an analysis.
3. ONE THING to do differently.
"I will get to the numbers faster."
4. NAME THE NEXT ROUND OUT LOUD.
"This one is low-level. Classes and
responsibilities, not scale."

Step four is the one people skip and it is the most valuable. The commonest failure in a back-to-back loop is running the wrong framework — doing capacity estimation for a vending machine, or drawing boxes when they asked for classes. Saying which round you are in, in the first minute, prevents it entirely.

"This sounds like a low-level design question, so I am going to spend most of the time on the class model and the extension points rather than on scale. Tell me if you would rather I went the other way."

What each round is actually scoring

HIGH-LEVEL

- did they ask about scale before designing?
- do the numbers change any decision?
- did they get to a deep dive, or just draw?
- can they name a trade-off in both directions?
- did they mention failure, monitoring, cost?

LOW-LEVEL

- is there one class per responsibility?
- could I add a new variant without editing existing classes?
- are the interfaces small?
- did they name a pattern and JUSTIFY it, rather than reciting it?
- did their design survive the extension question?

The last line of each is the one that carries most of the weight.

4 The picture

The loop, and the cloth between the rounds:

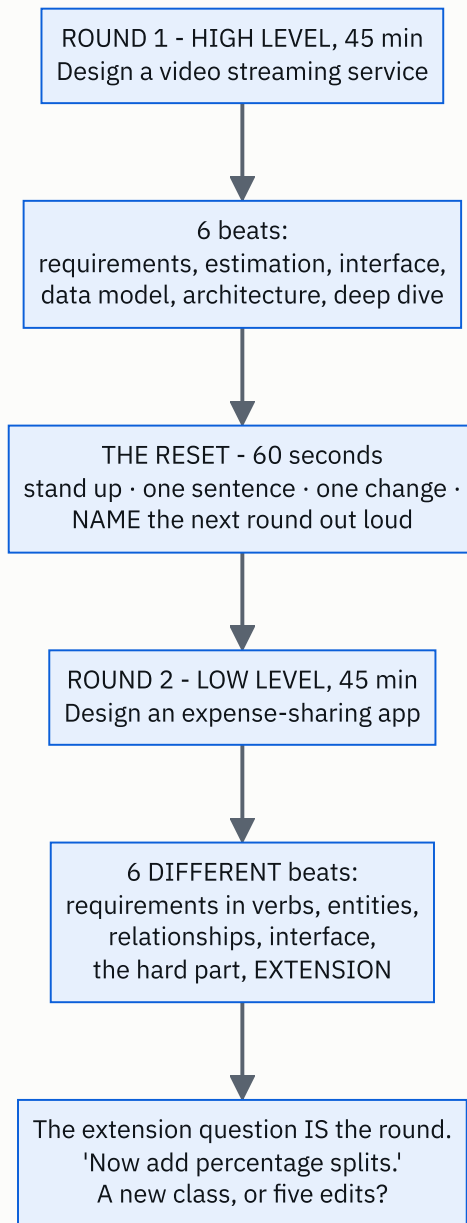


FIGURE 179.2

The two failure shapes, side by side:

HIGH-LEVEL, DONE BADLY

40 min of boxes
no numbers anywhere
every component named,
nothing examined
"we'd add a cache"
(where? invalidated how?)
no failure story
no cost, no monitoring

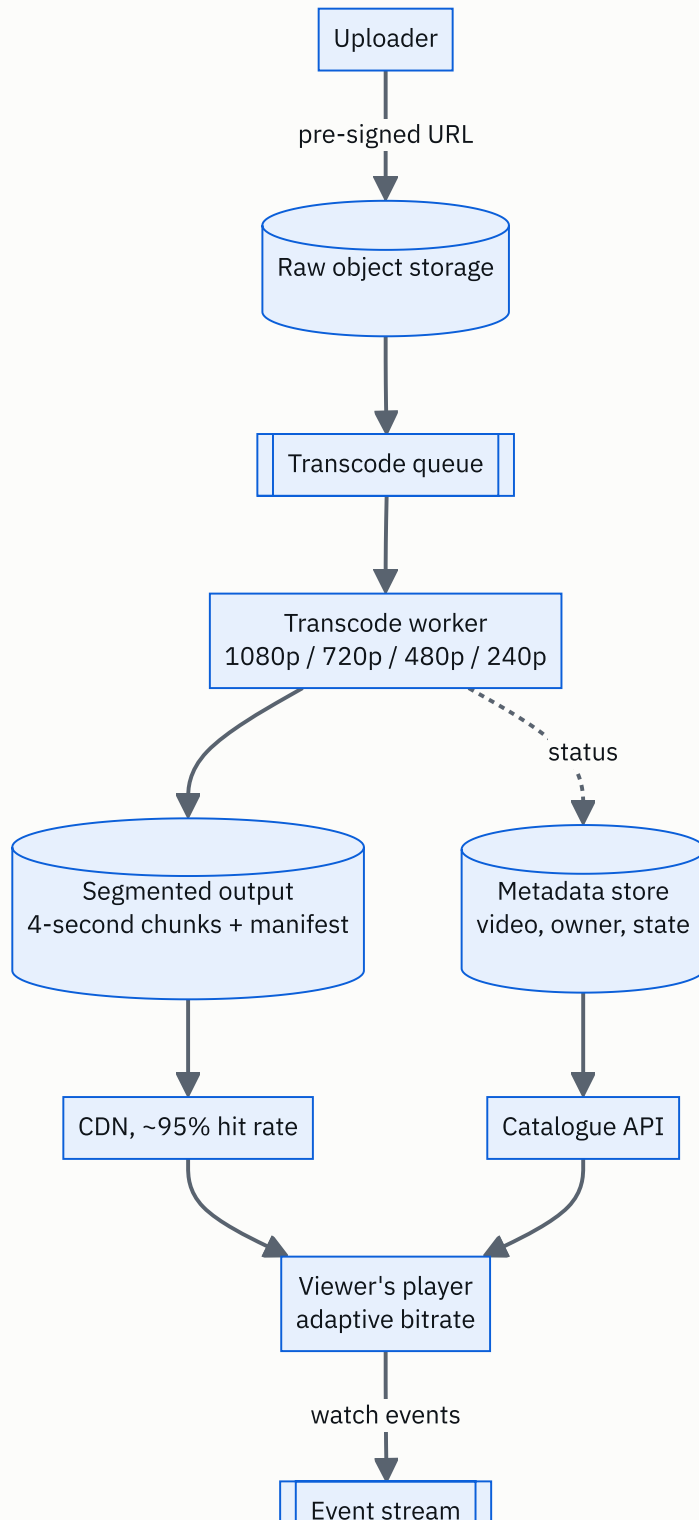
"Breadth without depth."

LOW-LEVEL, DONE BADLY

one class called Manager
that does everything
no interfaces
"if the split type is
equal... else if exact...
else if percentage..."
→ and then "add a new
split type" means
editing that method

"Has read about patterns.
Has not written software
that had to change."

The video streaming architecture from round one:



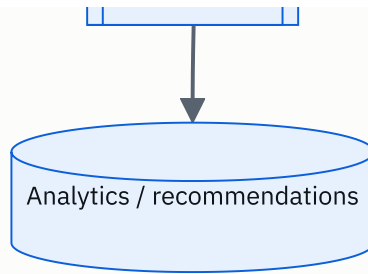


FIGURE 179.3

Two things to notice, because they are the design. The upload never passes through your service — the client goes straight to object storage with a pre-signed URL, which removes hundreds of megabytes a second from your servers. And the viewer never touches your origin either — the CDN does, at a 95% hit rate, which is the difference between a bandwidth bill you can pay and one you cannot.

5 How it actually works

ROUND 1 — Design a video streaming service (45 minutes)

Beat 1, requirements (minutes 0-5).

“Let me agree the core first. I will assume: a creator uploads a video; the system transcodes it into several qualities; a viewer searches or browses and plays one, with the quality adapting to their connection. I am leaving out comments, live streaming, monetisation and recommendations unless you want them — tell me now if any of those is the point.

Non-functional: how many users? ... Say a hundred million daily. Playback must start in about two seconds and must not stutter — so buffering is the real quality bar, not availability of the catalogue page. Uploads can be slow; nobody minds waiting minutes for a video to process, so the upload path can be asynchronous and eventually consistent, and the playback path cannot. Is that the right split?”

Beat 2, estimation (minutes 5-10). The numbers are in section 6 and they decide three things: object storage rather than a database, a CDN rather than an origin, and transcoding as an asynchronous pipeline.

Beat 3, interface (minutes 10-14).

```

POST /videos (title, size) → upload_url, video_id
POST /videos/{id}/complete → ok, transcoding starts
GET /videos/{id} → metadata + manifest_url
GET /videos/{id}/manifest → the list of quality levels and segments
GET /search?q= → [video]
POST /videos/{id}/events (position, quality, buffering)

```

“The upload returns a pre-signed URL rather than accepting the bytes, and I would say why: at 460 megabytes a second of uploads, routing them through my own service would need a fleet of machines doing nothing but copying.”

Beat 4, data model (minutes 14-19).

```

videos      (video_id, owner_id, title, duration, state,
             created_at)
renditions  (video_id, quality, manifest_url, bytes)
watch       (user_id, video_id, position, updated_at)

THE OBJECT STORE holds:
  raw/{video_id}           the original
  hls/{video_id}/{quality}/seg_N.ts  4-second segments
  hls/{video_id}/master.m3u8  the manifest

ACCESS PATTERNS
  "the metadata for one video" → key lookup, cached hard
  "videos by this creator"     → partition by owner_id
  "where was I in this video" → key lookup by
                               (user_id, video_id)
→ metadata is small and relational: Postgres, with a
  read-through cache. The video bytes never go near it.

```

Beat 5, architecture (minutes 19-28). The diagram in section 4, plus the two paths walked out loud.

“Upload: the client asks for an upload URL, puts the file straight into object storage, then calls `complete`. That publishes a message. A pool of transcode workers picks it up and produces four qualities, each cut into four-second segments, plus a manifest. Each worker writes its output back to object storage and updates the video’s state. When all renditions are done, the video becomes playable.

Playback: the player fetches the manifest, then pulls segments from the CDN. It measures its own download speed and switches quality between segments — that is adaptive bitrate, and it is why segments are short. Watch events go to an event stream, not to the API, because they are high-volume and nobody needs them synchronously.”

Beat 6, deep dive (minutes 28-45). Offer three:

“I could go deeper on the transcoding pipeline and how it handles a failed worker, on how the CDN is kept warm for a video that suddenly becomes popular, or on the storage cost, which I think is the most surprising number here. Which is most useful?”

And the transcoding deep dive, since it is the one with the real trade-off:

*“Transcoding one hour of video takes roughly an hour of CPU per output quality, so four qualities is four CPU-hours. The obvious design gives one video to one worker, and then a two-hour film takes eight hours to become playable.***

So instead I split the video into chunks first* — say one minute each — and fan them out across many workers in parallel. A two-hour film becomes 120 chunks times 4 qualities, which is 480 independent jobs. With 100 workers that is under five minutes instead of eight hours.***

The costs, and I want to name all three. The chunk boundaries have to fall on key-frames, or the segments will not join cleanly. The jobs must be idempotent, because a worker will die mid-chunk and the job will be retried — writing to a deterministic path in object storage gives that for free. And you need a completion tracker: 480 jobs must all finish before the video flips to playable, so something has to count them, and that counter is the piece that fails silently if you are not watching it.

Which is the metric I would alert on: the age of the oldest video still stuck in ‘transcoding’.”

And the close (last five minutes):

“Monitoring: the four golden signals per service, plus two custom ones — the rebuffer ratio, which is the real quality measure, and the oldest video stuck in transcoding. SLO: 99.9% on playback start, which is forty-three minutes a month. Security: pre-signed URLs expire in fifteen minutes, and playback URLs are signed per user so a link cannot be shared indefinitely. Cost: dominated by egress, which is why the CDN is in the design rather than an optimisation. And the weakness of my design is the completion tracker — if it drifts, videos sit unpublished with no user-facing symptom at all.”

ROUND 2 — Design an expense-sharing app (45 minutes)

Beat 1, requirements as verbs (minutes 0-5).

“Let me get the operations rather than the features. A user can create a group. A user can add an expense to a group, paid by one person, split between several. A user can see what they owe and what they are owed. A user can settle up.”

And one question that shapes everything: how many ways can an expense be split? ... Equally, by exact amounts, and by percentage. Good — that is the part that will change, so it is where the design has to be flexible.”

I am leaving out authentication, currencies and notifications unless you want them.”

Beat 2, entities (minutes 5-10). The nouns, and what each one knows.

User	id, name, email	
Group	id, name, members	
Expense	id, group, description, amount, paid_by, splits, created_at	
Split	user, amount	← what ONE person owes for ONE expense
Balance	derived, never stored: who owes whom, and how much	

“The important decision here is that Balance is derived, not stored. I could keep a running balance per pair of users, and then every edit or deletion of an old expense has to correctly unwind it. Deriving it from the expense list is slower and cannot drift. I would derive it and cache the result, and I would say that trade-off out loud rather than choosing silently.”

Beat 3, relationships (minutes 10-14).

Group	1	---	*	Expense	a group owns its expenses
Expense	1	---	*	Split	an expense owns its splits
Split	*	---	1	User	a split refers to a user
Group	*	---	*	User	membership

LIFETIME: deleting a group deletes its expenses, which deletes their splits. A User is NOT owned by anything.

INVARIANT, and it is the one worth stating:
 for every Expense, $\text{sum}(\text{split.amount}) = \text{expense.amount}$
 → enforced in the Expense constructor, so an invalid Expense cannot exist. Not checked later.

Beat 4, the interface (minutes 14-19).

```
class ExpenseService:
    def add_expense(self, group_id, description, amount,
                    paid_by, strategy) → Expense: ...
    def balances(self, group_id) → dict[UserId, Decimal]: ...
    def who_owes_whom(self, group_id) → list[Settlement]: ...
    def settle(self, group_id, payer, payee, amount) → None: ...
```

“Four methods. `balances` returns a net position per person; `who_owes_whom` turns that into actual transfers, and separating those two is deliberate — they are different questions and one of them has an interesting algorithm behind it.”

Beat 5, the hard part: the thing that will change (minutes 19-30).

“The split types are the part that will change, so that is where the design has to absorb change.”

The version I would not write:

```
def compute_splits(self, kind, amount, participants, values):
    if kind == "EQUAL":
        ...
    elif kind == "EXACT":
        ...
    elif kind == "PERCENT":
        ...
```

That method has to be edited every time a new split type appears, which is the open-closed principle being violated in one screen. And every edit risks the two types that already worked.

Instead: a strategy interface.

```
class SplitStrategy(Protocol):
    def split(self, amount: Decimal,
              participants: list[UserId]) → list[Split]: ...

class EqualSplit:
    def split(self, amount, participants):
        share = (amount / len(participants)).quantize(CENTS)
        splits = [Split(user, share) for user in participants]
        # the rounding remainder goes to ONE named person, never nowhere
        remainder = amount - share * len(participants)
        splits[0] = Split(splits[0].user, splits[0].amount + remainder)
        return splits

class ExactSplit:
    def __init__(self, amounts: dict[UserId, Decimal]):
        self.amounts = amounts

    def split(self, amount, participants):
        if sum(self.amounts.values()) != amount:
            raise ValueError("exact splits must sum to the total")
        return [Split(user, self.amounts[user]) for user in participants]
```

Adding a percentage split is a new class and no edits anywhere. That sentence is what the round is testing.

And I want to point at the rounding, because it is the detail that separates people. Thirty rupees between three people is fine. Ten rupees between three is 3.33 each, which is 9.99, and one paisa has vanished. So the remainder has to go somewhere deliberate — I give it to the payer. And every amount is a `Decimal`, never a float*, because 0.1 plus 0.2 is not 0.3 in binary floating point and money must not do that.”*

Beat 6, the extension question (minutes 30-45). This is the round.

They will ask for something. The three that come up:

“Now add a split where two people pay for one expense.”

“That changes `paid_by` from one user to a list of contributions. `Expense` grows a `payments` list alongside `splits`, with the same invariant — the payments must sum to the total. The strategies are untouched, because they only ever decided who owes what, not who paid. And the balance calculation becomes: for each expense, each payer is credited what they put in and each participant is debited what they owe. One change to one entity.”

“Now simplify the debts — minimise the number of transfers.”

“That is a nice one, and it is a greedy algorithm on the net balances.

Compute each person’s net position — positive means they are owed, negative means they owe. The positives and negatives each sum to the same total. Then repeatedly take the largest creditor and the largest debtor and settle the smaller of the two amounts between them. **Each transfer zeroes at least one person,** so with `n` people there are at most `n - 1` transfers.***

***I would be careful to state what it does not do: it does not find the provable minimum*, which is NP-hard in general.** It finds a good answer with at most $n - 1$ transfers, which is what people actually want* — and I would say that rather than claim optimality.*

Design-wise this is a new class — `DebtSimplifier` — that takes balances and returns settlements. It touches nothing existing, because `balances` was already a separate method from `who_owes_whom`. That separation, which looked like fussiness in beat four, is what makes this free.*“*

“Now support multiple currencies.”

“This is the one that hurts, and I would say so. An `Amount` becomes a value object of a number and a currency, and every arithmetic operation has to refuse to add two different currencies. Then a rate is needed, and a rate has a time — an expense from March must be settled at March’s rate or at today’s, and that is a product decision, not a technical one, so I would ask.

What this shows is that money being a bare `Decimal` was a simplification, not a design. If I expected currencies from the start I would have made `Amount` a value object on day one, and everything else would be unchanged. That is a real cost of my design and I would rather name it than defend it.*“*

6 The numbers

The video streaming estimation, in full.

```
ASSUMPTIONS, said out loud
100,000,000 daily active viewers
each watches 5 videos a day, averaging 10 minutes
500,000 videos uploaded a day, averaging 10 minutes
1080p is about 5 Mbit/s; 720p 2.5; 480p 1; 240p 0.4
```

Playback traffic — the number that decides the whole design.

```
watch-minutes per day
100,000,000 x 5 x 10 = 5,000,000,000 minutes/day
                      = 83,300,000 hours/day

assume an average delivered bitrate of 2 Mbit/s
83,300,000 hours x 3,600 s x 2 Mbit/s
= 599,760,000,000 Mbit/day
= 599,760,000,000 / 8 = 74,970,000,000 MB
= ~75 PB/day of video delivered

per second: 75 PB / 86,400 = ~870 GB/second

→ AT $0.09/GB OF DIRECT EGRESS THAT WOULD BE
75,000,000 GB x $0.09 = $6,750,000 PER DAY.

→ which is why this is a CDN problem, and why the CDN
is in the design from the first minute rather than
added as an optimisation. This single number is the
architecture.
```

Storage.

$500,000 \text{ uploads/day} \times 10 \text{ minutes} = 5,000,000 \text{ minutes/day}$

RAW, at ~50 MB per minute of 1080p:

$5,000,000 \times 50 \text{ MB} = 250 \text{ TB/day of originals}$

RENDITIONS, four qualities, roughly 90 MB per minute
for all four together:

$5,000,000 \times 90 \text{ MB} = 450 \text{ TB/day}$

TOTAL: ~700 TB/day = 255 PB/year

at \$0.023/GB-month for the hot tier:

$700,000 \text{ GB/day} \times 30 \text{ days} = 21,000,000 \text{ GB in month one}$

$\times \$0.023 = \$483,000/\text{month}$, and growing every month

→ so tiering is not optional. Most videos are watched
in their first week:

7 days hot, then infrequent access, then archive

→ and the raw originals go to deep archive
immediately after transcoding, since they are only
needed if you re-transcode.

Transcoding capacity.

$5,000,000 \text{ minutes of video uploaded per day}$

$\times 4 \text{ output qualities}$

$= 20,000,000 \text{ minutes of transcoding work per day}$

at roughly real time on one CPU core:

$20,000,000 \text{ minutes} = 333,333 \text{ core-hours/day}$

$/ 24 \text{ hours} = \sim 13,900 \text{ cores running constantly}$

→ ~870 sixteen-core machines, flat out, all day.

AND THIS IS WHY IT IS A SPOT-INSTANCE WORKLOAD:

the jobs are interruptible and idempotent, so a 70%
discount applies directly.

$870 \text{ machines} \times \$0.60/\text{hour} \times 730 \text{ hours} = \$381,000/\text{month}$

on spot at 25% of that = ~\$95,000/month

Playback start latency, which is the actual quality bar.

TARGET: video starts in under 2 seconds

manifest fetch, from the CDN	~50 ms
first segment (4 s of 480p, ~500 KB)	~200 ms on a 20 Mbit/s line
player buffers 2 segments before starting	~400 ms

	~650 ms

→ comfortable, and the reason is that playback STARTS at a low quality and steps up. Starting at 1080p would need 2.5 MB before the first frame.

→ which is the whole argument for adaptive bitrate, and it is a latency argument rather than a bandwidth one.

And the low-level round's only arithmetic, which is the rounding.

10.00 split three ways
 $10.00 / 3 = 3.3333\dots$
rounded to 3.33 each
 $3.33 \times 3 = 9.99$
→ ONE PAISA IS MISSING

over 1,000,000 expenses that is 10,000 rupees unaccounted for, and every one of them is a support ticket.

THE FIX: compute the shares, then give the remainder to a named person - the payer - deliberately.

AND: Decimal, never float.
 $0.1 + 0.2 = 0.30000000000000004$ in binary floating point. Money must never do that.

7 The trade-offs

The high-level round's trade-offs, stated as they should be in the room.

Transcode per video or per chunk. Per video is simple: one job, one worker, no coordination. It also means a two-hour film takes eight CPU-hours and is unplayable for most of a working day. Chunked transcoding parallelises to minutes and costs you three things: keyframe-aligned boundaries, idempotent

jobs because workers die, and a completion tracker that must count 480 finished jobs before the video goes live. I would chunk, and I would say that the tracker is the piece that fails silently.

Store every rendition, or transcode on demand. Storing four qualities of everything is 450 TB a day of output for content most of which nobody watches twice. Transcoding on demand is far cheaper in storage and adds seconds to playback start, which is the one thing that must not be slow. The real answer is a split: store renditions eagerly for popular content and lazily for the long tail — and “popular” is knowable, because view counts are extremely skewed.

Push to the CDN or let it pull. Pre-pushing a new video to every edge costs bandwidth for content that may never be watched. Pulling on demand means the first viewer at each edge waits. For a creator with ten million subscribers, pre-warming is obviously right; for the long tail it is obviously wrong — so the decision is per video, driven by the creator’s audience size.

The low-level round’s trade-offs.

Derive balances or store them. Stored balances make “what do I owe” a single read, and every edit or deletion of an old expense must correctly unwind them — which is where the bugs live, and they are the kind that produce a wrong number silently. Deriving from the expense list cannot drift and costs a scan. I would derive and cache, invalidating on any expense change — and I would say that I am choosing correctness over a read that is already fast enough.

Strategy objects or a switch. A switch is fewer files and it is honestly fine for two variants. The moment a third arrives, every addition edits a method that two working variants depend on. I would use strategies here specifically because the interviewer told me there were already three split types, which is the signal that more are coming. Reaching for a pattern without that signal is over-engineering, and saying why you chose it is the difference between the two.

And the honest cost of the back-to-back loop itself.

Two forty-five minute design rounds in a morning is genuinely tiring, and the second one is where the average candidate scores worse — not from lack of knowledge but from carrying the first round in. Sharada’s answer is the only one that works: put it on the board, wipe your hands, and name the next round out loud before you start it. The single most expensive mistake in a design loop is running the wrong framework — estimating queries per second for a vending machine, or drawing boxes when the question was about classes.

8 In the interview

How it gets asked

- “Two design rounds today — a high-level one now and a low-level one after the break.” — plan for the reset.
- “Design a video streaming service.” — six words, then silence.
- “Design an expense-sharing app.” — sounds smaller, and is not.
- “Now add percentage splits.” — the extension question. This IS the low-level round.
- “Which part would you like to go deeper on?” — pick the one with a real trade-off, not the easy one.

The first ninety seconds

Round one, high level:

“Before I draw — here is how I would like to use the time: five minutes on requirements and scope, five on numbers, then the API and data model quickly, then the architecture, keeping about fifteen minutes to go deep on whatever you find most interesting.

On scope: I will assume the core is upload, transcode, browse and play, with quality adapting to the viewer’s connection. I am leaving out live streaming, comments, monetisation and recommendations — say if any of those is actually the point.

And one requirement that will shape everything: uploads can be slow and asynchronous — nobody minds waiting minutes for processing — but playback must start in about two seconds and must never stutter. So I have one path that can be eventually consistent and one that cannot, and that split is the design.

Shall I put numbers on it?”

Round two, low level — and the first sentence is different on purpose:

“This sounds like a low-level design question, so I am going to spend most of the time on entities, responsibilities and how the design absorbs change, rather than on scale. Stop me if you would rather I went the other way.

*Let me get the operations as verbs first. Create a group. Add an expense, paid by someone, split between several people. See what I owe and what I am owed. Settle up.***

And one question that will decide the shape: how many ways can an expense be split? ... Equal, exact and percentage. Then that is the part that will change, and it is where my design has to be flexible.”

The follow-ups

“Go deeper on the transcoding pipeline.”

“Transcoding one hour of video takes roughly one hour of CPU per output quality, so four qualities is four CPU-hours per hour of video. At five million uploaded minutes a day times four qualities, that is twenty million minutes of work a day — about 13,900 cores running constantly, or 870 sixteen-core machines flat out.

The naive design gives one video to one worker. A two-hour film then takes eight CPU-hours and is unplayable for most of a day, which is a bad experience for exactly the creators you most want.

So I chunk first. Split the source into one-minute pieces, and fan them out. A two-hour film becomes 120 chunks times four qualities — 480 independent jobs — and with a hundred workers that is under five minutes instead of eight hours.

Three costs, and I would name all three rather than let you find them.

The chunk boundaries must land on keyframes, or the re-assembled segments will not decode cleanly at the joins.

The jobs must be idempotent, because workers will die mid-chunk and the job will be retried. Writing output to a deterministic path in object storage gives that for free — a retry simply overwrites the same object. And that also makes this a perfect spot-instance workload: interruptible, restartable, and a 70% discount — on 870 machines that is a saving of about 285,000 dollars a month.

And there has to be a completion tracker: all 480 jobs must finish before the video is published. That counter is the piece that fails silently — if it drifts, videos sit unpublished and no user-facing metric moves. So the metric I would alert on is the age of the oldest video still in the transcoding state, which is a symptom rather than a cause, and there is no natural alternative signal.

One more decision worth surfacing: whether to store all four renditions eagerly. At 450 terabytes a day of output for content that is mostly watched once, I would store eagerly for popular creators and lazily for the long tail — view counts are extremely skewed, so that split is knowable rather than a guess.”

“Now add percentage splits to the expense app.”

“A new class, and nothing else changes — which is the answer I designed for.

`PercentageSplit` implements the same `SplitStrategy` interface: given the total and the participants, it returns a list of splits. `Expense`, `Group`, `Balance` and the service are all untouched.

That is the open-closed principle doing actual work: open for extension, closed for modification. And the reason it is free here is a decision I made in the first ten minutes — putting the split logic behind an interface rather than in a conditional inside `add_expense`.

The version I deliberately avoided was `if kind == 'EQUAL' ... elif ... elif ...`, and it is worth saying what is wrong with it. Not that it is ugly. That every new split type edits a method the two working types depend on, so adding percentage risks breaking equal, and the test suite for that method grows combinatorially.

Two details inside the new class, because they are where the real bugs are.

Validation belongs in the strategy, not in the caller. Percentages must sum to a hundred; exact amounts must sum to the total. Each strategy knows its own rule, and it raises in the constructor rather than returning something invalid — the same reason the `Expense` invariant is enforced in its constructor.

And rounding. A hundred rupees split 33/33/34 is fine, but 33.3/33.3/33.4 percent of ten rupees is 3.33, 3.33, 3.34. Whenever the shares do not divide exactly, the remainder must go somewhere named — I give it to the payer. And every amount is a `Decimal`, never a float, because binary floating point cannot represent 0.1 and money must never do that.”

“How do you handle the second round when the first one went badly?”

“Deliberately, in about sixty seconds, and I would do the same thing whether it went badly or well — because being pleased with yourself is as dangerous as being rattled.

First, physically stop. Stand up, get water, look at something else. **It sounds trivial and it is the part that actually works,** because the state I need to leave behind is partly postural.

Second, one sentence about the round that finished. *‘I never got to the deep dive.’* **One sentence, not an analysis.** An analysis in the corridor is just the previous round continuing.

Third, one thing to do differently. *‘I will get to the numbers by minute eight.’* **One thing.** A list of five is a list I will not follow.

Fourth, and this is the one people skip: name the next round out loud before it starts. *‘This one is low-level. Classes and responsibilities, not scale.’* **The commonest failure in a back-to-back loop is running the wrong framework** — doing capacity estimation for a vending machine, or drawing boxes when they asked for a class model. **Saying which round I am in, in the first minute, prevents it entirely,** and it also gives the interviewer a chance to correct me if I have read it wrong.

And I would say the framing out loud if it helps: two design rounds is not one long round. It is two rounds, and what makes them separate is the deliberate stop in between. If I carry the first one in, I spend the second one designing the previous system.”

The model answer

“Two design rounds, back to back. Begin.”

“Let me say how I will run both, because they are different exercises and I want to be explicit about that.

The high-level round I run in six beats: requirements, estimation, interface, data model, architecture, deep dive. The order is fixed because each one decides the next — I cannot choose a store before I know the access patterns, and I cannot know the read-to-write ratio before I have asked how many users there are. Beats one to five are deliberately shallow so that I arrive at the deep dive with fifteen minutes, because that is where the signal is.

And in that round every number must be followed by ‘so...’. For video streaming, the number that decides the architecture is 75 petabytes a day of delivered video — which at direct egress prices would be six and three-quarter million dollars a day. That single figure is why a CDN is in the design from the first minute rather than mentioned as an optimisation at minute forty.

The low-level round is six different beats: requirements as verbs, entities, relationships, interface, the part that will change, and the extension question. Almost no arithmetic, and the whole round is really beat six. They will ask for a feature I did not plan for, and the score is whether my design absorbs it with a new class or requires me to edit five existing ones.

So in beat one I actively hunt for what will change. *‘How many ways can an expense be split?’* Three — so splits go behind an interface, and adding a fourth is a new class and no edits. And I would justify the pattern rather than recite it: I am using a strategy here because you told me there are already three variants, which is the signal that more are coming. Without that signal it would be over-engineering.

In both rounds I close the same way: what fails, what I would monitor, and one honest weakness of my own design. For the streaming system it is the transcode completion tracker, which fails silently and needs an alert on the oldest stuck video. For the expense app it is that money is a bare decimal, which was a simplification and not a design — the moment currencies appear, `Amount` should have been a value object from day one.

And between the two rounds I will take a minute, stand up, and say out loud which round I am about to be in. Two design rounds is not one long round; it is two rounds, and the stop in between is what makes them separate. Otherwise I spend the second one designing the first one again.”

9 Recall card

Two rounds, two different exercises. **HIGH-LEVEL**: six beats — requirements, estimation, interface, data model, architecture, deep dive — with every number followed by “so...”, and beats 1-5 deliberately shallow so you reach the deep dive with fifteen minutes. **LOW-LEVEL**: six **DIFFERENT** beats — requirements as **VERBS**, entities, relationships, interface, the part that will change, and the **EXTENSION** question. Almost no arithmetic, and beat 6 **IS** the round.

The reset between them is Sharada’s cloth, and it is sixty seconds. Physically stop · **ONE** sentence about the round that finished · **ONE** thing to change · and **NAME THE NEXT ROUND OUT LOUD**. The commonest failure in a back-to-back loop is running the wrong framework — estimating queries per second for a vending machine, or drawing boxes when they asked for classes. “Forty pots is not one long job.”

Video streaming, and the one number that is the architecture: 100M viewers × 5 videos × 10 min at ~2 Mbit/s = ~75 PB/day delivered, ~870 GB/second — \$6.75 MILLION A DAY at direct egress. So the CDN is in the design at minute one. Uploads go straight to object storage via a pre-signed URL (460 MB/s never touches your servers). **Chunk before transcoding** — a 2-hour film is 480 jobs, five minutes instead of eight hours — at the cost of **keyframe-aligned boundaries**, **idempotent jobs** (workers die; deterministic output paths), and a completion tracker that fails **SILENTLY**. Alert on the age of the oldest video stuck in transcoding. Interruptible + idempotent = **spot instances**, 70% off.

Expense sharing, and the one decision that is the round: put split types behind a `SplitStrategy` interface. Adding percentage splits is then a new class and zero edits — where `if kind == 'EQUAL' ... elif ...` means every addition edits a method two working variants depend on. **Justify the pattern by the signal** (“you told me there are already three”), or it is over-engineering. **Derive balances rather than store them** — stored balances drift when an old expense is edited. **Enforce** `sum(splits) == amount` in the **CONSTRUCTOR**, so an invalid expense cannot exist.

Money: `Decimal`, never float ($0.1 + 0.2 \neq 0.3$), and **when shares do not divide exactly the remainder goes to a NAMED person** — 10.00 split three ways is 9.99, and a missing paisa across a million expenses is ten thousand rupees of support tickets. **Close both rounds the same way: what fails, what you would monitor, and one honest weakness of your own design**. A design with no named weakness is one nobody looked at.

PRACTICE

Practice — The full mock, coding and design

DSA topic: Full mock: two problems, forty-five minutes **System design topic:** Full mock: one high-level design, one low-level design

Code these, in this order

Today is not practice. Today is the round. Set a clock, start it, and do not stop it for anything.

The rules, and breaking any one of them means you practised something else:

```
[ ] a visible clock: 20 minutes, then 25 minutes
[ ] talking out loud from the first second
[ ] NO running the code. Not once.
[ ] NO looking anything up
[ ] NO pausing
[ ] recorded
[ ] scored immediately, honestly
```

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Task Scheduler	LeetCode 621 (Medium)	Simulating first, then noticing the structure — and saying the sentence between them.
2	Count of Smaller Numbers After Self	LeetCode 315 (Hard)	Recognising merge sort doing an unfamiliar job.

Then, after the round, and only then

Score both problems on the rubric before you look at any solution.

PROBLEM SOLVING	0-4	CODING	0-4
COMMUNICATION	0-4	TESTING	0-4

16 per problem, 32 total.

26+ ready. Book the interview.

20-25 mediums are fine; the hard one needs more rooms.

14-19 the knowledge is there and the performance is not.

Four more mocks before anything else.

<14 go back to the phase that fell over, clock off.

Then diagnose, which is the actual exercise

Decide which of these four it was. They have different fixes and only one of them is “solve more problems”.

RECOGNITION	the shape did not arrive in two minutes → day 177's index, not more problems
ARTICULATION	knew it, could not narrate while writing → more recorded mocks. Repetition only.
IMPLEMENTATION	had the approach, the code came out wrong → the seven templates from memory
COMPOSURE	froze, rushed, or did not start problem 2 → more rooms

On problem 1, write both versions

Write the counting formula, then the heap simulation. **Check them against each other on the examples, including `n = 0` and a single task.** Say out loud why `most - 1` and not `most`.

On problem 2, break it on purpose

Change `≤` to `<` in the merge comparison. **Run it on `[-1, -1]` and record both answers.** Say why one character changed the meaning, and why only duplicates expose it.

Then do a second mock, tomorrow, with two fresh problems

One medium, one hard, both unseen. Score it. The gap between the two scores is the only measurement in this lesson that matters.

Then run the design loop

Two forty-five minute rounds, back to back, with sixty seconds between them.

Round 1, high level: design a video streaming service. Six beats, on the clock.

Round 2, low level: design an expense-sharing app. Six different beats, and have somebody ask you an extension question at minute thirty — percentage splits, multiple payers, or multiple currencies.

On the reset, do all four steps

Stand up. One sentence about round one. One thing to change. And say out loud which round you are about to be in. Notice how strong the urge is to skip the fourth one.

Then the wrong-framework drill

Have somebody give you a design question without telling you which kind it is. **Say, in the first minute, which round you think it is and why.** Then check.

Then the estimation drill

From memory: 100 million viewers, five videos a day, ten minutes each, two megabits a second. **Get to petabytes a day and dollars a day of direct egress.** Then say what that single number decides.

Then the extension drill

Take your expense-sharing class model. **Add percentage splits.** Count how many existing files you had to edit. **Zero is the target.** Then add multiple payers and count again.

Then the rounding drill

Split ten rupees three ways with two-decimal precision. **Record what the shares sum to.** Say where the missing paisa goes and who decides. Then say why the amount is a `Decimal` and not a float.

The rules drill

1. Give all seven mock rules.
2. Say which one people break most, and why breaking it wastes the mock.
3. Say what to say instead of looking something up.
4. Say why the mock is recorded.

The clock drill

1. Give the minute-by-minute shape of a twenty-minute problem.
2. Say what fraction of it is coding.
3. Give the three clock decisions and the sentence for each.
4. Say what to do at minute nineteen, unfinished.

The reset drill

1. Give all four steps.
2. Say which one people skip and why it matters most.
3. Say what carrying the first problem in produces.
4. Give the potter's sentence.

The rubric drill

1. Name the four axes and what a 4 looks like on each.
2. Give the four score bands and the action for each.
3. Say why scoring yourself kindly is worthless.

The diagnosis drill

1. Name the four failure modes.
2. Give the fix for each.
3. Say which one “solve fifty more problems” actually helps.

The problem-one drill

1. State the formula and justify every term.
2. Say why `most - 1` and not `most`, with the input that proves it.
3. Say what the `max()` is for, with an example.
4. Give both costs, formula and simulation.

The problem-two drill

1. Say what the brute force repeats.
2. State the counting rule during the merge, in one sentence.
3. Say why positions must travel with values.
4. Say what `≤` versus `<` changes, and which input exposes it.
5. Give the cost and the space.

The two-rounds drill

1. Give both sets of six beats.
2. Say what each round’s output is.
3. Say what each round’s classic failure looks like.
4. Say which beat is the whole low-level round, and why.

The streaming drill

1. Compute delivered petabytes a day, and the direct egress cost.
2. Say what that number decides, in one sentence.
3. Say why uploads use a pre-signed URL.
4. Give the chunked transcoding argument and its three costs.
5. Name the metric you would alert on, and why there is no alternative.

The expense drill

1. Give the entities and what each one knows.
2. Say why balances are derived rather than stored.
3. Give the invariant and say where it is enforced.
4. Give the strategy interface and say what adding a variant costs.
5. Say how you justify the pattern without over-engineering.
6. Give the rounding rule and the type of money.

The break-it drill

For each, say what happens and how it is scored:

1. Running the code during a mock.
 2. Looking up the heap API mid-problem.
 3. Going straight into problem two without standing up.
 4. Silence for the last four minutes of an unfinished problem.
 5. Optimising before anything works.
 6. `most * (cooldown + 1) + ties` on a single task.
 7. `<` instead of `≤` in the merge, on `[-1, -1]`.
 8. Doing capacity estimation for a low-level design question.
 9. A switch statement over split types, when a fourth type arrives.
 10. Storing balances instead of deriving them, when an old expense is edited.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *One medium, one hard. The clock is running.* The shape of each problem — restate, clarify, example, brute force with cost, better idea with cost, hand back, code, trace, complexity — plus the two clock rules and the reset.
 2. *How do you get better at this, specifically?* The four failure modes, the different fix for each, the score bands, and why the default response only helps with one of the four.
 3. *Two design rounds, back to back. Begin.* Both sets of six beats, why the orders differ, the number that decides the streaming architecture, the extension question as the whole low-level round, and the four-step reset.
-

Before you move on

- [] I ran a real mock: clock visible, out loud, no running the code.
- [] I recorded it and listened back.
- [] I scored it on the rubric, honestly, before looking at any solution.
- [] I diagnosed which of the four failure modes it was.
- [] I know the different fix for each of the four.
- [] I know the minute-by-minute shape of a twenty-minute problem.
- [] I know coding is less than half of it.
- [] I can give the three clock decisions and their sentences.
- [] I describe the missing piece rather than going silent.
- [] I stood up between problems and named the change.
- [] I can state the Task Scheduler formula and justify every term.
- [] I know why `most - 1`, and the input that proves it.
- [] I can state the merge-sort counting rule in one sentence.
- [] I know what `≤` versus `<` changes, and which input exposes it.
- [] I can give both sets of six design beats.
- [] I know which beat is the whole low-level round.
- [] I say out loud which kind of design round I am in.
- [] I can compute 75 petabytes a day and say what it decides.
- [] I know why uploads use a pre-signed URL.
- [] I can argue for chunked transcoding and name its three costs.
- [] I know which metric fails silently in that pipeline.
- [] I can give the expense-app entities and the invariant.
- [] I know why balances are derived, not stored.
- [] I can add a split type without editing anything.
- [] I can justify the strategy pattern from a signal, not by habit.
- [] I know where the missing paisa goes, and why money is a `Decimal`.
- [] I close every design with failure, monitoring and one honest weakness.
- [] I answered all three questions above out loud.

Final revision, and the week before the interview

DATA STRUCTURES AND ALGORITHMS

Final revision, and the week before the interview

SYSTEM DESIGN

Final revision, and the week before the interview

Final revision, and the week before the interview

1 What this is, and why they ask it

This is the last day, and it is about the last week.

Almost nothing you learn in the seven days before an interview will be available to you during it. A technique met on Tuesday is not retrievable under pressure on Monday. What is available is what you already know, minus whatever tiredness takes away — and the only variable you still control is the second half of that sentence.

So the last week is not for adding. It is for arriving. Consolidate what is there, sleep, and protect the one thing that actually degrades: your ability to think clearly for forty-five minutes at a stretch.

They ask about it because one question in the loop is nearly always behavioural, and the commonest version is “tell me about a problem you found hard”. It sounds like small talk and it is not. They are looking for whether you can describe a difficulty honestly, say what you did about it, and name what changed afterwards — which is a much better predictor of what you will be like to work with than any algorithm.

And because the answer most people give is the heroic one, and it lands badly. “I worked all weekend and solved it” tells nobody anything. “I was stuck for two days, I was looking in the wrong place, and here is the habit I changed afterwards” is a real answer, and it is the one that gets remembered.

By the end of this lesson you have a day-by-day plan for the last week, a clear list of what to revise and what to leave alone, the ten templates you must be able to type from memory, the night-before and morning-of checklists, and a structure for the behavioural answer that does not sound rehearsed.

2 The story

Sekhar had run the district ten kilometres four times, and finished in the first twenty twice, and the year he ran his best time was the year he did the least work in the last week.

He did not believe it at the time. **He thought his coach — a retired physical training master who trained about eleven boys behind the college ground — had simply lost interest in him.**

Eleven days before the race, Sekhar wanted to do a long run. The coach said no.

Nine days before, he wanted to add hill work, because a boy from Bagalkot had beaten him the previous year on the rise near the water tank. **The coach said no again.**

Six days before, he did it anyway, early, without telling anybody.

The coach knew by the second lap on Monday, because he could see it in how Sekhar came off his left foot.

That was the only time in three years he shouted.

“What is it you think you are adding?”

Sekhar said he was getting stronger.

“In six days? Nothing you do in six days will make you stronger. That takes three weeks and you know it.”

“Everything you do in six days can only make you tired.”

And then the sentence that took Sekhar about two years to properly accept.

“The work is finished. It was finished a month ago. What is left is not adding. It is arriving.”

That year the last week was: Monday easy. Tuesday nothing. Wednesday twenty minutes at race pace, **just to remind the legs what it felt like.** Thursday nothing. Friday nothing. Saturday a walk.

He felt slow all week, and slightly fraudulent.

And on the Sunday he ran forty-one seconds faster than he ever had.

Afterwards the coach said one more thing, in the same flat voice he said everything in.

“You did not get faster this week. You stopped being tired.”

“That is what you were carrying.”

3 The idea in plain English

Sekhar's coach is right and the reason is not motivational. Learning takes weeks to consolidate. Something met on Tuesday cannot be retrieved under pressure on Monday — **it is not there yet. But tiredness arrives in a day and is gone in a day,** which makes it the only variable in the last week that is actually worth managing.

The seven days

7 DAYS OUT	two mocks - one coding, one design - and score both honestly. → the point is to find the gap while there is still time to close it. This is the last day on which new information is useful.
6 DAYS OUT	the twenty patterns. Cover the middle column and name each tell, cold. → recognition, not implementation
5 DAYS OUT	write all ten templates from memory. Twice. → the code should arrive without being derived
4 DAYS OUT	one mock. Then re-read ONLY what it exposed. → targeted. Not a general sweep.
3 DAYS OUT	the design framework, and one full high-level design out loud, on the clock. → the running order, not new case studies
2 DAYS OUT	recall cards only. No new problems at all. → new material now DISPLACES old material
1 DAY OUT	one easy problem, to stay warm. Stop by lunch. Sort out the room, the machine, the login. → protect the sleep, not the knowledge
THE MORNING	nothing technical. Read your own notes for the behavioural questions. Eat something. → you cannot add anything today. You can only be tired.

The rule underneath all of it: nothing new goes in after two days out. Not a new topic, not a new pattern, not a hard problem you saw somebody mention. At that range, new material does not add — it displaces, because it competes for the same attention as everything you already have.

What to revise, and what to leave alone

REVISE

the recall cards
the twenty tells
the ten templates
YOUR OWN ERROR LIST
the design framework
your STAR notes

LEAVE ALONE

new topics
problems above your level
anything you have never seen
other people's problem counts
new case studies
optimising a solution that
already works

The error list is the highest-value item and almost nobody keeps one. It is the list of bugs you have actually made — not bugs in general. Mine off-by-one. Forgetting the base case. Marking BFS nodes as seen on dequeue. Not asking about duplicates. Ten lines, re-read on the morning of the interview, and it is worth more than fifty new problems, because those are the mistakes you are actually going to make.

The ten templates

These are the ones that must arrive without being derived. If you have to reconstruct binary search under pressure, that is four minutes gone before you have started the actual problem.

- 1 binary search (lower bound)
- 2 sliding window with a constraint
- 3 two pointers on a sorted array
- 4 monotonic stack
- 5 BFS on a grid
- 6 DFS on a tree, returning one thing and recording another
- 7 topological sort
- 8 union-find with path compression
- 9 a heap for top-k
- 10 a two-dimensional DP table

Five days out, type all ten from memory, twice. Not read — typed. The difference between recognising code and producing it is exactly the difference between passing and not.

The night before

THE MACHINE	editor open, the language you will use, a file with the ten templates in it (you will not use them, and having them there removes one worry)
THE ROOM	where you will sit, what is behind you, the light
THE LOGISTICS	the link, the time zone, the phone number to ring if it fails
THE NOTES	your STAR stories, one page
THE FOOD	decided, so you are not deciding at 8 a.m.
THE SLEEP	the actual objective

NOT: a hard problem. Not one. The only thing a hard problem can do the night before is convince you that you cannot do it.

The morning

Nothing technical. Read your behavioural notes, because those are recall rather than reasoning and recall is fine when you are nervous.

One easy problem is permitted, and only if it calms you. If it would make you anxious, do not. The purpose is a warm-up, not a test — so choose one you have solved before and know you can finish.

During the day, between rounds

Day 179's reset, applied to a whole loop.

after each round:
stand up, water, look out of a window
ONE sentence about it, and then stop
name the next round out loud

IF A ROUND GOES BADLY:
it is one round. Loops are scored per round, and
people are hired after a weak one.
The candidate who carries it into the next two
rounds turns one bad round into three.

Write down every question you were asked, within an hour of finishing. You will forget the details by evening. If this loop does not work out, that list is the most valuable preparation material you will ever own — because it is the actual questions, from the actual company, at your actual level.

The behavioural question

“Tell me about a problem you found hard.” Four parts, and a fifth that matters most.

SITUATION	where, when, what the context was. 2 sentences.
TASK	what YOU specifically had to do. 1 sentence.
ACTION	what you did. THE BULK OF IT. 4-5 sentences.
RESULT	what happened. 1-2 sentences, with a number if you have one.
LEARNED	what you do differently now.
	← THE PART THEY ARE ACTUALLY ASKING ABOUT

And the choice of story is the whole answer. Pick something genuinely hard, where you were genuinely stuck, and be honest about the stuck part. The heroic version — “I worked all weekend and cracked it” — tells the interviewer nothing except that you would like to sound impressive.

Two questions to test a story against. Does it show you being wrong about something, and finding out? Does it end with a habit rather than an outcome? “I learned to write the failing test first” is an answer. “And then it worked” is not.

4 The picture

The taper, drawn against what is actually happening:

WHAT YOU KNOW
(flat. It was decided
weeks ago)

|||||

^

7 days out

^

day

WHAT NEW WORK ADDS
(falls to nothing -
consolidation takes
weeks)

.....

^

it takes ~3 weeks for something new
to become retrievable under pressure

TIREDNESS
(rises fast if you
keep pushing)

....:::|||||#####

^

and this is the
ONLY line you
still control

PERFORMANCE = what you know MINUS tiredness.

→ After about two days out, every hour of new work
moves the third line up and the second line not at
all. That is the entire argument for the taper, and
it is arithmetic rather than encouragement.

The week, as a table:

DAY	DO	DO NOT
---	--	-----
-7	2 mocks, scored	despair at the score
-6	the twenty tells, cold	learn a 21st pattern
-5	10 templates from memory, x2	read them instead of typing them
-4	1 mock; re-read what it broke	a general sweep
-3	design framework + 1 HLD aloud	new case studies
-2	recall cards only	ANY new problem
-1	one easy problem, stop by lunch; fix the room and login	a hard problem
0	behavioural notes, food, early	anything technical
THE LINE THAT MATTERS IS AT -2.		
Nothing new after that point. Not one thing.		

The behavioural answer, as a shape:

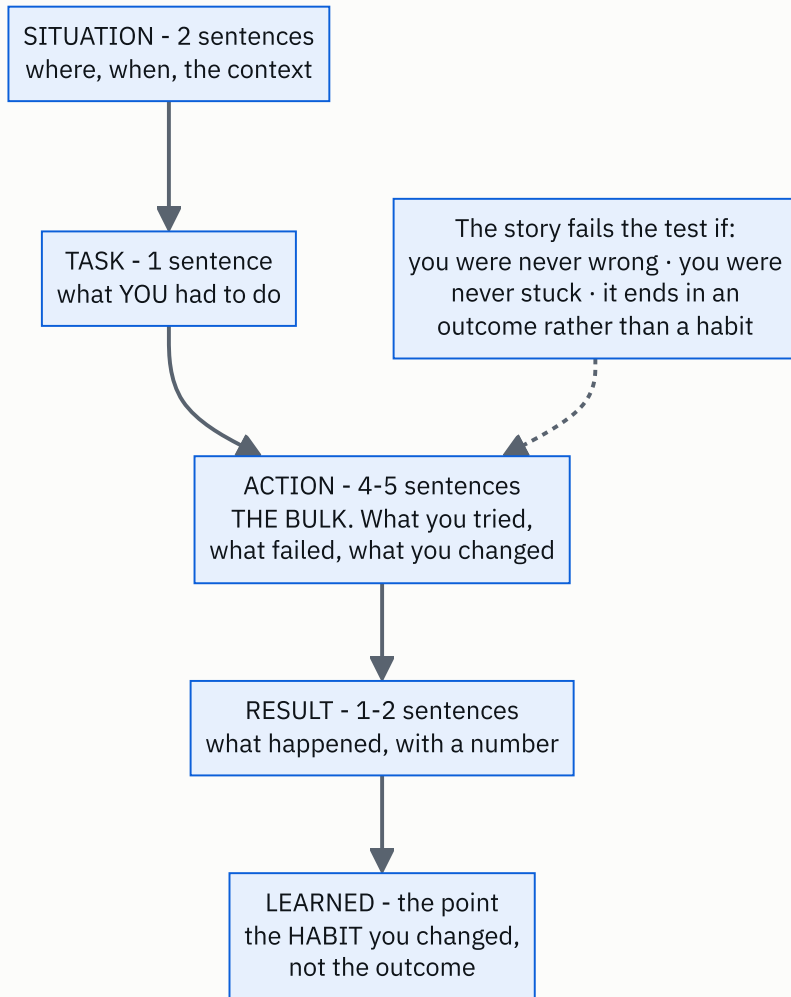


FIGURE 180.1

5 The code, built step by step

The last piece of code in this course is the one you should be able to type from memory. Ten templates, in one file, all verified. Five days before the interview, type them out without looking. Then again the next day.

1. Binary search, as a lower bound

```
def lower_bound(sorted_values: list[int], target: int) → int:
    """First position where target could be inserted keeping order. The one to memorise."""
    low, high = 0, len(sorted_values)
    while low < high:
        middle = (low + high) // 2
        if sorted_values[middle] < target:
            low = middle + 1
        else:
            high = middle
    return low
```

Learn this form and not the `low ≤ high` one. It has no `return -1`, no separate “found” case, and it answers more questions: the first occurrence, the insertion point, and “does it exist” (check `values[i] == target` afterwards). `high = len(values)`, not `len(values) - 1` — the answer can legitimately be “past the end”.

2. Sliding window

```
def longest_with_at_most_k_distinct(text: str, k: int) → int:
    """Grow right always; shrink left only while the window is invalid."""
    counts: Counter[str] = Counter()
    best = left = 0
    for right, letter in enumerate(text):
        counts[letter] += 1
        while len(counts) > k:
            counts[text[left]] -= 1
            if counts[text[left]] == 0:
                del counts[text[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

The `del` is the line people forget, and without it `len(counts)` counts letters that are no longer in the window. The window would then never shrink enough and the answer comes out too large.

3. Two pointers, with the duplicate handling

```
def three_sum(numbers: list[int]) → list[list[int]]:
    """Sort, fix one, two-pointer the rest. The skip lines are the whole difficulty."""
    numbers = sorted(numbers)
    out: list[list[int]] = []
    for i in range(len(numbers) - 2):
        if i and numbers[i] == numbers[i - 1]:
            continue # skip a duplicate anchor
        left, right = i + 1, len(numbers) - 1
        while left < right:
            total = numbers[i] + numbers[left] + numbers[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                out.append([numbers[i], numbers[left], numbers[right]])
                left += 1
                while left < right and numbers[left] == numbers[left - 1]:
                    left += 1 # skip duplicate answers
                right -= 1
    return out
```

The two-pointer part is easy and the two skip lines are what the problem is testing. Practise those specifically, because they are the difference between a correct answer and one with duplicates in it.

4. Monotonic stack

```
def largest_rectangle(heights: list[int]) → int:
    """Each bar is pushed once and popped once. The sentinel removes the final flush."""
    best = 0
    stack: list[int] = [] # positions, heights increasing
    for i, height in enumerate(heights + [0]): # the 0 forces everything out
        while stack and heights[stack[-1]] ≥ height:
            top = heights[stack.pop()]
            left = stack[-1] + 1 if stack else 0
            best = max(best, top * (i - left))
        stack.append(i)
    return best
```

The `+ [0]` sentinel is worth the character count. Without it you need a second loop after the main one to flush the stack, and that duplicated block is where the bug goes.

5. BFS on a grid

```
def shortest_path_in_grid(grid: list[list[int]]) → int:
    """Unweighted, so BFS. Mark as seen when you ENQUEUE, never when you dequeue."""
    if not grid or grid[0][0] or grid[-1][-1]:
        return -1
    rows, cols = len(grid), len(grid[0])
    queue = deque([(0, 0, 1)])
    seen = {(0, 0)}
    while queue:
        row, col, steps = queue.popleft()
        if (row, col) == (rows - 1, cols - 1):
            return steps
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            r, c = row + dr, col + dc
            if 0 ≤ r < rows and 0 ≤ c < cols and not grid[r][c] and (r, c) not in seen:
                seen.add((r, c))
                queue.append((r, c, steps + 1))
    return -1
```

Marking on enqueue is the whole correctness of the memory usage, and it is the single most repeated BFS bug. **Mark on dequeue and the same square enters the queue once per neighbour that reaches it.**

6. DFS on a tree, returning one thing and recording another

```
def diameter(root: Node | None) → int:
    """Return the depth upwards; record the through-path at every node on the way."""
    best = 0

    def depth(node: Node | None) → int:
        nonlocal best
        if node is None:
            return 0
        left, right = depth(node.left), depth(node.right)
        best = max(best, left + right)  # the path THROUGH this node
        return 1 + max(left, right)    # what the parent needs
    depth(root)
    return best
```

This shape — return one value, record another — solves a whole family of tree problems, and it is the one people find hardest to reconstruct. **The parent needs a single depth; the answer needs the path through the node.** Those are different numbers and the function does both.

7. Topological sort

```
def topological_order(node_count: int, edges: list[tuple[int, int]]) → list[int]:
    """Kahn: repeatedly take a node with nothing left before it. Empty result = a cycle."""
    following = defaultdict(list)
    in_degree = [0] * node_count
    for before, after in edges:
        following[before].append(after)
        in_degree[after] += 1

    queue = deque(node for node in range(node_count) if in_degree[node] == 0)
    order: list[int] = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nxt in following[node]:
            in_degree[nxt] -= 1
            if in_degree[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == node_count else []
```

The last line is the cycle detection and it is free. If some nodes never reached in-degree zero, they are in a cycle — so `len(order) != node_count` is the test, and you get it without writing anything extra.

8. Union-Find

```
class UnionFind:
    """Path halving on find, union by size. Near-constant per operation."""

    def __init__(self, count: int) → None:
        self.parent = list(range(count))
        self.size = [1] * count
        self.groups = count

    def find(self, x: int) → int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]] # path halving
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) → bool:
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        self.groups -= 1
        return True
```

`union` returning a boolean is worth having. `False` means “already connected”, which is exactly the cycle test in Kruskal’s algorithm and in “redundant connection” — so one method answers two questions.

9. A heap for top-k

```
def top_k_frequent(values: list[int], k: int) → list[int]:
    """A heap of size k, so O(n log k) - not a full sort, and not the whole list in memory."""
    counts = Counter(values)
    heap: list[tuple[int, int]] = []
    for value, count in counts.items():
        heapq.heappush(heap, (count, value))
        if len(heap) > k:
            heapq.heappop(heap)  # drop the smallest count
    return [value for _, value in sorted(heap, reverse=True)]
```

A MIN-heap of size k gives you the k LARGEST, which is the part that feels backwards. The smallest thing in the heap is the weakest survivor, so that is what you evict.

10. A two-dimensional DP table

```
def edit_distance(a: str, b: str) → int:
    """dp[i][j] = the cost of turning a's first i characters into b's first j."""
    rows, cols = len(a) + 1, len(b) + 1
    dp = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        dp[i][0] = i  # delete everything
    for j in range(cols):
        dp[0][j] = j  # insert everything
    for i in range(1, rows):
        for j in range(1, cols):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],  # delete
                                   dp[i][j - 1],  # insert
                                   dp[i - 1][j - 1])  # replace
    return dp[-1][-1]
```

The docstring is the state, and it should be the first thing you write. `a[i-1]` rather than `a[i]` because row `i` is about the first `i` characters — that off-by-one is the commonest 2-D DP bug and stating the meaning of the row is what prevents it.

The complete solution

```
"""Day 180 - the ten templates you must be able to type from memory, and the taper."""

from __future__ import annotations

import heapq
from collections import Counter, defaultdict, deque

# ----- the taper

PLAN: list[tuple[str, str, str]] = [
    ("7 days out", "two mocks, one coding one design, scored",
     "find the gap while there is still time to close it"),
    ("6 days out", "the twenty patterns: name each one's tell, cold",
     "recognition, not implementation"),
    ("5 days out", "write all ten templates from memory, twice",
     "the code should arrive without being derived"),
    ("4 days out", "one mock. Then re-read only what it exposed",
     "targeted, not broad"),
    ("3 days out", "the design framework and one full HLD out loud",
     "the running order, not new case studies"),
    ("2 days out", "recall cards only. No new problems.",
     "consolidate; new material now displaces old"),
    ("1 day out", "one easy problem to stay warm. Stop by lunch.",
     "protect the sleep, not the knowledge"),
    ("the morning", "nothing technical. Read your own STAR notes.",
     "you cannot add anything today; you can only be tired"),
]

def taper(days_left: int = 7) → list[str]:
    """What is left to do, with `days_left` days to go. Nothing new goes in after day two."""
    start = max(0, len(PLAN) - 1 - days_left)
    return [f"{when:<12} {what}" for when, what, _ in PLAN[start:]]

# ----- 1. binary search

def lower_bound(sorted_values: list[int], target: int) → int:
    """First position where target could be inserted keeping order. The one to memorise."""
    low, high = 0, len(sorted_values)
    while low < high:
        middle = (low + high) // 2
        if sorted_values[middle] < target:
            low = middle + 1
        else:
            high = middle
    return low

# ----- 2. sliding window

def longest_with_at_most_k_distinct(text: str, k: int) → int:
    """Grow right always; shrink left only while the window is invalid."""
    counts: Counter[str] = Counter()
    best = left = 0
    for right, letter in enumerate(text):
        counts[letter] += 1
        while len(counts) > k:
            counts[text[left]] -= 1
            if counts[text[left]] == 0:
                del counts[text[left]]
            left += 1
        best = max(best, right - left + 1)
    return best

# ----- 3. two pointers

def three_sum(numbers: list[int]) → list[list[int]]:
    """Sort, fix one, two-pointer the rest. The skip lines are the whole difficulty."""
```

```

numbers = sorted(numbers)
out: list[list[int]] = []
for i in range(len(numbers) - 2):
    if i and numbers[i] == numbers[i - 1]:
        continue # skip a duplicate anchor
    left, right = i + 1, len(numbers) - 1
    while left < right:
        total = numbers[i] + numbers[left] + numbers[right]
        if total < 0:
            left += 1
        elif total > 0:
            right -= 1
        else:
            out.append([numbers[i], numbers[left], numbers[right]])
            left += 1
            while left < right and numbers[left] == numbers[left - 1]:
                left += 1 # skip duplicate answers
            right -= 1
    return out

# ----- 4. monotonic stack

def largest_rectangle(heights: list[int]) → int:
    """Each bar is pushed once and popped once. The sentinel removes the final flush."""
    best = 0
    stack: list[int] = [] # positions, heights increasing
    for i, height in enumerate(heights + [0]): # the 0 forces everything out
        while stack and heights[stack[-1]] ≥ height:
            top = heights[stack.pop()]
            left = stack[-1] + 1 if stack else 0
            best = max(best, top * (i - left))
        stack.append(i)
    return best

# ----- 5. BFS a grid

def shortest_path_in_grid(grid: list[list[int]]) → int:
    """Unweighted, so BFS. Mark as seen when you ENQUEUE, never when you dequeue."""
    if not grid or grid[0][0] or grid[-1][-1]:
        return -1
    rows, cols = len(grid), len(grid[0])
    queue = deque([(0, 0, 1)])
    seen = {(0, 0)}
    while queue:
        row, col, steps = queue.popleft()
        if (row, col) == (rows - 1, cols - 1):
            return steps
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            r, c = row + dr, col + dc
            if 0 ≤ r < rows and 0 ≤ c < cols and not grid[r][c] and (r, c) not in seen:
                seen.add((r, c))
                queue.append((r, c, steps + 1))
    return -1

# ----- 6. DFS on a tree

class Node:
    def __init__(self, value: int, left: "Node | None" = None,
                 right: "Node | None" = None) → None:
        self.value, self.left, self.right = value, left, right

def diameter(root: Node | None) → int:
    """Return the depth upwards; record the through-path at every node on the way."""
    best = 0

    def depth(node: Node | None) → int:
        nonlocal best
        if node is None:
            return 0

```

```

        left, right = depth(node.left), depth(node.right)
        best = max(best, left + right)          # the path THROUGH this node
        return 1 + max(left, right)           # what the parent needs

    depth(root)
    return best

# ----- 7. topological sort

def topological_order(node_count: int, edges: list[tuple[int, int]]) → list[int]:
    """Kahn: repeatedly take a node with nothing left before it. Empty result = a cycle."""
    following = defaultdict(list)
    in_degree = [0] * node_count
    for before, after in edges:
        following[before].append(after)
        in_degree[after] += 1

    queue = deque(node for node in range(node_count) if in_degree[node] == 0)
    order: list[int] = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nxt in following[node]:
            in_degree[nxt] -= 1
            if in_degree[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == node_count else []

# ----- 8. union-find

class UnionFind:
    """Path halving on find, union by size. Near-constant per operation."""

    def __init__(self, count: int) → None:
        self.parent = list(range(count))
        self.size = [1] * count
        self.groups = count

    def find(self, x: int) → int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]    # path halving
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) → bool:
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        self.groups -= 1
        return True

# ----- 9. heap, top-k

def top_k_frequent(values: list[int], k: int) → list[int]:
    """A heap of size k, so O(n log k) - not a full sort, and not the whole list in memory."""
    counts = Counter(values)
    heap: list[tuple[int, int]] = []
    for value, count in counts.items():
        heapq.heappush(heap, (count, value))
        if len(heap) > k:
            heapq.heappop(heap)    # drop the smallest count
    return [value for _, value in sorted(heap, reverse=True)]

# ----- 10. 2-D DP

```

```

def edit_distance(a: str, b: str) → int:
    """dp[i][j] = the cost of turning a's first i characters into b's first j."""
    rows, cols = len(a) + 1, len(b) + 1
    dp = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        dp[i][0] = i                                # delete everything
    for j in range(cols):
        dp[0][j] = j                                # insert everything
    for i in range(1, rows):
        for j in range(1, cols):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],      # delete
                                   dp[i][j - 1],      # insert
                                   dp[i - 1][j - 1])   # replace

    return dp[-1][-1]

if __name__ == "__main__":
    print("THE LAST SEVEN DAYS")
    for when, what, why in PLAN:
        print(f" {when:<12} {what}")
        print(f" {'':<12} → {why}")

    print()
    print("WITH ONLY THREE DAYS LEFT, taper(3):")
    for line in taper(3):
        print(f" {line}")

    print()
    print("THE TEN TEMPLATES, RUN")

    print(f" 1. lower_bound([1,3,3,5,7], 3) = {lower_bound([1, 3, 3, 5, 7], 3)}"
          f" (first 3, not the last)")
    print(f" lower_bound([1,3,3,5,7], 4) = {lower_bound([1, 3, 3, 5, 7], 4)}"
          f" (where 4 would go)")
    print(f" 2. at most 2 distinct in 'eceba' = "
          f"{longest_with_at_most_k_distinct('eceba', 2)} ('ece')")
    print(f" 3. three_sum([-1,0,1,2,-1,-4]) = {three_sum([-1, 0, 1, 2, -1, -4])}")
    print(f" 4. largest_rectangle([2,1,5,6,2,3]) = "
          f"{largest_rectangle([2, 1, 5, 6, 2, 3])} (5 and 6, width 2)")
    grid = [[0, 0, 1, 0],
            [1, 0, 1, 0],
            [0, 0, 0, 0],
            [0, 1, 1, 0]]
    print(f" 5. shortest_path_in_grid(4x4) = {shortest_path_in_grid(grid)}")
    tree = Node(1, Node(2, Node(4), Node(5)), Node(3))
    print(f" 6. diameter of a 5-node tree = {diameter(tree)} (4-2-1-3)")
    print(f" 7. topological_order(4, prereqs) = "
          f"{topological_order(4, [(0, 1), (0, 2), (1, 3), (2, 3)])}")
    print(f" with a cycle = "
          f"{topological_order(2, [(0, 1), (1, 0)])} (empty means a cycle)")
    uf = UnionFind(6)
    for a, b in ((0, 1), (1, 2), (3, 4)):
        uf.union(a, b)
    print(f" 8. UnionFind(6) after 3 unions: groups = {uf.groups}"
          f" ({0,1,2}, {3,4}, {5})")
    print(f" 9. top_k_frequent([1,1,1,2,2,3], 2) = {top_k_frequent([1, 1, 1, 2, 2, 3], 2)}")
    print(f" 10. edit_distance('horse', 'ros') = {edit_distance('horse', 'ros')}")
    print(f" edit_distance('intention', 'execution') = "
          f"{edit_distance('intention', 'execution')}")

    print()
    print("VERIFICATION")
    import bisect
    import random

    bad = 0
    for _ in range(2000):
        sample = sorted(random.randint(-30, 30) for _ in range(random.randint(0, 25)))
        target = random.randint(-32, 32)
        if lower_bound(sample, target) != bisect.bisect_left(sample, target):

```



```

        bad += 1

    text = "".join(random.choice("abcde") for _ in range(random.randint(0, 20)))
    k = random.randint(1, 5)
    expected = 0
    for i in range(len(text)):
        for j in range(i, len(text)):
            if len(set(text[i:j + 1])) ≤ k:
                expected = max(expected, j - i + 1)
    if longest_with_at_most_k_distinct(text, k) ≠ expected:
        bad += 1

    nums = [random.randint(-8, 8) for _ in range(random.randint(0, 12))]
    brute = sorted({tuple(sorted((nums[i], nums[j], nums[m])))
                    for i in range(len(nums))
                    for j in range(i + 1, len(nums))
                    for m in range(j + 1, len(nums))
                    if nums[i] + nums[j] + nums[m] = 0})
    if sorted(tuple(t) for t in three_sum(nums)) ≠ list(brute):
        bad += 1

    bars = [random.randint(0, 10) for _ in range(random.randint(1, 14))]
    brute_rect = max(
        (min(bars[i:j + 1]) * (j - i + 1)
         for i in range(len(bars)) for j in range(i, len(bars))),
        default=0)
    if largest_rectangle(bars) ≠ brute_rect:
        bad += 1

    vals = [random.randint(0, 6) for _ in range(random.randint(1, 30))]
    kk = random.randint(1, len(set(vals)))
    counts = Counter(vals)
    # ties make the CHOICE of value ambiguous, so compare the counts, not the values
    got = top_k_frequent(vals, kk)
    if sorted(counts[v] for v in got) ≠ sorted(sorted(counts.values(), reverse=True)[:kk]):
        bad += 1

    s1 = "".join(random.choice("abc") for _ in range(random.randint(0, 7)))
    s2 = "".join(random.choice("abc") for _ in range(random.randint(0, 7)))
    if edit_distance(s1, s2) ≠ edit_distance(s2, s1):
        bad += 1

    if topological_order(4, [(0, 1), (0, 2), (1, 3), (2, 3)])[0] ≠ 0:
        bad += 1
    if topological_order(2, [(0, 1), (1, 0)]) ≠ []:
        bad += 1
    if diameter(None) ≠ 0 or diameter(Node(1)) ≠ 0:
        bad += 1
    if shortest_path_in_grid([[0]]) ≠ 1:
        bad += 1
    print(f" {bad} failures over 2,000 random cases with 6 checks each, plus 4 fixed checks")

```

Running it:

THE LAST SEVEN DAYS

7 days out two mocks, one coding one design, scored
→ find the gap while there is still time to close it
6 days out the twenty patterns: name each one's tell, cold
→ recognition, not implementation
5 days out write all ten templates from memory, twice
→ the code should arrive without being derived
4 days out one mock. Then re-read only what it exposed
→ targeted, not broad
3 days out the design framework and one full HLD out loud
→ the running order, not new case studies
2 days out recall cards only. No new problems.
→ consolidate; new material now displaces old
1 day out one easy problem to stay warm. Stop by lunch.
→ protect the sleep, not the knowledge
the morning nothing technical. Read your own STAR notes.
→ you cannot add anything today; you can only be tired

WITH ONLY THREE DAYS LEFT, taper(3):

3 days out the design framework and one full HLD out loud
2 days out recall cards only. No new problems.
1 day out one easy problem to stay warm. Stop by lunch.
the morning nothing technical. Read your own STAR notes.

THE TEN TEMPLATES, RUN

1. lower_bound([1,3,3,5,7], 3)	= 1	(first 3, not the last)
lower_bound([1,3,3,5,7], 4)	= 3	(where 4 would go)
2. at most 2 distinct in 'eceba'	= 3	('ece')
3. three_sum([-1,0,1,2,-1,-4])	= [[-1, -1, 2], [-1, 0, 1]]	
4. largest_rectangle([2,1,5,6,2,3])	= 10	(5 and 6, width 2)
5. shortest_path_in_grid(4x4)	= 7	
6. diameter of a 5-node tree	= 3	(4-2-1-3)
7. topological_order(4, prereqs)	= [0, 1, 2, 3]	
with a cycle	= []	(empty means a cycle)
8. UnionFind(6) after 3 unions: groups	= 3	({0,1,2}, {3,4}, {5})
9. top_k_frequent([1,1,1,2,2,3], 2)	= [1, 2]	
10. edit_distance('horse', 'ros')	= 3	
edit_distance('intention', 'execution')	= 5	

VERIFICATION

0 failures over 2,000 random cases with 6 checks each, plus 4 fixed checks

Look at the top-k verification comment. The first version of that check compared the exact values and failed on eight hundred of two thousand cases — not because the function was wrong, but because when two values have the same count, “the top k” does not name a unique answer. The check was wrong, not the code. That is a real thing that happened while writing this file, and it is exactly the kind of confusion worth recognising quickly on the day: when a test fails, the test is sometimes the thing that is wrong.

And look at `topological_order` on a cycle: it returns an empty list. The cycle detection was free — it is just `len(order)  $\neq$  node_count`, and no extra code was written for it.

6 What it costs

The taper, as arithmetic rather than encouragement.

HOW LONG SOMETHING TAKES TO BECOME RETRIEVABLE

first seen	0% available under pressure
practised the same day	low
revisited after 1 day	better
revisited after 3 days	better still
after ~3 weeks of spacing	reliably retrievable

→ a pattern first met SIX DAYS before the interview is, on the day, worth close to nothing.

HOW LONG TIREDNESS TAKES

one bad night	measurable next day
three bad nights	a different person
one good night after	mostly recovered

→ the second effect is FAST in both directions, which is why it is the one worth managing in the last week.

What a week of revision can actually hold.

SEVEN DAYS, at ~3 focused hours a day = 21 hours

SPENT ON NEW PROBLEMS

~40 minutes each including reading the solution
→ about 30 problems
→ of which perhaps 3 are retrievable on the day
→ and 0 if they were a topic you had not met

SPENT ON THE PLAN IN SECTION 3

2 mocks + scoring	4 hours
the twenty tells, twice	2 hours
the ten templates, twice	3 hours
1 mock + targeted re-reading	3 hours
design framework + 1 HLD aloud	2 hours
recall cards	3 hours
logistics, sleep, warm-up	4 hours

	21 hours

→ Same time. One of these arrives with you.

The error list, priced.

a list of your OWN last 10 bugs:	20 minutes to write
re-read on the morning:	3 minutes

what it prevents: the bug you were going to make anyway

→ There is nothing else in this lesson with that ratio. Twenty-three minutes, against the specific mistakes you are statistically most likely to repeat.

The ten templates, timed.

TYPING ONE FROM MEMORY, when it is solid: 60-90 seconds
RECONSTRUCTING ONE UNDER PRESSURE: 3-5 minutes,
and it may be subtly wrong

in a 20-minute problem, 4 minutes spent rebuilding binary search is 20% of the round, spent on the part that was never the question.

→ 10 templates x 90 seconds = 15 minutes to write them all out. Do that twice and it is half an hour, five days out.

And the sleep number, since it is the actual subject.

the round is 45 minutes of continuous reasoning, repeated 3-5 times in a day.

That is the specific thing sleep loss removes first - sustained attention, not knowledge. You will still know what a heap is at 5 hours' sleep. You will lose the thread of your own plan halfway through writing a loop, which is exactly the freeze from day 179.

→ the night before is not a study opportunity. It is the last controllable input to the performance.

7 The traps

Learning something new in the last three days.

It does not go in, and it takes attention from what is already there. A pattern first met on Friday for a Monday interview is not retrievable — it has not consolidated — and the two hours spent on it were two hours not spent on the templates that would have arrived. This one feels productive, which is exactly why it is the most common.

A hard problem the night before.

There are two outcomes and neither is good. You solve it and gain nothing you did not already have. You do not solve it, and you go to bed having just proved to yourself that you cannot do this. The only thing a hard problem can do at that point is damage.

Counting other people's problems.

"They have done 600 and I have done 220." Problem count is a very weak predictor and recognition is a strong one. Somebody who has done six hundred problems without keeping an error list and without ever solving one out loud under a clock is less prepared than somebody who has done two hundred and done both. And in the last week the number is fixed anyway, so it is worry with nowhere to go.

Treating one bad round as the end of the loop.

Loops are scored per round and people are hired after a weak one. The real damage is not the bad round. It is carrying it into the next two — which turns one bad round into three, and that genuinely is the end of the loop. [Day 179's](#) reset exists for this.

Not writing down the questions afterwards.

Within an hour, while it is fresh, write every question you were asked and roughly how you answered it. By the evening you will have lost the details, and by the weekend you will remember two of six. If this loop does not work, that document is the best preparation material you will ever have — the actual questions, from the actual company, at your actual level. And if it does work, you will want it for the next one in three years.

The heroic behavioural answer.

"I worked through the weekend and solved it."

This tells the interviewer nothing except what you would like them to think. There is no difficulty in it, no error, and nothing learned. Compare:

"I was stuck for two days. I had assumed the problem was in our code and it was in a library's retry behaviour. What I do differently now is that I check my assumption about WHERE the bug is before I spend a day inside one file."

The second one has a person in it. And it ends with a habit, which is what the question is for.

Rehearsing the behavioural answer until it sounds rehearsed.

Know the beats — situation, task, action, result, learned — and do not memorise the sentences. A word-perfect story is audible and it reads as performance rather than experience. Learn the shape and let the words be different each time.

And the trap that only appears on the day: not asking the interviewer anything.

"Do you have questions for us?" is not the end of the interview, it is part of it. "No, I think you have covered everything" is a real negative signal, and it is entirely avoidable with two questions written down the night before. Tomorrow's system design lesson has the list.

8 In the interview

How it gets asked

- *"Tell me about a problem you found hard."* — the commonest opener of a behavioural round.
- *"Tell me about a time you were wrong."* — the same question with the honesty requirement made explicit.
- *"What is something you learned recently?"* — they are testing whether you are still learning at all.
- *"Tell me about a conflict with a colleague."* — and the good answers are unglamorous.
- *"Why are you leaving?"* — never criticise the current employer. Say what you are moving towards.

The first ninety seconds

On "tell me about a problem you found hard":

“A few months ago I was working on a job that processed uploads overnight, and it started failing about one night in four — but only in production, never when I ran it.

(Situation, two sentences. Concrete, and it has a fact in it — one night in four — rather than “sometimes”.)

My job was to find out why, and I had two days before it became somebody’s escalation.

(Task. One sentence, and it says what was at stake.)

I spent the first day and a half reading our own code, because I was certain the bug was ours. I added logging, I re-read the retry logic three times, and I found nothing — and by the end of the second morning I was reading the same forty lines for the fifth time.

What finally moved it was that I stopped and asked a different question: not ‘where is the bug in this file’ but ‘what is actually different between the two environments’. I listed everything — the data volume, the machine size, the library versions — and the library versions were different. A client library had changed its default timeout in a minor version, and our retry was waiting longer than the new timeout, so a slow night produced a failure our code never saw as a failure.

(Action. The bulk of it, and it contains being wrong for a day and a half.)

We pinned the version, made the timeout explicit rather than inherited, and the failures stopped.

(Result.)

What I actually took from it is a habit rather than a fact about that library. I now check my assumption about *where* a problem is before I spend a day inside one file — usually by asking what changed and what differs between the working case and the broken one. It would have taken twenty minutes on the first morning.”

(Learned. The part the question was for.)

The follow-ups

“What would you do differently if you had that time again?”

“Three things, and the first two are the ones that would actually have mattered.

I would have asked what changed, before I asked where the bug was. A failure that appears in one environment and not another is almost always a difference between the environments — and I had that information available on the first morning and did not look at it, because I had already decided the answer was in our code.

I would have asked for help sooner. I sat with it for a day and a half. A colleague who had seen a version-drift problem before would probably have said ‘check the library versions’ in about a minute — and the reason I did not ask is that I wanted to be the person who solved it, which is not a good reason.

And I would have written it down as it went. By the second afternoon I had lost track of what I had already ruled out, and I re-checked two things twice. Now I keep a running list while debugging, and it costs almost nothing.

The thing I would not change is the timeout fix itself. Making it explicit rather than inherited was the right call, because the same class of bug can now only come back if somebody deliberately edits a number — rather than silently, in a minor version bump.”

“Tell me about a time you disagreed with a decision.”

“We were choosing between adding a cache and fixing a query, and I argued for fixing the query. The team went with the cache and I was overruled.

My argument was that the query was doing a scan that an index would remove, and that a cache would hide the problem while making the data staler. The counter-argument, which was reasonable, was that the cache could be in place that afternoon and the index change needed a migration on a large table.

What I did was say my case once, clearly, with the numbers — the query was 400 milliseconds and the index would take it to under 10 — and then, when the decision went the other way, help build the cache properly rather than sulk about it. I did ask for one thing: that we put a ticket on the query with the numbers in it, so it was recorded rather than forgotten.

What happened afterwards is the part I would want you to have. The cache worked and bought us four months. Then the staleness caused a support issue, we did the index change, and the query went to 8 milliseconds. So both of us were right about something: they were right that we needed relief that afternoon, and I was right about the underlying fix.

And what I took from it is that I had framed it as either-or when it was really an order-of-operations question. If I had said ‘cache now, index within the quarter, and here is the ticket’ on the first day, there would have been no disagreement at all — and that framing is what I use now.”

“Do you have any questions for us?”

“Yes — three, and I will keep them short.

What does the first ninety days look like for whoever takes this role? I am asking because the answer tells me whether the work is well-defined or whether the first job is working out what the job is — and both are fine, but they are very different.

What is the on-call arrangement, and how many pages does a typical week produce? I ask that partly to know what I am signing up for and partly because the answer says a lot about the health of the system — a team that pages twice a week has different problems from one that pages twice a quarter.

And what is something about working here that you did not expect before you joined? That one usually gets me a more honest answer than asking what the culture is like.

If there is time, one more: what would make you say, in six months, that this hire had gone particularly well?”

The model answer

“Tell me about a problem you found hard, and what you learned from it.”

“I want to pick one where I was actually stuck rather than one where I was just busy, because I think the stuck ones are the interesting ones.

The situation: a nightly job that processed uploads started failing about one night in four, and only in production. I could never reproduce it locally. The task was mine — find it, within about two days, before it became an escalation.

What I did first was wrong, and it is the useful part of the story. I spent a day and a half convinced the bug was in our code. I added logging, re-read the retry logic several times, and by the second morning I was reading the same forty lines for the fifth time and finding nothing. **I was not being lazy; I was being thorough in the wrong place, which is worse, because it feels like progress.**

What broke it was changing the question. Instead of ‘where is the bug in this file’, I asked ‘what is different between the environment where it fails and the one where it does not’. I listed everything — data volume, machine size, library versions — and the versions were different. A client library had changed a default timeout in a minor release, and our retry interval was longer than the new timeout, so a slow night produced a failure that our error handling never recognised as one.

We pinned the version, made the timeout explicit instead of inherited, and the failures stopped. That part took forty minutes.

Three things I actually took from it.

The first is a habit: I check my assumption about *where* a problem is before I spend a day inside one file. Usually by asking what changed, and what differs between the working case and the broken one. That would have cost twenty minutes on the first morning.

The second is that I should have asked for help much sooner. A day and a half is far too long, and the reason I did not ask is that I wanted to be the one who found it — which is a bad reason, and I recognise it now when it comes up.

The third is smaller and I use it constantly: I keep a running list while debugging of what I have ruled out. By the second afternoon I had re-checked two things twice because I had lost track.

And the broader thing, which is why I chose this story: the failure was in the boundary between our code and something else, and I only looked inside our code. Most of the genuinely hard bugs I have met since have been on a boundary of some kind — between two services, between a library version and an assumption, between what a system promised and what it actually did. So that is where I look earlier now.“

9 Recall card

The last week is not for adding. It is for arriving. Something new takes about three weeks to become retrievable under pressure, so a pattern met six days out is worth close to nothing on the day — while tiredness arrives in one night and leaves in one night, which makes it the only variable still worth managing. Nothing new goes in after TWO DAYS OUT. At that range new material does not add, it displaces.

The plan. –7 two mocks, scored · –6 the twenty tells, cold · –5 the ten templates typed from memory, twice · –4 one mock, then re-read ONLY what it exposed · –3 the design framework and one HLD out loud · –2 recall cards only, no new problems · –1 one easy problem, stop by lunch, fix the room and the login · the morning: nothing technical. Revise the recall cards, the twenty tells, the ten templates, YOUR OWN ERROR LIST and your STAR notes. Leave alone new topics, new case studies, and other people's problem counts.

The error list is the best ratio in this lesson: twenty minutes to write your own last ten bugs, three minutes to re-read on the morning — and it targets the mistakes you are statistically most likely to repeat. The ten templates must arrive without being derived: reconstructing binary search under pressure costs four minutes, which is 20% of a twenty-minute round spent on the part that was never the question.

On the day: no hard problem the night before (you either learn nothing or convince yourself you cannot do it); reset between rounds — stand up, one sentence, name the next round; one bad round is one round, and carrying it into the next two is what turns it into three; write every question down within an hour, because by the evening the details are gone and that list is the best preparation material you will ever own.

The behavioural answer is SITUATION (2 sentences) · TASK (1) · ACTION (4-5, the bulk) · RESULT (1-2, with a number) · LEARNED — and the last one is what the question was for. Pick a story where you were genuinely stuck and be honest

about it: “I worked all weekend and solved it” contains no difficulty, no error and nothing learned. **Test a story with two questions: does it show you being wrong and finding out, and does it end in a HABIT rather than an outcome? Learn the beats, never the sentences** — word-perfect is audible. And “no, you’ve covered everything” is a real negative; have two questions written down.

Final revision, and the week before the interview

1 What this is, and why they ask it

This is the one page. Everything from a hundred and eighty days of system design, compressed into what you would actually re-read the night before: **the two running orders, the numbers, and the trade-off pairs you must be able to argue in both directions.**

And it is the last thing in the course, so it ends where the interview does: with them asking whether you have any questions.

That question is not the end of the interview. It is part of it. “No, I think you covered everything” is a real negative signal — it says you have not thought about whether you want this job, only about whether they want you. **And it is entirely avoidable with two questions written down the night before.**

They ask it because **the questions you ask are the clearest evidence of what you think matters. Somebody who asks about the on-call rota and what a typical week’s paging looks like has operated something.** Somebody who asks only about the technology stack has not, yet. **And the way the answers come back tells you as much about the team as the answers themselves.**

And because it is genuinely the last thing you control. By that point the design rounds are decided. **What is still open is whether you leave the room having also evaluated them** — which matters, because the interview is a decision in two directions and most candidates only make one of them.

By the end of this lesson you have a single revision page for both design rounds, the numbers sheet, the trade-off vocabulary in both directions, a plan for the design side of the last week, the questions to ask, and what the answers to them actually mean.

2 The story

The flat was on the third floor and there was no lift, and Latha had decided she liked it before she reached the top of the stairs.

Her mother had come with her, which Latha had agreed to reluctantly and was already regretting.

The owner showed them round. **Latha asked about the rent, and the deposit, and whether they were allowed to paint the walls.**

Her mother asked four questions, and not one of them was about the flat.

“How many hours does the water come?”

“Which floor does the pressure stop reaching, in April?”

“How long have the people below been here?”

“Why did the last tenants leave?”

Latha was quietly mortified and looked at the balcony.

But she noticed something on the way back down the stairs.

The owner had answered the first three easily and quickly. He knew the water timings to the half hour, and he knew which floors suffered in summer, **and he said both without having to think.**

On the fourth he hesitated. Only a little — half a second — **and then said the family had moved to Hyderabad for work.**

Her mother said nothing about any of it in the auto home.

Then, over dinner: **“He was telling the truth about the water. He was not telling the whole truth about why they left.”**

Latha asked how she could possibly know that.

“I do not know it. I noticed it. Those are different things, and noticing is what the questions are actually for.”

And then the thing Latha repeated to her friends for years afterwards.

“You think you ask questions to get information. Half of it is that.”

“The other half is that the way a man answers a question about his own house tells you what it is like to live in it.”

They took the flat. **The water was exactly as described.** And in the second year the people below turned out to be precisely why the previous family had left — **which her mother had guessed and could not have proved, and about which she was gracious enough to say nothing at all.**

3 The idea in plain English

Latha's mother asks two kinds of question at once. One kind gets an answer. The other kind gets a reaction — and the reaction is often the more useful of the two.

That is exactly the shape of the last five minutes of an interview.

The one page — round one, high level

```
1 REQUIREMENTS 3-5 features IN, the rest explicitly OUT
                  scale · read:write · latency · availability
                  · CONSISTENCY · durability
2 ESTIMATION    DAU → actions → /86,400 → peak x2-3
                  → bytes → storage/year → bandwidth
                  EVERY NUMBER FOLLOWED BY "SO..."
3 INTERFACE      5-6 operations. Cursor pagination.
                  Idempotency keys on creates.
4 DATA MODEL    entities, keys, ACCESS PATTERNS
                  → then choose the store, with a reason
5 ARCHITECTURE   draw it, then WALK the read path and the
                  write path out loud
6 DEEP DIVE      offer three, let them pick. This is where
                  the signal is.

CLOSE monitoring & what pages · the SLO in minutes ·
          auth/authz & what you would not log · monthly cost ·
          what happens when each box dies ·
          ONE HONEST WEAKNESS OF YOUR OWN DESIGN
```

The one page — round two, low level

```
1 REQUIREMENTS in VERBS. "add an expense", "settle up"
2 ENTITIES      the nouns, and what each one KNOWS
3 RELATIONSHIPS who holds whom; what owns whose lifetime;
                  the INVARIANT, enforced in the constructor
4 INTERFACE     the public methods, with signatures
5 THE HARD PART the thing that will CHANGE - and the
                  pattern that absorbs it
6 EXTENSION     "now add X". A new class, or five edits?
                  THIS IS THE ROUND.
```

AND: say which kind of round you are in, in the first minute. Running the wrong framework is the commonest failure in a back-to-back loop.

The building blocks, in one list

Every high-level design is assembled from about fifteen things. If you can say what each is for and what it costs, you can build any of them.

LOAD BALANCER	spreads traffic; health checks; the entry point for TLS
CDN	puts bytes near users; the answer to an egress bill
CACHE	a copy nearer the reader; cache-aside vs write-through; INVALIDATION is the hard part
QUEUE / LOG	decouples producer from consumer; turns an outage into a delay
OBJECT STORE	blobs. Never a database.
RELATIONAL DB	transactions, joins, constraints
WIDE-COLUMN	huge, append-only, read by one key
SEARCH INDEX	free text; eventually consistent with the source of truth
REPLICA	read scaling and failover; replication lag is the cost
SHARD	write scaling; the shard key is the decision you live with
RATE LIMITER	protects everything behind it
CIRCUIT BREAKER	stops a slow dependency becoming an outage
WORKER POOL	the asynchronous half of the system
API GATEWAY	auth, limits, routing, one front door
MONITORING	metrics detect · traces localise · logs diagnose

The trade-off pairs

You should be able to argue each of these in both directions, in about thirty seconds. That is what “and now argue the opposite” is testing.

SQL	vs	NoSQL
strong	vs	eventual consistency
synchronous	vs	asynchronous
normalise	vs	denormalise
fan-out on write	vs	fan-out on read
cache-aside	vs	write-through
monolith	vs	microservices
stateful	vs	stateless services
push	vs	pull
commit capacity	vs	on-demand

In every pair, the second option is not “the modern one”. It is the one that is right under different conditions, and naming the condition is the answer. “Eventual consistency, because a like count may be a few seconds stale — but not for the payment, where I would take the coordination cost.”

The design side of the last week

- 7 one full design mock, on the clock, out loud, scored
- 6 the two running orders, from memory
- 5 the numbers sheet - recite it cold
- 4 one design mock. Then re-read only what it exposed.
- 3 one high-level design out loud, end to end, with the closing checklist
- 2 recall cards only. No new case studies.
- 1 write your questions for THEM. Two, minimum.
- 0 nothing technical

Same rule as the coding side: nothing new after two days out. A case study first read on Friday is not a case study you can design on Monday — you will produce a half-remembered version of somebody else's answer, which is worse than reasoning from the building blocks.

The questions you ask them

Four kinds, and each one tells you something different.

ABOUT THE WORK

"What does the first ninety days look like?"

→ is the work defined, or is defining it the job?

ABOUT THE SYSTEM

"What is the on-call rota, and how many pages does a typical week produce?"

→ the single most informative question you can ask about engineering health

ABOUT THE TEAM

"What did you not expect before you joined?"

→ gets a more honest answer than "what is the culture like"

ABOUT YOU

"What would make you say, in six months, that this hire had gone particularly well?"

→ and it gives you the actual success criteria

And listen the way Latha's mother listened. A hesitation on "how many pages a week" is information. A specific answer — "about two, and one of them is usually the same flaky job" — is a healthy team describing a real system. A vague one — "it's manageable" — is a different kind of answer, and you have learned something either way.

4 The picture

The whole syllabus as one reference architecture — every building block, in the place it belongs:

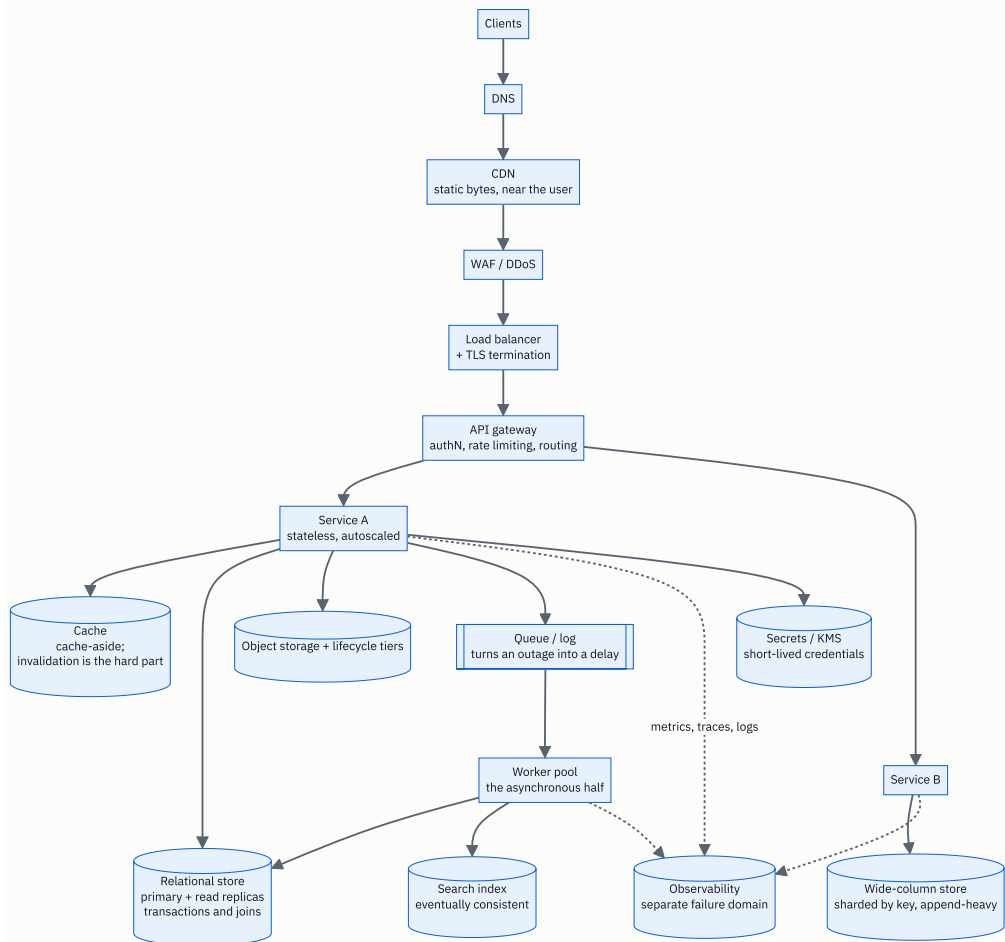


FIGURE 180.2

If you can point at every box and say what it is for, what it costs, and what happens when it dies, you can design any of the case studies — because they are all this diagram with different boxes emphasised. A feed is heavy on the queue and the wide-column store. A video service is heavy on object storage and the CDN. A payment system is heavy on the relational store and light on everything else.

The night-before card, which is the whole course on one screen:

```

+-----+
| THE TWO RUNNING ORDERS |
| HLD: requirements · estimation · interface · |
| data model · architecture · deep dive |
| LLD: verbs · entities · relationships · interface · |
| the thing that changes · EXTENSION |
+-----+
| ALWAYS SAY |
| - here is how I would like to use the time |
| - these features in, these OUT |
| - "so..." after every number |
| - shall I go deeper here, or move on? |
| - one honest weakness of my own design |
+-----+
| THE NUMBERS |
| 1 day = 86,400 s (call it 100,000) |
| memory 100 ns · SSD 100 us · same-zone 0.5 ms · |
| cross-region 50-150 ms |
| 1 machine ~10,000 req/s · Postgres ~5,000 writes/s · |
| Redis ~100,000 ops/s |
| row ~1 KB · photo ~2 MB · 1080p ~50 MB/min |
| storage $0.023/GB-mo · EGRESS $0.09/GB · |
| cross-zone $0.01/GB EACH WAY |
| 99.9% = 43 min/month · 99.99% = 4.3 · 99.999% = 26 s |
+-----+
| THE CLOSE |
| monitoring · SLO in minutes · security · cost · |
| failure per box · one honest weakness |
+-----+
| MY TWO QUESTIONS FOR THEM |
| 1. _____ |
| 2. _____ |
+-----+

```

The last box is deliberately blank. Fill it in the night before, in your own words, for that specific company — and having two written down means that “do you have any questions” is never the moment you go quiet.

5 How it actually works

How to revise a design track, since it is not like revising code

You cannot practise system design by reading case studies. Reading one is pleasant and produces almost no retrievable skill, because the difficulty was never in knowing what a cache is — it is in producing a running order under time pressure with somebody watching.

LOW VALUE

reading a case study

memorising an architecture

more case studies

watching a video

HIGH VALUE

designing one OUT LOUD, timed,
then reading the case study
being able to say what each
building block costs
the same case study a second
time, a week later, from
memory
recording yourself and
listening back

The single most effective exercise: pick a product, set forty-five minutes, and design it out loud with nobody there. Then compare with a written-up version, and note only the things you did not think to ask.

The two running orders, and why they must be automatic

Under pressure you will not invent a structure. You will do whatever you have done before — which for most candidates is “start drawing boxes”, because that is the enjoyable part and the only part they have practised.

Say the opening sentence out loud enough times that it is automatic:

“Before I draw anything — here is how I would like to use the time...”

Fifteen seconds, and it changes the whole round, because from then on you are running it and they are responding, rather than the other way round.

Taking the round’s notes

Keep the requirements and the numbers visible for the whole round. Top-left corner, and do not rub them out. You will refer back to them constantly — “*we said a hundred-to-one read ratio, so...*” — **and so will the interviewer.**

And in a remote round, where the whiteboard is a shared document, say what you are doing while you scroll. “*I am going back up to the requirements for a second.*” **Silence plus a moving cursor is confusing from the other side.**

The last five minutes

1. state the closing checklist, unprompted
monitoring · SLO · security · cost · failure ·
one honest weakness
2. ask your questions - the ones you wrote down
3. LISTEN to how they are answered, not only to what
the answers are
4. thank them, and ask what the next step is and when
→ a specific answer is a good sign; a vague one
is information too

And afterwards, within the hour: write down every question you were asked. The design questions especially, because they are reused — the same company will ask a very similar one at the next level.

The offer conversation, briefly

It is out of scope for this course and one thing is worth saying, because it is the same skill.

“What are your salary expectations?” is a question with a structure, exactly like a design question. Deflect once — “I would rather understand the role first; what is the band for this level?” — and if pressed, give a range you have researched rather than a number you hope for. The same principle as the design round: gather information before committing to a position.

What to do when a round has clearly gone badly

Finish it properly anyway. Loops are scored per round and the interviewers usually confer afterwards — a candidate who visibly gave up in round two is remembered differently from one who was struggling and stayed constructive.

And then do the reset from [day 179](#). Stand up, one sentence, name the next round. The damage from a bad round is almost never the round. It is the next two.

6 The numbers

This is the sheet. Recite it cold, five days out.

Time.

1 day = 86,400 seconds → call it 100,000
1 month = 2.6 million seconds
1 year = 31.5 million seconds

1 million/day = ~12 per second
1 billion/day = ~12,000 per second
100 million/day = ~1,200 per second

Latency.

memory read	100 ns	
SSD read	100 microseconds	(1,000x memory)
network, same zone	0.5 ms	
network, same region	1-2 ms	
network, cross-region	50-150 ms	
spinning disk seek	10 ms	

→ an in-process call is ~10 ns, so a network call is 50,000-100,000x slower. That single ratio is the microservices argument.

Throughput, order of magnitude.

one machine, simple HTTP	~10,000 req/s
one Postgres primary	~5,000 writes/s
one Redis instance	~100,000 ops/s
one Kafka partition	~10 MB/s

→ "700 writes a second against 5,000 for one primary, so I would not shard yet" is a real decision made in eight seconds.

Sizes.

UUID	16 bytes
timestamp	8 bytes
a short row	100 B - 1 KB
a structured log line	~300 bytes
a thumbnail	~200 KB
a compressed photo	~2 MB
1 minute of 1080p	~50 MB

Availability.

```
99%          7 h 12 m per month
99.9%        43.2 minutes
99.95%       21.6 minutes
99.99%       4.32 minutes
99.999%      25.9 SECONDS
```

```
in series:    0.999^8 = 99.2% → 5 h 44 m
in parallel:  two at 99% → 99.99%
```

→ IN SERIES YOU MULTIPLY AVAILABILITIES AND GET WORSE.
IN PARALLEL YOU MULTIPLY FAILURE RATES AND GET BETTER.

Money.

```
object storage      $0.023 per GB-month
archive tier        $0.004 (instant) / $0.001 (deep)
EGRESS to internet  $0.09 per GB      ← the surprise
cross-zone          $0.01 per GB, EACH WAY
a small machine     ~$0.10 per hour
managed logs        ~$0.50 per GB ingested
```

```
→ storing 30 TB for a month: $690
  sending it out once:      $2,700
  SERVING IS 4x MORE EXPENSIVE THAN STORING.
```

A worked estimate, so the sheet has been used at least once.

```
100,000,000 DAU x 20 reads/day
= 2,000,000,000 reads/day
= 23,000 reads/second (peak x3 = ~70,000)
```

```
100,000,000 x 0.2 writes/day
= 20,000,000 writes/day
= 231 writes/second (peak x3 = ~700)
```

```
RATIO 100:1 → read-optimised: cache hard, read
              replicas before sharding, fan out on write
```

```
700 peak writes vs ~5,000/s for one Postgres primary
→ ONE primary. Do not shard.
```

```
70,000 peak reads vs ~10,000 req/s per machine
→ 7 machines, so 15-20 with headroom and redundancy
```

And one last number, which is about the interview rather than the system.


```
A 45-MINUTE ROUND
3  introductions
5  requirements
5  estimation
4  interface
5  data model
9  architecture
9  deep dive
5  closing checklist and YOUR QUESTIONS
---
45
```

→ the last five minutes are on the plan. They are not spare time, and they are the only part of the round where you are the one asking.

7 The trade-offs

The pairs, each argued both ways in a sentence. This is the vocabulary the whole design track was building towards.

SQL or NoSQL. Relational when you need transactions, joins and constraints, and when the shape of the data is stable — which is most business data, and the default should be Postgres. **Wide-column** when the volume is enormous, the access is by one key, and the write rate exceeds what a single primary can take. I would not use NoSQL to avoid designing a schema; that is the most common wrong reason.

Strong or eventual consistency. Strong where a disagreement is visible and harmful — an account balance, an inventory count for the last item, a payment. **Eventual** where staleness is invisible or tolerable — a like count, a follower list, a search index. The question to ask the interviewer is “what does the user see during the gap?”, and the answer is a product decision rather than a technical default.

Synchronous or asynchronous. Synchronous when the caller cannot proceed without the answer. Asynchronous for everything else — and it is the single most effective way to stop availability multiplying, because a dependency being down becomes a delay rather than a failure. The cost is eventual consistency and a queue you now have to operate.

Normalise or denormalise. Normalise for correctness and for data that is written often. Denormalise for reads, deliberately, when the join is the bottleneck — and accept that you now have two copies that can disagree, which needs an owner and

an update path.

Fan out on write or on read. On write when reads vastly outnumber writes, so the expensive work is done once. **On read when the fan-out is enormous** — the celebrity problem. **The real answer is usually the hybrid, with a threshold you can change without a deploy.**

Cache-aside or write-through. Cache-aside is simple and the cache can be stale on a write. Write-through keeps them consistent and puts the cache on the write path, so a cache problem becomes a write problem. I would use cache-aside with a short expiry by default, and only take write-through where staleness is actually harmful.

Monolith or microservices. The deciding factor is the number of teams, not the size of the system. One team: a monolith, and splitting is pure cost. Several teams blocked on one release train: extract along team lines. And the modular monolith is the usual right answer in between.

Commit capacity or stay on demand. Commit to the baseline that has been running every hour for six months — roughly 40% off. Never commit to the peak. And put anything interruptible on spot, for 70-90% off, provided it can handle a two-minute termination notice.

And the meta-trade-off, which is the one worth ending on. Every one of those pairs has a right answer only once you know the conditions, and the conditions come from the requirements — which is why the running order puts requirements first and why every number is followed by “so...”. **A candidate who has memorised the right-hand column of every pair has learned fashion.** **A candidate who asks what the read-to-write ratio is, and then picks, has learned engineering.**

8 In the interview

How it gets asked

- *“Do you have any questions for us?”* — always, and it is part of the interview.
- *“Is there anything you would like to know about the team?”* — the same question, softer.
- *“What are you looking for in your next role?”* — answer honestly; a mismatch helps both of you.
- *“Why are you leaving?”* — never criticise the current employer; say what you are moving towards.

- *“What are your salary expectations?”* — deflect once, then give a researched range.

The first ninety seconds

On “do you have any questions for us”:

“Yes — three, and I will keep them short.

What does the first ninety days look like for whoever takes this role? I am asking because the answer tells me whether the work is already well defined or whether working out what the work is is the first job. Both are fine and they are very different, and I would rather know which.

What is the on-call arrangement, and roughly how many pages does a typical week produce? Partly to know what I am signing up for, and partly because that number says a great deal about the health of the system. A team that pages twice a week has different problems from one that pages twice a quarter, and neither answer would put me off — I would just want to know which one I am joining.

And what is something about working here that you did not expect before you joined? I find that gets a more honest answer than asking what the culture is like.

If we have time, one more: what would make you say, in six months, that this hire had gone particularly well?”

And then listen to how they answer, not only to what they say. A specific answer to the paging question — “about two a week, and one is usually the same flaky job we keep meaning to fix” — is a healthy team describing a real system. A vague one is also an answer.

The follow-ups

“What would you want to know about the system itself?”

“Four things, and each one tells me something I could not learn from the outside.

What is the deployment story — how long from merging a change to it being live, and how long to roll it back? The rollback number is the one I actually care about, because a team that can go back in five minutes ships freely, and a team that takes an hour deploys rarely and therefore in large risky batches. **That single number tells me most of what daily life is like.**

What is the oldest part of the system that everybody is nervous about? Every system has one, so a denial is more worrying than an answer. And I would want to know whether there is a plan for it or whether it is simply avoided.

What does the observability look like — if a user reports a slow request, how long does it take to find out which service was slow? If the answer is ‘we look at the trace’, that is a mature system. If it is ‘we grep the logs on four machines’, I know what my first six months would usefully contain.

And how are architectural decisions actually made? One person, a design review, a written proposal? There is no wrong answer, but knowing it tells me how a disagreement would go, and that matters more to me than the technology stack.”

“What would you want to know about the team?”

“Things that are hard to fake, mostly.

How long have the people on the team been here, and when did the last person leave and why? That is a version of Latha’s mother’s fourth question — I am not expecting a scandal, and the way it is answered is as informative as the answer.

How does the team decide what to work on? Whether an engineer can say ‘this needs three weeks of reliability work’ and be heard is a real property of a team, and it connects to something concrete: if there is an error budget policy, is the feature freeze actually honoured, or is it overridden every time?

What does a code review typically look like — how long, how many rounds, what gets blocked on? It tells me about standards and about how people talk to each other, at once.

And what is the split between new work and maintaining what exists? Any honest answer is useful. A team that says ‘ninety percent new features’ is either very young or not telling me about the other forty percent.”

“Is there anything you would like to add?”

“Two things, briefly.

First, on the design round — I said the fan-out worker was the weakest part of my design, and I want to be clear that I would treat that as work rather than as a caveat. The concrete version is a queue-lag metric with an alert on it, because that failure has no user-facing symptom and would otherwise run for hours. I mention it because naming a weakness and not having a plan for it is only half an answer.

Second, something you did not ask about that I think is relevant. The thing I have found hardest and most useful over the last year is getting better at saying where I am when I am stuck — what I know, what is blocking me, and what I am about to try. It sounds like a communication point and it is really an engineering one: it is what lets somebody else help, and it is how I found the last two problems I could not find alone.

And thank you — this has been a genuinely interesting conversation, which I do not say about all of them. What is the next step, and when would you expect to know?”

The model answer

“Do you have any questions for us?”

“I do, and I have thought about which ones actually matter to me rather than which ones sound good — so some of these are quite practical.

First, the work itself. What does the first ninety days look like for this role? I want to know whether the problem is defined and needs solving, or whether defining it is the job. I would be happy with either, and they are different jobs and I would rather know which one I am accepting.

Second, the system. What is the time from merging a change to it being live — and more importantly, how long to roll one back? I ask about rollback specifically because that number decides everything downstream: a team that can go back in five minutes ships small changes often, and a team that takes an hour ships rarely and therefore in large batches, which is more dangerous. It is the single most informative number about daily engineering life.

Third, on-call. What is the rota, and roughly how many pages does a typical week produce? Not because I am reluctant to be on call — I think you should operate what you build — but because that number is the best available measure of whether the system is healthy. And I would want to know whether pages are actionable, or whether people have started ignoring them, which is the failure mode that actually causes outages.

Fourth, the team. What is something about working here that you did not expect before you joined? I ask that instead of asking about the culture, because it usually gets a real answer.

And last, about me: what would make you say, in six months, that this hire had gone particularly well? Partly because I would like to know the actual criteria, and partly because if the answer surprises me, that is something we should both know now rather than in six months.

I would also say — and this is the honest part — that I am trying to work out whether I want to work here, not only whether you want me. The design rounds today told you quite a lot about how I think. These questions are the part where I find out the same kind of thing about you — and I think that is a fair trade, and probably better for both of us than my nodding politely and asking nothing.

Thank you. What is the next step, and when would you expect to know?“

9 Recall card

Two running orders, and say which round you are in within the first minute. HLD: requirements (3-5 in, the rest explicitly OUT; scale, read:write, latency, availability, CONSISTENCY) · estimation (every number followed by “so...”) · interface · data model from ACCESS PATTERNS · architecture, then WALK both paths · deep dive, offered as a choice. LLD: verbs · entities · relationships and the invariant in the constructor · interface · the thing that will CHANGE · EXTENSION, which is the round. Close both with monitoring, the SLO in minutes, security, cost, failure per box, and ONE HONEST WEAKNESS OF YOUR OWN.

The numbers, cold. 1 day $\approx$ 100,000 s. memory 100 ns · SSD 100 μ s · same-zone 0.5 ms · cross-region 50-150 ms — **and an in-process call is $\sim$ 10 ns, so a network hop is 50,000-100,000 $\times$ slower.** One machine $\sim$ 10,000 req/s · Postgres $\sim$ 5,000 writes/s · Redis $\sim$ 100,000 ops/s. Row $\sim$ 1 KB · photo $\sim$ 2 MB · 1080p $\sim$ 50 MB/min. Storage \$0.023/GB-mo · EGRESS \$0.09/GB · cross-zone \$0.01/GB EACH WAY — serving 30 TB costs $4\times$ storing it. 99.9% = 43 min/month, 99.99% = 4.3, 99.999% = 26 seconds. In series you multiply availabilities and get worse; in parallel you multiply failure rates and get better.

Ten trade-off pairs, argued BOTH ways in thirty seconds each — SQL/NoSQL, strong/eventual, sync/async, normalise/denormalise, fan-out on write/read, cache-aside/write-through, monolith/microservices, stateful/stateless, push/pull, committed/on-demand. The second option is never “the modern one” — it is right under different conditions, and naming the condition IS the answer.

Revise design by DESIGNING, not reading. Forty-five minutes, out loud, timed, alone — then compare, and note only what you did not think to ask. **Nothing new after two days out:** a case study first read on Friday becomes a half-remembered version of somebody else’s answer, which is worse than reasoning from the building blocks. If you can say what each of the fifteen building blocks is for, what it costs, and what happens when it dies, you can design any of them.

“Do you have any questions for us?” is part of the interview, and “no, you’ve covered everything” is a real negative. Ask about the first ninety days (is the work defined, or is defining it the job), rollback time (the number that decides daily engineering life), pages per week (the best available measure of system health), and “what did you not expect before you joined?” Then listen the way Latha’s mother listened — the way an answer comes back tells you as much as the answer. Write down every question you were asked within the hour, and ask what the next step is and when.

PRACTICE

Practice — The last week, and the questions you ask

DSA topic: Final revision, and the week before the interview **System design topic:** Final revision, and the week before the interview

Code these, in this order

There are no new problems today, and that is deliberate. The last exercise in this course is the ten templates, typed from memory, twice.

#	TEMPLATE	WHAT IT MUST DO WITHOUT BEING DERIVED	THE LINE PEOPLE GET WRONG
1	Binary search, lower bound	First position where the target could be inserted	<code>high = len(values)</code> , not <code>len - 1</code>
2	Sliding window with a constraint	Longest stretch with at most k distinct	deleting the key when its count hits zero
3	Two pointers on a sorted list	Three-sum with no duplicate answers	both skip lines
4	Monotonic stack	Largest rectangle in a histogram	the sentinel that flushes the stack
5	BFS on a grid	Fewest steps to the far corner	marking as seen on ENQUEUE
6	DFS on a tree	Diameter: return one value, record another	the two different numbers
7	Topological sort	An order, or empty if there is a cycle	<code>len(order) != n</code> as the cycle test
8	Union-Find	Connected components, near constant time	path halving, and union returning a bool
9	Heap for top-k	The k most frequent	a MIN-heap gives you the k LARGEST
10	Two-dimensional DP	Edit distance	<code>a[i-1]</code> , because row <code>i</code> is the first <code>i</code> characters

The rule

Type them. Do not read them. Then run them against the examples in the lesson. Then do it again tomorrow. Anything that takes more than ninety seconds, or comes out wrong, goes on a list — and that list is what you revise on day five.

Then write your error list

Go back through the last three months of your own solutions and find the bugs you actually made. Not bugs in general — yours.

Ten lines. Something like:

1. off-by-one in the window's right edge
2. forgot the empty-input case
3. marked BFS nodes as seen on dequeue
4. did not ask about duplicates
5. used `/` where I meant `//`
6. did not bracket a shift
7. started coding before stating the cost
8. forgot to state complexity unprompted
9. defended an idea after a hint
10. did not test the case I was unsure about

Twenty minutes to write. Three minutes to re-read on the morning. Nothing else in this course has that ratio.

Then design one thing out loud, timed

Forty-five minutes, a clock, nobody there. Pick a product you use. Run the six beats. Then read somebody's written-up version and note only the things you did not think to ask — not the things you got differently, the things you did not think to ask.

Then run through the low-level order once

Pick something small — a lift, a card game, a booking form. **Requirements as verbs, entities, relationships, interface, the thing that will change.** Then have somebody ask you to add a feature, and count how many existing files you would have to edit.

Then recite the numbers sheet, cold

Time, latency, throughput, sizes, availability, money. Out loud, without looking. Then use two of them to answer “do I need to shard?” in one sentence.

Then write your two questions

For the specific company. In your own words. Write them down where you will see them on the day.

And write your STAR notes — two or three stories, in beats not sentences. Situation, task, action, result, learned. Test each one: were you actually wrong about something? Does it end with a habit?

Then the taper drill

Say the seven-day plan from memory. Then say what happens if the interview is in three days instead of seven, and which items you drop first.

Then stop

That is the last exercise in this course. Everything after this point is the interview, and the most useful thing you can do the day before is nothing.

The templates drill

1. Type all ten from memory. Time each one.
2. For each, say the line people get wrong.
3. Say why `lower_bound` is the binary search worth memorising.
4. Say what `union` returning a boolean is for.
5. Say why a min-heap gives you the k largest.

The taper drill

1. Give the seven-day plan, day by day.
2. Say what the line at minus two days is, and why.
3. Say how long something new takes to become retrievable.
4. Say why tiredness is the only variable worth managing.
5. Say what to drop first if you only have three days.

The revise-or-leave drill

1. Give six things to revise.
2. Give six things to leave alone.
3. Say which single item has the best ratio, and its numbers.
4. Say why other people's problem counts are worthless to you now.

The night-before drill

1. Give the six things to sort out the night before.
2. Say what the actual objective of the evening is.
3. Say what a hard problem can do for you at that point.
4. Say what belongs on the one page you re-read.

The on-the-day drill

1. Say what you do the morning of.
2. Give the reset between rounds, all four steps.
3. Say what one bad round means, and what turns it into three.
4. Say what to do within an hour of finishing, and why.

The behavioural drill

1. Give the five beats of a STAR answer, with lengths.
2. Say which beat is the point of the question.
3. Give the two tests a story must pass.
4. Say what is wrong with “I worked all weekend and solved it”.
5. Say why you learn the beats and never the sentences.

The two-orders drill

1. Give both six-beat running orders.
2. Say the opening sentence for a design round.
3. Say what each round’s closing checklist contains.
4. Say what happens if you run the wrong framework.

The building-blocks drill

1. Name fifteen building blocks.
2. For any five, say what it is for, what it costs, and what happens when it dies.
3. Say what makes a feed design different from a video design, in terms of which blocks dominate.

The trade-offs drill

For each pair, argue both directions in thirty seconds:

1. SQL / NoSQL
2. strong / eventual consistency
3. synchronous / asynchronous

4. normalise / denormalise
5. fan-out on write / on read
6. cache-aside / write-through
7. monolith / microservices
8. committed / on-demand capacity

The numbers drill

1. Seconds in a day, and the number you round it to.
2. Four latency figures.
3. Four throughput figures.
4. Availability at four levels, in minutes.
5. Storage, egress and cross-zone prices.
6. The ratio between serving 30 TB and storing it.
7. Series against parallel, in one sentence.

The questions drill

1. Give four questions to ask, one from each category.
2. Say what a specific answer to the paging question tells you.
3. Say what a vague one tells you.
4. Say what is wrong with “no, you’ve covered everything”.
5. Say what you ask at the very end.

The break-it drill

For each, say what happens:

1. Learning a new pattern three days before the interview.
 2. A hard problem the night before.
 3. Comparing your problem count with somebody else’s.
 4. Carrying a bad round into the next one.
 5. Not writing down the questions afterwards.
 6. A word-perfect rehearsed behavioural answer.
 7. Reading case studies instead of designing out loud.
 8. Running the high-level framework in a low-level round.
 9. Answering “do you have any questions” with “no”.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Tell me about a problem you found hard, and what you learned from it.* Situation, task, action — with you being wrong in it — result with a number, and the habit you changed. Not the heroic version.
 2. *What are you going to do in the seven days before your interview?* The day-by-day plan, the line at minus two days, what you revise and what you leave, and why tiredness is the only variable left.
 3. *Do you have any questions for us?* Four questions from four categories, why each one is informative, and what you listen for in how they are answered.
-

Before the interview

- [] I can type all ten templates from memory in under ninety seconds each.
- [] I know the line people get wrong in each of them.
- [] I have written my own error list — my bugs, not bugs in general.
- [] I know the seven-day plan and the rule at minus two days.
- [] I know nothing new goes in after that, and why.
- [] I know what to revise and what to leave alone.
- [] I will not do a hard problem the night before.
- [] The machine, the room, the link and the food are decided.
- [] I have the twenty patterns and their tells, cold.
- [] I have both design running orders, from memory.
- [] I can recite the numbers sheet without looking.
- [] I can argue all ten trade-off pairs in both directions.
- [] I can name fifteen building blocks and what each costs.
- [] I say which kind of design round I am in, in the first minute.
- [] I close every design with monitoring, SLO, security, cost, failure and one honest weakness.
- [] I have two or three STAR stories, as beats and not sentences.
- [] Each story has me being wrong in it, and ends with a habit.
- [] I have two questions written down for this specific company.
- [] I know what to listen for in how they answer.

- [] I will reset between rounds: stand up, one sentence, name the next round.
 - [] I know one bad round is one round.
 - [] I will write down every question within an hour of finishing.
 - [] I answered all three questions above out loud.
-

After day 180

That is the course.

One hundred and eighty days, two tracks, and the last of them ends where the interview does — with you asking the questions.

What you have is not a list of solved problems. It is twenty patterns you can recognise from a sentence, ten templates that arrive without being derived, two running orders that hold under pressure, a page of numbers that turn opinions into decisions, and the habit of saying what you are thinking while you think it.

None of that expires. Go and use it.

The nine sections, and what each is for

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but $50\text{M} \times 20 \times 2\text{KB} = 2\text{TB}$.

7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.

Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

Bits and maths

Day 171 · dsa — Binary, bits, and why they matter

Binary is place value with two instead of ten: the places are 1, 2, 4, 8, 16, each twice the last, each digit 0 or 1. $13 = 8 + 4 + 1 = 1101$. **n bits gives 2^n values, 0 to $2^n - 1$.** Know $2^{10} \approx$ a thousand, $2^{20} \approx$ a million, $2^{30} \approx$ a billion.

Converting, two rules. In: **divide by two, read the remainders UPWARDS** (downwards gives $1011 = 11$, a plausible wrong answer with no self-check). Out: **left to right, double and add the next bit.**

Six operators, one sentence each. AND = **both**. OR = **either**. XOR = **exactly one**, which is the same as **different**. NOT flips. **$\ll 1$ doubles, $\gg 1$ halves rounding down** — the same place-value fact, and the rounding is just the bit that fell off. **$1 \ll k$ is 2^k , the single-bit mask.**

The four edits, all built from $1 \ll \text{position}$: GET $(n \gg p) \& 1$; SET $n \mid (1 \ll p)$; CLEAR $n \& \sim(1 \ll p)$; TOGGLE $n \wedge (1 \ll p)$.

Three tricks. **$n \& (n - 1)$ clears the lowest set bit** — subtracting one flips it and ones-fills below, so ANDing removes exactly it. That gives **Kernighan's count** (one step per SET bit: 1 instead of 32 for $1 \ll 31$, but no better when all bits are set) and **$n > 0$ and $(n \& (n-1)) = 0$ for powers of two** — **the guard is not decoration, or zero passes.** **$x \& (size - 1)$ is $x \% size$ for power-of-two sizes:** one CPU cycle against twenty to forty, which is why hash tables and ring buffers are sized that way.

Negatives are two's complement — flip and add one — so one adder handles both signs and the carry falls off the end. The range is asymmetric (-128 to 127). Python is arbitrary-width, so **$\sim x = -x - 1$, not a pattern — mask with $\& 0xFF$ or $\& 0xFFFFFFFF$.** And **$\gg$ is arithmetic:** **$-1 \gg 5$ is still -1 , so a `while n: n  $\gg=$  1` loop HANGS on a negative.** Always bracket shifts — **$1 \ll n - 1$ is $1 \ll (n-1)$, not $(1 \ll n) - 1$.**

Day 172 · dsa — The bit tricks every interview uses

Two core moves, and they are opposites. `n & (n - 1)` **CLEARs** the lowest set bit — `n - 1` borrows, so the lowest 1 flips and everything below fills with ones; above it nothing changed. `n & -n` **KEEPS ONLY** the lowest set bit — `-n` is `~n + 1`, the same ripple from the other side. Almost every trick is one of these in a loop.

Masks: `1 << k` is one bit; `(1 << k) - 1` is `k` ones at the bottom; `~(1 << k)` is a hole. Always bracket — `1 << k - 1` is `1 << (k-1)`, silently wrong.

One-line tests. Power of two: `n > 0` and `n & (n-1) == 0` — the guard is not decoration, `0 & -1 == 0`. All ones (0, 1, 3, 7, 15): `n & (n+1) == 0` — the carry runs off the top. Odd: `n & 1`.

Counting. Kernighan `while n: n &= n-1` — **one step per SET bit** (1 against 32 for `1 << 31`, tied at 255), and it **HANGS on a negative in Python**, so guard or mask. Every count at once: `bits[i] = bits[i & (i-1)] + 1`, valid because `i & (i-1) < i`. Hamming distance = set bits of `a ^ b`, because **XOR is “different”**.

A subset IS a number. `for mask in range(1 << n)` lists all 2^n subsets; `mask >> i & 1` tests item `i`; `n * 2^n` is why bitmask problems stop at `n = 20`. Walk sparse masks with isolate-then-clear. Every submask: `sub = (sub - 1) & mask`, **append-then-break-on-zero** — 3^n , not 4^n , and testing at the top skips the empty submask.

Day 173 · dsa — XOR problems

One fact runs the whole family: `a ^ a = 0`, `a ^ 0 = a`, and order does not matter. So XORing a list cancels everything that comes in pairs. **The rearrangement argument IS the proof** — group the duplicates, every bracket is zero. XOR is “is the count of ones in this column odd?”, asked independently per column, so nothing carries and nothing overflows.

One loner: XOR the list. One missing from 0..n: XOR the list against the range (seed the accumulator with `len(numbers)`, or the top of the range has no partner). **Prefer this to the sum formula, which overflows.**

Two loners is the real question. XORing everything gives `a ^ b`, which does not determine the pair — 6 could be 3^5 or 1^7 . A set bit in `a ^ b` means they **DISAGREE** there, so isolate one with `both & -both` and split the list on it: **duplicates agree everywhere so pairs stay together; the loners are forced apart**. Then `second = both ^ first` — no third pass. **Needs exactly two loners, or it silently returns (0, 0).**

k copies means counting modulo k. Three times each: count every bit position mod 3, 32 passes, $O(32n)$. XOR is the `k = 2` case for free. **In Python, fix the sign at the end (`answer - (1 << 32)` when bit 31 is set) or minus three comes back as 4,294,967,293.**

XOR is its own inverse, so it does prefix sums. `xor(l..r) = prefix[r+1] ^ prefix[l]`, and `prefix[i] = prefix[k]` means that stretch XORs to zero. **The XOR of 0..n has period four:** `n, 1, n+1, 0` for `n%4 = 0,1,2,3`. **Biggest XOR pair:** build the answer from the top bit down and ask `wanted ^ prefix in prefixes` — $O(32n)$, not $O(n^2)$. And the trap that passes small tests: plain XOR looks correct whenever every repeat count happens to be even.

Day 174 · dsa — Primes, GCD, and modular arithmetic

Primality: test only to the SQUARE ROOT — if `n = a×b`, they cannot both exceed $\sqrt{n}$, so 1,000 divisions instead of a million. **Then step by 6 and test `6k ± 1`, a third again. 1 is not prime; 2 is, and is the only even prime — write the base cases first. Beyond $\sim 10^{14}$, name Miller–Rabin.**

The SIEVE is the change of viewpoint: cross off multiples instead of testing numbers. Start at `p × p` (everything smaller already has a smaller prime factor) and stop when `p × p > limit`. $O(n \log \log n)$ — $\sim 2.8\text{M}$ operations for a million, about $100\times$ faster than testing each — and memory limits it before time does (8 MB / 80 MB / 800 MB in Python; use a bytearray, a bitset, or a segmented sieve for 10^9). **78,498 primes below a million.** A smallest-prime-factor sieve then factorises anything in $\leq \log_2 n$ steps.

Euclid: `gcd(a, b) = gcd(b, a mod b)`, because a common divisor of both also divides the remainder — so the set of common divisors never changes. The picture is cutting the largest squares off a rectangle. $O(\log \min(a,b))$, worst case consecutive Fibonacci numbers; `gcd(a, 0) = a`. `lcm = a // gcd * b` — **DIVIDE FIRST**, or `a*b` overflows 64 bits around 3×10^9 .

Under a modulus: +, − and $\times$ pass through; $\div$ does NOT. Division becomes multiplication by an inverse. Fermat, when the modulus is prime: `a(m-2) mod m` — one fast power. On a composite modulus it silently returns a wrong number (the “inverse” of 3 mod 10 comes out as 1); use extended Euclid there, and note there is no inverse at all when `a` shares a factor with `m`. In C/Java/Go `%` takes the dividend’s sign, so write `((a-b) % m + m) % m`; Python’s `%` is already non-negative.

Fast power: square and multiply, one step per BIT of the exponent — 30 steps for 10^9 , 60 for 10^{18} . Forgetting `% m` inside the loop does not crash, it just stops finishing. `10^9 + 7` is used because it is prime (Fermat works), fits in a signed 32-bit integer, and two values under it multiply to under 10^{18} so `a*b % m` is safe in 64 bits.

Day 175 · dsa — The combinatorics you actually need

Two questions decide everything: DOES ORDER MATTER, and CAN THINGS REPEAT. Order + no repeats = nPr ; no order + no repeats = nCr ; order + repeats = n^n ; no order + repeats = stars and bars, $C(n+r-1, r)$. Derive, do not recall: n choices, then $n-1$, then $n-2$ — and stop after r factors.

The most useful sentence in the subject: “I counted each answer once per arrangement, so I divide by the number of arrangements.” `nCr = nPr / r!`. MISSISSIPPI = $11!/(1!4!4!2!) = 34,650$. When a count comes out too big you have double counted — usually the fix is to count the COMPLEMENT. And the multiplication principle needs independence; nothing warns you when the second choice is limited by the first.

NEVER COMPUTE A FACTORIAL. `21!` overflows a 64-bit integer, so `C(21,2)` = 210 comes back as nonsense. Build up: `result = result * (n - i) // (i + 1)`, multiply before you divide (the division is exact only after the multiplication), and use `r = min(r, n-r)` — 3 iterations, not 17, for `C(20,17)`. `//` not `/`: the float version is right on small inputs and quietly wrong past ~15 digits.

`C(n,r) = C(n-1,r-1) + C(n-1,r)` is “the item is in, or it is out” — which is why counting problems become DP, and why Pascal’s triangle IS the grid-paths table. Row n sums to 2^n , because it is every subset sorted by size. Pascal is $O(n^2)$ time and space — right for many small queries, impossible at $n = 100,000$.

Modulo 10^9+7 is the tell that the answer is astronomical. Precompute `fact[]` and `inv_fact[]`, then `C(n,r) = fact[n] · inv_fact[r] · inv_fact[n-r]` — three lookups, $O(1)$ per query. Build the inverses with ONE fast power at the top and a walk down (`inv_fact[i-1] = inv_fact[i] * i`), $15\times$ faster than one power each. Requires $n < \text{modulus}$, else Lucas’ theorem. Recognise the shapes: grid paths = $C(m+n-2, m-1)$; Catalan = $C(2n,n)/(n+1) = 1, 1, 2, 5, 14, 42, 132$ for brackets, tree shapes and parenthesisations; subsets = 2^n .

Day 176 · dsa — Bits and maths revision and mock round

Ask “which of the twelve”, not “what is this”. The triggers: “O(1) space” + repeats → XOR (one loner: XOR all; two: XOR then split on both & -both; three copies: count each column mod 3). “ $n \leq 20$ ” → bitmask over `range(1 << n)`. “all primes below N” → sieve (per group, not per number). “one number prime?” → trial division to $\sqrt{n}$. “modulo 10^9+7 ” → factorial tables + Fermat inverse. “exponent up to 10^9 ” → square and multiply. “how many ways” → does order matter, can things repeat.

Read the constraints BEFORE thinking about the problem — that is where the setter names the intended solution. Then: is the work per number or per group? (sieve vs test each) — does anything shrink structurally? — does the answer need dividing? `n & (n-1)`, Euclid and fast power are one idea: shrink by a structural step, not by one unit, and linear becomes logarithmic. When a limit is 10^9 , the answer is almost never a loop.

Say the recognition sentence out loud before writing a line. “ANDing a range keeps only the common prefix.” “A product is decided independently per prime, so I multiply the counts.” If you cannot say the sentence, you are not ready to write — and writing anyway is how twenty minutes disappear.

The silent failures, none of which raise. `1 << n - 1` is `1 << (n-1)`. `is_power_of_two(0)` is True without the guard. Plain XOR is right whenever repeat counts happen to be even — `[2,2,2,3]` gives 1. Fermat on a composite modulus returns a plausible non-inverse. `21!` overflows 64 bits, so `C(21,2)=210` comes out as nonsense. `/` instead of `//` in `nCr` lies past ~15 digits. And `while n: n &= n-1` on a negative hangs, which is worse than crashing.

Edge cases in this phase are always 0, 1, 2 and a negative. Where there is a clever version and an explainable one — the ones/twos state machine against counting columns, a trie against the greedy prefix — write the explainable one and say the clever one exists. Under a clock, being able to justify what you wrote beats a constant factor, because the reasoning is what is being graded.

Final mocks and revision

Day 177 · dsa — The twenty patterns, on one page

Twenty shapes, not twenty algorithms. 1 two pointers (sorted + a pair) · 2 sliding window (contiguous + constraint) · 3 fast/slow (cycle, middle) · 4 prefix sums (range queries, sums to k) · 5 binary search (sorted, want a position) · 6 binary

search on the ANSWER (“minimise the maximum”) · 7 sort then sweep (intervals, greedy) · 8 hashing (“seen before”, counting) · 9 monotonic stack (next greater) · 10 heap (top-k, k-merge) · 11 linked-list surgery · 12 tree traversal · 13 BST properties · 14 backtracking (“return ALL”) · 15 graph traversal + topo sort (a grid IS a graph) · 16 shortest paths · 17 union-find (“connected?” while joining) · 18 trie (prefixes) · 19 DP (ways/best, overlapping) · 20 bits and maths.

Read the constraints BEFORE the problem — the setter names the intended solution there. $n \leq 20$ → exponential/bitmask. $n \leq 1,000$ → $O(n^2)$, a 2-D table. $n \leq 10^5$ → $O(n \log n)$. $n \leq 10^9$ → a formula, never a loop. “O(1) extra space” is forbidding the hash map. “Return all” → backtracking. “Number of ways” → DP or counting. “Best value” → DP or greedy. “A position” → binary search.

Then: what structure does the input have (sorted → two pointers/binary search; tree → traversal; grid → graph; prefixes → trie; things joining → union-find), and what is the slow version repeating? Re-summing → prefix sums. Re-scanning ahead → monotonic stack. Re-solving → DP. Sorting to take five → heap. Almost every pattern is the same move: notice repeated work and stop repeating it.

Say the PRECONDITION when you name the pattern, because violating one gives a wrong answer, not a crash. Two pointers needs sorted. Sliding window breaks on NEGATIVES (shrinking can raise the sum — use prefix sums + a map). Greedy needs an exchange argument (`coins=[1,3,4]` , `amount=6` : greedy 4+1+1, optimal 3+3). Dijkstra needs non-negative weights. Binary search needs monotonicity.

Hold two or three candidates, never one. “Longest” fires window AND DP — the separating question is *contiguous?*. “Minimum cost” fires Dijkstra AND DP — *weighted graph?*. “Sorted” fires two pointers AND binary search. Recognition is the whole game: thirty seconds to the shape leaves ten minutes to write and five to test; nine minutes of brute force leaves a rushed implementation of the same code. And the perennial bugs: BFS marks seen on ENQUEUE, backtracking needs the undo, and you append `chosen[:]` , not `chosen` .

Day 178 · dsa — How to think out loud in a coding round

You are graded on what you SAY, not what you thought. The interviewer cannot see inside your head, and they are not testing whether you can code alone in a room — they are testing what it is like to solve a hard problem sitting next to you. Most companies score four separate axes: problem solving, coding, COMMUNICATION and TESTING — and two of those you can be excellent at even on a problem you do not finish.

The arc of 45 minutes: 0-2 restate · 2-5 the four questions · 5-7 an example by hand · 7-10 brute force with its cost, then the better idea with its cost, then “shall I write that?” · 10-30 code · 30-38 test out loud · 38-45 complexity and the follow-up. The first line of code is at minute TEN, and that is correct. Always say the brute force first — it proves you can find a solution before improving one.

The four questions, on every problem, even when you can guess: HOW BIG? WHAT WEIRD INPUTS (empty, duplicates, negatives, already sorted)? WHAT DO I RETURN WHEN THERE IS NO ANSWER? MAY I MODIFY THE INPUT? Listen to the answers — “n up to 100,000” has just ruled out $O(n^2)$. They cost about a minute and are the cheapest insurance in the round.

Narrate DECISIONS, not keystrokes. “I’m using a set so lookups are constant” — not “now I open a bracket”. **Thirty seconds is the silence limit**; after that say what you are doing, even “I’m working out whether this bound is inclusive”. **When stuck: say so, then shrink the problem, or work a small case by hand, or state what you know and what is blocking** — that last sentence is what lets the interviewer give you a hint, and they want to. Take hints by repeating them back in your own words; “no, because...” is the most expensive phrase in the round.

Test by SAYING THE VALUES, not by asserting. “The set is empty, so the loop never runs, so it returns 0” is a test; “I think that’s right” is a hope. **Test the thing you were unsure about while writing**, which is where your bugs actually live — **finding your own bug is a strong positive**. State the complexity unprompted. And manage the clock: at minute 25 with nothing working, say “let me get the straight-forward version down first and optimise if there is time” — that sentence turns a probable fail into a pass surprisingly often.

Day 179 · dsa — Full mock: two problems, forty-five minutes

Performing is a separate skill from knowing, and it only improves in the room. Meera’s four seconds were never about the notes. **The rules of a real mock: a visible clock (20 + 25), talk continuously, NO RUNNING THE CODE, no looking anything up, no pausing, record it, and score it honestly straight afterwards. Rule three matters most** — running the code is how you avoid learning to read your own code, and in the room there is no run button.

Less than half of each problem is coding. 3 min restate and clarify · 3 min brute force with its cost then the better idea with its cost then “shall I write that?” · 9 min code · 4 min trace and edges · 1 min complexity. **The seven minutes of slack at the end come entirely from the seven minutes of talking at the start.**

The minute at 0:20 is the RESET and it is not padding. Stand up. One sentence about the first problem, one thing to change, then start the second as though the first had not happened — carrying it across is how a strong candidate produces a weak second half. Three clock decisions, each with a sentence: at minute 10 with nothing written, write the brute force (“so that something works”); with a known gap, name it and fix it; at the end unfinished, DESCRIBE the missing piece. Partial credit is real and only available to people who say what is missing.

Score four axes out of four, per problem: problem solving, coding, communication, testing — 32 total. 26+ book the interview · 20-25 mediums fine, the hard one needs more rooms · 14-19 four more mocks · under 14 go back to the phase that fell over with the clock off. A mock scored generously has told you nothing.

Diagnose WHICH of four things failed, because they have different fixes. **Recognition** → re-read the pattern index, not more problems. **Articulation** → more recorded mocks; it is a motor skill. **Implementation** → write the seven templates from memory until automatic. **Composure** → more rooms. **The default response to a bad mock — solve fifty more problems — only helps with one of the four.** And say every loop’s invariant out loud before writing the loop: constructing and transcribing at the same time is what produces the freeze.

Day 180 · dsa — Final revision, and the week before the interview

The last week is not for adding. It is for arriving. Something new takes about three weeks to become retrievable under pressure, so a pattern met six days out is worth close to nothing on the day — while tiredness arrives in one night and leaves in one night, which makes it the only variable still worth managing. Nothing new goes in after TWO DAYS OUT. At that range new material does not add, it displaces.

The plan. –7 two mocks, scored · –6 the twenty tells, cold · –5 the ten templates typed from memory, twice · –4 one mock, then re-read ONLY what it exposed · –3 the design framework and one HLD out loud · –2 recall cards only, no new problems · –1 one easy problem, stop by lunch, fix the room and the login · the morning: nothing technical. Revise the recall cards, the twenty tells, the ten templates, YOUR OWN ERROR LIST and your STAR notes. Leave alone new topics, new case studies, and other people’s problem counts.

The error list is the best ratio in this lesson: twenty minutes to write your own last ten bugs, three minutes to re-read on the morning — and it targets the mistakes you are statistically most likely to repeat. The ten templates must arrive without

being derived: reconstructing binary search under pressure costs four minutes, which is 20% of a twenty-minute round spent on the part that was never the question.

On the day: no hard problem the night before (you either learn nothing or convince yourself you cannot do it); **reset between rounds — stand up, one sentence, name the next round; one bad round is one round, and carrying it into the next two is what turns it into three; write every question down within an hour,** because by the evening the details are gone and that list is the best preparation material you will ever own.

The behavioural answer is SITUATION (2 sentences) · TASK (1) · ACTION (4-5, the bulk) · RESULT (1-2, with a number) · LEARNED — and the last one is what the question was for. Pick a story where you were genuinely stuck and be honest about it: “I worked all weekend and solved it” contains no difficulty, no error and nothing learned. Test a story with two questions: does it show you being wrong and finding out, and does it end in a HABIT rather than an outcome? Learn the beats, never the sentences — word-perfect is audible. And “no, you’ve covered everything” is a real negative; have two questions written down.

Reliability, security, and the interview itself

Day 171 · system-design — Monitoring, metrics, and alerting

Three pillars, and it is a sequence rather than a choice: METRICS to detect (cheap, aggregated), TRACES to localise (sampled ~1%), LOGS to diagnose (expensive, full detail). `trace_id` on every log line is the join key. Roughly 9 GB/day of metrics against 100 GB/day of logs and 1 TB/day of unsampled traces at 100M requests.

Four golden signals: LATENCY (split successful from failed — a fast error drags the average down), TRAFFIC, ERRORS (as a fraction, never a count), SATURATION (the only leading indicator). RED for services, USE for resources.

The average describes nobody: 950 requests at 10 ms and 50 at 2,000 ms averages 109.5 ms, which no request took. $p_{50} = 10$ ms, $p_{99} = 2,000$ ms. **And the tail compounds — 20 backend calls means 18% of page loads hit a p99 call; 100 calls means 63%. You cannot average percentiles either** (p_{99} s of 100 ms and 10,000 ms average to 5,050 ms when the truth is ~100 ms — $50\times$ wrong); **sum HISTOGRAM BUCKETS instead.** Counters and histograms aggregate; summaries do not.

Alert on SYMPTOMS, not causes — you cannot enumerate causes in advance but there are about five symptoms. **Every page must be actionable, urgent, and need a human**; anything else is a dashboard entry or a ticket. **Three to five paging alerts per service**, each with `for: 5m`, a ratio not a count, the current value, and a **run-book link with a DO NOT line**. **Saturation is the one cause worth paging on**. **The fix is always subtraction** — and if under half of pages are real problems, the alerting is the problem.

SLI is the measurement, SLO the internal target, SLA the contract (always looser). Error budget: $99.9\% = 43$ minutes a month; $99.99\% = 4.3$; $99.999\% = 26$ seconds, at which no human can respond and recovery must be fully automatic. Each nine costs $\sim 10\times$ more. **And CARDINALITY is what actually kills metrics systems** — one `user_id` label turns 1.25M series into 12.5 trillion — **so no unbounded value is ever a label**. **Monitoring lives in a separate failure domain**, or the outage takes it with you.

Day 172 · system-design — Logging and distributed tracing

One identifier, minted at the front door, carried everywhere. The trace id goes in the `traceparent` header (W3C Trace Context) between services and in a **context object** (`contextvars`, `ThreadLocal`, `context.Context`, `AsyncLocalStorage`) inside one — and it goes on every log line. **A trace that stops halfway is almost always a hop nobody carried context across: a queue, a thread pool, a third party**.

A SPAN is one unit of work with a name, a duration, an id and a parent id. **A TRACE is every span sharing a trace id, as a tree**. Read the waterfall and ask **which bar is wide because it is working and which is wide because it is waiting** — the slow-looking service is usually just waiting for the real culprit.

Structured, not prose: `{"event":..., "trace_id":..., "duration_ms":...}`, because you cannot count sentences. **Mandatory fields: timestamp, level, service, trace id**. **NEVER log passwords, auth headers, tokens, full card numbers or one-time codes** — redact by field name in the library, because the real leak is somebody logging the whole request object during a bug hunt.

The arithmetic that decides everything. $100\text{M requests} \times 8 \text{ services} \times 3 \text{ lines} \times 300 \text{ bytes} = 720 \text{ GB/day of logs}$, against **9 GB/day of metrics** — **eighty times** — and over **\$100k/year** on per-gigabyte pricing. Traces unsampled are 600 GB/day; **1% head sampling plus every error and every p99 breach** brings it to $\sim 9 \text{ GB/day}$. Cut the log bill with levels, then retention tiers (7 days hot + 90 cold), then sampling successes, then a label-only store like Loki.

Metrics detect, traces localise, logs diagnose — in that order. Trace ids belong in logs and spans and NEVER in a metric label (100M series a day). Write logs to stdout and let a shipper forward them, never a network call on the request path; rotate the files, or a full disk takes the process down; and keep the whole stack in a separate failure domain from the product.

Day 173 · system-design — SLAs, SLOs, and error budgets

SLI is the MEASUREMENT (good events / valid events — a ratio, never a count). SLO is the INTERNAL TARGET plus a window (“99.9% over a rolling 28 days”). SLA is the EXTERNAL CONTRACT with money attached, and it is ALWAYS LOOSER than the SLO — same number means the first internal miss is already a penalty. Typical layering: SLA 99.9 / SLO 99.95 / alert at 99.97. Credits are a statement of seriousness, not insurance.

The table, from 43,200 minutes in a 30-day month. $99\% = 7h12m \cdot 99.9\% = 43.2 \text{ min} \cdot 99.99\% = 4.32 \text{ min} \cdot 99.999\% = 25.9 \text{ seconds}$. Twenty-six seconds is less time than a human needs to read an alert, so at five nines nothing human is in the recovery path — and each nine costs roughly $10\times$ the last.

The error budget is a RESOURCE, not an accident allowance. 99.9% of 3 billion monthly requests = 3,000,000 failures allowed; a 20-minute outage at 1,157 req/s spends 46% of the month in one incident. Spend it on risky deploys, migrations and experiments. Policy: $>25\%$ left ship freely; $<25\%$ no risky changes; 0% feature freeze — with a named override, or the rule dies the first time it is inconvenient. An unspent budget means you were too cautious or over-provisioned.

Alert on BURN RATE, or you find out at month end. burn = actual error rate / allowed. 1% errors against a 0.1% allowance = $10\times$ = budget gone in 3 days. Standard pair: $14.4\times$ for 1 hour (2% of budget) pages; $6\times$ for 6 hours (5%) tickets — the fast one catches outages, the slow one catches the 0.15% leak that eats the month invisibly.

IN SERIES YOU MULTIPLY AVAILABILITIES AND GET WORSE; IN PARALLEL YOU MULTIPLY FAILURE RATES AND GET BETTER. Six dependencies at 99.9% $\rightarrow 0.999^6 = 99.4\%$, four and a half hours a month; twenty hops caps you at 98%. Fix by making calls optional, caching the last good answer, or adding independent parallel replicas (two at 99% $\rightarrow 99.99\%$) — and “independent” is doing enormous work, because shared racks, pipelines and DNS correlate failures. 100% is the wrong target: unachievable, unaffordable, and invisible behind the user’s own connection.

Day 174 · system-design — Deployments: blue-green, canary, and rollback

Every deployment runs **TWO VERSIONS AT ONCE** against **ONE DATABASE** — that overlap is where everything goes wrong, so every change must be backward compatible with the version before it. **Five strategies**: **RECREATE** (downtime, fine for batch), **ROLLING** (free, no extra capacity, but rollback is as slow as rollout), **BLUE-GREEN** (rollback in seconds, costs $2 \times$ capacity, database still shared), **CANARY** (the only one that finds load-dependent bugs, and it is slow), **FEATURE FLAGS** (deploy $\neq$ release; instant rollback; costs flag debt, n flags = 2^n untested combinations).

The number that decides your culture: time from “this is bad” to “the old version is serving.” Under five minutes and you can take risks; an hour and you deploy rarely, which makes each deploy bigger and more dangerous. **ROLL BACK FIRST, INVESTIGATE AFTERWARDS** — understanding takes an hour, undoing takes minutes.

LIVENESS restarts the container; **READINESS** removes it from the load balancer and leaves it running. A service is alive long before it is ready — **only a liveness check means real users hit a cold instance**; wiring liveness to the slow condition gives a restart loop. **Do not make readiness depend hard on a shared dependency, or the whole fleet leaves the load balancer at once.** Drain connections for ~ 30 s: $1,000 \text{ req/s} \times 200 \text{ ms} = 200 \text{ in-flight per machine}$, so an undrained 10-machine rollout throws away 2,000 requests, and twenty deploys a month is **13% of a three-nines error budget**.

Canary arithmetic, which almost nobody has. 1% of 100M requests/day = 11.6 req/s, so detecting a **0.5% error rate takes ~ 29 minutes**, and a 0.05% one takes ~ 4.8 hours. **A ten-minute 1% canary detects an outage, not a regression** — hence the 1/5/25/50/100 ramp. **Compare against the stable version at the same moment, never a fixed threshold.**

You can run two versions of code; you cannot run two versions of data. **EXPAND AND CONTRACT**: add column $\rightarrow$ dual write $\rightarrow$ backfill in batches $\rightarrow$ read new $\rightarrow$ drop the old one weeks later. **Additive first, destructive last, with a gap long enough that rollback still works.** Backfill 500M rows in 10,000-row batches (~ 2 hours), never one **UPDATE**. **And no strategy rolls back an irreversible action** — a sent email, a taken payment, a consumed message, a dropped column — **so arrange for the irreversible step to be last, small, and separately controlled.**

Day 175 · system-design — Security in a design interview

AUTHENTICATION is who you are, answered **ONCE** at the edge. **AUTHORISATION** is what you may do, answered **ON EVERY REQUEST FOR EVERY OBJECT** — and that is where real breaches live. The classic bug is **insecure direct object reference**: a valid user asks for `/invoices/8813` and gets it. The fix is **not unguessable ids (that is hiding)**; it is `WHERE id = ? AND account_id = ?` in the **SAME** statement, and in multi-tenant systems **row-level security** below the application, so a missing filter returns nothing rather than everything.

Passwords: `bcrypt` / `scrypt` / `Argon2id`, salted, ~250 ms per hash. Never a fast hash. SHA-256 at 10^{10} guesses/s cracks an 8-character lowercase password ($26^8 = 209$ billion) in **21 seconds**; `bcrypt` at 4 guesses/s/core takes ~1,650 years. The salt stops one crack breaking every shared password. Length beats complexity — two more characters is $\times 676$. And 250 ms of CPU a stranger can spend is a DoS vector, so rate-limit login hardest, per account AND per address.

JWT = stateless scale, **NO REVOCATION** until expiry. **Session** = a lookup, instant revocation. The practical shape: **15-minute access token** + **revocable refresh token**, so a stolen token is worth 15 minutes and disabling an account takes effect within 15. **TLS everywhere including internally (mTLS between services)** — a hard shell with a soft inside fails the moment anything gets in. Encryption at rest protects against a stolen disk **AND NOTHING ELSE**, because the application holds the key; field-level encryption is the real defence and costs you indexing. No secrets in the repo — history is forever; prefer short-lived workload-identity credentials, and envelope encryption with a master key that never leaves the KMS.

The five that actually happen, with the mechanism: **SQL injection** → **parameterised statements** (query and values travel separately, so values are never parsed as SQL); **XSS** → **context-correct output encoding** + **CSP**; **CSRF** → **SameSite cookies** + a token; **SSRF** → an allowlist of destinations, never a blocklist; **credential stuffing** → **MFA**, per-account and per-address limits, breached-password checks. **WebAuthn** is phishing-resistant by construction — it will not sign for the wrong domain.

Defence in depth is an assumption: every layer will fail; the question is what the next one stops. Ask “if this service is taken over tonight, what can the attacker reach?” — not “do we trust it”. The **audit log** is **NOT** the **application log**: append-only, a separate store the app cannot rewrite, years of retention, and it records **denied** attempts, which carry the most signal. $5\text{M writes/day} \times 500\text{ B} = 2.5$

GB/day, 6.4 TB over seven years, ~\$147/month — cheap and non-negotiable. And the finding in most reviews is not a missing control; it is one that was quietly bypassed months ago while everything kept working.

Day 176 · system-design — Cost: the constraint nobody mentions

Cost is a design constraint, not a report. Four line items dominate: **COMPUTE** (usually only 30-50%), **STORAGE** (cheap, only grows), **DATA TRANSFER** (the surprise), **MANAGED SERVICES** (observability is often the largest single line) — plus a fifth that is not on the bill: **people**, which is why “build it ourselves to save money” is usually wrong.

Data transfer is where the shock is. **IN** is free; **OUT** to the internet is ~\$0.09/GB; **BETWEEN ZONES** is ~\$0.01/GB EACH WAY. Storing 30 TB for a month is ~\$690; sending it out once is \$2,700 — serving is 4× more expensive than storing. And cross-zone chatter appears on no architecture diagram: 8 internal calls × 100M requests × 20 KB = 16 TB/day, two-thirds crossing, both ways ≈ \$6,420/month — more than the entire saving from committing the compute. Fix it with zone-aware routing, not by collapsing to one zone.

Buy compute three ways at once: **COMMIT** the baseline (~40% off for 1 year, 60-70% for 3, and you pay whether you use it), **ON-DEMAND** for the peak, **SPOT** for anything interruptible (70-90% off, two minutes’ notice, never a stateful leader). Serverless is cheapest idle and dearest busy. Commit to the floor, never the peak.

Optimise in order of effort, because the free things are bigger. 1. Delete what is unused. 2. Switch off non-production outside working hours (used 45 of 168 hours — ~73% saving, and it is Ramesh’s empty fridge). 3. Right-size. 4. Commit. 5. Tier storage. 6. Stop the bytes moving — CDN, compression, co-location. 7. Change the architecture — **LAST**, and where everyone starts. A worked example goes \$29,500 → \$10,500, a 65% cut, with no product code changed.

You cannot reduce what you cannot see. Tag everything and enforce it with a policy that refuses untagged resources — the “untagged” row is where abandoned projects live. Showback changes behaviour almost as much as chargeback with far fewer arguments. A cost bug pages nobody, so it runs until the invoice — set a daily spend alert per tag on the **RATE** of change, and price changes in the pull request. Track cost per DAU and per 1,000 requests: the direction matters more than the total, and a bill growing faster than usage is a leak. And before buying a nine, ask what an hour of downtime actually costs.

Day 177 · system-design — Microservices versus monolith, argued both ways

A **MONOLITH** is one deployable with one transaction boundary; **MICROSERVICES** are many deployables, each **OWNING ITS OWN DATA**. The data ownership is the definition — many services sharing one database is a **distributed monolith**: every cost of the split, none of the independence. **Never split by technical layer; split by business capability**.

FOR (each with a condition): independent DEPLOYMENT (the real one — a release train several teams contend for), **independent SCALING** (only when profiles differ $\sim 10\times$), **FAULT ISOLATION** (only if the calls are optional), **TEAM AUTONOMY** (Conway's law: the architecture fights the org chart and the org chart wins). **AGAINST: you LOSE TRANSACTIONS** (sagas, compensations, an observable inconsistency window, refunds that are not inverses, idempotency everywhere — paid on every future feature); **availability MULTIPLIES DOWNWARD**; **every call is $\sim 100,000\times$ slower and can now fail, time out or succeed twice; ~ 3 hours/service/month of upkeep, forever**.

The arithmetic to have ready. 8 services at 99.9% = $0.999^8 = 99.2\% \rightarrow 5\text{h } 44\text{m/month}$ instead of 43 minutes, with nothing built worse. In-process call $\sim 10\text{ns}$ against a network call $\sim 1\text{ms} \rightarrow 8$ calls add 8 ms to every request. 16 TB/day of internal traffic $\approx \$6,420/\text{month cross-zone}$, where function calls cost nothing. **40 services $\times 3\text{h} = 120\text{h/month} = 0.75$ of an engineer on upkeep alone**. And relationships are $n(n-1)/2$ — 6 services is 15 pairs, 40 is 780.

The usual right answer is the **MODULAR MONOLITH**: one deployable and one transaction, with hard internal boundaries enforced by tooling. Shopify does it deliberately; **Segment split and publicly moved back. Migrate with the STRANGLER FIG — a facade, then one capability at a time, each step reversible — never a rewrite**. And there is a **fixed platform cost that arrives with service number two**: tracing, per-service metrics and pipelines, service discovery, contract tests, and a laptop story.

Team count decides it, not taste. 1 team $\rightarrow$ monolith. 3 teams $\rightarrow$ modular monolith. 8 teams blocked on one release train $\rightarrow$ extract along team lines. 50 teams $\rightarrow$ microservices plus a platform team. **A service per TEAM, never per developer, and never more services than there are people to own them. Splitting one thing into six gives you six things and a problem between every two of them — so every split must be paid for by a specific pressure, and “we might be big one day” does not pay for it**.

Day 178 · system-design — The system design interview framework, memorised

Six beats, fixed order, because each one's output is the next one's input. 1 REQUIREMENTS (5) · 2 ESTIMATION (5) · 3 INTERFACE (5) · 4 DATA MODEL (5) · 5 ARCHITECTURE (10) · 6 DEEP DIVE (10-15). **Announce the plan in the first fifteen seconds** — it puts you in control and lets the interviewer redirect you before you waste twenty minutes. **Beats 1-5 are deliberately shallow; beat 6 is deliberately deep.** The two classic failures are **too broad** (40 minutes of boxes, nothing examined) and **too narrow** (30 minutes on the cache before anyone agreed what the product does).

Requirements has two halves and candidates give only the first. Functional: **3-5 features IN and the rest explicitly OUT** — naming what you exclude shows you can scope. Non-functional: **scale, read:write, latency, availability, CONSISTENCY, durability.** **Push hardest on consistency** — “must a post appear instantly?” is a product question that changes the whole architecture.

Estimation is “how many plates”, and every number must be followed by “so...”. DAU → actions → $\div 86,400$ (call it 100,000) → **peak** $\times 2-3$ → bytes → storage/year → bandwidth. **100M DAU $\times$ 20 reads = 23,000 reads/s against 230 writes/s = 100:1 — SO cache hard, read replicas before sharding, fan out on write.** 20M photos $\times$ 2 MB = **40 TB/day**, so object storage and a CDN, never a database. **700 peak writes against $\sim 5,000$ /s for one Postgres primary — SO do not shard yet, and say that deliberately.**

Memorise the reference numbers: memory 100 ns · SSD 100 μ s · same-zone network 0.5 ms · cross-region 50-150 ms · one app machine $\sim 10,000$ req/s · one Postgres $\sim 5,000$ writes/s · one Redis $\sim 100,000$ ops/s · photo 2 MB · row ~ 1 KB · egress \$0.09/GB. They turn “should I shard?” into an eight-second decision.

Design the data model from ACCESS PATTERNS, not a tidy entity diagram; walk the read path and the write path out loud (a drawing nobody walks through is just boxes); **offer the interviewer a choice of deep dive.** **Close with the checklist almost nobody reaches:** monitoring and what pages, the SLO in minutes with its error budget, auth and what you would refuse to log, a monthly cost, what happens when each box dies — and one honest weakness of your own design. *“The fan-out worker fails silently, so I would alert on queue lag.”* A design with no named weakness is a design nobody looked at.

Day 179 · system-design — Full mock: one high-level design, one low-level design

Two rounds, two different exercises. HIGH-LEVEL: six beats — requirements, estimation, interface, data model, architecture, deep dive — with every number followed by “so...”, and beats 1-5 deliberately shallow so you reach the deep dive with fifteen minutes. LOW-LEVEL: six DIFFERENT beats — requirements as VERBS, entities, relationships, interface, the part that will change, and the EXTENSION question. Almost no arithmetic, and beat 6 IS the round.

The reset between them is Sharada’s cloth, and it is sixty seconds. Physically stop · ONE sentence about the round that finished · ONE thing to change · and NAME THE NEXT ROUND OUT LOUD. The commonest failure in a back-to-back loop is running the wrong framework — estimating queries per second for a vending machine, or drawing boxes when they asked for classes. “Forty pots is not one long job.”

Video streaming, and the one number that is the architecture: 100M viewers × 5 videos × 10 min at ~2 Mbit/s = ~75 PB/day delivered, ~870 GB/second — \$6.75 MILLION A DAY at direct egress. So the CDN is in the design at minute one. Uploads go straight to object storage via a pre-signed URL (460 MB/s never touches your servers). Chunk before transcoding — a 2-hour film is 480 jobs, five minutes instead of eight hours — at the cost of keyframe-aligned boundaries, idempotent jobs (workers die; deterministic output paths), and a completion tracker that fails SILENTLY. Alert on the age of the oldest video stuck in transcoding. Interruptible + idempotent = spot instances, 70% off.

Expense sharing, and the one decision that is the round: put split types behind a SplitStrategy interface. Adding percentage splits is then a new class and zero edits — where if kind = 'EQUAL' ... elif ... means every addition edits a method two working variants depend on. Justify the pattern by the signal (“you told me there are already three”), or it is over-engineering. Derive balances rather than store them — stored balances drift when an old expense is edited. Enforce sum(splits) = amount in the CONSTRUCTOR, so an invalid expense cannot exist.

Money: Decimal, never float (0.1 + 0.2 ≠ 0.3), and when shares do not divide exactly the remainder goes to a NAMED person — 10.00 split three ways is 9.99, and a missing paisa across a million expenses is ten thousand rupees of support tickets. Close both rounds the same way: what fails, what you would monitor, and one honest weakness of your own design. A design with no named weakness is one nobody looked at.

Day 180 · system-design — Final revision, and the week before the interview

Two running orders, and say which round you are in within the first minute. HLD: requirements (3-5 in, the rest explicitly OUT; scale, read:write, latency, availability, CONSISTENCY) · estimation (every number followed by “so...”) · interface · data model from ACCESS PATTERNS · architecture, then WALK both paths · deep dive, offered as a choice. LLD: verbs · entities · relationships and the invariant in the constructor · interface · the thing that will CHANGE · EXTENSION, which is the round. Close both with monitoring, the SLO in minutes, security, cost, failure per box, and ONE HONEST WEAKNESS OF YOUR OWN.

The numbers, cold. 1 day $\approx$ 100,000 s. memory 100 ns · SSD 100 μ s · same-zone 0.5 ms · cross-region 50-150 ms — **and an in-process call is $\sim$ 10 ns, so a network hop is 50,000-100,000 $\times$ slower.** One machine $\sim$ 10,000 req/s · Postgres $\sim$ 5,000 writes/s · Redis $\sim$ 100,000 ops/s. Row $\sim$ 1 KB · photo $\sim$ 2 MB · 1080p $\sim$ 50 MB/min. Storage \$0.023/GB-mo · EGRESS \$0.09/GB · cross-zone \$0.01/GB EACH WAY — serving 30 TB costs $4\times$ storing it. 99.9% = 43 min/month, 99.99% = 4.3, 99.999% = 26 seconds. In series you multiply availabilities and get worse; in parallel you multiply failure rates and get better.

Ten trade-off pairs, argued BOTH ways in thirty seconds each — SQL/NoSQL, strong/eventual, sync/async, normalise/denormalise, fan-out on write/read, cache-aside/write-through, monolith/microservices, stateful/stateless, push/pull, committed/on-demand. The second option is never “the modern one” — it is right under different conditions, and naming the condition IS the answer.

Revise design by DESIGNING, not reading. Forty-five minutes, out loud, timed, alone — then compare, and note only what you did not think to ask. **Nothing new after two days out:** a case study first read on Friday becomes a half-remembered version of somebody else’s answer, which is worse than reasoning from the building blocks. If you can say what each of the fifteen building blocks is for, what it costs, and what happens when it dies, you can design any of them.

“Do you have any questions for us?” is part of the interview, and “no, you’ve covered everything” is a real negative. Ask about the first ninety days (is the work defined, or is defining it the job), rollback time (the number that decides daily engineering life), pages per week (the best available measure of system health), and “what did you not expect before you joined?” Then listen the way Latha’s mother listened — the way an answer comes back tells you as much as the answer. Write down every question you were asked within the hour, and ask what the next step is and when.

The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
171	Binary, bits, and why they matter	Monitoring, metrics, and alerting
172	The bit tricks every interview uses	Logging and distributed tracing
173	XOR problems	SLAs, SLOs, and error budgets
174	Primes, GCD, and modular arithmetic	Deployments: blue-green, canary, and rollback
175	The combinatorics you actually need	Security in a design interview
176	Bits and maths revision and mock round	Cost: the constraint nobody mentions
177	The twenty patterns, on one page	Microservices versus monolith, argued both ways
178	How to think out loud in a coding round	The system design interview framework, memorised
179	Full mock: two problems, forty-five minutes	Full mock: one high-level design, one low-level design
180	Final revision, and the week before the interview	Final revision, and the week before the interview

Glossary

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

4

4 kb [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

A

a replicated log [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

abstract class [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

acid [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [day 033](../day-033-window-with-a-map/README.md).

adaptee [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

adapter [Day 059](#) — An adapter is an implementation that translates to a specific technology (PostgresOrderRepository).

add a tie-breaker [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

algorithm [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

always comparable [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

amortised [Day 005](#) — The word for that is amortised: list.append is O(1) amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

anaemic model [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

and a direction [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

anti-diagonals [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of $r + c$.

anti-sniping [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

approximate [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

array [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [\[day 009\]\(../day-009-what-an-array-is/README.md\)](#).

artefact [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

assignment rule [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

at-least-once [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

atomic [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

atomicity [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

availability [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that **every** non-failing node answers.

aws ecs [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

B

backend [Day 002](#) — The backend is the code that never leaves the kitchen.

bearer token [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

best case [Day 002](#) — The best case is 1, the worst case is n.

binary tree [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

blast radius [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

block [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

blocked [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

both [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

boundary [Day 118](#) — The unifying idea worth stating: **two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.**

bounded context [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

bounded optimism [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

breadth-first [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [\[tomorrow\]\(../day-101-bfs-level-order/README.md\)](#).

break point [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

broken links [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

bucket by time [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel_id, month).

bucket counts [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's histogram_quantile does, and it is why counters and histograms are the metric types that aggregate and...

bulkhead [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

burst size [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

C

cache [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

cache it [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

cache stampede [Day 090](#) — And the first cold morning is a cache stampede.

candidates [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

capacity [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

cdn [Day 070](#) — CDN — a geographically distributed caching proxy.

certificate [Day 006](#) — When your browser connects to google.com, the machine sends back a certificate — a small document containing the name it claims (google.com), a public key, an expiry date, and a signature from a Certificate Authority.

chaining [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

channels [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

checks permission [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

chunks [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

circular buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

class [Day 043](#) — A class is the description; an object is one thing made from that description.

client [Day 068](#) — The client — the wall socket, which will only accept one shape.

collapse key [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

collapsing writes [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

collection [Day 016](#) — A collection is the set of things: /bags.

column [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

compact [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

comparator [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

comparison [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

complete [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

component [Day 069](#) — The tumblers are the component — the real object that does the actual job.

composite [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

composite key [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

composition [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

composition root [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually main() or a startup module.

connection [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([[day 006](#)](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([[day 008](#)](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

connection-oriented [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

consequence [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

consistency [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

constraints [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

constructor [Day 044](#) — `_init_` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

container [Day 013](#) — A container is an ordinary process — the same kind of process you met on [[day 008](#)](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

container image [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

containerd [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

context [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

coordinator [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

correlation id [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

cost [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

count map [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

counter [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

cri-o [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

critical path [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

cycle [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

D

dangerous [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

dashed [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

dau [Day 097](#) — DAU — daily active users.

deep-copied [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

deleted [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

delta of delta [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

dependency [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

depth [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

depth-first [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

deque [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [\[day 074\]\(../day-074-deques-and-window-max/README.md\)](#) — holding indices, arranged so that their values are decreasing from front to back.

diminishing returns [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

direct addressing [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

domain name [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

draw two diagrams [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

duck typing [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

dummy node [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

E

earlier [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

edge cases [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

edge compute [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

effect [Day 113](#) — Say it that way: "exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key." Systems that advertise exactly-once are doing precisely this.

elastic scaling [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

election timeout [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

empty [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

etl [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

event object [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`.

eventual consistency [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

exactly [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

exactly correct [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

exactly one [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

exclusive lock [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

expiry [Day 090](#) — The dates are expiry — a completely separate mechanism.

F

facts [Day 082](#) — So: a FinePolicy. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

false negative [Day 124](#) — A false negative is failing to notice a machine that really has died.

false positive [Day 124](#) — A false positive is declaring a healthy machine dead.

fanout [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

fee [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day.*

fifo [Day 090](#) — FIFO — evict the oldest inserted.

file [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

filled diamond [Day 050](#) — A filled diamond means *composition* — the parts die with the whole, so Order ♦ — OrderLine because a line item has no meaning without its order.

finite state machine [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

float [Day 081](#) — Devi's second tin is a float: coins reserved so tomorrow can start.

four-direction move [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

fragile [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently.

frontend [Day 002](#) — The frontend is the code that crosses the counter.

G

galloping mode [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

gap [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

generator expression [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

greedy simulation [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

guest operating system [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

H

hash map [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

hash ring [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

hateoas [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.

header interface [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

heap [Day 117](#) — The six letters on the table are the heap: one candidate per list.

heapsort [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

heartbeat [Day 124](#) — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

hidden spof [Day 111](#) — The shared approach road is a hidden SPOF — two components that are not independent.

hiding mechanism [Day 052](#) — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

highest node [Day 104](#) — Every path has exactly one highest node — the point where it turns around.

historical snapshot [Day 029](#) — | Copy | Instead of | Why | |---|---|---| | posts.comment_count | COUNT(*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer_name | joining to customers | a historical snapshot: the name at the time of the order | | order_items.unit_price | joining to products | the price *then*, which must...

hollow triangle [Day 050](#) — Three: inheritance — a line with a hollow triangle pointing at the parent:

hook [Day 075](#) — award is a hook: the base class implements it as *doing nothing*, so a subclass may override it and most do not.

horizontal scaling [Day 098](#) — Horizontal scaling means putting more machines beside it and dividing the work.

hypervisor [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

I

image [Day 013](#) — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

implicit graph [Day 130](#) — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [day 125](../day-125-what-a-graph-is/README.md).

in parallel [Day 111](#) — Components in parallel — any one will do — multiply their *failure* probabilities, which is a completely different shape.

in series [Day 111](#) — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

in-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

index [Day 025](#) — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

indexed heap [Day 114](#) — Hence the indexed heap: a hash map from value to index, updated on every swap.

inheritance [Day 071](#) — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

inorder [Day 110](#) — The standing order is inorder: left subtree, then root, then right subtree.

input [Day 008](#) — In a problem: the input — its type, its size, and what is in it.

integrity [Day 006](#) — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

intent plus multiplicity [Day 070](#) — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

interface [Day 052](#) — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

invariant [Day 028](#) — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

is possible [Day 046](#) — $> *$ Find the smallest X such that some task is possible. $*$ $> > *$ Find the largest X such that some condition still holds. $*$

item [Day 016](#) — An item is one of them: `/bags/213`.

iterator [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

J

json [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [[day 007](#)](../day-007-space-complexity/README.md); it looks like `{"id": 42, "name": "Zoya"}`.

jump [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

jwt [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from **somebody else**, so a user can log in with Google without your service ever...

K

k closest points [Day 116](#) — k closest points — the same shape with a computed key.

k most frequent [Day 116](#) — k most frequent — count first, then top-k on the counts.

k-th largest specifically [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

key [Day 023](#) — The key is as much of the design as the algorithm.

key property [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

L

lag [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

largest [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position $k - 1$; - the 1st largest is at position $n - 1$, so the k-th largest is at position $n - k$.

latency [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

lazy [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

lazy deletion [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

lazy loading [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

lazy refill [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

leader [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

leaf [Day 098](#) — He is a leaf — an item with no children.

lease [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

ledger entries [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

left [Day 040](#) — Remove the strip to the left: `prefix[r2+1][c1]`.

lfu [Day 090](#) — LFU — evict the least frequently used.

linear scan [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

linearizability [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

list [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

list of lists [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

load balancer [Day 098](#) — Sending students to the right counter is a load balancer.

load factor [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

load shedding [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

log-structured merge tree [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

logical cohesion [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

logical time [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by **causality** rather than by wall time.

loop [Day 001](#) — A loop is a statement that says, "do the following again and again".

loop body [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

low cohesion [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

lru [Day 090](#) — LRU — evict the least recently used.

M

manifest [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS `.m3u8` or DASH `.mpd` — listing every rendition and every segment.

many times [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

many-to-many [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

matrix [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

mau [Day 097](#) — MAU — monthly active users.

max-heap [Day 116](#) — The mirror rule: to find the k smallest, keep a max-heap of size k — in Python, a min-heap of negated values.

method resolution order [Day 052](#) — The Python answer. The method resolution order — `D._mro_` — is computed by a rule called C3 linearisation, and `super()` follows that order rather than jumping straight to a parent.

milliseconds [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

min-heap [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

minimum spanning forest [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

mixin [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

monotone [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

monotonic deque [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is $O(n)$ overall.

N

name mangling [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

namespace [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

negative [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [\[day 037\]\(../day-037-prefix-sums/README.md\)](#).

negative infinity [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

negatives [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

nested loop [Day 001](#) — A nested loop is a loop inside another loop.

network partition [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

never equal [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

never forgets [Day 090](#) — Needs a third structure and a `min_frequency` counter to stay $O(1)$, and it never forgets: an entry that was hot last month holds a huge count and will not leave.

never persisted [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

no children [Day 121](#) — It has no children — nothing continues past it.

node [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

not a map [Day 067](#) — | not a map — heap or sorted array |

O

$o(1)$ [Day 079](#) — tail — a reference to the last node, which makes append $O(1)$ instead of $O(n)$.

object [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

observers [Day 072](#) — The objects on the list are the observers.

occupied [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

offset [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

on first touch [Day 041](#) — Each `post.author` is a *different table*, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

one-shot [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

opens [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

orchestrator [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, `systemd` on a virtual machine.

origin [Day 101](#) — The storeroom is the origin — the source of truth.

origin pull [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

orphan rows [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

orphans [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

out-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

overloading [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

overriding [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

P

page [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

parallel edges [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then A—B twice genuinely **is** a cycle, and skipping "the parent vertex" wrongly ignores it.

parameters [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

parent [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

partial failure [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, *"I do not know"*, and almost every difficult problem in the rest of this phase is a consequence of it.

participants [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

partition key [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows **sorted inside** that bucket.

partitions [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

path [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

port [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

prefix sums [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

preorder [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

priority queue [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

procedure [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

process [Day 008](#) — A process is a running program with its own private memory.

program [Day 001](#) — A program is a sequence of statements.

protecting state [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

Q

qps [Day 097](#) — QPS stands for **queries per second** — the number of requests arriving at your system each second.

queue [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [day 73](../day-073-queues/README.md).

quorum [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [day 117](../day-117-merge-k-sorted/README.md).

R

race condition [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

ram [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

rank [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

rebalancing [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

rebuilding the tree [Day 109](#) — Repacking the whole cart is rebuilding the tree — $O(n)$.

recursion [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

references [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

refill rate [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

register [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

relative epsilon [Day 048](#) — The fix is a relative epsilon — stop when the gap is small *compared to the value*:

removal [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

replica [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

replica divergence [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

request-aware routing [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

resolver [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

resources [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

returned [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."*

ring buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position $7 + 1$ is position 0.

role interface [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

root [Day 093](#) — The root is the loft itself, written /.

rotation [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

routine [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

row [Day 026](#) — A row is one of those things.

run [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

S

same remainder [Day 038](#) — A stretch is divisible by k when the prefixes at its two ends leave the same remainder — so carry running $\% k$ and count matching remainders.

schema [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

scoping [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are *not* building.

script [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

seam [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

search space [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

selectivity [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

sentinel [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

separator [Day 019](#) — The string before the `.join` is the separator — `"".join` glues with nothing between, `",".join` puts a comma and a space between each pair.

sequence diagram [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

serialisable [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

service credits [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

session [Day 020](#) — A session is a record the server keeps about a logged-in user.

session guarantees [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

set [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

shallow-copied [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a *reference* to the same arrangement, and changing it from either end changed it for both.

shape [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.

shard [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

shared lock [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

shared store [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

singleton [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

singly linked list [Day 076](#) — A singly linked list — where each node knows only the **next** one — cannot do that.

slice [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

smallest [Day 046](#) — `>` **Find the smallest X such that some task is possible.** `>` **Find the largest X such that some condition still holds.**

snapshot [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

snipe [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

sorted region [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in **which end** the region grows from and **how** the next value gets there.

sorting everything [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

splits [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

spof [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

squares [Day 003](#) — | Example | `|---|---|---|` | $O(1)$ | constant | does not change | reading items[5] | | $O(\log n)$ | logarithmic | goes up by one step | halving until you reach 1 | | $O(n)$ | linear | doubles | one loop over the list | | $O(n \log n)$ | linearithmic | slightly more than doubles | sorting | | $O(n^2)$ | quadratic | goes up four times | every...

stable [Day 051](#) — Python's sort is Timsort: $O(n \log n)$ worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$.

stack [Day 075](#) — The fridge is a stack: last in, first out.

stack frame [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

stack overflow [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

state [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

statements [Day 001](#) — A program is a sequence of statements.

status [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

still one machine [Day 112](#) — | Move | Price | `|---|---|` | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

stonith [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

strategy [Day 071](#) — Strategy turns an algorithm into a value.

strictly shorter [Day 072](#) — `left[i]` — the index of the nearest bar to the left that is strictly shorter.

structural [Day 058](#) — In Python, typing.Protocol is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing ([\[day 052\]\(../day-052-quadratic-sorts/README.md\)](#)).

subarray [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

subject [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

subscription list [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

substring [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

subtree [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

surge multiplier especially [Day 088](#) — The surge multiplier especially is frozen.

swap [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

system call [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

T

table [Day 026](#) — A table holds one kind of thing.

tail call [Day 088](#) — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

tail-recursive [Day 090](#) — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

tcp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

template [Day 092](#) — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

test double [Day 053](#) — A test double is a stand-in you pass in during a test.

the return address [Day 088](#) — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

the running median [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and Λ^* — the nearest unvisited node - top-k problems — [\[day 116\]\(../day-116-top-k/README.md\)](#) - merging k sorted lists — [\[day 117\]\(../day-117-merge-k-sorted/README.md\)](#) - the running median — [\[day 118\]\(../day-118-two-heaps/README.md\)](#), with two heaps -...

third table [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: `posts.author_id` | | many-to-many | a post has many tags, a tag has many posts | a third table: `post_tags(post_id, tag_id)` | | one-to-one | a user has one profile | either side; usually the optional one |

thread [Day 008](#) — A thread is one line of execution inside a process, and all the threads in a process share that memory.

three distinct orderings [Day 092](#) — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1].

three-way partition [Day 054](#) — The fix is a three-way partition: split into **less than**, **equal to**, and **greater than**, and recurse only into the outer two.

tight coupling [Day 061](#) — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

time-to-live [Day 037](#) — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

tiny constraint [Day 097](#) — A tiny constraint: $n \leq 8$ means factorial, $n \leq 20$ means 2^n , target ≤ 500 with small candidates means combination sum.

toast [Day 134](#) — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

token [Day 019](#) — The server verifies them and issues a token — a session id or a signed token.

tombstone [Day 011](#) — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

top-k problems [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

total order [Day 123](#) — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

transitions itself [Day 071](#) — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

transitive dependency [Day 029](#) — This is a transitive dependency: `id` → `department_id` → `department_name`.

tree [Day 040](#) — A domain of independent records — a tree — tolerates anything.

ttl [Day 003](#) — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

tuple [Day 005](#) — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

two [Day 087](#) — In fib, each call makes two — so the work explodes.

two implementations [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

U

udp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

under-fetching [Day 021](#) — And from [[day 015](#)](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the $N+1$ problem when it happens in a list.

union filesystem [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

upper bound [Day 044](#) — The upper bound is the first index where $\text{nums}[i] > 4$, which is index 4 — one past the run.

uri [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

url [Day 001](#) — A URL is an address for something on the internet.

V

value object [Day 051](#) — A value object is defined entirely by its values.

version [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

vertex [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

vertical scaling [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

victim [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

virtual address space [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

virtual machine [Day 013](#) — A virtual machine is an emulated computer.

volatile [Day 009](#) — It is also volatile: switch the power off and it is gone.

W

weight [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

which class [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

which is hours [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

within [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

worst case [Day 002](#) — The best case is 1, the worst case is n .

write skew [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

write-around with invalidation [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

write-through [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.

Y

yes [Day 093](#) — | Problem | Recurse on | Duplicates in input | Extra rule | $|-|---|---|---|$ | Combinations (LC 77) | $i + 1$ | no | stop at size k | | Combination Sum (LC 39) | i — reuse allowed | no | stop when remaining $= 0$ | | Combination Sum II (LC 40) | $i + 1$ — each used once | yes | sort, and skip when $i > \text{start}$ and $c[i] = c[i-1]$ | |...

References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

FOR THE WHOLE BOOK

Python Software Foundation. *The Python Language Reference and Standard Library*. <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

Python Wiki. *TimeComplexity - cost of the built-in container operations*. <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

David Galles, University of San Francisco. *Data Structure Visualizations*. <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

Steven Halim and others. *VisuAlgo - visualising data structures and algorithms*. <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

MIT OpenCourseWare. *6.006 Introduction to Algorithms*. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

LeetCode. *Problem set*. <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

DATA STRUCTURES AND ALGORITHMS, BY PART

PART XVIII · BITS AND MATHS (DAYS 171–176)

Python Software Foundation. *Bitwise operations on integers*. <https://docs.python.org/3/library/stdtypes.html#bitwise-operations-on-integer-types> [PSF Licence]

Python integers are arbitrary precision, which changes some classic tricks.

Sean Eron Anderson, Stanford. *Bit Twiddling Hacks*. <https://graphics.stanford.edu/~seander/bithacks.html> [Public domain]

The reference collection. Explicitly placed in the public domain by its author.

Wikipedia. *Modular arithmetic*. https://en.wikipedia.org/wiki/Modular_arithmetic [CC BY-SA]

Behind the $10^{*9} + 7$ that appears throughout competitive problems.

PART XIX · FINAL MOCKS AND REVISION (DAYS 177–180)

Google Careers. *How we hire - interview preparation*. <https://www.google.com/about/careers/applications/how-we-hire/> [Free to read]

What one large employer says publicly about its own process.

Wikipedia. *Spaced repetition.* https://en.wikipedia.org/wiki/Spaced_repetition [CC BY-SA]

The evidence behind the recall cards in Appendix B.

SYSTEM DESIGN, BY PHASE

RELIABILITY, SECURITY, AND THE INTERVIEW ITSELF (DAYS 171-180)

Google. *The Site Reliability Workbook.* <https://sre.google/workbook/table-of-contents/> [Free to read online]

SLIs, SLOs and error budgets, with worked examples.

OWASP. *OWASP Top Ten.* <https://owasp.org/www-project-top-ten/> [CC BY-SA]

The vulnerability list a security question is almost always drawn from.

OWASP. *Cheat Sheet Series.* <https://cheatsheetseries.owasp.org/> [CC BY-SA]

Authentication, session management and input validation, in checklist form.

Mozilla. *MDN - Web security.* <https://developer.mozilla.org/en-US/docs/Web/Security> [CC BY-SA 2.5]

CORS, CSP and same-origin, explained without hand-waving.

Prometheus Authors. *Prometheus documentation.* <https://prometheus.io/docs/introduction/overview/>
[Apache 2.0]

Metric types and alerting rules, for the monitoring chapters.

Colophon: how this book was made

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

TYPE, TOOLS AND LIBRARIES

SIL International, after Matthew Carter. *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

Bold Monday for IBM. *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

JetBrains. *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]

All code, output and ASCII diagrams.

Google. *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]

The Devanagari on the covers and title page.

Knut Sveidqvist and contributors. *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]

Renders every flow chart, sequence diagram and tree.

Georg Brandl and contributors. *Pygments*. <https://pygments.org/> [BSD 2-Clause]

Colours the code. The palette used here was written for this book.

Waylan Limberg and contributors. *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]

Turns the lesson sources into the pages you are reading.

Martin Thoma and contributors. *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]

Joins the covers, front matter and body, and writes the bookmarks.

The Chromium Authors. *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]

The typesetting engine. No proprietary software was used to make this book.

HOW A PAGE WAS MADE

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

Problem index

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
171	Number of 1 Bits	LeetCode 191 (Easy)	The naive loop, then $n \& (n-1)$.
171	Power of Two	LeetCode 231 (Easy)	The one-liner, and the guard everyone omits.
171	Counting Bits	LeetCode 338 (Easy)	Bits meeting DP — the recurrence is one line.
171	Reverse Bits	LeetCode 190 (Easy)	Fixed width, and why Python needs to be told it.
171	Sum of Two Integers	LeetCode 371 (Medium)	Addition from AND, XOR and shift — and Python's traps.
171	Bitwise AND of Numbers Range	LeetCode 201 (Medium)	The common prefix, and why the answer is a shift.
172	Hamming Distance	LeetCode 461 (Easy)	That you know XOR means "different". One line.
172	Counting Bits	LeetCode 338 (Easy)	The one-line recurrence, and why the smaller answer is already there.
172	Reverse Bits	LeetCode 190 (Easy)	Fixed width, and why Python has to be told what the width is.
172	Subsets	LeetCode 78 (Medium)	A subset is a number. The iterative bitmask answer, not backtracking.
172	Single Number	LeetCode 136 (Easy)	Pairs cancel. Warm-up for tomorrow.
172	Maximum Product of Word Lengths	LeetCode 318 (Medium)	A word as a 26-bit mask, and "no shared letters" as $a \& b == 0$.
173	Single Number	LeetCode 136 (Easy)	Pairs cancel, and that the constraint named the technique.
173	Missing Number	LeetCode 268 (Easy)	Supplying the partners yourself, and why XOR beats the sum.
173	Single Number III	LeetCode 260 (Medium)	The split bit. This is the one that separates people.
173	Single Number II	LeetCode 137 (Medium)	Counting modulo three, and Python's sign problem.
173	Set Mismatch	LeetCode 645 (Easy)	One doubled, one missing — the split trick again, in disguise.
173	Maximum XOR of Two Numbers in an Array	LeetCode 421 (Medium)	Greedy from the top bit, with a set lookup instead of a second loop.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
174	Count Primes	LeetCode 204 (Medium)	The sieve, and why you start at $p \times p$.
174	Greatest Common Divisor of Strings	LeetCode 1071 (Easy)	That Euclid is about structure, not just numbers.
174	Ugly Number II	LeetCode 264 (Medium)	Factors as generators, and the three-pointer merge.
174	Pow(x, n)	LeetCode 50 (Medium)	Square and multiply, plus the negative-exponent edge case.
174	Super Pow	LeetCode 372 (Medium)	Fast power with the exponent given as digits.
174	Fraction to Recurring Decimal	LeetCode 166 (Medium)	Remainders repeating, which is modular arithmetic in disguise.
175	Unique Paths	LeetCode 62 (Medium)	That a path is a choice of which steps go down.
175	Pascal's Triangle II	LeetCode 119 (Easy)	One row in $O(n)$ space, by the multiplicative rule.
175	Unique Binary Search Trees	LeetCode 96 (Medium)	Catalan, arrived at through the split-and-multiply recurrence.
175	Combinations	LeetCode 77 (Medium)	Generating them, not counting them — and the size of the output.
175	Count Sorted Vowel Strings	LeetCode 1641 (Medium)	Stars and bars in disguise, or a four-line DP.
175	Number of Ways to Divide a Long Corridor	LeetCode 2147 (Hard)	The multiplication principle over independent gaps, modulo a prime.
176	Bitwise AND of Numbers Range	LeetCode 201 (Medium)	That a range of numbers shares a binary prefix.
176	Single Number II	LeetCode 137 (Medium)	Noticing that pairing fails, and counting modulo three.
176	Closest Prime Numbers in Range	LeetCode 2523 (Medium)	Per group, not per number — plus one fact about prime gaps.
176	Count Ways to Make Array With Product	LeetCode 1735 (Hard)	The capstone: sieve, factorise, stars and bars, modular inverse.
177	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Sliding window, and the "seen before the window started" line.
177	Koko Eating Bananas	LeetCode 875 (Medium)	Binary search on the answer — the pattern people fail to recognise.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
177	Daily Temperatures	LeetCode 739 (Medium)	Monotonic stack, and that it holds positions rather than values.
177	Course Schedule	LeetCode 207 (Medium)	That "prerequisites" means a graph, and cycles mean impossible.
177	Subarray Sum Equals K	LeetCode 560 (Medium)	Why this is prefix sums and NOT a sliding window.
177	Coin Change	LeetCode 322 (Medium)	Naming the state before writing the loop, and why greedy fails.
178	Longest Consecutive Sequence	LeetCode 128 (Medium)	The full arc: brute force, its cost, the one-line fix, and why it is linear.
178	Product of Array Except Self	LeetCode 238 (Medium)	Clarifying the constraint — no division, $O(1)$ extra — before writing.
178	Merge Intervals	LeetCode 56 (Medium)	Stating the sort key and its reason before touching the loop.
178	Word Break	LeetCode 139 (Medium)	Naming the state out loud before writing the DP.
179	Task Scheduler	LeetCode 621 (Medium)	Simulating first, then noticing the structure — and saying the sentence between them.
179	Count of Smaller Numbers After Self	LeetCode 315 (Hard)	Recognising merge sort doing an unfamiliar job.

H

Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

#

4 kb, 324, 599, 600, 611

A

a replicated log, 599
abstract class, 599
acid, 383, 385, 599, 600, 617
adaptee, 599
adapter, 599
add a tie-breaker, 599
algorithm, 14, 26, 65, 83, 170, 176, 194, 196, 345, ...
always comparable, 599, 610
amortised, 599
anaemic model, 599
and a direction, 599
anti-diagonals, 599
anti-sniping, 599
approximate, 599
array, 134, 163, 187, 193, 198, 221, 294, 297, 298, ...
artefact, 600, 602
assignment rule, 600
at-least-once, 600, 604
atomic, 392, 394, 396, 401, 599, 600
atomicity, 599, 600
availability, 45, 143, 145, 148, 153, 316, 384, 386, 436, ...
aws ecs, 600, 610

B

backend, 34, 38, 45, 49, 53, 588, 600, 617
bearer token, 94, 103, 600
best case, 600, 617
binary tree, 234, 239, 242, 243, 245, 600
blast radius, 103, 259, 263, 270, 286, 600
block, 187, 190, 193, 263, 264, 280, 285, 355, 368, ...
blocked, 382, 391, 392, 396, 398, 564, 567, 594, 600
both, 3, 5, 6, 8, 10, 16, 18, 19, 20, ...
boundary, 324, 337, 380, 381, 382, 401, 600, 604, 609, ...
bounded context, 382, 600
bounded optimism, 600

breadth-first, 350, 360, 365, 600

break point, 600

broken links, 52, 601, 610

bucket by time, 601

bucket counts, 35, 49, 601

bulkhead, 601

burst size, 601, 612

C

cache, 36, 50, 52, 53, 148, 160, 187, 193, 202, ...
cache it, 601
cache stampede, 601
candidates, 79, 172, 174, 188, 266, 599, 601, 616, 617
capacity, 9, 39, 44, 147, 205, 317, 357, 360, 362, ...
cdn, 147, 319, 320, 324, 325, 329, 330, 333, 334, ...
certificate, 260, 601
chaining, 601
channels, 601
checks permission, 601
chunks, 471, 477, 478, 483, 496, 499, 510, 601
circular buffer, 601, 612
class, 97, 149, 206, 355, 356, 360, 361, 363, 365, ...
client, 41, 147, 261, 386, 445, 557, 601, 608, 612
collapse key, 602
collapsing writes, 602
collection, 358, 362, 476, 535, 602, 607, 617, 618
column, 10, 36, 77, 119, 120, 121, 123, 125, 129, ...
compact, 602, 603
comparator, 602
comparison, 130, 270, 367, 602, 615
complete, 15, 43, 73, 125, 143, 182, 241, 299, 351, ...
component, 361, 382, 441, 446, 491, 495, 602, 606, 609, ...
composite, 172, 176, 292, 602
composite key, 602
composition, 602, 605
composition root, 602
connection, 37, 41, 42, 207, 361, 385, 388, 602, 607
connection-oriented, 602
consequence, 602, 611, 613

consistency, 386, 436, 438, 439, 453, 491, 554, 555, 570, ...
constraints, 293, 337, 356, 362, 400, 555, 602, 604
constructor, 501, 514, 554, 596, 599, 602
container, 97, 202, 316, 360, 600, 602, 603, 618
container image, 600, 602
containerd, 603
context, 96, 104, 107, 382, 527, 529, 589, 600, 603
coordinator, 385, 394, 603, 611
correlation id, 603
cost, 19, 22, 26, 28, 33, 41, 43, 44, 45, ...
count map, 603
counter, 3, 39, 41, 84, 90, 145, 152, 259, 262, ...
cri-o, 603
critical path, 603
cycle, 21, 28, 29, 30, 46, 57, 60, 291, 315, ...

D

dangerous, 263, 603
dashed, 603
dau, 228, 229, 323, 326, 335, 436, 439, 440, 453, ...
deep-copied, 603
deleted, 318, 603, 604, 610, 616
delta of delta, 603
dependency, 165, 361, 389, 555, 603, 610, 616
depth, 36, 41, 48, 50, 258, 263, 279, 280, 350, ...
depth-first, 350, 360, 365, 603
deque, 347, 351, 358, 362, 369, 403, 472, 476, 525, ...
diminishing returns, 604
direct addressing, 604
domain name, 604
draw two diagrams, 604
duck typing, 604
dummy node, 350, 604

E

earlier, 90, 186, 206, 250, 276, 307, 311, 374, 418, ...
edge cases, 307, 312, 313, 409, 413, 418, 420, 422, 427, ...
edge compute, 604
effect, 31, 36, 207, 278, 279, 386, 455, 483, 541, ...
elastic scaling, 604
election timeout, 604
empty, 77, 80, 85, 88, 101, 117, 122, 236, 315, ...
etl, 22, 80, 101, 132, 142, 196, 197, 228, 257, ...
event object, 604
eventual consistency, 386, 555, 604
exactly, 4, 5, 6, 7, 9, 10, 11, 13, 14, ...
exactly correct, 604
exactly one, 6, 7, 9, 10, 11, 13, 14, 16, 18, ...
exclusive lock, 604

expiry, 270, 279, 564, 592, 601, 605, 608, 616

F

facts, 63, 87, 117, 126, 128, 164, 165, 167, 197, ...
false negative, 605
false positive, 605
fanout, 605
fee, 5, 34, 36, 38, 45, 49, 115, 148, 166, ...
fifo, 605
file, 22, 23, 80, 81, 82, 96, 97, 107, 134, ...
filled diamond, 605
finite state machine, 605
float, 23, 82, 134, 178, 191, 238, 248, 249, 250, ...
four-direction move, 605
fragile, 605
frontend, 605

G

galloping mode, 605
gap, 3, 4, 5, 6, 9, 20, 144, 166, 203, ...
generator expression, 605
greedy simulation, 605
guest operating system, 605, 606

H

hash map, 353, 367, 371, 372, 373, 375, 376, 423, 585, ...
hash ring, 605
hateoas, 605
header interface, 606
heap, 33, 37, 43, 44, 46, 52, 53, 93, 96, ...
heapsort, 606
heartbeat, 606
hidden spof, 606
hiding mechanism, 606
highest node, 606
historical snapshot, 606
hollow triangle, 606
hook, 606
horizontal scaling, 606
hypervisor, 605, 606

I

image, 380, 393, 436, 437, 445, 600, 602, 603, 606
implicit graph, 606
in parallel, 148, 161, 384, 562, 590, 606
in series, 148, 152, 158, 160, 161, 166, 381, 387, 392, ...
in-degree, 351, 606, 610
index, 13, 96, 97, 101, 105, 262, 272, 274, 278, ...
indexed heap, 606

inheritance, 606
inorder, 351, 354, 361, 606, 613
input, 22, 23, 27, 55, 80, 81, 123, 129, 132, ...
integrity, 607
intent plus multiplicity, 607
interface, 381, 435, 436, 439, 443, 453, 491, 492, 493, ...
invariant, 501, 554, 607
is possible, 607, 614
item, 9, 20, 29, 66, 68, 71, 74, 85, 87, ...
iterator, 607

J

json, 97, 98, 101, 103, 106, 607
jump, 153, 315, 430, 607, 609
jwt, 261, 270, 271, 279, 283, 284, 285, 287, 592, ...

K

k closest points, 607
k most frequent, 607
k-th largest specifically, 607
key, 28, 43, 52, 53, 133, 158, 259, 262, 263, ...
key property, 607

L

lag, 9, 42, 92, 93, 158, 183, 201, 206, 208, ...
largest, 130, 170, 177, 194, 198, 232, 233, 243, 245, ...
latency, 33, 48, 52, 53, 101, 106, 143, 148, 160, ...
lazy, 32, 549, 608
lazy deletion, 608
lazy loading, 608
lazy refill, 608
leader, 326, 331, 334, 593, 604, 608, 615
leaf, 608, 614
lease, 200, 201, 213, 216, 218, 380, 382, 385, 389, ...
ledger entries, 608
left, 4, 5, 6, 7, 11, 12, 13, 15, 22, ...
lfu, 608
linear scan, 608
linearizability, 608
list, 31, 45, 52, 66, 68, 70, 71, 73, 74, ...
list of lists, 608
load balancer, 147, 153, 202, 204, 207, 319, 555, 557, 608
load factor, 608
load shedding, 608
log-structured merge tree, 608
logical cohesion, 608
logical time, 608
loop, 11, 13, 15, 16, 17, 18, 19, 20, 21, ...
loop body, 418, 429, 608

low cohesion, 608
lru, 599, 608

M

manifest, 496, 498, 506, 608
many times, 32, 174, 194, 256, 345, 352, 358, 361, 369, ...
many-to-many, 609, 615
matrix, 608, 609
mau, 609
max-heap, 471, 483, 609
method resolution order, 609
milliseconds, 33, 385, 603, 609
min-heap, 472, 534, 571, 573, 609
minimum spanning forest, 609
mixin, 609
monotone, 357, 362, 609
monotonic deque, 609

N

name mangling, 609
namespace, 321, 609
negative, 7, 12, 14, 16, 17, 18, 22, 56, 57, ...
negative infinity, 609
negatives, 57, 69, 73, 368, 376, 585, 604, 609
nested loop, 367, 609
network partition, 610
never equal, 599, 610
never forgets, 610
never persisted, 610
no children, 608, 610
node, 36, 50, 53, 96, 104, 234, 243, 245, 251, ...
not a map, 610

O

$O(1)$, 21, 77, 78, 79, 115, 123, 125, 129, 130, ...
object, 28, 35, 94, 96, 98, 103, 104, 105, 107, ...
observers, 610
occupied, 603, 604, 610
offset, 610
on first touch, 610
one-shot, 610
opens, 149, 610
orchestrator, 600, 610
origin, 96, 177, 319, 324, 411, 445, 473, 476, 498, ...
origin pull, 610
orphan rows, 610
orphans, 601, 610
out-degree, 606, 610
overloading, 610
overriding, 611

P

page, 21, 28, 31, 34, 36, 38, 39, 42, 43, ...
parallel edges, 611
parameters, 439, 611
parent, 92, 93, 94, 95, 102, 104, 106, 107, 112, ...
partial failure, 611
participants, 501, 502, 603, 611, 613
partition key, 601, 611
partitions, 611
path, 28, 45, 46, 51, 52, 101, 106, 107, 113, ...
port, 14, 15, 23, 26, 35, 39, 49, 57, 58, ...
prefix sums, 348, 354, 355, 360, 365, 367, 368, 609, 611, ...
preorder, 611
priority queue, 611
procedure, 291, 353, 401, 611
process, 91, 97, 150, 207, 208, 385, 388, 561, 602, ...
program, 237, 248, 352, 354, 355, 361, 365, 602, 611, ...
protecting state, 611

Q

qps, 439, 491, 611
queue, 34, 36, 41, 48, 50, 92, 93, 95, 96, ...
quorum, 612

R

race condition, 612
ram, 20, 35, 39, 41, 46, 49, 53, 65, 70, ...
rank, 39, 352, 355, 360, 393, 612
rebalancing, 612
rebuilding the tree, 612
recursion, 68, 367, 370, 479, 612
references, 601, 610, 612, 618
refill rate, 601, 612
register, 259, 266, 612
relative epsilon, 612
removal, 211, 217, 612
replica, 148, 153, 210, 319, 322, 325, 444, 555, 557, ...
replica divergence, 612
request-aware routing, 612
resolver, 612
resources, 34, 53, 318, 322, 334, 390, 588, 593, 612, ...
returned, 42, 605, 612, 615
ring buffer, 21, 601, 612
role interface, 612
root, 172, 173, 178, 182, 183, 185, 186, 190, 191, ...
rotation, 45, 97, 613
routine, 81, 101, 172, 211, 213, 318, 322, 435, 613
row, 7, 8, 10, 11, 26, 32, 36, 39, 46, ...

run, 3, 17, 29, 32, 42, 43, 46, 48, 51, ...

S

same remainder, 613
schema, 199, 202, 203, 204, 208, 211, 212, 213, 214, ...
scoping, 443, 613
script, 263, 274, 356, 361, 362, 363, 365, 407, 408, ...
seam, 381, 391, 396, 613
search space, 275, 348, 399, 613
selectivity, 613
sentinel, 531, 536, 604, 613
separator, 613
sequence diagram, 613, 620
serialisable, 613
service credits, 613
session, 94, 207, 261, 271, 607, 613, 614, 616, 619
session guarantees, 613
set, 3, 7, 8, 9, 11, 13, 14, 15, 16, ...
shallow-copied, 613
shape, 72, 80, 85, 86, 100, 108, 111, 113, 132, ...
shard, 438, 442, 443, 444, 445, 446, 449, 452, 453, ...
shared lock, 614
shared store, 614
singleton, 614
singly linked list, 614
slice, 179, 201, 214, 217, 296, 614
smallest, 179, 182, 183, 184, 188, 189, 297, 300, 349, ...
snapshot, 269, 316, 319, 320, 325, 606, 614
snipe, 614
sorted region, 614
sorting everything, 614
splits, 127, 128, 235, 355, 473, 494, 500, 501, 502, ...
spof, 606, 614
squares, 176, 177, 194, 198, 582, 614
stable, 207, 213, 217, 218, 356, 563, 591, 614
stack, 41, 55, 97, 100, 103, 106, 107, 189, 349, ...
stack frame, 189, 614
stack overflow, 614
state, 20, 29, 56, 71, 78, 79, 85, 100, 123, ...
statements, 611, 614, 621
status, 37, 40, 42, 496, 614
still one machine, 607, 614
stonith, 615
strategy, 89, 199, 201, 211, 212, 213, 218, 219, 222, ...
strictly shorter, 615
structural, 292, 293, 611, 615
subarray, 360, 368, 615, 624
subject, 615
subscription list, 615

substring, 355, 360, 361, 363, 365, 615, 623
subtree, 606, 611, 615
surge multiplier especially, 615
swap, 66, 67, 74, 75, 81, 112, 115, 133, 180, ...
system call, 615

T

table, 3, 9, 21, 24, 27, 28, 29, 30, 41, ...
tail call, 615
tail-recursive, 615
tcp, 602, 615, 616
template, 356, 362, 363, 365, 402, 467, 516, 524, 525, ...
test double, 615
the return address, 615
the running median, 606, 615, 616
third table, 609, 615
thread, 62, 96, 104, 107, 486, 542, 589, 600, 616
three distinct orderings, 616
three-way partition, 616
tight coupling, 616
time-to-live, 616
tiny constraint, 616
toast, 616
token, 94, 103, 107, 221, 258, 260, 261, 263, 264, ...
tombstone, 616
top-k problems, 606, 615, 616
total order, 616
transitions itself, 616
transitive dependency, 616
tree, 9, 86, 92, 102, 107, 200, 234, 239, 242, ...
ttl, 32, 145, 167, 175, 189, 194, 196, 272, 290, ...
tuple, 123, 125, 180, 181, 183, 239, 242, 297, 300, ...

two, 3, 4, 5, 6, 7, 8, 9, 10, 11, ...
two implementations, 616

U

udp, 615, 616
under-fetching, 616
union filesystem, 616
upper bound, 612, 617
uri, 1, 149, 150, 205, 209, 227, 258, 264, 281, ...
url, 37, 40, 51, 263, 274, 276, 278, 437, 445, ...

V

value object, 503, 513, 617
version, 5, 56, 92, 130, 131, 204, 205, 206, 207, ...
vertex, 603, 610, 611, 617
vertical scaling, 617
victim, 617
virtual address space, 617
virtual machine, 600, 610, 617
volatile, 617

W

weight, 3, 4, 5, 6, 10, 20, 206, 210, 212, ...
which class, 617
which is hours, 617
within, 143, 208, 217, 261, 272, 278, 279, 289, 316, ...
worst case, 20, 26, 59, 83, 188, 194, 197, 198, 221, ...
write skew, 617
write-around with invalidation, 617
write-through, 555, 564, 570, 575, 597, 617

Y

yes, 9, 62, 66, 89, 97, 98, 99, 101, 109, ...

Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.